

MkniX

0.9.0

Generated by Doxygen 1.9.1

1 Mknix	2
1.1 Installation	2
1.2 Tests	2
1.3 Usage	2
1.4 References	2
1.5 License	2
2 mknix Input File Manual	2
2.1 Overview	2
2.2 Top-Level Commands	3
2.2.1 TITLE	3
2.2.2 WORKINGDIR	3
2.2.3 DIMENSION	3
2.2.4 GRAVITY	3
2.2.5 CONTACT	3
2.2.6 VISUALIZATION	3
2.2.7 OUTPUT	3
2.2.8 SMOOTHING	3
2.2.9 INITIALTEMPERATURE	3
2.3 MATERIALS Section	3
2.3.1 PLSTRAIN (Plane-strain mechanical material)	3
2.3.2 THERMAL (Thermal material, constant properties)	4
2.3.3 POROSITY (Porous medium extension; requires a THERMAL entry for the same mat_id)	4
2.3.4 FILES (Temperature-dependent thermal properties from files)	4
2.4 SYSTEM Section	5
2.5 RIGIDBODIES Section	5
2.5.1 PENALTY / AUGMENTED	5
2.5.2 ALPHA	5
2.5.3 MASSPOINT	5
2.5.4 BAR	5
2.5.5 CHAIN	6
2.5.6 GENERIC2D	6
2.5.7 GENERIC3D	6
2.6 FLEXBODIES Section	6
2.6.1 MESHFREE / FEMESH	6
2.7 BODYPOINTS Section	7
2.8 JOINTS Section	7
2.8.1 AXIS – additional sub-command	7
2.8.2 AUGMENTED tolerance	7
2.8.3 CLEARANCE – additional sub-command	7
2.8.4 THERMALSPHERICAL – additional sub-command	7
2.9 LOADS Section	7

2.9.1 THERMALBODY	8
2.9.2 THERMALFLUX1D	8
2.9.3 RADIATION	9
2.10 MOTION Section	9
2.11 SIGNALS Section	9
2.12 ANALYSIS Section	9
2.12.1 STATIC	9
2.12.2 THERMALSTATIC	9
2.12.3 THERMALDYNAMIC	9
2.12.4 THERMOMECHANICALDYNAMIC	9
2.12.5 DYNAMIC	9
2.13 Complete Minimal Example	10
3 Namespace Index	10
3.1 Namespace List	10
4 Hierarchical Index	10
4.1 Class Hierarchy	10
5 Class Index	13
5.1 Class List	13
6 File Index	15
6.1 File List	15
7 Namespace Documentation	20
7.1 Imx Namespace Reference	20
7.2 mknix Namespace Reference	20
7.2.1 Detailed Description	22
7.2.2 Typedef Documentation	22
7.2.3 Enumeration Type Documentation	22
7.2.4 Function Documentation	23
8 Class Documentation	28
8.1 mknix::Analysis Class Reference	28
8.1.1 Detailed Description	29
8.1.2 Constructor & Destructor Documentation	29
8.1.3 Member Function Documentation	29
8.1.4 Member Data Documentation	31
8.2 mknix::AnalysisDynamic Class Reference	31
8.2.1 Detailed Description	32
8.2.2 Constructor & Destructor Documentation	32
8.2.3 Member Function Documentation	33
8.3 mknix::AnalysisStatic Class Reference	34

8.3.1 Detailed Description	35
8.3.2 Constructor & Destructor Documentation	35
8.3.3 Member Function Documentation	36
8.4 mknix::AnalysisThermalDynamic Class Reference	36
8.4.1 Detailed Description	37
8.4.2 Constructor & Destructor Documentation	38
8.4.3 Member Function Documentation	38
8.5 mknix::AnalysisThermalStatic Class Reference	39
8.5.1 Detailed Description	40
8.5.2 Constructor & Destructor Documentation	40
8.5.3 Member Function Documentation	41
8.6 mknix::AnalysisThermoMechanicalDynamic Class Reference	41
8.6.1 Detailed Description	42
8.6.2 Constructor & Destructor Documentation	43
8.6.3 Member Function Documentation	43
8.7 mknix::Body Class Reference	44
8.7.1 Detailed Description	47
8.7.2 Constructor & Destructor Documentation	47
8.7.3 Member Function Documentation	47
8.7.4 Member Data Documentation	56
8.8 mknix::BoundaryGroup Class Reference	57
8.8.1 Detailed Description	58
8.8.2 Constructor & Destructor Documentation	58
8.8.3 Member Function Documentation	58
8.8.4 Member Data Documentation	59
8.9 mknix::boxFIR Class Reference	59
8.9.1 Detailed Description	60
8.9.2 Constructor & Destructor Documentation	60
8.9.3 Member Function Documentation	60
8.10 mknix::Cell Class Reference	60
8.10.1 Detailed Description	63
8.10.2 Constructor & Destructor Documentation	63
8.10.3 Member Function Documentation	63
8.10.4 Member Data Documentation	70
8.11 mknix::CellBoundary Class Reference	71
8.11.1 Detailed Description	72
8.11.2 Constructor & Destructor Documentation	72
8.11.3 Member Function Documentation	73
8.11.4 Member Data Documentation	74
8.12 mknix::CellBoundaryLinear Class Reference	75
8.12.1 Detailed Description	76
8.12.2 Constructor & Destructor Documentation	76

8.12.3 Member Function Documentation	78
8.12.4 Member Data Documentation	79
8.13 mknix::CellRect Class Reference	79
8.13.1 Detailed Description	81
8.13.2 Constructor & Destructor Documentation	81
8.13.3 Member Function Documentation	83
8.14 mknix::CellTetrahedron Class Reference	83
8.14.1 Detailed Description	84
8.14.2 Constructor & Destructor Documentation	84
8.14.3 Member Function Documentation	85
8.14.4 Member Data Documentation	86
8.15 mknix::CellTriang Class Reference	86
8.15.1 Detailed Description	88
8.15.2 Constructor & Destructor Documentation	88
8.15.3 Member Function Documentation	90
8.15.4 Member Data Documentation	91
8.16 mknix::CompBar Class Reference	91
8.16.1 Detailed Description	91
8.16.2 Constructor & Destructor Documentation	91
8.16.3 Member Function Documentation	91
8.17 mknix::Constraint Class Reference	92
8.17.1 Detailed Description	93
8.17.2 Constructor & Destructor Documentation	94
8.17.3 Member Function Documentation	94
8.17.4 Member Data Documentation	99
8.18 mknix::ConstraintClearance Class Reference	100
8.18.1 Detailed Description	101
8.18.2 Constructor & Destructor Documentation	101
8.18.3 Member Function Documentation	102
8.18.4 Member Data Documentation	103
8.19 mknix::ConstraintContact Class Reference	103
8.19.1 Detailed Description	104
8.19.2 Constructor & Destructor Documentation	105
8.19.3 Member Function Documentation	105
8.19.4 Member Data Documentation	106
8.20 mknix::ConstraintDistance Class Reference	106
8.20.1 Detailed Description	107
8.20.2 Constructor & Destructor Documentation	108
8.20.3 Member Function Documentation	108
8.20.4 Member Data Documentation	109
8.21 mknix::ConstraintFixedAxis Class Reference	110
8.21.1 Detailed Description	111

8.21.2 Constructor & Destructor Documentation	111
8.21.3 Member Function Documentation	112
8.21.4 Member Data Documentation	112
8.22 mknix::ConstraintFixedCoordinates Class Reference	113
8.22.1 Detailed Description	114
8.22.2 Constructor & Destructor Documentation	114
8.22.3 Member Function Documentation	115
8.22.4 Member Data Documentation	115
8.23 mknix::ConstraintThermal Class Reference	116
8.23.1 Detailed Description	117
8.23.2 Constructor & Destructor Documentation	117
8.23.3 Member Function Documentation	117
8.24 mknix::ConstraintThermalFixed Class Reference	118
8.24.1 Detailed Description	119
8.24.2 Constructor & Destructor Documentation	120
8.24.3 Member Function Documentation	120
8.24.4 Member Data Documentation	121
8.25 mknix::Contact Class Reference	121
8.25.1 Detailed Description	121
8.25.2 Constructor & Destructor Documentation	121
8.25.3 Member Function Documentation	122
8.26 Imx::DenseMatrix< T > Class Template Reference	122
8.26.1 Detailed Description	122
8.27 mknix::ElemTetrahedron Class Reference	123
8.27.1 Detailed Description	124
8.27.2 Constructor & Destructor Documentation	124
8.27.3 Member Function Documentation	125
8.28 mknix::ElemTriangle Class Reference	126
8.28.1 Detailed Description	127
8.28.2 Constructor & Destructor Documentation	127
8.28.3 Member Function Documentation	128
8.29 mknix::FlexBody Class Reference	129
8.29.1 Detailed Description	130
8.29.2 Constructor & Destructor Documentation	131
8.29.3 Member Function Documentation	131
8.29.4 Member Data Documentation	136
8.30 mknix::FlexFrameGalerkin Class Reference	136
8.30.1 Detailed Description	138
8.30.2 Constructor & Destructor Documentation	138
8.30.3 Member Function Documentation	139
8.31 mknix::FlexGlobalGalerkin Class Reference	142
8.31.1 Detailed Description	144

8.31.2 Constructor & Destructor Documentation	144
8.31.3 Member Function Documentation	144
8.32 mknix::Force Class Reference	148
8.32.1 Detailed Description	148
8.32.2 Constructor & Destructor Documentation	149
8.32.3 Member Function Documentation	149
8.33 mknix::GaussPoint Class Reference	150
8.33.1 Detailed Description	152
8.33.2 Constructor & Destructor Documentation	152
8.33.3 Member Function Documentation	153
8.33.4 Member Data Documentation	157
8.34 mknix::GaussPoint2D Class Reference	159
8.34.1 Detailed Description	160
8.34.2 Constructor & Destructor Documentation	160
8.34.3 Member Function Documentation	162
8.35 mknix::GaussPoint3D Class Reference	169
8.35.1 Detailed Description	171
8.35.2 Constructor & Destructor Documentation	171
8.35.3 Member Function Documentation	172
8.36 mknix::GaussPointBoundary Class Reference	179
8.36.1 Detailed Description	180
8.36.2 Constructor & Destructor Documentation	180
8.36.3 Member Function Documentation	181
8.36.4 Member Data Documentation	183
8.37 mknix::Load Class Reference	183
8.37.1 Detailed Description	184
8.37.2 Constructor & Destructor Documentation	184
8.37.3 Member Function Documentation	184
8.37.4 Member Data Documentation	185
8.38 mknix::LoadThermal Class Reference	185
8.38.1 Detailed Description	186
8.38.2 Constructor & Destructor Documentation	186
8.38.3 Member Function Documentation	186
8.38.4 Member Data Documentation	187
8.39 mknix::LoadThermalBody Class Reference	188
8.39.1 Detailed Description	188
8.39.2 Constructor & Destructor Documentation	188
8.39.3 Member Function Documentation	188
8.39.4 Member Data Documentation	190
8.40 mknix::LoadThermalBoundary1D Class Reference	191
8.40.1 Detailed Description	191
8.40.2 Constructor & Destructor Documentation	191

8.40.3 Member Function Documentation	192
8.40.4 Member Data Documentation	193
8.41 mknix::Material Class Reference	193
8.41.1 Detailed Description	194
8.41.2 Constructor & Destructor Documentation	195
8.41.3 Member Function Documentation	195
8.42 Imx::Matrix< T > Class Template Reference	206
8.42.1 Detailed Description	206
8.43 mknix::MknixWrapper Class Reference	206
8.43.1 Detailed Description	207
8.43.2 Constructor & Destructor Documentation	207
8.43.3 Member Function Documentation	207
8.44 mknix::Motion Class Reference	207
8.44.1 Detailed Description	208
8.44.2 Constructor & Destructor Documentation	209
8.44.3 Member Function Documentation	209
8.45 mknix::Node Class Reference	210
8.45.1 Detailed Description	212
8.45.2 Constructor & Destructor Documentation	212
8.45.3 Member Function Documentation	213
8.46 mknix::Point Class Reference	221
8.46.1 Detailed Description	223
8.46.2 Constructor & Destructor Documentation	223
8.46.3 Member Function Documentation	225
8.46.4 Friends And Related Function Documentation	234
8.46.5 Member Data Documentation	235
8.47 mknix::Radiation Class Reference	236
8.47.1 Detailed Description	237
8.47.2 Constructor & Destructor Documentation	237
8.47.3 Member Function Documentation	238
8.48 mknix::Reader Class Reference	238
8.48.1 Detailed Description	238
8.48.2 Constructor & Destructor Documentation	239
8.48.3 Member Function Documentation	239
8.49 mknix::ReaderConstraints Class Reference	240
8.49.1 Detailed Description	240
8.49.2 Constructor & Destructor Documentation	240
8.49.3 Member Function Documentation	241
8.50 mknix::ReaderFlex Class Reference	241
8.50.1 Detailed Description	242
8.50.2 Constructor & Destructor Documentation	242
8.50.3 Member Function Documentation	242

8.51	mknix::ReaderRigid Class Reference	243
8.51.1	Detailed Description	243
8.51.2	Constructor & Destructor Documentation	243
8.51.3	Member Function Documentation	244
8.52	mknix::RigidBar Class Reference	244
8.52.1	Detailed Description	246
8.52.2	Constructor & Destructor Documentation	246
8.52.3	Member Function Documentation	247
8.53	mknix::RigidBody Class Reference	249
8.53.1	Detailed Description	251
8.53.2	Constructor & Destructor Documentation	251
8.53.3	Member Function Documentation	251
8.53.4	Member Data Documentation	255
8.54	mknix::RigidBody2D Class Reference	256
8.54.1	Detailed Description	257
8.54.2	Constructor & Destructor Documentation	258
8.54.3	Member Function Documentation	258
8.55	mknix::RigidBody3D Class Reference	260
8.55.1	Detailed Description	262
8.55.2	Constructor & Destructor Documentation	262
8.55.3	Member Function Documentation	263
8.56	mknix::RigidBodyMassPoint Class Reference	265
8.56.1	Detailed Description	266
8.56.2	Constructor & Destructor Documentation	266
8.56.3	Member Function Documentation	266
8.57	mknix::SectionReader Class Reference	268
8.57.1	Detailed Description	268
8.57.2	Constructor & Destructor Documentation	268
8.57.3	Member Function Documentation	268
8.58	mknix::ShapeFunction Class Reference	269
8.58.1	Detailed Description	271
8.58.2	Constructor & Destructor Documentation	271
8.58.3	Member Function Documentation	272
8.58.4	Member Data Documentation	274
8.59	mknix::ShapeFunctionLinear Class Reference	274
8.59.1	Detailed Description	275
8.59.2	Constructor & Destructor Documentation	276
8.59.3	Member Function Documentation	276
8.60	mknix::ShapeFunctionLinearX Class Reference	277
8.60.1	Detailed Description	278
8.60.2	Constructor & Destructor Documentation	278
8.60.3	Member Function Documentation	278

8.61 mknix::ShapeFunctionMLS Class Reference	279
8.61.1 Detailed Description	280
8.61.2 Constructor & Destructor Documentation	280
8.61.3 Member Function Documentation	280
8.62 mknix::ShapeFunctionMLS2D Class Reference	281
8.62.1 Detailed Description	281
8.62.2 Constructor & Destructor Documentation	282
8.62.3 Member Function Documentation	282
8.63 mknix::ShapeFunctionMLS3D Class Reference	282
8.63.1 Detailed Description	283
8.63.2 Constructor & Destructor Documentation	284
8.63.3 Member Function Documentation	284
8.64 mknix::ShapeFunctionRBF Class Reference	284
8.64.1 Detailed Description	285
8.64.2 Constructor & Destructor Documentation	286
8.64.3 Member Function Documentation	286
8.65 mknix::ShapeFunctionTetrahedron Class Reference	287
8.65.1 Detailed Description	288
8.65.2 Constructor & Destructor Documentation	288
8.65.3 Member Function Documentation	289
8.66 mknix::ShapeFunctionTriangle Class Reference	290
8.66.1 Detailed Description	291
8.66.2 Constructor & Destructor Documentation	291
8.66.3 Member Function Documentation	291
8.67 mknix::ShapeFunctionTriangleSigned Class Reference	292
8.67.1 Detailed Description	293
8.67.2 Constructor & Destructor Documentation	293
8.67.3 Member Function Documentation	293
8.68 mknix::Simulation Class Reference	294
8.68.1 Detailed Description	297
8.68.2 Member Enumeration Documentation	297
8.68.3 Constructor & Destructor Documentation	297
8.68.4 Member Function Documentation	297
8.68.5 Friends And Related Function Documentation	324
8.69 mknix::MknixWrapper::SimulationWrapper Struct Reference	325
8.69.1 Detailed Description	325
8.69.2 Member Data Documentation	325
8.70 mknix::System Class Reference	325
8.70.1 Detailed Description	328
8.70.2 Constructor & Destructor Documentation	328
8.70.3 Member Function Documentation	329
8.70.4 Friends And Related Function Documentation	337

8.70.5 Member Data Documentation	337
8.71 mknix::SystemChain Class Reference	338
8.71.1 Detailed Description	340
8.71.2 Constructor & Destructor Documentation	340
8.71.3 Member Function Documentation	340
8.72 mknix::ThermalBody Class Reference	342
8.72.1 Detailed Description	343
8.72.2 Constructor & Destructor Documentation	343
8.72.3 Member Function Documentation	343
8.72.4 Member Data Documentation	347
8.73 Imx::Vector< T > Class Template Reference	347
8.73.1 Detailed Description	347
9 File Documentation	347
9.1 analysis.cpp File Reference	347
9.2 analysis.h File Reference	348
9.3 analysisdynamic.cpp File Reference	349
9.4 analysisdynamic.h File Reference	349
9.5 analysisstatic.cpp File Reference	350
9.6 analysisstatic.h File Reference	350
9.7 analysisthermaldynamic.cpp File Reference	351
9.8 analysisthermaldynamic.h File Reference	352
9.9 analysisthermalstatic.cpp File Reference	353
9.10 analysisthermalstatic.h File Reference	353
9.11 analysisthermomechanicaldynamic.cpp File Reference	354
9.12 analysisthermomechanicaldynamic.h File Reference	354
9.13 body.cpp File Reference	355
9.14 body.h File Reference	356
9.15 bodyflex.cpp File Reference	357
9.16 bodyflex.h File Reference	357
9.17 bodyflexframegalerkin.cpp File Reference	358
9.18 bodyflexframegalerkin.h File Reference	359
9.19 bodyflexglobalgalerkin.cpp File Reference	360
9.20 bodyflexglobalgalerkin.h File Reference	361
9.21 bodyrigid.cpp File Reference	361
9.22 bodyrigid.h File Reference	362
9.23 bodyrigid0D.cpp File Reference	363
9.24 bodyrigid0D.h File Reference	363
9.25 bodyrigid1D.cpp File Reference	364
9.26 bodyrigid1D.h File Reference	365
9.27 bodyrigid2D.cpp File Reference	366
9.28 bodyrigid2D.h File Reference	367

9.29 bodyrigid3D.cpp File Reference	367
9.30 bodyrigid3D.h File Reference	368
9.31 bodythermal.cpp File Reference	369
9.32 bodythermal.h File Reference	369
9.33 boundarygroup.cpp File Reference	370
9.34 boundarygroup.h File Reference	371
9.35 cell.cpp File Reference	371
9.36 cell.h File Reference	372
9.36.1 Detailed Description	373
9.37 cellboundary.cpp File Reference	373
9.38 cellboundary.h File Reference	373
9.39 cellboundarylinear.cpp File Reference	374
9.40 cellboundarylinear.h File Reference	374
9.40.1 Detailed Description	375
9.41 cellrect.cpp File Reference	375
9.42 cellrect.h File Reference	376
9.42.1 Detailed Description	377
9.43 celltetrahedron.cpp File Reference	377
9.44 celltetrahedron.h File Reference	378
9.44.1 Detailed Description	378
9.45 celltriang.cpp File Reference	379
9.46 celltriang.h File Reference	379
9.46.1 Detailed Description	380
9.47 common.cpp File Reference	380
9.48 common.h File Reference	381
9.49 compbar.cpp File Reference	382
9.50 compbar.h File Reference	382
9.51 constraint.cpp File Reference	382
9.52 constraint.h File Reference	383
9.53 constraintclearance.cpp File Reference	383
9.54 constraintclearance.h File Reference	384
9.55 constraintcontact.cpp File Reference	385
9.56 constraintcontact.h File Reference	385
9.57 constraintdistance.cpp File Reference	386
9.58 constraintdistance.h File Reference	386
9.59 constraintfixedaxis.cpp File Reference	387
9.60 constraintfixedaxis.h File Reference	388
9.61 constraintfixedcoordinates.cpp File Reference	388
9.62 constraintfixedcoordinates.h File Reference	389
9.63 constraintthermal.cpp File Reference	390
9.64 constraintthermal.h File Reference	390
9.65 constraintthermalfixed.cpp File Reference	391

9.66 constraintthermalfixed.h File Reference	391
9.67 dictionary.h File Reference	392
9.68 elemtetrahedron.cpp File Reference	392
9.69 elemtetrahedron.h File Reference	393
9.70 elemtriangle.cpp File Reference	394
9.71 elemtriangle.h File Reference	394
9.72 force.cpp File Reference	395
9.73 force.h File Reference	396
9.74 gausspoint.cpp File Reference	397
9.75 gausspoint.h File Reference	397
9.75.1 Detailed Description	398
9.76 gausspoint2D.cpp File Reference	398
9.77 gausspoint2D.h File Reference	399
9.78 gausspoint3D.cpp File Reference	399
9.79 gausspoint3D.h File Reference	400
9.80 gausspointboundary.cpp File Reference	401
9.81 gausspointboundary.h File Reference	401
9.82 generalcontact.cpp File Reference	402
9.83 generalcontact.h File Reference	402
9.84 input_manual.md File Reference	403
9.85 load.cpp File Reference	403
9.86 load.h File Reference	403
9.87 loadradiation.cpp File Reference	404
9.88 loadradiation.h File Reference	405
9.89 loadthermal.cpp File Reference	406
9.90 loadthermal.h File Reference	406
9.91 loadthermalbody.cpp File Reference	407
9.92 loadthermalbody.h File Reference	407
9.93 loadthermalboundary1D.cpp File Reference	408
9.94 loadthermalboundary1D.h File Reference	409
9.95 mainpage.md File Reference	410
9.96 material.cpp File Reference	410
9.97 material.h File Reference	410
9.98 mknixWrapper.cpp File Reference	411
9.99 mknixWrapper.h File Reference	411
9.100 motion.cpp File Reference	412
9.101 motion.h File Reference	413
9.102 node.cpp File Reference	413
9.103 node.h File Reference	414
9.104 point.cpp File Reference	415
9.105 point.h File Reference	415
9.105.1 Detailed Description	416

9.106 reader.cpp File Reference	416
9.107 reader.h File Reference	416
9.108 readerconstraints.cpp File Reference	417
9.109 readerconstraints.h File Reference	418
9.110 readerflex.cpp File Reference	419
9.111 readerflex.h File Reference	419
9.112 readerrigid.cpp File Reference	420
9.113 readerrigid.h File Reference	421
9.114 sectionreader.cpp File Reference	421
9.115 sectionreader.h File Reference	422
9.116 shapefunction.cpp File Reference	423
9.117 shapefunction.h File Reference	424
9.117.1 Detailed Description	424
9.118 shapefunctionlinear-x.cpp File Reference	425
9.119 shapefunctionlinear-x.h File Reference	425
9.120 shapefunctionlinear.cpp File Reference	426
9.121 shapefunctionlinear.h File Reference	426
9.122 shapefunctionMLS.cpp File Reference	427
9.123 shapefunctionMLS.h File Reference	428
9.123.1 Detailed Description	429
9.124 shapefunctionMLS2D.cpp File Reference	429
9.125 shapefunctionMLS2D.h File Reference	430
9.126 shapefunctionMLS3D.cpp File Reference	430
9.127 shapefunctionMLS3D.h File Reference	431
9.128 shapefunctionRBF.cpp File Reference	432
9.129 shapefunctionRBF.h File Reference	432
9.129.1 Detailed Description	433
9.130 shapefunctiontetrahedron.cpp File Reference	433
9.131 shapefunctiontetrahedron.h File Reference	434
9.132 shapefunctiontriangle.cpp File Reference	435
9.133 shapefunctiontriangle.h File Reference	435
9.134 shapefunctiontriangle3D.cpp File Reference	436
9.135 shapefunctiontriangle3D.h File Reference	437
9.136 simulation.cpp File Reference	438
9.137 simulation.h File Reference	439
9.137.1 Macro Definition Documentation	439
9.138 system.cpp File Reference	439
9.139 system.h File Reference	440
9.140 systemchain.cpp File Reference	441
9.141 systemchain.h File Reference	441

1 MkniX

MkniX is a simulation library for nonlinear thermo-mechanical multibody systems using FEM and mesh-free methods. It includes a standalone executable for forward simulations, and is used in other projects such as [WHAM](#) and [ALICIA](#) for real-time and inverse thermal operational monitoring, respectively.

For details on the formulation, applications and benchmarking, consult the author's [PhD Thesis](#).

1.1 Installation

MkniX is built using CMake (version 2.8 or greater). From the MkniX root directory:

```
cmake -B<build_dir> -H. -DCMAKE_BUILD_TYPE=<Debug|Release>
```

Example:

```
cmake -Bbuild -H. -DCMAKE_BUILD_TYPE=Debug
```

1.2 Tests

After building, run the test suite from the build directory:

```
cd build/tests
ctest
```

1.3 Usage

Run MkniX with an input file as the sole argument:

```
mknix <input_file.mknix>
```

For a full description of the input file format and all available commands see the [mknix Input File Manual](#).

1.4 References

1. Iglesias, Daniel. *On the application of meshfree methods to the nonlinear dynamics of multibody systems*, 2017. [doi:10.20868/upm.thesis.39121](#)
 2. Iglesias, D., García Orden, J.C. *Galerkin meshfree methods applied to the nonlinear dynamics of flexible multibody systems*. Multibody Syst Dyn 25, 203–224 (2011). [doi:10.1007/s11044-010-9224-9](#)
 3. D. Iglesias, J. C. García Orden, B. Brañas, J.M. Carmona, J. Molla. *Application of Galerkin meshfree methods to nonlinear thermo-mechanical simulation of solids under extremely high pulsed loading*. Fusion Engineering and Design, Vol. 88(9–10), 2744–2747, 2013. [doi:10.1016/j.fusengdes.2013.02.158](#)
-

1.5 License

Copyright © 2015 Daniel Iglesias.

MkniX is free software: you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

MkniX is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the [GNU Lesser General Public License](#) for more details.

2 mknix Input File Manual

2.1 Overview

An mknix input file is a plain-text file read sequentially by keyword. Keywords are case-sensitive. Comments begin with // and extend to the end of the line. The general structure is:

```
TITLE <name>
WORKINGDIR <path>
DIMENSION <dim>
MATERIALS
...
ENDMATERIALS
SYSTEM <name>
...
```

```
ENDSYSTEM
ANALYSIS
...
ENDANALYSIS
```

2.2 Top-Level Commands

2.2.1 TITLE

```
TITLE <name>
```

Sets the simulation title (single token, no spaces).

2.2.2 WORKINGDIR

```
WORKINGDIR <path>
```

Changes the working directory. All subsequent file paths are relative to this directory.

2.2.3 DIMENSION

```
DIMENSION <2|3>
```

Sets the spatial dimension of the problem.

2.2.4 GRAVITY

```
GRAVITY <gx> <gy> <gz>
```

Sets the gravity vector components.

2.2.5 CONTACT

```
CONTACT <GLOBAL|NONE>
```

Sets the contact detection strategy.

2.2.6 VISUALIZATION

```
VISUALIZATION <ON|OFF>
```

Enables or disables visualization output.

2.2.7 OUTPUT

```
OUTPUT MATRICES
```

Enables output of system matrices to file.

2.2.8 SMOOTHING

```
SMOOTHING <OFF|LOCAL|CONSTANT|GLOBAL>
```

Sets the stress-smoothing strategy.

2.2.9 INITIALTEMPERATURE

```
INITIALTEMPERATURE <value>
```

Sets the global initial temperature for all nodes.

2.3 MATERIALS Section

```
MATERIALS
// one or more material definitions
ENDMATERIALS
```

2.3.1 PLSTRAIN (Plane-strain mechanical material)

```
PLSTRAIN <mat_id> <E> <nu> <density>
```

- `mat_id` – integer material identifier
- `E` – Young's modulus
- `nu` – Poisson's ratio
- `density` – mass density

2.3.2 THERMAL (Thermal material, constant properties)

THERMAL <mat_id> <Cp> <kappa> <beta> <density>

- Cp – heat capacity
- kappa – thermal conductivity
- beta – thermal expansion coefficient
- density – mass density

2.3.3 POROSITY (Porous medium extension; requires a THERMAL entry for the same mat_id)

POROSITY <mat_id> <Rsl> <T_bulk> [porosityCapacity]

- mat_id – integer material identifier (must match a THERMAL entry)
- Rsl – fluid-solid thermal resistance
- T_bulk – initial bulk fluid temperature
- porosityCapacity (optional) – fluid volumetric heat capacity (if provided, fluid temperature is updated dynamically)

Details:

- If porosityCapacity is specified (and non-zero), the fluid temperature is automatically updated during the simulation according to the power balance, using the provided capacity value. *Note* that this porosityCapacity is currently dependant on the time step parameter. This feature is experimental and may be removed in future versions, as this effect should be implemented as a body property, in order to have a proper residual and tangent contributions.
- If omitted, the fluid temperature remains constant at T_bulk (backward compatible with previous input files).

2.3.4 FILES (Temperature-dependent thermal properties from files)

```
FILES
CAPACITY      <mat_id> <filename>
CONDUCTIVITY  <mat_id> <filename>
BETA          <mat_id> <filename>
DENSITY       <mat_id> <filename>
RESISTANCE    <mat_id> <filename> [COORDS <x|y|z|xy|xz|yz>]
RESISTANCE    <mat_id> [COORDS <x|y|z|xy|xz|yz>] <filename>
ENDFILES
```

CAPACITY, CONDUCTIVITY, BETA, and DENSITY files are two-column, whitespace-separated text files with no header:

File type	Column 1	Column 2
CAPACITY	temperature	heat capacity
CONDUCTIVITY	temperature	thermal conductivity
BETA	temperature	thermal expansion coefficient
DENSITY	temperature	mass density

RESISTANCE supports coordinate-dependent interpolation via optional COORDS:

- x, y, z: 1D table (two columns: key and resistance)
- xy, xz, yz: 2D regular grid table
- Default when omitted: COORDS x

For COORDS x|y|z, use a two-column file:

File type	Column 1	Column 2
RESISTANCE	coordinate value	interface thermal resistance

For COORDS $xy|xz|yz$, use a regular-grid file:

- Row 1: first value ignored, remaining values are key2 coordinates.
- Column 1 (rows 2..N): key1 coordinates.
- Body: resistance values on the (key1, key2) grid.

Examples:

```
CAPACITY 1 cp_vs_temp.txt
CONDUCTIVITY 1 kappa_vs_temp.txt
BETA 1 beta_vs_temp.txt
DENSITY 1 density_vs_temp.txt
RESISTANCE 1 COORDS xz resistance_xz.txt
RESISTANCE 2 resistance_y.txt COORDS y
```

2.4 SYSTEM Section

```
SYSTEM <name>
  RIGIDBODIES ... ENDRIGIDBODIES
  FLEXBODIES ... ENDFLEXBODIES
  BODYPOINTS ... ENDBODYPOINTS
  JOINTS ... ENDJOINTS
  LOADS ... ENDLLOADS
  ENVIRONMENT ... ENDENVIRONMENT
  MOTION <node_id> ... ENDMOTION
  SCALE <sx> <sy> <sz>
  MIRROR <x|y|z>
  SHIFT <dx> <dy> <dz>
  SIGNALS ... ENDSIGNALS
ENDSYSTEM
```

SCALE, MIRROR, and SHIFT apply immediately to all nodes defined so far.

2.5 RIGIDBODIES Section

```
RIGIDBODIES
  PENALTY | AUGMENTED
  ALPHA <value>
  MASSPOINT <name> ... ENDMASSPOINT
  BAR <name> ... ENDBAR
  CHAIN <name> ... ENDCHAIN
  GENERIC2D <name> ... ENDGENERIC2D
  GENERIC3D <name> ... ENDGENERIC3D
ENDRIGIDBODIES
```

2.5.1 PENALTY / AUGMENTED

Sets the constraint enforcement method (penalty or augmented Lagrange). Default is penalty.

2.5.2 ALPHA

ALPHA <value>

Sets the penalty / augmented-Lagrange stiffness factor.

2.5.3 MASSPOINT

```
MASSPOINT <name>
  NODEA <x> <y> <z>
  MASS <value>
ENDMASSPOINT
```

2.5.4 BAR

```
BAR <name>
  NODEA <x> <y> <z>
  NODEB <x> <y> <z>
  MASS <value>
  OUTPUT ENERGY
ENDBAR
```

2.5.5 CHAIN

```
CHAIN <name>
  NODEA      <x> <y> <z>
  NODEB      <x> <y> <z>
  MASS       <value>
  SEGMENTS   <n>
  LENGTH     <value>
  TIMELENGTH <time> <length> // repeatable
  OUTPUT     ENERGY
ENDCHAIN
```

TIMELENGTH may appear multiple times to define a time-varying chain length.

2.5.6 GENERIC2D

```
GENERIC2D <name>
  MASS      <value>
  IXX       <value>
  IYY       <value>
  IXY       <value>
  DENSITY   <factor>
  POSITION   <xCoG> <yCoG> <theta>
  TRIANGLES FILE <meshfile>
  OUTPUT    ENERGY
ENDGENERIC2D
```

meshfile is a mesh file with format: first line <nNodes> <nCells>, then <id> <x> <y> <z> per node, then connectivity.

2.5.7 GENERIC3D

```
GENERIC3D <name>
  MASS      <value>
  IXX <value> IYY <value> IZZ <value>
  IXY <value> IYZ <value> IXZ <value>
  DENSITYFACTOR <factor>
  POSITION    <xCoG> <yCoG> <zCoG>
  OUTPUT     ENERGY
ENDGENERIC3D
```

2.6 FLEXBODIES Section

```
FLEXBODIES
  MESHFREE <name> ... ENDMESHFREE
  FEMESH   <name> ... ENDFEMESH
  SHARENODES <nameA> <nameB>
ENDFLEXBODIES
```

SHARENODES makes two bodies share their node lists (for coupled domains).

2.6.1 MESHFREE / FEMESH

Both use the same sub-keywords:

```
MESHFREE <name>
  FORMULATION <THERMAL|MECHANICAL|THERMOMECHANICAL>
  METHOD       <EFG|RPIM>
  OUTPUT      <STRESS|ENERGY>
  INITIALTEMPERATURE <value>
  BOUNDARYGROUP <bgName> ... ENDBOUNDARYGROUP
  NODES       ...
  CELLS       ...
  MESH        ...
ENDMESHFREE
```

```
2.6.1.1 BOUNDARYGROUP BOUNDARYGROUP <name>
  METHOD <formulation> <nGPs> <alpha>
  FILE <meshfile>
ENDBOUNDARYGROUP
```

2.6.1.2 NODES – Inline definition Rectangular patch:

```
NODES
RECTANGULAR <nX> <nY> <x1> <y1> <x2> <y2>
```

Creates $(n_x+1) \times (n_y+1)$ nodes on a regular grid from (x_1, y_1) to (x_2, y_2) .

Grid from file (triangles or quads):

```
NODES
GRID TRIANGLES <meshfile>
NODES
GRID QUADS     <meshfile>
```

2.6.1.3 CELLS – Integration cell definition CELLS

```
<mat_id> <nGPs> <alpha> <dc>
RECTANGULAR <mat_id> <dcx> <dcy> <nx> <ny> <x1> <y1> <x2> <y2>
```

2.6.1.4 MESH – Combined node + cell from file MESH

```
<mat_id> <nGPs> <alpha>
TRIANGLES <meshfile>
QUADS <meshfile>
```

2.7 BODYPOINTS Section

```
BODYPOINTS
  <bodyName> <x> <y> <z> // rigid body point
  <bodyName> <x> <y> <z> <alpha> <dc> // flex body point
ENDBODYPOINTS
```

Adds extra nodes to already-defined bodies.

2.8 JOINTS Section

```
JOINTS
  PENALTY | AUGMENTED [<tolerance>]
  ALPHA <value>
  SPHERICAL <name> ... ENDSPHERICAL
  DISTANCE <name> ... ENDDISTANCE
  AXIS <name> ... ENDAXIS
  CLEARANCE <name> ... ENDCLEARANCE
  THERMALSPHERICAL <name> ... ENDTHERMALSPHERICAL
ENDJOINTS
```

The <name> field corresponds to a unique joint name defined by the user. The ... parts will have the following information:

All joint types share NODEA and NODEB sub-commands:

```
NODEA <bodyName>.<nodeId>
NODEB <bodyName>.<nodeId>
```

Use GROUND as the body name to fix a node to the ground frame. For grounded joints, write GROUND without .nodeId:

```
NODEA GROUND
NODEB <bodyName>.<nodeId>
```

or

```
NODEA <bodyName>.<nodeId>
NODEB GROUND
```

2.8.1 AXIS – additional sub-command

```
DIRECTION <x|y|z>
```

2.8.2 AUGMENTED tolerance

```
AUGMENTED <tolerance>
```

Optional tolerance used by augmented constraints to decide convergence. If omitted, the default tolerance is 5 . 0.

2.8.3 CLEARANCE – additional sub-command

```
TOLERANCE <value>
```

2.8.4 THERMALSPHERICAL – additional sub-command

THERMALSPHERICAL supports an optional initial temperature override:

```
TEMPERATURE <value>
```

If TEMPERATURE is provided, it initializes both nodes in that thermal joint with the specified value. If omitted, the nodes are initialized using the global INITIALTEMPERATURE value.

This initialization is persistent through thermal analysis setup (it is not overwritten by the global thermal initialization step).

2.9 LOADS Section

```
LOADS
  FORCE <body>.<node> <fx> <fy> <fz>
  THERMALFLUENCE <body>.<node> <value>
  THERMALOUTPUT <body>.<node>
  THERMALOUTPUT MAX_INTERFACE_TEMP
  THERMALBODY <bodyName> VALUE <value>
  THERMALBODY <bodyName> FILE <filename>
```

```

THERMALBODY      <bodyName> FILE3D   <filename>
THERMALBODY      <bodyName> FILE_VTK <filename>
THERMALBODY      <bodyName> TIMEFILE <timeFilename>
THERMALFLUX1D    <body>.<boundaryGroup> ... ENDTHERMALFLUX1D
RADIATION        ... ENDRADIATION
ENDLOADS

```

2.9.1 THERMALBODY

Applies a volumetric heat source to a thermal body.

Constant value:

```
THERMALBODY <bodyName> VALUE <value>
```

1-D spatial distribution from file:

```
THERMALBODY <bodyName> FILE <filename>
```

The file contains two whitespace-separated columns (X coordinate, load value), one row per point:

```

0.0  1000.0
0.05 1500.0
0.10 1200.0

```

2-D spatial distribution from file (regular grid):

```
THERMALBODY <bodyName> FILE <filename>
```

The file uses a matrix layout. The first row lists the `key2` coordinate values (first cell is ignored). Each subsequent row starts with the `key1` coordinate, followed by the load values for each `key2`:

```

0.0  0.0  0.05  0.10
0.0  1000  1100  1200
0.05 1500  1600  1700
0.10 1200  1300  1400

```

The parser automatically distinguishes 1-D from 2-D based on the number of columns.

3-D spatial distribution from file:

```
THERMALBODY <bodyName> FILE3D <filename>
```

The file contains four whitespace-separated columns (`x`, `y`, `z`, load value), one data row per point. Any line that does not parse as exactly four numbers is silently skipped, so the file may begin with an arbitrary header (units, comments, etc.):

```

# x[m]  y[m]  z[m]  q[W/m3]
0.0  0.0  0.0  1000.0
0.05 0.0  0.0  1500.0
0.05 0.02 0.01 1800.0

```

The load at a query point is obtained by trilinear interpolation through `interpolate3D`.

VTK Unstructured Grid (requires `HAVE_VTK` at build time):

```
THERMALBODY <bodyName> FILE_VTK <file.vtu>
```

Reads a VTK XML Unstructured Grid file (`.vtu`). The first scalar point-data array in the file is used as the source distribution. At each query point the value is computed using VTK's native shape-function interpolation (`vtkCellLocator + FindCell`). Points that fall outside the grid are projected onto the nearest cell face and interpolated there. If `MkniX` is compiled without VTK support (`HAVE_VTK` not defined), this option emits an error message and the load is not applied.

Optional time scaling (with `VALUE`, `FILE`, `FILE3D`, or `FILE_VTK`):

```

THERMALBODY <bodyName> VALUE      <value>    TIMEFILE <timeFilename>
THERMALBODY <bodyName> FILE       <filename> TIMEFILE <timeFilename>
THERMALBODY <bodyName> FILE3D     <filename> TIMEFILE <timeFilename>
THERMALBODY <bodyName> FILE_VTK   <filename> TIMEFILE <timeFilename>

```

Pure time-dependent source:

```
THERMALBODY <bodyName> TIMEFILE <timeFilename>
```

`TIMEFILE` is a two-column file: time vs scale factor. For pure time-dependent input, the spatial source defaults to `1.0` and is scaled in time.

2.9.2 THERMALFLUX1D

Applies a 1-D heat flux to a boundary group.

```

THERMALFLUX1D <bodyName>.<boundaryGroupName>
  FILE      <fluxFilename>
  TIMEFILE  <timeFilename>
  SCALE     <factor>
ENDTHERMALFLUX1D

```

- `FILE` – two-column file: coordinate vs flux value
 - `TIMEFILE` – two-column file: time vs scale factor
 - `SCALE` – multiplies all flux values by a constant factor
-

2.9.3 RADIATION

```
RADIATION
  STATIC3D | STATIC2D
  SKIPLINES <n>
  LENGTHFACTOR <factor>
  SCALEAXIS <sx> <sy> <sz>
  DOSEFACTOR <factor>
  MAPFILE <filename>
ENDRADIATION
```

MAPFILE contains rows of <x> <y> [<z>] <dose>. STATIC3D expects 4 columns; STATIC2D expects 3 columns (z is assumed 0). SKIPLINES discards header lines.

2.10 MOTION Section

```
MOTION <nodeId>
  TIMECONF <time> <ux> <uy> <uz> // repeatable
ENDMOTION
```

Prescribes time-varying displacement on a ground node. Multiple TIMECONF entries form a piecewise-linear trajectory.

2.11 SIGNALS Section

```
SIGNALS
  MECHANICALOUTPUT <signalName> <bodyName>.<nodeId>
  MECHANICALINPUT <signalName> <constraintName1> [<constraintName2> ...]
ENDSIGNALS
```

2.12 ANALYSIS Section

```
ANALYSIS
  STATIC ... ENDSTATIC
  DYNAMIC ... ENDDYNAMIC
  THERMALSTATIC ... ENDTHERMALSTATIC
  THERMALDYNAMIC ... ENDTHERMALDYNAMIC
  THERMOMECHANICALDYNAMIC ... ENDTHERMOMECHANICALDYNAMIC
ENDANALYSIS
```

All analysis types share EPSILON and TIME. Transient types also require INTEGRATOR.

2.12.1 STATIC

```
STATIC
  EPSILON <tol>
  TIME <t_end>
ENDSTATIC
```

2.12.2 THERMALSTATIC

```
THERMALSTATIC
  EPSILON <tol>
  TIME <t_end>
ENDTHERMALSTATIC
```

2.12.3 THERMALDYNAMIC

```
THERMALDYNAMIC
  EPSILON <tol>
  INTEGRATOR <BDF-1|BDF-2|...>
  TIME <t0> <tf> <dt>
ENDTHERMALDYNAMIC
```

2.12.4 THERMOMECHANICALDYNAMIC

```
THERMOMECHANICALDYNAMIC
  EPSILON <tol>
  INTEGRATOR <type>
  TIME <t0> <tf> <dt>
ENDTHERMOMECHANICALDYNAMIC
```

2.12.5 DYNAMIC

```
DYNAMIC
  EPSILON <tol>
  INTEGRATOR NEWMARK <beta> <gamma>
  INTEGRATOR NEWMARK-ALPHA <beta> <gamma>
```

```

INTEGRATOR HHT-SIMPLE      <alpha>
INTEGRATOR HHT-GENERALIZED <alpha>
TIME        <t0> <tf> <dt>
ENDDYNAMIC

```

2.13 Complete Minimal Example

```

// Thermal transient analysis of a two-triangle FEM mesh
TITLE 2triangfem
DIMENSION 2
MATERIALS
  THERMAL 1 1348 224 0 1750
ENDMATERIALS
SYSTEM 2triangles
FLEXBODIES
  FEMESH triangle
  FORMULATION THERMAL
  MESH
    1 1 3.5
  TRIANGLES 2triangles.dat
ENDFEMESH
ENDFLEXBODIES
JOINTS
ENDJOINTS
LOADS
  THERMALFLUENCE triangle.2 5E6
ENDLOADS
ENDSYSTEM
ANALYSIS
  THERMALDYNAMIC
    EPSILON 1E-6
    INTEGRATOR BDF-1
    TIME 0.0 10E-3 1E-3
  ENDTHERMALDYNAMIC
ENDANALYSIS

```

3 Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Imx	20
mknix	20

4 Hierarchical Index

4.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

mknix::Analysis	28
mknix::AnalysisDynamic	31
mknix::AnalysisStatic	34
mknix::AnalysisThermalDynamic	36
mknix::AnalysisThermalStatic	39
mknix::AnalysisThermoMechanicalDynamic	41
mknix::Body	44
mknix::FlexBody	129
mknix::FlexFrameGalerkin	136

mknix::FlexGlobalGalerkin	142
mknix::RigidBody	249
mknix::RigidBar	244
mknix::RigidBody2D	256
mknix::RigidBody3D	260
mknix::RigidBodyMassPoint	265
mknix::BoundaryGroup	57
mknix::boxFIR	59
mknix::Cell	60
mknix::CellRect	79
mknix::CellTetrahedron	83
mknix::ElemTetrahedron	123
mknix::CellTriang	86
mknix::ElemTriangle	126
mknix::CellBoundary	71
mknix::CellBoundaryLinear	75
mknix::CompBar	91
mknix::Constraint	92
mknix::ConstraintClearance	100
mknix::ConstraintContact	103
mknix::ConstraintDistance	106
mknix::ConstraintFixedAxis	110
mknix::ConstraintFixedCoordinates	113
mknix::ConstraintThermal	116
mknix::ConstraintThermalFixed	118
mknix::Contact	121
Imx::DenseMatrix< T >	122
Imx::DenseMatrix< data_type >	122
Imx::DenseMatrix< double >	122
mknix::Load	183
mknix::Force	148
mknix::Radiation	236

mknix::LoadThermal	185
mknix::LoadThermalBody	188
mknix::LoadThermalBoundary1D	191
mknix::Material	193
lmx::Matrix< T >	206
lmx::Matrix< data_type >	206
mknix::MknixWrapper	206
mknix::Point	221
mknix::GaussPoint	150
mknix::GaussPoint2D	159
mknix::GaussPoint3D	169
mknix::GaussPointBoundary	179
mknix::Node	210
mknix::Reader	238
mknix::ReaderConstraints	240
mknix::ReaderFlex	241
mknix::ReaderRigid	243
mknix::SectionReader	268
mknix::ShapeFunction	269
mknix::ShapeFunctionLinear	274
mknix::ShapeFunctionLinearX	277
mknix::ShapeFunctionMLS	279
mknix::ShapeFunctionMLS2D	281
mknix::ShapeFunctionMLS3D	282
mknix::ShapeFunctionRBF	284
mknix::ShapeFunctionTetrahedron	287
mknix::ShapeFunctionTriangle	290
mknix::ShapeFunctionTriangleSigned	292
mknix::Simulation	294
mknix::MknixWrapper::SimulationWrapper	325
mknix::System	325
mknix::Motion	207

mknix::SystemChain	338
mknix::ThermalBody	342
lmx::Vector< T >	347
lmx::Vector< data_type >	347
lmx::Vector< double >	347

5 Class Index

5.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

mknix::Analysis	28
mknix::AnalysisDynamic	31
mknix::AnalysisStatic	34
mknix::AnalysisThermalDynamic	36
mknix::AnalysisThermalStatic	39
mknix::AnalysisThermoMechanicalDynamic	41
mknix::Body	44
mknix::BoundaryGroup	57
mknix::boxFIR	59
mknix::Cell	60
mknix::CellBoundary	71
mknix::CellBoundaryLinear	75
mknix::CellRect	79
mknix::CellTetrahedron	83
mknix::CellTriang	86
mknix::CompBar	91
mknix::Constraint	92
mknix::ConstraintClearance	100
mknix::ConstraintContact	103
mknix::ConstraintDistance	106
mknix::ConstraintFixedAxis	110
mknix::ConstraintFixedCoordinates	113
mknix::ConstraintThermal	116

mknix::ConstraintThermalFixed	118
mknix::Contact	121
lmx::DenseMatrix< T >	122
mknix::ElemTetrahedron	123
mknix::ElemTriangle	126
mknix::FlexBody	129
mknix::FlexFrameGalerkin	136
mknix::FlexGlobalGalerkin	142
mknix::Force	148
mknix::GaussPoint	150
mknix::GaussPoint2D	159
mknix::GaussPoint3D	169
mknix::GaussPointBoundary	179
mknix::Load	183
mknix::LoadThermal	185
mknix::LoadThermalBody	188
mknix::LoadThermalBoundary1D	191
mknix::Material	193
lmx::Matrix< T >	206
mknix::MknixWrapper	206
mknix::Motion	207
mknix::Node	210
mknix::Point	221
mknix::Radiation	236
mknix::Reader	238
mknix::ReaderConstraints	240
mknix::ReaderFlex	241
mknix::ReaderRigid	243
mknix::RigidBar	244
mknix::RigidBody	249
mknix::RigidBody2D	256
mknix::RigidBody3D	260

mknix::RigidBodyMassPoint	265
mknix::SectionReader	268
mknix::ShapeFunction	269
mknix::ShapeFunctionLinear	274
mknix::ShapeFunctionLinearX	277
mknix::ShapeFunctionMLS	279
mknix::ShapeFunctionMLS2D	281
mknix::ShapeFunctionMLS3D	282
mknix::ShapeFunctionRBF	284
mknix::ShapeFunctionTetrahedron	287
mknix::ShapeFunctionTriangle	290
mknix::ShapeFunctionTriangleSigned	292
mknix::Simulation	294
mknix::MknixWrapper::SimulationWrapper	325
mknix::System	325
mknix::SystemChain	338
mknix::ThermalBody	342
Imx::Vector< T >	347

6 File Index

6.1 File List

Here is a list of all files with brief descriptions:

analysis.cpp	347
analysis.h	348
analysisdynamic.cpp	349
analysisdynamic.h	349
analysisstatic.cpp	350
analysisstatic.h	350
analysisthermaldynamic.cpp	351
analysisthermaldynamic.h	352
analysisthermalstatic.cpp	353
analysisthermalstatic.h	353

analysisisthermomechanicaldynamic.cpp	354
analysisisthermomechanicaldynamic.h	354
body.cpp	355
body.h	356
bodyflex.cpp	357
bodyflex.h	357
bodyflexframegalerkin.cpp	358
bodyflexframegalerkin.h	359
bodyflexglobalgalerkin.cpp	360
bodyflexglobalgalerkin.h	361
bodyrigid.cpp	361
bodyrigid.h	362
bodyrigid0D.cpp	363
bodyrigid0D.h	363
bodyrigid1D.cpp	364
bodyrigid1D.h	365
bodyrigid2D.cpp	366
bodyrigid2D.h	367
bodyrigid3D.cpp	367
bodyrigid3D.h	368
bodythermal.cpp	369
bodythermal.h	369
boundarygroup.cpp	370
boundarygroup.h	371
cell.cpp	371
cell.h	
Background cells for integration	372
cellboundary.cpp	373
cellboundary.h	373
cellboundarylinear.cpp	374
cellboundarylinear.h	
Background cells for boundary integration	374
cellrect.cpp	375

cellrect.h	
Background cells for integration	376
celltetrahedron.cpp	377
celltetrahedron.h	
Background cells for integration	378
celltriang.cpp	379
celltriang.h	
Background cells for integration	379
common.cpp	380
common.h	381
compbar.cpp	382
compbar.h	382
constraint.cpp	382
constraint.h	383
constraintclearance.cpp	383
constraintclearance.h	384
constraintcontact.cpp	385
constraintcontact.h	385
constraintdistance.cpp	386
constraintdistance.h	386
constraintfixedaxis.cpp	387
constraintfixedaxis.h	388
constraintfixedcoordinates.cpp	388
constraintfixedcoordinates.h	389
constraintthermal.cpp	390
constraintthermal.h	390
constraintthermalfixed.cpp	391
constraintthermalfixed.h	391
dictionary.h	392
elemtetrahedron.cpp	392
elemtetrahedron.h	393
elemtriangle.cpp	394
elemtriangle.h	394

force.cpp	395
force.h	396
gausspoint.cpp	397
gausspoint.h	
Point for numerical integration	397
gausspoint2D.cpp	398
gausspoint2D.h	399
gausspoint3D.cpp	399
gausspoint3D.h	400
gausspointboundary.cpp	401
gausspointboundary.h	401
generalcontact.cpp	402
generalcontact.h	402
load.cpp	403
load.h	403
loadradiation.cpp	404
loadradiation.h	405
loadthermal.cpp	406
loadthermal.h	406
loadthermalbody.cpp	407
loadthermalbody.h	407
loadthermalboundary1D.cpp	408
loadthermalboundary1D.h	409
material.cpp	410
material.h	410
mknixWrapper.cpp	411
mknixWrapper.h	411
motion.cpp	412
motion.h	413
node.cpp	413
node.h	414
point.cpp	415

point.h	
Point of interest	415
reader.cpp	416
reader.h	416
readerconstraints.cpp	417
readerconstraints.h	418
readerflex.cpp	419
readerflex.h	419
readerrigid.cpp	420
readerrigid.h	421
sectionreader.cpp	421
sectionreader.h	422
shapefunction.cpp	423
shapefunction.h	
Function for interpolation of basic variables	424
shapefunctionlinear-x.cpp	425
shapefunctionlinear-x.h	425
shapefunctionlinear.cpp	426
shapefunctionlinear.h	426
shapefunctionMLS.cpp	427
shapefunctionMLS.h	
Function for aproximation of basic variables by Moving Least Squares fit	428
shapefunctionMLS2D.cpp	429
shapefunctionMLS2D.h	430
shapefunctionMLS3D.cpp	430
shapefunctionMLS3D.h	431
shapefunctionRBF.cpp	432
shapefunctionRBF.h	
Function for interpolation of basic variables by means of Radial basis Functions	432
shapefunctiontetrahedron.cpp	433
shapefunctiontetrahedron.h	434
shapefunctiontriangle.cpp	435
shapefunctiontriangle.h	435
shapefunctiontriangle3D.cpp	436

shapefunctiontriangle3D.h	437
simulation.cpp	438
simulation.h	439
system.cpp	439
system.h	440
systemchain.cpp	441
systemchain.h	441

7 Namespace Documentation

7.1 Imx Namespace Reference

Classes

- class [Vector](#)
- class [Matrix](#)
- class [DenseMatrix](#)

7.2 mknix Namespace Reference

Classes

- class [boxFIR](#)
- class [Cell](#)
- class [CellBoundary](#)
- class [CellBoundaryLinear](#)
- class [CellRect](#)
- class [CellTetrahedron](#)
- class [CellTriang](#)
- class [CompBar](#)
- class [ElemTetrahedron](#)
- class [ElemTriangle](#)
- class [GaussPoint](#)
- class [GaussPoint2D](#)
- class [GaussPoint3D](#)
- class [GaussPointBoundary](#)
- class [Material](#)
- class [Node](#)
- class [Point](#)
- class [ShapeFunction](#)
- class [ShapeFunctionLinearX](#)
- class [ShapeFunctionLinear](#)
- class [ShapeFunctionMLS](#)
- class [ShapeFunctionMLS2D](#)
- class [ShapeFunctionMLS3D](#)
- class [ShapeFunctionRBF](#)
- class [ShapeFunctionTetrahedron](#)
- class [ShapeFunctionTriangle](#)
- class [ShapeFunctionTriangleSigned](#)
- class [Reader](#)
- class [ReaderConstraints](#)

- class [ReaderFlex](#)
- class [ReaderRigid](#)
- class [SectionReader](#)
- class [Analysis](#)
- class [AnalysisDynamic](#)
- class [AnalysisStatic](#)
- class [AnalysisThermalDynamic](#)
- class [AnalysisThermalStatic](#)
- class [AnalysisThermoMechanicalDynamic](#)
- class [MknixWrapper](#)
- class [Simulation](#)
- class [Body](#)
- class [FlexBody](#)
- class [FlexFrameGalerkin](#)
- class [FlexGlobalGalerkin](#)
- class [RigidBody](#)
- class [RigidBodyMassPoint](#)
- class [RigidBodyBar](#)
- class [RigidBody2D](#)
- class [RigidBody3D](#)
- class [ThermalBody](#)
- class [BoundaryGroup](#)
- class [Constraint](#)
- class [ConstraintClearance](#)
- class [ConstraintContact](#)
- class [ConstraintDistance](#)
- class [ConstraintFixedAxis](#)
- class [ConstraintFixedCoordinates](#)
- class [ConstraintThermal](#)
- class [ConstraintThermalFixed](#)
- class [Force](#)
- class [Contact](#)
- class [Load](#)
- class [Radiation](#)
- class [LoadThermal](#)
- class [LoadThermalBody](#)
- class [LoadThermalBoundary1D](#)
- class [Motion](#)
- class [System](#)
- class [SystemChain](#)

Typedefs

- typedef double [data_type](#)

Enumerations

- enum class [SourceType](#) {
 [CONSTANT](#) = 0 , [SOURCE_1D](#) = 1 , [SOURCE_2D](#) = 2 , [SOURCE_3D](#) = 3 ,
 [SOURCE_VTK](#) = 4 }

Identifies which spatial source is active, in ascending priority order.

Functions

- double [interpolate1D](#) (double key, const std::map< double, double > &theMap)
Performs linear interpolation on a 1D map of (key, value) pairs.
- double [interpolate2D](#) (double key1, double key2, const std::map< double, std::map< double, double >> &the2DMap)
Performs bilinear interpolation on a 2D nested map of data points.
- double [interpolate3D](#) (double key1, double key2, double key3, const std::map< double, std::map< double, std::map< double, double >>> &the3DMap)
Performs trilinear interpolation on a 3D nested map of data points.
- std::vector< double > [doubles_in_vector](#) (const std::string &str)
Parses all whitespace-separated double values from a string.
- std::vector< std::vector< double > > [read_lines](#) (std::istream &stm)
Reads all lines from an input stream and parses each as a row of doubles.
- bool [isNumber](#) (const std::string &str)
Checks if a string represents a number.
- void [readFile3D](#) (const std::string &fileName, std::map< double, std::map< double, std::map< double, double >>> &dest)
Reads a 3D source distribution file into a nested map.
- template<class T, class... Args>
std::unique_ptr< T > [make_unique](#) (Args &&... args)
- void [setSignal](#) (std::string node, std::vector< double >)
Placeholder function for setting a signal on a node (currently a no-op).
- std::vector< double > [getSignal](#) (const std::string &node)
Placeholder function for retrieving a signal from a node (currently returns an empty vector).
- constexpr double [deg2rad](#) (double deg)

7.2.1 Detailed Description

Author

Daniel Iglesias <daniel@extremo>

7.2.2 Typedef Documentation

7.2.2.1 data_type `typedef double mknix::data\_type`

Definition at line 46 of file common.h.

7.2.3 Enumeration Type Documentation

7.2.3.1 SourceType `enum mknix::SourceType [strong]`

Identifies which spatial source is active, in ascending priority order.

Enumerator

CONSTANT	
SOURCE_1D	
SOURCE_2D	
SOURCE_3D	
SOURCE_VTK	

Definition at line 35 of file loadthermalbody.h.

7.2.4 Function Documentation

7.2.4.1 deg2rad() `constexpr double mknix::deg2rad ( double deg ) [constexpr]`

Definition at line 518 of file body.cpp.

Here is the caller graph for this function:



7.2.4.2 doubles_in_vector() `std::vector< double > mknix::doubles_in_vector ( const std::string & str )`

Parses all whitespace-separated double values from a string.

Uses a string stream and an istream_iterator to extract every floating-point number present in the string, from left to right.

Parameters

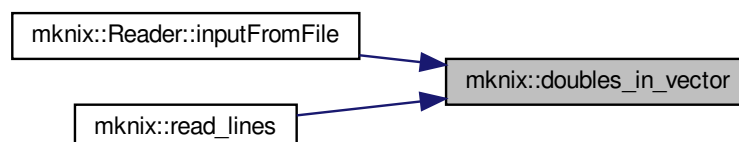
<i>str</i>	Input string containing whitespace-separated numeric values.
------------	--

Returns

A vector of doubles parsed from the string, in order.

Definition at line 285 of file common.cpp.

Here is the caller graph for this function:



7.2.4.3 getSignal() `std::vector<double> mknix::getSignal ( const std::string & node )`

Placeholder function for retrieving a signal from a node (currently returns an empty vector).

Parameters

<i>node</i>	Name of the node.
-------------	-------------------

Returns

An empty vector.

Definition at line 482 of file simulation.cpp.

7.2.4.4 interpolate1D() `double mknix::interpolate1D (`
`double key,`
`const std::map< double, double > & theMap )`

Performs linear interpolation on a 1D map of (key, value) pairs.

Given a lookup key and a sorted map of data points, returns the linearly interpolated value. If the key is beyond the upper bound of the map the last value is returned; if it is below the lower bound the first value is returned.

Parameters

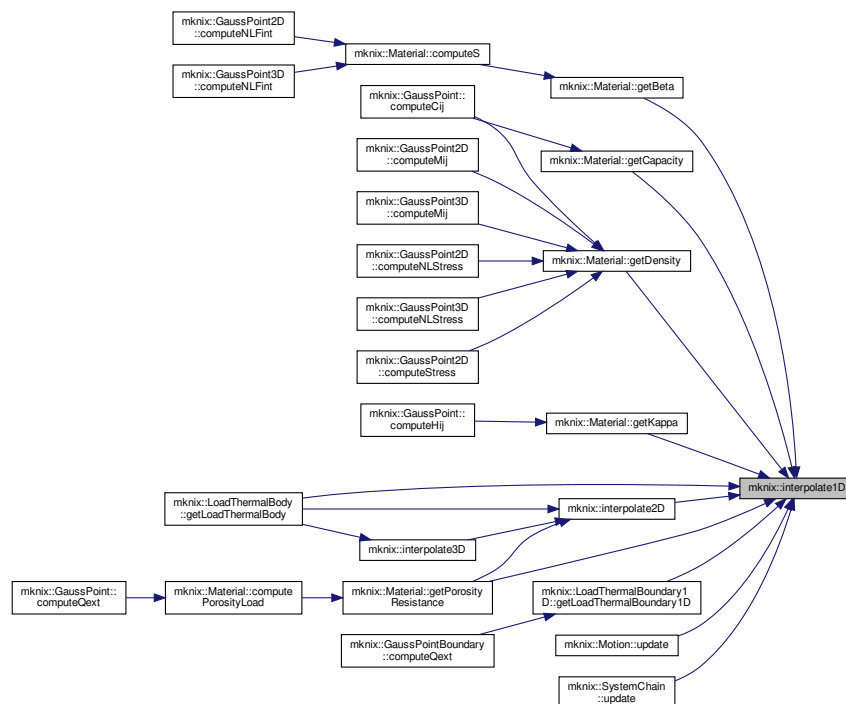
<i>key</i>	The lookup key to interpolate at.
<i>theMap</i>	A sorted map of (key, value) data points.

Returns

The interpolated value at the given key.

Definition at line 48 of file common.cpp.

Here is the caller graph for this function:



7.2.4.5 interpolate2D() `double mknix::interpolate2D (`
`double key1,`
`double key2,`
`const std::map< double, std::map< double, double >> & the2DMap )`

Performs bilinear interpolation on a 2D nested map of data points.

Interpolates along the first key axis by finding the two bracketing rows and calling interpolate1D on each row for the second key, then linearly combines the two results. Returns 0.0 if the map is empty.

Parameters

<i>key1</i>	The first-axis lookup key.
<i>key2</i>	The second-axis lookup key.
<i>the2DMap</i>	A sorted nested map: outer key → inner (key, value) map.

Returns

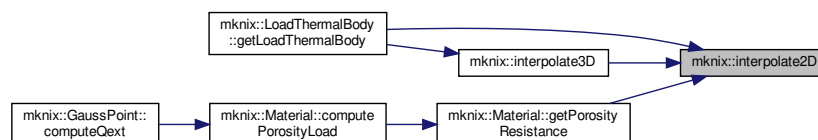
The bilinearly interpolated value at (key1, key2).

Definition at line 80 of file common.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4.6 interpolate3D() `double mknix::interpolate3D (`
`double key1,`
`double key2,`
`double key3,`
`const std::map< double, std::map< double, std::map< double, double >>> & the3DMap )`

Performs trilinear interpolation on a 3D nested map of data points.

Interpolates along the first key axis by finding the two bracketing slices and calling interpolate2D on each slice for (key2, key3), then linearly combines the two results. Returns 0.0 if the map is empty.

Parameters

<i>key1</i>	The first-axis lookup key (x coordinate).
-------------	---

Parameters

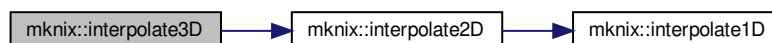
<i>key2</i>	The second-axis lookup key (y coordinate).
<i>key3</i>	The third-axis lookup key (z coordinate).
<i>the3DMap</i>	A sorted nested map: outer key → 2D nested map of (key, value) pairs.

Returns

The trilinearly interpolated value at (key1, key2, key3).

Definition at line 139 of file common.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



7.2.4.7 isNumber() `bool mknix::isNumber ( const std::string & str )`

Checks if a string represents a number.

Parameters

<i>str</i>	The string to check. Can be empty.
------------	------------------------------------

Returns

True if the string represents a number, False otherwise.

Definition at line 318 of file common.cpp.

7.2.4.8 make_unique() `template<class T , class... Args> std::unique_ptr<T> mknix::make_unique ( Args &&... args )`

Stand-in for `std::make_unique` included in C++14

Definition at line 61 of file common.h.

7.2.4.9 read_lines() `std::vector< std::vector< double > > mknix::read_lines ( std::istream & stm )`

Reads all lines from an input stream and parses each as a row of doubles.

Iterates over lines until EOF, calling `doubles_in_vector` on each line and collecting the resulting rows into a 2D vector.

Parameters

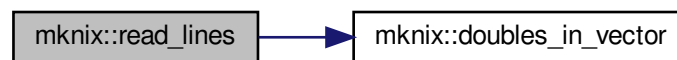
<i>stm</i>	Input stream to read lines from.
------------	----------------------------------

Returns

A 2D vector where each inner vector contains the doubles from one line.

Definition at line 305 of file `common.cpp`.

Here is the call graph for this function:



7.2.4.10 readFile3D() `void mknix::readFile3D ( const std::string & fileName, std::map< double, std::map< double, std::map< double, double >>> & dest )`

Reads a 3D source distribution file into a nested map.

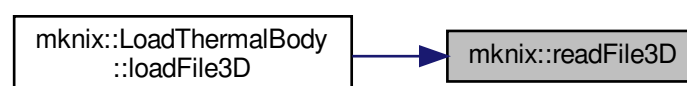
Reads the file line by line. Lines that cannot be parsed as exactly four whitespace-separated floating-point numbers (x, y, z, value) are silently skipped, allowing the file to contain an arbitrary header with units or other human-readable information.

Parameters

<i>fileName</i>	Path to the file containing the 3D source data.
<i>dest</i>	Output nested map to populate: <code>dest[x][y][z] = value</code> .

Definition at line 338 of file `common.cpp`.

Here is the caller graph for this function:



7.2.4.11 setSignal() `void mknix::setSignal (`
`std::string node,`
`std::vector< double > )`

Placeholder function for setting a signal on a node (currently a no-op).

Parameters

<i>node</i>	Name of the node.
-------------	-------------------

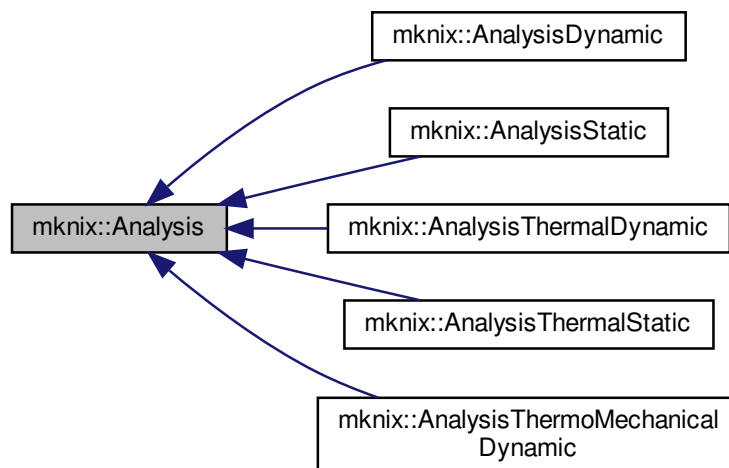
Definition at line 472 of file simulation.cpp.

8 Class Documentation

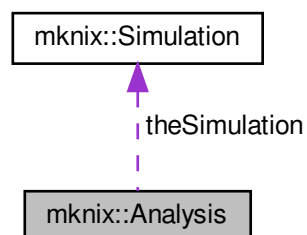
8.1 mknix::Analysis Class Reference

```
#include <analysis.h>
```

Inheritance diagram for mknix::Analysis:



Collaboration diagram for mknix::Analysis:



Public Member Functions

- [Analysis](#) ()
- [Analysis](#) ([Simulation](#) *)
- virtual [~Analysis](#) ()
- virtual std::string [type](#) ()=0
- void [setEpsilon](#) (double epsilon_in)
- virtual void [solve](#) ([lmx::Vector](#)< [data_type](#) > *, [lmx::Vector](#)< [data_type](#) > *=0, [lmx::Vector](#)< [data_type](#) > *=0)=0
- virtual void [init](#) ([lmx::Vector](#)< [data_type](#) > *q_in, [lmx::Vector](#)< [data_type](#) > *qdot_in, int verbosity)=0
- virtual void [nextStep](#) ()=0

Protected Attributes

- [Simulation](#) * [theSimulation](#)
- double [epsilon](#)

8.1.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 37 of file analysis.h.

8.1.2 Constructor & Destructor Documentation

8.1.2.1 [Analysis](#)() [1/2] `mknix::Analysis::Analysis ( )`

Definition at line 26 of file analysis.cpp.

8.1.2.2 [Analysis](#)() [2/2] `mknix::Analysis::Analysis ( Simulation * simulation\_in )`

Definition at line 31 of file analysis.cpp.

8.1.2.3 [~Analysis](#)() `mknix::Analysis::~~Analysis ( )` [virtual]

Definition at line 38 of file analysis.cpp.

8.1.3 Member Function Documentation

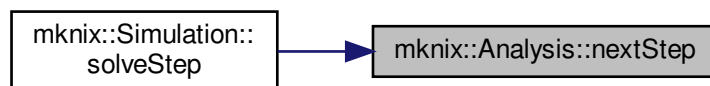
8.1.3.1 [init](#)() `virtual void mknix::Analysis::init ( lmx::Vector< data\_type > * q\_in, lmx::Vector< data\_type > * qdot\_in, int verbosity )` [pure virtual]

Implemented in [mknix::AnalysisThermoMechanicalDynamic](#), [mknix::AnalysisThermalStatic](#), [mknix::AnalysisThermalDynamic](#), [mknix::AnalysisStatic](#), and [mknix::AnalysisDynamic](#).

8.1.3.2 nextStep() `virtual void mknix::Analysis::nextStep ( ) [pure virtual]`

Implemented in [mknix::AnalysisThermoMechanicalDynamic](#), [mknix::AnalysisThermalStatic](#), [mknix::AnalysisThermalDynamic](#), [mknix::AnalysisStatic](#), and [mknix::AnalysisDynamic](#).

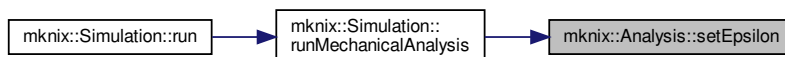
Here is the caller graph for this function:



8.1.3.3 setEpsilon() `void mknix::Analysis::setEpsilon ( double epsilon_in ) [inline]`

Definition at line 48 of file `analysis.h`.

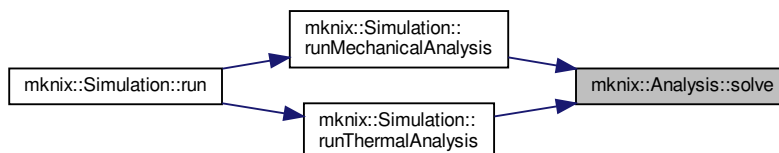
Here is the caller graph for this function:



8.1.3.4 solve() `virtual void mknix::Analysis::solve ( lmx::Vector< data_type > * , lmx::Vector< data_type > * = 0, lmx::Vector< data_type > * = 0 ) [pure virtual]`

Implemented in [mknix::AnalysisThermoMechanicalDynamic](#), [mknix::AnalysisThermalStatic](#), [mknix::AnalysisThermalDynamic](#), [mknix::AnalysisStatic](#), and [mknix::AnalysisDynamic](#).

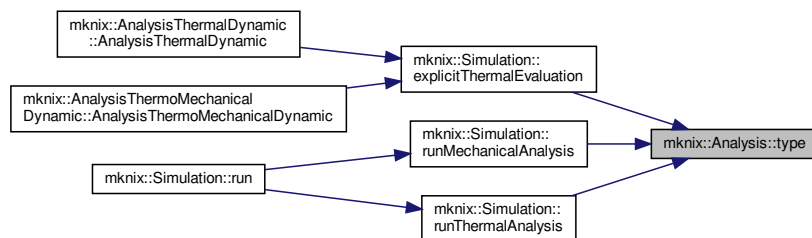
Here is the caller graph for this function:



8.1.3.5 type() `virtual std::string mknix::Analysis::type ( ) [pure virtual]`

Implemented in [mknix::AnalysisThermoMechanicalDynamic](#), [mknix::AnalysisThermalStatic](#), [mknix::AnalysisThermalDynamic](#), [mknix::AnalysisStatic](#), and [mknix::AnalysisDynamic](#).

Here is the caller graph for this function:



8.1.4 Member Data Documentation

8.1.4.1 `epsilon` `double mknix::Analysis::epsilon` [protected]

Definition at line 63 of file `analysis.h`.

8.1.4.2 `theSimulation` `Simulation* mknix::Analysis::theSimulation` [protected]

Definition at line 62 of file `analysis.h`.

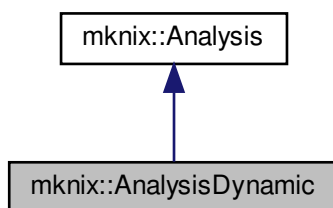
The documentation for this class was generated from the following files:

- [analysis.h](#)
- [analysis.cpp](#)

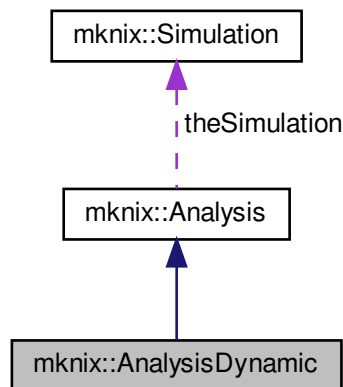
8.2 mknix::AnalysisDynamic Class Reference

```
#include <analysisdynamic.h>
```

Inheritance diagram for `mknix::AnalysisDynamic`:



Collaboration diagram for `mknix::AnalysisDynamic`:



Public Member Functions

- [AnalysisDynamic\(\)](#)
- [AnalysisDynamic\(Simulation *, double, double, double, const char *, double, double, double\)](#)
- [~AnalysisDynamic\(\)](#)
- `std::string type()`
- `void solve(lmx::Vector< data_type > *, lmx::Vector< data_type > *, lmx::Vector< data_type > *)` override
- `void nextStep()` override
- `void init(lmx::Vector< data_type > *q_in, lmx::Vector< data_type > *qdot_in, int verbosity)` override

Additional Inherited Members

8.2.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `analysisdynamic.h`.

8.2.2 Constructor & Destructor Documentation

8.2.2.1 AnalysisDynamic() [1/2] `mknix::AnalysisDynamic::AnalysisDynamic()`

Definition at line 26 of file `analysisdynamic.cpp`.

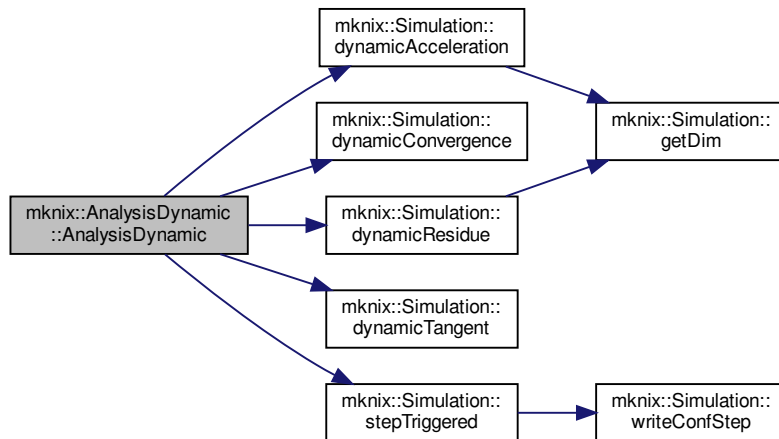
8.2.2.2 AnalysisDynamic() [2/2] `mknix::AnalysisDynamic::AnalysisDynamic()`

```

Simulation * simulation_in,
double to_in,
double tf_in,
double At_in,
const char * integrator_in,
double par1 = -1.,
double par2 = -1.,
double par3 = -1. )

```

Definition at line 32 of file analysisdynamic.cpp.
Here is the call graph for this function:



8.2.2.3 ~AnalysisDynamic() mknix::AnalysisDynamic::~~AnalysisDynamic ()

Definition at line 83 of file analysisdynamic.cpp.

8.2.3 Member Function Documentation

8.2.3.1 init() void mknix::AnalysisDynamic::init (
 lmx::Vector< data_type > * q_in,
 lmx::Vector< data_type > * qdot_in,
 int verbosity) [override], [virtual]

Implements [mknix::Analysis](#).

Definition at line 107 of file analysisdynamic.cpp.

8.2.3.2 nextStep() void mknix::AnalysisDynamic::nextStep () [override], [virtual]

Implements [mknix::Analysis](#).

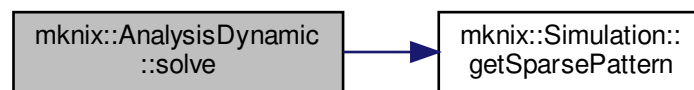
Definition at line 102 of file analysisdynamic.cpp.

8.2.3.3 solve() void mknix::AnalysisDynamic::solve (
 lmx::Vector< data_type > * q_in,
 lmx::Vector< data_type > * qdot_in,
 lmx::Vector< data_type > *) [override], [virtual]

Implements [mknix::Analysis](#).

Definition at line 88 of file analysisdynamic.cpp.

Here is the call graph for this function:



8.2.3.4 type() `std::string mknix::AnalysisDynamic::type ( ) [inline], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 40 of file `analysisdynamic.h`.

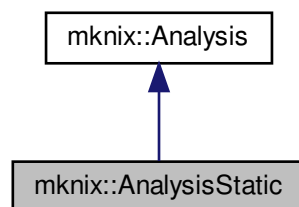
The documentation for this class was generated from the following files:

- [analysisdynamic.h](#)
- [analysisdynamic.cpp](#)

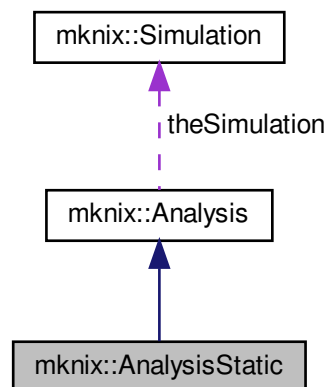
8.3 mknix::AnalysisStatic Class Reference

```
#include <analysisstatic.h>
```

Inheritance diagram for `mknix::AnalysisStatic`:



Collaboration diagram for mknix::AnalysisStatic:



Public Member Functions

- [AnalysisStatic](#) ()
- [AnalysisStatic](#) ([Simulation](#) *, double)
- [~AnalysisStatic](#) ()
- [std::string type](#) ()
- void [solve](#) ([lmx::Vector](#)< [data_type](#) > *, [lmx::Vector](#)< [data_type](#) > *, [lmx::Vector](#)< [data_type](#) > *) override
- void [nextStep](#) () override
- void [init](#) ([lmx::Vector](#)< [data_type](#) > *q_in, [lmx::Vector](#)< [data_type](#) > *qdot_in, int verbosity) override

Additional Inherited Members

8.3.1 Detailed Description

Author

AUTHORS <MAILS>

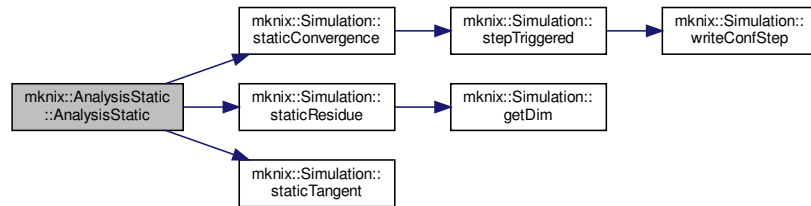
Definition at line 31 of file analysisstatic.h.

8.3.2 Constructor & Destructor Documentation

8.3.2.1 AnalysisStatic() [1/2] `mknix::AnalysisStatic::AnalysisStatic ( )`
 Definition at line 26 of file analysisstatic.cpp.

8.3.2.2 AnalysisStatic() [2/2] `mknix::AnalysisStatic::AnalysisStatic (
 Simulation * simulation_in,
 double time_in)`
 Definition at line 32 of file analysisstatic.cpp.

Here is the call graph for this function:



8.3.2.3 ~AnalysisStatic() `mknix::AnalysisStatic::~~AnalysisStatic ( )`
 Definition at line 44 of file `analysisstatic.cpp`.

8.3.3 Member Function Documentation

8.3.3.1 init() `void mknix::AnalysisStatic::init (`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in,`
`int verbosity ) [inline], [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 49 of file `analysisstatic.h`.

8.3.3.2 nextStep() `void mknix::AnalysisStatic::nextStep ( ) [inline], [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 47 of file `analysisstatic.h`.

8.3.3.3 solve() `void mknix::AnalysisStatic::solve (`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in = 0,`
`lmx::Vector< data_type > * not_used = 0 ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 49 of file `analysisstatic.cpp`.

8.3.3.4 type() `std::string mknix::AnalysisStatic::type ( ) [inline], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 40 of file `analysisstatic.h`.

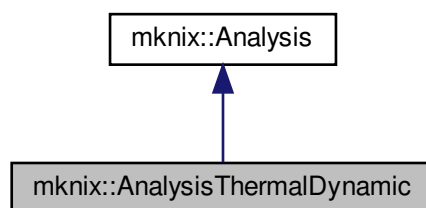
The documentation for this class was generated from the following files:

- [analysisstatic.h](#)
- [analysisstatic.cpp](#)

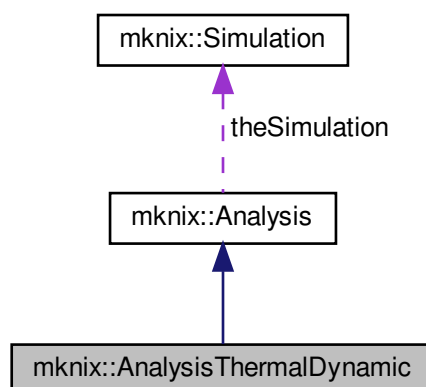
8.4 mknix::AnalysisThermalDynamic Class Reference

```
#include <analysisisthermaldynamic.h>
```

Inheritance diagram for mknix::AnalysisThermalDynamic:



Collaboration diagram for mknix::AnalysisThermalDynamic:



Public Member Functions

- [AnalysisThermalDynamic](#) ()
- [AnalysisThermalDynamic](#) ([Simulation](#) *, double, double, double, char *)
- [~AnalysisThermalDynamic](#) ()
- [std::string type](#) ()
- void [init](#) ([Imx::Vector](#)< [data_type](#) > *q_in, [Imx::Vector](#)< [data_type](#) > *qdot_in, int verbosity) override
- void [nextStep](#) () override
- void [solve](#) ([Imx::Vector](#)< [data_type](#) > *, [Imx::Vector](#)< [data_type](#) > *, [Imx::Vector](#)< [data_type](#) > *) override

Additional Inherited Members

8.4.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `analysisthermaldynamic.h`.

8.4.2 Constructor & Destructor Documentation

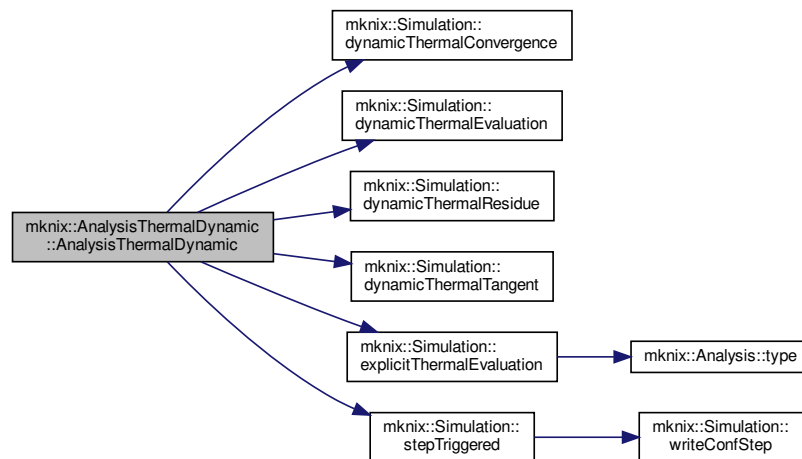
8.4.2.1 AnalysisThermalDynamic() [1/2] `mknix::AnalysisThermalDynamic::AnalysisThermalDynamic ( )`

Definition at line 26 of file `analysisthermaldynamic.cpp`.

8.4.2.2 AnalysisThermalDynamic() [2/2] `mknix::AnalysisThermalDynamic::AnalysisThermalDynamic ( Simulation * simulation_in, double to_in, double tf_in, double At_in, char * integrator_in )`

Definition at line 32 of file `analysisthermaldynamic.cpp`.

Here is the call graph for this function:



8.4.2.3 ~AnalysisThermalDynamic() `mknix::AnalysisThermalDynamic::~~AnalysisThermalDynamic ( )`

Definition at line 67 of file `analysisthermaldynamic.cpp`.

8.4.3 Member Function Documentation

8.4.3.1 init() `void mknix::AnalysisThermalDynamic::init ( lmx::Vector< data_type > * q_in, lmx::Vector< data_type > * qdot_in, int verbosity ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 71 of file `analysisthermaldynamic.cpp`.

8.4.3.2 nextStep() `void mknix::AnalysisThermalDynamic::nextStep ( ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 81 of file analysisthermaldynamic.cpp.

8.4.3.3 solve() `void mknix::AnalysisThermalDynamic::solve (`
`lmx::Vector< data_type > * qt_in,`
`lmx::Vector< data_type > * qdot_in = 0,`
`lmx::Vector< data_type > * not_used = 0 ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 87 of file analysisthermaldynamic.cpp.

8.4.3.4 type() `std::string mknix::AnalysisThermalDynamic::type ( ) [inline], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 40 of file analysisthermaldynamic.h.

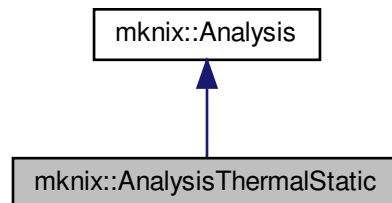
The documentation for this class was generated from the following files:

- [analysisthermaldynamic.h](#)
- [analysisthermaldynamic.cpp](#)

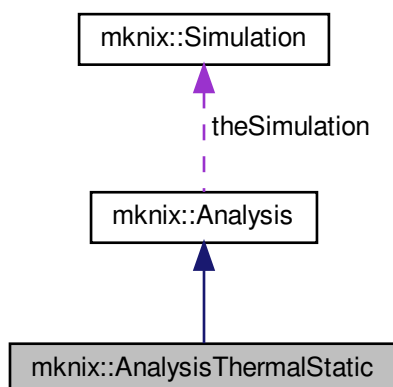
8.5 mknix::AnalysisThermalStatic Class Reference

```
#include <analysisthermalstatic.h>
```

Inheritance diagram for mknix::AnalysisThermalStatic:



Collaboration diagram for `mknix::AnalysisThermalStatic`:



Public Member Functions

- [AnalysisThermalStatic](#) ()
- [AnalysisThermalStatic](#) ([Simulation](#) *, double)
- [~AnalysisThermalStatic](#) ()
- `std::string` [type](#) ()
- void [solve](#) ([lmx::Vector](#)< [data_type](#) > *, [lmx::Vector](#)< [data_type](#) > *, [lmx::Vector](#)< [data_type](#) > *) override
- void [nextStep](#) () override
- void [init](#) ([lmx::Vector](#)< [data_type](#) > *q_in, [lmx::Vector](#)< [data_type](#) > *qdot_in, int verbosity) override

Additional Inherited Members

8.5.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `analysisthermalstatic.h`.

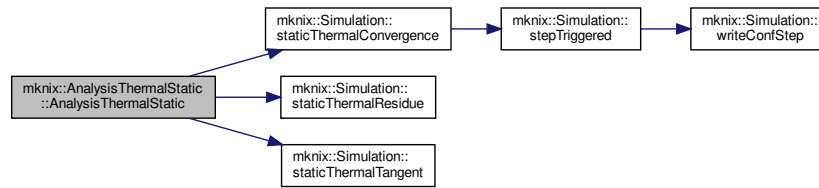
8.5.2 Constructor & Destructor Documentation

8.5.2.1 [AnalysisThermalStatic](#)() [1/2] `mknix::AnalysisThermalStatic::AnalysisThermalStatic ( )`
 Definition at line 26 of file `analysisthermalstatic.cpp`.

8.5.2.2 [AnalysisThermalStatic](#)() [2/2] `mknix::AnalysisThermalStatic::AnalysisThermalStatic (`
 [Simulation](#) * *simulation_in*,
 double *time_in*)

Definition at line 32 of file `analysisthermalstatic.cpp`.

Here is the call graph for this function:



8.5.2.3 ~AnalysisThermalStatic() `mknix::AnalysisThermalStatic::~~AnalysisThermalStatic ( )`
 Definition at line 44 of file `analysisthermalstatic.cpp`.

8.5.3 Member Function Documentation

8.5.3.1 init() `void mknix::AnalysisThermalStatic::init (`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in,`
`int verbosity ) [inline], [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 49 of file `analysisthermalstatic.h`.

8.5.3.2 nextStep() `void mknix::AnalysisThermalStatic::nextStep ( ) [inline], [override], [virtual]`
 Implements [mknix::Analysis](#).

Definition at line 47 of file `analysisthermalstatic.h`.

8.5.3.3 solve() `void mknix::AnalysisThermalStatic::solve (`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in = 0,`
`lmx::Vector< data_type > * not_used = 0 ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 49 of file `analysisthermalstatic.cpp`.

8.5.3.4 type() `std::string mknix::AnalysisThermalStatic::type ( ) [inline], [virtual]`
 Implements [mknix::Analysis](#).

Definition at line 40 of file `analysisthermalstatic.h`.

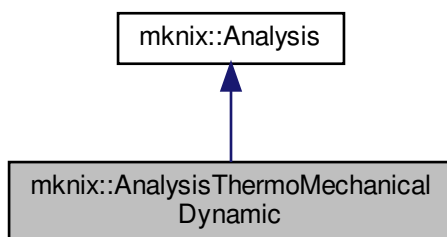
The documentation for this class was generated from the following files:

- [analysisthermalstatic.h](#)
- [analysisthermalstatic.cpp](#)

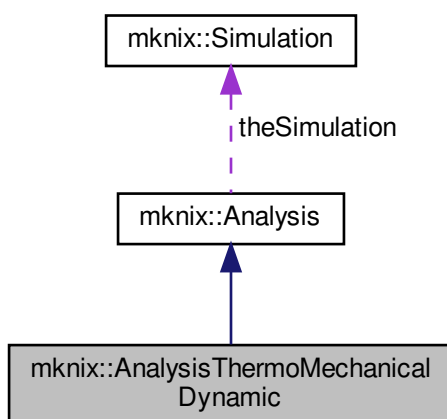
8.6 mknix::AnalysisThermoMechanicalDynamic Class Reference

```
#include <analysisthermomechanicaldynamic.h>
```

Inheritance diagram for `mknix::AnalysisThermoMechanicalDynamic`:



Collaboration diagram for `mknix::AnalysisThermoMechanicalDynamic`:



Public Member Functions

- [AnalysisThermoMechanicalDynamic](#) ()
- [AnalysisThermoMechanicalDynamic](#) ([Simulation](#) *, double, double, double, char *)
- [~AnalysisThermoMechanicalDynamic](#) ()
- `std::string type` ()
- `void solve` ([Imx::Vector](#)< [data_type](#) > *, [Imx::Vector](#)< [data_type](#) > *, [Imx::Vector](#)< [data_type](#) > *) override
- `void nextStep` () override
- `void init` ([Imx::Vector](#)< [data_type](#) > *q_in, [Imx::Vector](#)< [data_type](#) > *qdot_in, int verbosity) override

Additional Inherited Members

8.6.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 33 of file `analysisthermomechanicaldynamic.h`.

8.6.2 Constructor & Destructor Documentation

8.6.2.1 AnalysisThermoMechanicalDynamic() [1/2] mknix::AnalysisThermoMechanicalDynamic::↔

AnalysisThermoMechanicalDynamic ()

Definition at line 26 of file analysisthermomechanicaldynamic.cpp.

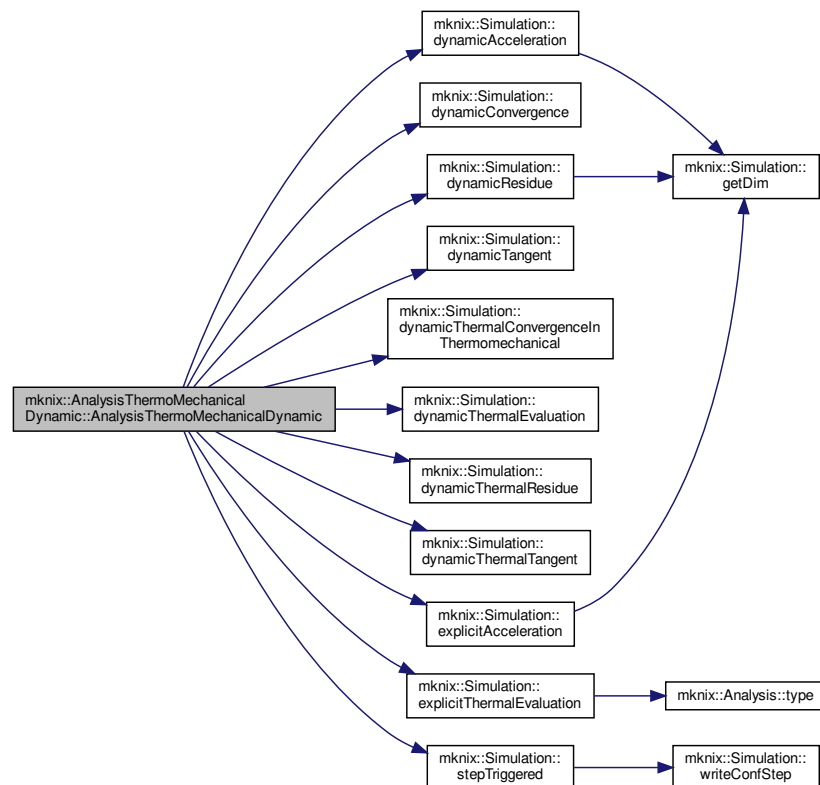
8.6.2.2 AnalysisThermoMechanicalDynamic() [2/2] mknix::AnalysisThermoMechanicalDynamic::↔

AnalysisThermoMechanicalDynamic (

```
Simulation * simulation_in,
double to_in,
double tf_in,
double At_in,
char * integrator_in )
```

Definition at line 32 of file analysisthermomechanicaldynamic.cpp.

Here is the call graph for this function:



8.6.2.3 ~AnalysisThermoMechanicalDynamic() mknix::AnalysisThermoMechanicalDynamic::~~Analysis↔

ThermoMechanicalDynamic ()

Definition at line 89 of file analysisthermomechanicaldynamic.cpp.

8.6.3 Member Function Documentation

8.6.3.1 init() `void mknix::AnalysisThermoMechanicalDynamic::init (`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in,`
`int verbosity ) [inline], [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 51 of file `analysisthermomechanicaldynamic.h`.

8.6.3.2 nextStep() `void mknix::AnalysisThermoMechanicalDynamic::nextStep ( ) [inline], [override],`
`[virtual]`

Implements [mknix::Analysis](#).

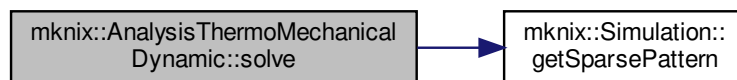
Definition at line 49 of file `analysisthermomechanicaldynamic.h`.

8.6.3.3 solve() `void mknix::AnalysisThermoMechanicalDynamic::solve (`
`lmx::Vector< data_type > * qt_in,`
`lmx::Vector< data_type > * q_in,`
`lmx::Vector< data_type > * qdot_in = 0 ) [override], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 94 of file `analysisthermomechanicaldynamic.cpp`.

Here is the call graph for this function:



8.6.3.4 type() `std::string mknix::AnalysisThermoMechanicalDynamic::type ( ) [inline], [virtual]`

Implements [mknix::Analysis](#).

Definition at line 42 of file `analysisthermomechanicaldynamic.h`.

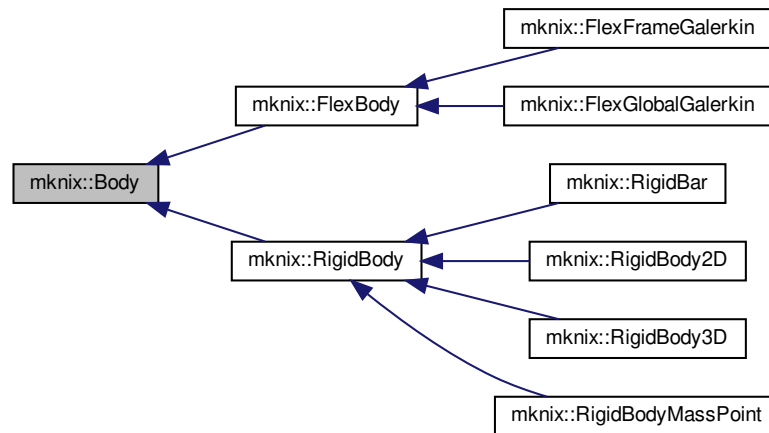
The documentation for this class was generated from the following files:

- [analysisthermomechanicaldynamic.h](#)
- [analysisthermomechanicaldynamic.cpp](#)

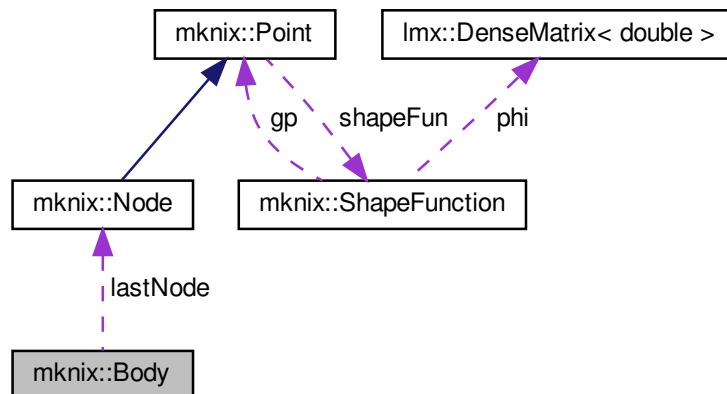
8.7 mknix::Body Class Reference

```
#include <body.h>
```

Inheritance diagram for mknix::Body:



Collaboration diagram for mknix::Body:



Public Member Functions

- [Body](#) ()
Default constructor. Initializes the body with thermal simulation enabled and energy computation disabled.
- [Body](#) (std::string)
Constructor with 1 parameter.
- virtual [~Body](#) ()
Destructor. Releases memory for temperature vectors, cells, boundary groups and volumetric heat sources.
- virtual std::string [getType](#) ()=0
- virtual void [initialize](#) ()
Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.

- virtual void `calcCapacityMatrix` ()
Computes the local Capacity of the material body by calling each cell's cascade function.
- virtual void `calcConductivityMatrix` ()
Computes the local Conductivity of the material body by calling each cell's cascade function.
- virtual void `calcExternalHeat` ()
Computes the local volumetric heat vector of the material body by calling each cell's cascade function.
- virtual void `assembleCapacityMatrix` (Imx::Matrix< data_type > &)
Assembles the local conductivity into the global matrix by calling each cell's cascade function.
- virtual void `assembleConductivityMatrix` (Imx::Matrix< data_type > &)
Assembles the local conductivity into the global matrix by calling each cell's cascade function.
- virtual void `assembleExternalHeat` (Imx::Vector< data_type > &)
Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.
- virtual void `calcMassMatrix` ()=0
- virtual void `calcExternalForces` ()=0
- virtual void `assembleMassMatrix` (Imx::Matrix< data_type > &)=0
- virtual void `assembleExternalForces` (Imx::Vector< data_type > &)=0
- void `setTemperature` (double)
Sets the temperature of every node in the body.
- virtual void `setMechanical` ()
- virtual void `setOutput` (std::string)=0
- virtual void `outputStep` (const Imx::Vector< data_type > &, const Imx::Vector< data_type > &)=0
- virtual void `outputStep` (const Imx::Vector< data_type > &)=0
- virtual void `outputStep` ()
Postprocess and store thermal step results for any analysis.
- virtual void `outputToFile` (std::ofstream *)
Streams the data stored during the analysis to a file.
- void `outputVTK` ()
Prepare for saving VTK files.
- virtual void `addNode` (Node *node_in)
- void `addNodes` (std::vector< Node * > &nodes_in)
- virtual int `getNodesSize` ()
- std::vector< Node * > & `getNodes` ()
- virtual Node * `getNode` (int node_number)
- virtual Node * `getLastNode` ()
- virtual void `addBoundaryLine` (Point *node1, Point *node2)
- virtual void `addBoundaryConnectivity` (std::vector< int > connectivity_in)
Appends a connectivity record (list of node indices) to the boundary connectivity table.
- virtual int `getBoundarySize` ()
- virtual Point * `getBoundaryFirstNode` ()
- virtual Point * `getBoundaryNextNode` (Point *node_in)
- void `addBoundaryGroup` (std::string boundaryName_in)
- void `addNodeToBoundaryGroup` (int nodeNumber_in, std::string boundaryName_in)
- void `addCellToBoundaryGroup` (CellBoundary *cell_in, std::string boundaryName_in)
- void `setLoadThermalInBoundaryGroup` (LoadThermalBoundary1D *load_in, std::string boundaryName_in)
- int `getCellLastNumber` ()
- virtual void `addCell` (int num, Cell *cell_in)
- virtual void `writeBodyInfo` (std::ofstream *)=0
- virtual void `writeBoundaryNodes` (std::vector< Point * > &)=0
- virtual void `writeBoundaryConnectivity` (std::vector< std::vector< Point * > > &)=0
- virtual void `translate` (double x, double y, double z)
Translates all nodes of the body by the given displacement vector.
- virtual void `rotate` (double phi, double theta, double psi)
Rotates all nodes of the body using Euler angles (ZYX convention).
- virtual void `addLoadThermal` (LoadThermalBody *theLoad)

Protected Attributes

- `std::string title`
- `Node * lastNode`
- `std::vector< Node * > nodes`
- `std::vector< Node * > bondedBodyNodes`
- `std::map< std::string, BoundaryGroup * > boundaryGroups`
- `std::vector< std::vector< int > > boundaryConnectivity`
- `std::map< Point *, Point * > linearBoundary`
- `std::map< int, Cell * > cells`
- `bool computeEnergy`
- `bool isThermal`
- `std::vector< Imx::Vector< data_type > * > temperature`
- `std::vector< LoadThermalBody * > volumetricHeatSources`

8.7.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 47 of file body.h.

8.7.2 Constructor & Destructor Documentation

8.7.2.1 Body() [1/2] `mknix::Body::Body ( )`

Default constructor. Initializes the body with thermal simulation enabled and energy computation disabled.
Definition at line 51 of file body.cpp.

8.7.2.2 Body() [2/2] `mknix::Body::Body ( std::string title_in )`

Constructor with 1 parameter.

Parameters

<i>title_in</i>	Name of body in the system. Will be the same as the associated material body
-----------------	--

Definition at line 62 of file body.cpp.

8.7.2.3 ~Body() `mknix::Body::~~Body ( ) [virtual]`

Destructor. Releases memory for temperature vectors, cells, boundary groups and volumetric heat sources.
Definition at line 73 of file body.cpp.

8.7.3 Member Function Documentation

8.7.3.1 addBoundaryConnectivity() `void mknix::Body::addBoundaryConnectivity ( std::vector< int > connectivity_in ) [virtual]`

Appends a connectivity record (list of node indices) to the boundary connectivity table.

Parameters

<i>connectivity</i> ↔ <i>_in</i>	Vector of node indices defining one boundary segment.
-------------------------------------	---

Definition at line 497 of file body.cpp.

8.7.3.2 addBoundaryGroup() void mknix::Body::addBoundaryGroup (
 std::string *boundaryName_in*) [inline]

Definition at line 165 of file body.h.

8.7.3.3 addBoundaryLine() virtual void mknix::Body::addBoundaryLine (
 Point * *node1*,
 Point * *node2*) [inline], [virtual]

Definition at line 131 of file body.h.

8.7.3.4 addCell() virtual void mknix::Body::addCell (
 int *num*,
 Cell * *cell_in*) [inline], [virtual]

Definition at line 191 of file body.h.

8.7.3.5 addCellToBoundaryGroup() void mknix::Body::addCellToBoundaryGroup (
 CellBoundary * *cell_in*,
 std::string *boundaryName_in*) [inline]

Definition at line 176 of file body.h.

8.7.3.6 addLoadThermal() virtual void mknix::Body::addLoadThermal (
 LoadThermalBody * *theLoad*) [inline], [virtual]

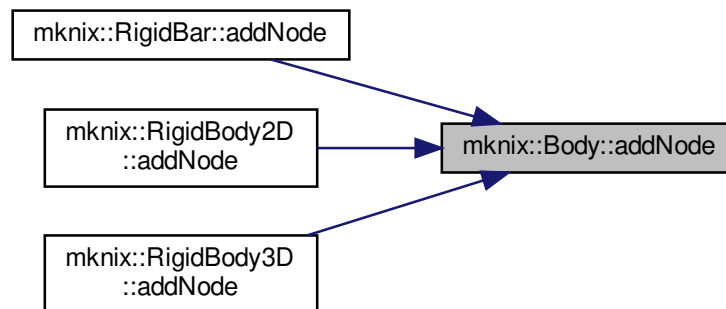
Definition at line 207 of file body.h.

8.7.3.7 addNode() virtual void mknix::Body::addNode (
 Node * *node_in*) [inline], [virtual]

Reimplemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), and [mknix::RigidBar](#).

Definition at line 101 of file body.h.

Here is the caller graph for this function:



8.7.3.8 addNodes() `void mknix::Body::addNodes (`
`std::vector< Node * > & nodes_in ) [inline]`

Definition at line 106 of file `body.h`.

8.7.3.9 addNodeToBoundaryGroup() `void mknix::Body::addNodeToBoundaryGroup (`
`int nodeNumber_in,`
`std::string boundaryName_in ) [inline]`

Definition at line 170 of file `body.h`.

8.7.3.10 assembleCapacityMatrix() `void mknix::Body::assembleCapacityMatrix (`
`lmx::Matrix< data_type > & globalCapacity ) [virtual]`

Assembles the local conductivity into the global matrix by calling each cell's cascade function.

Parameters

<i>globalCapacity</i>	Reference to the global matrix of the thermal simulation.
-----------------------	---

Returns

`void`

Definition at line 219 of file `body.cpp`.

8.7.3.11 assembleConductivityMatrix() `void mknix::Body::assembleConductivityMatrix (`
`lmx::Matrix< data_type > & globalConductivity ) [virtual]`

Assembles the local conductivity into the global matrix by calling each cell's cascade function.

Parameters

<i>globalConductivity</i>	Reference to the global matrix of the thermal simulation.
---------------------------	---

Returns

void

Definition at line 235 of file body.cpp.

8.7.3.12 assembleExternalForces() `virtual void mknix::Body::assembleExternalForces (
 lmx::Vector< data_type > &) [pure virtual]`

Implemented in [mknix::RigidBody](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.13 assembleExternalHeat() `void mknix::Body::assembleExternalHeat (
 lmx::Vector< data_type > & globalExternalHeat) [virtual]`

Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.

Returns

void

Definition at line 250 of file body.cpp.

8.7.3.14 assembleMassMatrix() `virtual void mknix::Body::assembleMassMatrix (
 lmx::Matrix< data_type > &) [pure virtual]`

Implemented in [mknix::RigidBody](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.15 calcCapacityMatrix() `void mknix::Body::calcCapacityMatrix ( ) [virtual]`

Computes the local Capacity of the material body by calling each cell's cascade function.

Returns

void

Definition at line 171 of file body.cpp.

8.7.3.16 calcConductivityMatrix() `void mknix::Body::calcConductivityMatrix ( ) [virtual]`

Computes the local Conductivity of the material body by calling each cell's cascade function.

Returns

void

Definition at line 186 of file body.cpp.

8.7.3.17 calcExternalForces() `virtual void mknix::Body::calcExternalForces ( ) [pure virtual]`

Implemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), [mknix::RigidBar](#), [mknix::RigidBodyMassPoint](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.18 calcExternalHeat() `void mknix::Body::calcExternalHeat ( ) [virtual]`

Computes the local volumetric heat vector of the material body by calling each cell's cascade function.

Returns

void

Definition at line 201 of file body.cpp.

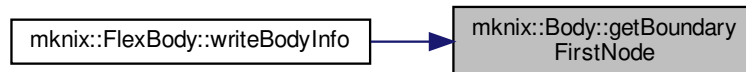
8.7.3.19 calcMassMatrix() `virtual void mknix::Body::calcMassMatrix ( ) [pure virtual]`

Implemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), [mknix::RigidBar](#), [mknix::RigidBodyMassPoint](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.20 getBoundaryFirstNode() `virtual Point* mknix::Body::getBoundaryFirstNode ( ) [inline], [virtual]`

Definition at line 155 of file body.h.

Here is the caller graph for this function:



8.7.3.21 getBoundaryNextNode() `virtual Point* mknix::Body::getBoundaryNextNode ( Point * node_in ) [inline], [virtual]`

Definition at line 160 of file body.h.

8.7.3.22 getBoundarySize() `virtual int mknix::Body::getBoundarySize ( ) [inline], [virtual]`

Definition at line 150 of file body.h.

Here is the caller graph for this function:



8.7.3.23 getCellLastNumber() `int mknix::Body::getCellLastNumber ( ) [inline]`

Definition at line 186 of file body.h.

8.7.3.24 getLastNode() `virtual Node* mknix::Body::getLastNode ( ) [inline], [virtual]`

Definition at line 126 of file body.h.

Here is the caller graph for this function:



8.7.3.25 getNode() `virtual Node* mknix::Body::getNode (`
`int node_number ) [inline], [virtual]`

Reimplemented in [mknix::RigidBody](#), and [mknix::FlexBody](#).

Definition at line 121 of file body.h.

8.7.3.26 getNodes() `std::vector<Node*> mknix::Body::getNodes ( ) [inline]`

Definition at line 116 of file body.h.

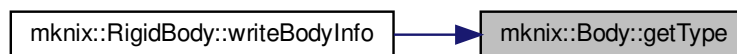
8.7.3.27 getNodesSize() `virtual int mknix::Body::getNodesSize ( ) [inline], [virtual]`

Definition at line 111 of file body.h.

8.7.3.28 getType() `virtual std::string mknix::Body::getType ( ) [pure virtual]`

Implemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), [mknix::RigidBodyBar](#), [mknix::RigidBodyMassPoint](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

Here is the caller graph for this function:



8.7.3.29 initialize() `void mknix::Body::initialize ( ) [virtual]`

Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.

Returns

`void`

Reimplemented in [mknix::FlexBody](#).

Definition at line 108 of file body.cpp.

8.7.3.30 outputStep() [1/3] `void mknix::Body::outputStep ( ) [virtual]`

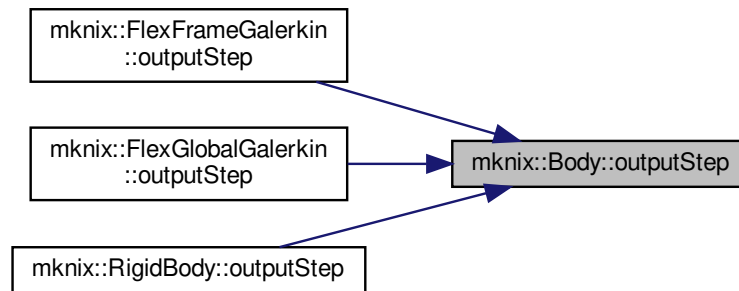
Postprocess and store thermal step results for any analysis.

Returns

void

Definition at line 283 of file body.cpp.

Here is the caller graph for this function:



8.7.3.31 outputStep() [2/3] virtual void mknix::Body::outputStep (
 const [lmx::Vector](#)< [data_type](#) > &) [pure virtual]

Implemented in [mknix::RigidBody](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.32 outputStep() [3/3] virtual void mknix::Body::outputStep (
 const [lmx::Vector](#)< [data_type](#) > & ,
 const [lmx::Vector](#)< [data_type](#) > &) [pure virtual]

Implemented in [mknix::RigidBody](#), [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.7.3.33 outputToFile() void mknix::Body::outputToFile (
 std::ofstream * *outFile*) [virtual]

Streams the data stored during the analysis to a file.

Parameters

<i>outFile</i>	Output files
----------------	--------------

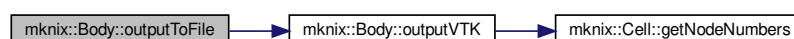
Returns

void

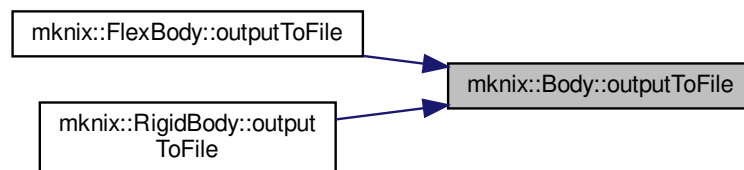
Reimplemented in [mknix::RigidBody](#), and [mknix::FlexBody](#).

Definition at line 321 of file body.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.34 outputVTK() `void mknix::Body::outputVTK ( )`

Prepare for saving VTK files.

Returns

void

Definition at line 375 of file body.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.7.3.35 rotate() `void mknix::Body::rotate (` `double phi,` `double theta,` `double psi ) [virtual]`

Rotates all nodes of the body using Euler angles (ZYX convention).

Parameters

<i>phi</i>	Rotation angle about the X axis (degrees).
------------	--

Parameters

<i>theta</i>	Rotation angle about the Y axis (degrees).
<i>psi</i>	Rotation angle about the Z axis (degrees).

Definition at line 529 of file body.cpp.

Here is the call graph for this function:



8.7.3.36 setLoadThermalInBoundaryGroup() `void mknix::Body::setLoadThermalInBoundaryGroup ( LoadThermalBoundary1D * load_in, std::string boundaryName_in ) [inline]`

Definition at line 181 of file body.h.

8.7.3.37 setMechanical() `virtual void mknix::Body::setMechanical ( ) [inline], [virtual]`

Definition at line 83 of file body.h.

8.7.3.38 setOutput() `virtual void mknix::Body::setOutput ( std::string ) [pure virtual]`

Implemented in [mknix::RigidBody](#), and [mknix::FlexBody](#).

8.7.3.39 setTemperature() `void mknix::Body::setTemperature ( double temp_in )`

Sets the temperature of every node in the body.

Parameters

<i>temp_in</i>	Temperature value to assign to all nodes.
----------------	---

Definition at line 269 of file body.cpp.

8.7.3.40 translate() `void mknix::Body::translate ( double x_in, double y_in, double z_in ) [virtual]`

Translates all nodes of the body by the given displacement vector.

Parameters

Parameters

$x \leftrightarrow$ _in	Displacement along the X axis.
$y \leftrightarrow$ _in	Displacement along the Y axis.
$z \leftrightarrow$ _in	Displacement along the Z axis.

Definition at line 508 of file body.cpp.

8.7.3.41 writeBodyInfo() virtual void mknix::Body::writeBodyInfo (
std::ofstream *) [pure virtual]

Implemented in [mknix::RigidBody](#), and [mknix::FlexBody](#).

8.7.3.42 writeBoundaryConnectivity() virtual void mknix::Body::writeBoundaryConnectivity (
std::vector< std::vector< [Point](#) * >> &) [pure virtual]

Implemented in [mknix::FlexBody](#).

8.7.3.43 writeBoundaryNodes() virtual void mknix::Body::writeBoundaryNodes (
std::vector< [Point](#) * > &) [pure virtual]

Implemented in [mknix::RigidBody](#), and [mknix::FlexBody](#).

8.7.4 Member Data Documentation

8.7.4.1 bondedBodyNodes std::vector<[Node](#)*> mknix::Body::bondedBodyNodes [protected]

Definition at line 217 of file body.h.

8.7.4.2 boundaryConnectivity std::vector<std::vector<int> > mknix::Body::boundaryConnectivity [protected]

Definition at line 219 of file body.h.

8.7.4.3 boundaryGroups std::map<std::string, [BoundaryGroup](#)*> mknix::Body::boundaryGroups [protected]

Definition at line 218 of file body.h.

8.7.4.4 cells std::map<int, [Cell](#)*> mknix::Body::cells [protected]

Map of integration cells.

Definition at line 222 of file body.h.

8.7.4.5 computeEnergy bool mknix::Body::computeEnergy [protected]

Definition at line 224 of file body.h.

8.7.4.6 isThermal `bool mknix::Body::isThermal [protected]`
 Definition at line 225 of file body.h.

8.7.4.7 lastNode `Node* mknix::Body::lastNode [protected]`
 Definition at line 215 of file body.h.

8.7.4.8 linearBoundary `std::map<Point*, Point*> mknix::Body::linearBoundary [protected]`
 Map of linear boundary.
 Definition at line 220 of file body.h.

8.7.4.9 nodes `std::vector<Node*> mknix::Body::nodes [protected]`
 Definition at line 216 of file body.h.

8.7.4.10 temperature `std::vector<lmx::Vector<data_type>*> mknix::Body::temperature [protected]`
 Definition at line 226 of file body.h.

8.7.4.11 title `std::string mknix::Body::title [protected]`
 Definition at line 214 of file body.h.

8.7.4.12 volumetricHeatSources `std::vector<LoadThermalBody*> mknix::Body::volumetricHeatSources [protected]`
 Definition at line 227 of file body.h.

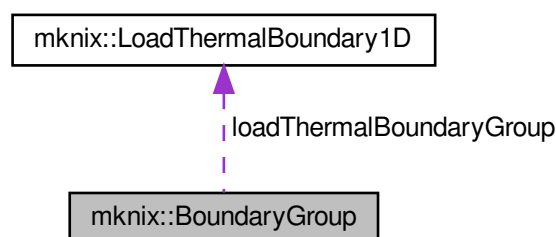
The documentation for this class was generated from the following files:

- [body.h](#)
- [body.cpp](#)

8.8 mknix::BoundaryGroup Class Reference

```
#include <boundarygroup.h>
```

Collaboration diagram for mknix::BoundaryGroup:



Public Member Functions

- [BoundaryGroup](#) ()
Default constructor.
- virtual [~BoundaryGroup](#) ()
Destructor. Releases memory for all boundary cell objects.
- virtual void [initialize](#) ()
Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.
- virtual void [calcExternalHeat](#) ()
Computes the local volumetric heat vector of the material body by calling each cell's cascade function.
- virtual void [assembleExternalHeat](#) ([lmx::Vector](#)< [data_type](#) > &)
Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.
- virtual void [addNode](#) ([Node](#) *node_in)
- virtual void [addCell](#) ([CellBoundary](#) *cell_in)
- virtual void [setLoadThermal](#) ([LoadThermalBoundary1D](#) *theLoad)

Protected Attributes

- [std::vector](#)< [Node](#) * > [nodes](#)
- [std::map](#)< int, [CellBoundary](#) * > [cells](#)
- [LoadThermalBoundary1D](#) * [loadThermalBoundaryGroup](#)

8.8.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 41 of file [boundarygroup.h](#).

8.8.2 Constructor & Destructor Documentation

8.8.2.1 [BoundaryGroup](#)() `mknix::BoundaryGroup::BoundaryGroup ( )`

Default constructor.

Definition at line 29 of file [boundarygroup.cpp](#).

8.8.2.2 [~BoundaryGroup](#)() `mknix::BoundaryGroup::~~BoundaryGroup ( )` [virtual]

Destructor. Releases memory for all boundary cell objects.

Definition at line 46 of file [boundarygroup.cpp](#).

8.8.3 Member Function Documentation

8.8.3.1 [addCell](#)() `virtual void mknix::BoundaryGroup::addCell ( CellBoundary * cell_in )` [inline], [virtual]

Definition at line 64 of file [boundarygroup.h](#).

8.8.3.2 [addNode](#)() `virtual void mknix::BoundaryGroup::addNode ( Node * node_in )` [inline], [virtual]

Definition at line 59 of file [boundarygroup.h](#).

8.8.3.3 assembleExternalHeat() `void mknix::BoundaryGroup::assembleExternalHeat (
lmx::Vector< data_type > & globalExternalHeat) [virtual]`

Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.

Returns

`void`

Definition at line 107 of file `boundarygroup.cpp`.

8.8.3.4 calcExternalHeat() `void mknix::BoundaryGroup::calcExternalHeat ( ) [virtual]`

Computes the local volumetric heat vector of the material body by calling each cell's cascade function.

Returns

`void`

Definition at line 89 of file `boundarygroup.cpp`.

8.8.3.5 initialize() `void mknix::BoundaryGroup::initialize ( ) [virtual]`

Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.

Returns

`void`

Definition at line 60 of file `boundarygroup.cpp`.

8.8.3.6 setLoadThermal() `virtual void mknix::BoundaryGroup::setLoadThermal (
LoadThermalBoundary1D * theLoad) [inline], [virtual]`

Definition at line 71 of file `boundarygroup.h`.

8.8.4 Member Data Documentation

8.8.4.1 cells `std::map<int, CellBoundary *> mknix::BoundaryGroup::cells [protected]`

Map of integration cells.

Definition at line 79 of file `boundarygroup.h`.

8.8.4.2 loadThermalBoundaryGroup `LoadThermalBoundary1D* mknix::BoundaryGroup::loadThermal↔
 BoundaryGroup [protected]`

Definition at line 81 of file `boundarygroup.h`.

8.8.4.3 nodes `std::vector<Node *> mknix::BoundaryGroup::nodes [protected]`

Definition at line 78 of file `boundarygroup.h`.

The documentation for this class was generated from the following files:

- [boundarygroup.h](#)
- [boundarygroup.cpp](#)

8.9 mknix::boxFIR Class Reference

```
#include <common.h>
```


Public Member Functions

- [boxFIR](#) (int)
Constructs a box (moving-average) FIR filter with the given number of coefficients.
- void [filter](#) (vector< double > &)
Applies the box FIR filter in-place to a signal vector.

8.9.1 Detailed Description

Definition at line 66 of file common.h.

8.9.2 Constructor & Destructor Documentation

8.9.2.1 boxFIR() `mknix::boxFIR::boxFIR (int _numCoeffs )`

Constructs a box (moving-average) FIR filter with the given number of coefficients.

The effective filter length is doubled internally. If the resulting length is less than 1 it is clamped to 2. All coefficients are set to 1/numCoeffs so that the filter computes a uniform moving average.

Parameters

<code>_numCoeffs</code>	Desired half-length of the filter; the internal length will be 2 * <code>_numCoeffs</code> .
-------------------------	--

Definition at line 197 of file common.cpp.

8.9.3 Member Function Documentation

8.9.3.1 filter() `void mknix::boxFIR::filter (vector< double > & a )`

Applies the box FIR filter in-place to a signal vector.

Smooths the input signal using a centred moving-average strategy: the beginning and end regions of the signal are handled by holding the boundary values, while the interior is filtered using a look-ahead window of numCoeffs/2 samples. The result overwrites the input vector.

Parameters

<code>a</code>	Signal vector to be filtered in-place.
----------------	--

Definition at line 223 of file common.cpp.

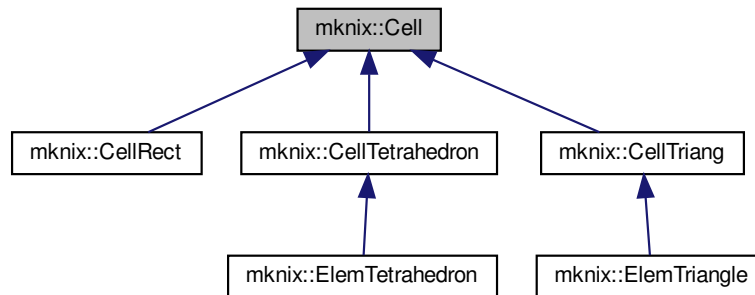
The documentation for this class was generated from the following files:

- [common.h](#)
- [common.cpp](#)

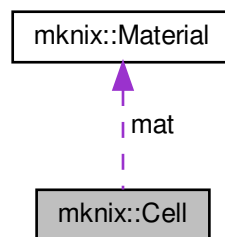
8.10 mknix::Cell Class Reference

```
#include <cell.h>
```

Inheritance diagram for mknix::Cell:



Collaboration diagram for mknix::Cell:



Public Member Functions

- [Cell](#) ()
Default constructor for [Cell](#).
- [Cell](#) ([Material](#) &, std::string, double, int)
Constructs a [Cell](#) with a material, formulation type, influence radius factor, and number of Gauss points.
- virtual [~Cell](#) ()
Destructor. Releases all dynamically allocated Gauss points.
- bool [setMaterialIfLayer](#) ([Material](#) &, double)
Reassigns the material for all Gauss points if the cell belongs to a thin layer.
- virtual void [initialize](#) (std::vector< [Node](#) * > &)
Initializes the cell by finding support nodes for each Gauss point (meshfree formulations).
- virtual void [computeShapeFunctions](#) ()
Computes and stores shape function values at each Gauss point using the selected formulation.
- void [computeCapacityGaussPoints](#) ()
Computes the local thermal capacity (heat capacity) contribution at each Gauss point.
- void [assembleCapacityGaussPoints](#) (lmx::Matrix< [data_type](#) > &)
Assembles Gauss-point capacity contributions into the global thermal capacity matrix.
- void [computeConductivityGaussPoints](#) ()

- Computes the local thermal conductivity contribution at each Gauss point.*

 - void `assembleConductivityGaussPoints` (`Imx::Matrix< data_type > &`)

Assembles Gauss-point conductivity contributions into the global conductivity matrix.
- void `computeQextGaussPoints` (`std::vector< LoadThermalBody * >`)

Computes the external thermal load vector contribution at each Gauss point.
- void `assembleQextGaussPoints` (`Imx::Vector< data_type > &`)

Assembles external thermal load contributions from Gauss points into the global heat vector.
- void `computeMGaussPoints` ()

Computes the local mass matrix contribution at each Gauss point.
- void `assembleMGaussPoints` (`Imx::Matrix< data_type > &`)

Assembles Gauss-point mass contributions into the global mass matrix.
- void `computeFintGaussPoints` ()

Computes the linear internal force vector contribution at each Gauss point.
- void `computeNLFintGaussPoints` ()

Computes the nonlinear internal force vector contribution at each Gauss point.
- void `assembleFintGaussPoints` (`Imx::Vector< data_type > &`)

Assembles internal force contributions from Gauss points into the global internal force vector.
- void `computeFextGaussPoints` ()

Computes the external force vector contribution at each Gauss point.
- void `assembleFextGaussPoints` (`Imx::Vector< data_type > &`)

Assembles external force contributions from Gauss points into the global external force vector.
- void `computeKGaussPoints` ()

Computes the linear tangent (stiffness) matrix contribution at each Gauss point.
- void `computeNLKGaussPoints` ()

Computes the nonlinear tangent matrix contribution at each Gauss point.
- void `assembleKGaussPoints` (`Imx::Matrix< data_type > &`)

Assembles tangent matrix contributions from Gauss points into the global tangent matrix.
- void `assembleRGaussPoints` (`Imx::Vector< data_type > &`, `int`)

Computes linear stress at each Gauss point and assembles the result into a body stress vector.
- void `assembleNLRGaussPoints` (`Imx::Vector< data_type > &`, `int`)

Computes nonlinear (large-deformation) stress at each Gauss point and assembles the result.
- double `calcPotentialEGaussPoints` (`const Imx::Vector< data_type > &`)

Accumulates the potential (gravitational/body-force) energy across all Gauss points.
- double `calcKineticEGaussPoints` (`const Imx::Vector< data_type > &`)

Accumulates the kinetic energy across all Gauss points.
- double `calcElasticEGaussPoints` ()

Accumulates the elastic strain energy across all Gauss points.
- void `outputConnectivityToFile` (`std::ofstream *`)

Writes the node numbers of this cell's body points to an output file.
- virtual void `gnuplotOut` (`std::ofstream &`, `std::ofstream &`)=0
- void `gnuplotOutStress` (`std::ofstream &`)

Outputs Gauss-point stress values to a file suitable for gnuplot visualization.
- `std::vector< int >` `getNodeNumbers` ()

Returns the global node numbers of all body points belonging to this cell.
- int `getNumberOfNodes` ()

Protected Attributes

- [Material](#) * [mat](#)
- std::string [formulation](#)
- double [alpha](#)
- int [nGPoints](#)
- std::vector< [GaussPoint](#) * > [gPoints](#)
- std::vector< [GaussPoint](#) * > [gPoints_MC](#)
- double [jacobian](#)
- std::vector< [Point](#) * > [bodyPoints](#)
- double [dc](#)

8.10.1 Detailed Description

Author

Daniel Iglesias

Definition at line 56 of file cell.h.

8.10.2 Constructor & Destructor Documentation

8.10.2.1 Cell() [1/2] `mnix::Cell::Cell ( )`

Default constructor for [Cell](#).

Definition at line 16 of file cell.cpp.

8.10.2.2 Cell() [2/2] `mnix::Cell::Cell (`

```
    Material & material\_in,
    std::string formulation\_in,
    double alpha\_in,
    int nGPoints\_in )
```

Constructs a [Cell](#) with a material, formulation type, influence radius factor, and number of Gauss points.

Parameters

<i>material_in</i>	Reference to the material assigned to this cell.
<i>formulation_in</i>	String identifying the meshfree formulation (e.g. "EFG", "RPIM").
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of integration Gauss points per direction.

Definition at line 28 of file cell.cpp.

8.10.2.3 ~Cell() `mnix::Cell::~~Cell ( ) [virtual]`

Destructor. Releases all dynamically allocated Gauss points.

Definition at line 43 of file cell.cpp.

8.10.3 Member Function Documentation

8.10.3.1 assembleCapacityGaussPoints() `void mnix::Cell::assembleCapacityGaussPoints (`

```
    lmx::Matrix< data\_type > & globalCapacity )
```

Assembles Gauss-point capacity contributions into the global thermal capacity matrix.

Parameters

<i>globalCapacity</i>	Global thermal capacity matrix to be updated.
-----------------------	---

Definition at line 165 of file cell.cpp.

8.10.3.2 assembleConductivityGaussPoints() `void mknix::Cell::assembleConductivityGaussPoints (
 lmx::Matrix< data_type > & globalConductivity)`

Assembles Gauss-point conductivity contributions into the global conductivity matrix.

Parameters

<i>globalConductivity</i>	Global thermal conductivity matrix to be updated.
---------------------------	---

Definition at line 189 of file cell.cpp.

8.10.3.3 assembleFextGaussPoints() `void mknix::Cell::assembleFextGaussPoints (
 lmx::Vector< data_type > & globalFext)`

Assembles external force contributions from Gauss points into the global external force vector.

Parameters

<i>globalFext</i>	Global external force vector to be updated.
-------------------	---

Definition at line 301 of file cell.cpp.

8.10.3.4 assembleFintGaussPoints() `void mknix::Cell::assembleFintGaussPoints (
 lmx::Vector< data_type > & globalFint)`

Assembles internal force contributions from Gauss points into the global internal force vector.

Parameters

<i>globalFint</i>	Global internal force vector to be updated.
-------------------	---

Definition at line 276 of file cell.cpp.

8.10.3.5 assembleKGaussPoints() `void mknix::Cell::assembleKGaussPoints (
 lmx::Matrix< data_type > & globalTangent)`

Assembles tangent matrix contributions from Gauss points into the global tangent matrix.

Parameters

<i>globalTangent</i>	Global tangent (stiffness) matrix to be updated.
----------------------	--

Definition at line 338 of file cell.cpp.

8.10.3.6 assembleMGaussPoints() `void mknix::Cell::assembleMGaussPoints (
 lmx::Matrix< data_type > & globalMass)`

Assembles Gauss-point mass contributions into the global mass matrix.

Parameters

<i>globalMass</i>	Global mass matrix to be updated.
-------------------	-----------------------------------

Definition at line 239 of file cell.cpp.

8.10.3.7 assembleNLRGaussPoints() `void mknix::Cell::assembleNLRGaussPoints (
 lmx::Vector< data_type > & globalStress,
 int firstNode)`

Computes nonlinear (large-deformation) stress at each Gauss point and assembles the result.

Parameters

<i>globalStress</i>	Body-level stress resultant vector to be updated.
<i>firstNode</i>	Global index of the first node in this body.

Definition at line 369 of file cell.cpp.

8.10.3.8 assembleQextGaussPoints() `void mknix::Cell::assembleQextGaussPoints (
 lmx::Vector< data_type > & globalQext)`

Assembles external thermal load contributions from Gauss points into the global heat vector.

Parameters

<i>globalQext</i>	Global external heat load vector to be updated.
-------------------	---

Definition at line 214 of file cell.cpp.

8.10.3.9 assembleRGaussPoints() `void mknix::Cell::assembleRGaussPoints (
 lmx::Vector< data_type > & globalStress,
 int firstNode)`

Computes linear stress at each Gauss point and assembles the result into a body stress vector.

Parameters

<i>globalStress</i>	Body-level stress resultant vector to be updated.
<i>firstNode</i>	Global index of the first node in this body.

Definition at line 352 of file cell.cpp.

8.10.3.10 calcElasticEGaussPoints() `double mknix::Cell::calcElasticEGaussPoints ( )`
 Accumulates the elastic strain energy across all Gauss points.

Returns

Total elastic energy contribution from this cell.

Definition at line 419 of file cell.cpp.

8.10.3.11 calcKineticEGaussPoints() `double mknix::Cell::calcKineticEGaussPoints (
 const lmx::Vector< data_type > & qdot)`

Accumulates the kinetic energy across all Gauss points.

Parameters

<i>qdot</i>	Current global velocity state vector.
-------------	---------------------------------------

Returns

Total kinetic energy contribution from this cell.

Definition at line 403 of file cell.cpp.

8.10.3.12 calcPotentialEGaussPoints() `double mknix::Cell::calcPotentialEGaussPoints ( const lmx::Vector< data_type > & q )`

Accumulates the potential (gravitational/body-force) energy across all Gauss points.

Parameters

<i>q</i>	Current global displacement state vector.
----------	---

Returns

Total potential energy contribution from this cell.

Definition at line 386 of file cell.cpp.

8.10.3.13 computeCapacityGaussPoints() `void mknix::Cell::computeCapacityGaussPoints ( )`

Computes the local thermal capacity (heat capacity) contribution at each Gauss point.

Definition at line 153 of file cell.cpp.

8.10.3.14 computeConductivityGaussPoints() `void mknix::Cell::computeConductivityGaussPoints ( )`

Computes the local thermal conductivity contribution at each Gauss point.

Definition at line 177 of file cell.cpp.

8.10.3.15 computeFextGaussPoints() `void mknix::Cell::computeFextGaussPoints ( )`

Computes the external force vector contribution at each Gauss point.

Definition at line 288 of file cell.cpp.

8.10.3.16 computeFintGaussPoints() `void mknix::Cell::computeFintGaussPoints ( )`

Computes the linear internal force vector contribution at each Gauss point.

Definition at line 251 of file cell.cpp.

8.10.3.17 computeKGaussPoints() `void mknix::Cell::computeKGaussPoints ( )`

Computes the linear tangent (stiffness) matrix contribution at each Gauss point.

Definition at line 313 of file cell.cpp.

8.10.3.18 computeMGaussPoints() `void mknix::Cell::computeMGaussPoints ( )`

Computes the local mass matrix contribution at each Gauss point.

Definition at line 226 of file cell.cpp.

8.10.3.19 computeNLIntGaussPoints() `void mknix::Cell::computeNLIntGaussPoints ( )`

Computes the nonlinear internal force vector contribution at each Gauss point.

Definition at line 263 of file cell.cpp.

8.10.3.20 computeNLKGaussPoints() `void mknix::Cell::computeNLKGaussPoints ( )`

Computes the nonlinear tangent matrix contribution at each Gauss point.

Definition at line 325 of file cell.cpp.

8.10.3.21 computeQextGaussPoints() `void mknix::Cell::computeQextGaussPoints (`

`std::vector< LoadThermalBody * > loadThermalBody_in )`

Computes the external thermal load vector contribution at each Gauss point.

Parameters

<code>loadThermalBody_in</code>	Vector of body thermal loads applied to this cell.
---------------------------------	--

Definition at line 202 of file cell.cpp.

8.10.3.22 computeShapeFunctions() `void mknix::Cell::computeShapeFunctions ( ) [virtual]`

Computes and stores shape function values at each Gauss point using the selected formulation.

Reimplemented in [mknix::ElemTriangle](#), and [mknix::ElemTetrahedron](#).

Definition at line 134 of file cell.cpp.

8.10.3.23 getNodeNumbers() `std::vector< int > mknix::Cell::getNodeNumbers ( )`

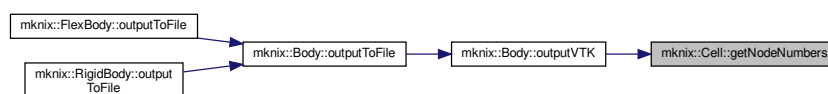
Returns the global node numbers of all body points belonging to this cell.

Returns

Vector of node numbers.

Definition at line 465 of file cell.cpp.

Here is the caller graph for this function:

**8.10.3.24 getNumberOfNodes()** `int mknix::Cell::getNumberOfNodes ( ) [inline]`

Definition at line 122 of file cell.h.

8.10.3.25 gnuplotOut() `virtual void mknix::Cell::gnuplotOut (`

`std::ofstream & ,
std::ofstream &) [pure virtual]`

Implemented in [mknix::CellTriang](#), [mknix::CellTetrahedron](#), and [mknix::CellRect](#).

8.10.3.26 gnuplotOutStress() `void mknix::Cell::gnuplotOutStress (
 std::ofstream & gptension)`

Outputs Gauss-point stress values to a file suitable for gnuplot visualization.

Parameters

<i>gptension</i>	Output file stream for stress/tension data.
------------------	---

Definition at line 450 of file cell.cpp.

8.10.3.27 initialize() `void mknix::Cell::initialize (
 std::vector< Node * > & nodes_in) [virtual]`

Initializes the cell by finding support nodes for each Gauss point (meshfree formulations).

Parameters

<i>nodes_in</i>	Vector of domain nodes used to build the support neighbourhood.
-----------------	---

Reimplemented in [mknix::ElemTriangle](#), [mknix::ElemTetrahedron](#), and [mknix::CellRect](#).

Definition at line 104 of file cell.cpp.

8.10.3.28 outputConnectivityToFile() `void mknix::Cell::outputConnectivityToFile (
 std::ofstream * outfile)`

Writes the node numbers of this cell's body points to an output file.

Parameters

<i>outfile</i>	Pointer to the open output file stream.
----------------	---

Definition at line 435 of file cell.cpp.

8.10.3.29 setMaterialIfLayer() `bool mknix::Cell::setMaterialIfLayer (
 Material & newMat,
 double thickness)`

Reassigns the material for all Gauss points if the cell belongs to a thin layer.

Detects whether the cell is within a layer of the given thickness by checking if any pair of body-points is closer than the thickness. If so, the material is replaced.

Parameters

<i>newMat</i>	The replacement material.
<i>thickness</i>	Maximum inter-node distance that qualifies the cell as part of the layer.

Returns

True if the material was changed, false otherwise.

Definition at line 66 of file cell.cpp.

8.10.4 Member Data Documentation

8.10.4.1 alpha double mknix::Cell::alpha [protected]
Definition at line 130 of file cell.h.

8.10.4.2 bodyPoints std::vector< [Point*](#) > mknix::Cell::bodyPoints [protected]
Definition at line 135 of file cell.h.

8.10.4.3 dc double mknix::Cell::dc [protected]
Definition at line 136 of file cell.h.

8.10.4.4 formulation std::string mknix::Cell::formulation [protected]
Definition at line 129 of file cell.h.

8.10.4.5 gPoints std::vector<[GaussPoint*](#)> mknix::Cell::gPoints [protected]
Definition at line 132 of file cell.h.

8.10.4.6 gPoints_MC std::vector<[GaussPoint*](#)> mknix::Cell::gPoints_MC [protected]
Definition at line 133 of file cell.h.

8.10.4.7 jacobian double mknix::Cell::jacobian [protected]
Definition at line 134 of file cell.h.

8.10.4.8 mat [Material*](#) mknix::Cell::mat [protected]
Definition at line 128 of file cell.h.

8.10.4.9 nGPoints int mknix::Cell::nGPoints [protected]
number of Gauss Points
Definition at line 131 of file cell.h.

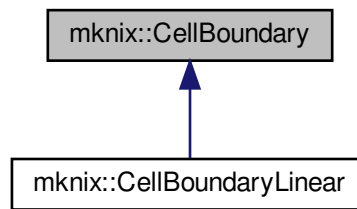
The documentation for this class was generated from the following files:

- [cell.h](#)
- [cell.cpp](#)

8.11 mknix::CellBoundary Class Reference

```
#include <cellboundary.h>
```

Inheritance diagram for `mknix::CellBoundary`:



Public Member Functions

- `CellBoundary` ()
Default constructor for `CellBoundary`.
- `CellBoundary` (std::string, double, int)
Constructs a `CellBoundary` with formulation, influence radius factor, and number of Gauss points.
- virtual `~CellBoundary` ()
Destructor. Releases all dynamically allocated boundary Gauss points.
- virtual void `initialize` (std::vector< `Node` * > &)
Initializes boundary Gauss points by finding support nodes (meshfree formulations only).
- virtual void `computeShapeFunctions` ()
Computes shape function values at each boundary Gauss point.
- void `computeQextGaussPoints` (`LoadThermalBoundary1D` *)
Computes the external thermal boundary load contribution at each Gauss point.
- void `assembleQextGaussPoints` (`Imx::Vector`< `data_type` > &)
Assembles boundary thermal load contributions from Gauss points into the global heat vector.

Protected Attributes

- std::string `formulation`
- double `alpha`
- int `nGPoints`
- std::vector< `GaussPointBoundary` * > `gPoints`
- double `jacobian`
- std::vector< `Point` * > `bodyPoints`
- double `dc`

8.11.1 Detailed Description

Author

Daniel Iglesias

Definition at line 55 of file `cellboundary.h`.

8.11.2 Constructor & Destructor Documentation

8.11.2.1 CellBoundary() [1/2] `mknix::CellBoundary::CellBoundary ( )`

Default constructor for [CellBoundary](#).

Definition at line 16 of file `cellboundary.cpp`.

8.11.2.2 CellBoundary() [2/2] `mknix::CellBoundary::CellBoundary (`

```
    std::string formulation_in,
    double alpha_in,
    int nGPoints_in )
```

Constructs a [CellBoundary](#) with formulation, influence radius factor, and number of Gauss points.

Parameters

<i>formulation_in</i>	String identifying the formulation (e.g. "EFG", "RPIM").
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of integration Gauss points.

Definition at line 27 of file `cellboundary.cpp`.

8.11.2.3 ~CellBoundary() `mknix::CellBoundary::~~CellBoundary ( ) [virtual]`

Destructor. Releases all dynamically allocated boundary Gauss points.

Definition at line 40 of file `cellboundary.cpp`.

8.11.3 Member Function Documentation**8.11.3.1 assembleQextGaussPoints()** `void mknix::CellBoundary::assembleQextGaussPoints (`

```
    lm::Vector< data_type > & globalQext )
```

Assembles boundary thermal load contributions from Gauss points into the global heat vector.

Parameters

<i>globalQext</i>	Global external heat load vector to be updated.
-------------------	---

Definition at line 101 of file `cellboundary.cpp`.

8.11.3.2 computeQextGaussPoints() `void mknix::CellBoundary::computeQextGaussPoints (`

```
    LoadThermalBoundary1D * loadThermalBoundary1D_in )
```

Computes the external thermal boundary load contribution at each Gauss point.

Parameters

<i>loadThermalBoundary1D_in</i>	Pointer to the 1D boundary thermal load object.
---------------------------------	---

Definition at line 89 of file `cellboundary.cpp`.

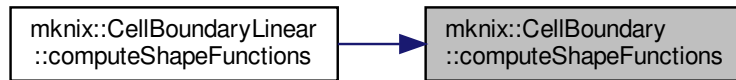
8.11.3.3 computeShapeFunctions() `void mknix::CellBoundary::computeShapeFunctions ( ) [virtual]`

Computes shape function values at each boundary Gauss point.

Reimplemented in [mknix::CellBoundaryLinear](#).

Definition at line 69 of file `cellboundary.cpp`.

Here is the caller graph for this function:



8.11.3.4 initialize() `void mknix::CellBoundary::initialize (`
`std::vector< Node * > & nodes_in ) [virtual]`

Initializes boundary Gauss points by finding support nodes (meshfree formulations only).

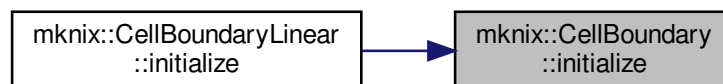
Parameters

<code>nodes_in</code>	Vector of domain nodes used to build the support neighbourhood.
-----------------------	---

Reimplemented in [mnix::CellBoundaryLinear](#).

Definition at line 53 of file cellboundary.cpp.

Here is the caller graph for this function:



8.11.4 Member Data Documentation

8.11.4.1 alpha `double mknix::CellBoundary::alpha [protected]`

Definition at line 81 of file cellboundary.h.

8.11.4.2 bodyPoints `std::vector< Point* > mknix::CellBoundary::bodyPoints [protected]`

Definition at line 85 of file cellboundary.h.

8.11.4.3 dc `double mknix::CellBoundary::dc [protected]`

Definition at line 86 of file cellboundary.h.

8.11.4.4 formulation `std::string mknix::CellBoundary::formulation [protected]`

Definition at line 80 of file cellboundary.h.

8.11.4.5 gPoints `std::vector<GaussPointBoundary*> mknix::CellBoundary::gPoints` [protected]
 Definition at line 83 of file cellboundary.h.

8.11.4.6 jacobian `double mknix::CellBoundary::jacobian` [protected]
 Definition at line 84 of file cellboundary.h.

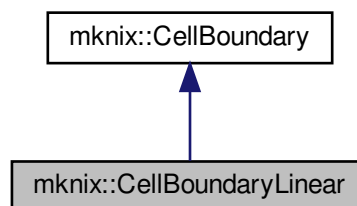
8.11.4.7 nGPoints `int mknix::CellBoundary::nGPoints` [protected]
 number of Gauss Points
 Definition at line 82 of file cellboundary.h.
 The documentation for this class was generated from the following files:

- [cellboundary.h](#)
- [cellboundary.cpp](#)

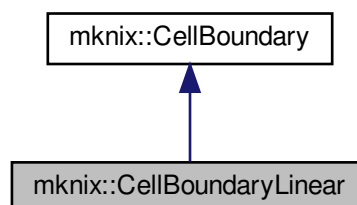
8.12 mknix::CellBoundaryLinear Class Reference

`#include <cellboundarylinear.h>`

Inheritance diagram for mknix::CellBoundaryLinear:



Collaboration diagram for mknix::CellBoundaryLinear:



Public Member Functions

- [CellBoundaryLinear](#) ()
Default constructor for [CellBoundaryLinear](#).
- [CellBoundaryLinear](#) (std::string, double, int, [Point](#) *, [Point](#) *)

Constructs a 1D linear boundary cell between two points.

- `CellBoundaryLinear` (std::string, double, int, `Point` *, `Point` *, double)

Constructs a 1D linear boundary cell between two points with an explicit nodal spacing.

- `~CellBoundaryLinear` ()

Destructor for `CellBoundaryLinear`.

- virtual void `initialize` (std::vector< `Node` * > &)

Initializes boundary Gauss points, delegating to the base class for meshfree cases and adding FEM support nodes directly for non-meshfree formulations.

- virtual void `computeShapeFunctions` ()

Computes shape function values at each Gauss point, using 1D linear functions for FEM.

Protected Member Functions

- void `createGaussPoints` ()

Creates and positions Gauss points along the 1D boundary segment using standard quadrature rules.

Protected Attributes

- `cofe::TensorRank2`< 3, double > `points`

8.12.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file `cellboundarylinear.h`.

8.12.2 Constructor & Destructor Documentation

8.12.2.1 `CellBoundaryLinear`() [1/3] `mknix::CellBoundaryLinear::CellBoundaryLinear` ()

Default constructor for `CellBoundaryLinear`.

Definition at line 13 of file `cellboundarylinear.cpp`.

8.12.2.2 `CellBoundaryLinear`() [2/3] `mknix::CellBoundaryLinear::CellBoundaryLinear` (

```
std::string formulation_in,
double alpha_in,
int nGPoints_in,
Point * n1_in,
Point * n2_in )
```

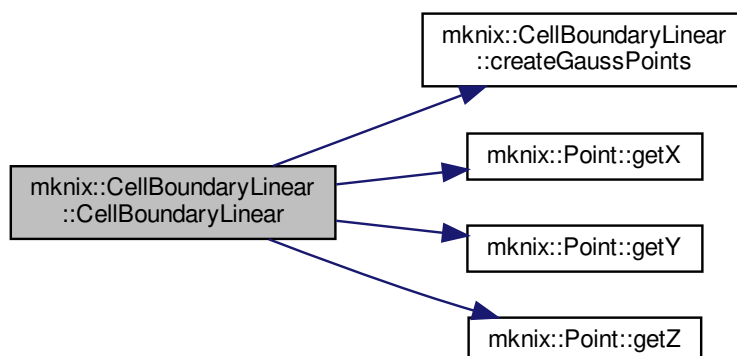
Constructs a 1D linear boundary cell between two points.

Parameters

<code>formulation_in</code>	String identifying the formulation.
<code>alpha_in</code>	Influence radius scaling factor.
<code>nGPoints_in</code>	Number of Gauss points along the boundary.
<code>n1_in</code>	Pointer to the first boundary point.
<code>n2_in</code>	Pointer to the second boundary point.

Definition at line 26 of file `cellboundarylinear.cpp`.

Here is the call graph for this function:



8.12.2.3 CellBoundaryLinear() [3/3] `mknix::CellBoundaryLinear::CellBoundaryLinear (`
`std::string formulation_in,`
`double alpha_in,`
`int nGPoints_in,`
`Point * n1_in,`
`Point * n2_in,`
`double dc_in )`

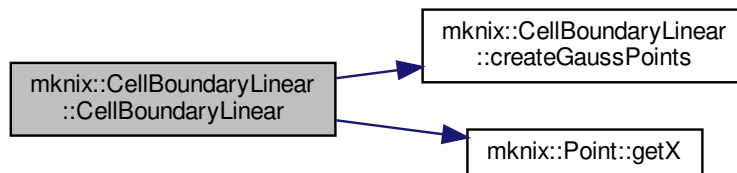
Constructs a 1D linear boundary cell between two points with an explicit nodal spacing.

Parameters

<i>formulation_in</i>	String identifying the formulation.
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of Gauss points along the boundary.
<i>n1_in</i>	Pointer to the first boundary point.
<i>n2_in</i>	Pointer to the second boundary point.
<i>dc_in</i>	Characteristic nodal spacing override.

Definition at line 63 of file cellboundarylinear.cpp.

Here is the call graph for this function:



8.12.2.4 `~CellBoundaryLinear()` `mknix::CellBoundaryLinear::~~CellBoundaryLinear ( )`

Destructor for [CellBoundaryLinear](#).

Definition at line 94 of file `cellboundarylinear.cpp`.

8.12.3 Member Function Documentation

8.12.3.1 `computeShapeFunctions()` `void mknix::CellBoundaryLinear::computeShapeFunctions ( )`

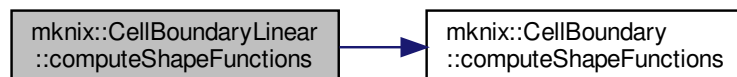
[virtual]

Computes shape function values at each Gauss point, using 1D linear functions for FEM.

Reimplemented from [mknix::CellBoundary](#).

Definition at line 127 of file `cellboundarylinear.cpp`.

Here is the call graph for this function:

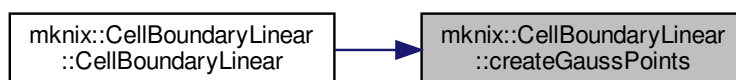


8.12.3.2 `createGaussPoints()` `void mknix::CellBoundaryLinear::createGaussPoints ( )` [protected]

Creates and positions Gauss points along the 1D boundary segment using standard quadrature rules.

Definition at line 145 of file `cellboundarylinear.cpp`.

Here is the caller graph for this function:



8.12.3.3 initialize() `void mknix::CellBoundaryLinear::initialize (`
`std::vector< Node * > & nodes_in ) [virtual]`

Initializes boundary Gauss points, delegating to the base class for meshfree cases and adding FEM support nodes directly for non-meshfree formulations.

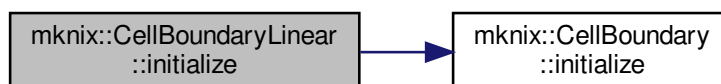
Parameters

<code>nodes_</code> <code>_in</code>	Vector of domain nodes.
---	-------------------------

Reimplemented from [mknix::CellBoundary](#).

Definition at line 104 of file `cellboundarylinear.cpp`.

Here is the call graph for this function:



8.12.4 Member Data Documentation

8.12.4.1 points `cofe::TensorRank2<3, double> mknix::CellBoundaryLinear::points [protected]`
 position of vertex points

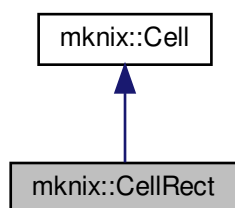
Definition at line 46 of file `cellboundarylinear.h`.

The documentation for this class was generated from the following files:

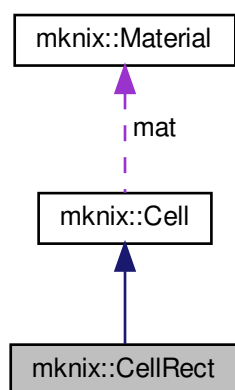
- [cellboundarylinear.h](#)
- [cellboundarylinear.cpp](#)

8.13 mknix::CellRect Class Reference

```
#include <cellrect.h>
```



Collaboration diagram for mknix::CellRect:



Public Member Functions

- `CellRect ()`
Default constructor for CellRect.
- `CellRect (Material &, std::string, double, int, double, double, double, double, double, double, double, double, double, double, double, double)`
Constructs a 2D rectangular integration cell defined by four corner points.
- `CellRect (Material &, std::string, double, int, double, double, double, double, double, double, double, double, double, double, double, double, double, double, double, double, double)`
Constructs a 3D hexahedral integration cell defined by eight corner points.
- `~CellRect ()`
Destructor for CellRect.
- `void initialize (std::vector< Node * > &)`
Initializes Gauss points by searching for support nodes within the cell bounding box.
- `void gnuplotOut (std::ofstream &, std::ofstream &)`
Outputs the cell geometry and Gauss point positions to files for gnuplot visualization.

Additional Inherited Members

8.13.1 Detailed Description

Author

Daniel Iglesias

Definition at line 49 of file cellrect.h.

8.13.2 Constructor & Destructor Documentation

8.13.2.1 CellRect() [1/3] mknix::CellRect::CellRect ()

Default constructor for [CellRect](#).

Definition at line 13 of file cellrect.cpp.

8.13.2.2 CellRect() [2/3] mknix::CellRect::CellRect (

```
Material & material_in,
std::string formulation_in,
double alpha_in,
int nGPoints_in,
double x1_in,
double y1_in,
double x2_in,
double y2_in,
double x3_in,
double y3_in,
double x4_in,
double y4_in,
double dcx_in,
double dcy_in,
double minX_in,
double minY_in,
double maxX_in,
double maxY_in )
```

Constructs a 2D rectangular integration cell defined by four corner points.

Parameters

<i>material_in</i>	Reference to the material.
<i>formulation_in</i>	Meshfree formulation string (e.g. "EFG").
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of Gauss points per direction.
<i>x1_in,y1_in</i>	Coordinates of corner 1.
<i>x2_in,y2_in</i>	Coordinates of corner 2.
<i>x3_in,y3_in</i>	Coordinates of corner 3.
<i>x4_in,y4_in</i>	Coordinates of corner 4.
<i>dcx_in,dcy_in</i>	Characteristic nodal spacing in x and y.
<i>minX_in,minY_in,maxX_in,maxY_in</i>	Bounding box for support node search.

Definition at line 31 of file cellrect.cpp.

```

8.13.2.3 CellRect() [3/3] mknix::CellRect::CellRect (
    Material & material_in,
    std::string formulation_in,
    double alpha_in,
    int nGPoints_in,
    double x1_in,
    double y1_in,
    double z1_in,
    double x2_in,
    double y2_in,
    double z2_in,
    double x3_in,
    double y3_in,
    double z3_in,
    double x4_in,
    double y4_in,
    double z4_in,
    double x5_in,
    double y5_in,
    double z5_in,
    double x6_in,
    double y6_in,
    double z6_in,
    double x7_in,
    double y7_in,
    double z7_in,
    double x8_in,
    double y8_in,
    double z8_in,
    double dcx_in,
    double dcy_in,
    double dcz_in,
    double minX_in,
    double minY_in,
    double minZ_in,
    double maxX_in,
    double maxY_in,
    double maxZ_in )

```

Constructs a 3D hexahedral integration cell defined by eight corner points.

Parameters

<i>material_in</i>	Reference to the material.
<i>formulation_in</i>	Meshfree formulation string.
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of Gauss points per direction.
<i>x1_in..z8_in</i>	Coordinates of the eight hex corners.
<i>dcx_in,dcy_in,dcz_in</i>	Characteristic nodal spacing in x, y, z.
<i>minX_in,minY_in,minZ_in,maxX_in,maxY_in,maxZ_in</i>	Bounding box for support node search.

Definition at line 82 of file cellrect.cpp.

```

8.13.2.4 ~CellRect() mknix::CellRect::~~CellRect ( )

```

Destructor for [CellRect](#).

Definition at line 146 of file cellrect.cpp.

8.13.3 Member Function Documentation

8.13.3.1 gnuplotOut() `void mknix::CellRect::gnuplotOut (`
`std::ofstream & data,`
`std::ofstream & gpdata ) [virtual]`

Outputs the cell geometry and Gauss point positions to files for gnuplot visualization.

Parameters

<i>data</i>	Output file stream for cell corner coordinates.
<i>gpdata</i>	Output file stream for Gauss point coordinates.

Implements [mknix::Cell](#).

Definition at line 366 of file cellrect.cpp.

8.13.3.2 initialize() `void mknix::CellRect::initialize (`
`std::vector< Node * > & nodes_in ) [virtual]`

Initializes Gauss points by searching for support nodes within the cell bounding box.

Parameters

<i>nodes</i> ↔ <i>_in</i>	Vector of domain nodes.
------------------------------	-------------------------

Reimplemented from [mknix::Cell](#).

Definition at line 349 of file cellrect.cpp.

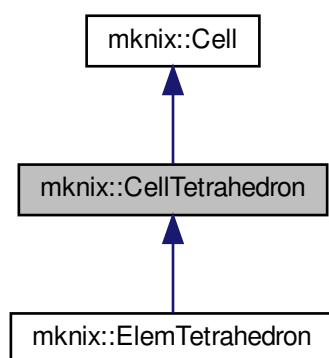
The documentation for this class was generated from the following files:

- [cellrect.h](#)
- [cellrect.cpp](#)

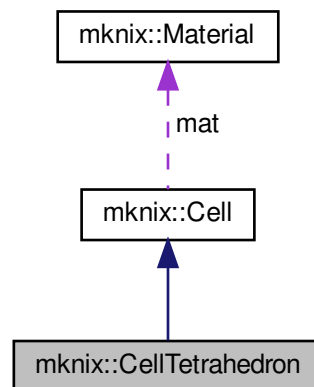
8.14 mknix::CellTetrahedron Class Reference

```
#include <celltetrahedron.h>
```

Inheritance diagram for mknix::CellTetrahedron:



Collaboration diagram for `mnix::CellTetrahedron`:



Public Member Functions

- [CellTetrahedron](#) ()
Default constructor for [CellTetrahedron](#).
- [CellTetrahedron](#) ([Material](#) &, `std::string`, `double`, `int`, [Point](#) *, [Point](#) *, [Point](#) *, [Point](#) *)
Constructs a tetrahedral integration cell from four corner points.
- virtual [~CellTetrahedron](#) ()
Destructor for [CellTetrahedron](#).
- void [gnuplotOut](#) (`std::ofstream` &, `std::ofstream` &)
Outputs tetrahedral cell geometry and Gauss points to files for gnuplot visualization.

Protected Member Functions

- void [createGaussPoints](#) ()
Creates and positions Gauss points inside the tetrahedron using standard quadrature rules.

Protected Attributes

- `cofe::TensorRank2` < 3, `double` > [points](#)

8.14.1 Detailed Description

Author

Daniel Iglesias

Definition at line 44 of file `celltetrahedron.h`.

8.14.2 Constructor & Destructor Documentation

8.14.2.1 [CellTetrahedron](#)() [1/2] `mnix::CellTetrahedron::CellTetrahedron ( )`

Default constructor for [CellTetrahedron](#).

Definition at line 14 of file `celltetrahedron.cpp`.

8.14.2.2 CellTetrahedron() [2/2] `mknix::CellTetrahedron::CellTetrahedron (`
`Material & material_in,`
`std::string formulation_in,`
`double alpha_in,`
`int nGPoints_in,`
`Point * n1_in,`
`Point * n2_in,`
`Point * n3_in,`
`Point * n4_in )`

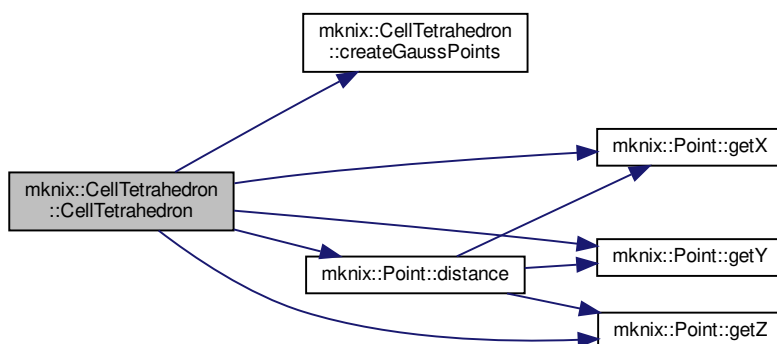
Constructs a tetrahedral integration cell from four corner points.

Parameters

<i>material_in</i>	Reference to the material.
<i>formulation_in</i>	Meshfree formulation string.
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of integration Gauss points.
<i>n1_in</i>	Pointer to corner point 1.
<i>n2_in</i>	Pointer to corner point 2.
<i>n3_in</i>	Pointer to corner point 3.
<i>n4_in</i>	Pointer to corner point 4.

Definition at line 30 of file celltetrahedron.cpp.

Here is the call graph for this function:



8.14.2.3 ~CellTetrahedron() `mknix::CellTetrahedron::~~CellTetrahedron ( )` [virtual]

Destructor for [CellTetrahedron](#).

Definition at line 83 of file celltetrahedron.cpp.

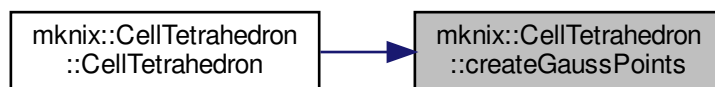
8.14.3 Member Function Documentation

8.14.3.1 createGaussPoints() `void mknix::CellTetrahedron::createGaussPoints ( )` [protected]

Creates and positions Gauss points inside the tetrahedron using standard quadrature rules.

Definition at line 91 of file celltetrahedron.cpp.

Here is the caller graph for this function:



8.14.3.2 gnuplotOut() `void mknix::CellTetrahedron::gnuplotOut (`
 `std::ofstream & data,`
 `std::ofstream & gpdata ) [virtual]`

Outputs tetrahedral cell geometry and Gauss points to files for gnuplot visualization.

Parameters

<i>data</i>	Output file stream for cell geometry.
<i>gpdata</i>	Output file stream for Gauss point positions.

Implements [mnix::Cell](#).

Definition at line 175 of file celltetrahedron.cpp.

8.14.4 Member Data Documentation

8.14.4.1 points `cofe::TensorRank2<3, double> mknix::CellTetrahedron::points [protected]`
 position of vertex points

Definition at line 47 of file celltetrahedron.h.

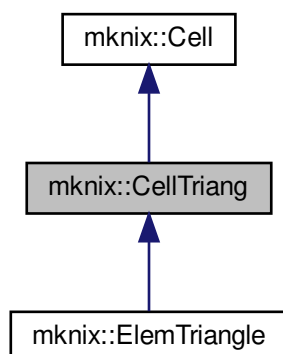
The documentation for this class was generated from the following files:

- [celltetrahedron.h](#)
- [celltetrahedron.cpp](#)

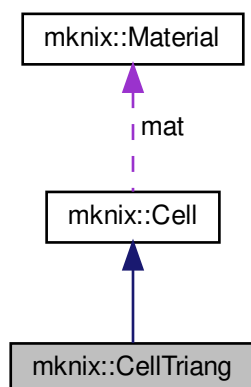
8.15 mknix::CellTriang Class Reference

```
#include <celltriang.h>
```

Inheritance diagram for mknix::CellTriang:



Collaboration diagram for mknix::CellTriang:



Public Member Functions

- [CellTriang](#) ()
Default constructor for [CellTriang](#).
- [CellTriang](#) (Material &, std::string, double, int, [Point](#) *, [Point](#) *, [Point](#) *)
Constructs a triangular integration cell from three points, computing dc automatically.
- [CellTriang](#) (Material &, std::string, double, int, [Point](#) *, [Point](#) *, [Point](#) *, double)
Constructs a triangular integration cell with an explicit characteristic nodal spacing.
- virtual [~CellTriang](#) ()
Destructor for [CellTriang](#).
- void [gnuplotOut](#) (std::ofstream &, std::ofstream &)
Outputs triangle cell geometry and Gauss point positions to files for gnuplot visualization.

Protected Member Functions

- void [createGaussPoints](#) ()

Creates and positions Gauss points inside the triangle using standard quadrature rules.

Protected Attributes

- `cofe::TensorRank2` < 3, double > [points](#)

8.15.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file `celltriang.h`.

8.15.2 Constructor & Destructor Documentation

8.15.2.1 `CellTriang()` [1/3] `mknix::CellTriang::CellTriang ( )`

Default constructor for [CellTriang](#).

Definition at line 13 of file `celltriang.cpp`.

8.15.2.2 `CellTriang()` [2/3] `mknix::CellTriang::CellTriang (`

```
    Material & material_in,  
    std::string formulation_in,  
    double alpha_in,  
    int nGPoints_in,  
    Point * n1_in,  
    Point * n2_in,  
    Point * n3_in )
```

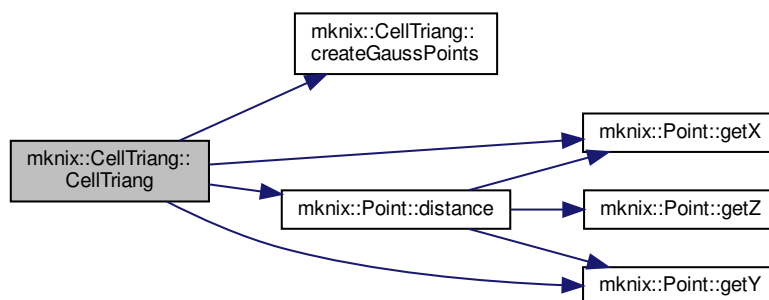
Constructs a triangular integration cell from three points, computing dc automatically.

Parameters

<code>material_in</code>	Reference to the material.
<code>formulation_in</code>	Meshfree formulation string.
<code>alpha_in</code>	Influence radius scaling factor.
<code>nGPoints_in</code>	Number of integration Gauss points.
<code>n1_in</code>	Pointer to triangle vertex 1.
<code>n2_in</code>	Pointer to triangle vertex 2.
<code>n3_in</code>	Pointer to triangle vertex 3.

Definition at line 28 of file `celltriang.cpp`.

Here is the call graph for this function:



8.15.2.3 CellTriang() [3/3] mknix::CellTriang::CellTriang (

```

Material & material_in,
std::string formulation_in,
double alpha_in,
int nGPoints_in,
Point * n1_in,
Point * n2_in,
Point * n3_in,
double dc_in )

```

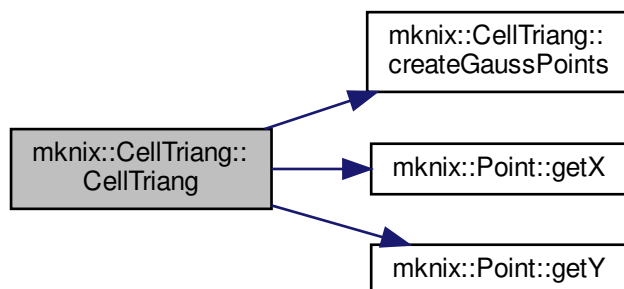
Constructs a triangular integration cell with an explicit characteristic nodal spacing.

Parameters

<i>material_in</i>	Reference to the material.
<i>formulation_in</i>	Meshfree formulation string.
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of integration Gauss points.
<i>n1_in</i>	Pointer to triangle vertex 1.
<i>n2_in</i>	Pointer to triangle vertex 2.
<i>n3_in</i>	Pointer to triangle vertex 3.
<i>dc_in</i>	Explicit characteristic nodal spacing.

Definition at line 73 of file celltriang.cpp.

Here is the call graph for this function:



8.15.2.4 `~CellTriang()` `mknix::CellTriang::~~CellTriang ( ) [virtual]`

Destructor for [CellTriang](#).

Definition at line 109 of file `celltriang.cpp`.

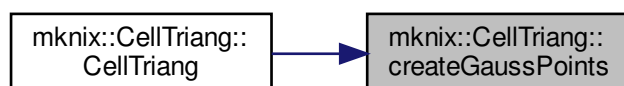
8.15.3 Member Function Documentation

8.15.3.1 `createGaussPoints()` `void mknix::CellTriang::createGaussPoints ( ) [protected]`

Creates and positions Gauss points inside the triangle using standard quadrature rules.

Definition at line 117 of file `celltriang.cpp`.

Here is the caller graph for this function:



8.15.3.2 `gnuplotOut()` `void mknix::CellTriang::gnuplotOut ( std::ofstream & data, std::ofstream & gpdata ) [virtual]`

Outputs triangle cell geometry and Gauss point positions to files for gnuplot visualization.

Parameters

<i>data</i>	Output file stream for cell corner coordinates.
<i>gpdata</i>	Output file stream for Gauss point coordinates.

Implements [mknix::Cell](#).

Definition at line 331 of file celltriang.cpp.

8.15.4 Member Data Documentation

8.15.4.1 points `cofe::TensorRank2<3, double> mknix::CellTriang::points` [protected]
position of vertex points

Definition at line 46 of file celltriang.h.

The documentation for this class was generated from the following files:

- [celltriang.h](#)
- [celltriang.cpp](#)

8.16 mknix::CompBar Class Reference

```
#include <compbar.h>
```

Public Member Functions

- [CompBar](#) ()
- [CompBar](#) (int, [Node](#) *, [Node](#) *)
- [~CompBar](#) ()
- void [addToRender](#) (vtkRenderer *)
- void [removeFromRender](#) (vtkRenderer *)
- void [updatePoints](#) ()

8.16.1 Detailed Description

Definition at line 38 of file compbar.h.

8.16.2 Constructor & Destructor Documentation

8.16.2.1 CompBar() [1/2] `mknix::CompBar::CompBar ( )`

8.16.2.2 CompBar() [2/2] `mknix::CompBar::CompBar (`
`int ,`
`Node * ,`
`Node * )`

8.16.2.3 ~CompBar() `mknix::CompBar::~~CompBar ( )`

8.16.3 Member Function Documentation

8.16.3.1 addToRender() `void mknix::CompBar::addToRender (`
`vtkRenderer * )`

8.16.3.2 removeFromRender() `void mknix::CompBar::removeFromRender (`
`vtkRenderer * )`

8.16.3.3 updatePoints() `void mknix::CompBar::updatePoints ( )`

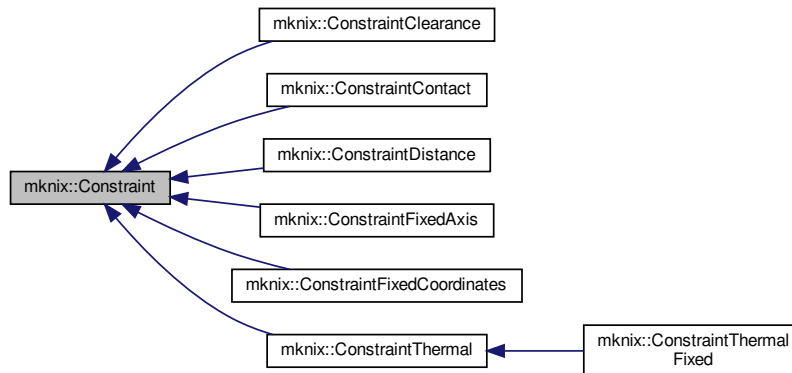
The documentation for this class was generated from the following file:

- [compbar.h](#)

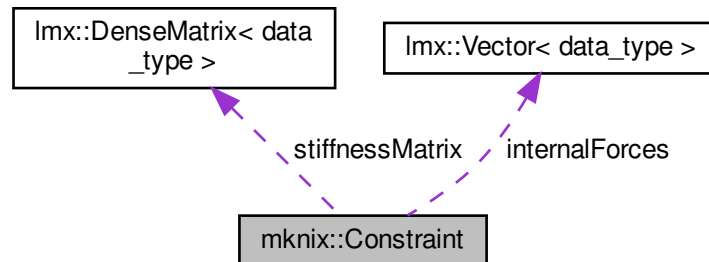
8.17 mknix::Constraint Class Reference

```
#include <constraint.h>
```

Inheritance diagram for mknix::Constraint:



Collaboration diagram for mknix::Constraint:



Public Member Functions

- [Constraint](#) ()
Default constructor. Reads `alpha` and `method` from the global [Simulation](#) settings.
- [Constraint](#) (double &, std::string &)
Constructor with explicit `alpha` and `method` parameters.
- [Constraint](#) (double &, std::string &, int)
Constructor with explicit parameters including problem dimension.
- virtual [~Constraint](#) ()
Destructor.
- void [setTitle](#) (std::string &title_in)

- `std::string getTitle ()`
- `Imx::Vector< data_type > & getInternalForces ()`
- `Imx::DenseMatrix< data_type > & getStiffnessMatrix ()`
- `virtual void writeJointInfo (std::ofstream *)`
Writes the joint type, title and node numbers to the output file.
- `virtual void calcPhi ()=0`
- `virtual void calcPhiq ()=0`
- `virtual void calcPhiqq ()=0`
- `virtual void calcInternalForces ()`
Computes the local internal force vector from the current constraint residual and Lagrange multiplier estimates.
- `virtual void calcTangentMatrix ()`
Computes the local tangent stiffness matrix from the second derivatives of the constraint.
- `virtual void assembleInternalForces (Imx::Vector< data_type > &)`
Assembles the local internal forces into the global force vector using shape-function mapping.
- `virtual void assembleTangentMatrix (Imx::Matrix< data_type > &)`
Assembles the local tangent matrix into the global tangent matrix using shape-function mapping.
- `virtual bool checkAugmented ()`
Checks whether the AUGMENTED method has converged. Updates Lagrange multipliers if not.
- `virtual void clearAugmented ()`
Resets the Lagrange multipliers to zero for the next augmented-Lagrangian cycle.
- `virtual Node * getNode (size_t nodeNumber)`
- `virtual size_t getNodeCount ()`
- `void outputStep (const Imx::Vector< data_type > &, const Imx::Vector< data_type > &)`
Stores the current internal force vector for post-processing (dynamic step).
- `void outputStep (const Imx::Vector< data_type > &)`
Stores the current internal force vector for post-processing (static step).
- `void outputToFile (std::ofstream *)`
Writes stored constraint force history to the output file.

Protected Attributes

- `int dim`
- `int iter_augmented`
- `double alpha`
- `double augmentedTolerance`
- `std::string method`
- `std::string title`
- `std::vector< Node * > nodes`
- `Imx::Vector< data_type > internalForces`
- `std::vector< Imx::Vector< data_type > > internalForcesOutput`
- `Imx::DenseMatrix< data_type > stiffnessMatrix`
- `std::vector< double > lambda`
- `std::vector< double > phi`
- `std::vector< Imx::Vector< data_type > > phi_q`
- `std::vector< Imx::DenseMatrix< data_type > > phi_qq`

8.17.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 34 of file constraint.h.

8.17.2 Constructor & Destructor Documentation

8.17.2.1 **Constraint()** [1/3] `mknix::Constraint::Constraint ( )`

Default constructor. Reads alpha and method from the global [Simulation](#) settings.

Definition at line 31 of file `constraint.cpp`.

8.17.2.2 **Constraint()** [2/3] `mknix::Constraint::Constraint ( double & alpha_in, std::string & method_in )`

Constructor with explicit alpha and method parameters.

Parameters

<i>alpha_in</i>	Augmented-Lagrangian penalty parameter.
<i>method_in</i>	Constraint enforcement method (e.g. "LAGRANGE", "AUGMENTED").

Definition at line 46 of file `constraint.cpp`.

8.17.2.3 **Constraint()** [3/3] `mknix::Constraint::Constraint ( double & alpha_in, std::string & method_in, int dim_in )`

Constructor with explicit parameters including problem dimension.

Parameters

<i>alpha_in</i>	Augmented-Lagrangian penalty parameter.
<i>method_in</i>	Constraint enforcement method.
<i>dim_in</i>	Problem dimension (2 or 3).

Definition at line 62 of file `constraint.cpp`.

8.17.2.4 **~Constraint()** `mknix::Constraint::~~Constraint ( )` [virtual]

Destructor.

Definition at line 75 of file `constraint.cpp`.

8.17.3 Member Function Documentation

8.17.3.1 **assembleInternalForces()** `void mknix::Constraint::assembleInternalForces ( lmx::Vector< data_type > & globalInternalForces )` [virtual]

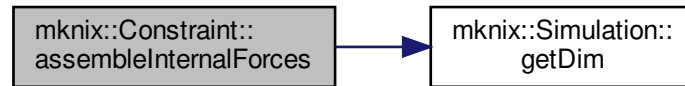
Assembles the local internal forces into the global force vector using shape-function mapping.

Parameters

<i>globalInternalForces</i>	Reference to the global internal force vector.
-----------------------------	--

Reimplemented in [mknix::ConstraintThermal](#).

Definition at line 149 of file constraint.cpp.
Here is the call graph for this function:



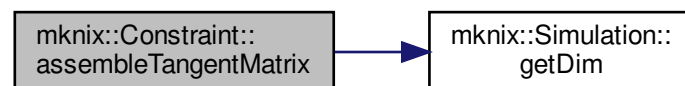
8.17.3.2 assembleTangentMatrix() `void mknix::Constraint::assembleTangentMatrix (
 lmx::Matrix< data_type > & globalTangent) [virtual]`

Assembles the local tangent matrix into the global tangent matrix using shape-function mapping.

Parameters

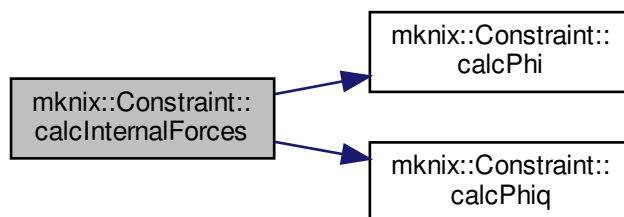
<i>globalTangent</i>	Reference to the global tangent (stiffness) matrix.
----------------------	---

Reimplemented in [mknix::ConstraintThermal](#).
Definition at line 184 of file constraint.cpp.
Here is the call graph for this function:



8.17.3.3 calcInternalForces() `void mknix::Constraint::calcInternalForces ( ) [virtual]`
Computes the local internal force vector from the current constraint residual and Lagrange multiplier estimates.
Definition at line 104 of file constraint.cpp.

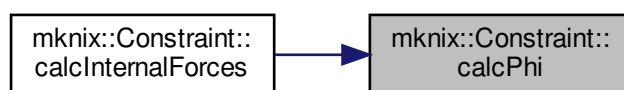
Here is the call graph for this function:



8.17.3.4 `calcPhi()` `virtual void mknix::Constraint::calcPhi ( ) [pure virtual]`

Implemented in [mknix::ConstraintThermalFixed](#), [mknix::ConstraintFixedCoordinates](#), [mknix::ConstraintFixedAxis](#), [mknix::ConstraintDistance](#), [mknix::ConstraintContact](#), and [mknix::ConstraintClearance](#).

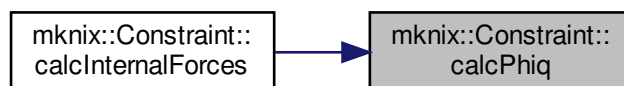
Here is the caller graph for this function:



8.17.3.5 `calcPhiq()` `virtual void mknix::Constraint::calcPhiq ( ) [pure virtual]`

Implemented in [mknix::ConstraintThermalFixed](#), [mknix::ConstraintFixedCoordinates](#), [mknix::ConstraintFixedAxis](#), [mknix::ConstraintDistance](#), [mknix::ConstraintContact](#), and [mknix::ConstraintClearance](#).

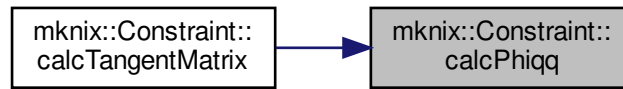
Here is the caller graph for this function:



8.17.3.6 `calcPhiqq()` `virtual void mknix::Constraint::calcPhiqq ( ) [pure virtual]`

Implemented in [mknix::ConstraintThermalFixed](#), [mknix::ConstraintFixedCoordinates](#), [mknix::ConstraintFixedAxis](#), [mknix::ConstraintDistance](#), [mknix::ConstraintContact](#), and [mknix::ConstraintClearance](#).

Here is the caller graph for this function:

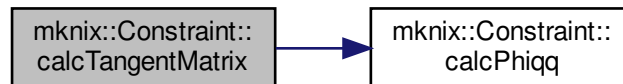


8.17.3.7 calcTangentMatrix() `void mknix::Constraint::calcTangentMatrix ( ) [virtual]`

Computes the local tangent stiffness matrix from the second derivatives of the constraint.

Definition at line 121 of file `constraint.cpp`.

Here is the call graph for this function:



8.17.3.8 checkAugmented() `bool mknix::Constraint::checkAugmented ( ) [virtual]`

Checks whether the AUGMENTED method has converged. Updates Lagrange multipliers if not.

Returns

True if converged or method is not AUGMENTED; false otherwise.

Reimplemented in [mknix::ConstraintThermalFixed](#).

Definition at line 238 of file `constraint.cpp`.

8.17.3.9 clearAugmented() `void mknix::Constraint::clearAugmented ( ) [virtual]`

Resets the Lagrange multipliers to zero for the next augmented-Lagrangian cycle.

Definition at line 278 of file `constraint.cpp`.

8.17.3.10 getInternalForces() `lmx::Vector<data_type>& mknix::Constraint::getInternalForces ( ) [inline]`

Definition at line 55 of file `constraint.h`.

8.17.3.11 getNode() `virtual Node* mknix::Constraint::getNode ( size_t nodeNumber ) [inline], [virtual]`

Definition at line 85 of file constraint.h.

8.17.3.12 getNodeCount() `virtual size_t mknix::Constraint::getNodeCount ( ) [inline], [virtual]`

Definition at line 90 of file constraint.h.

8.17.3.13 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::Constraint::getStiffnessMatrix ( ) [inline]`

Definition at line 60 of file constraint.h.

8.17.3.14 getTitle() `std::string mknix::Constraint::getTitle ( ) [inline]`

Definition at line 50 of file constraint.h.

8.17.3.15 outputStep() [1/2] `void mknix::Constraint::outputStep ( const lmx::Vector< data_type > & q )`

Stores the current internal force vector for post-processing (static step).

Parameters

<i>q</i>	Global configuration vector (unused here).
----------	--

Definition at line 321 of file constraint.cpp.

8.17.3.16 outputStep() [2/2] `void mknix::Constraint::outputStep ( const lmx::Vector< data_type > & q, const lmx::Vector< data_type > & qdot )`

Stores the current internal force vector for post-processing (dynamic step).

Parameters

<i>q</i>	Global configuration vector (unused here).
<i>qdot</i>	Global velocity vector (unused here).

Definition at line 308 of file constraint.cpp.

8.17.3.17 outputToFile() `void mknix::Constraint::outputToFile ( std::ofstream * outFile )`

Writes stored constraint force history to the output file.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 334 of file constraint.cpp.

8.17.3.18 setTitle() `void mknix::Constraint::setTitle ( std::string & title_in ) [inline]`

Definition at line 45 of file constraint.h.

8.17.3.19 writeJointInfo() `void mknix::Constraint::writeJointInfo ( std::ofstream * outfile ) [virtual]`

Writes the joint type, title and node numbers to the output file.

Parameters

<i>outfile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 84 of file constraint.cpp.

8.17.4 Member Data Documentation

8.17.4.1 alpha `double mknix::Constraint::alpha [protected]`

Definition at line 103 of file constraint.h.

8.17.4.2 augmentedTolerance `double mknix::Constraint::augmentedTolerance [protected]`

Definition at line 104 of file constraint.h.

8.17.4.3 dim `int mknix::Constraint::dim [protected]`

Definition at line 102 of file constraint.h.

8.17.4.4 internalForces `lmx::Vector<data_type> mknix::Constraint::internalForces [protected]`

Definition at line 108 of file constraint.h.

8.17.4.5 internalForcesOutput `std::vector<lmx::Vector<data_type> > mknix::Constraint::internalForcesOutput [protected]`

Definition at line 109 of file constraint.h.

8.17.4.6 iter_augmented `int mknix::Constraint::iter_augmented [protected]`

Definition at line 102 of file constraint.h.

8.17.4.7 lambda `std::vector<double> mknix::Constraint::lambda [protected]`

Definition at line 111 of file constraint.h.

8.17.4.8 method `std::string mknix::Constraint::method [protected]`

Definition at line 105 of file constraint.h.

8.17.4.9 nodes `std::vector<Node*> mknix::Constraint::nodes [protected]`

Definition at line 107 of file constraint.h.

8.17.4.10 phi `std::vector<double> mknix::Constraint::phi` [protected]

Definition at line 112 of file `constraint.h`.

8.17.4.11 phi_q `std::vector<lmx::Vector<data_type> > mknix::Constraint::phi_q` [protected]

Definition at line 113 of file `constraint.h`.

8.17.4.12 phi_qq `std::vector<lmx::DenseMatrix<data_type> > mknix::Constraint::phi_qq` [protected]

Definition at line 114 of file `constraint.h`.

8.17.4.13 stiffnessMatrix `lmx::DenseMatrix<data_type> mknix::Constraint::stiffnessMatrix` [protected]

Definition at line 110 of file `constraint.h`.

8.17.4.14 title `std::string mknix::Constraint::title` [protected]

Definition at line 106 of file `constraint.h`.

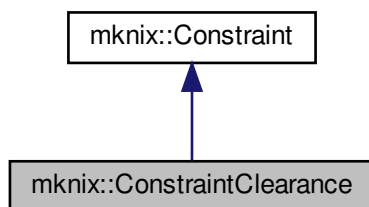
The documentation for this class was generated from the following files:

- [constraint.h](#)
- [constraint.cpp](#)

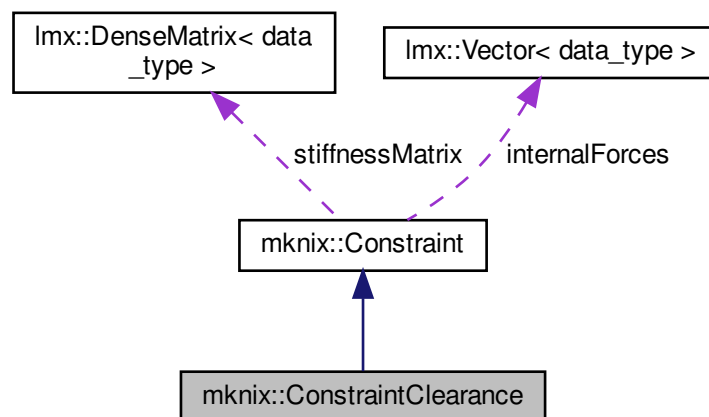
8.18 mknix::ConstraintClearance Class Reference

```
#include <constraintclearance.h>
```

Inheritance diagram for `mknix::ConstraintClearance`:



Collaboration diagram for mknix::ConstraintClearance:



Public Member Functions

- [ConstraintClearance](#) ()
Default constructor.
- [~ConstraintClearance](#) ()
Destructor.
- [ConstraintClearance](#) (Node *, Node *, double &, double &, std::string &)
Constructor for a clearance (radial gap) constraint between two nodes.
- void [calcPhi](#) ()
Evaluates $\phi = r_t^2 - r_h^2$ where r_t is the current inter-node distance.
- void [calcPhiq](#) ()
Computes the gradient of ϕ with respect to the nodal coordinates. Sets ϕ_q to zero when the gap is within the clearance radius.
- void [calcPhiqq](#) ()
Computes the Hessian of ϕ with respect to the nodal coordinates. Sets ϕ_{qq} to zero when the gap is within the clearance radius.
- [Imx::Vector< data_type >](#) & [getInternalForces](#) ()
- [Imx::DenseMatrix< data_type >](#) & [getStiffnessMatrix](#) ()

Protected Attributes

- double [rh](#)
- double [rt](#)

8.18.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file constraintclearance.h.

8.18.2 Constructor & Destructor Documentation

8.18.2.1 ConstraintClearance() [1/2] `mknix::ConstraintClearance::ConstraintClearance ( )`

Default constructor.

Definition at line 32 of file `constraintclearance.cpp`.

8.18.2.2 ~ConstraintClearance() `mknix::ConstraintClearance::~~ConstraintClearance ( )`

Destructor.

Definition at line 66 of file `constraintclearance.cpp`.

8.18.2.3 ConstraintClearance() [2/2] `mknix::ConstraintClearance::ConstraintClearance (`

```
Node * a_in,
Node * b_in,
double & rh_in,
double & alpha_in,
std::string & method_in )
```

Constructor for a clearance (radial gap) constraint between two nodes.

Parameters

<code>a_in</code>	Pointer to node A (inner body centre).
<code>b_in</code>	Pointer to node B (outer body centre).
<code>rh_in</code>	Reference (maximum allowed) distance.
<code>alpha_in</code>	Penalty parameter.
<code>method_in</code>	Enforcement method keyword.

Definition at line 45 of file `constraintclearance.cpp`.

8.18.3 Member Function Documentation

8.18.3.1 calcPhi() `void mknix::ConstraintClearance::calcPhi ( ) [virtual]`

Evaluates $\phi = r_t^2 - r_h^2$ where r_t is the current inter-node distance.

Implements [mknix::Constraint](#).

Definition at line 73 of file `constraintclearance.cpp`.

8.18.3.2 calcPhiq() `void mknix::ConstraintClearance::calcPhiq ( ) [virtual]`

Computes the gradient of ϕ with respect to the nodal coordinates. Sets ϕ_q to zero when the gap is within the clearance radius.

Implements [mknix::Constraint](#).

Definition at line 86 of file `constraintclearance.cpp`.

8.18.3.3 calcPhiqq() `void mknix::ConstraintClearance::calcPhiqq ( ) [virtual]`

Computes the Hessian of ϕ with respect to the nodal coordinates. Sets ϕ_{qq} to zero when the gap is within the clearance radius.

Implements [mknix::Constraint](#).

Definition at line 118 of file `constraintclearance.cpp`.

8.18.3.4 getInternalForces() `lmx::Vector<data_type>& mknix::ConstraintClearance::getInternalForces ( ) [inline]`

Definition at line 46 of file `constraintclearance.h`.

8.18.3.5 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::ConstraintClearance::get↔
StiffnessMatrix () [inline]`

Definition at line 51 of file `constraintclearance.h`.

8.18.4 Member Data Documentation

8.18.4.1 rh `double mknix::ConstraintClearance::rh [protected]`

Definition at line 57 of file `constraintclearance.h`.

8.18.4.2 rt `double mknix::ConstraintClearance::rt [protected]`

Definition at line 58 of file `constraintclearance.h`.

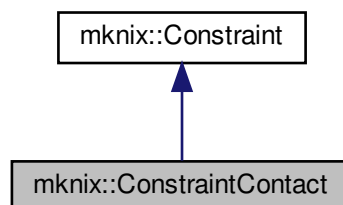
The documentation for this class was generated from the following files:

- [constraintclearance.h](#)
- [constraintclearance.cpp](#)

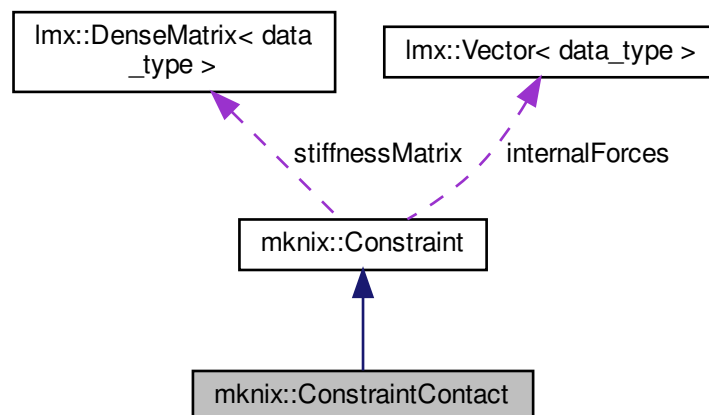
8.19 mknix::ConstraintContact Class Reference

```
#include <constraintcontact.h>
```

Inheritance diagram for `mknix::ConstraintContact`:



Collaboration diagram for `mknix::ConstraintContact`:



Public Member Functions

- [ConstraintContact](#) ()
Default constructor.
- [~ConstraintContact](#) ()
Destructor.
- [ConstraintContact](#) ([Node](#) *, [Node](#) *, [Node](#) *, double &, std::string &)
Constructor for a contact constraint between a surface segment and a contact point.
- void [calcPhi](#) ()
Evaluates the contact gap function phi using the surface normal and signed gap distance.
- void [calcPhiq](#) ()
Computes the gradient of phi with respect to the nodal coordinates. Returns zero when the contact gap is open ($rt > 0$).
- void [calcPhiqq](#) ()
Computes the Hessian of phi with respect to the nodal coordinates.
- [Imx::Vector< data_type > & getInternalForces](#) ()
- [Imx::DenseMatrix< data_type > & getStiffnessMatrix](#) ()
- double [getGap](#) ()

Protected Attributes

- std::vector< double > [normal](#)
- double [rh](#)
- double [rt](#)

8.19.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `constraintcontact.h`.

8.19.2 Constructor & Destructor Documentation

8.19.2.1 ConstraintContact() [1/2] mknix::ConstraintContact::ConstraintContact ()

Default constructor.

Definition at line 31 of file constraintcontact.cpp.

8.19.2.2 ~ConstraintContact() mknix::ConstraintContact::~~ConstraintContact ()

Destructor.

Definition at line 66 of file constraintcontact.cpp.

8.19.2.3 ConstraintContact() [2/2] mknix::ConstraintContact::ConstraintContact (

```
Node * q1_in,
Node * q2_in,
Node * p_in,
double & alpha_in,
std::string & method_in )
```

Constructor for a contact constraint between a surface segment and a contact point.

Parameters

<i>q1_in</i>	Pointer to the first node defining the contact surface.
<i>q2_in</i>	Pointer to the second node defining the contact surface.
<i>p_in</i>	Pointer to the contacting node.
<i>alpha_in</i>	Penalty parameter.
<i>method_in</i>	Enforcement method keyword.

Definition at line 44 of file constraintcontact.cpp.

8.19.3 Member Function Documentation

8.19.3.1 calcPhi() void mknix::ConstraintContact::calcPhi () [virtual]

Evaluates the contact gap function phi using the surface normal and signed gap distance.

Implements [mknix::Constraint](#).

Definition at line 73 of file constraintcontact.cpp.

8.19.3.2 calcPhiq() void mknix::ConstraintContact::calcPhiq () [virtual]

Computes the gradient of phi with respect to the nodal coordinates. Returns zero when the contact gap is open ($rt > 0$).

Implements [mknix::Constraint](#).

Definition at line 95 of file constraintcontact.cpp.

8.19.3.3 calcPhiqq() void mknix::ConstraintContact::calcPhiqq () [virtual]

Computes the Hessian of phi with respect to the nodal coordinates.

Implements [mknix::Constraint](#).

Definition at line 135 of file constraintcontact.cpp.

8.19.3.4 getGap() `double mknix::ConstraintContact::getGap ( ) [inline]`
 Definition at line 56 of file constraintcontact.h.

8.19.3.5 getInternalForces() `lmx::Vector<data_type>& mknix::ConstraintContact::getInternalForces ( ) [inline]`
 Definition at line 46 of file constraintcontact.h.

8.19.3.6 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::ConstraintContact::getStiffnessMatrix ( ) [inline]`
 Definition at line 51 of file constraintcontact.h.

8.19.4 Member Data Documentation

8.19.4.1 normal `std::vector<double> mknix::ConstraintContact::normal [protected]`
 Definition at line 62 of file constraintcontact.h.

8.19.4.2 rh `double mknix::ConstraintContact::rh [protected]`
 Definition at line 63 of file constraintcontact.h.

8.19.4.3 rt `double mknix::ConstraintContact::rt [protected]`
 Definition at line 64 of file constraintcontact.h.

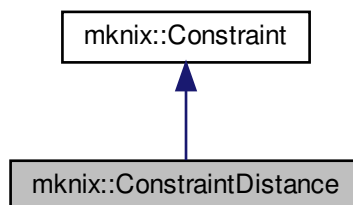
The documentation for this class was generated from the following files:

- [constraintcontact.h](#)
- [constraintcontact.cpp](#)

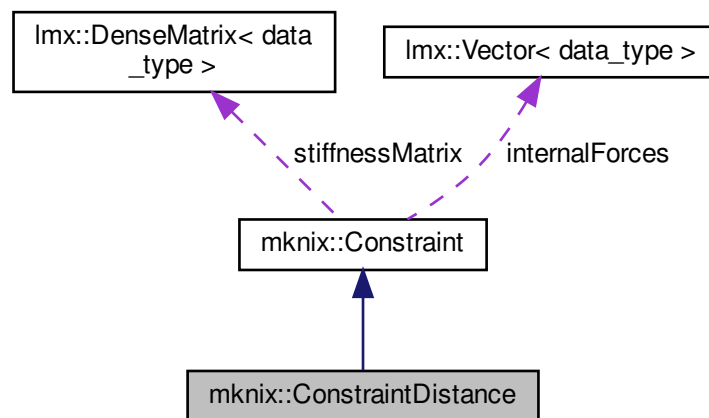
8.20 mknix::ConstraintDistance Class Reference

`#include <constraintdistance.h>`

Inheritance diagram for mknix::ConstraintDistance:



Collaboration diagram for mknix::ConstraintDistance:



Public Member Functions

- [ConstraintDistance](#) ()
Default constructor.
- [~ConstraintDistance](#) ()
Destructor.
- [ConstraintDistance](#) (Node *, Node *, double &, std::string &)
Constructor for a fixed-distance constraint between two nodes.
- void [calcRo](#) ()
Computes and stores the reference distance ro between the two nodes at the current configuration.
- void [calcPhi](#) ()
Evaluates $\phi = r_t^2 - r_o^2$ where r_t is the current inter-node distance.
- void [calcPhiq](#) ()
Computes the gradient of phi with respect to the nodal coordinates.
- void [calcPhiqq](#) ()
Computes the Hessian of phi with respect to the nodal coordinates.
- void [setLenght](#) (double new_ro)
- [Imx::Vector< data_type > &getInternalForces](#) ()
- [Imx::DenseMatrix< data_type > &getStiffnessMatrix](#) ()

Protected Attributes

- double [ro](#)
- double [rt](#)

8.20.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file constraintdistance.h.

8.20.2 Constructor & Destructor Documentation

8.20.2.1 ConstraintDistance() [1/2] `mknix::ConstraintDistance::ConstraintDistance ( )`

Default constructor.

Definition at line 31 of file `constraintdistance.cpp`.

8.20.2.2 ~ConstraintDistance() `mknix::ConstraintDistance::~~ConstraintDistance ( )`

Destructor.

Definition at line 74 of file `constraintdistance.cpp`.

8.20.2.3 ConstraintDistance() [2/2] `mknix::ConstraintDistance::ConstraintDistance (`

```
Node * a_in,
Node * b_in,
double & alpha_in,
std::string & method_in )
```

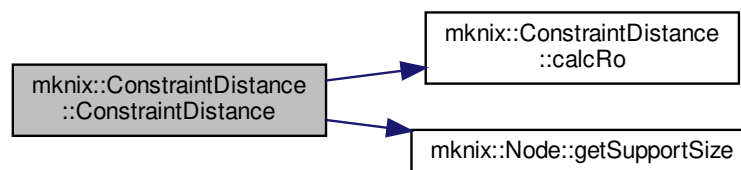
Constructor for a fixed-distance constraint between two nodes.

Parameters

<i>a_in</i>	Pointer to node A.
<i>b_in</i>	Pointer to node B.
<i>alpha_in</i>	Penalty parameter.
<i>method_in</i>	Enforcement method keyword.

Definition at line 43 of file `constraintdistance.cpp`.

Here is the call graph for this function:



8.20.3 Member Function Documentation

8.20.3.1 calcPhi() `void mknix::ConstraintDistance::calcPhi ( ) [virtual]`

Evaluates $\phi = r_t^2 - r_o^2$ where r_t is the current inter-node distance.

Implements [mknix::Constraint](#).

Definition at line 97 of file `constraintdistance.cpp`.

8.20.3.2 calcPhiq() `void mknix::ConstraintDistance::calcPhiq ( ) [virtual]`

Computes the gradient of ϕ with respect to the nodal coordinates.

Implements [mknix::Constraint](#).

Definition at line 130 of file constraintdistance.cpp.

8.20.3.3 calcPhiqq() `void mknix::ConstraintDistance::calcPhiqq ( ) [virtual]`

Computes the Hessian of phi with respect to the nodal coordinates.

Implements [mknix::Constraint](#).

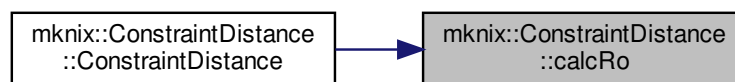
Definition at line 145 of file constraintdistance.cpp.

8.20.3.4 calcRo() `void mknix::ConstraintDistance::calcRo ( )`

Computes and stores the reference distance ro between the two nodes at the current configuration.

Definition at line 81 of file constraintdistance.cpp.

Here is the caller graph for this function:



8.20.3.5 getInternalForces() `lmx::Vector<data_type>& mknix::ConstraintDistance::getInternalForces ( ) [inline]`

Definition at line 53 of file constraintdistance.h.

8.20.3.6 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::ConstraintDistance::getStiffnessMatrix ( ) [inline]`

Definition at line 58 of file constraintdistance.h.

8.20.3.7 setLenght() `void mknix::ConstraintDistance::setLenght ( double new_ro ) [inline]`

Definition at line 48 of file constraintdistance.h.

8.20.4 Member Data Documentation

8.20.4.1 ro `double mknix::ConstraintDistance::ro [protected]`

Definition at line 64 of file constraintdistance.h.

8.20.4.2 rt `double mknix::ConstraintDistance::rt [protected]`

Definition at line 65 of file constraintdistance.h.

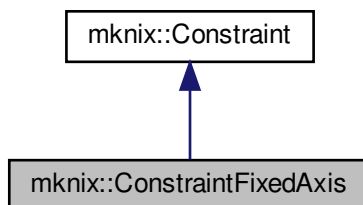
The documentation for this class was generated from the following files:

- [constraintdistance.h](#)
- [constraintdistance.cpp](#)

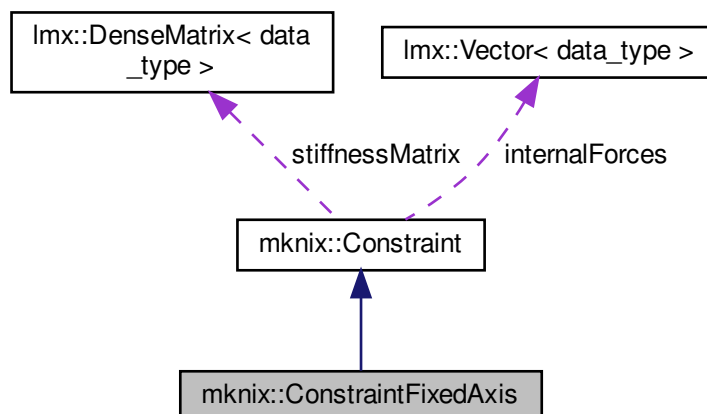
8.21 mknix::ConstraintFixedAxis Class Reference

```
#include <constraintfixedaxis.h>
```

Inheritance diagram for mknix::ConstraintFixedAxis:



Collaboration diagram for mknix::ConstraintFixedAxis:



Public Member Functions

- [ConstraintFixedAxis](#) ()
Default constructor.
- [ConstraintFixedAxis](#) ([Node](#) *, [Node](#) *, std::string &, double &, std::string &)
Constructor for a constraint that fixes the relative displacement along one axis.
- [~ConstraintFixedAxis](#) ()
Destructor.
- void [calcPhi](#) ()
Evaluates $\phi = r_t - r_o$, the current minus the reference displacement along the fixed axis.
- void [calcPhiq](#) ()
Computes the gradient of ϕ using the shape-function values of the support nodes.
- void [calcPhiqq](#) ()
Hessian of ϕ is zero (linear constraint); this is a no-op.

- [Imx::Vector< data_type > & getInternalForces \(\)](#)
- [Imx::DenseMatrix< data_type > & getStiffnessMatrix \(\)](#)

Protected Attributes

- `std::string` [axisName](#)
- `double` [ro](#)
- `double` [rt](#)

8.21.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `constraintfixedaxis.h`.

8.21.2 Constructor & Destructor Documentation

8.21.2.1 ConstraintFixedAxis() [1/2] `mknix::ConstraintFixedAxis::ConstraintFixedAxis ( )`

Default constructor.

Definition at line 31 of file `constraintfixedaxis.cpp`.

8.21.2.2 ConstraintFixedAxis() [2/2] `mknix::ConstraintFixedAxis::ConstraintFixedAxis (`

```
Node * a_in,
Node * b_in,
std::string & axisName_in,
double & alpha_in,
std::string & method_in )
```

Constructor for a constraint that fixes the relative displacement along one axis.

Parameters

<code>a_in</code>	Pointer to node A.
<code>b_in</code>	Pointer to node B.
<code>axisName_in</code>	Name of the locked axis: "x", "y", or "z".
<code>alpha_in</code>	Penalty parameter.
<code>method_in</code>	Enforcement method keyword.

Definition at line 45 of file `constraintfixedaxis.cpp`.

Here is the call graph for this function:



8.21.2.3 ~ConstraintFixedAxis() `mknix::ConstraintFixedAxis::~~ConstraintFixedAxis ( )`
Destructor.
Definition at line 79 of file `constraintfixedaxis.cpp`.

8.21.3 Member Function Documentation

8.21.3.1 calcPhi() `void mknix::ConstraintFixedAxis::calcPhi ( ) [virtual]`
Evaluates $\phi = r_t - r_o$, the current minus the reference displacement along the fixed axis.
Implements [mknix::Constraint](#).
Definition at line 86 of file `constraintfixedaxis.cpp`.

8.21.3.2 calcPhiq() `void mknix::ConstraintFixedAxis::calcPhiq ( ) [virtual]`
Computes the gradient of ϕ using the shape-function values of the support nodes.
Implements [mknix::Constraint](#).
Definition at line 101 of file `constraintfixedaxis.cpp`.

8.21.3.3 calcPhiqq() `void mknix::ConstraintFixedAxis::calcPhiqq ( ) [virtual]`
Hessian of ϕ is zero (linear constraint); this is a no-op.
Implements [mknix::Constraint](#).
Definition at line 159 of file `constraintfixedaxis.cpp`.

8.21.3.4 getInternalForces() `lmx::Vector<data_type>& mknix::ConstraintFixedAxis::getInternalForces ( ) [inline]`
Definition at line 46 of file `constraintfixedaxis.h`.

8.21.3.5 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::ConstraintFixedAxis::getStiffnessMatrix ( ) [inline]`
Definition at line 51 of file `constraintfixedaxis.h`.

8.21.4 Member Data Documentation

8.21.4.1 axisName `std::string mknix::ConstraintFixedAxis::axisName [protected]`
Definition at line 57 of file `constraintfixedaxis.h`.

8.21.4.2 ro `double mknix::ConstraintFixedAxis::ro [protected]`
Definition at line 58 of file `constraintfixedaxis.h`.

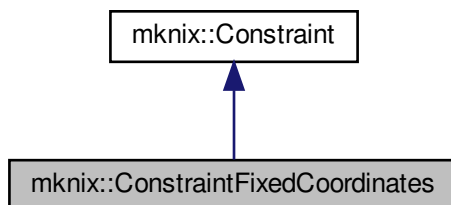
8.21.4.3 rt `double mknix::ConstraintFixedAxis::rt [protected]`
Definition at line 59 of file `constraintfixedaxis.h`.
The documentation for this class was generated from the following files:

- [constraintfixedaxis.h](#)
- [constraintfixedaxis.cpp](#)

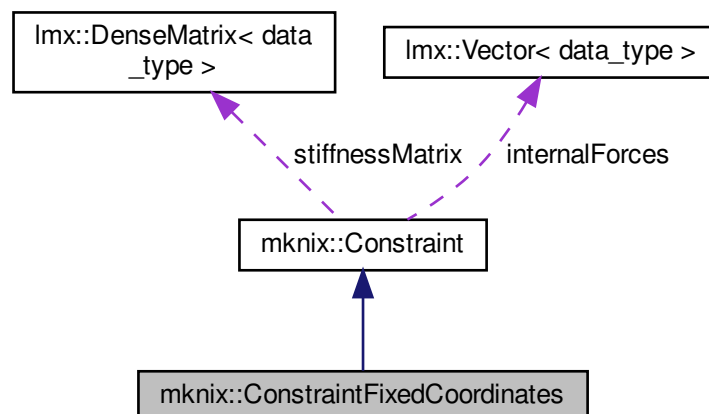
8.22 mknix::ConstraintFixedCoordinates Class Reference

```
#include <constraintfixedcoordinates.h>
```

Inheritance diagram for mknix::ConstraintFixedCoordinates:



Collaboration diagram for mknix::ConstraintFixedCoordinates:



Public Member Functions

- [ConstraintFixedCoordinates](#) ()
Default constructor.
- [ConstraintFixedCoordinates](#) (Node *, Node *, double &, std::string &)
Constructor for a constraint that fixes the relative displacement in all coordinate directions.
- [~ConstraintFixedCoordinates](#) ()
Destructor.
- void [calcPhi](#) ()
Evaluates $\phi[k]$ = relative displacement along axis k minus the reference value.
- void [calcPhiq](#) ()
Computes the gradient of each ϕ component using shape-function values of support nodes.
- void [calcPhiqq](#) ()
Hessian of ϕ is zero (linear constraint); this is a no-op.

- `Imx::Vector< data_type > & getInternalForces ()`
- `Imx::DenseMatrix< data_type > & getStiffnessMatrix ()`

Protected Attributes

- double `rxo`
- double `ryo`
- double `rzo`
- double `rxt`
- double `ryt`
- double `rzt`

8.22.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `constraintfixedcoordinates.h`.

8.22.2 Constructor & Destructor Documentation

8.22.2.1 ConstraintFixedCoordinates() [1/2] `mknix::ConstraintFixedCoordinates::ConstraintFixedCoordinates ( )`

Default constructor.

Definition at line 31 of file `constraintfixedcoordinates.cpp`.

8.22.2.2 ConstraintFixedCoordinates() [2/2] `mknix::ConstraintFixedCoordinates::ConstraintFixedCoordinates (`

```
Node * a_in,
Node * b_in,
double & alpha_in,
std::string & method_in )
```

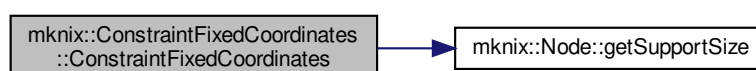
Constructor for a constraint that fixes the relative displacement in all coordinate directions.

Parameters

<code>a_in</code>	Pointer to node A.
<code>b_in</code>	Pointer to node B.
<code>alpha_in</code>	Penalty parameter.
<code>method_in</code>	Enforcement method keyword.

Definition at line 44 of file `constraintfixedcoordinates.cpp`.

Here is the call graph for this function:



8.22.2.3 ~ConstraintFixedCoordinates() `mknix::ConstraintFixedCoordinates::~~ConstraintFixedCoordinates ( )`

Destructor.

Definition at line 88 of file `constraintfixedcoordinates.cpp`.

8.22.3 Member Function Documentation

8.22.3.1 calcPhi() `void mknix::ConstraintFixedCoordinates::calcPhi ( ) [virtual]`

Evaluates $\phi[k]$ = relative displacement along axis k minus the reference value.

Implements [mknix::Constraint](#).

Definition at line 95 of file `constraintfixedcoordinates.cpp`.

8.22.3.2 calcPhiq() `void mknix::ConstraintFixedCoordinates::calcPhiq ( ) [virtual]`

Computes the gradient of each phi component using shape-function values of support nodes.

Implements [mknix::Constraint](#).

Definition at line 114 of file `constraintfixedcoordinates.cpp`.

8.22.3.3 calcPhiqq() `void mknix::ConstraintFixedCoordinates::calcPhiqq ( ) [virtual]`

Hessian of phi is zero (linear constraint); this is a no-op.

Implements [mknix::Constraint](#).

Definition at line 147 of file `constraintfixedcoordinates.cpp`.

8.22.3.4 getInternalForces() `lmx::Vector<data_type>& mknix::ConstraintFixedCoordinates::getInternalForces ( ) [inline]`

Definition at line 46 of file `constraintfixedcoordinates.h`.

8.22.3.5 getStiffnessMatrix() `lmx::DenseMatrix<data_type>& mknix::ConstraintFixedCoordinates::getStiffnessMatrix ( ) [inline]`

Definition at line 51 of file `constraintfixedcoordinates.h`.

8.22.4 Member Data Documentation

8.22.4.1 rxo `double mknix::ConstraintFixedCoordinates::rxo [protected]`

Definition at line 57 of file `constraintfixedcoordinates.h`.

8.22.4.2 rxt `double mknix::ConstraintFixedCoordinates::rxt [protected]`

Definition at line 58 of file `constraintfixedcoordinates.h`.

8.22.4.3 ryo `double mknix::ConstraintFixedCoordinates::ryo [protected]`

Definition at line 57 of file `constraintfixedcoordinates.h`.

8.22.4.4 ryt `double mknix::ConstraintFixedCoordinates::ryt [protected]`

Definition at line 58 of file `constraintfixedcoordinates.h`.

8.22.4.5 rzo `double mknix::ConstraintFixedCoordinates::rzo [protected]`
Definition at line 57 of file `constraintfixedcoordinates.h`.

8.22.4.6 rzt `double mknix::ConstraintFixedCoordinates::rzt [protected]`
Definition at line 58 of file `constraintfixedcoordinates.h`.

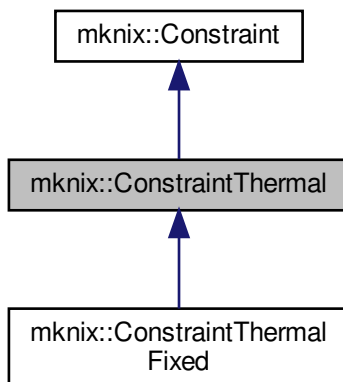
The documentation for this class was generated from the following files:

- [constraintfixedcoordinates.h](#)
- [constraintfixedcoordinates.cpp](#)

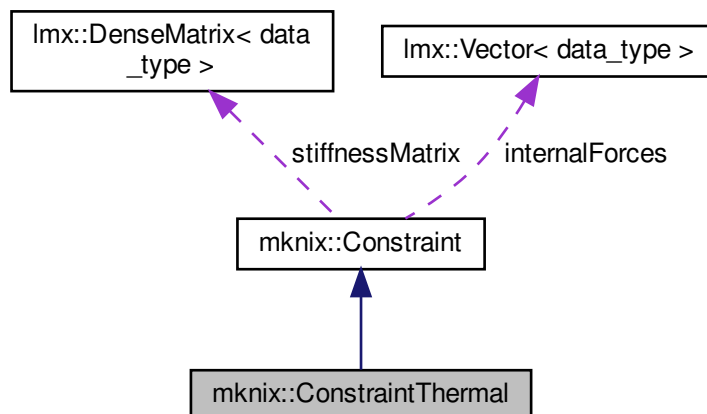
8.23 mknix::ConstraintThermal Class Reference

```
#include <constraintthermal.h>
```

Inheritance diagram for `mknix::ConstraintThermal`:



Collaboration diagram for `mknix::ConstraintThermal`:



Public Member Functions

- [ConstraintThermal](#) ()
Default constructor.
- [ConstraintThermal](#) (double &, std::string &)
Constructor with penalty parameter and method.
- [~ConstraintThermal](#) ()
Destructor.
- virtual void [assembleInternalForces](#) (Imx::Vector< [data_type](#) > &)
Assembles the thermal constraint internal forces into the global thermal load vector.
- virtual void [assembleTangentMatrix](#) (Imx::Matrix< [data_type](#) > &)
Assembles the thermal constraint tangent matrix into the global thermal tangent matrix.

Additional Inherited Members

8.23.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 35 of file constraintthermal.h.

8.23.2 Constructor & Destructor Documentation

8.23.2.1 [ConstraintThermal\(\)](#) [1/2] mknix::ConstraintThermal::ConstraintThermal ()

Default constructor.

Definition at line 31 of file constraintthermal.cpp.

8.23.2.2 [ConstraintThermal\(\)](#) [2/2] mknix::ConstraintThermal::ConstraintThermal (double & *alpha_in*, std::string & *method_in*)

Constructor with penalty parameter and method.

Parameters

<i>alpha_in</i>	Penalty parameter.
<i>method_in</i>	Enforcement method keyword.

Definition at line 42 of file constraintthermal.cpp.

8.23.2.3 [~ConstraintThermal\(\)](#) mknix::ConstraintThermal::~~ConstraintThermal ()

Destructor.

Definition at line 51 of file constraintthermal.cpp.

8.23.3 Member Function Documentation

8.23.3.1 [assembleInternalForces\(\)](#) void mknix::ConstraintThermal::assembleInternalForces (Imx::Vector< [data_type](#) > & *globalInternalForces*) [virtual]

Assembles the thermal constraint internal forces into the global thermal load vector.

Parameters

<i>globalInternalForces</i>	Reference to the global internal heat vector.
-----------------------------	---

Reimplemented from [mknix::Constraint](#).

Definition at line 59 of file constraintthermal.cpp.

8.23.3.2 assembleTangentMatrix() `void mknix::ConstraintThermal::assembleTangentMatrix (
 lmx::Matrix< data_type > & globalTangent) [virtual]`

Assembles the thermal constraint tangent matrix into the global thermal tangent matrix.

Parameters

<i>globalTangent</i>	Reference to the global tangent matrix.
----------------------	---

Reimplemented from [mknix::Constraint](#).

Definition at line 94 of file constraintthermal.cpp.

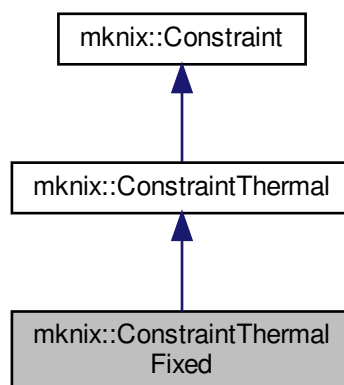
The documentation for this class was generated from the following files:

- [constraintthermal.h](#)
- [constraintthermal.cpp](#)

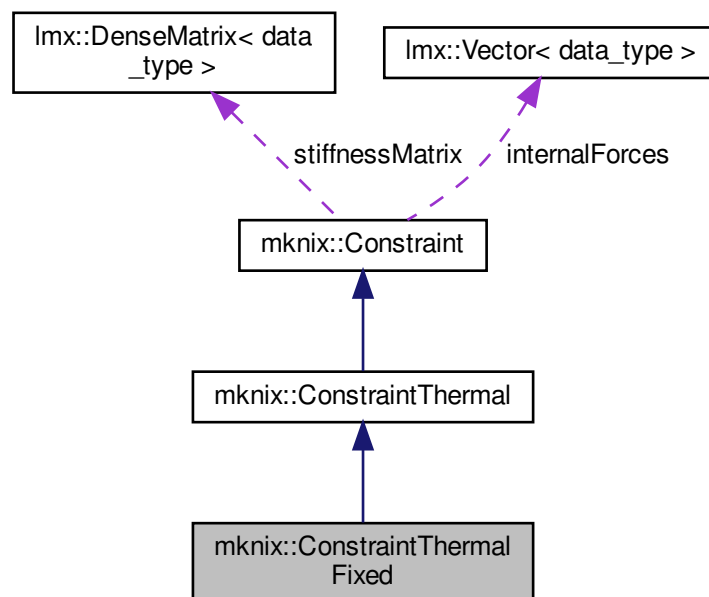
8.24 mknix::ConstraintThermalFixed Class Reference

```
#include <constraintthermalfixed.h>
```

Inheritance diagram for mknix::ConstraintThermalFixed:



Collaboration diagram for mknix::ConstraintThermalFixed:



Public Member Functions

- [ConstraintThermalFixed](#) ()
Default constructor.
- [ConstraintThermalFixed](#) (Node *, Node *, double &, std::string &)
Constructor for a fixed thermal constraint (isothermal boundary between two nodes).
- [~ConstraintThermalFixed](#) ()
Destructor.
- void [calcPhi](#) ()
Evaluates the thermal constraint $\phi = T_b - T_a$ (temperature difference).
- void [calcPhiq](#) ()
Computes the gradient of the thermal constraint using shape-function values.
- void [calcPhiqq](#) ()
Hessian of ϕ is zero (linear thermal constraint); this is a no-op.
- bool [checkAugmented](#) ()
Checks whether the AUGMENTED method has converged for this thermal constraint. Uses temperature-dependent tolerances (tighter above 300 K).

Protected Attributes

- double [Tt](#)

8.24.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file constraintthermalfixed.h.

8.24.2 Constructor & Destructor Documentation

8.24.2.1 ConstraintThermalFixed() [1/2] `mknix::ConstraintThermalFixed::ConstraintThermalFixed ( )`

Default constructor.

Definition at line 31 of file `constraintthermalfixed.cpp`.

8.24.2.2 ConstraintThermalFixed() [2/2] `mknix::ConstraintThermalFixed::ConstraintThermalFixed ( Node * a_in, Node * b_in, double & alpha_in, std::string & method_in )`

Constructor for a fixed thermal constraint (isothermal boundary between two nodes).

Parameters

<i>a_in</i>	Pointer to the reference temperature node.
<i>b_in</i>	Pointer to the constrained temperature node.
<i>alpha_in</i>	Penalty parameter.
<i>method_in</i>	Enforcement method keyword.

Definition at line 44 of file `constraintthermalfixed.cpp`.

Here is the call graph for this function:



8.24.2.3 ~ConstraintThermalFixed() `mknix::ConstraintThermalFixed::~~ConstraintThermalFixed ( )`

Destructor.

Definition at line 77 of file `constraintthermalfixed.cpp`.

8.24.3 Member Function Documentation

8.24.3.1 calcPhi() `void mknix::ConstraintThermalFixed::calcPhi ( ) [virtual]`

Evaluates the thermal constraint $\phi = T_b - T_a$ (temperature difference).

Implements [mknix::Constraint](#).

Definition at line 84 of file `constraintthermalfixed.cpp`.

8.24.3.2 calcPhiq() `void mknix::ConstraintThermalFixed::calcPhiq ( ) [virtual]`

Computes the gradient of the thermal constraint using shape-function values.

Implements [mknix::Constraint](#).

Definition at line 97 of file constraintthermalfixed.cpp.

8.24.3.3 calcPhiqq() `void mknix::ConstraintThermalFixed::calcPhiqq ( ) [virtual]`

Hessian of phi is zero (linear thermal constraint); this is a no-op.

Implements [mknix::Constraint](#).

Definition at line 117 of file constraintthermalfixed.cpp.

8.24.3.4 checkAugmented() `bool mknix::ConstraintThermalFixed::checkAugmented ( ) [virtual]`

Checks whether the AUGMENTED method has converged for this thermal constraint. Uses temperature-dependent tolerances (tighter above 300 K).

Returns

True if converged or method is not AUGMENTED; false otherwise.

Reimplemented from [mknix::Constraint](#).

Definition at line 126 of file constraintthermalfixed.cpp.

8.24.4 Member Data Documentation

8.24.4.1 Tt `double mknix::ConstraintThermalFixed::Tt [protected]`

Definition at line 49 of file constraintthermalfixed.h.

The documentation for this class was generated from the following files:

- [constraintthermalfixed.h](#)
- [constraintthermalfixed.cpp](#)

8.25 mknix::Contact Class Reference

```
#include <generalcontact.h>
```

Public Member Functions

- [Contact](#) ()
- [Contact](#) ([Simulation](#) *, double)
- [~Contact](#) ()
- void [createPoints](#) ()
- void [updatePoints](#) ()
- void [createPolys](#) ()
- void [updateLines](#) ()
- void [createDelaunay](#) ()
- void [updateDelaunay](#) ()
- void [createDrawingObjects](#) ()
- void [createDrawingContactObjects](#) ()
- void [drawObjects](#) ()

8.25.1 Detailed Description

Definition at line 51 of file generalcontact.h.

8.25.2 Constructor & Destructor Documentation

8.25.2.1 Contact() [1/2] `mknix::Contact::Contact ( )`

8.25.2.2 Contact() [2/2] `mknix::Contact::Contact (
Simulation * ,
double)`

8.25.2.3 ~Contact() `mknix::Contact::~~Contact ( ) [inline]`
Definition at line 56 of file `generalcontact.h`.

8.25.3 Member Function Documentation

8.25.3.1 createDelaunay() `void mknix::Contact::createDelaunay ( )`

8.25.3.2 createDrawingContactObjects() `void mknix::Contact::createDrawingContactObjects ( )`

8.25.3.3 createDrawingObjects() `void mknix::Contact::createDrawingObjects ( )`

8.25.3.4 createPoints() `void mknix::Contact::createPoints ( )`

8.25.3.5 createPolys() `void mknix::Contact::createPolys ( )`

8.25.3.6 drawObjects() `void mknix::Contact::drawObjects ( )`

8.25.3.7 updateDelaunay() `void mknix::Contact::updateDelaunay ( )`

8.25.3.8 updateLines() `void mknix::Contact::updateLines ( )`

8.25.3.9 updatePoints() `void mknix::Contact::updatePoints ( )`

The documentation for this class was generated from the following file:

- [generalcontact.h](#)

8.26 Imx::DenseMatrix< T > Class Template Reference

8.26.1 Detailed Description

```
template<typename T>  
class Imx::DenseMatrix< T >
```

Definition at line 40 of file `cellrect.h`.

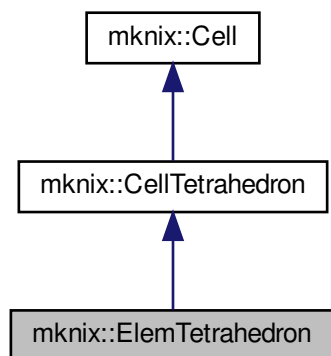
The documentation for this class was generated from the following file:

- [cellrect.h](#)

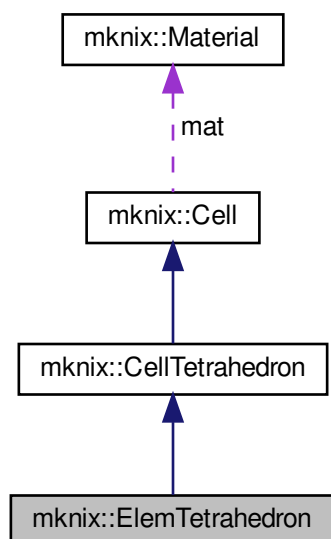
8.27 mknix::ElemTetrahedron Class Reference

```
#include <elemtetrahedron.h>
```

Inheritance diagram for mknix::ElemTetrahedron:



Collaboration diagram for mknix::ElemTetrahedron:



Public Member Functions

- [ElemTetrahedron](#) ()
Default constructor for [ElemTetrahedron](#).
- [ElemTetrahedron](#) ([Material](#) &, double, int, [Node](#) *, [Node](#) *, [Node](#) *, [Node](#) *)
Constructs an FEM tetrahedral element from four nodes.

- `~ElemTetrahedron ()`
Destructor for [ElemTetrahedron](#).
- `void initialize (std::vector< Node * > &)`
Initializes the FEM tetrahedral element by assigning the four corner nodes as support nodes for all Gauss points and creating separate mass-consistent Gauss points.
- `void computeShapeFunctions ()`
Computes FEM shape functions at each Gauss point by filling the B matrix.
- `void createGaussPoints_MC ()`
Creates mass-consistent Gauss points (gPoints_MC) with a higher-order quadrature rule than the standard stress Gauss points.

Additional Inherited Members

8.27.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `elemtetrahedron.h`.

8.27.2 Constructor & Destructor Documentation

8.27.2.1 ElemTetrahedron() [1/2] `mknix::ElemTetrahedron::ElemTetrahedron ()`

Default constructor for [ElemTetrahedron](#).

Definition at line 31 of file `elemtetrahedron.cpp`.

8.27.2.2 ElemTetrahedron() [2/2] `mknix::ElemTetrahedron::ElemTetrahedron (`

```
Material & material_in,  
double alpha_in,  
int nGPoints_in,  
Node * n1_in,  
Node * n2_in,  
Node * n3_in,  
Node * n4_in )
```

Constructs an FEM tetrahedral element from four nodes.

Parameters

<code>material_in</code>	Reference to the material.
<code>alpha_in</code>	Influence radius scaling factor.
<code>nGPoints_in</code>	Number of integration Gauss points.
<code>n1_in</code>	Pointer to node 1.
<code>n2_in</code>	Pointer to node 2.
<code>n3_in</code>	Pointer to node 3.
<code>n4_in</code>	Pointer to node 4.

Definition at line 47 of file `elemtetrahedron.cpp`.

8.27.2.3 ~ElemTetrahedron() `mknix::ElemTetrahedron::~~ElemTetrahedron ()`

Destructor for [ElemTetrahedron](#).

Definition at line 69 of file `elemtetrahedron.cpp`.

8.27.3 Member Function Documentation

8.27.3.1 computeShapeFunctions() `void mknix::ElemTetrahedron::computeShapeFunctions ( ) [virtual]`

Computes FEM shape functions at each Gauss point by filling the B matrix.

Reimplemented from [mknix::Cell](#).

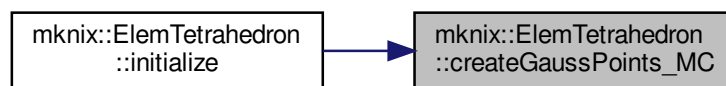
Definition at line 107 of file `elemtetrahedron.cpp`.

8.27.3.2 createGaussPoints_MC() `void mknix::ElemTetrahedron::createGaussPoints_MC ( )`

Creates mass-consistent Gauss points (`gPoints_MC`) with a higher-order quadrature rule than the standard stress Gauss points.

Definition at line 123 of file `elemtetrahedron.cpp`.

Here is the caller graph for this function:



8.27.3.3 initialize() `void mknix::ElemTetrahedron::initialize ( std::vector< Node * > & nodes_in ) [virtual]`

Initializes the FEM tetrahedral element by assigning the four corner nodes as support nodes for all Gauss points and creating separate mass-consistent Gauss points.

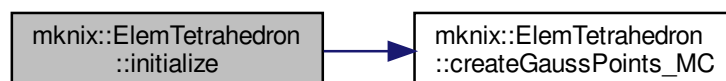
Parameters

<code>nodes_in</code>	Vector of domain nodes (unused directly; corners are added explicitly).
-----------------------	---

Reimplemented from [mknix::Cell](#).

Definition at line 79 of file `elemtetrahedron.cpp`.

Here is the call graph for this function:



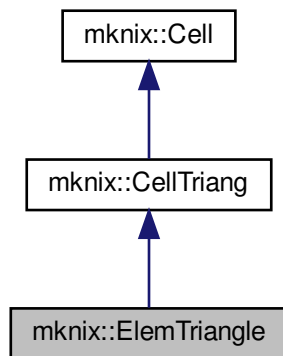
The documentation for this class was generated from the following files:

- [elemtetrahedron.h](#)
- [elemtetrahedron.cpp](#)

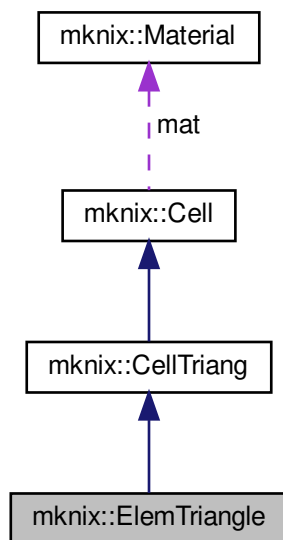
8.28 mknix::ElemTriangle Class Reference

```
#include <elemtriangle.h>
```

Inheritance diagram for mknix::ElemTriangle:



Collaboration diagram for mknix::ElemTriangle:



Public Member Functions

- [ElemTriangle](#) ()
Default constructor for [ElemTriangle](#).
- [ElemTriangle](#) ([Material](#) &, double, int, [Node](#) *, [Node](#) *, [Node](#) *)
Constructs an FEM triangular element from three nodes.

- [~ElemTriangle](#) ()
Destructor for [ElemTriangle](#).
- void [initialize](#) (std::vector< [Node](#) * > &)
Initializes the FEM triangle by assigning the three corner nodes as support nodes for all Gauss points and creating separate mass-consistent Gauss points.
- void [computeShapeFunctions](#) ()
Computes FEM shape functions at each Gauss point by filling the B matrix.

Protected Member Functions

- void [createGaussPoints_MC](#) ()
Creates mass-consistent Gauss points (gPoints_MC) with a higher-order quadrature rule than the standard stress Gauss points.

Additional Inherited Members

8.28.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file elemtriangle.h.

8.28.2 Constructor & Destructor Documentation

8.28.2.1 ElemTriangle() [1/2] mknix::ElemTriangle::ElemTriangle ()

Default constructor for [ElemTriangle](#).

Definition at line 31 of file elemtriangle.cpp.

8.28.2.2 ElemTriangle() [2/2] mknix::ElemTriangle::ElemTriangle (

```
Material & material_in,
double alpha_in,
int nGPoints_in,
Node * n1_in,
Node * n2_in,
Node * n3_in )
```

Constructs an FEM triangular element from three nodes.

Parameters

<i>material_in</i>	Reference to the material.
<i>alpha_in</i>	Influence radius scaling factor.
<i>nGPoints_in</i>	Number of integration Gauss points.
<i>n1_in</i>	Pointer to node 1.
<i>n2_in</i>	Pointer to node 2.
<i>n3_in</i>	Pointer to node 3.

Definition at line 46 of file elemtriangle.cpp.

8.28.2.3 ~ElemTriangle() mknix::ElemTriangle::~~ElemTriangle ()

Destructor for [ElemTriangle](#).

Definition at line 66 of file elemtriangle.cpp.

8.28.3 Member Function Documentation

8.28.3.1 computeShapeFunctions() `void mknix::ElemTriangle::computeShapeFunctions ( ) [virtual]`

Computes FEM shape functions at each Gauss point by filling the B matrix.

Reimplemented from [mknix::Cell](#).

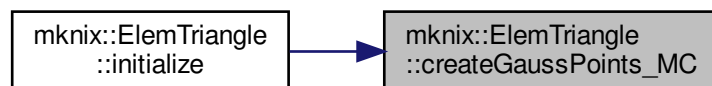
Definition at line 103 of file elemtriangle.cpp.

8.28.3.2 createGaussPoints_MC() `void mknix::ElemTriangle::createGaussPoints_MC ( ) [protected]`

Creates mass-consistent Gauss points (gPoints_MC) with a higher-order quadrature rule than the standard stress Gauss points.

Definition at line 120 of file elemtriangle.cpp.

Here is the caller graph for this function:



8.28.3.3 initialize() `void mknix::ElemTriangle::initialize ( std::vector< Node * > & nodes_in ) [virtual]`

Initializes the FEM triangle by assigning the three corner nodes as support nodes for all Gauss points and creating separate mass-consistent Gauss points.

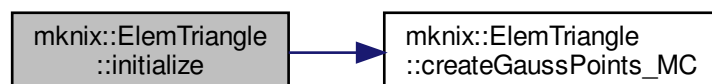
Parameters

<code>nodes_in</code>	Vector of domain nodes (unused directly; corners are added explicitly).
-----------------------	---

Reimplemented from [mknix::Cell](#).

Definition at line 76 of file elemtriangle.cpp.

Here is the call graph for this function:



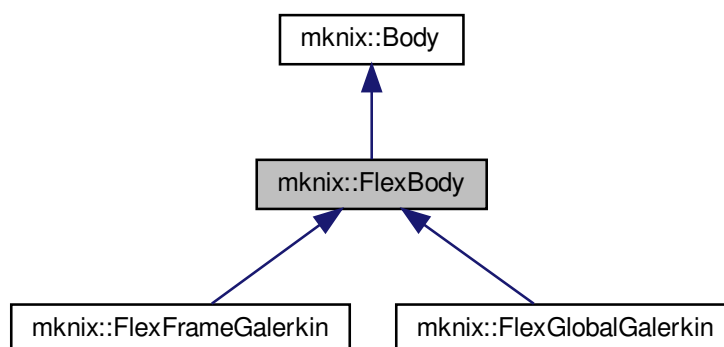
The documentation for this class was generated from the following files:

- [elemtriangle.h](#)
- [elemtriangle.cpp](#)

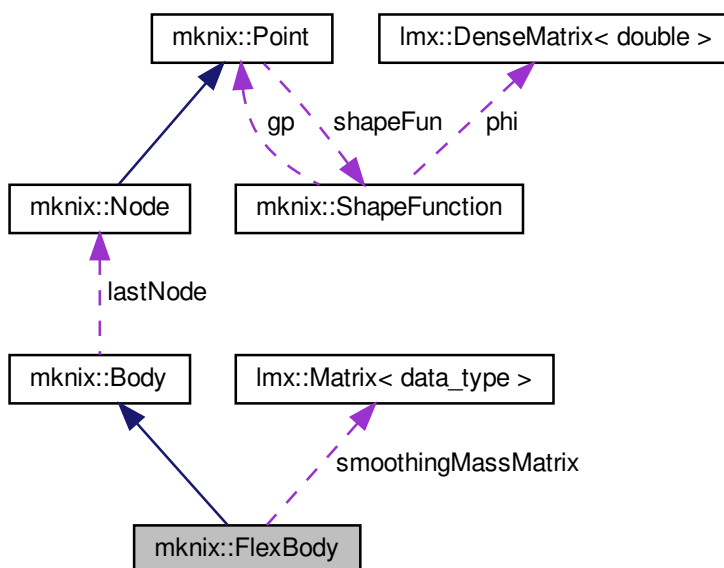
8.29 mknix::FlexBody Class Reference

```
#include <bodyflex.h>
```

Inheritance diagram for mknix::FlexBody:



Collaboration diagram for mknix::FlexBody:



Public Member Functions

- [FlexBody](#) ()

Default constructor. Initializes the flexible body with nonlinear formulation and no stress/energy output.

- `FlexBody` (`std::string`)

Constructor with title.

- virtual `~FlexBody` ()

Destructor.

- virtual void `initialize` () override

Initializes the body, then finds support nodes and solves shape functions for all output and body integration points.

- `Point` * `getBodyPoint` (int `point_number`)
- virtual `Node` * `getNode` (int `node_number`) override
- `Point` * `getLastBodyPoint` ()
- virtual void `setType` (`std::string` `type_in`)=0
- virtual void `setFormulation` (`std::string` `formulation_in`)
- virtual void `calcInternalForces` ()=0
- virtual void `calcTangentMatrix` ()=0
- virtual void `assembleInternalForces` (`Imx::Vector`< `data_type` > &)=0
- virtual void `assembleTangentMatrix` (`Imx::Matrix`< `data_type` > &)=0
- void `addBodyPoint` (`Point` *, `std::string`)

Adds a body integration point, sets its interpolation method (RBF or MLS), and registers it in the list of body points.

- void `addPoint` (`Node` *)

Adds a copy of the given `Node` as a non-integration output point (MLS type).

- void `addPoint` (int, double, double, double, double, double)

Creates a new `Node` output point from coordinates, sets its influence parameters, finds support nodes, and solves its MLS shape function.

- virtual int `getNumberOfPoints` ()
- void `setOutput` (`std::string`) override

Activates a flag for output data at the end of the analysis.

- void `outputToFile` (`std::ofstream` *) override

Streams the data stored during the analysis to a file.

- void `writeBodyInfo` (`std::ofstream` *) override

Writes body mesh/meshfree connectivity information to the output file.

- void `writeBoundaryNodes` (`std::vector`< `Point` * > &) override

Collects all boundary nodes of the body into the provided vector.

- void `writeBoundaryConnectivity` (`std::vector`< `std::vector`< `Point` * > > &) override

Builds the ordered boundary connectivity list by following the `linearBoundary` map and appending the traversal to `connectivity_nodes`.

Protected Attributes

- `std::string` `formulation`
- `std::vector`< `Node` * > `points`
- `std::vector`< `Point` * > `bodyPoints`
- bool `computeStress`
- bool `computeEnergy`
- `Imx::Matrix`< `data_type` > `smoothingMassMatrix`
- `std::vector`< `Imx::Vector`< `data_type` > > `stresses`
- `std::vector`< `Imx::Vector`< `data_type` > * > `energies`

8.29.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 36 of file `bodyflex.h`.

8.29.2 Constructor & Destructor Documentation

8.29.2.1 FlexBody() [1/2] mknix::FlexBody::FlexBody ()

Default constructor. Initializes the flexible body with nonlinear formulation and no stress/energy output.
Definition at line 31 of file bodyflex.cpp.

8.29.2.2 FlexBody() [2/2] mknix::FlexBody::FlexBody (std::string title_in)

Constructor with title.

Parameters

<i>title</i> ↔ _in	Name identifier for this flexible body.
-----------------------	---

Definition at line 44 of file bodyflex.cpp.

8.29.2.3 ~FlexBody() mknix::FlexBody::~FlexBody () [virtual]

Destructor.

Definition at line 55 of file bodyflex.cpp.

8.29.3 Member Function Documentation

8.29.3.1 addBodyPoint() void mknix::FlexBody::addBodyPoint (Point * point_in, std::string method_in)

Adds a body integration point, sets its interpolation method (RBF or MLS), and registers it in the list of body points.

Parameters

<i>point</i> ↔ _in	Pointer to the Point object to add.
<i>method</i> ↔ _in	Interpolation method keyword: "RPIM"/"RBF" or "EFG"/"MLS".

Definition at line 86 of file bodyflex.cpp.

8.29.3.2 addPoint() [1/2] void mknix::FlexBody::addPoint (int nodeNumber, double x, double y, double z, double alpha, double dc)

Creates a new [Node](#) output point from coordinates, sets its influence parameters, finds support nodes, and solves its MLS shape function.

Parameters

<i>nodeNumber</i>	Node identifier.
<i>x</i>	X coordinate.

Parameters

<i>y</i>	Y coordinate.
<i>z</i>	Z coordinate.
<i>alpha</i>	Influence domain scaling factor.
<i>dc</i>	Characteristic length for the influence domain.

Definition at line 132 of file bodyflex.cpp.

8.29.3.3 addPoint() [2/2] `void mknix::FlexBody::addPoint (
Node * node_in)`

Adds a copy of the given [Node](#) as a non-integration output point (MLS type).

Parameters

<i>node_in</i>	Pointer to the source node to copy.
----------------	-------------------------------------

Definition at line 111 of file bodyflex.cpp.

8.29.3.4 assembleInternalForces() `virtual void mknix::FlexBody::assembleInternalForces (
lmx::Vector< data_type > &) [pure virtual]`

Implemented in [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.29.3.5 assembleTangentMatrix() `virtual void mknix::FlexBody::assembleTangentMatrix (
lmx::Matrix< data_type > &) [pure virtual]`

Implemented in [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.29.3.6 calcInternalForces() `virtual void mknix::FlexBody::calcInternalForces ( ) [pure virtual]`

Implemented in [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

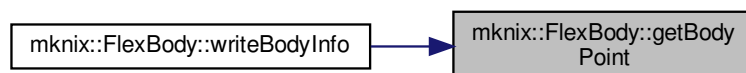
8.29.3.7 calcTangentMatrix() `virtual void mknix::FlexBody::calcTangentMatrix ( ) [pure virtual]`

Implemented in [mknix::FlexGlobalGalerkin](#), and [mknix::FlexFrameGalerkin](#).

8.29.3.8 getBodyPoint() `Point* mknix::FlexBody::getBodyPoint (
int point_number) [inline]`

Definition at line 48 of file bodyflex.h.

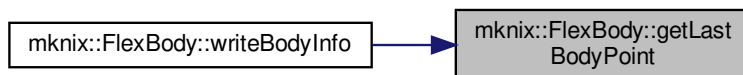
Here is the caller graph for this function:



8.29.3.9 getLastBodyPoint() `Point* mknix::FlexBody::getLastBodyPoint ( ) [inline]`

Definition at line 67 of file bodyflex.h.

Here is the caller graph for this function:

**8.29.3.10 getNode()** `virtual Node* mknix::FlexBody::getNode ( int node_number ) [inline], [override], [virtual]`

Reimplemented from [mknix::Body](#).

Definition at line 55 of file bodyflex.h.

Here is the caller graph for this function:

**8.29.3.11 getNumberOfPoints()** `virtual int mknix::FlexBody::getNumberOfPoints ( ) [inline], [virtual]`

Definition at line 96 of file bodyflex.h.

8.29.3.12 initialize() `void mknix::FlexBody::initialize ( ) [override], [virtual]`

Initializes the body, then finds support nodes and solves shape functions for all output and body integration points.

Reimplemented from [mknix::Body](#).

Definition at line 63 of file bodyflex.cpp.

8.29.3.13 outputToFile() `void mknix::FlexBody::outputToFile ( std::ofstream * outFile ) [override], [virtual]`

Streams the data stored during the analysis to a file.

Parameters

<code>outFile</code>	Output files
----------------------	--------------

Returns

`void`

Reimplemented from [mknix::Body](#).

Definition at line 174 of file bodyflex.cpp.
Here is the call graph for this function:



8.29.3.14 setFormulation() `virtual void mknix::FlexBody::setFormulation ( std::string formulation_in ) [inline], [virtual]`

Definition at line 74 of file bodyflex.h.

8.29.3.15 setOutput() `void mknix::FlexBody::setOutput ( std::string outputType_in ) [override], [virtual]`

Activates a flag for output data at the end of the analysis.

See also

[outputToFile\(\)](#)

[outputStep\(\)](#)

Parameters

<i>outputType_in</i>	Keyword of the flag. Options are: [STRESS, ENERGY]
----------------------	--

Returns

void

Implements [mnix::Body](#).

Definition at line 155 of file bodyflex.cpp.

8.29.3.16 setType() `virtual void mknix::FlexBody::setType ( std::string type_in ) [pure virtual]`

Implemented in [mnix::FlexGlobalGalerkin](#), and [mnix::FlexFrameGalerkin](#).

8.29.3.17 writeBodyInfo() `void mknix::FlexBody::writeBodyInfo ( std::ofstream * outFile ) [override], [virtual]`

Writes body mesh/meshfree connectivity information to the output file.

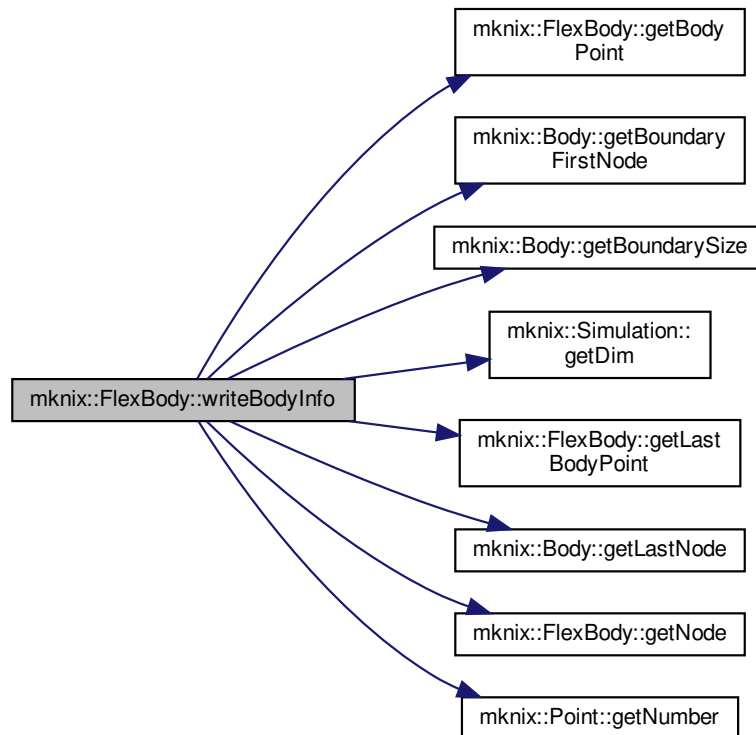
Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Implements [mnix::Body](#).

Definition at line 211 of file bodyflex.cpp.

Here is the call graph for this function:



8.29.3.18 writeBoundaryConnectivity() `void mknix::FlexBody::writeBoundaryConnectivity ( std::vector< std::vector< Point * >> & ) [override], [virtual]`

Builds the ordered boundary connectivity list by following the linearBoundary map and appending the traversal to connectivity_nodes.

Parameters

<i>connectivity_nodes</i>	Vector of node chains to which the boundary loop is appended.
---------------------------	---

Implements [mknix::Body](#).

Definition at line 316 of file bodyflex.cpp.

8.29.3.19 writeBoundaryNodes() `void mknix::FlexBody::writeBoundaryNodes ( std::vector< Point * > & boundary_nodes ) [override], [virtual]`

Collects all boundary nodes of the body into the provided vector.

Parameters

<i>boundary_nodes</i>	Vector to which the boundary node pointers are appended.
-----------------------	--

Implements [mknix::Body](#).

Definition at line 303 of file bodyflex.cpp.

8.29.4 Member Data Documentation

8.29.4.1 **bodyPoints** `std::vector<Point*> mknix::FlexBody::bodyPoints [protected]`

Points to define integration domain

Definition at line 121 of file bodyflex.h.

8.29.4.2 **computeEnergy** `bool mknix::FlexBody::computeEnergy [protected]`

Definition at line 124 of file bodyflex.h.

8.29.4.3 **computeStress** `bool mknix::FlexBody::computeStress [protected]`

Definition at line 123 of file bodyflex.h.

8.29.4.4 **energies** `std::vector<lmx::Vector<data_type>*> mknix::FlexBody::energies [protected]`

Definition at line 128 of file bodyflex.h.

8.29.4.5 **formulation** `std::string mknix::FlexBody::formulation [protected]`

Definition at line 118 of file bodyflex.h.

8.29.4.6 **points** `std::vector<Node*> mknix::FlexBody::points [protected]`

Additional points to define loads or constraints

Definition at line 119 of file bodyflex.h.

8.29.4.7 **smoothingMassMatrix** `lmx::Matrix<data_type> mknix::FlexBody::smoothingMassMatrix [protected]`

Definition at line 126 of file bodyflex.h.

8.29.4.8 **stresses** `std::vector<lmx::Vector<data_type> > mknix::FlexBody::stresses [protected]`

Definition at line 127 of file bodyflex.h.

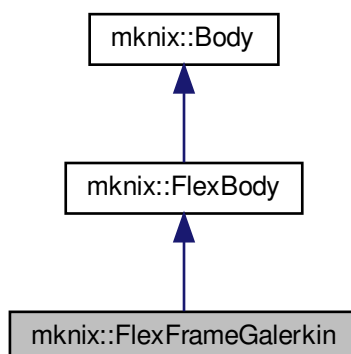
The documentation for this class was generated from the following files:

- [bodyflex.h](#)
- [bodyflex.cpp](#)

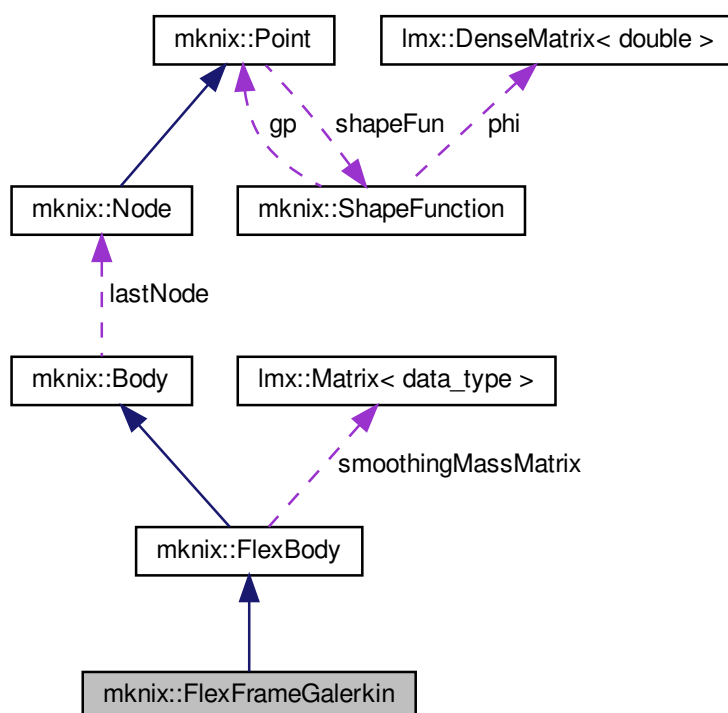
8.30 mknix::FlexFrameGalerkin Class Reference

```
#include <bodyflexframegalerkin.h>
```

Inheritance diagram for mknix::FlexFrameGalerkin:



Collaboration diagram for mknix::FlexFrameGalerkin:



Public Member Functions

- [FlexFrameGalerkin](#) ()
Default constructor.

- [FlexFrameGalerkin](#) (std::string)
Constructor with title.
- [~FlexFrameGalerkin](#) ()
Destructor.
- std::string [getType](#) ()
- void [setType](#) (std::string type_in)
- void [calcMassMatrix](#) ()
Computes the local mass matrix for all cells using Gauss-point integration.
- void [calcInternalForces](#) ()
- void [calcExternalForces](#) ()
Computes the local external force vector for all cells using Gauss-point integration.
- void [calcTangentMatrix](#) ()
- void [assembleMassMatrix](#) (Imx::Matrix< data_type > &)
Assembles the local mass matrix into the global mass matrix.
- void [assembleInternalForces](#) (Imx::Vector< data_type > &)
Assembles the local internal forces into the global internal force vector.
- void [assembleExternalForces](#) (Imx::Vector< data_type > &)
Assembles the local external forces into the global external force vector.
- void [assembleTangentMatrix](#) (Imx::Matrix< data_type > &)
Assembles the local tangent stiffness matrix into the global tangent matrix.
- void [outputStep](#) (const Imx::Vector< data_type > &, const Imx::Vector< data_type > &)
Postprocess and store step results for dynamic analysis.
- void [outputStep](#) (const Imx::Vector< data_type > &)
Postprocess and store step results for static analysis.

Additional Inherited Members

8.30.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file bodyflexframegalerkin.h.

8.30.2 Constructor & Destructor Documentation

8.30.2.1 FlexFrameGalerkin() [1/2] `mknix::FlexFrameGalerkin::FlexFrameGalerkin ( )`

Default constructor.

Definition at line 31 of file bodyflexframegalerkin.cpp.

8.30.2.2 FlexFrameGalerkin() [2/2] `mknix::FlexFrameGalerkin::FlexFrameGalerkin ( std::string title_in )`

Constructor with title.

Parameters

<i>title</i> <i>_in</i>	Name identifier for this body.
----------------------------	--------------------------------

Definition at line 41 of file bodyflexframegalerkin.cpp.

8.30.2.3 ~FlexFrameGalerkin() mknix::FlexFrameGalerkin::~~FlexFrameGalerkin ()

Destructor.

Definition at line 50 of file bodyflexframegalerkin.cpp.

8.30.3 Member Function Documentation**8.30.3.1 assembleExternalForces()** void mknix::FlexFrameGalerkin::assembleExternalForces (
 lmx::Vector< data_type > & globalExternalForces) [virtual]

Assembles the local external forces into the global external force vector.

Parameters

<i>globalExternalForces</i>	Reference to the global external force vector.
-----------------------------	--

Implements [mknix::Body](#).

Definition at line 195 of file bodyflexframegalerkin.cpp.

8.30.3.2 assembleInternalForces() void mknix::FlexFrameGalerkin::assembleInternalForces (
 lmx::Vector< data_type > & globalInternalForces) [virtual]

Assembles the local internal forces into the global internal force vector.

Parameters

<i>globalInternalForces</i>	Reference to the global internal force vector.
-----------------------------	--

Implements [mknix::FlexBody](#).

Definition at line 178 of file bodyflexframegalerkin.cpp.

8.30.3.3 assembleMassMatrix() void mknix::FlexFrameGalerkin::assembleMassMatrix (
 lmx::Matrix< data_type > & globalMass) [virtual]

Assembles the local mass matrix into the global mass matrix.

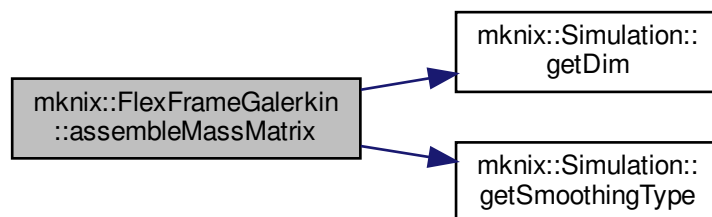
Parameters

<i>globalMass</i>	Reference to the global mass matrix.
-------------------	--------------------------------------

Implements [mknix::Body](#).

Definition at line 142 of file bodyflexframegalerkin.cpp.

Here is the call graph for this function:



8.30.3.4 assembleTangentMatrix() `void mknix::FlexFrameGalerkin::assembleTangentMatrix (
 lmx::Matrix< data_type > & globalTangent) [virtual]`

Assembles the local tangent stiffness matrix into the global tangent matrix.

Parameters

<i>globalTangent</i>	Reference to the global tangent matrix.
----------------------	---

Implements [mknix::FlexBody](#).

Definition at line 212 of file `bodyflexframegalerkin.cpp`.

8.30.3.5 calcExternalForces() `void mknix::FlexFrameGalerkin::calcExternalForces ( ) [virtual]`

Computes the local external force vector for all cells using Gauss-point integration.

Implements [mknix::Body](#).

Definition at line 99 of file `bodyflexframegalerkin.cpp`.

8.30.3.6 calcInternalForces() `void mknix::FlexFrameGalerkin::calcInternalForces ( ) [virtual]`

Implements [mknix::FlexBody](#).

Definition at line 70 of file `bodyflexframegalerkin.cpp`.

8.30.3.7 calcMassMatrix() `void mknix::FlexFrameGalerkin::calcMassMatrix ( ) [virtual]`

Computes the local mass matrix for all cells using Gauss-point integration.

Implements [mknix::Body](#).

Definition at line 57 of file `bodyflexframegalerkin.cpp`.

8.30.3.8 calcTangentMatrix() `void mknix::FlexFrameGalerkin::calcTangentMatrix ( ) [virtual]`

Implements [mknix::FlexBody](#).

Definition at line 112 of file `bodyflexframegalerkin.cpp`.

8.30.3.9 getType() `std::string mknix::FlexFrameGalerkin::getType ( ) [inline], [virtual]`

Implements [mknix::Body](#).

Definition at line 40 of file `bodyflexframegalerkin.h`.

8.30.3.10 outputStep() [1/2] `void mknix::FlexFrameGalerkin::outputStep (`
`const lmx::Vector< data_type > & q ) [virtual]`

Postprocess and store step results for static analysis.

Parameters

q	Global configuration vector
-----	-----------------------------

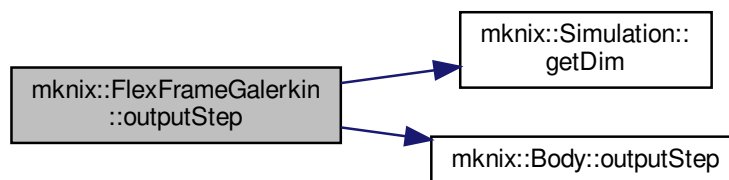
Returns

void

Implements [mknix::Body](#).

Definition at line 295 of file bodyflexframegalerkin.cpp.

Here is the call graph for this function:



8.30.3.11 outputStep() [2/2] `void mknix::FlexFrameGalerkin::outputStep (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot ) [virtual]`

Postprocess and store step results for dynamic analysis.

Parameters

q	Global configuration vector
$qdot$	Global configuration first derivative vector

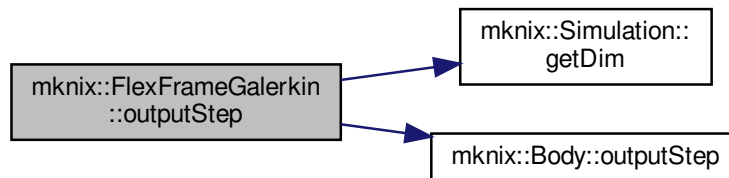
Returns

void

Implements [mknix::Body](#).

Definition at line 231 of file bodyflexframegalerkin.cpp.

Here is the call graph for this function:



8.30.3.12 setType() `void mknix::FlexFrameGalerkin::setType ( std::string type_in ) [inline], [virtual]`

Implements [mknix::FlexBody](#).

Definition at line 45 of file bodyflexframegalerkin.h.

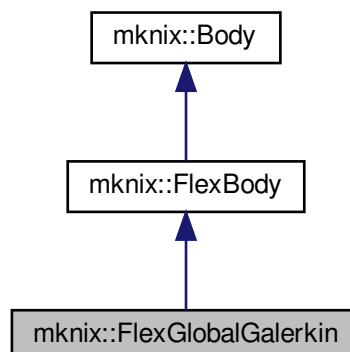
The documentation for this class was generated from the following files:

- [bodyflexframegalerkin.h](#)
- [bodyflexframegalerkin.cpp](#)

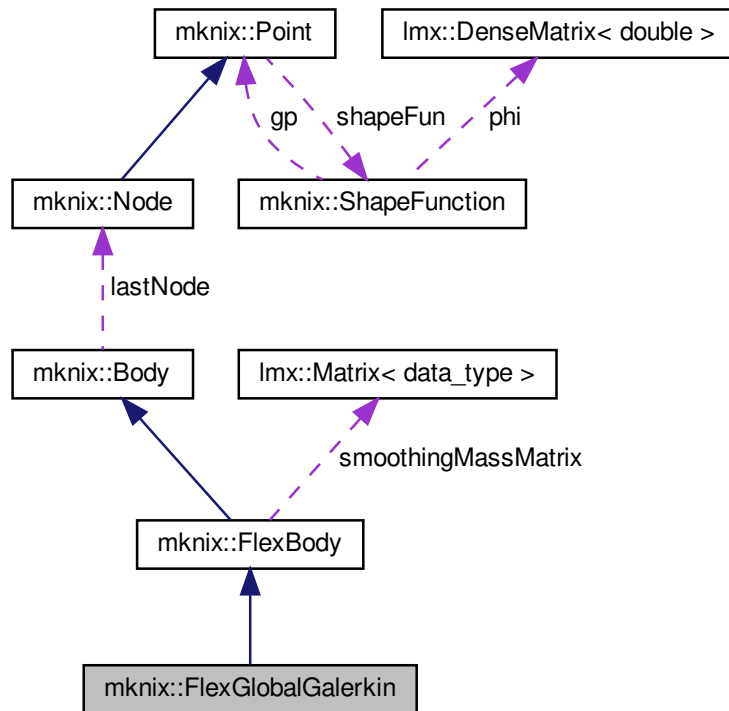
8.31 mknix::FlexGlobalGalerkin Class Reference

```
#include <bodyflexglobalgalerkin.h>
```

Inheritance diagram for `mknix::FlexGlobalGalerkin`:



Collaboration diagram for mknix::FlexGlobalGalerkin:



Public Member Functions

- [FlexGlobalGalerkin](#) ()
Default constructor.
- [FlexGlobalGalerkin](#) (std::string)
Constructor with title.
- [~FlexGlobalGalerkin](#) ()
Destructor.
- std::string [getType](#) ()
- void [setType](#) (std::string type_in)
- void [calcMassMatrix](#) ()
Computes the local mass and external force matrices for all cells. External forces are computed here because they depend on mass and gravity.
- void [calcInternalForces](#) ()
Computes the local internal force vector for all cells (linear or nonlinear formulation).
- void [calcExternalForces](#) ()
No-op in GlobalGalerkin formulation; external forces are computed in [calcMassMatrix\(\)](#).
- void [calcTangentMatrix](#) ()
Computes the local tangent stiffness matrix for all cells (linear or nonlinear formulation).
- void [assembleMassMatrix](#) (Imx::Matrix< data_type > &)
Assembles the local mass matrix into the global mass matrix.
- void [assembleInternalForces](#) (Imx::Vector< data_type > &)
Assembles the local internal forces into the global internal force vector.

- void `assembleExternalForces` (`lmx::Vector< data_type > &`)
Assembles the local external forces into the global external force vector.
- void `assembleTangentMatrix` (`lmx::Matrix< data_type > &`)
Assembles the local tangent stiffness matrix into the global tangent matrix.
- void `outputStep` (`const lmx::Vector< data_type > &`, `const lmx::Vector< data_type > &`)
Postprocess and store step results for dynamic analysis.
- void `outputStep` (`const lmx::Vector< data_type > &`)
Postprocess and store step results for static analysis.

Additional Inherited Members

8.31.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `bodyflexglobalgalerkin.h`.

8.31.2 Constructor & Destructor Documentation

8.31.2.1 `FlexGlobalGalerkin()` [1/2] `mknix::FlexGlobalGalerkin::FlexGlobalGalerkin ( )`

Default constructor.

Definition at line 31 of file `bodyflexglobalgalerkin.cpp`.

8.31.2.2 `FlexGlobalGalerkin()` [2/2] `mknix::FlexGlobalGalerkin::FlexGlobalGalerkin ( std::string title_in )`

Constructor with title.

Parameters

<code>title_in</code>	Name identifier for this body.
-----------------------	--------------------------------

Definition at line 41 of file `bodyflexglobalgalerkin.cpp`.

8.31.2.3 `~FlexGlobalGalerkin()` `mknix::FlexGlobalGalerkin::~~FlexGlobalGalerkin ( )`

Destructor.

Definition at line 50 of file `bodyflexglobalgalerkin.cpp`.

8.31.3 Member Function Documentation

8.31.3.1 `assembleExternalForces()` `void mknix::FlexGlobalGalerkin::assembleExternalForces ( lmx::Vector< data_type > & globalExternalForces ) [virtual]`

Assembles the local external forces into the global external force vector.

Parameters

<code>globalExternalForces</code>	Reference to the global external force vector.
-----------------------------------	--

Implements `mknix::Body`.

Definition at line 187 of file bodyflexglobalgalerkin.cpp.

8.31.3.2 assembleInternalForces() `void mknix::FlexGlobalGalerkin::assembleInternalForces (
 lmx::Vector< data_type > & globalInternalForces) [virtual]`

Assembles the local internal forces into the global internal force vector.

Parameters

<i>globalInternalForces</i>	Reference to the global internal force vector.
-----------------------------	--

Implements [mknix::FlexBody](#).

Definition at line 172 of file bodyflexglobalgalerkin.cpp.

8.31.3.3 assembleMassMatrix() `void mknix::FlexGlobalGalerkin::assembleMassMatrix (
 lmx::Matrix< data_type > & globalMass) [virtual]`

Assembles the local mass matrix into the global mass matrix.

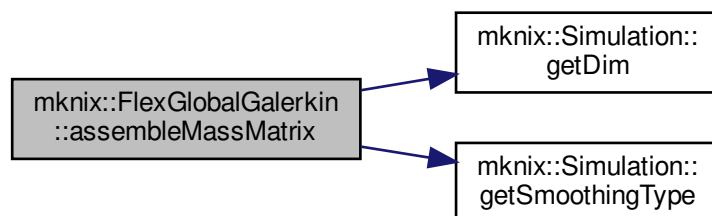
Parameters

<i>globalMass</i>	Reference to the global mass matrix.
-------------------	--------------------------------------

Implements [mknix::Body](#).

Definition at line 138 of file bodyflexglobalgalerkin.cpp.

Here is the call graph for this function:



8.31.3.4 assembleTangentMatrix() `void mknix::FlexGlobalGalerkin::assembleTangentMatrix (
 lmx::Matrix< data_type > & globalTangent) [virtual]`

Assembles the local tangent stiffness matrix into the global tangent matrix.

Parameters

<i>globalTangent</i>	Reference to the global tangent matrix.
----------------------	---

Implements [mknix::FlexBody](#).

Definition at line 202 of file bodyflexglobalgalerkin.cpp.

8.31.3.5 calcExternalForces() `void mknix::FlexGlobalGalerkin::calcExternalForces ( ) [virtual]`
 No-op in GlobalGalerkin formulation; external forces are computed in [calcMassMatrix\(\)](#).
 Implements [mknix::Body](#).
 Definition at line 98 of file `bodyflexglobalgalerkin.cpp`.

8.31.3.6 calcInternalForces() `void mknix::FlexGlobalGalerkin::calcInternalForces ( ) [virtual]`
 Computes the local internal force vector for all cells (linear or nonlinear formulation).
 Implements [mknix::FlexBody](#).
 Definition at line 73 of file `bodyflexglobalgalerkin.cpp`.

8.31.3.7 calcMassMatrix() `void mknix::FlexGlobalGalerkin::calcMassMatrix ( ) [virtual]`
 Computes the local mass and external force matrices for all cells. External forces are computed here because they depend on mass and gravity.
 Implements [mknix::Body](#).
 Definition at line 58 of file `bodyflexglobalgalerkin.cpp`.

8.31.3.8 calcTangentMatrix() `void mknix::FlexGlobalGalerkin::calcTangentMatrix ( ) [virtual]`
 Computes the local tangent stiffness matrix for all cells (linear or nonlinear formulation).
 Implements [mknix::FlexBody](#).
 Definition at line 112 of file `bodyflexglobalgalerkin.cpp`.

8.31.3.9 getType() `std::string mknix::FlexGlobalGalerkin::getType ( ) [inline], [virtual]`
 Implements [mknix::Body](#).
 Definition at line 40 of file `bodyflexglobalgalerkin.h`.

8.31.3.10 outputStep() [1/2] `void mknix::FlexGlobalGalerkin::outputStep (`
 `const lmx::Vector< data_type > & q ) [virtual]`
 Postprocess and store step results for static analysis.

Parameters

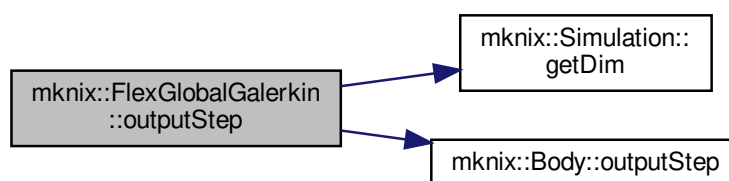
<i>q</i>	Global configuration vector
----------	-----------------------------

Returns

void

Implements [mknix::Body](#).
 Definition at line 277 of file `bodyflexglobalgalerkin.cpp`.

Here is the call graph for this function:



8.31.3.11 outputStep() [2/2] `void mknix::FlexGlobalGalerkin::outputStep (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot ) [virtual]`

Postprocess and store step results for dynamic analysis.

Parameters

<i>q</i>	Global configuration vector
<i>qdot</i>	Global configuration first derivative vector

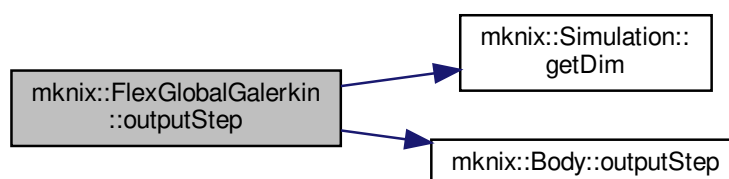
Returns

void

Implements [mknix::Body](#).

Definition at line 219 of file `bodyflexglobalgalerkin.cpp`.

Here is the call graph for this function:



8.31.3.12 setType() `void mknix::FlexGlobalGalerkin::setType (`
`std::string type_in ) [inline], [virtual]`

Implements [mknix::FlexBody](#).

Definition at line 45 of file `bodyflexglobalgalerkin.h`.

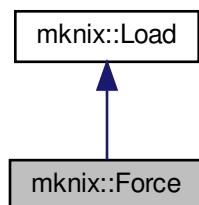
The documentation for this class was generated from the following files:

- [bodyflexglobalgalerkin.h](#)
- [bodyflexglobalgalerkin.cpp](#)

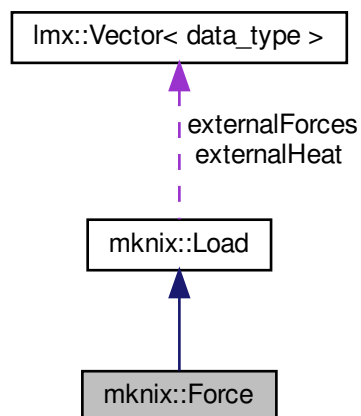
8.32 mknix::Force Class Reference

```
#include <force.h>
```

Inheritance diagram for mknix::Force:



Collaboration diagram for mknix::Force:



Public Member Functions

- [Force](#) ()
Default constructor.
- [Force](#) ([Node](#) *, double, double, double)
Constructor that creates a concentrated force applied to a single node.
- [~Force](#) ()
Destructor.
- virtual void [outputToFile](#) (std::ofstream *)
Writes force data to the output file (currently a no-op).

Additional Inherited Members

8.32.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file force.h.

8.32.2 Constructor & Destructor Documentation

8.32.2.1 Force() [1/2] mknix::Force::Force ()

Default constructor.

Definition at line 31 of file force.cpp.

8.32.2.2 Force() [2/2] mknix::Force::Force (

```
Node * node_in,
double fx_in,
double fy_in,
double fz_in )
```

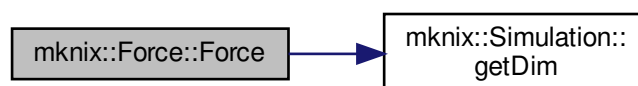
Constructor that creates a concentrated force applied to a single node.

Parameters

<i>node_in</i>	Pointer to the node where the force is applied.
<i>fx_in</i>	Force component along X.
<i>fy_in</i>	Force component along Y.
<i>fz_in</i>	Force component along Z (used in 3D only).

Definition at line 43 of file force.cpp.

Here is the call graph for this function:



8.32.2.3 ~Force() mknix::Force::~~Force ()

Destructor.

Definition at line 58 of file force.cpp.

8.32.3 Member Function Documentation

8.32.3.1 outputToFile() void mknix::Force::outputToFile (std::ofstream * outFile) [virtual]

Writes force data to the output file (currently a no-op).

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Implements [mknix::Load](#).

Definition at line 66 of file `force.cpp`.

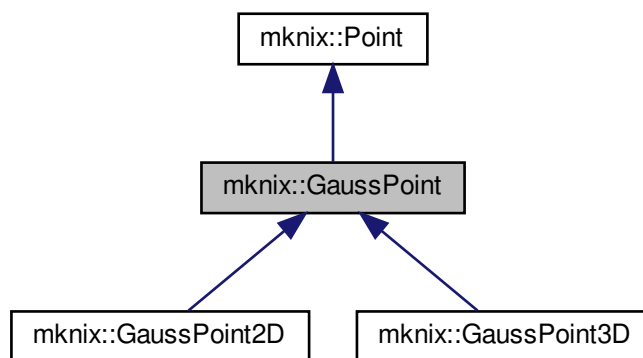
The documentation for this class was generated from the following files:

- [force.h](#)
- [force.cpp](#)

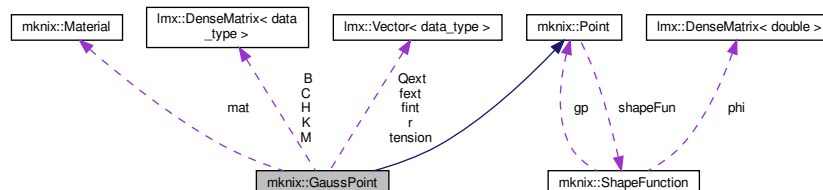
8.33 mknix::GaussPoint Class Reference

```
#include <gausspoint.h>
```

Inheritance diagram for `mknix::GaussPoint`:



Collaboration diagram for `mknix::GaussPoint`:



Public Member Functions

- [GaussPoint](#) ()
Default constructor for [GaussPoint](#).
- [GaussPoint](#) (int dim_in, double alpha_in, double weight_in, double jacobian_in, [Material](#) *mat_in, int num, double coor_x, double coor_y, double dc_in, bool)
Constructs a 2D Gauss point with coordinates (x, y).
- [GaussPoint](#) (int dim_in, double alpha_in, double weight_in, double jacobian_in, [Material](#) *mat_in, int num, double coor_x, double coor_y, double coor_z, double dc_in, bool)

- virtual `~GaussPoint ()`
Constructs a 3D Gauss point with coordinates (x, y, z).
- virtual void `shapeFunSolve (std::string, double)` override
Computes and stores shape function values at this point using the specified meshfree method.
- virtual void `fillFEmatrices ()=0`
- void `setMaterial (Material &mat_in)`
- void `computeCij ()`
Computes the local thermal capacity (heat capacity) matrix contribution C at this Gauss point.
- void `computeHij ()`
Computes the local thermal conductivity matrix contribution H at this Gauss point.
- void `computeQext (std::vector< LoadThermalBody * >)`
Computes the external thermal load vector contribution Qext at this Gauss point, including volumetric heat sources and porosity cooling.
- virtual void `computeFint ()=0`
- virtual void `computeFext ()=0`
- virtual void `computeMij ()=0`
- virtual void `computeKij ()=0`
- virtual void `computeStress ()=0`
- virtual void `computeNLStress ()=0`
- virtual void `computeNLFint ()=0`
- virtual void `computeNLKij ()=0`
- void `assembleCij (Imx::Matrix< data_type > &)`
Assembles the local capacity matrix C into the global thermal capacity matrix.
- void `assembleHij (Imx::Matrix< data_type > &)`
Assembles the local conductivity matrix H into the global conductivity matrix.
- void `assembleQext (Imx::Vector< data_type > &)`
Assembles the local external heat vector Qext into the global heat load vector.
- virtual void `assembleMij (Imx::Matrix< data_type > &)=0`
- virtual void `assembleKij (Imx::Matrix< data_type > &)=0`
- virtual void `assembleRi (Imx::Vector< data_type > &, int)=0`
- virtual void `assembleFint (Imx::Vector< data_type > &)=0`
- virtual void `assembleFext (Imx::Vector< data_type > &)=0`
- virtual double `calcPotentialE (const Imx::Vector< data_type > &)=0`
- virtual double `calcKineticE (const Imx::Vector< data_type > &)=0`
- virtual double `calcElasticE ()=0`
- void `gnuplotOutStress (std::ofstream &)`
Outputs the Gauss point position and stress value to a gnuplot-compatible file.

Protected Attributes

- int `num`
- double `weight`
- `Material * mat`
- bool `stressPoint`
- double `avgTemp`
- `Imx::DenseMatrix< data_type > B`
- `Imx::DenseMatrix< data_type > C`
- `Imx::DenseMatrix< data_type > H`
- `Imx::DenseMatrix< data_type > M`
- `Imx::DenseMatrix< data_type > K`
- `Imx::Vector< data_type > tension`
- `Imx::Vector< data_type > r`
- `Imx::Vector< data_type > Qext`
- `Imx::Vector< data_type > fint`
- `Imx::Vector< data_type > fext`

8.33.1 Detailed Description

Author

Daniel Iglesias

Definition at line 46 of file gausspoint.h.

8.33.2 Constructor & Destructor Documentation

8.33.2.1 GaussPoint() [1/3] `mkknix::GaussPoint::GaussPoint ( )`

Default constructor for [GaussPoint](#).

Definition at line 17 of file gausspoint.cpp.

8.33.2.2 GaussPoint() [2/3] `mkknix::GaussPoint::GaussPoint (`

```
int dim_in,
double alpha_in,
double weight_in,
double jacobian_in,
Material * mat_in,
int num_in,
double coor_x,
double coor_y,
double dc_in,
bool stressPoint_in )
```

Constructs a 2D Gauss point with coordinates (x, y).

Parameters

<i>dim_in</i>	Spatial dimension.
<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian of the mapping to the reference domain.
<i>mat_in</i>	Pointer to the material at this point.
<i>num_in</i>	Index of this Gauss point within its cell.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>dc_in</i>	Characteristic nodal spacing.
<i>stressPoint_in</i>	True if this point is used for stress smoothing.

Definition at line 35 of file gausspoint.cpp.

8.33.2.3 GaussPoint() [3/3] `mkknix::GaussPoint::GaussPoint (`

```
int dim_in,
double alpha_in,
double weight_in,
double jacobian_in,
Material * mat_in,
int num_in,
double coor_x,
double coor_y,
double coor_z,
```

```
double dc_in,
bool stressPoint_in )
```

Constructs a 3D Gauss point with coordinates (x, y, z).

Parameters

<i>dim_in</i>	Spatial dimension.
<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian of the mapping.
<i>mat_in</i>	Pointer to the material.
<i>num_in</i>	Index of this point within its cell.
<i>coord_x</i>	X coordinate.
<i>coord_y</i>	Y coordinate.
<i>coord_z</i>	Z coordinate.
<i>dc_in</i>	Characteristic nodal spacing.
<i>stressPoint_in</i>	True if this point is used for stress smoothing.

Definition at line 67 of file gausspoint.cpp.

8.33.2.4 ~GaussPoint() mknix::GaussPoint::~~GaussPoint () [virtual]

Destructor for [GaussPoint](#).

Definition at line 89 of file gausspoint.cpp.

8.33.3 Member Function Documentation

8.33.3.1 assembleCij() void mknix::GaussPoint::assembleCij (lmx::Matrix< data_type > & globalCapacity)

Assembles the local capacity matrix C into the global thermal capacity matrix.

Parameters

<i>globalCapacity</i>	Global thermal capacity matrix to be updated.
-----------------------	---

Definition at line 289 of file gausspoint.cpp.

8.33.3.2 assembleFext() virtual void mknix::GaussPoint::assembleFext (lmx::Vector< data_type > &) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.3 assembleFint() virtual void mknix::GaussPoint::assembleFint (lmx::Vector< data_type > &) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.4 assembleHij() void mknix::GaussPoint::assembleHij (lmx::Matrix< data_type > & globalConductivity)

Assembles the local conductivity matrix H into the global conductivity matrix.

Parameters

<i>globalConductivity</i>	Global conductivity matrix to be updated.
---------------------------	---

Definition at line 308 of file gausspoint.cpp.

8.33.3.5 assembleKij() virtual void mknix::GaussPoint::assembleKij (
 `lmx::Matrix< data_type > &`) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.6 assembleMij() virtual void mknix::GaussPoint::assembleMij (
 `lmx::Matrix< data_type > &`) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.7 assembleQext() void mknix::GaussPoint::assembleQext (
 `lmx::Vector< data_type > & globalHeat`)

Assembles the local external heat vector Qext into the global heat load vector.

Parameters

<i>globalHeat</i>	Global external heat load vector to be updated.
-------------------	---

Definition at line 327 of file gausspoint.cpp.

8.33.3.8 assembleRi() virtual void mknix::GaussPoint::assembleRi (
 `lmx::Vector< data_type > &` ,
 `int`) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.9 calcElasticE() virtual double mknix::GaussPoint::calcElasticE () [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.10 calcKineticE() virtual double mknix::GaussPoint::calcKineticE (
 `const lmx::Vector< data_type > &`) [pure virtual]

Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.11 calcPotentialE() virtual double mknix::GaussPoint::calcPotentialE (
 `const lmx::Vector< data_type > &`) [pure virtual]

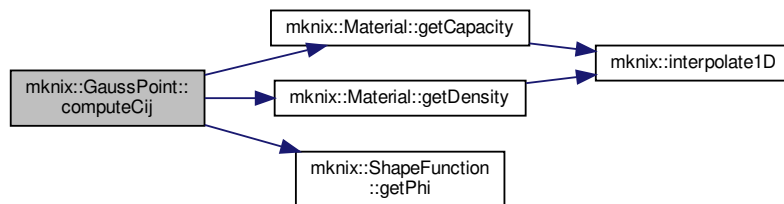
Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.12 computeCij() void mknix::GaussPoint::computeCij ()

Computes the local thermal capacity (heat capacity) matrix contribution C at this Gauss point.

Definition at line 141 of file gausspoint.cpp.

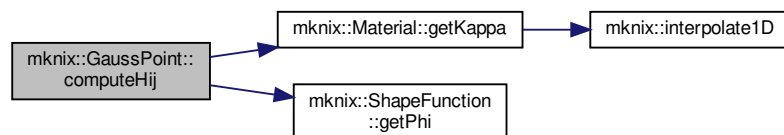
Here is the call graph for this function:



8.33.3.13 computeFext() `virtual void mknix::GaussPoint::computeFext ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.14 computeFint() `virtual void mknix::GaussPoint::computeFint ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.15 computeHij() `void mknix::GaussPoint::computeHij ( )`
 Computes the local thermal conductivity matrix contribution H at this Gauss point.
 Definition at line 196 of file gausspoint.cpp.
 Here is the call graph for this function:



8.33.3.16 computeKij() `virtual void mknix::GaussPoint::computeKij ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.17 computeMij() `virtual void mknix::GaussPoint::computeMij ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.18 computeNLFint() `virtual void mknix::GaussPoint::computeNLFint ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.19 computeNLKij() `virtual void mknix::GaussPoint::computeNLKij ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.20 computeNLStress() `virtual void mknix::GaussPoint::computeNLStress ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.21 computeQext() `void mknix::GaussPoint::computeQext (
 std::vector< LoadThermalBody * > loadThermalBody_in)`

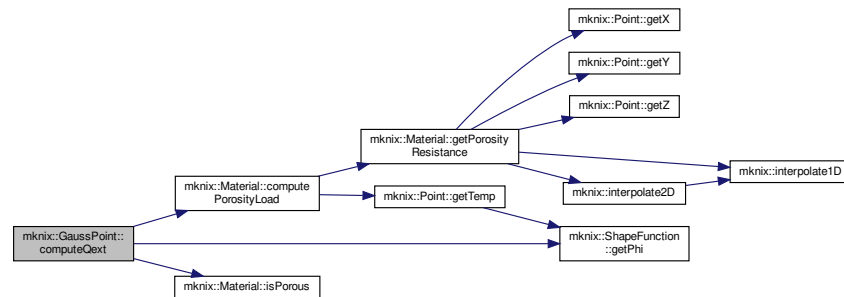
Computes the external thermal load vector contribution Qext at this Gauss point, including volumetric heat sources and porosity cooling.

Parameters

<code>loadThermalBody_↵ _in</code>	Vector of body thermal load objects.
--	--------------------------------------

Definition at line 259 of file gausspoint.cpp.

Here is the call graph for this function:



8.33.3.22 computeStress() `virtual void mknix::GaussPoint::computeStress ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.23 fillFEmatrices() `virtual void mknix::GaussPoint::fillFEmatrices ( ) [pure virtual]`
 Implemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

8.33.3.24 gnuplotOutStress() `void mknix::GaussPoint::gnuplotOutStress (
 std::ofstream & gptension)`

Outputs the Gauss point position and stress value to a gnuplot-compatible file.

Parameters

<code>gptension</code>	Output file stream for stress data.
------------------------	-------------------------------------

Definition at line 342 of file gausspoint.cpp.

8.33.3.25 setMaterial() `void mknix::GaussPoint::setMaterial (
Material & mat_in) [inline]`

Definition at line 66 of file gausspoint.h.

8.33.3.26 shapeFunSolve() `void mknix::GaussPoint::shapeFunSolve (`
`std::string type_in,`
`double q_in ) [override], [virtual]`

Computes and stores shape function values at this point using the specified meshfree method.

Parameters

<i>type_in</i>	Shape function type: "RBF" for radial basis functions or "MLS" for moving least squares.
<i>q_in</i>	Shape parameter (overridden internally to 0.5).

Reimplemented from [mknix::Point](#).

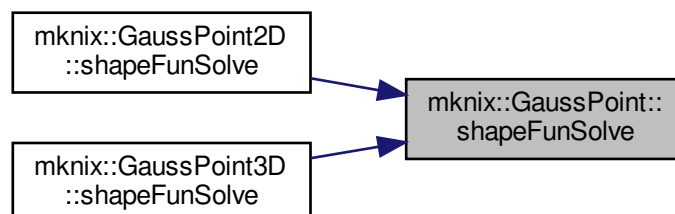
Reimplemented in [mknix::GaussPoint3D](#), and [mknix::GaussPoint2D](#).

Definition at line 99 of file gausspoint.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.33.4 Member Data Documentation

8.33.4.1 avgTemp `double mknix::GaussPoint::avgTemp [protected]`
 Definition at line 122 of file gausspoint.h.

8.33.4.2 B `lmx::DenseMatrix<data_type> mknix::GaussPoint::B [protected]`
 Definition at line 124 of file gausspoint.h.

8.33.4.3 C `lmx::DenseMatrix<data_type> mknix::GaussPoint::C` [protected]
Definition at line 125 of file gausspoint.h.

8.33.4.4 fext `lmx::Vector<data_type> mknix::GaussPoint::fext` [protected]
Definition at line 135 of file gausspoint.h.

8.33.4.5 fint `lmx::Vector<data_type> mknix::GaussPoint::fint` [protected]
Definition at line 134 of file gausspoint.h.

8.33.4.6 H `lmx::DenseMatrix<data_type> mknix::GaussPoint::H` [protected]
Definition at line 126 of file gausspoint.h.

8.33.4.7 K `lmx::DenseMatrix<data_type> mknix::GaussPoint::K` [protected]
Definition at line 128 of file gausspoint.h.

8.33.4.8 M `lmx::DenseMatrix<data_type> mknix::GaussPoint::M` [protected]
Definition at line 127 of file gausspoint.h.

8.33.4.9 mat `Material* mknix::GaussPoint::mat` [protected]
Definition at line 120 of file gausspoint.h.

8.33.4.10 num `int mknix::GaussPoint::num` [protected]
Definition at line 118 of file gausspoint.h.

8.33.4.11 Qext `lmx::Vector<data_type> mknix::GaussPoint::Qext` [protected]
Definition at line 133 of file gausspoint.h.

8.33.4.12 r `lmx::Vector<data_type> mknix::GaussPoint::r` [protected]
Definition at line 131 of file gausspoint.h.

8.33.4.13 stressPoint `bool mknix::GaussPoint::stressPoint` [protected]
Definition at line 121 of file gausspoint.h.

8.33.4.14 tension `lmx::Vector<data_type> mknix::GaussPoint::tension` [protected]
Definition at line 130 of file gausspoint.h.

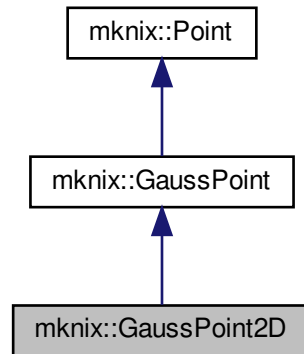
8.33.4.15 weight `double mknix::GaussPoint::weight` [protected]
Definition at line 119 of file gausspoint.h.
The documentation for this class was generated from the following files:

- [gausspoint.h](#)
- [gausspoint.cpp](#)

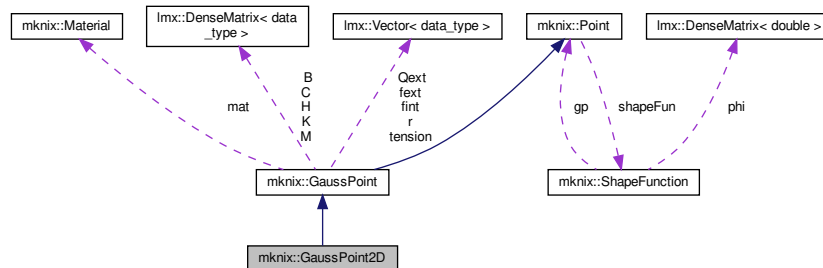
8.34 mknix::GaussPoint2D Class Reference

```
#include <gausspoint2D.h>
```

Inheritance diagram for mknix::GaussPoint2D:



Collaboration diagram for mknix::GaussPoint2D:



Public Member Functions

- [GaussPoint2D](#) ()
Default constructor for [GaussPoint2D](#).
- [GaussPoint2D](#) (double alpha_in, double weight_in, double jacobian_in, [Material](#) *mat_in, int num, double coor_x, double coor_y, double dc_in, bool stressPoint_in)
Constructs a 2D Gauss point with (x, y) coordinates.
- [GaussPoint2D](#) (double alpha_in, double weight_in, double jacobian_in, [Material](#) *mat_in, int num, double coor_x, double coor_y, double coor_z, double dc_in, bool stressPoint_in)
Constructs a 2D Gauss point with (x, y, z) coordinates (z used for out-of-plane position).
- [~GaussPoint2D](#) ()
Destructor for [GaussPoint2D](#).
- void [shapeFunSolve](#) (std::string, double) override
Computes meshfree shape functions, fills the B matrix, and accumulates smoothing weights.
- void [fillFEmatrices](#) () override
Computes FEM triangular shape functions and fills the B matrix for FEM elements.
- void [computeMij](#) () override

- Computes the local 2D mass matrix contribution M at this Gauss point.*

 - void `computeKij` () override
- Computes the linear 2D tangent (stiffness) matrix contribution K at this Gauss point.*

 - void `computeStress` () override
- Computes linear Cauchy stress and assembles the smoothed stress resultant vector r .*

 - void `computeNLStress` () override
- Computes nonlinear (large-deformation) Cauchy stress via the deformation gradient and assembles the smoothed stress resultant vector r .*

 - void `computeFint` () override
- Computes the linear internal force vector contribution $fint = K * u$ at this Gauss point.*

 - void `computeFext` () override
- Computes the external force vector contribution $fext = M * g$ at this Gauss point.*

 - void `computeNLFint` () override
- Computes the nonlinear internal force vector from the 1st Piola-Kirchhoff stress.*

 - void `computeNLKij` () override
- Computes the nonlinear tangent (material + initial stress) matrix contribution K .*

 - void `assembleMij` (Imx::Matrix< data_type > &) override
- Assembles the local mass matrix M into the global mass matrix.*

 - void `assembleKij` (Imx::Matrix< data_type > &) override
- Assembles the local tangent matrix K into the global tangent matrix.*

 - void `assembleRi` (Imx::Vector< data_type > &, int) override
- Assembles the smoothed stress resultant vector r into the body-level stress vector.*

 - void `assembleFint` (Imx::Vector< data_type > &) override
- Assembles the local internal force vector $fint$ into the global internal force vector.*

 - void `assembleFext` (Imx::Vector< data_type > &) override
- Assembles the local external force vector $fext$ into the global external force vector.*

 - double `calcPotentialE` (const Imx::Vector< data_type > &) override
- Computes the potential energy contribution ($fext \cdot q$) at this Gauss point.*

 - double `calcKineticE` (const Imx::Vector< data_type > &) override
- Computes the kinetic energy contribution ($0.5 * \dot{q}^T * M * \dot{q}$) at this Gauss point.*

 - double `calcElasticE` () override
- Computes the elastic strain energy density at this Gauss point using the stored material model.*

Additional Inherited Members

8.34.1 Detailed Description

Author

Daniel Iglesias

Definition at line 46 of file gausspoint2D.h.

8.34.2 Constructor & Destructor Documentation

8.34.2.1 GaussPoint2D() [1/3] mknix::GaussPoint2D::GaussPoint2D ()

Default constructor for `GaussPoint2D`.

Definition at line 17 of file gausspoint2D.cpp.

8.34.2.2 GaussPoint2D() [2/3] mknix::GaussPoint2D::GaussPoint2D (

```

    double alpha_in,
    double weight_in,
    double jacobian_in,
    Material * mat_in,
    int num_in,
    double coor_x,
    double coor_y,
    double dc_in,
    bool stressPoint_in )

```

Constructs a 2D Gauss point with (x, y) coordinates.

Parameters

<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian of the cell mapping.
<i>mat_in</i>	Pointer to the material.
<i>num_in</i>	Index within the parent cell.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>dc_in</i>	Characteristic nodal spacing.
<i>stressPoint_in</i>	True if this point participates in stress smoothing.

Definition at line 34 of file gausspoint2D.cpp.

8.34.2.3 GaussPoint2D() [3/3] mknix::GaussPoint2D::GaussPoint2D (

```

    double alpha_in,
    double weight_in,
    double jacobian_in,
    Material * mat_in,
    int num_in,
    double coor_x,
    double coor_y,
    double coor_z,
    double dc_in,
    bool stressPoint_in )

```

Constructs a 2D Gauss point with (x, y, z) coordinates (z used for out-of-plane position).

Parameters

<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian of the cell mapping.
<i>mat_in</i>	Pointer to the material.
<i>num_in</i>	Index within the parent cell.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>coor_z</i>	Z coordinate.
<i>dc_in</i>	Characteristic nodal spacing.
<i>stressPoint_in</i>	True if this point participates in stress smoothing.

Definition at line 58 of file gausspoint2D.cpp.

8.34.2.4 `~GaussPoint2D()` `mknix::GaussPoint2D::~~GaussPoint2D ( )`

Destructor for [GaussPoint2D](#).

Definition at line 72 of file gausspoint2D.cpp.

8.34.3 Member Function Documentation

8.34.3.1 `assembleFext()` `void mknix::GaussPoint2D::assembleFext ( lmx::Vector< data_type > & globalFext ) [override], [virtual]`

Assembles the local external force vector `fext` into the global external force vector.

Parameters

<code>globalFext</code>	Global external force vector to be updated.
-------------------------	---

Implements [mknix::GaussPoint](#).

Definition at line 704 of file gausspoint2D.cpp.

8.34.3.2 `assembleFint()` `void mknix::GaussPoint2D::assembleFint ( lmx::Vector< data_type > & globalFint ) [override], [virtual]`

Assembles the local internal force vector `fint` into the global internal force vector.

Parameters

<code>globalFint</code>	Global internal force vector to be updated.
-------------------------	---

Implements [mknix::GaussPoint](#).

Definition at line 686 of file gausspoint2D.cpp.

8.34.3.3 `assembleKij()` `void mknix::GaussPoint2D::assembleKij ( lmx::Matrix< data_type > & globalTangent ) [override], [virtual]`

Assembles the local tangent matrix `K` into the global tangent matrix.

Parameters

<code>globalTangent</code>	Global tangent (stiffness) matrix to be updated.
----------------------------	--

Implements [mknix::GaussPoint](#).

Definition at line 619 of file gausspoint2D.cpp.

8.34.3.4 `assembleMij()` `void mknix::GaussPoint2D::assembleMij ( lmx::Matrix< data_type > & globalMass ) [override], [virtual]`

Assembles the local mass matrix `M` into the global mass matrix.

Parameters

<code>globalMass</code>	Global mass matrix to be updated.
-------------------------	-----------------------------------

Implements [mknix::GaussPoint](#).

Definition at line 590 of file gausspoint2D.cpp.

8.34.3.5 assembleRi() `void mknix::GaussPoint2D::assembleRi (
 lmx::Vector< data_type > & bodyR,
 int firstNode) [override], [virtual]`

Assembles the smoothed stress resultant vector *r* into the body-level stress vector.

Parameters

<i>bodyR</i>	Body-level stress resultant vector to be updated.
<i>firstNode</i>	Global index of the first node in the parent body.

Implements [mknix::GaussPoint](#).

Definition at line 648 of file gausspoint2D.cpp.

Here is the call graph for this function:



8.34.3.6 calcElasticE() `double mknix::GaussPoint2D::calcElasticE ( ) [override], [virtual]`

Computes the elastic strain energy density at this Gauss point using the stored material model.

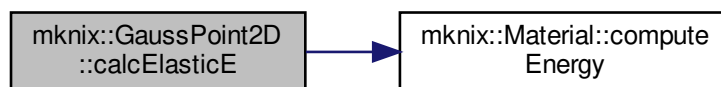
Returns

Elastic strain energy contribution.

Implements [mknix::GaussPoint](#).

Definition at line 770 of file gausspoint2D.cpp.

Here is the call graph for this function:



8.34.3.7 calcKineticE() `double mknix::GaussPoint2D::calcKineticE (
 const lmx::Vector< data_type > & qdot) [override], [virtual]`

Computes the kinetic energy contribution ($0.5 * \dot{q}^T * M * \dot{q}$) at this Gauss point.

Parameters

<i>qdot</i>	Current global velocity state vector.
-------------	---------------------------------------

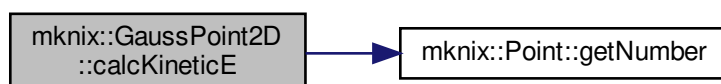
Returns

Kinetic energy contribution.

Implements [mnix::GaussPoint](#).

Definition at line 743 of file gausspoint2D.cpp.

Here is the call graph for this function:



8.34.3.8 calcPotentialE() `double mnix::GaussPoint2D::calcPotentialE ( const lmx::Vector< data\_type > & q ) [override], [virtual]`

Computes the potential energy contribution ($\text{fext} \cdot \mathbf{q}$) at this Gauss point.

Parameters

<i>q</i>	Current global displacement state vector.
----------	---

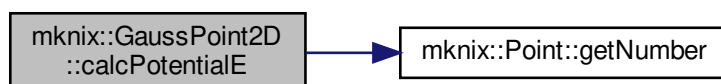
Returns

Potential energy contribution.

Implements [mnix::GaussPoint](#).

Definition at line 723 of file gausspoint2D.cpp.

Here is the call graph for this function:

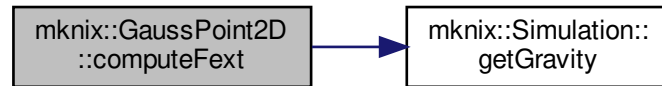


8.34.3.9 computeFext() `void mnix::GaussPoint2D::computeFext ( ) [override], [virtual]`

Computes the external force vector contribution $\text{fext} = \mathbf{M} * \mathbf{g}$ at this Gauss point.

Implements [mnix::GaussPoint](#).

Definition at line 362 of file gausspoint2D.cpp.
Here is the call graph for this function:



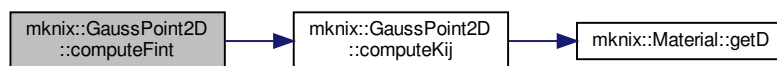
8.34.3.10 `computeFint()` `void mknix::GaussPoint2D::computeFint ( ) [override], [virtual]`

Computes the linear internal force vector contribution $\mathbf{fint} = \mathbf{K} * \mathbf{u}$ at this Gauss point.

Implements [mknix::GaussPoint](#).

Definition at line 341 of file gausspoint2D.cpp.

Here is the call graph for this function:



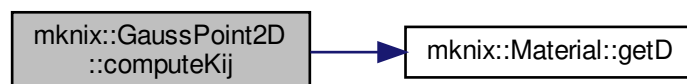
8.34.3.11 `computeKij()` `void mknix::GaussPoint2D::computeKij ( ) [override], [virtual]`

Computes the linear 2D tangent (stiffness) matrix contribution $\mathbf{K}$ at this Gauss point.

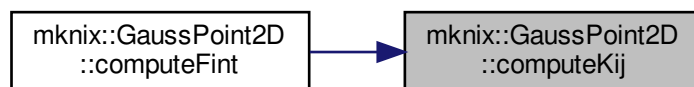
Implements [mknix::GaussPoint](#).

Definition at line 192 of file gausspoint2D.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



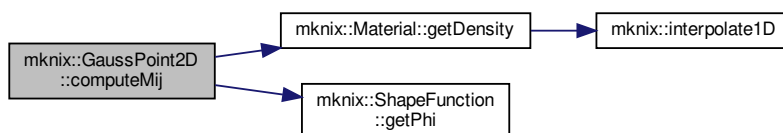
8.34.3.12 `computeMij()` `void mknix::GaussPoint2D::computeMij ( ) [override], [virtual]`

Computes the local 2D mass matrix contribution M at this Gauss point.

Implements [mknix::GaussPoint](#).

Definition at line 167 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



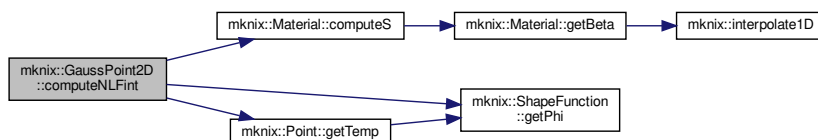
8.34.3.13 `computeNLFint()` `void mknix::GaussPoint2D::computeNLFint ( ) [override], [virtual]`

Computes the nonlinear internal force vector from the 1st Piola-Kirchhoff stress.

Implements [mknix::GaussPoint](#).

Definition at line 384 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



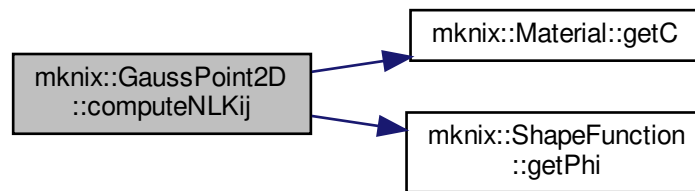
8.34.3.14 `computeNLKij()` `void mknix::GaussPoint2D::computeNLKij ( ) [override], [virtual]`

Computes the nonlinear tangent (material + initial stress) matrix contribution K.

Implements [mknix::GaussPoint](#).

Definition at line 417 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



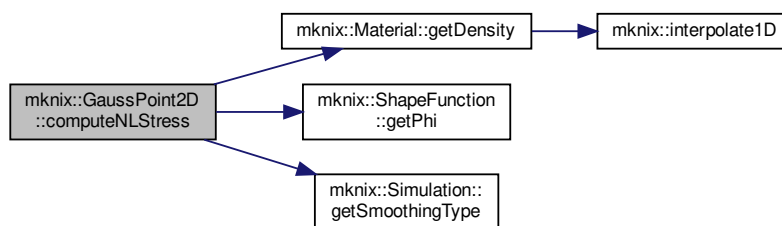
8.34.3.15 computeNLStress() `void mknix::GaussPoint2D::computeNLStress ( ) [override], [virtual]`

Computes nonlinear (large-deformation) Cauchy stress via the deformation gradient and assembles the smoothed stress resultant vector `r`.

Implements [mknix::GaussPoint](#).

Definition at line 293 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



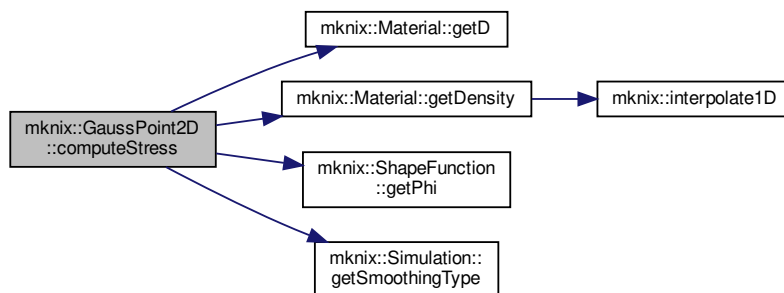
8.34.3.16 computeStress() `void mknix::GaussPoint2D::computeStress ( ) [override], [virtual]`

Computes linear Cauchy stress and assembles the smoothed stress resultant vector `r`.

Implements [mknix::GaussPoint](#).

Definition at line 226 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



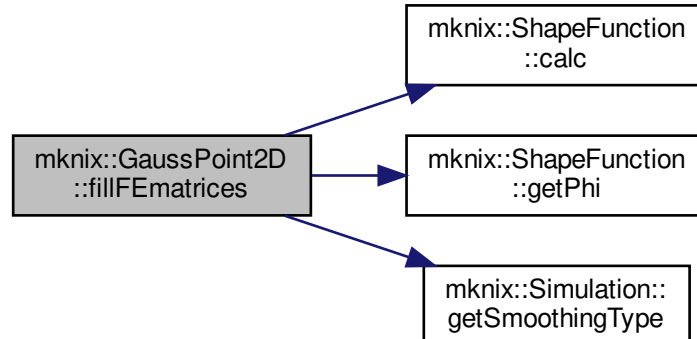
8.34.3.17 fillFEmatrices() `void mknix::GaussPoint2D::fillFEmatrices ( ) [override], [virtual]`

Computes FEM triangular shape functions and fills the B matrix for FEM elements.

Implements [mknix::GaussPoint](#).

Definition at line 123 of file `gausspoint2D.cpp`.

Here is the call graph for this function:



8.34.3.18 shapeFunSolve() `void mknix::GaussPoint2D::shapeFunSolve (`

```

    std::string type_in,
    double q_in ) [override], [virtual]

```

Computes meshfree shape functions, fills the B matrix, and accumulates smoothing weights.

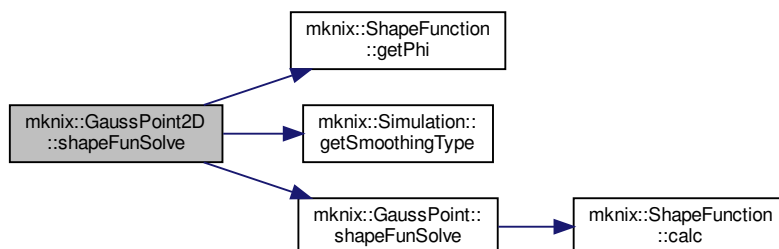
Parameters

<code>type_in</code>	Shape function type ("RBF" or "MLS").
<code>q_in</code>	Shape parameter (overridden internally).

Reimplemented from [mknix::GaussPoint](#).

Definition at line 82 of file gausspoint2D.cpp.

Here is the call graph for this function:



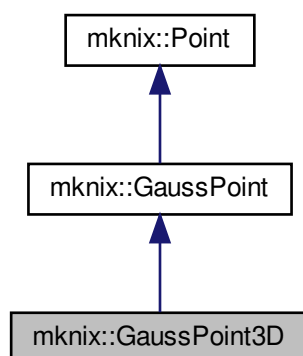
The documentation for this class was generated from the following files:

- [gausspoint2D.h](#)
- [gausspoint2D.cpp](#)

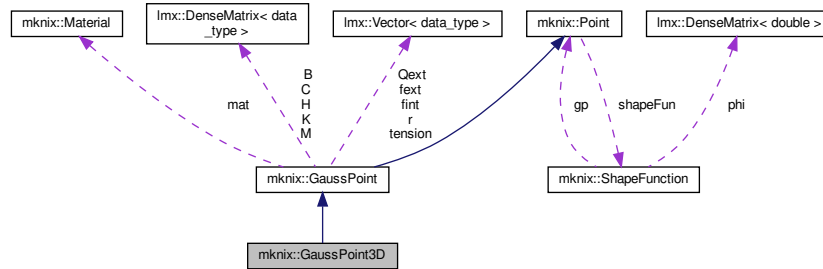
8.35 mknix::GaussPoint3D Class Reference

```
#include <gausspoint3D.h>
```

Inheritance diagram for `mknix::GaussPoint3D`:



Collaboration diagram for `mknix::GaussPoint3D`:



Public Member Functions

- [GaussPoint3D](#) ()
Default constructor for [GaussPoint3D](#).
- [GaussPoint3D](#) (double alpha_in, double weight_in, double jacobian_in, [Material](#) *mat_in, int num, double coor_x, double coor_y, double coor_z, double dc_in, bool stressPoint_in)
Constructs a 3D Gauss point with (x, y, z) coordinates.
- [~GaussPoint3D](#) ()
Destructor for [GaussPoint3D](#).
- void [shapeFunSolve](#) (std::string, double) override
Computes meshfree shape functions, fills the 3D B matrix, and accumulates smoothing weights.
- void [fillFEMatrices](#) () override
Computes FEM tetrahedral shape functions and fills the 3D B matrix for FEM elements.
- void [computeMij](#) () override
Computes the local 3D mass matrix contribution M at this Gauss point.
- void [computeKij](#) () override
Computes the linear 3D tangent (stiffness) matrix contribution K at this Gauss point.
- void [computeStress](#) () override
Computes linear 3D Cauchy stress and assembles the smoothed stress resultant vector r.
- void [computeNLStress](#) () override
Computes nonlinear (large-deformation) 3D Cauchy stress and assembles the stress resultant r.
- void [computeFint](#) () override
Computes the linear 3D internal force vector contribution $fint = K * u$.
- void [computeFext](#) () override
Computes the 3D external force vector contribution $fext = M * g$.
- void [computeNLFint](#) () override
Computes the nonlinear 3D internal force vector from the 1st Piola-Kirchhoff stress.
- void [computeNLKij](#) () override
Computes the nonlinear 3D tangent (material + initial stress) matrix contribution K.
- void [assembleMij](#) (lmx::Matrix< data_type > &) override
Assembles the local 3D mass matrix M into the global mass matrix.
- void [assembleKij](#) (lmx::Matrix< data_type > &) override
Assembles the local 3D tangent matrix K into the global tangent matrix.
- void [assembleRi](#) (lmx::Vector< data_type > &, int) override
Assembles the 3D smoothed stress resultant vector r into the body-level stress vector.
- void [assembleFint](#) (lmx::Vector< data_type > &) override
Assembles the local 3D internal force vector fint into the global internal force vector.

- void [assembleFext](#) (Imx::Vector< [data_type](#) > &) override
Assembles the local 3D external force vector fext into the global external force vector.
- double [calcPotentialE](#) (const Imx::Vector< [data_type](#) > &) override
Computes the 3D potential energy contribution ($fext \cdot q$) at this Gauss point.
- double [calcKineticE](#) (const Imx::Vector< [data_type](#) > &) override
*Computes the 3D kinetic energy contribution ($0.5 * \dot{q}^T * M * \dot{q}$) at this Gauss point.*
- double [calcElasticE](#) () override
Computes the 3D elastic strain energy density using the stored material model.

Additional Inherited Members

8.35.1 Detailed Description

Author

Daniel Iglesias

Definition at line 46 of file gausspoint3D.h.

8.35.2 Constructor & Destructor Documentation

8.35.2.1 GaussPoint3D() [1/2] mknix::GaussPoint3D::GaussPoint3D ()

Default constructor for [GaussPoint3D](#).

Definition at line 17 of file gausspoint3D.cpp.

8.35.2.2 GaussPoint3D() [2/2] mknix::GaussPoint3D::GaussPoint3D (

```
double alpha_in,
double weight_in,
double jacobian_in,
Material * mat_in,
int num_in,
double coor_x,
double coor_y,
double coor_z,
double dc_in,
bool stressPoint_in )
```

Constructs a 3D Gauss point with (x, y, z) coordinates.

Parameters

<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian of the cell mapping.
<i>mat_in</i>	Pointer to the material.
<i>num_in</i>	Index within the parent cell.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>coor_z</i>	Z coordinate.
<i>dc_in</i>	Characteristic nodal spacing.
<i>stressPoint_in</i>	True if this point participates in stress smoothing.

Definition at line 35 of file gausspoint3D.cpp.

8.35.2.3 `~GaussPoint3D()` `mknix::GaussPoint3D::~~GaussPoint3D ( )`

Destructor for [GaussPoint3D](#).

Definition at line 66 of file `gausspoint3D.cpp`.

8.35.3 Member Function Documentation

8.35.3.1 `assembleFext()` `void mknix::GaussPoint3D::assembleFext ( lmx::Vector< data_type > & globalFext ) [override], [virtual]`

Assembles the local 3D external force vector `fext` into the global external force vector.

Parameters

<code>globalFext</code>	Global external force vector to be updated.
-------------------------	---

Implements [mknix::GaussPoint](#).

Definition at line 700 of file `gausspoint3D.cpp`.

8.35.3.2 `assembleFint()` `void mknix::GaussPoint3D::assembleFint ( lmx::Vector< data_type > & globalFint ) [override], [virtual]`

Assembles the local 3D internal force vector `fint` into the global internal force vector.

Parameters

<code>globalFint</code>	Global internal force vector to be updated.
-------------------------	---

Implements [mknix::GaussPoint](#).

Definition at line 682 of file `gausspoint3D.cpp`.

8.35.3.3 `assembleKij()` `void mknix::GaussPoint3D::assembleKij ( lmx::Matrix< data_type > & globalTangent ) [override], [virtual]`

Assembles the local 3D tangent matrix `K` into the global tangent matrix.

Parameters

<code>globalTangent</code>	Global tangent (stiffness) matrix to be updated.
----------------------------	--

Implements [mknix::GaussPoint](#).

Definition at line 632 of file `gausspoint3D.cpp`.

8.35.3.4 `assembleMij()` `void mknix::GaussPoint3D::assembleMij ( lmx::Matrix< data_type > & globalMass ) [override], [virtual]`

Assembles the local 3D mass matrix `M` into the global mass matrix.

Parameters

<code>globalMass</code>	Global mass matrix to be updated.
-------------------------	-----------------------------------

Implements [mknix::GaussPoint](#).

Definition at line 602 of file `gausspoint3D.cpp`.

8.35.3.5 assembleRi() `void mknix::GaussPoint3D::assembleRi (
 lmx::Vector< data_type > & bodyR,
 int firstNode) [override], [virtual]`

Assembles the 3D smoothed stress resultant vector *r* into the body-level stress vector.

Parameters

<i>bodyR</i>	Body-level stress resultant vector to be updated.
<i>firstNode</i>	Global index of the first node in the parent body.

Implements [mknix::GaussPoint](#).

Definition at line 661 of file gausspoint3D.cpp.

8.35.3.6 calcElasticE() `double mknix::GaussPoint3D::calcElasticE ( ) [override], [virtual]`
 Computes the 3D elastic strain energy density using the stored material model.

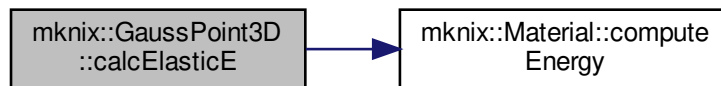
Returns

Elastic strain energy contribution.

Implements [mknix::GaussPoint](#).

Definition at line 765 of file gausspoint3D.cpp.

Here is the call graph for this function:



8.35.3.7 calcKineticE() `double mknix::GaussPoint3D::calcKineticE (
 const lmx::Vector< data_type > & qdot) [override], [virtual]`
 Computes the 3D kinetic energy contribution $(0.5 * \dot{q}^T * M * \dot{q})$ at this Gauss point.

Parameters

<i>qdot</i>	Current global velocity state vector.
-------------	---------------------------------------

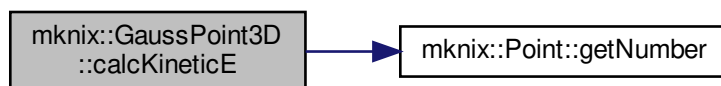
Returns

Kinetic energy contribution.

Implements [mknix::GaussPoint](#).

Definition at line 739 of file gausspoint3D.cpp.

Here is the call graph for this function:



8.35.3.8 calcPotentialE() `double mknix::GaussPoint3D::calcPotentialE ( const lmx::Vector< data_type > & q ) [override], [virtual]`

Computes the 3D potential energy contribution ($\text{fext} \cdot \mathbf{q}$) at this Gauss point.

Parameters

q	Current global displacement state vector.
-----	---

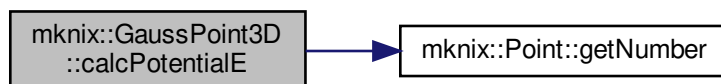
Returns

Potential energy contribution.

Implements [mknix::GaussPoint](#).

Definition at line 719 of file gausspoint3D.cpp.

Here is the call graph for this function:



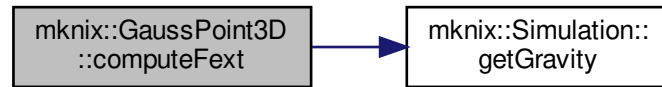
8.35.3.9 computeFext() `void mknix::GaussPoint3D::computeFext ( ) [override], [virtual]`

Computes the 3D external force vector contribution $\text{fext} = \mathbf{M} * \mathbf{g}$.

Implements [mknix::GaussPoint](#).

Definition at line 362 of file gausspoint3D.cpp.

Here is the call graph for this function:



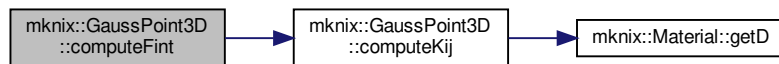
8.35.3.10 `computeFint()` `void mknix::GaussPoint3D::computeFint ( ) [override], [virtual]`

Computes the linear 3D internal force vector contribution $\mathbf{fint} = \mathbf{K} * \mathbf{u}$.

Implements [mknix::GaussPoint](#).

Definition at line 341 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



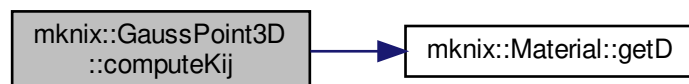
8.35.3.11 `computeKij()` `void mknix::GaussPoint3D::computeKij ( ) [override], [virtual]`

Computes the linear 3D tangent (stiffness) matrix contribution $\mathbf{K}$ at this Gauss point.

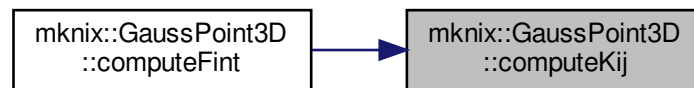
Implements [mknix::GaussPoint](#).

Definition at line 184 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



Here is the caller graph for this function:



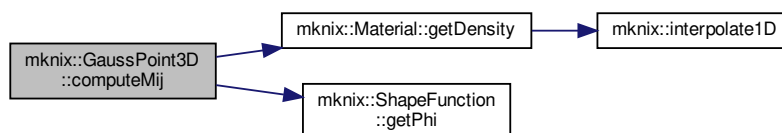
8.35.3.12 `computeMij()` `void mknix::GaussPoint3D::computeMij ( ) [override], [virtual]`

Computes the local 3D mass matrix contribution M at this Gauss point.

Implements [mknix::GaussPoint](#).

Definition at line 156 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



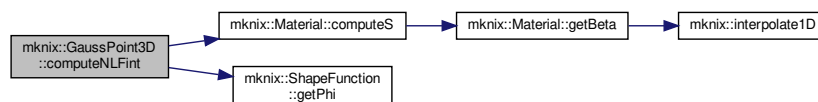
8.35.3.13 `computeNLFint()` `void mknix::GaussPoint3D::computeNLFint ( ) [override], [virtual]`

Computes the nonlinear 3D internal force vector from the 1st Piola-Kirchhoff stress.

Implements [mknix::GaussPoint](#).

Definition at line 384 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



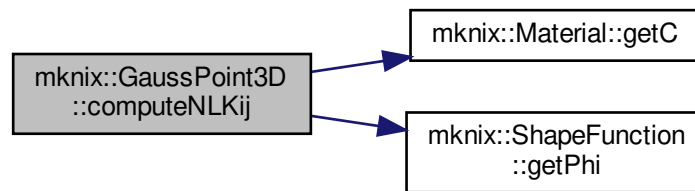
8.35.3.14 `computeNLKij()` `void mknix::GaussPoint3D::computeNLKij ( ) [override], [virtual]`

Computes the nonlinear 3D tangent (material + initial stress) matrix contribution K.

Implements [mknix::GaussPoint](#).

Definition at line 417 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



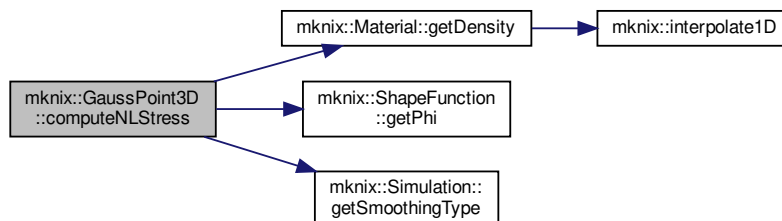
8.35.3.15 `computeNLStress()` `void mknix::GaussPoint3D::computeNLStress ( ) [override], [virtual]`

Computes nonlinear (large-deformation) 3D Cauchy stress and assembles the stress resultant `r`.

Implements [mknix::GaussPoint](#).

Definition at line 275 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



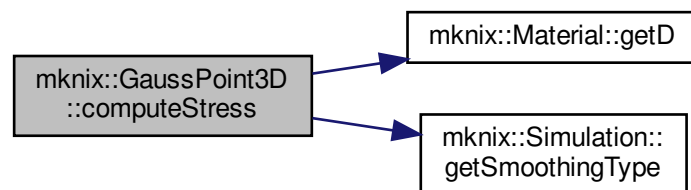
8.35.3.16 `computeStress()` `void mknix::GaussPoint3D::computeStress ( ) [override], [virtual]`

Computes linear 3D Cauchy stress and assembles the smoothed stress resultant vector `r`.

Implements [mknix::GaussPoint](#).

Definition at line 223 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



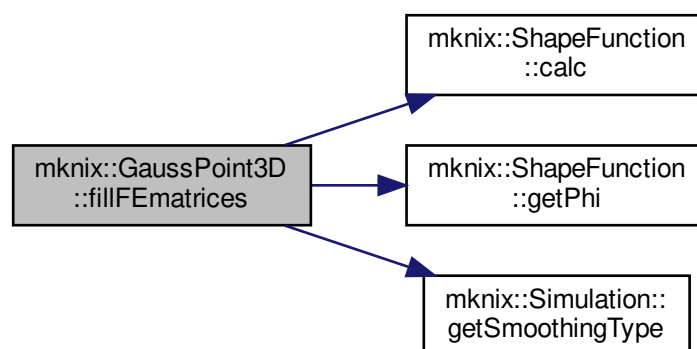
8.35.3.17 fillFEmatrices() `void mknix::GaussPoint3D::fillFEmatrices ( ) [override], [virtual]`

Computes FEM tetrahedral shape functions and fills the 3D B matrix for FEM elements.

Implements [mknix::GaussPoint](#).

Definition at line 116 of file `gausspoint3D.cpp`.

Here is the call graph for this function:



8.35.3.18 shapeFunSolve() `void mknix::GaussPoint3D::shapeFunSolve (` `std::string type_in,` `double q_in ) [override], [virtual]`

Computes meshfree shape functions, fills the 3D B matrix, and accumulates smoothing weights.

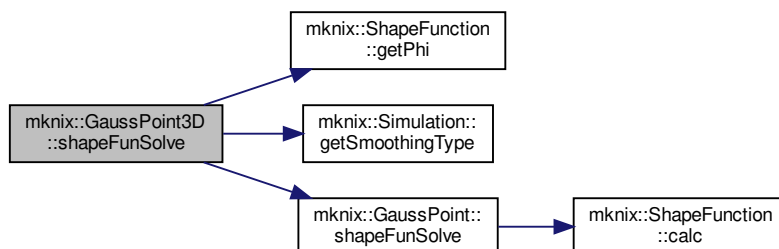
Parameters

<i>type_in</i>	Shape function type ("RBF" or "MLS").
<i>q_in</i>	Shape parameter (overridden internally).

Reimplemented from [mknix::GaussPoint](#).

Definition at line 76 of file gausspoint3D.cpp.

Here is the call graph for this function:



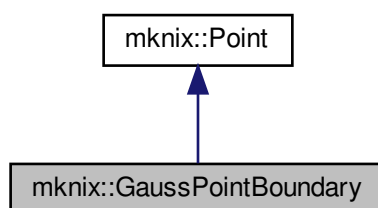
The documentation for this class was generated from the following files:

- [gausspoint3D.h](#)
- [gausspoint3D.cpp](#)

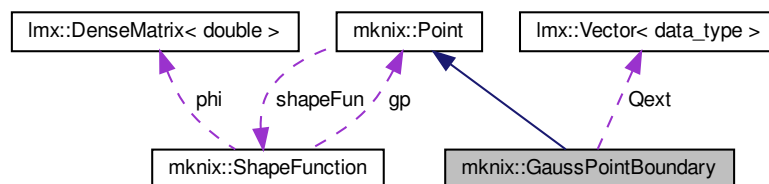
8.36 mknix::GaussPointBoundary Class Reference

```
#include <gausspointboundary.h>
```

Inheritance diagram for `mknix::GaussPointBoundary`:



Collaboration diagram for `mknix::GaussPointBoundary`:



Public Member Functions

- [GaussPointBoundary](#) ()
Default constructor for [GaussPointBoundary](#).
- [GaussPointBoundary](#) (int dim_in, double alpha_in, double weight_in, double jacobian_in, int num_in, double coor_x, double dc_in)
Constructs a 1D boundary Gauss point with a single x coordinate.
- [GaussPointBoundary](#) (int dim_in, double alpha_in, double weight_in, double jacobian_in, int num_in, double coor_x, double coor_y, double dc_in)
Constructs a boundary Gauss point with (x, y) coordinates.
- [~GaussPointBoundary](#) ()
Destructor for [GaussPointBoundary](#).
- virtual void [shapeFunSolve](#) (std::string, double) override
Computes shape function values at this boundary point and initializes internal vectors.
- void [computeQext](#) ([LoadThermalBoundary1D](#) *)
Computes the external thermal boundary heat load contribution Qext at this point.
- void [assembleQext](#) (Imx::Vector< [data_type](#) > &)
Assembles the local boundary heat load vector Qext into the global heat load vector.

Protected Member Functions

- void [initializeMatVecs](#) ()
Initializes internal vectors to the correct size based on the number of support nodes.

Protected Attributes

- int [num](#)
- double [weight](#)
- Imx::Vector< [data_type](#) > [Qext](#)

8.36.1 Detailed Description

Author

Daniel Iglesias

Definition at line 45 of file gausspointboundary.h.

8.36.2 Constructor & Destructor Documentation

8.36.2.1 [GaussPointBoundary](#)() [1/3] `mnix::GaussPointBoundary::GaussPointBoundary ( )`

Default constructor for [GaussPointBoundary](#).

Definition at line 18 of file gausspointboundary.cpp.

8.36.2.2 [GaussPointBoundary](#)() [2/3] `mnix::GaussPointBoundary::GaussPointBoundary (`

```
int dim_in,
double alpha_in,
double weight_in,
double jacobian_in,
int num_in,
double coor_x,
double dc_in )
```

Constructs a 1D boundary Gauss point with a single x coordinate.

Parameters

<i>dim_in</i>	Spatial dimension of the parent domain.
<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian value for the boundary mapping.
<i>num_in</i>	Index of this point within its boundary cell.
<i>coor_x</i>	X coordinate of the boundary Gauss point.
<i>dc_in</i>	Characteristic nodal spacing.

Definition at line 33 of file gausspointboundary.cpp.

8.36.2.3 GaussPointBoundary() [3/3] `mknix::GaussPointBoundary::GaussPointBoundary (`
`int dim_in,`
`double alpha_in,`
`double weight_in,`
`double jacobian_in,`
`int num_in,`
`double coor_x,`
`double coor_y,`
`double dc_in )`

Constructs a boundary Gauss point with (x, y) coordinates.

Parameters

<i>dim_in</i>	Spatial dimension of the parent domain.
<i>alpha_in</i>	Influence radius scaling factor.
<i>weight_in</i>	Quadrature weight.
<i>jacobian_in</i>	Jacobian value for the boundary mapping.
<i>num_in</i>	Index of this point within its boundary cell.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>dc_in</i>	Characteristic nodal spacing.

Definition at line 57 of file gausspointboundary.cpp.

8.36.2.4 ~GaussPointBoundary() `mknix::GaussPointBoundary::~~GaussPointBoundary ( )`
Destructor for [GaussPointBoundary](#).

Definition at line 74 of file gausspointboundary.cpp.

8.36.3 Member Function Documentation

8.36.3.1 assembleQext() `void mknix::GaussPointBoundary::assembleQext (`
`lmx::Vector< data_type > & globalHeat )`

Assembles the local boundary heat load vector Qext into the global heat load vector.

Parameters

<i>globalHeat</i>	Global external heat load vector to be updated.
-------------------	---

Definition at line 151 of file gausspointboundary.cpp.

8.36.3.2 computeQext() `void mknix::GaussPointBoundary::computeQext (
 LoadThermalBoundary1D * loadThermalBoundary_in)`

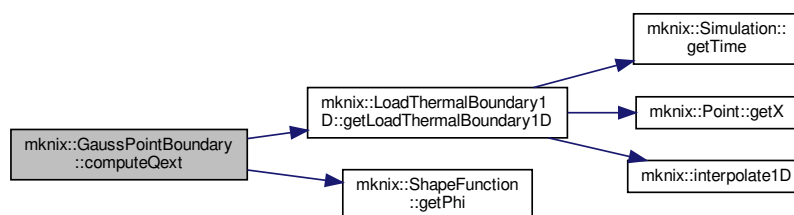
Computes the external thermal boundary heat load contribution Qext at this point.

Parameters

<i>loadThermalBoundary_in</i>	Pointer to the 1D boundary thermal load object.
-------------------------------	---

Definition at line 131 of file gausspointboundary.cpp.

Here is the call graph for this function:

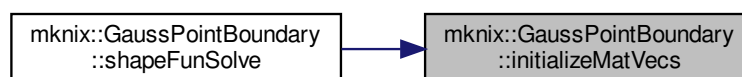


8.36.3.3 initializeMatVecs() `void mknix::GaussPointBoundary::initializeMatVecs ( ) [protected]`

Initializes internal vectors to the correct size based on the number of support nodes.

Definition at line 171 of file gausspointboundary.cpp.

Here is the caller graph for this function:



8.36.3.4 shapeFunSolve() `void mknix::GaussPointBoundary::shapeFunSolve (
 std::string type_in,
 double q_in) [override], [virtual]`

Computes shape function values at this boundary point and initializes internal vectors.

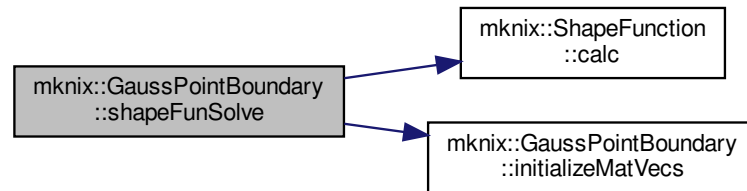
Parameters

<i>type_in</i>	Shape function type: "RBF", "MLS", or "1D-X".
<i>q_in</i>	Shape parameter (overridden internally to 0.5).

Reimplemented from [mknix::Point](#).

Definition at line 84 of file gausspointboundary.cpp.

Here is the call graph for this function:



8.36.4 Member Data Documentation

8.36.4.1 num `int mknix::GaussPointBoundary::num` [protected]

Definition at line 81 of file gausspointboundary.h.

8.36.4.2 Qext `lmx::Vector<data_type> mknix::GaussPointBoundary::Qext` [protected]

Definition at line 84 of file gausspointboundary.h.

8.36.4.3 weight `double mknix::GaussPointBoundary::weight` [protected]

Definition at line 82 of file gausspointboundary.h.

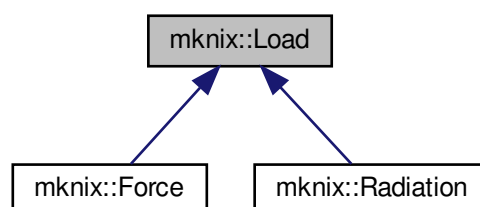
The documentation for this class was generated from the following files:

- [gausspointboundary.h](#)
- [gausspointboundary.cpp](#)

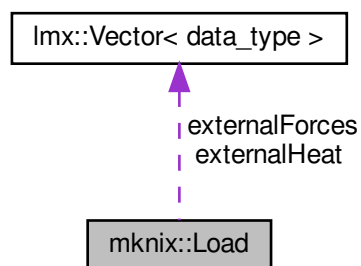
8.37 mknix::Load Class Reference

```
#include <load.h>
```

Inheritance diagram for `mknix::Load`:



Collaboration diagram for `mknix::Load`:



Public Member Functions

- [Load](#) ()
Default constructor.
- virtual [~Load](#) ()
Destructor.
- virtual void [assembleExternalForces](#) ([Imx::Vector< data_type >](#) &)
Assembles the local external force vector into the global force vector.
- virtual void [outputToFile](#) (`std::ofstream *`)=0

Protected Attributes

- `std::vector< Node \* >` [nodes](#)
- [Imx::Vector< data_type >](#) [externalForces](#)
- [Imx::Vector< data_type >](#) [externalHeat](#)

8.37.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 33 of file `load.h`.

8.37.2 Constructor & Destructor Documentation

8.37.2.1 `Load()` `mknix::Load::Load ( )`

Default constructor.

Definition at line 31 of file `load.cpp`.

8.37.2.2 `~Load()` `mknix::Load::~~Load ( )` [virtual]

Destructor.

Definition at line 39 of file `load.cpp`.

8.37.3 Member Function Documentation

8.37.3.1 assembleExternalForces() `void mknix::Load::assembleExternalForces (
lmx::Vector< data_type > & globalExternalForces) [virtual]`

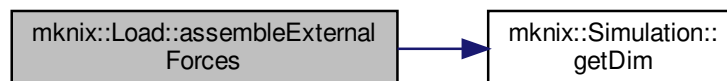
Assembles the local external force vector into the global force vector.

Parameters

<i>globalExternalForces</i>	Reference to the global external force vector.
-----------------------------	--

Definition at line 48 of file load.cpp.

Here is the call graph for this function:



8.37.3.2 outputToFile() `virtual void mknix::Load::outputToFile (
std::ofstream *) [pure virtual]`

Implemented in [mknix::Radiation](#), and [mknix::Force](#).

8.37.4 Member Data Documentation

8.37.4.1 externalForces `lmx::Vector<data\_type> mknix::Load::externalForces [protected]`

Definition at line 46 of file load.h.

8.37.4.2 externalHeat `lmx::Vector<data\_type> mknix::Load::externalHeat [protected]`

Definition at line 47 of file load.h.

8.37.4.3 nodes `std::vector<Node\*> mknix::Load::nodes [protected]`

Definition at line 45 of file load.h.

The documentation for this class was generated from the following files:

- [load.h](#)
- [load.cpp](#)

8.38 mknix::LoadThermal Class Reference

```
#include <loadthermal.h>
```

Public Member Functions

- [LoadThermal](#) ()
Default constructor.
- [LoadThermal](#) ([Node](#) *, double)
Constructor with node and fluence value.

- virtual `~LoadThermal()`
Destructor.
- virtual void `insertNodesXCoordinates` (`std::vector< double > &`)
Appends the X coordinates of the loaded nodes to the provided vector.
- virtual void `updateLoad` (`double load_in`)
- virtual void `assembleExternalHeat` (`lmx::Vector< data_type > &`)
Assembles the thermal load value into the global external heat vector.
- virtual void `outputToFile` (`std::ofstream *`)
- void `getMaxTemp` (`double &`)
Updates maxTemp_in with the maximum temperature among all loaded nodes.

Protected Attributes

- `std::vector< Node * > nodes`
- `data_type externalHeat`

8.38.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 33 of file loadthermal.h.

8.38.2 Constructor & Destructor Documentation

8.38.2.1 LoadThermal() [1/2] `mknix::LoadThermal::LoadThermal()`

Default constructor.

Definition at line 31 of file loadthermal.cpp.

8.38.2.2 LoadThermal() [2/2] `mknix::LoadThermal::LoadThermal()`

`Node * node_in,`
`double fluence_in`

Constructor with node and fluence value.

Parameters

<code>node_in</code>	Pointer to the node where the heat load is applied.
<code>fluence_in</code>	Heat fluence (thermal load) value.

Definition at line 40 of file loadthermal.cpp.

8.38.2.3 ~LoadThermal() `mknix::LoadThermal::~~LoadThermal()` [virtual]

Destructor.

Definition at line 49 of file loadthermal.cpp.

8.38.3 Member Function Documentation

8.38.3.1 assembleExternalHeat() `void mknix::LoadThermal::assembleExternalHeat()` [virtual]

`lmx::Vector< data_type > & globalExternalHeat`

Assembles the thermal load value into the global external heat vector.

Parameters

<i>globalExternalHeat</i>	Reference to the global external heat vector.
---------------------------	---

Definition at line 74 of file loadthermal.cpp.

8.38.3.2 getMaxTemp() `void mknix::LoadThermal::getMaxTemp ( double & maxTemp_in )`

Updates maxTemp_in with the maximum temperature among all loaded nodes.

Parameters

<i>maxTemp_in</i>	Reference to the running maximum temperature value.
-------------------	---

Definition at line 94 of file loadthermal.cpp.

8.38.3.3 insertNodesXCoordinates() `void mknix::LoadThermal::insertNodesXCoordinates ( std::vector< double > & x_coordinates ) [virtual]`

Appends the X coordinates of the loaded nodes to the provided vector.

Parameters

<i>x_coordinates</i>	Vector to which node X coordinates are appended.
----------------------	--

Definition at line 57 of file loadthermal.cpp.

8.38.3.4 outputToFile() `virtual void mknix::LoadThermal::outputToFile ( std::ofstream * ) [inline], [virtual]`

Definition at line 51 of file loadthermal.h.

8.38.3.5 updateLoad() `virtual void mknix::LoadThermal::updateLoad ( double load_in ) [inline], [virtual]`

Definition at line 44 of file loadthermal.h.

8.38.4 Member Data Documentation

8.38.4.1 externalHeat `data_type mknix::LoadThermal::externalHeat [protected]`

Definition at line 58 of file loadthermal.h.

8.38.4.2 nodes `std::vector<Node*> mknix::LoadThermal::nodes [protected]`

Definition at line 57 of file loadthermal.h.

The documentation for this class was generated from the following files:

- [loadthermal.h](#)
- [loadthermal.cpp](#)

8.39 mknix::LoadThermalBody Class Reference

```
#include <loadthermalbody.h>
```

Public Member Functions

- [LoadThermalBody](#) ()
Default constructor. Initializes the constant heat source value to zero.
- virtual [~LoadThermalBody](#) ()
Destructor.
- virtual double [getLoadThermalBody](#) (Point *)
*Computes the thermal body load at a given point. * Returns the heat source value at the location of the provided point. If a 2D spatial distribution is available it is interpolated using the point's X and Y coordinates; if only a 1D distribution is available it is interpolated using the X coordinate; otherwise the constant source value is used. The result is further scaled by the time-dependent factor interpolated from the time map, if one has been loaded.*
- virtual void [setConstantValue](#) (double value_in)
- void [loadTimeFile](#) (const std::string &fileName)
Loads a time-dependent load scale factor from a file.
- void [addLoad](#) (double distance_in, double load_in)
- void [addLoad](#) (double key1_in, double key2_in, double load_in)
- void [addLoad](#) (double key1_in, double key2_in, double key3_in, double load_in)
- void [loadFile3D](#) (const std::string &fileName)

Protected Attributes

- double [constantSouce](#)
- [SourceType](#) [m_sourceType](#) = [SourceType::CONSTANT](#)
- bool [m_hasTimeScale](#) = false
- std::map< double, double > [m_source](#)
- std::map< double, std::map< double, double > > [m_source2D](#)
- std::map< double, std::map< double, std::map< double, double > > > [m_source3D](#)
- std::map< double, double > [m_time](#)

8.39.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 40 of file loadthermalbody.h.

8.39.2 Constructor & Destructor Documentation

8.39.2.1 LoadThermalBody() `mknix::LoadThermalBody::LoadThermalBody ( )`

Default constructor. Initializes the constant heat source value to zero.

Definition at line 43 of file loadthermalbody.cpp.

8.39.2.2 ~LoadThermalBody() `mknix::LoadThermalBody::~~LoadThermalBody ( )` [virtual]

Destructor.

Definition at line 51 of file loadthermalbody.cpp.

8.39.3 Member Function Documentation

8.39.3.1 addLoad() [1/3] `void mknix::LoadThermalBody::addLoad (`
`double distance_in,`
`double load_in ) [inline]`

Definition at line 58 of file loadthermalbody.h.

8.39.3.2 addLoad() [2/3] `void mknix::LoadThermalBody::addLoad (`
`double key1_in,`
`double key2_in,`
`double key3_in,`
`double load_in ) [inline]`

Definition at line 72 of file loadthermalbody.h.

8.39.3.3 addLoad() [3/3] `void mknix::LoadThermalBody::addLoad (`
`double key1_in,`
`double key2_in,`
`double load_in ) [inline]`

Definition at line 65 of file loadthermalbody.h.

8.39.3.4 getLoadThermalBody() `double mknix::LoadThermalBody::getLoadThermalBody (`
`Point * point_in ) [virtual]`

Computes the thermal body load at a given point. * Returns the heat source value at the location of the provided point. If a 2D spatial distribution is available it is interpolated using the point's X and Y coordinates; if only a 1D distribution is available it is interpolated using the X coordinate; otherwise the constant source value is used. The result is further scaled by the time-dependent factor interpolated from the time map, if one has been loaded.

Parameters

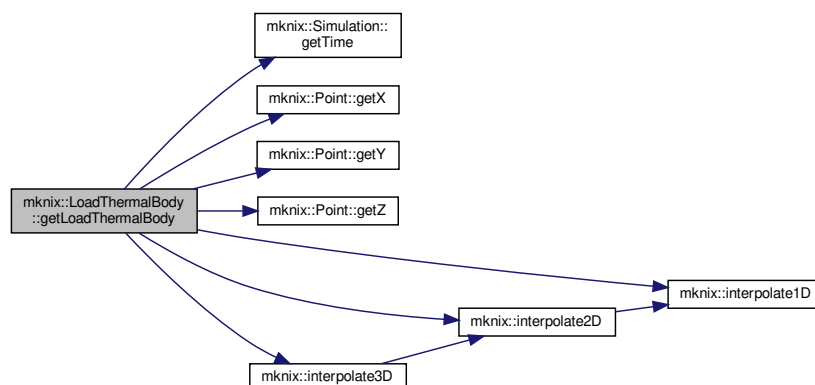
<i>point_in</i>	Pointer to the Point at which the load is evaluated.
-----------------	--

Returns

The thermal body load value at the given point and current simulation time.

Definition at line 95 of file loadthermalbody.cpp.

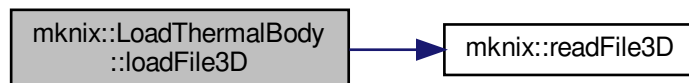
Here is the call graph for this function:



8.39.3.5 loadFile3D() `void mknix::LoadThermalBody::loadFile3D (`
`const std::string & fileName ) [inline]`

Definition at line 78 of file loadthermalbody.h.

Here is the call graph for this function:



8.39.3.6 loadTimeFile() `void mknix::LoadThermalBody::loadTimeFile (`
`const std::string & fileName )`

Loads a time-dependent load scale factor from a file.

Reads pairs of (time, load_factor) values from the specified file and stores them in the internal time map for use during simulation.

Parameters

<i>fileName</i>	Path to the file containing time and load factor pairs.
-----------------	---

Definition at line 63 of file loadthermalbody.cpp.

8.39.3.7 setConstantValue() `virtual void mknix::LoadThermalBody::setConstantValue (`
`double value_in ) [inline], [virtual]`

Definition at line 51 of file loadthermalbody.h.

8.39.4 Member Data Documentation

8.39.4.1 constantSouce `double mknix::LoadThermalBody::constantSouce [protected]`

Definition at line 95 of file loadthermalbody.h.

8.39.4.2 m_hasTimeScale `bool mknix::LoadThermalBody::m_hasTimeScale = false [protected]`

Definition at line 97 of file loadthermalbody.h.

8.39.4.3 m_source `std::map<double, double> mknix::LoadThermalBody::m_source [protected]`

Definition at line 98 of file loadthermalbody.h.

8.39.4.4 m_source2D `std::map<double, std::map<double, double> > mknix::LoadThermalBody::m_source2D [protected]`

Definition at line 99 of file loadthermalbody.h.

8.39.4.5 m_source3D `std::map<double, std::map<double, std::map<double, double> > > mknix::LoadThermalBody::m_source3D` [protected]
 Definition at line 100 of file loadthermalbody.h.

8.39.4.6 m_sourceType `SourceType mknix::LoadThermalBody::m_sourceType = SourceType::CONSTANT` [protected]
 Definition at line 96 of file loadthermalbody.h.

8.39.4.7 m_time `std::map<double, double> mknix::LoadThermalBody::m_time` [protected]
 Definition at line 101 of file loadthermalbody.h.
 The documentation for this class was generated from the following files:

- [loadthermalbody.h](#)
- [loadthermalbody.cpp](#)

8.40 mknix::LoadThermalBoundary1D Class Reference

```
#include <loadthermalboundary1D.h>
```

Public Member Functions

- [LoadThermalBoundary1D](#) ()
Default constructor.
- [~LoadThermalBoundary1D](#) ()
Destructor.
- void [loadFile](#) (std::string)
Reads (X, load) pairs from a file and stores them in the spatial load map.
- void [loadTimeFile](#) (std::string)
Reads (time, load_factor) pairs from a file and stores them in the time-scale map.
- void [scaleLoad](#) (double)
Scales all spatial load values by a constant factor.
- double [getLoadThermalBoundary1D](#) (Point *)
Returns the thermal boundary load at the given point's X coordinate, optionally scaled by the time-dependent factor.

Protected Attributes

- std::map< double, double > [loadmap](#)
- std::map< double, double > [timemap](#)

8.40.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 35 of file loadthermalboundary1D.h.

8.40.2 Constructor & Destructor Documentation

8.40.2.1 LoadThermalBoundary1D() `mknix::LoadThermalBoundary1D::LoadThermalBoundary1D ( )`
 Default constructor.
 Definition at line 32 of file loadthermalboundary1D.cpp.

8.40.2.2 `~LoadThermalBoundary1D()` `mknix::LoadThermalBoundary1D::~~LoadThermalBoundary1D ( )`
 Destructor.
 Definition at line 40 of file `loadthermalboundary1D.cpp`.

8.40.3 Member Function Documentation

8.40.3.1 `getLoadThermalBoundary1D()` `double mknix::LoadThermalBoundary1D::getLoadThermalBoundary1D (`
`Point * thePoint )`

Returns the thermal boundary load at the given point's X coordinate, optionally scaled by the time-dependent factor.

Parameters

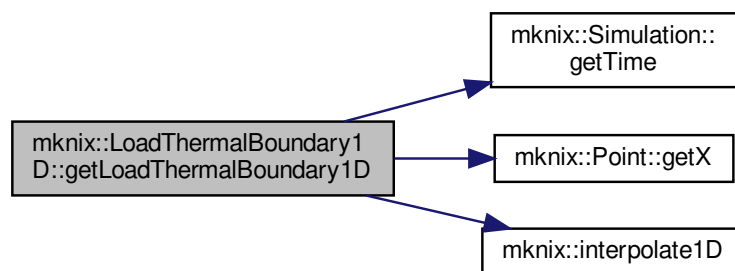
<i>thePoint</i>	Pointer to the boundary integration point.
-----------------	--

Returns

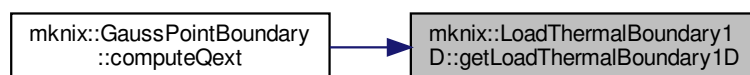
Interpolated load value at the point's position and current simulation time.

Definition at line 110 of file `loadthermalboundary1D.cpp`.

Here is the call graph for this function:



Here is the caller graph for this function:



8.40.3.2 `loadFile()` `void mknix::LoadThermalBoundary1D::loadFile (`
`std::string fileName )`

Reads (X, load) pairs from a file and stores them in the spatial load map.

Parameters

<i>fileName</i>	Path to the spatial load data file.
-----------------	-------------------------------------

Definition at line 48 of file loadthermalboundary1D.cpp.

8.40.3.3 loadTimeFile() `void mknix::LoadThermalBoundary1D::loadTimeFile ( std::string fileName )`

Reads (time, load_factor) pairs from a file and stores them in the time-scale map.

Parameters

<i>fileName</i>	Path to the time-scale data file.
-----------------	-----------------------------------

Definition at line 71 of file loadthermalboundary1D.cpp.

8.40.3.4 scaleLoad() `void mknix::LoadThermalBoundary1D::scaleLoad ( double loadFactor_in )`

Scales all spatial load values by a constant factor.

Parameters

<i>loadFactor_in</i>	Multiplicative factor to apply to every load entry.
----------------------	---

Definition at line 94 of file loadthermalboundary1D.cpp.

8.40.4 Member Data Documentation

8.40.4.1 loadmap `std::map<double, double> mknix::LoadThermalBoundary1D::loadmap [protected]`

Definition at line 53 of file loadthermalboundary1D.h.

8.40.4.2 timemap `std::map<double, double> mknix::LoadThermalBoundary1D::timemap [protected]`

Definition at line 54 of file loadthermalboundary1D.h.

The documentation for this class was generated from the following files:

- [loadthermalboundary1D.h](#)
- [loadthermalboundary1D.cpp](#)

8.41 mknix::Material Class Reference

```
#include <material.h>
```

Public Member Functions

- [Material](#) ()
Default constructor. Initializes all material properties to zero.
- [~Material](#) ()
Destructor for [Material](#).
- double [getE](#) ()
- double [getMu](#) ()

- double [getDensity](#) (double temp_in=0)
- double [getCapacity](#) (double temp_in=0)
- double [getKappa](#) (double temp_in=0)
- double [getBeta](#) (double temp_in=0)
- [Imx::DenseMatrix](#)< double > & [getD](#) ()
- [Imx::DenseMatrix](#)< double > & [getC](#) ()
- double [getPorosityResistance](#) ([Point](#) *)
Returns the porosity resistance at a given point, interpolating from tabulated data if available.
- double [computePorosityLoad](#) ([Point](#) *)
Computes the cooling power per unit volume from the porous medium and updates the fluid temperature.
- double [getPorosityFluidTemp](#) ()
- void [update](#) (int convergence)
Updates material state variables at the end of a nonlinear iteration or converged step.
- const std::vector< double > & [getFluidTempHistory](#) () const
- void [setPorosityFluidTemp](#) (double temp_in)
- bool [isPorous](#) ()
- void [setThermalProps](#) (double capacity_in, double kappa_in, double beta_in, double density_in)
Sets the thermal material properties.
- void [setPorosityProps](#) (double resistance_in, double fluid_temperature_in, double porosityCapacity_in=0.0)
Sets the porosity material properties and marks the material as porous.
- void [setMechanicalProps](#) (int dim_in, double young_in, double poisson_in, double density_in)
Sets the mechanical material properties and pre-computes the elastic stiffness matrices D and C.
- void [addThermalCapacity](#) (double temp_in, double capacity_in)
- void [addThermalConductivity](#) (double temp_in, double conductivity_in)
- void [addThermalExpansion](#) (double temp_in, double beta_in)
- void [addThermalDensity](#) (double temp_in, double density_in)
- void [addVariableResistance](#) (double distance_in, double resistance_in)
- void [addVariableResistance](#) (double key1_in, double key2_in, double resistance_in)
- void [setVariableResistanceCoords](#) (const std::string &coords_in)
- void [computeD](#) ()
Computes the small-strain elastic constitutive matrix D (Voigt notation, plane stress or 3D).
- void [computeC](#) ()
Computes the large-strain elastic constitutive tensor C in Voigt notation.
- double [getCsym](#) (int &i, int &j, int &k, int &l)
Returns the fully-symmetrized minor-symmetric component of the material tangent tensor.
- void [computeS](#) (cofe::TensorRank2Sym< 2, double > &S, const cofe::TensorRank2< 2, double > &F, double)
Computes the 2nd Piola-Kirchhoff stress S for a 2D deformation gradient using St. Venant-Kirchhoff.
- void [computeS](#) (cofe::TensorRank2Sym< 3, double > &S, const cofe::TensorRank2< 3, double > &F)
Computes the 2nd Piola-Kirchhoff stress S for a 3D deformation gradient using St. Venant-Kirchhoff.
- double [computeEnergy](#) (const cofe::TensorRank2< 2, double > &S)
Computes the strain energy density for a 2D deformation using the St. Venant-Kirchhoff model.
- double [computeEnergy](#) (const cofe::TensorRank2< 3, double > &S)
Computes the strain energy density for a 3D deformation using the St. Venant-Kirchhoff model.
- void [outputToFile](#) (std::ofstream &outFile)
Writes the material state (porosity flag, fluid temperature history) to an output file.

8.41.1 Detailed Description

Author

Daniel Iglesias

Definition at line 34 of file material.h.

8.41.2 Constructor & Destructor Documentation

8.41.2.1 **Material()** `mknix::Material::Material ( )`

Default constructor. Initializes all material properties to zero.
Definition at line 29 of file material.cpp.

8.41.2.2 **~Material()** `mknix::Material::~~Material ( )`

Destructor for [Material](#).
Definition at line 47 of file material.cpp.

8.41.3 Member Function Documentation

8.41.3.1 **addThermalCapacity()** `void mknix::Material::addThermalCapacity ( double temp_in, double capacity_in ) [inline]`

Definition at line 132 of file material.h.

8.41.3.2 **addThermalConductivity()** `void mknix::Material::addThermalConductivity ( double temp_in, double conductivity_in ) [inline]`

Definition at line 136 of file material.h.

8.41.3.3 **addThermalDensity()** `void mknix::Material::addThermalDensity ( double temp_in, double density_in ) [inline]`

Definition at line 144 of file material.h.

8.41.3.4 **addThermalExpansion()** `void mknix::Material::addThermalExpansion ( double temp_in, double beta_in ) [inline]`

Definition at line 140 of file material.h.

8.41.3.5 **addVariableResistance()** [1/2] `void mknix::Material::addVariableResistance ( double distance_in, double resistance_in ) [inline]`

Definition at line 148 of file material.h.

8.41.3.6 **addVariableResistance()** [2/2] `void mknix::Material::addVariableResistance ( double key1_in, double key2_in, double resistance_in ) [inline]`

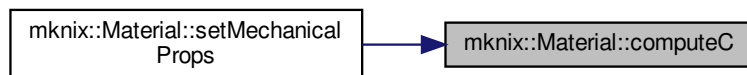
Definition at line 152 of file material.h.

8.41.3.7 computeC() `void mknix::Material::computeC ( )`

Computes the large-strain elastic constitutive tensor C in Voigt notation.

Definition at line 214 of file material.cpp.

Here is the caller graph for this function:

**8.41.3.8 computeD()** `void mknix::Material::computeD ( )`

Computes the small-strain elastic constitutive matrix D (Voigt notation, plane stress or 3D).

Definition at line 175 of file material.cpp.

Here is the caller graph for this function:

**8.41.3.9 computeEnergy()** `[1/2] double mknix::Material::computeEnergy (`
`const cofe::TensorRank2< 2, double > & F )`

Computes the strain energy density for a 2D deformation using the St. Venant-Kirchhoff model.

Parameters

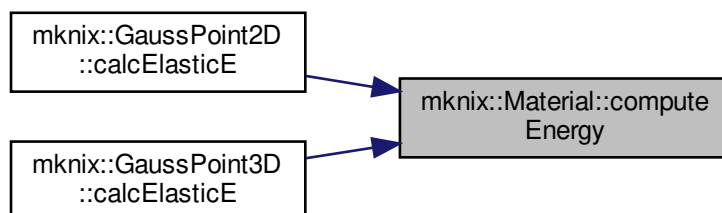
F	2D deformation gradient tensor.
-----	---------------------------------

Returns

Elastic strain energy density.

Definition at line 337 of file material.cpp.

Here is the caller graph for this function:



8.41.3.10 computeEnergy() [2/2] `double mknix::Material::computeEnergy (`
`const cofe::TensorRank2< 3, double > & F )`

Computes the strain energy density for a 3D deformation using the St. Venant-Kirchhoff model.

Parameters

F	3D deformation gradient tensor.
-----	---------------------------------

Returns

Elastic strain energy density.

Definition at line 370 of file material.cpp.

8.41.3.11 computePorosityLoad() `double mknix::Material::computePorosityLoad (`
`Point * point_in )`

Computes the cooling power per unit volume from the porous medium and updates the fluid temperature.

Parameters

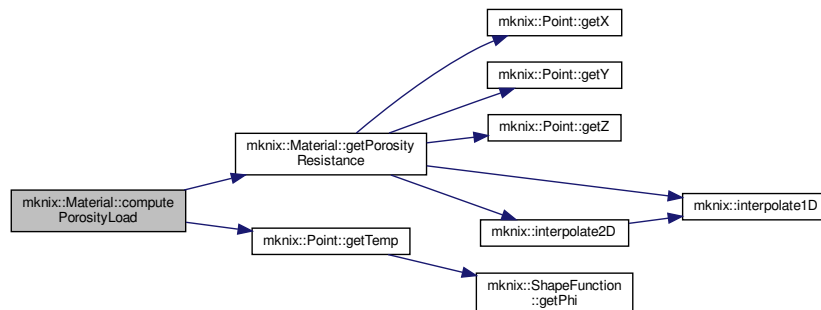
<code>point_in</code>	Pointer to the evaluation point (provides local temperature and coordinates).
-----------------------	---

Returns

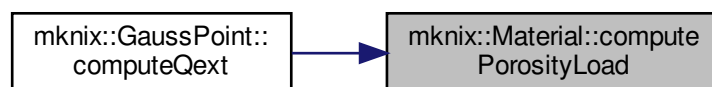
Cooling power: $(T_{\text{solid}} - T_{\text{fluid}}) / R_{\text{porosity}}$.

Definition at line 356 of file material.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.41.3.12 computeS() [1/2] `void mknix::Material::computeS (`
`cofe::TensorRank2Sym< 2, double > & S,`
`const cofe::TensorRank2< 2, double > & F,`
`double temperature_in )`

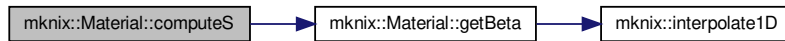
Computes the 2nd Piola-Kirchhoff stress S for a 2D deformation gradient using St. Venant-Kirchhoff.

Parameters

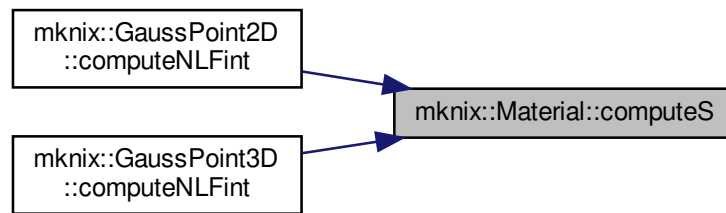
S	Output 2D symmetric stress tensor.
F	Input 2D deformation gradient tensor.
$temperature_{in}$	Current temperature (used for thermal expansion correction).

Definition at line 283 of file material.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.41.3.13 computeS() [2/2] `void mknix::Material::computeS (`
`cofe::TensorRank2Sym< 3, double > & S,`
`const cofe::TensorRank2< 3, double > & F )`

Computes the 2nd Piola-Kirchhoff stress S for a 3D deformation gradient using St. Venant-Kirchhoff.

Parameters

S	Output 3D symmetric stress tensor.
F	Input 3D deformation gradient tensor.

Definition at line 314 of file material.cpp.

8.41.3.14 getBeta() `double mknix::Material::getBeta (`
`double temp_in = 0 ) [inline]`

Definition at line 90 of file material.h.

Here is the call graph for this function:



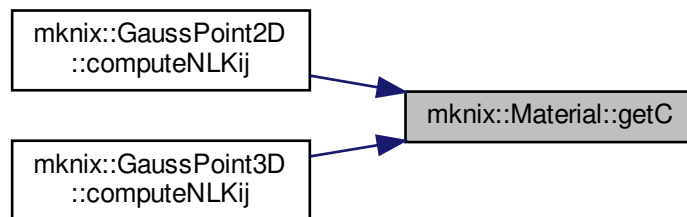
Here is the caller graph for this function:



8.41.3.15 `getC()` `lmx::DenseMatrix<double>& mknix::Material::getC ( ) [inline]`

Definition at line 99 of file material.h.

Here is the caller graph for this function:



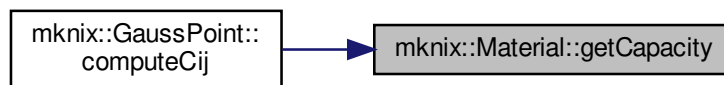
8.41.3.16 `getCapacity()` `double mknix::Material::getCapacity ( double temp_in = 0 ) [inline]`

Definition at line 80 of file material.h.

Here is the call graph for this function:



Here is the caller graph for this function:



8.41.3.17 getCsym() `double mknix::Material::getCsym (`
`int & i,`
`int & j,`
`int & k,`
`int & l )`

Returns the fully-symmetrized minor-symmetric component of the material tangent tensor.

Parameters

<i>i</i>	Row index (0-based).
<i>j</i>	Column index (0-based).
<i>k</i>	Second row index.
<i>l</i>	Second column index.

Returns

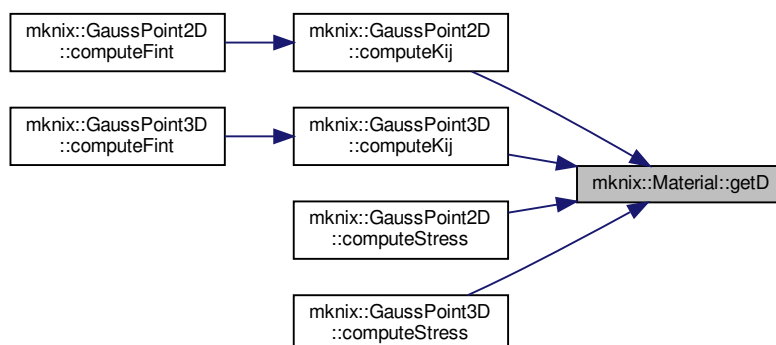
The value $0.25 \cdot (C_{ijkl} + C_{ijlk} + C_{jikl} + C_{jilk})$.

Definition at line 256 of file material.cpp.

8.41.3.18 getD() `lmx::DenseMatrix<double>& mknix::Material::getD ( ) [inline]`

Definition at line 95 of file material.h.

Here is the caller graph for this function:



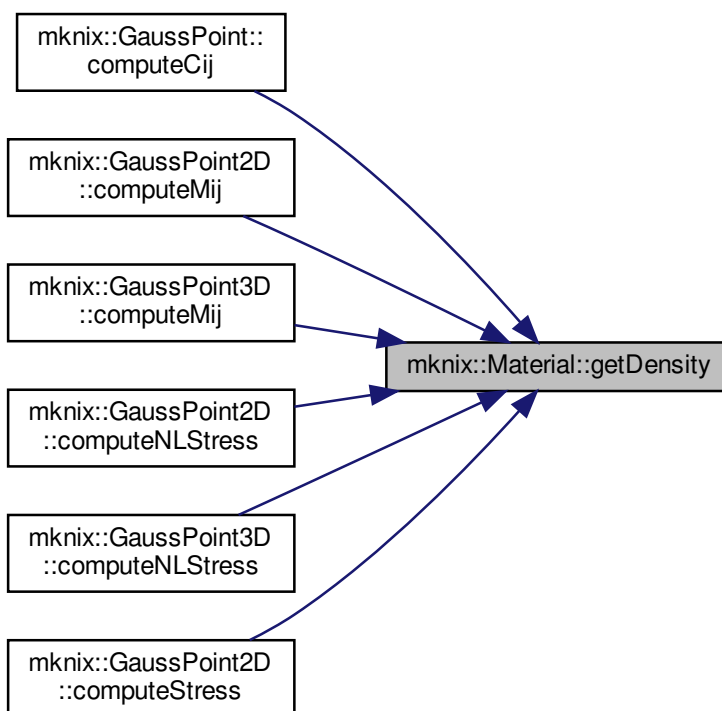
8.41.3.19 getDensity() `double mknix::Material::getDensity (`
`double temp_in = 0 ) [inline]`

Definition at line 75 of file material.h.

Here is the call graph for this function:



Here is the caller graph for this function:



8.41.3.20 getE() `double mknix::Material::getE ( ) [inline]`
Definition at line 67 of file material.h.

8.41.3.21 getFluidTempHistory() `const std::vector<double>& mknix::Material::getFluidTempHistory ( ) const [inline]`
Definition at line 113 of file material.h.

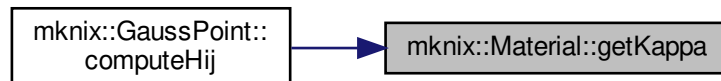
8.41.3.22 getKappa() `double mknix::Material::getKappa ( double temp_in = 0 ) [inline]`

Definition at line 85 of file material.h.

Here is the call graph for this function:



Here is the caller graph for this function:



8.41.3.23 getMu() `double mknix::Material::getMu ( ) [inline]`

Definition at line 71 of file material.h.

8.41.3.24 getPorosityFluidTemp() `double mknix::Material::getPorosityFluidTemp ( ) [inline]`

Definition at line 106 of file material.h.

8.41.3.25 getPorosityResistance() `double mknix::Material::getPorosityResistance ( Point * point_in )`

Returns the porosity resistance at a given point, interpolating from tabulated data if available.

Parameters

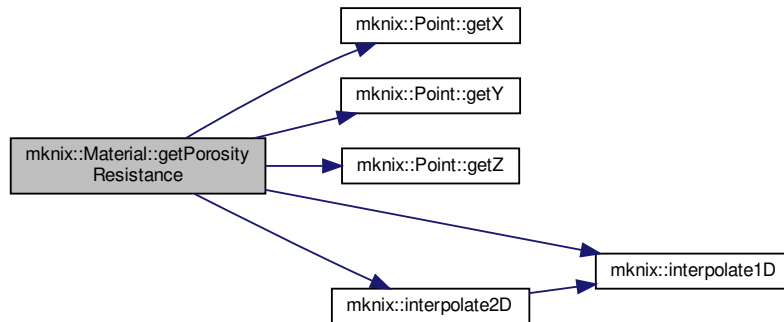
<i>point_in</i>	Pointer to the point at which to evaluate the resistance.
-----------------	---

Returns

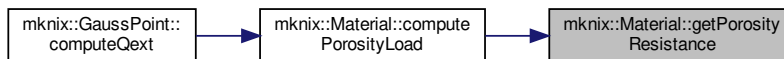
Porosity resistance value (scalar or interpolated from 1D/2D tables).

Definition at line 56 of file material.cpp.

Here is the call graph for this function:



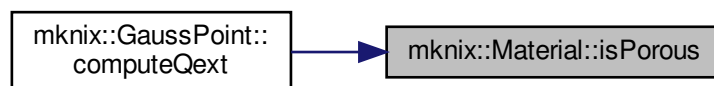
Here is the caller graph for this function:



8.41.3.26 isPorous() `bool mknix::Material::isPorous ( ) [inline]`

Definition at line 123 of file material.h.

Here is the caller graph for this function:



8.41.3.27 outputToFile() `void mknix::Material::outputToFile ( std::ofstream * outFile )`

Writes the material state (porosity flag, fluid temperature history) to an output file.

Parameters

<i>outFile</i>	Pointer to the open output file stream.
----------------	---

Definition at line 388 of file material.cpp.

8.41.3.28 setMechanicalProps() `void mknix::Material::setMechanicalProps (`
`int dim_in,`
`double young_in,`
`double poisson_in,`
`double density_in )`

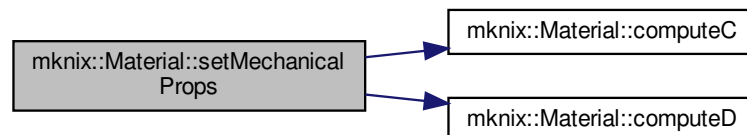
Sets the mechanical material properties and pre-computes the elastic stiffness matrices D and C.

Parameters

<i>dim_in</i>	Spatial dimension (2 for plane stress, 3 for full 3D).
<i>young_in</i>	Young's modulus.
<i>poisson_in</i>	Poisson's ratio.
<i>density_in</i>	Mass density.

Definition at line 141 of file material.cpp.

Here is the call graph for this function:



8.41.3.29 setPorosityFluidTemp() `void mknix::Material::setPorosityFluidTemp (`
`double temp_in ) [inline]`

Definition at line 118 of file material.h.

8.41.3.30 setPorosityProps() `void mknix::Material::setPorosityProps (`
`double resistance_in,`
`double fluid_temperature_in,`
`double porosityCapacity_in = 0.0 )`

Sets the porosity material properties and marks the material as porous.

Parameters

<i>resistance_in</i>	Volumetric thermal resistance of the porous medium.
<i>fluid_temperature_in</i>	Initial fluid temperature.
<i>porosityCapacity_in</i>	Thermal capacity of the fluid.

Definition at line 124 of file material.cpp.

8.41.3.31 setThermalProps() `void mknix::Material::setThermalProps (`
`double capacity_in,`
`double kappa_in,`
`double beta_in,`
`double density_in )`

Sets the thermal material properties.

Parameters

<i>capacity_in</i>	Specific heat capacity.
<i>kappa_in</i>	Thermal conductivity.
<i>beta_in</i>	Thermal expansion coefficient.
<i>density_in</i>	Mass density.

Definition at line 110 of file material.cpp.

8.41.3.32 setVariableResistanceCoords() `void mknix::Material::setVariableResistanceCoords (`
`const std::string & coords_in ) [inline]`

Definition at line 156 of file material.h.

8.41.3.33 update() `void mknix::Material::update (`
`int convergence )`

Updates material state variables at the end of a nonlinear iteration or converged step.

Parameters

<i>convergence</i>	1 if the step has converged (commit history), 0 to revert to the last converged state.
--------------------	--

Definition at line 157 of file material.cpp.

The documentation for this class was generated from the following files:

- [material.h](#)
- [material.cpp](#)

8.42 Imx::Matrix< T > Class Template Reference

8.42.1 Detailed Description

```
template<typename T>
class Imx::Matrix< T >
```

Definition at line 41 of file cell.h.

The documentation for this class was generated from the following file:

- [cell.h](#)

8.43 mknix::MknixWrapper Class Reference

```
#include <mknixWrapper.h>
```

Classes

- struct [SimulationWrapper](#)

Public Member Functions

- [MknixWrapper](#) (const std::string &input_file_name, const int input_detail=0)
- virtual [~MknixWrapper](#) ()
- void [init](#) (double temperatures)
- void [run](#) (double *heatFluence, double *temperatures)

8.43.1 Detailed Description

Definition at line 7 of file mknixWrapper.h.

8.43.2 Constructor & Destructor Documentation

8.43.2.1 MknixWrapper() `mknix::MknixWrapper::MknixWrapper (`
 `const std::string & input_file_name,`
 `const int input_detail = 0 )`

Definition at line 9 of file mknixWrapper.cpp.

8.43.2.2 ~MknixWrapper() `mknix::MknixWrapper::~~MknixWrapper ( )` [virtual], [default]

8.43.3 Member Function Documentation

8.43.3.1 init() `void mknix::MknixWrapper::init (`
 `double temperatures )`

Definition at line 21 of file mknixWrapper.cpp.

8.43.3.2 run() `void mknix::MknixWrapper::run (`
 `double * heatFluence,`
 `double * temperatures )`

Definition at line 26 of file mknixWrapper.cpp.

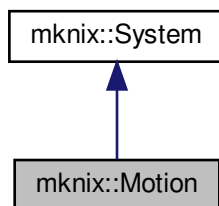
The documentation for this class was generated from the following files:

- [mknixWrapper.h](#)
- [mknixWrapper.cpp](#)

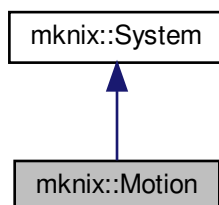
8.44 mknix::Motion Class Reference

```
#include <motion.h>
```

Inheritance diagram for mknix::Motion:



Collaboration diagram for mknix::Motion:



Public Member Functions

- [Motion](#) ()
Default constructor.
- [Motion](#) (Node *)
Constructor that binds a prescribed motion to a node.
- [~Motion](#) ()
Destructor.
- void [setNode](#) (Node *node_in)
- void [setTimeUx](#) (std::map< double, double > &timeUX_in)
- void [setTimeUy](#) (std::map< double, double > &timeUY_in)
- void [setTimeUz](#) (std::map< double, double > &timeUZ_in)
- void [update](#) (double)
Updates the prescribed displacements of the bound node by interpolating the stored time-displacement maps at the current simulation time.

Additional Inherited Members

8.44.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 33 of file motion.h.

8.44.2 Constructor & Destructor Documentation

8.44.2.1 Motion() [1/2] mknix::Motion::Motion ()

Default constructor.

Definition at line 30 of file motion.cpp.

8.44.2.2 Motion() [2/2] mknix::Motion::Motion (Node * node_in)

Constructor that binds a prescribed motion to a node.

Parameters

<i>node_in</i>	Pointer to the node whose position will be driven by this motion.
----------------	---

Definition at line 38 of file motion.cpp.

8.44.2.3 ~Motion() mknix::Motion::~Motion ()

Destructor.

Definition at line 50 of file motion.cpp.

8.44.3 Member Function Documentation

8.44.3.1 setNode() void mknix::Motion::setNode (Node * node_in) [inline]

Definition at line 43 of file motion.h.

8.44.3.2 setTimeUx() void mknix::Motion::setTimeUx (std::map< double, double > & timeUX_in) [inline]

Definition at line 49 of file motion.h.

8.44.3.3 setTimeUy() void mknix::Motion::setTimeUy (std::map< double, double > & timeUY_in) [inline]

Definition at line 54 of file motion.h.

8.44.3.4 setTimeUz() void mknix::Motion::setTimeUz (std::map< double, double > & timeUZ_in) [inline]

Definition at line 59 of file motion.h.

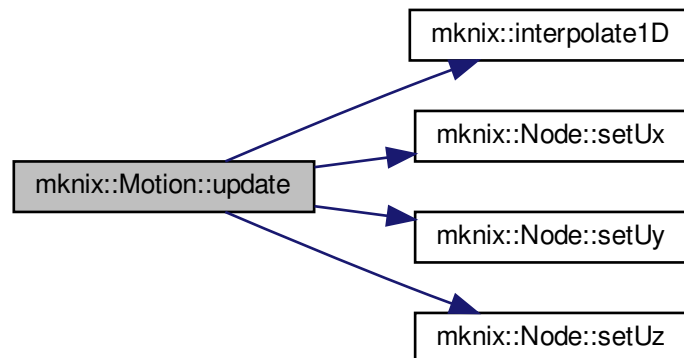
8.44.3.5 update() void mknix::Motion::update (double theTime) [virtual]

Updates the prescribed displacements of the bound node by interpolating the stored time-displacement maps at the current simulation time.

Parameters

<i>theTime</i>	Current simulation time.
----------------	--------------------------

Reimplemented from [mknix::System](#).
Definition at line 59 of file motion.cpp.
Here is the call graph for this function:



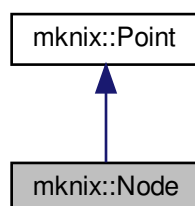
The documentation for this class was generated from the following files:

- [motion.h](#)
- [motion.cpp](#)

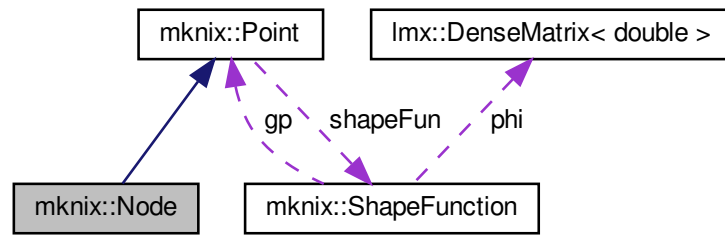
8.45 mknix::Node Class Reference

```
#include <node.h>
```

Inheritance diagram for `mknix::Node`:



Collaboration diagram for mknix::Node:



Public Member Functions

- [Node](#) ()
Default constructor for [Node](#).
- [Node](#) (const [Node](#) &)
Copy constructor. Copies position, displacement, and temperature from another [Node](#).
- [Node](#) (const [Node](#) *)
Pointer-based copy constructor. Copies position, displacement, and temperature from another [Node](#).
- [Node](#) (int i_in, double x_in, double y_in, double z_in)
Constructs a [Node](#) at a given position with initial displacement equal to its coordinates.
- [~Node](#) ()
Destructor for [Node](#).
- void [addWeight](#) (double w_in)
- const double & [getWeight](#) () const
- const int & [getThermalNumber](#) () const
- const void [setNumber](#) (int num_in)
- const void [setThermalNumber](#) (int num_in)
- const double & [getqt](#) () const
- double [getqx](#) (int i) const
- double [getUx](#) () const
- double [getUy](#) () const
- double [getUz](#) () const
- double [getU](#) (int i) const
- double [getConfig](#) (int dof) const override
Returns the current configuration coordinate for the given degree of freedom. For meshfree nodes (MLS, 1D, 2D, 3D), evaluates the interpolated value from support nodes; for FEM nodes, returns the stored displacement directly.
- double [getTemp](#) () const override
Returns the current temperature at this node. For MLS nodes, interpolates from support node temperatures; for FEM nodes, returns the stored temperature directly.
- size_t [getSupportSize](#) (int deriv)
Returns the number of support nodes for a given derivative order.
- int [getSupportNodeNumber](#) (int deriv, int s_node)
Returns the global node number of a support node at a given derivative order and index.
- double [getShapeFunValue](#) (int deriv, int s_node)
Returns the shape function value (phi) for a given derivative order and support node index.
- void [setUx](#) (double ux_in)
- void [setUy](#) (double uy_in)

- void [setUz](#) (double uz_in)
- void [setqx](#) (const [lmx::Vector](#)< [data_type](#) > &globalConf, int dim)
Updates the node's displacement components from a global configuration vector.
- void [setqt](#) (const [lmx::Vector](#)< [data_type](#) > &globalConf)
Updates the node's temperature from a global temperature vector.
- void [setqt](#) (double temp_in)
- void [setX](#) (double X_in)
- void [setY](#) (double Y_in)
- void [setZ](#) (double Z_in)

Additional Inherited Members

8.45.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 32 of file node.h.

8.45.2 Constructor & Destructor Documentation

8.45.2.1 [Node\(\)](#) [1/4] `mknix::Node::Node ( )`

Default constructor for [Node](#).

Definition at line 29 of file node.cpp.

8.45.2.2 [Node\(\)](#) [2/4] `mknix::Node::Node ( const Node & p_node_in )`

Copy constructor. Copies position, displacement, and temperature from another [Node](#).

Parameters

p_node_in	Reference to the source Node .
---------------------------	--

Definition at line 35 of file node.cpp.

8.45.2.3 [Node\(\)](#) [3/4] `mknix::Node::Node ( const Node * p_node_in )`

Pointer-based copy constructor. Copies position, displacement, and temperature from another [Node](#).

Parameters

p_node_in	Pointer to the source Node .
---------------------------	--

Definition at line 47 of file node.cpp.

8.45.2.4 [Node\(\)](#) [4/4] `mknix::Node::Node ( int i_in, double x_in, double y_in, double z_in )`

Constructs a [Node](#) at a given position with initial displacement equal to its coordinates.

Parameters

i_{in}	Global node index.
x_{in}	X coordinate.
y_{in}	Y coordinate.
z_{in}	Z coordinate.

Definition at line 62 of file node.cpp.

8.45.2.5 ~Node() `mknix::Node::~~Node ( )`

Destructor for [Node](#).

Definition at line 75 of file node.cpp.

8.45.3 Member Function Documentation

8.45.3.1 addWeight() `void mknix::Node::addWeight ( double w_in ) [inline]`

Definition at line 45 of file node.h.

8.45.3.2 getConf() `double mknix::Node::getConf ( int gdl ) const [override], [virtual]`

Returns the current configuration coordinate for the given degree of freedom. For meshfree nodes (MLS, 1D, 2D, 3D), evaluates the interpolated value from support nodes; for FEM nodes, returns the stored displacement directly.

Parameters

gdl	Degree of freedom index (0 = x, 1 = y, 2 = z).
-------	--

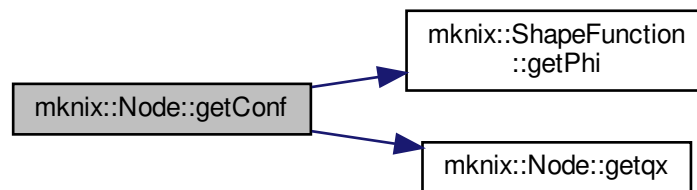
Returns

Interpolated or direct configuration value.

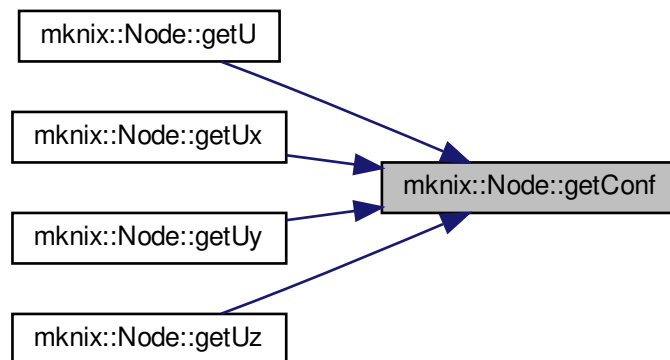
Reimplemented from [mknix::Point](#).

Definition at line 87 of file node.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.45.3.3 `getqt()` `const double& mknix::Node::getqt ( ) const [inline]`

Definition at line 82 of file `node.h`.

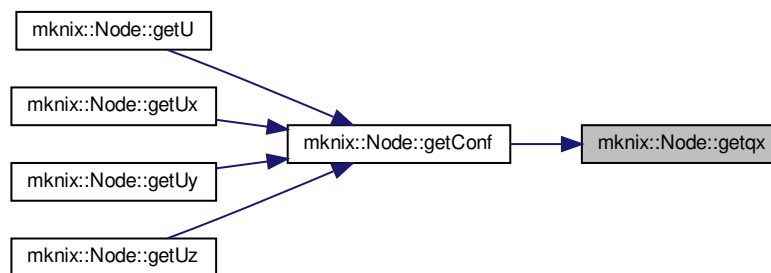
Here is the caller graph for this function:



8.45.3.4 `getqx()` `double mknix::Node::getqx ( int i ) const [inline]`

Definition at line 87 of file node.h.

Here is the caller graph for this function:



8.45.3.5 getShapeFunValue() `double mknix::Node::getShapeFunValue (`
 `int deriv,`
 `int s_node )`

Returns the shape function value (phi) for a given derivative order and support node index.

Parameters

<i>deriv</i>	Derivative order (0 = value, 1 = dx, 2 = dy, etc.).
<i>s_node</i>	Index of the support node within the local support set.

Returns

Shape function value; 1.0 for standard FEM nodes at derivative order 0.

Definition at line 227 of file node.cpp.

Here is the call graph for this function:



8.45.3.6 getSupportNodeNumber() `int mknix::Node::getSupportNodeNumber (`
 `int deriv,`
 `int s_node ) [virtual]`

Returns the global node number of a support node at a given derivative order and index.

Parameters

<i>deriv</i>	Derivative order.
<i>s_node</i>	Index of the support node within the local support set.

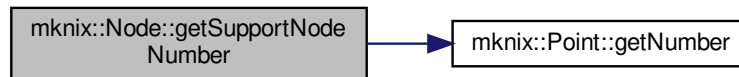
Returns

Global node number of the support node.

Reimplemented from [mknix::Point](#).

Definition at line 186 of file node.cpp.

Here is the call graph for this function:



8.45.3.7 getSupportSize() `size_t mknix::Node::getSupportSize ( int deriv ) [virtual]`

Returns the number of support nodes for a given derivative order.

Parameters

<i>deriv</i>	Derivative order (0 = value, >0 = derivative).
--------------	--

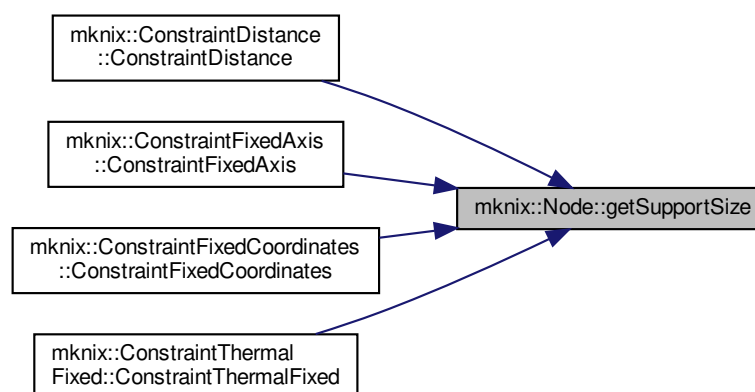
Returns

Number of support nodes, or 1 for standard FEM nodes.

Reimplemented from [mknix::Point](#).

Definition at line 147 of file node.cpp.

Here is the caller graph for this function:



8.45.3.8 getTemp() `double mknix::Node::getTemp ( ) const [override], [virtual]`

Returns the current temperature at this node. For MLS nodes, interpolates from support node temperatures; for FEM nodes, returns the stored temperature directly.

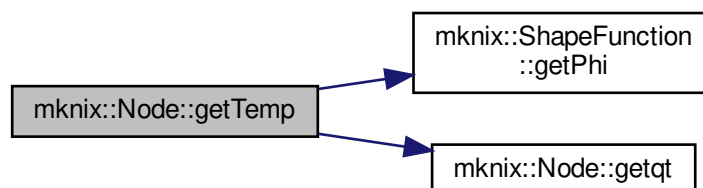
Returns

Temperature value.

Reimplemented from [mknix::Point](#).

Definition at line 119 of file node.cpp.

Here is the call graph for this function:



8.45.3.9 getThermalNumber() `const int& mknix::Node::getThermalNumber ( ) const [inline]`

Definition at line 55 of file node.h.

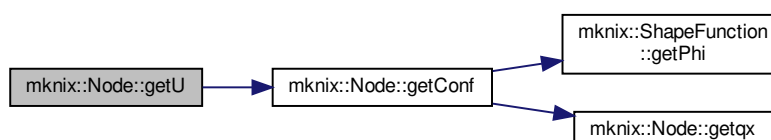
Here is the caller graph for this function:



8.45.3.10 getU() `double mknix::Node::getU ( int i ) const [inline]`

Definition at line 117 of file node.h.

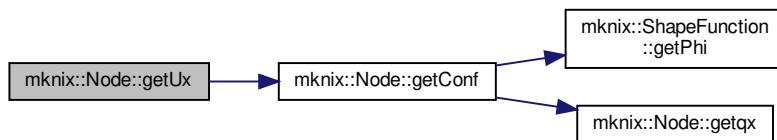
Here is the call graph for this function:



8.45.3.11 getUx() `double mknix::Node::getUx ( ) const [inline]`

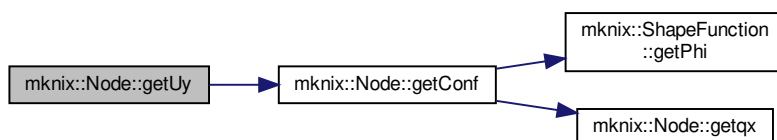
Definition at line 102 of file node.h.

Here is the call graph for this function:

**8.45.3.12 getUy()** `double mknix::Node::getUy ( ) const [inline]`

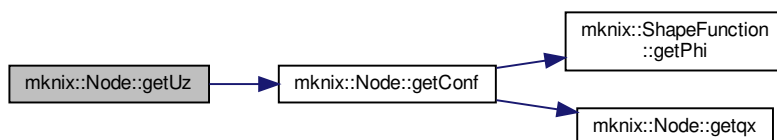
Definition at line 107 of file node.h.

Here is the call graph for this function:

**8.45.3.13 getUz()** `double mknix::Node::getUz ( ) const [inline]`

Definition at line 112 of file node.h.

Here is the call graph for this function:

**8.45.3.14 getWeight()** `const double& mknix::Node::getWeight ( ) const [inline]`

Definition at line 50 of file node.h.

8.45.3.15 setNumber() `const void mknix::Node::setNumber (`
`int num_in ) [inline]`

Definition at line 60 of file node.h.

8.45.3.16 setqt() [1/2] `void mknix::Node::setqt ( const lmx::Vector< data_type > & globalTemp )`

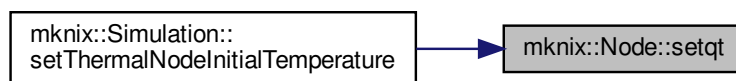
Updates the node's temperature from a global temperature vector.

Parameters

<i>globalTemp</i>	Global temperature state vector.
-------------------	----------------------------------

Definition at line 263 of file node.cpp.

Here is the caller graph for this function:



8.45.3.17 setqt() [2/2] `void mknix::Node::setqt ( double temp_in ) [inline]`

Definition at line 162 of file node.h.

8.45.3.18 setqx() `void mknix::Node::setqx ( const lmx::Vector< data_type > & globalConf, int dim )`

Updates the node's displacement components from a global configuration vector.

Parameters

<i>globalConf</i>	Global displacement state vector.
<i>dim</i>	Spatial dimension (2 or 3).

Definition at line 249 of file node.cpp.

8.45.3.19 setThermalNumber() `const void mknix::Node::setThermalNumber ( int num_in ) [inline]`

Definition at line 65 of file node.h.

8.45.3.20 setUx() `void mknix::Node::setUx ( double ux_in ) [inline]`

Definition at line 143 of file node.h.

Here is the caller graph for this function:



8.45.3.21 setUy() `void mknix::Node::setUy (`
 `double uy_in ) [inline]`

Definition at line 148 of file node.h.

Here is the caller graph for this function:



8.45.3.22 setUz() `void mknix::Node::setUz (`
 `double uz_in ) [inline]`

Definition at line 153 of file node.h.

Here is the caller graph for this function:



8.45.3.23 setX() `void mknix::Node::setX (`
 `double X_in ) [inline]`

Definition at line 179 of file node.h.

8.45.3.24 setY() `void mknix::Node::setY (`
 `double Y_in ) [inline]`

Definition at line 185 of file node.h.

8.45.3.25 setZ() void mknix::Node::setZ (
 double Z_in) [inline]

Definition at line 191 of file node.h.

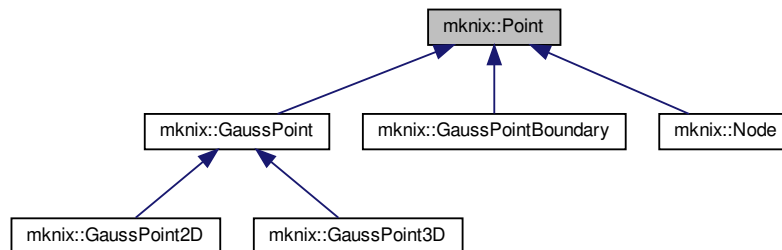
The documentation for this class was generated from the following files:

- [node.h](#)
- [node.cpp](#)

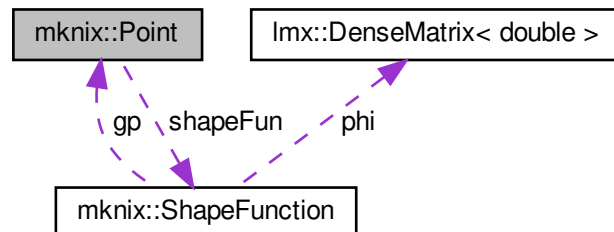
8.46 mknix::Point Class Reference

```
#include <point.h>
```

Inheritance diagram for mknix::Point:



Collaboration diagram for mknix::Point:



Public Member Functions

- [Point](#) ()
Default constructor for [Point](#).
- [Point](#) (const [Point](#) &point_in)
Copy constructor. Copies all fields from another [Point](#).
- [Point](#) (const [Point](#) *point_in)
Pointer-based copy constructor.
- [Point](#) (int i, double coor_x, double coor_y, double coor_z)
Constructs a [Point](#) at a given (x, y, z) position using the global simulation dimension.
- [Point](#) (int dim, int i, double coor_x, double coor_y, double coor_z, double alpha_in, double dc_in)
Constructs a [Point](#) with explicit dimension, index, coordinates, and meshfree parameters.

- `Point` (int `dim`, int `i`, double `coor_x`, double `coor_y`, double `coor_z`, double `jacobian_in`, double `alpha_in`, double `dc_in`)

Constructs a Gauss-point-style `Point` with an explicit Jacobian value.

- virtual `~Point` ()

Destructor. Frees the shape function object if one was allocated.

- const int & `getDim` () const
- const double & `getX` () const
- const double & `getY` () const
- const double & `getZ` () const
- const int & `getNumber` () const
- virtual double `getTemp` () const

Computes the interpolated temperature at this point from its support nodes.

- virtual double `getConf` (int) const

Computes the interpolated configuration coordinate for a given DOF from support nodes.

- double `distance` (`Point` &) const

Computes the Euclidean distance from this point to another.

- void `setShapeFunType` (std::string `type_in`)
- std::string `getShapeFunType` ()
- virtual size_t `getSupportSize` (int `deriv`=0)
- virtual int `getSupportNodeNumber` (int `deriv`, int `s_node`)

Returns the global node number of a support node by derivative order and index.

- void `setDc` (double `dc_in`)
- void `setAlphai` (double `alpha_in`)
- void `addSupportNode` (`Node` *)

Adds a single node to this point's support neighbourhood.

- void `findSupportNodes` (std::vector< `Node` * > &)

*Finds and stores all domain nodes within the influence radius ($\alpha * dc$) of this point.*

- void `findSupportNodes` (std::vector< `Node` * > &, double, double, double, double)

Finds support nodes within the influence radius, clamping the search box to a rectangular domain.

- virtual void `shapeFunSolve` (std::string, double)

Computes and stores shape function values at this point using the specified method.

- void `shapeFunSolve` (double `q_in`)
- void `setJacobian` (double `jac_in`)
- void `gnuplotOut` (std::ofstream &)

Writes the point coordinates, support node positions, and shape function values to gnuplot-compatible files for visualization.

- const std::vector< `Node` * > & `getSupportNodes` () const

Protected Attributes

- int `dim`
- int `num`
- double `X`
- double `Y`
- double `Z`
- double `alphai`
- double `dc`
- std::string `shapeFunType`
- `ShapeFunction` * `shapeFun`
- std::vector< `Node` * > `supportNodes`
- size_t `supportNodesSize`
- double `jacobian`

Friends

- class [GaussPoint](#)
- class [ShapeFunction](#)
- class [ShapeFunctionRBF](#)
- class [ShapeFunctionMLS](#)
- class [ShapeFunctionMLS2D](#)
- class [ShapeFunctionMLS3D](#)
- class [ShapeFunctionTetrahedron](#)
- class [ShapeFunctionTriangle](#)
- class [ShapeFunctionLinear](#)
- class [ShapeFunctionLinearX](#)

8.46.1 Detailed Description

Author

Daniel Iglesias

Definition at line 55 of file `point.h`.

8.46.2 Constructor & Destructor Documentation

8.46.2.1 `Point()` [1/6] `mknix::Point::Point ( )`

Default constructor for [Point](#).

Definition at line 19 of file `point.cpp`.

8.46.2.2 `Point()` [2/6] `mknix::Point::Point ( const Point & point_in )`

Copy constructor. Copies all fields from another [Point](#).

Parameters

<code>point↵ _in</code>	Reference to the source Point .
-----------------------------	---

Definition at line 27 of file `point.cpp`.

8.46.2.3 `Point()` [3/6] `mknix::Point::Point ( const Point * point_in )`

Pointer-based copy constructor.

Parameters

<code>point↵ _in</code>	Pointer to the source Point .
-----------------------------	---

Definition at line 47 of file `point.cpp`.

8.46.2.4 `Point()` [4/6] `mknix::Point::Point ( int i, double coor_x, double coor_y,`

```
double coor_z )
```

Constructs a [Point](#) at a given (x, y, z) position using the global simulation dimension.

Parameters

<i>i</i>	Global point index.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>coor_z</i>	Z coordinate.

Definition at line 70 of file point.cpp.

```
8.46.2.5 Point() [5/6] mknix::Point::Point (
    int dim_in,
    int i,
    double coor_x,
    double coor_y,
    double coor_z,
    double alpha_in,
    double dc_in )
```

Constructs a [Point](#) with explicit dimension, index, coordinates, and meshfree parameters.

Parameters

<i>dim_in</i>	Spatial dimension.
<i>i</i>	Global point index.
<i>coor_x</i>	X coordinate.
<i>coor_y</i>	Y coordinate.
<i>coor_z</i>	Z coordinate.
<i>alpha_{in}</i>	Influence radius scaling factor.
<i>dc_in</i>	Characteristic nodal spacing.

Definition at line 98 of file point.cpp.

```
8.46.2.6 Point() [6/6] mknix::Point::Point (
    int dim_in,
    int i,
    double coor_x,
    double coor_y,
    double coor_z,
    double jacobian_in,
    double alpha_in,
    double dc_in )
```

Constructs a Gauss-point-style [Point](#) with an explicit Jacobian value.

Parameters

<i>dim_in</i>	Spatial dimension.
<i>i</i>	Global point index.
<i>coor_x</i>	X coordinate.

Parameters

<i>coord_y</i>	Y coordinate.
<i>coord_z</i>	Z coordinate.
<i>jacobian_in</i>	Jacobian of the cell mapping.
<i>alpha_in</i>	Influence radius scaling factor.
<i>dc_in</i>	Characteristic nodal spacing.

Definition at line 130 of file point.cpp.

8.46.2.7 ~Point() `mknix::Point::~~Point ( ) [virtual]`
 Destructor. Frees the shape function object if one was allocated.
 Definition at line 155 of file point.cpp.

8.46.3 Member Function Documentation

8.46.3.1 addSupportNode() `void mknix::Point::addSupportNode (
 Node * node_in)`

Adds a single node to this point's support neighbourhood.

Parameters

<i>node_in</i>	Pointer to the node to add.
----------------	-----------------------------

Definition at line 220 of file point.cpp.

8.46.3.2 distance() `double mknix::Point::distance (
 Point & node_in) const`

Computes the Euclidean distance from this point to another.

Parameters

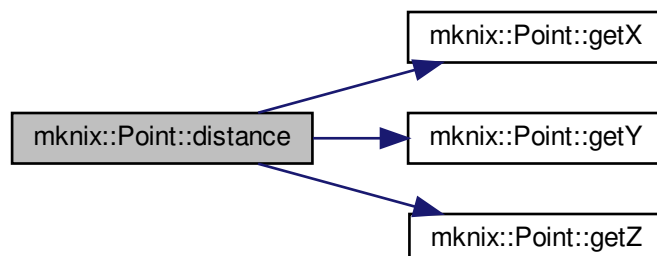
<i>node_in</i>	Reference to the other point.
----------------	-------------------------------

Returns

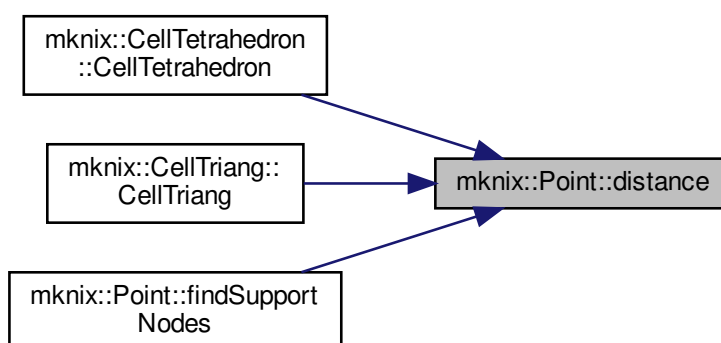
Euclidean distance.

Definition at line 197 of file point.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.3.3 findSupportNodes() [1/2] `void mknix::Point::findSupportNodes ( std::vector< Node * > & domainNodes )`

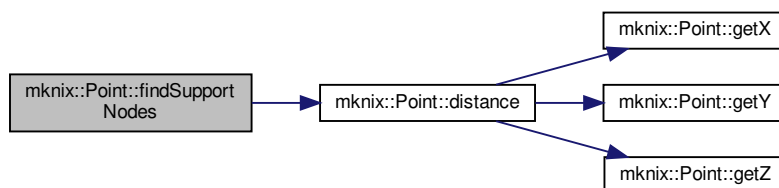
Finds and stores all domain nodes within the influence radius ($\alpha * dc$) of this point.

Parameters

<i>domainNodes</i>	Vector of all candidate domain nodes.
--------------------	---------------------------------------

Definition at line 231 of file point.cpp.

Here is the call graph for this function:



8.46.3.4 findSupportNodes() [2/2] `void mknix::Point::findSupportNodes (`
`std::vector< Node * > & domainNodes,`
`double domain_minX,`
`double domain_maxX,`
`double domain_minY,`
`double domain_maxY )`

Finds support nodes within the influence radius, clamping the search box to a rectangular domain.

Parameters

<i>domainNodes</i>	Vector of all candidate domain nodes.
<i>domain_minX,domain_maxX</i>	X extent of the rectangular domain.
<i>domain_minY,domain_maxY</i>	Y extent of the rectangular domain.

Definition at line 309 of file point.cpp.

8.46.3.5 getConf() `double mknix::Point::getConf (`
`int dof ) const [virtual]`

Computes the interpolated configuration coordinate for a given DOF from support nodes.

Parameters

<i>dof</i>	Degree of freedom index (0 = x, 1 = y, 2 = z).
------------	--

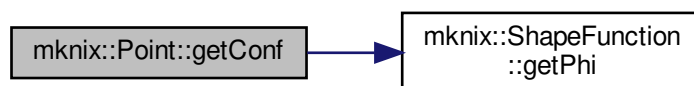
Returns

Interpolated configuration value.

Reimplemented in [mknix::Node](#).

Definition at line 182 of file point.cpp.

Here is the call graph for this function:



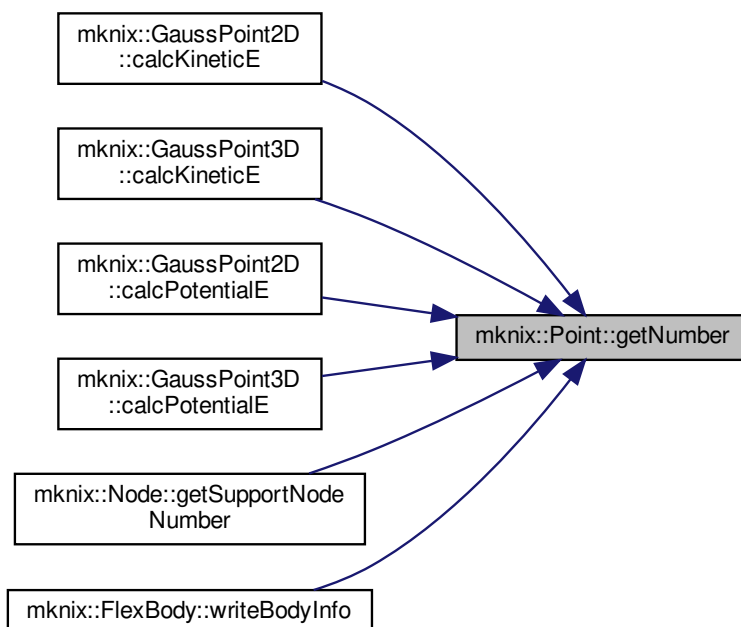
8.46.3.6 `getDim()` `const int& mknix::Point::getDim ( ) const [inline]`

Definition at line 97 of file point.h.

8.46.3.7 `getNumber()` `const int& mknix::Point::getNumber ( ) const [inline]`

Definition at line 117 of file point.h.

Here is the caller graph for this function:



8.46.3.8 getShapeFunType() `std::string mknix::Point::getShapeFunType ( ) [inline]`
 Definition at line 133 of file point.h.

8.46.3.9 getSupportNodeNumber() `int mknix::Point::getSupportNodeNumber (`
 `int deriv,`
 `int s_node ) [virtual]`

Returns the global node number of a support node by derivative order and index.

Parameters

<i>deriv</i>	Derivative order (currently unused; all derivatives use the same support).
<i>s_node</i>	Index within the support node set.

Returns

Global node number.

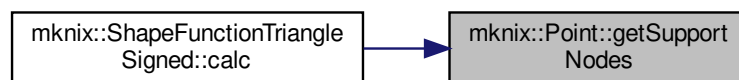
Reimplemented in [mknix::Node](#).

Definition at line 210 of file point.cpp.

8.46.3.10 getSupportNodes() `const std::vector<Node*>& mknix::Point::getSupportNodes ( ) const [inline]`

Definition at line 178 of file point.h.

Here is the caller graph for this function:



8.46.3.11 getSupportSize() `virtual size_t mknix::Point::getSupportSize (`
 `int deriv = 0 ) [inline], [virtual]`

Reimplemented in [mknix::Node](#).

Definition at line 138 of file point.h.

8.46.3.12 getTemp() `double mknix::Point::getTemp ( ) const [virtual]`

Computes the interpolated temperature at this point from its support nodes.

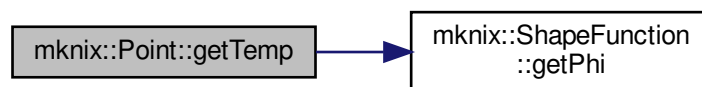
Returns

Interpolated temperature value.

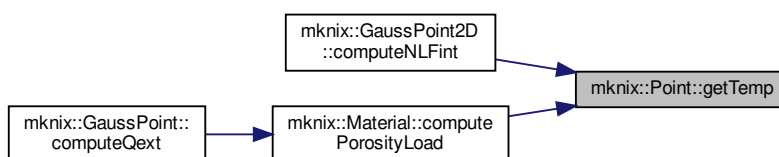
Reimplemented in [mknix::Node](#).

Definition at line 167 of file point.cpp.

Here is the call graph for this function:

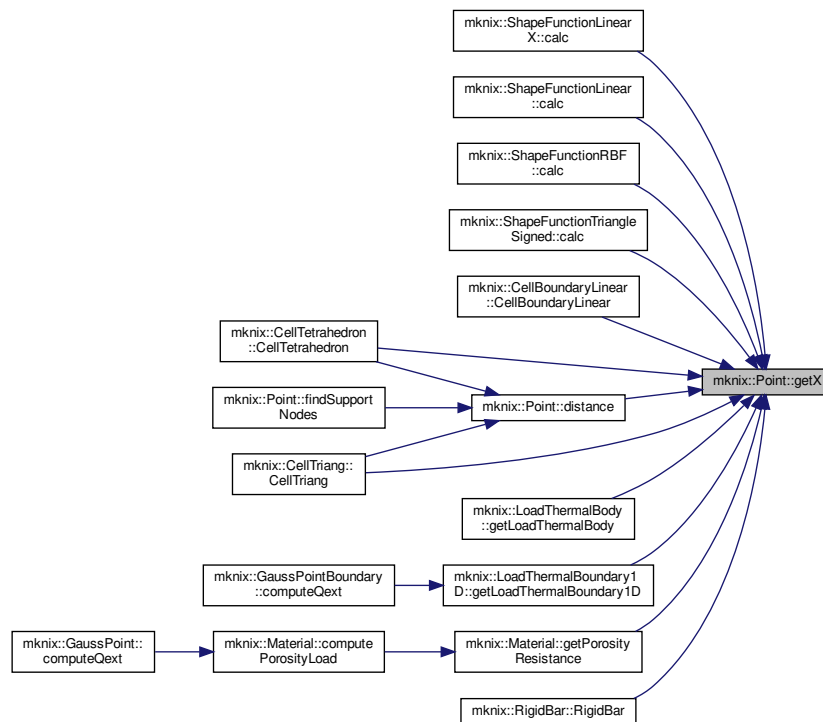


Here is the caller graph for this function:



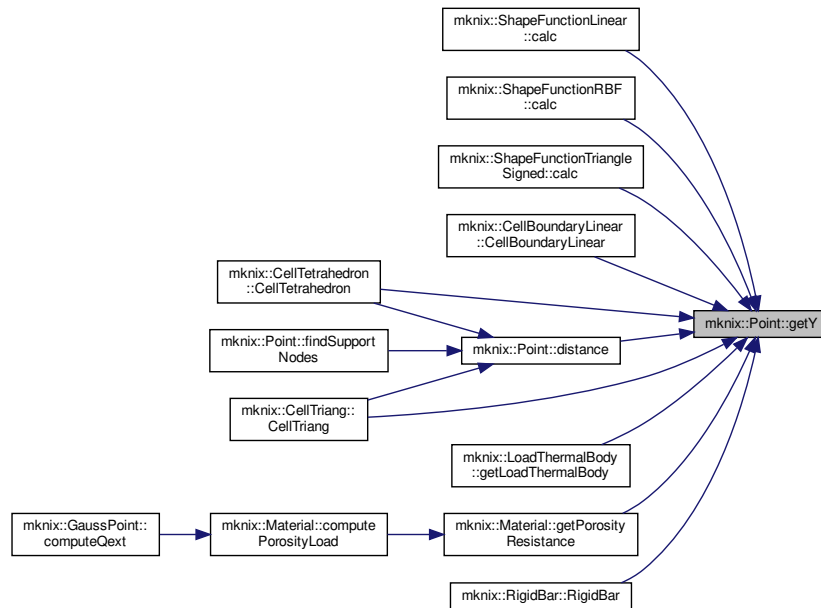
8.46.3.13 `getX()` `const double& mknix::Point::getX ( ) const [inline]`
Definition at line 102 of file `point.h`.

Here is the caller graph for this function:



8.46.3.14 getY() `const double& mknix::Point::getY ( ) const [inline]`
 Definition at line 107 of file point.h.

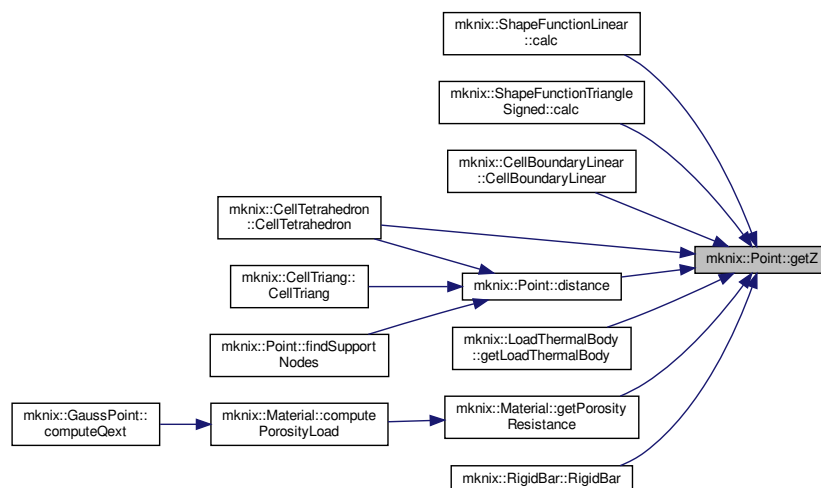
Here is the caller graph for this function:



8.46.3.15 `getZ()` `const double& mknix::Point::getZ ( ) const [inline]`

Definition at line 112 of file `point.h`.

Here is the caller graph for this function:



8.46.3.16 `gnuplotOut()` `void mknix::Point::gnuplotOut ( std::ofstream & gpdata )`

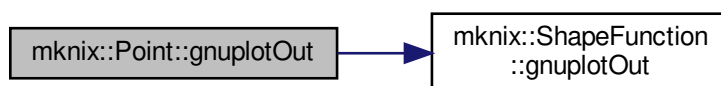
Writes the point coordinates, support node positions, and shape function values to gnuplot-compatible files for visualization.

Parameters

<i>gpdata</i>	Output file stream collecting all Gauss point coordinates.
---------------	--

Definition at line 435 of file point.cpp.

Here is the call graph for this function:



8.46.3.17 setAlphai() `void mknix::Point::setAlphai ( double alphai_in ) [inline]`

Definition at line 151 of file point.h.

8.46.3.18 setDc() `void mknix::Point::setDc ( double dc_in ) [inline]`

Definition at line 145 of file point.h.

8.46.3.19 setJacobian() `void mknix::Point::setJacobian ( double jac_in ) [inline]`

Definition at line 171 of file point.h.

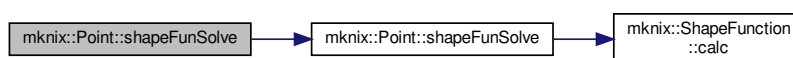
8.46.3.20 setShapeFunType() `void mknix::Point::setShapeFunType ( std::string type_in ) [inline]`

Definition at line 128 of file point.h.

8.46.3.21 shapeFunSolve() [1/2] `void mknix::Point::shapeFunSolve ( double q_in ) [inline]`

Definition at line 166 of file point.h.

Here is the call graph for this function:



8.46.3.22 shapeFunSolve() [2/2] `void mknix::Point::shapeFunSolve (`
`std::string type_in,`
`double q_in ) [virtual]`

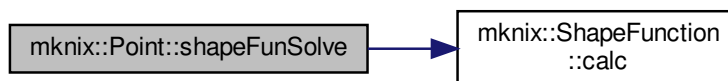
Computes and stores shape function values at this point using the specified method.

Parameters

<code>type_in</code>	Method string: "RBF", "MLS", "1D", "1D-X", "2D", or "3D".
<code>q_in</code>	Shape parameter for RBF (overridden to 0.5 for RBF/MLS).

Reimplemented in [mknix::GaussPointBoundary](#), [mknix::GaussPoint3D](#), [mknix::GaussPoint2D](#), and [mknix::GaussPoint](#).
Definition at line 375 of file point.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.46.4 Friends And Related Function Documentation

8.46.4.1 GaussPoint `friend class GaussPoint [friend]`

Definition at line 60 of file point.h.

8.46.4.2 ShapeFunction `friend class ShapeFunction [friend]`

Definition at line 62 of file point.h.

8.46.4.3 ShapeFunctionLinear `friend class ShapeFunctionLinear [friend]`

Definition at line 76 of file point.h.

8.46.4.4 ShapeFunctionLinearX `friend class ShapeFunctionLinearX [friend]`

Definition at line 78 of file point.h.

8.46.4.5 ShapeFunctionMLS friend class [ShapeFunctionMLS](#) [friend]
Definition at line 66 of file point.h.

8.46.4.6 ShapeFunctionMLS2D friend class [ShapeFunctionMLS2D](#) [friend]
Definition at line 68 of file point.h.

8.46.4.7 ShapeFunctionMLS3D friend class [ShapeFunctionMLS3D](#) [friend]
Definition at line 70 of file point.h.

8.46.4.8 ShapeFunctionRBF friend class [ShapeFunctionRBF](#) [friend]
Definition at line 64 of file point.h.

8.46.4.9 ShapeFunctionTetrahedron friend class [ShapeFunctionTetrahedron](#) [friend]
Definition at line 72 of file point.h.

8.46.4.10 ShapeFunctionTriangle friend class [ShapeFunctionTriangle](#) [friend]
Definition at line 74 of file point.h.

8.46.5 Member Data Documentation

8.46.5.1 alphai double mknix::Point::alphai [protected]
Definition at line 187 of file point.h.

8.46.5.2 dc double mknix::Point::dc [protected]
Definition at line 188 of file point.h.

8.46.5.3 dim int mknix::Point::dim [protected]
Definition at line 184 of file point.h.

8.46.5.4 jacobian double mknix::Point::jacobian [protected]
Definition at line 193 of file point.h.

8.46.5.5 num int mknix::Point::num [protected]
Definition at line 185 of file point.h.

8.46.5.6 shapeFun [ShapeFunction*](#) mknix::Point::shapeFun [protected]
Definition at line 190 of file point.h.

8.46.5.7 shapeFunType std::string mknix::Point::shapeFunType [protected]
Definition at line 189 of file point.h.

8.46.5.8 supportNodes `std::vector<Node*> mknix::Point::supportNodes` [protected]
Definition at line 191 of file point.h.

8.46.5.9 supportNodesSize `size_t mknix::Point::supportNodesSize` [protected]
Definition at line 192 of file point.h.

8.46.5.10 X `double mknix::Point::X` [protected]
Definition at line 186 of file point.h.

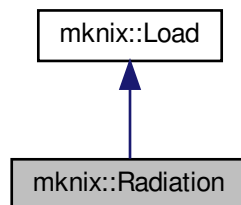
8.46.5.11 Y `double mknix::Point::Y` [protected]
Definition at line 186 of file point.h.

8.46.5.12 Z `double mknix::Point::Z` [protected]
Definition at line 186 of file point.h.
The documentation for this class was generated from the following files:

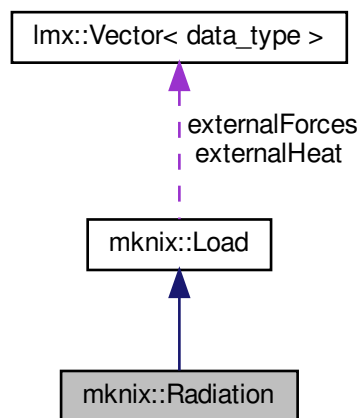
- [point.h](#)
- [point.cpp](#)

8.47 mknix::Radiation Class Reference

`#include <loadradiation.h>`
Inheritance diagram for mknix::Radiation:



Collaboration diagram for mknix::Radiation:



Public Member Functions

- [Radiation](#) ()
Default constructor.
- [~Radiation](#) ()
Destructor.
- void [addVoxel](#) (double, double, double, double)
Stores a radiation voxel value at the given spatial coordinates.
- void [outputToFile](#) (std::ofstream *)
Writes the radiation map (regular voxel grid) to the output file.

Additional Inherited Members

8.47.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 33 of file loadradiation.h.

8.47.2 Constructor & Destructor Documentation

8.47.2.1 Radiation() `mknix::Radiation::Radiation ( )`

Default constructor.

Definition at line 28 of file loadradiation.cpp.

8.47.2.2 ~Radiation() `mknix::Radiation::~~Radiation ( )`

Destructor.

Definition at line 36 of file loadradiation.cpp.

8.47.3 Member Function Documentation

8.47.3.1 addVoxel() `void mknix::Radiation::addVoxel (`
 `double x_in,`
 `double y_in,`
 `double z_in,`
 `double value_in )`

Stores a radiation voxel value at the given spatial coordinates.

Parameters

<code>x_in</code>	X coordinate of the voxel.
<code>y_in</code>	Y coordinate of the voxel.
<code>z_in</code>	Z coordinate of the voxel.
<code>value_in</code>	Radiation intensity value at this voxel.

Definition at line 48 of file loadradiation.cpp.

8.47.3.2 outputToFile() `void mknix::Radiation::outputToFile (`
 `std::ofstream * outFile ) [virtual]`

Writes the radiation map (regular voxel grid) to the output file.

Parameters

<code>outFile</code>	Pointer to the output file stream.
----------------------	------------------------------------

Implements [mknix::Load](#).

Definition at line 58 of file loadradiation.cpp.

The documentation for this class was generated from the following files:

- [loadradiation.h](#)
- [loadradiation.cpp](#)

8.48 mknix::Reader Class Reference

```
#include <reader.h>
```

Public Member Functions

- [Reader](#) ()
Default constructor. Opens the "output.reader" log file.
- [Reader](#) ([Simulation](#) *)
Constructs a [Reader](#) and associates it with the given [Simulation](#) instance.
- [~Reader](#) ()
Destructor. Cleans up dynamically allocated reader objects.
- void [inputFromFile](#) (const std::string &fileIn)
Parses the main input file, dispatching each top-level keyword to the appropriate reader.

8.48.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 51 of file reader.h.

8.48.2 Constructor & Destructor Documentation

8.48.2.1 Reader() [1/2] `mknix::Reader::Reader ( )`

Default constructor. Opens the "output.reader" log file.
Definition at line 72 of file reader.cpp.

8.48.2.2 Reader() [2/2] `mknix::Reader::Reader ( Simulation * simulation_in )`

Constructs a [Reader](#) and associates it with the given [Simulation](#) instance.

Parameters

<i>simulation</i> ↔ _in	Pointer to the active Simulation to populate.
----------------------------	---

Definition at line 84 of file reader.cpp.

8.48.2.3 ~Reader() `mknix::Reader::~~Reader ( )`

Destructor. Cleans up dynamically allocated reader objects.
Definition at line 160 of file reader.cpp.

8.48.3 Member Function Documentation

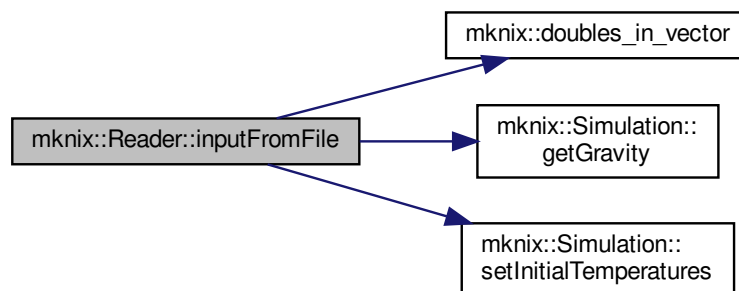
8.48.3.1 inputFromFile() `void mknix::Reader::inputFromFile ( const std::string & fileIn )`

Parses the main input file, dispatching each top-level keyword to the appropriate reader.

Parameters

<i>file</i> ↔ In	Path to the input file to read.
---------------------	---------------------------------

Definition at line 186 of file reader.cpp.
Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [reader.h](#)
- [reader.cpp](#)

8.49 mknix::ReaderConstraints Class Reference

```
#include <readerconstraints.h>
```

Public Member Functions

- [ReaderConstraints](#) ()
Default constructor. Initialises all pointers to null.
- [ReaderConstraints](#) ([Simulation](#) *, [std::ofstream](#) &, [std::ifstream](#) &)
Constructs a [ReaderConstraints](#) and binds it to the given simulation, output, and input streams.
- [~ReaderConstraints](#) ()
Destructor.
- void [readConstraints](#) ([System](#) *)
Reads and processes a JOINTS block, creating constraint objects for the given system.

8.49.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 44 of file readerconstraints.h.

8.49.2 Constructor & Destructor Documentation

8.49.2.1 [ReaderConstraints](#)() [1/2] `mknix::ReaderConstraints::ReaderConstraints ( )`

Default constructor. Initialises all pointers to null.

Definition at line 39 of file readerconstraints.cpp.

8.49.2.2 [ReaderConstraints](#)() [2/2] `mknix::ReaderConstraints::ReaderConstraints (`

```
    Simulation * simulation_in,  
    std::ofstream & output_in,  
    std::ifstream & input_in )
```

Constructs a [ReaderConstraints](#) and binds it to the given simulation, output, and input streams.

Parameters

<i>simulation_↵ _in</i>	Pointer to the active Simulation instance.
<i>output_in</i>	Reference to the output log file stream.
<i>input_in</i>	Reference to the input file stream to read from.

Definition at line 54 of file readerconstraints.cpp.

8.49.2.3 [~ReaderConstraints](#)() `mknix::ReaderConstraints::~~ReaderConstraints ( )`

Destructor.

Definition at line 68 of file readerconstraints.cpp.

8.49.3 Member Function Documentation

8.49.3.1 readConstraints() `void mknix::ReaderConstraints::readConstraints (
 System * system_in)`

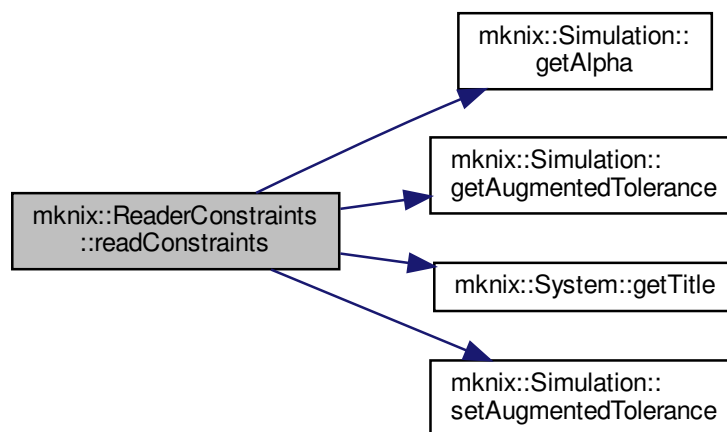
Reads and processes a JOINTS block, creating constraint objects for the given system.

Parameters

<code>system_in</code>	Pointer to the System whose constraints are being populated.
------------------------	--

Definition at line 76 of file readerconstraints.cpp.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [readerconstraints.h](#)
- [readerconstraints.cpp](#)

8.50 mknix::ReaderFlex Class Reference

```
#include <readerflex.h>
```

Public Member Functions

- [ReaderFlex](#) ()
Default constructor. Initialises all pointers to null.
- [ReaderFlex](#) ([Simulation](#) *, `std::ofstream &`, `std::ifstream &`)
Constructs a [ReaderFlex](#) and binds it to the given simulation, output, and input streams.
- [~ReaderFlex](#) ()
Destructor.
- `void` [readFlexBodies](#) ([System](#) *)
Reads and constructs all flexible bodies defined in the FLEXBODIES block.

8.50.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 36 of file readerflex.h.

8.50.2 Constructor & Destructor Documentation

8.50.2.1 ReaderFlex() [1/2] `mknix::ReaderFlex::ReaderFlex ( )`

Default constructor. Initialises all pointers to null.

Definition at line 39 of file readerflex.cpp.

8.50.2.2 ReaderFlex() [2/2] `mknix::ReaderFlex::ReaderFlex (`

```
Simulation * simulation_in,  
std::ofstream & output_in,  
std::ifstream & input_in )
```

Constructs a [ReaderFlex](#) and binds it to the given simulation, output, and input streams.

Parameters

<code>simulation_↔ _in</code>	Pointer to the active Simulation instance.
<code>output_in</code>	Reference to the output log file stream.
<code>input_in</code>	Reference to the input file stream to read from.

Definition at line 52 of file readerflex.cpp.

8.50.2.3 ~ReaderFlex() `mknix::ReaderFlex::~~ReaderFlex ( )`

Destructor.

Definition at line 66 of file readerflex.cpp.

8.50.3 Member Function Documentation

8.50.3.1 readFlexBodies() `void mknix::ReaderFlex::readFlexBodies (`

```
System * system_in )
```

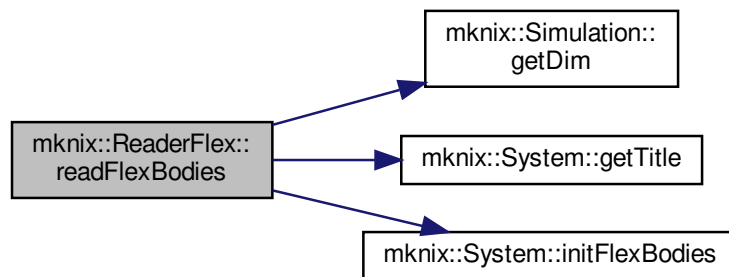
Reads and constructs all flexible bodies defined in the FLEXBODIES block.

Parameters

<code>system_↔ _in</code>	Pointer to the System that will own the flex bodies.
-------------------------------	--

Definition at line 78 of file readerflex.cpp.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [readerflex.h](#)
- [readerflex.cpp](#)

8.51 mknix::ReaderRigid Class Reference

```
#include <readerrigid.h>
```

Public Member Functions

- [ReaderRigid](#) ()
Default constructor. Initialises all pointers to null.
- [ReaderRigid](#) ([Simulation](#) *, `std::ofstream &`, `std::ifstream &`)
Constructs a [ReaderRigid](#) and binds it to the given simulation, output, and input streams.
- [~ReaderRigid](#) ()
Destructor.
- `void` [readRigidBodies](#) ([System](#) *)
Reads the RIGIDBODIES block and dispatches to the appropriate specific rigid body reader.

8.51.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 41 of file `readerrigid.h`.

8.51.2 Constructor & Destructor Documentation

8.51.2.1 [ReaderRigid](#)() [1/2] `mknix::ReaderRigid::ReaderRigid ( )`

Default constructor. Initialises all pointers to null.

Definition at line 38 of file `readerrigid.cpp`.

8.51.2.2 ReaderRigid() [2/2] `mknix::ReaderRigid::ReaderRigid (`
`Simulation * simulation_in,`
`std::ofstream & output_in,`
`std::ifstream & input_in )`

Constructs a [ReaderRigid](#) and binds it to the given simulation, output, and input streams.

Parameters

<code>simulation_↔ _in</code>	Pointer to the active Simulation instance.
<code>output_in</code>	Reference to the output log file stream.
<code>input_in</code>	Reference to the input file stream to read from.

Definition at line 51 of file `readerrigid.cpp`.

8.51.2.3 ~ReaderRigid() `mknix::ReaderRigid::~~ReaderRigid ( )`

Destructor.

Definition at line 64 of file `readerrigid.cpp`.

8.51.3 Member Function Documentation

8.51.3.1 readRigidBodies() `void mknix::ReaderRigid::readRigidBodies (`
`System * system_in )`

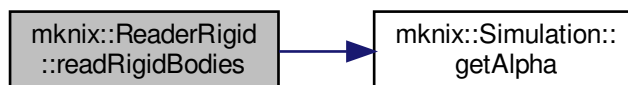
Reads the RIGIDBODIES block and dispatches to the appropriate specific rigid body reader.

Parameters

<code>system_↔ _in</code>	Pointer to the System that will own the rigid bodies.
-------------------------------	---

Definition at line 75 of file `readerrigid.cpp`.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [readerrigid.h](#)
- [readerrigid.cpp](#)

8.52 mknix::RigidBar Class Reference

```
#include <bodyrigid1D.h>
```


Destructor.

- `std::string getType ()`
- `void setInertia (double, int)`
Sets the rotational inertia about axis 0 (only one axis supported).
- `void setPosition (std::vector< double > &)`
Sets initial end-node positions from a centre-of-mass position and rotation angle.
- `void calcMassMatrix ()`
Builds the consistent local mass matrix for the bar (2-node linear element).
- `void calcExternalForces ()`
Computes the gravitational load vector: half of $-m \cdot g$ at each end node.
- `void addNode (Node *)`
Adds a domain node to the bar, assigning the two frame nodes as support nodes and solving the 1D shape function.
- `void writeBoundaryNodes (std::vector< Node * > &)`
Collects the frame nodes (bar endpoints) as boundary nodes.
- `void writeBoundaryConnectivity (std::vector< std::vector< Node * > > &)`
Builds the boundary connectivity using the frame nodes (bar endpoints).

Additional Inherited Members

8.52.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 32 of file bodyrigid1D.h.

8.52.2 Constructor & Destructor Documentation

8.52.2.1 RigidBody() [1/2] `mknix::RigidBody::RigidBody ( )`

Default constructor. Initializes the bar with zero moment of inertia.

Definition at line 30 of file bodyrigid1D.cpp.

8.52.2.2 RigidBody() [2/2] `mknix::RigidBody::RigidBody (`

```
std::string title_in,
Node * nodeA_in,
Node * nodeB_in,
double mass_in )
```

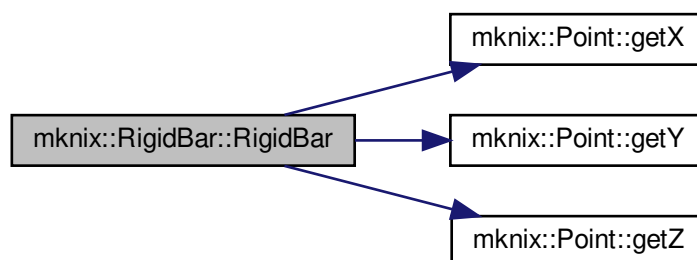
Constructor with title, end nodes and mass.

Parameters

<i>title_in</i>	Name identifier for this bar body.
<i>nodeA_↔_in</i>	Pointer to node A (first endpoint).
<i>nodeB_↔_in</i>	Pointer to node B (second endpoint).
<i>mass_in</i>	Total mass of the bar.

Definition at line 43 of file bodyrigid1D.cpp.

Here is the call graph for this function:



8.52.2.3 ~RigidBar() mknix::RigidBar::~~RigidBar ()

Destructor.

Definition at line 69 of file bodyrigid1D.cpp.

8.52.3 Member Function Documentation

8.52.3.1 addNode() void mknix::RigidBar::addNode (Node * node_in) [virtual]

Adds a domain node to the bar, assigning the two frame nodes as support nodes and solving the 1D shape function.

Parameters

<i>node</i> ↔ _in	Pointer to the domain node to add.
----------------------	------------------------------------

Reimplemented from [mknix::Body](#).

Definition at line 152 of file bodyrigid1D.cpp.

Here is the call graph for this function:



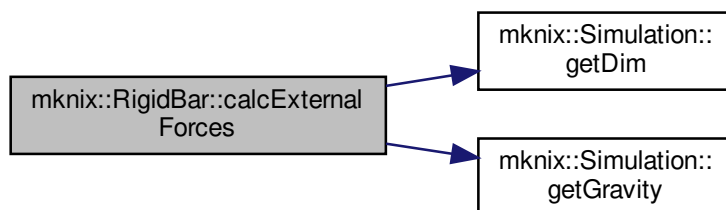
8.52.3.2 calcExternalForces() void mknix::RigidBar::calcExternalForces () [virtual]

Computes the gravitational load vector: half of -m*g at each end node.

Implements [mknix::Body](#).

Definition at line 130 of file bodyrigid1D.cpp.

Here is the call graph for this function:



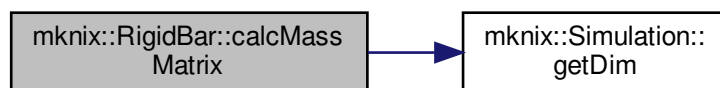
8.52.3.3 calcMassMatrix() `void mknix::RigidBody::calcMassMatrix ( ) [virtual]`

Builds the consistent local mass matrix for the bar (2-node linear element).

Implements [mknix::Body](#).

Definition at line 106 of file `bodyrigid1D.cpp`.

Here is the call graph for this function:



8.52.3.4 getType() `std::string mknix::RigidBody::getType ( ) [inline], [virtual]`

Implements [mknix::Body](#).

Definition at line 41 of file `bodyrigid1D.h`.

8.52.3.5 setInertia() `void mknix::RigidBody::setInertia ( double inertia_in, int axis ) [virtual]`

Sets the rotational inertia about axis 0 (only one axis supported).

Parameters

<i>inertia_in</i>	Inertia value to assign.
<i>axis</i>	Axis index (only 0 is valid).

Implements [mknix::RigidBody](#).

Definition at line 78 of file `bodyrigid1D.cpp`.

8.52.3.6 setPosition() `void mknix::RigidBody::setPosition (
 std::vector< double > & position) [virtual]`

Sets initial end-node positions from a centre-of-mass position and rotation angle.

Parameters

<i>position</i>	Vector [CoG_x, CoG_y, z, angle] defining the bar pose.
-----------------	--

Implements [mknix::RigidBody](#).

Definition at line 94 of file `bodyrigid1D.cpp`.

8.52.3.7 writeBoundaryConnectivity() `void mknix::RigidBody::writeBoundaryConnectivity (
 std::vector< std::vector< Node * > > & connectivity_nodes)`

Builds the boundary connectivity using the frame nodes (bar endpoints).

Parameters

<i>connectivity_nodes</i>	Vector of node chains to which the bar boundary is appended.
---------------------------	--

Definition at line 186 of file `bodyrigid1D.cpp`.

8.52.3.8 writeBoundaryNodes() `void mknix::RigidBody::writeBoundaryNodes (
 std::vector< Node * > & boundary_nodes)`

Collects the frame nodes (bar endpoints) as boundary nodes.

Parameters

<i>boundary_nodes</i>	Vector to which the frame node pointers are appended.
-----------------------	---

Definition at line 173 of file `bodyrigid1D.cpp`.

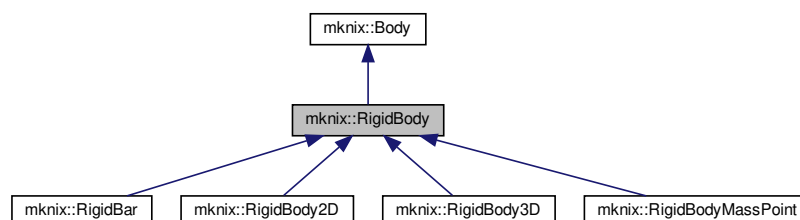
The documentation for this class was generated from the following files:

- [bodyrigid1D.h](#)
- [bodyrigid1D.cpp](#)

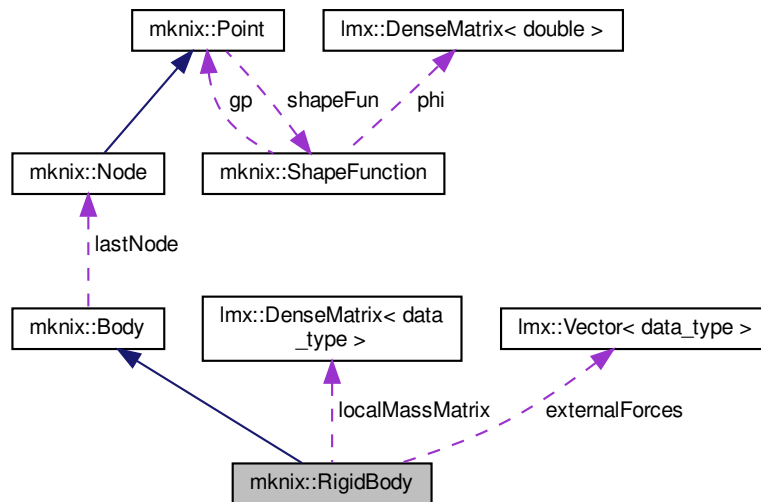
8.53 mknix::RigidBody Class Reference

```
#include <bodyrigid.h>
```

Inheritance diagram for `mknix::RigidBody`:



Collaboration diagram for `mknix::RigidBody`:



Public Member Functions

- `RigidBody ()`
Default constructor. Initializes a rigid body with zero mass and no energy computation.
- `RigidBody (std::string)`
Constructor with title.
- `virtual ~RigidBody ()`
Destructor.
- `void assembleMassMatrix (Imx::Matrix< data_type > &) override`
Assembles the local mass matrix contribution into the global mass matrix.
- `void assembleExternalForces (Imx::Vector< data_type > &) override`
Assembles the local external forces into the global external force vector.
- `void setMass (double mass_in)`
- `void setDensityFactor (double density_in)`
- `virtual void setInertia (double, int)=0`
- `virtual void setPosition (std::vector< double > &)=0`
- `void setOutput (std::string) override`
Enables energy computation output.
- `Node * getNode (int) override`
- `void outputStep (const Imx::Vector< data_type > &, const Imx::Vector< data_type > &) override`
Stores domain-node positions for the current dynamic step.
- `void outputStep (const Imx::Vector< data_type > &) override`
Stores domain-node positions for the current static step.
- `void outputToFile (std::ofstream *) override`
Writes stored domain-node positions and optional energy data to the output file.
- `void writeBodyInfo (std::ofstream *) override`
Writes rigid-body type and frame-node information to the output file.
- `virtual void writeBoundaryNodes (std::vector< Point * > &) override`
No-op in the base rigid-body class; specialized subclasses may override.
- `virtual void writeBoundaryConnectivity (std::vector< std::vector< Point * > > &) override`
No-op in the base rigid-body class; specialized subclasses may override.

Protected Attributes

- bool [computeEnergy](#)
- int [dim](#)
- double [mass](#)
- double [densityFactor](#)
- std::vector< [Node](#) * > [frameNodes](#)
- std::vector< [lmx::Vector](#)< [data_type](#) > * > [domainConf](#)
- [lmx::Vector](#)< [data_type](#) > [externalForces](#)
- [lmx::DenseMatrix](#)< [data_type](#) > [localMassMatrix](#)
- std::vector< [lmx::Vector](#)< [data_type](#) > * > [energy](#)

8.53.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 34 of file [bodyrigid.h](#).

8.53.2 Constructor & Destructor Documentation**8.53.2.1 RigidBody()** [1/2] `mknix::RigidBody::RigidBody ( )`

Default constructor. Initializes a rigid body with zero mass and no energy computation.

Definition at line 30 of file [bodyrigid.cpp](#).

8.53.2.2 RigidBody() [2/2] `mknix::RigidBody::RigidBody (
std::string title_in)`

Constructor with title.

Parameters

<i>title</i> ↔ <i>_in</i>	Name identifier for this rigid body.
------------------------------	--------------------------------------

Definition at line 43 of file [bodyrigid.cpp](#).

8.53.2.3 ~RigidBody() `mknix::RigidBody::~~RigidBody ( ) [virtual]`

Destructor.

Definition at line 55 of file [bodyrigid.cpp](#).

8.53.3 Member Function Documentation**8.53.3.1 assembleExternalForces()** `void mknix::RigidBody::assembleExternalForces (
lmx::Vector< data_type > & globalExternalForces) [override], [virtual]`

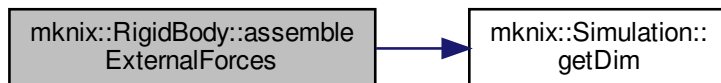
Assembles the local external forces into the global external force vector.

Parameters

<i>globalExternalForces</i>	Reference to the global external force vector.
-----------------------------	--

Implements [mknix::Body](#).

Definition at line 93 of file bodyrigid.cpp.
Here is the call graph for this function:



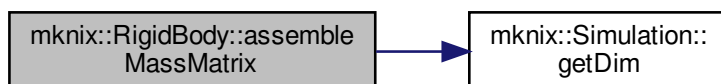
8.53.3.2 assembleMassMatrix() `void mknix::RigidBody::assembleMassMatrix (
 lmx::Matrix< data_type > & globalMass) [override], [virtual]`

Assembles the local mass matrix contribution into the global mass matrix.

Parameters

<i>globalMass</i>	Reference to the global mass matrix.
-------------------	--------------------------------------

Implements [mnix::Body](#).
Definition at line 64 of file bodyrigid.cpp.
Here is the call graph for this function:



8.53.3.3 getNode() `Node * mknix::RigidBody::getNode (
 int node_number) [override], [virtual]`

Reimplemented from [mnix::Body](#).

Definition at line 126 of file bodyrigid.cpp.

8.53.3.4 outputStep() `[1/2] void mknix::RigidBody::outputStep (
 const lmx::Vector< data_type > & q) [override], [virtual]`

Stores domain-node positions for the current static step.

Parameters

<i>q</i>	Global configuration vector.
----------	------------------------------

Implements [mnix::Body](#).
Definition at line 222 of file bodyrigid.cpp.

Here is the call graph for this function:



8.53.3.5 outputStep() [2/2] `void mknix::RigidBody::outputStep (`
 `const lmx::Vector< data_type > & q,`
 `const lmx::Vector< data_type > & qdot ) [override], [virtual]`

Stores domain-node positions for the current dynamic step.

Parameters

<i>q</i>	Global configuration vector.
<i>qdot</i>	Global velocity vector.

Implements [mknix::Body](#).

Definition at line 161 of file bodyrigid.cpp.

Here is the call graph for this function:



8.53.3.6 outputToFile() `void mknix::RigidBody::outputToFile (`
 `std::ofstream * outFile ) [override], [virtual]`

Writes stored domain-node positions and optional energy data to the output file.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Reimplemented from [mknix::Body](#).

Definition at line 266 of file bodyrigid.cpp.

Here is the call graph for this function:



8.53.3.7 setDensityFactor() `void mknix::RigidBody::setDensityFactor (`
`double density_in ) [inline]`

Definition at line 54 of file bodyrigid.h.

8.53.3.8 setInertia() `virtual void mknix::RigidBody::setInertia (`
`double ,`
`int ) [pure virtual]`

Implemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), [mknix::RigidBar](#), and [mknix::RigidBodyMassPoint](#).

8.53.3.9 setMass() `void mknix::RigidBody::setMass (`
`double mass_in ) [inline]`

Definition at line 49 of file bodyrigid.h.

8.53.3.10 setOutput() `void mknix::RigidBody::setOutput (`
`std::string outputType_in ) [override], [virtual]`

Enables energy computation output.

Parameters

<code>outputType_in</code>	Keyword; use "ENERGY" to activate energy output.
----------------------------	--

Implements [mknix::Body](#).

Definition at line 121 of file bodyrigid.cpp.

8.53.3.11 setPosition() `virtual void mknix::RigidBody::setPosition (`
`std::vector< double > & ) [pure virtual]`

Implemented in [mknix::RigidBody3D](#), [mknix::RigidBody2D](#), [mknix::RigidBar](#), and [mknix::RigidBodyMassPoint](#).

8.53.3.12 writeBodyInfo() `void mknix::RigidBody::writeBodyInfo (`
`std::ofstream * outFile ) [override], [virtual]`

Writes rigid-body type and frame-node information to the output file.

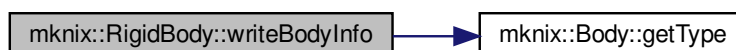
Parameters

<code>outFile</code>	Pointer to the output file stream.
----------------------	------------------------------------

Implements [mknix::Body](#).

Definition at line 310 of file bodyrigid.cpp.

Here is the call graph for this function:



8.53.3.13 writeBoundaryConnectivity() `void mknix::RigidBody::writeBoundaryConnectivity ( std::vector< std::vector< Point * > > & connectivity_nodes ) [override], [virtual]`

No-op in the base rigid-body class; specialized subclasses may override.

Parameters

<i>connectivity_nodes</i>	Output vector (unused here).
---------------------------	------------------------------

Definition at line 345 of file bodyrigid.cpp.

8.53.3.14 writeBoundaryNodes() `void mknix::RigidBody::writeBoundaryNodes ( std::vector< Point * > & boundary_nodes ) [override], [virtual]`

No-op in the base rigid-body class; specialized subclasses may override.

Parameters

<i>boundary_nodes</i>	Output vector (unused here).
-----------------------	------------------------------

Implements [mknix::Body](#).

Definition at line 330 of file bodyrigid.cpp.

8.53.4 Member Data Documentation

8.53.4.1 computeEnergy `bool mknix::RigidBody::computeEnergy [protected]`

Definition at line 81 of file bodyrigid.h.

8.53.4.2 densityFactor `double mknix::RigidBody::densityFactor [protected]`

Definition at line 83 of file bodyrigid.h.

8.53.4.3 dim `int mknix::RigidBody::dim [protected]`

Definition at line 82 of file bodyrigid.h.

8.53.4.4 domainConf `std::vector<lmx::Vector<data\_type> *> mknix::RigidBody::domainConf [protected]`

Definition at line 85 of file bodyrigid.h.

8.53.4.5 energy `std::vector<lmx::Vector<data\_type> *> mknix::RigidBody::energy [protected]`

Definition at line 88 of file bodyrigid.h.

8.53.4.6 externalForces `lmx::Vector<data\_type> mknix::RigidBody::externalForces [protected]`

Definition at line 86 of file bodyrigid.h.

8.53.4.7 frameNodes `std::vector<Node *> mknix::RigidBody::frameNodes [protected]`

Definition at line 84 of file bodyrigid.h.

8.53.4.8 localMassMatrix `lmx::DenseMatrix<data_type> mknix::RigidBody::localMassMatrix [protected]`
Definition at line 87 of file bodyrigid.h.

8.53.4.9 mass `double mknix::RigidBody::mass [protected]`
Definition at line 83 of file bodyrigid.h.

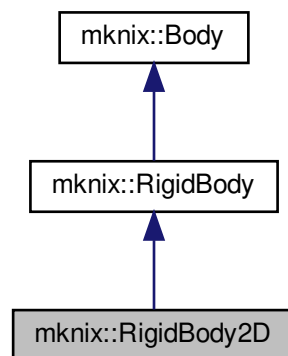
The documentation for this class was generated from the following files:

- [bodyrigid.h](#)
- [bodyrigid.cpp](#)

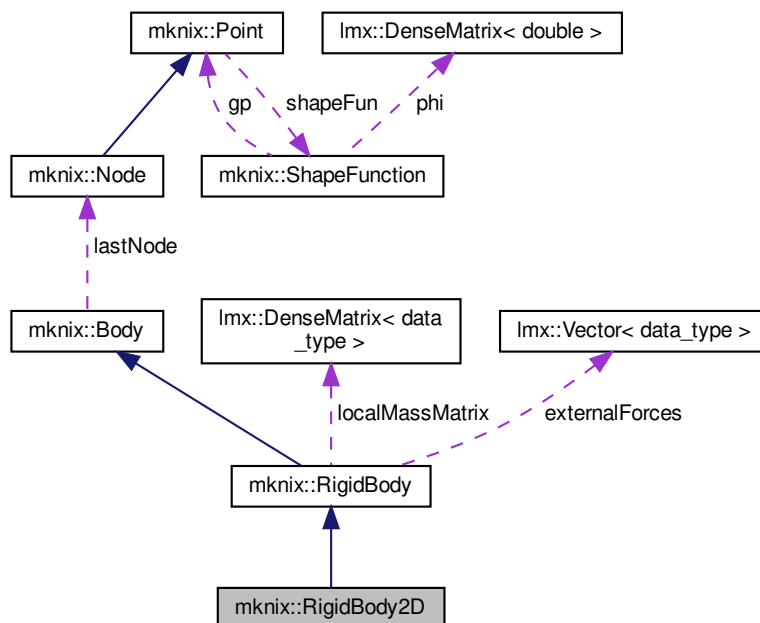
8.54 mknix::RigidBody2D Class Reference

`#include <bodyrigid2D.h>`

Inheritance diagram for mknix::RigidBody2D:



Collaboration diagram for mknix::RigidBody2D:



Public Member Functions

- [RigidBody2D](#) ()
Default constructor. Initializes the 2D rigid body with zero inertia terms.
- [RigidBody2D](#) (std::string, [Node](#) *, [Node](#) *, [Node](#) *)
Constructor with title and three frame nodes (CoG, x-director, y-director).
- [~RigidBody2D](#) ()
Destructor.
- std::string [getType](#) ()
- void [setInertia](#) (double, int)
Sets one component of the planar inertia tensor.
- void [setPosition](#) (std::vector< double > &)
Sets the initial pose of the body from a centre-of-mass position and rotation angle.
- void [calcMassMatrix](#) ()
Builds the local mass matrix for a planar rigid body using the body-inertia parameters.
- void [calcExternalForces](#) ()
Computes the gravitational external force vector applied at the CoG node.
- void [addNode](#) ([Node](#) *)
Adds a domain node, assigning the three frame nodes as support nodes and solving the 2D shape function.

Additional Inherited Members

8.54.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 32 of file bodyrigid2D.h.

8.54.2 Constructor & Destructor Documentation

8.54.2.1 RigidBody2D() [1/2] `mknix::RigidBody2D::RigidBody2D ( )`

Default constructor. Initializes the 2D rigid body with zero inertia terms.

Definition at line 30 of file `bodyrigid2D.cpp`.

8.54.2.2 RigidBody2D() [2/2] `mknix::RigidBody2D::RigidBody2D ( std::string title_in, Node * nodeA_in, Node * nodeB_in, Node * nodeC_in )`

Constructor with title and three frame nodes (CoG, x-director, y-director).

Parameters

<i>title_in</i>	Name identifier for this body.
<i>nodeA_in</i>	Pointer to the centre-of-gravity node.
<i>nodeB_in</i>	Pointer to the x-direction frame node.
<i>nodeC_in</i>	Pointer to the y-direction frame node.

Definition at line 45 of file `bodyrigid2D.cpp`.

8.54.2.3 ~RigidBody2D() `mknix::RigidBody2D::~~RigidBody2D ( )`

Destructor.

Definition at line 72 of file `bodyrigid2D.cpp`.

8.54.3 Member Function Documentation

8.54.3.1 addNode() `void mknix::RigidBody2D::addNode ( Node * node_in ) [virtual]`

Adds a domain node, assigning the three frame nodes as support nodes and solving the 2D shape function.

Parameters

<i>node_in</i>	Pointer to the domain node to add.
----------------	------------------------------------

Reimplemented from [mknix::Body](#).

Definition at line 167 of file `bodyrigid2D.cpp`.

Here is the call graph for this function:



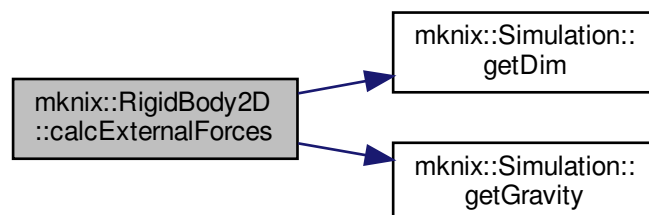
8.54.3.2 calcExternalForces() `void mknix::RigidBody2D::calcExternalForces ( ) [virtual]`

Computes the gravitational external force vector applied at the CoG node.

Implements [mknix::Body](#).

Definition at line 154 of file `bodyrigid2D.cpp`.

Here is the call graph for this function:



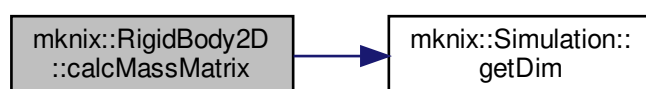
8.54.3.3 calcMassMatrix() `void mknix::RigidBody2D::calcMassMatrix ( ) [virtual]`

Builds the local mass matrix for a planar rigid body using the body-inertia parameters.

Implements [mknix::Body](#).

Definition at line 107 of file `bodyrigid2D.cpp`.

Here is the call graph for this function:



8.54.3.4 getType() `std::string mknix::RigidBody2D::getType ( ) [inline], [virtual]`

Implements [mknix::Body](#).

Definition at line 41 of file `bodyrigid2D.h`.

8.54.3.5 setInertia() `void mknix::RigidBody2D::setInertia (
double inertia_in,
int axis) [virtual]`

Sets one component of the planar inertia tensor.

Parameters

<i>inertia_in</i>	Value to assign.
<i>axis</i>	Component index: 0=Ixx, 1=Iyy, 2=Ixy.

Implements [mknix::RigidBody](#).

Definition at line 81 of file `bodyrigid2D.cpp`.

8.54.3.6 setPosition() `void mknix::RigidBody2D::setPosition (
std::vector< double > & position) [virtual]`

Sets the initial pose of the body from a centre-of-mass position and rotation angle.

Parameters

<i>position</i>	Vector [CoG_x, CoG_y, rotation_angle] defining the initial pose.
-----------------	--

Implements [mknix::RigidBody](#).

Definition at line 92 of file `bodyrigid2D.cpp`.

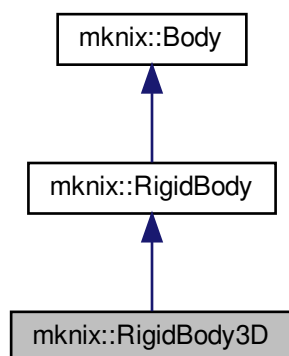
The documentation for this class was generated from the following files:

- [bodyrigid2D.h](#)
- [bodyrigid2D.cpp](#)

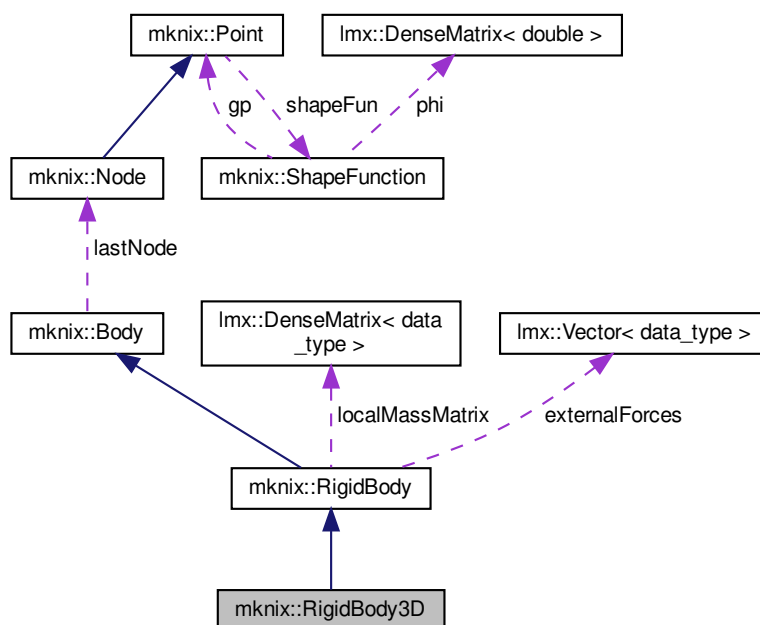
8.55 mknix::RigidBody3D Class Reference

```
#include <bodyrigid3D.h>
```

Inheritance diagram for mknix::RigidBody3D:



Collaboration diagram for mknix::RigidBody3D:



Public Member Functions

- [RigidBody3D](#) ()
Default constructor.
- [RigidBody3D](#) (std::string, [Node](#) *, [Node](#) *, [Node](#) *, [Node](#) *)
Constructor with title and four frame nodes (CoG, and three director nodes).
- [~RigidBody3D](#) ()

Destructor.

- `std::string getType ()`
- `void setInertia (double, int)`

Sets one component of the 3D inertia tensor.

- `void setPosition (std::vector< double > &)`

Sets the initial pose of the body from a centre-of-mass position.

- `void calcMassMatrix ()`

Builds the local 3D mass matrix from the principal inertia tensor components. Applies optional density scaling before assembling the block structure.

- `void calcExternalForces ()`

Computes the gravitational external force vector applied at the CoG node.

- `void addNode (Node *)`

Adds a domain node, assigning the four frame nodes as support nodes and solving the 3D shape function.

Additional Inherited Members

8.55.1 Detailed Description

Definition at line 29 of file `bodyrigid3D.h`.

8.55.2 Constructor & Destructor Documentation

8.55.2.1 RigidBody3D() [1/2] `mknix::RigidBody3D::RigidBody3D ( )`

Default constructor.

Definition at line 36 of file `bodyrigid3D.cpp`.

8.55.2.2 RigidBody3D() [2/2] `mknix::RigidBody3D::RigidBody3D (`

```
std::string title_in,
Node * node0_in,
Node * node1_in,
Node * node2_in,
Node * node3_in )
```

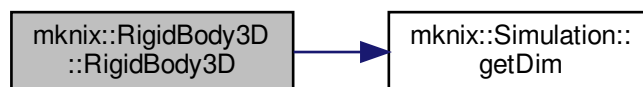
Constructor with title and four frame nodes (CoG, and three director nodes).

Parameters

<code>title_in</code>	Name identifier for this body.
<code>node0↔ _in</code>	Centre-of-gravity node.
<code>node1↔ _in</code>	X-direction frame node.
<code>node2↔ _in</code>	Y-direction frame node.
<code>node3↔ _in</code>	Z-direction frame node.

Definition at line 48 of file `bodyrigid3D.cpp`.

Here is the call graph for this function:



8.55.2.3 ~RigidBody3D() mknix::RigidBody3D::~~RigidBody3D ()

Destructor.

Definition at line 68 of file bodyrigid3D.cpp.

8.55.3 Member Function Documentation

8.55.3.1 addNode() void mknix::RigidBody3D::addNode (Node * node_in) [virtual]

Adds a domain node, assigning the four frame nodes as support nodes and solving the 3D shape function.

Parameters

<i>node_in</i>	Pointer to the domain node to add.
----------------	------------------------------------

Reimplemented from [mknix::Body](#).

Definition at line 213 of file bodyrigid3D.cpp.

Here is the call graph for this function:



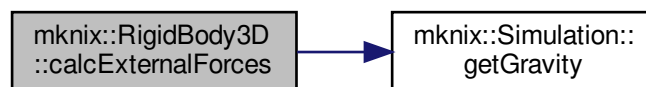
8.55.3.2 calcExternalForces() void mknix::RigidBody3D::calcExternalForces () [virtual]

Computes the gravitational external force vector applied at the CoG node.

Implements [mknix::Body](#).

Definition at line 198 of file bodyrigid3D.cpp.

Here is the call graph for this function:



8.55.3.3 calcMassMatrix() `void mknix::RigidBody3D::calcMassMatrix ( ) [virtual]`

Builds the local 3D mass matrix from the principal inertia tensor components. Applies optional density scaling before assembling the block structure.

Implements [mnix::Body](#).

Definition at line 116 of file `bodyrigid3D.cpp`.

8.55.3.4 getType() `std::string mknix::RigidBody3D::getType ( ) [inline], [virtual]`

Implements [mnix::Body](#).

Definition at line 39 of file `bodyrigid3D.h`.

8.55.3.5 setInertia() `void mknix::RigidBody3D::setInertia ( double inertia_in, int axis ) [virtual]`

Sets one component of the 3D inertia tensor.

Parameters

<i>inertia_in</i>	Value to assign.
<i>axis</i>	Component: 0=Ixx, 1=Iyy, 2=Izz, 3=Ixy, 4=Iyz, 5=Ixz.

Implements [mnix::RigidBody](#).

Definition at line 77 of file `bodyrigid3D.cpp`.

8.55.3.6 setPosition() `void mknix::RigidBody3D::setPosition ( std::vector< double > & position ) [virtual]`

Sets the initial pose of the body from a centre-of-mass position.

Parameters

<i>position</i>	Vector [CoG_x, CoG_y, CoG_z] defining the initial position.
-----------------	---

Implements [mnix::RigidBody](#).

Definition at line 92 of file `bodyrigid3D.cpp`.

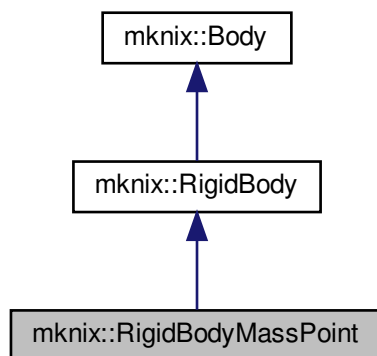
The documentation for this class was generated from the following files:

- [bodyrigid3D.h](#)
- [bodyrigid3D.cpp](#)

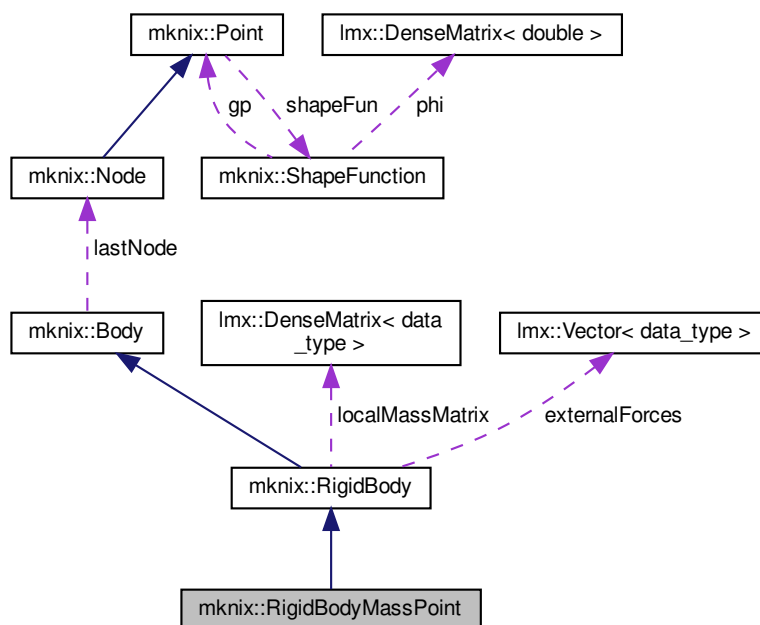
8.56 mknix::RigidBodyMassPoint Class Reference

```
#include <bodyrigid0D.h>
```

Inheritance diagram for mknix::RigidBodyMassPoint:



Collaboration diagram for mknix::RigidBodyMassPoint:



Public Member Functions

- [RigidBodyMassPoint](#) ()
Default constructor.
- [RigidBodyMassPoint](#) (std::string, [Node](#) *, double)

Constructor with title, node and mass.

- [~RigidBodyMassPoint](#) ()

Destructor.

- void [setInertia](#) (double, int)

Inertia assignment is not applicable to a mass point; logs an error.

- void [setPosition](#) (std::vector< double > &)

Sets the initial position of the mass point.

- void [calcMassMatrix](#) ()

Builds the local mass matrix: $m \cdot I$ for 2D, or $m \cdot I_3$ for 3D.

- void [calcExternalForces](#) ()

Computes the gravitational external force vector: $-m \cdot g$ for each dimension.

- std::string [getType](#) ()

Additional Inherited Members

8.56.1 Detailed Description

Definition at line 30 of file bodyrigid0D.h.

8.56.2 Constructor & Destructor Documentation

8.56.2.1 RigidBodyMassPoint() [1/2] `mknix::RigidBodyMassPoint::RigidBodyMassPoint ( )`

Default constructor.

Definition at line 35 of file bodyrigid0D.cpp.

8.56.2.2 RigidBodyMassPoint() [2/2] `mknix::RigidBodyMassPoint::RigidBodyMassPoint (`

`std::string title_in,`

`Node * nodeA_in,`

`double mass_in )`

Constructor with title, node and mass.

Parameters

<i>title_in</i>	Name identifier for this mass-point body.
<i>nodeA_in</i>	Pointer to the single frame node (centre of mass).
<i>mass_in</i>	Total mass of the point.

Definition at line 45 of file bodyrigid0D.cpp.

8.56.2.3 ~RigidBodyMassPoint() `mknix::RigidBodyMassPoint::~~RigidBodyMassPoint ( )`

Destructor.

Definition at line 60 of file bodyrigid0D.cpp.

8.56.3 Member Function Documentation

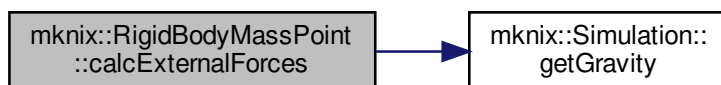
8.56.3.1 calcExternalForces() `void mknix::RigidBodyMassPoint::calcExternalForces ( ) [virtual]`

Computes the gravitational external force vector: $-m \cdot g$ for each dimension.

Implements [mknix::Body](#).

Definition at line 111 of file bodyrigid0D.cpp.

Here is the call graph for this function:



8.56.3.2 calcMassMatrix() `void mknix::RigidBodyMassPoint::calcMassMatrix ( ) [virtual]`

Builds the local mass matrix: $m \times I$ for 2D, or $m \times I3$ for 3D.

Implements [mknix::Body](#).

Definition at line 88 of file `bodyrigid0D.cpp`.

8.56.3.3 getType() `std::string mknix::RigidBodyMassPoint::getType ( ) [inline], [virtual]`

Implements [mknix::Body](#).

Definition at line 49 of file `bodyrigid0D.h`.

8.56.3.4 setInertia() `void mknix::RigidBodyMassPoint::setInertia ( double inertia_in, int axis ) [virtual]`

Inertia assignment is not applicable to a mass point; logs an error.

Parameters

<i>inertia_in</i>	Inertia value (ignored).
<i>axis</i>	Axis index (ignored).

Implements [mknix::RigidBody](#).

Definition at line 69 of file `bodyrigid0D.cpp`.

8.56.3.5 setPosition() `void mknix::RigidBodyMassPoint::setPosition ( std::vector< double > & position ) [virtual]`

Sets the initial position of the mass point.

Parameters

<i>position</i>	Vector containing [x, y] (or [x, y, z]) coordinates.
-----------------	--

Implements [mknix::RigidBody](#).

Definition at line 78 of file `bodyrigid0D.cpp`.

The documentation for this class was generated from the following files:

- [bodyrigid0D.h](#)
- [bodyrigid0D.cpp](#)

8.57 mknix::SectionReader Class Reference

```
#include <sectionreader.h>
```

Public Member Functions

- [SectionReader](#) (const std::string §ionName)
Constructs a [SectionReader](#) with the given section name.
- void [read](#) (std::ifstream &input, std::ofstream &log, size_t &line_no)
Reads and parses the section from the input stream, dispatching tokens to fields or sub-sections.
- const std::vector< std::pair< std::string, std::string > > & [getFields](#) ()
- const std::vector< [SectionReader](#) > & [getSubSections](#) ()
- [SectionReader](#) & [addField](#) (const std::string &fieldName)
Registers a valid field name for this section.
- [SectionReader](#) & [addSubSection](#) ([SectionReader](#) subSection)
Registers a nested sub-section reader.

8.57.1 Detailed Description

Definition at line 15 of file sectionreader.h.

8.57.2 Constructor & Destructor Documentation

8.57.2.1 [SectionReader\(\)](#) `mknix::SectionReader::SectionReader (const std::string & name )`

Constructs a [SectionReader](#) with the given section name.

Parameters

<i>name</i>	The keyword name of the section (e.g., "SYSTEM").
-------------	---

Definition at line 15 of file sectionreader.cpp.

8.57.3 Member Function Documentation

8.57.3.1 [addField\(\)](#) `mknix::SectionReader & mknix::SectionReader::addField (const std::string & fieldName )`

Registers a valid field name for this section.

Parameters

<i>fieldName</i>	The name of the field to accept.
------------------	----------------------------------

Returns

Reference to this [SectionReader](#) for chaining.

Definition at line 81 of file sectionreader.cpp.

8.57.3.2 [addSubSection\(\)](#) `mknix::SectionReader & mknix::SectionReader::addSubSection (mknix::SectionReader subSection )`

Registers a nested sub-section reader.

Parameters

<i>subSection</i>	The SectionReader instance for the sub-section.
-------------------	---

Returns

Reference to this [SectionReader](#) for chaining.

Definition at line 92 of file sectionreader.cpp.

8.57.3.3 getFields() `const std::vector<std::pair<std::string, std::string> >& mknix::SectionReader::getFields ( ) [inline]`

Definition at line 20 of file sectionreader.h.

8.57.3.4 getSubSections() `const std::vector<SectionReader>& mknix::SectionReader::getSubSections ( ) [inline]`

Definition at line 24 of file sectionreader.h.

8.57.3.5 read() `void mknix::SectionReader::read (
std::ifstream & input,
std::ofstream & log,
size_t & line_no)`

Reads and parses the section from the input stream, dispatching tokens to fields or sub-sections.

Parameters

<i>input</i>	Input file stream to read tokens from.
<i>log</i>	Output log stream for diagnostic messages.
<i>line_no</i>	Current line number, incremented as lines are consumed.

Definition at line 25 of file sectionreader.cpp.

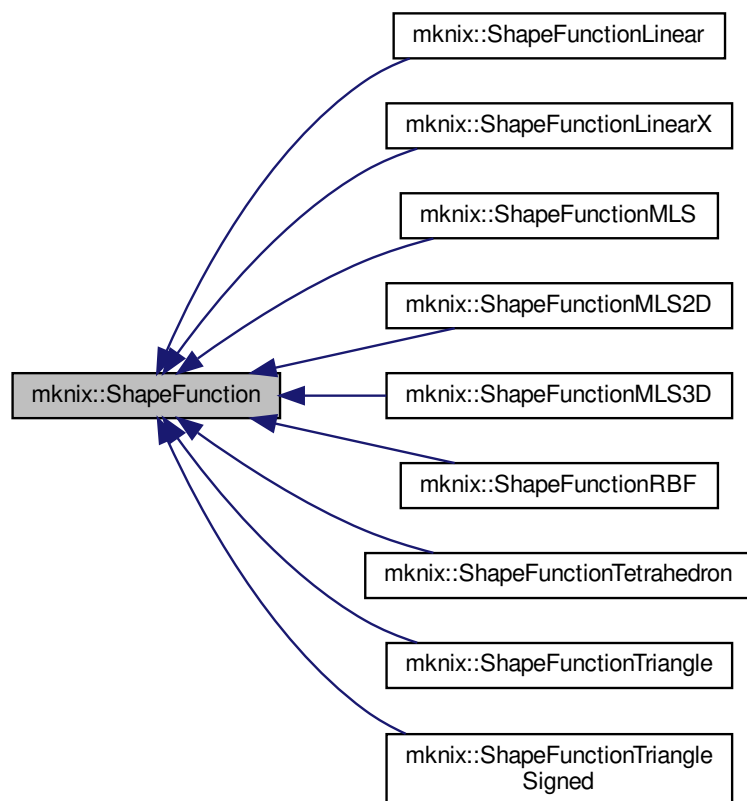
The documentation for this class was generated from the following files:

- [sectionreader.h](#)
- [sectionreader.cpp](#)

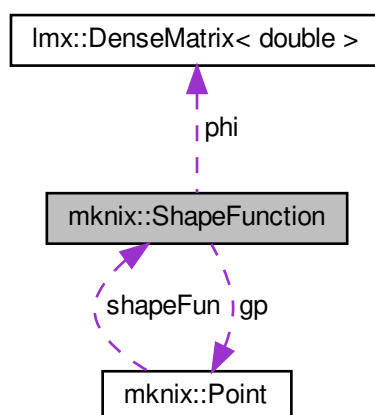
8.58 mknix::ShapeFunction Class Reference

```
#include <shapefunction.h>
```

Inheritance diagram for `mnix::ShapeFunction`:



Collaboration diagram for `mnix::ShapeFunction`:



Public Member Functions

- [ShapeFunction](#) ()
Default constructor. Initializes dimension from the global simulation setting.
- [ShapeFunction](#) (const [ShapeFunction](#) *)
Copy constructor from pointer. Copies dimension, phi matrix, and Gauss point reference.
- [ShapeFunction](#) ([Point](#) *)
Constructs a [ShapeFunction](#) attached to the given [Point](#) (Gauss point).
- virtual [~ShapeFunction](#) ()
Destructor for [ShapeFunction](#).
- virtual void [calc](#) ()=0
- virtual double [getPhi](#) (size_t i, size_t j)
- virtual void [setPhi](#) (double value_in, size_t i, size_t j)
- virtual void [outputValues](#) ()
Prints shape function values (phi and derivatives) for all support nodes to stdout.
- virtual void [gnuplotOut](#) ()
Writes shape function phi values and first derivatives for all support nodes to gnuplot-compatible output files.

Protected Attributes

- size_t [dim](#)
- [lmx::DenseMatrix](#)< double > [phi](#)
- [Point](#) * [gp](#)

8.58.1 Detailed Description

Author

Daniel Iglesias

Definition at line 44 of file shapefunction.h.

8.58.2 Constructor & Destructor Documentation**8.58.2.1 ShapeFunction() [1/3]** `mknix::ShapeFunction::ShapeFunction ( )`

Default constructor. Initializes dimension from the global simulation setting.

Definition at line 13 of file shapefunction.cpp.

8.58.2.2 ShapeFunction() [2/3] `mknix::ShapeFunction::ShapeFunction ( const ShapeFunction * sf_in )`

Copy constructor from pointer. Copies dimension, phi matrix, and Gauss point reference.

Parameters

<code>sf_in</code>	Pointer to the source ShapeFunction .
--------------------	---

Definition at line 22 of file shapefunction.cpp.

8.58.2.3 ShapeFunction() [3/3] `mknix::ShapeFunction::ShapeFunction ( Point * gp_in )`

Constructs a [ShapeFunction](#) attached to the given [Point](#) (Gauss point).

Parameters

<i>gp</i> ↔ _in	Pointer to the Point at which shape functions will be evaluated.
--------------------	--

Definition at line 34 of file shapefunction.cpp.

8.58.2.4 ~ShapeFunction() `mnix::ShapeFunction::~~ShapeFunction ( ) [virtual]`

Destructor for [ShapeFunction](#).

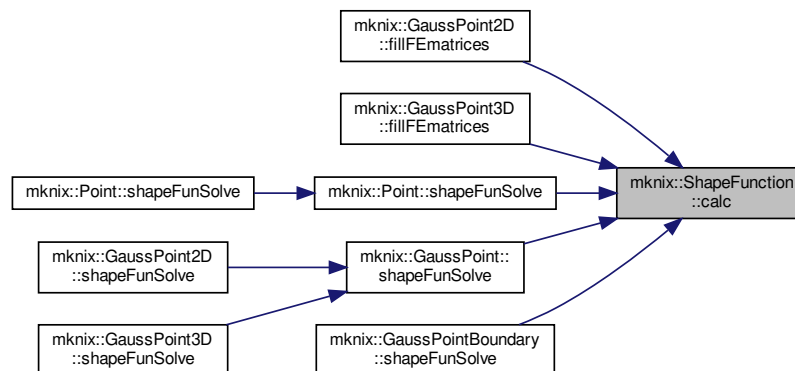
Definition at line 44 of file shapefunction.cpp.

8.58.3 Member Function Documentation

8.58.3.1 calc() `virtual void mnix::ShapeFunction::calc ( ) [pure virtual]`

Implemented in [mnix::ShapeFunctionTriangleSigned](#), [mnix::ShapeFunctionTriangle](#), [mnix::ShapeFunctionTetrahedron](#), [mnix::ShapeFunctionRBF](#), [mnix::ShapeFunctionMLS3D](#), [mnix::ShapeFunctionMLS2D](#), [mnix::ShapeFunctionMLS](#), [mnix::ShapeFunctionLinear](#), and [mnix::ShapeFunctionLinearX](#).

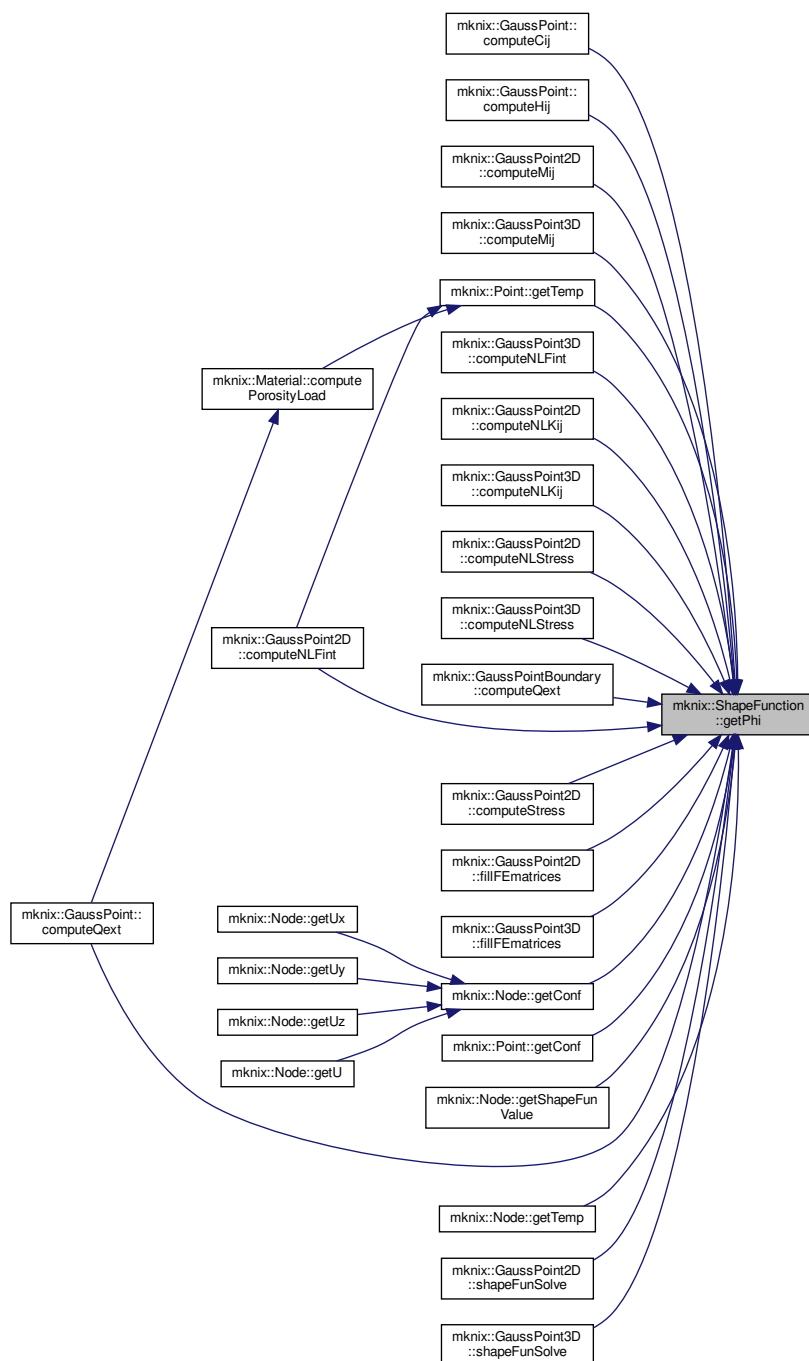
Here is the caller graph for this function:



8.58.3.2 getPhi() `virtual double mnix::ShapeFunction::getPhi ( size_t i, size_t j ) [inline], [virtual]`

Definition at line 63 of file shapefunction.h.

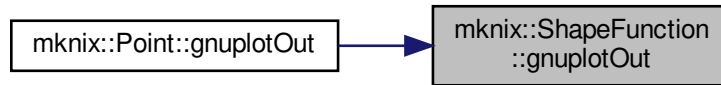
Here is the caller graph for this function:



8.58.3.3 gnuplotOut() void mknix::ShapeFunction::gnuplotOut () [virtual]

Writes shape function phi values and first derivatives for all support nodes to gnuplot-compatible output files.
Definition at line 83 of file shapefunction.cpp.

Here is the caller graph for this function:



8.58.3.4 outputValues() `void mknix::ShapeFunction::outputValues ( ) [virtual]`
 Prints shape function values (phi and derivatives) for all support nodes to stdout.
 Definition at line 51 of file shapefunction.cpp.

8.58.3.5 setPhi() `virtual void mknix::ShapeFunction::setPhi (`
 `double value_in,`
 `size_t i,`
 `size_t j ) [inline], [virtual]`
 Definition at line 68 of file shapefunction.h.

8.58.4 Member Data Documentation

8.58.4.1 dim `size_t mknix::ShapeFunction::dim [protected]`
 Definition at line 48 of file shapefunction.h.

8.58.4.2 gp `Point* mknix::ShapeFunction::gp [protected]`
 Definition at line 50 of file shapefunction.h.

8.58.4.3 phi `lmx::DenseMatrix<double> mknix::ShapeFunction::phi [protected]`
 Definition at line 49 of file shapefunction.h.

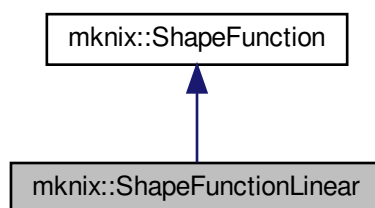
The documentation for this class was generated from the following files:

- [shapefunction.h](#)
- [shapefunction.cpp](#)

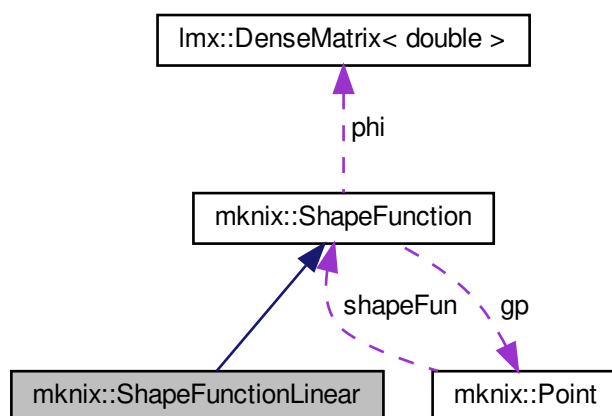
8.59 mknix::ShapeFunctionLinear Class Reference

```
#include <shapefunctionlinear.h>
```

Inheritance diagram for mknix::ShapeFunctionLinear:



Collaboration diagram for mknix::ShapeFunctionLinear:



Public Member Functions

- [ShapeFunctionLinear](#) ()
Default constructor for [ShapeFunctionLinear](#).
- [ShapeFunctionLinear](#) ([Point](#) *)
Constructs a [ShapeFunctionLinear](#) attached to the given [Point](#).
- [~ShapeFunctionLinear](#) ()
Destructor for [ShapeFunctionLinear](#).
- void [calc](#) ()
Computes linear (Lagrange) 1D shape functions and their derivatives using signed distance. The shape functions are defined over the segment between the two support nodes.

Additional Inherited Members

8.59.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file shapefunctionlinear.h.

8.59.2 Constructor & Destructor Documentation

8.59.2.1 ShapeFunctionLinear() [1/2] `mknix::ShapeFunctionLinear::ShapeFunctionLinear ( )`

Default constructor for [ShapeFunctionLinear](#).

Definition at line 30 of file shapefunctionlinear.cpp.

8.59.2.2 ShapeFunctionLinear() [2/2] `mknix::ShapeFunctionLinear::ShapeFunctionLinear ( Point * gp_in )`

Constructs a [ShapeFunctionLinear](#) attached to the given [Point](#).

Parameters

<code>gp_in</code>	Pointer to the evaluation Point (expects exactly 2 support nodes).
--------------------	--

Definition at line 41 of file shapefunctionlinear.cpp.

8.59.2.3 ~ShapeFunctionLinear() `mknix::ShapeFunctionLinear::~~ShapeFunctionLinear ( )`

Destructor for [ShapeFunctionLinear](#).

Definition at line 51 of file shapefunctionlinear.cpp.

8.59.3 Member Function Documentation

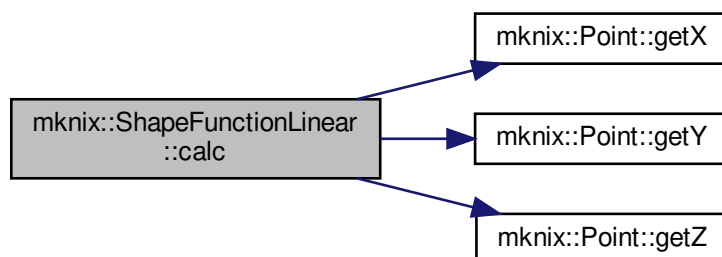
8.59.3.1 calc() `void mknix::ShapeFunctionLinear::calc ( ) [virtual]`

Computes linear (Lagrange) 1D shape functions and their derivatives using signed distance. The shape functions are defined over the segment between the two support nodes.

Implements [mknix::ShapeFunction](#).

Definition at line 60 of file shapefunctionlinear.cpp.

Here is the call graph for this function:



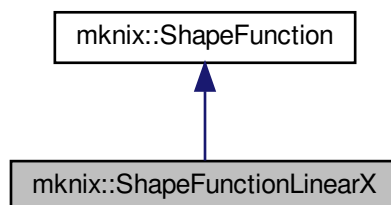
The documentation for this class was generated from the following files:

- [shapefunctionlinear.h](#)
- [shapefunctionlinear.cpp](#)

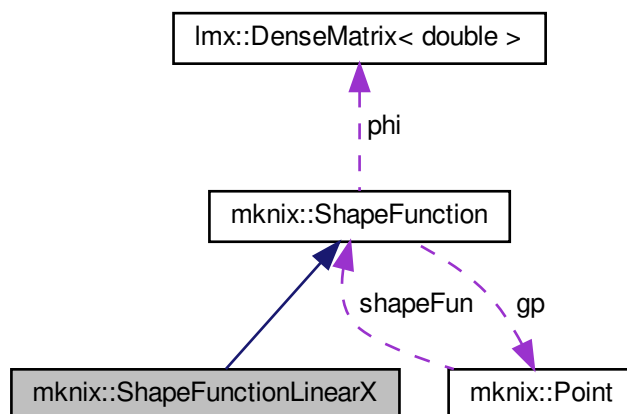
8.60 mknix::ShapeFunctionLinearX Class Reference

```
#include <shapefunctionlinear-x.h>
```

Inheritance diagram for mknix::ShapeFunctionLinearX:



Collaboration diagram for mknix::ShapeFunctionLinearX:



Public Member Functions

- [ShapeFunctionLinearX](#) ()
Default constructor for *ShapeFunctionLinearX*.
- [ShapeFunctionLinearX](#) (Point *)
Constructs a *ShapeFunctionLinearX* attached to the given *Point*.
- [~ShapeFunctionLinearX](#) ()
Destructor for *ShapeFunctionLinearX*.
- void [calc](#) ()

Computes linear (Lagrange) 1D shape functions using only the x-component of the nodal positions. This is a simplified version for boundary cells aligned with the X axis.

Additional Inherited Members

8.60.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file shapefunctionlinear-x.h.

8.60.2 Constructor & Destructor Documentation

8.60.2.1 ShapeFunctionLinearX() [1/2] `mnix::ShapeFunctionLinearX::ShapeFunctionLinearX ( )`

Default constructor for [ShapeFunctionLinearX](#).

Definition at line 30 of file shapefunctionlinear-x.cpp.

8.60.2.2 ShapeFunctionLinearX() [2/2] `mnix::ShapeFunctionLinearX::ShapeFunctionLinearX ( Point * gp_in )`

Constructs a [ShapeFunctionLinearX](#) attached to the given [Point](#).

Parameters

<code>gp_in</code>	Pointer to the evaluation Point (expects exactly 2 support nodes).
--------------------	--

Definition at line 41 of file shapefunctionlinear-x.cpp.

8.60.2.3 ~ShapeFunctionLinearX() `mnix::ShapeFunctionLinearX::~~ShapeFunctionLinearX ( )`

Destructor for [ShapeFunctionLinearX](#).

Definition at line 51 of file shapefunctionlinear-x.cpp.

8.60.3 Member Function Documentation

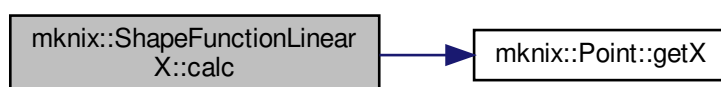
8.60.3.1 calc() `void mnix::ShapeFunctionLinearX::calc ( ) [virtual]`

Computes linear (Lagrange) 1D shape functions using only the x-component of the nodal positions. This is a simplified version for boundary cells aligned with the X axis.

Implements [mnix::ShapeFunction](#).

Definition at line 60 of file shapefunctionlinear-x.cpp.

Here is the call graph for this function:



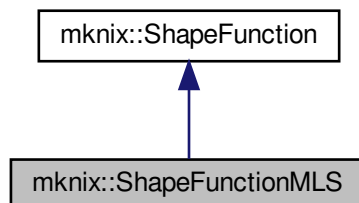
The documentation for this class was generated from the following files:

- [shapefunctionlinear-x.h](#)
- [shapefunctionlinear-x.cpp](#)

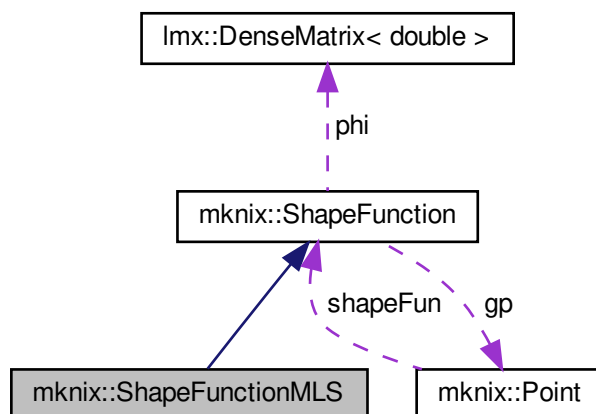
8.61 mknix::ShapeFunctionMLS Class Reference

```
#include <shapefunctionMLS.h>
```

Inheritance diagram for mknix::ShapeFunctionMLS:



Collaboration diagram for mknix::ShapeFunctionMLS:



Public Member Functions

- [ShapeFunctionMLS](#) ()
Default constructor for [ShapeFunctionMLS](#).
- [ShapeFunctionMLS](#) (int, int, int, double &, double &, [Point](#) *)
Constructs a Moving Least Squares (MLS) shape function for any spatial dimension.
- [~ShapeFunctionMLS](#) ()
Destructor for [ShapeFunctionMLS](#).
- void [calc](#) ()
Computes MLS shape functions and derivatives by calling `weight`, `moment matrix`, and `phi` routines.

Additional Inherited Members

8.61.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file shapefunctionMLS.h.

8.61.2 Constructor & Destructor Documentation

8.61.2.1 ShapeFunctionMLS() [1/2] `mknix::ShapeFunctionMLS::ShapeFunctionMLS ( )`

Default constructor for [ShapeFunctionMLS](#).

Container of B_I

Definition at line 13 of file shapefunctionMLS.cpp.

8.61.2.2 ShapeFunctionMLS() [2/2] `mknix::ShapeFunctionMLS::ShapeFunctionMLS (`

```
int nn_in,
int mm_in,
int weightType_in,
double & alpha_c_in,
double & d_c_in,
Point * gp_in )
```

Constructs a Moving Least Squares (MLS) shape function for any spatial dimension.

Parameters

<i>nn_in</i>	Number of support nodes.
<i>mm_in</i>	Polynomial basis order (1 = linear).
<i>weightType_in</i>	Weight function type (0 = exponential, 1 = cubic spline).
<i>alpha_c_in</i>	Influence radius scaling factor.
<i>d_c_in</i>	Characteristic nodal spacing.
<i>gp_in</i>	Pointer to the evaluation Point .

Definition at line 27 of file shapefunctionMLS.cpp.

8.61.2.3 ~ShapeFunctionMLS() `mknix::ShapeFunctionMLS::~~ShapeFunctionMLS ( )`

Destructor for [ShapeFunctionMLS](#).

Definition at line 95 of file shapefunctionMLS.cpp.

8.61.3 Member Function Documentation

8.61.3.1 calc() `void mknix::ShapeFunctionMLS::calc ( ) [virtual]`

Computes MLS shape functions and derivatives by calling weight, moment matrix, and phi routines.

Implements [mknix::ShapeFunction](#).

Definition at line 102 of file shapefunctionMLS.cpp.

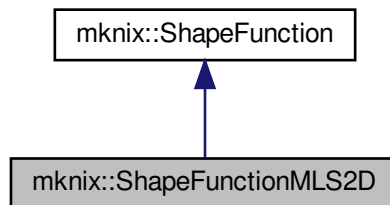
The documentation for this class was generated from the following files:

- [shapefunctionMLS.h](#)
- [shapefunctionMLS.cpp](#)

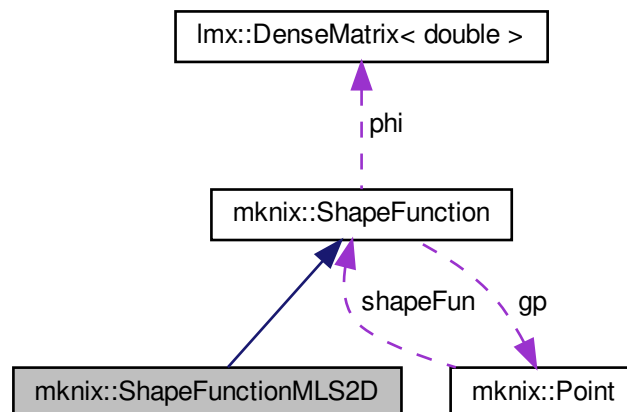
8.62 mknix::ShapeFunctionMLS2D Class Reference

```
#include <shapefunctionMLS2D.h>
```

Inheritance diagram for mknix::ShapeFunctionMLS2D:



Collaboration diagram for mknix::ShapeFunctionMLS2D:



Public Member Functions

- [ShapeFunctionMLS2D](#) ()
Default constructor for [ShapeFunctionMLS2D](#).
- [ShapeFunctionMLS2D](#) (int, int, int, double &, double &, [Point](#) *)
Constructs a 2D Moving Least Squares shape function.
- [~ShapeFunctionMLS2D](#) ()
Destructor for [ShapeFunctionMLS2D](#).
- void [calc](#) ()
Computes 2D MLS shape functions and derivatives.

Additional Inherited Members

8.62.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file shapefunctionMLS2D.h.

8.62.2 Constructor & Destructor Documentation**8.62.2.1 ShapeFunctionMLS2D()** [1/2] `mknix::ShapeFunctionMLS2D::ShapeFunctionMLS2D ( )`Default constructor for [ShapeFunctionMLS2D](#).Container of [B_I](#)

Definition at line 13 of file shapefunctionMLS2D.cpp.

8.62.2.2 ShapeFunctionMLS2D() [2/2] `mknix::ShapeFunctionMLS2D::ShapeFunctionMLS2D (`

```

    int nn_in,
    int mm_in,
    int weightType_in,
    double & alpha_c_in,
    double & d_c_in,
    Point * gp_in )

```

Constructs a 2D Moving Least Squares shape function.

Parameters

<i>nn_in</i>	Number of support nodes.
<i>mm_in</i>	Polynomial basis order (1 = linear).
<i>weightType_in</i>	Weight function type.
<i>alpha_c_in</i>	Influence radius scaling factor.
<i>d_c_in</i>	Characteristic nodal spacing.
<i>gp_in</i>	Pointer to the evaluation Point .

Definition at line 27 of file shapefunctionMLS2D.cpp.

8.62.2.3 ~ShapeFunctionMLS2D() `mknix::ShapeFunctionMLS2D::~~ShapeFunctionMLS2D ( )`Destructor for [ShapeFunctionMLS2D](#).

Definition at line 78 of file shapefunctionMLS2D.cpp.

8.62.3 Member Function Documentation**8.62.3.1 calc()** `void mknix::ShapeFunctionMLS2D::calc ( ) [virtual]`

Computes 2D MLS shape functions and derivatives.

Implements [mknix::ShapeFunction](#).

Definition at line 85 of file shapefunctionMLS2D.cpp.

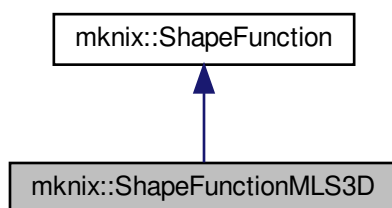
The documentation for this class was generated from the following files:

- [shapefunctionMLS2D.h](#)
- [shapefunctionMLS2D.cpp](#)

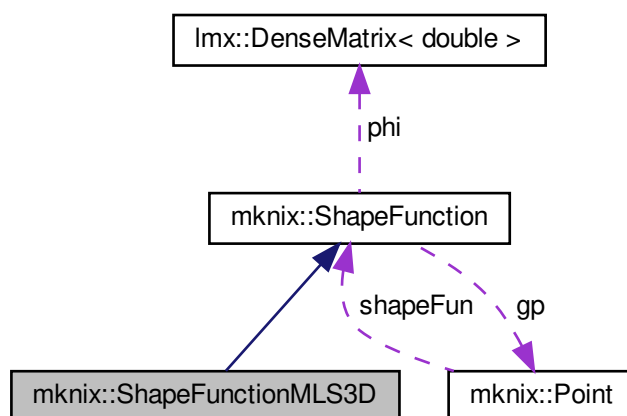
8.63 mknix::ShapeFunctionMLS3D Class Reference

#include <shapefunctionMLS3D.h>

Inheritance diagram for mknix::ShapeFunctionMLS3D:



Collaboration diagram for mknix::ShapeFunctionMLS3D:



Public Member Functions

- [ShapeFunctionMLS3D](#) ()
Default constructor for [ShapeFunctionMLS3D](#).
- [ShapeFunctionMLS3D](#) (int, int, int, double &, double &, [Point](#) *)
Constructs a 3D Moving Least Squares shape function.
- [~ShapeFunctionMLS3D](#) ()
Destructor for [ShapeFunctionMLS3D](#).
- void [calc](#) ()
Computes 3D MLS shape functions and derivatives.

Additional Inherited Members

8.63.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file shapefunctionMLS3D.h.

8.63.2 Constructor & Destructor Documentation**8.63.2.1 ShapeFunctionMLS3D()** [1/2] `mknix::ShapeFunctionMLS3D::ShapeFunctionMLS3D ( )`Default constructor for [ShapeFunctionMLS3D](#).Container of [B_I](#)

Definition at line 13 of file shapefunctionMLS3D.cpp.

8.63.2.2 ShapeFunctionMLS3D() [2/2] `mknix::ShapeFunctionMLS3D::ShapeFunctionMLS3D (`

```

    int nn_in,
    int mm_in,
    int weightType_in,
    double & alpha_c_in,
    double & d_c_in,
    Point * gp_in )

```

Constructs a 3D Moving Least Squares shape function.

Parameters

<i>nn_in</i>	Number of support nodes.
<i>mm_in</i>	Polynomial basis order (1 = linear).
<i>weightType_in</i>	Weight function type.
<i>alpha_c_in</i>	Influence radius scaling factor.
<i>d_c_in</i>	Characteristic nodal spacing.
<i>gp_in</i>	Pointer to the evaluation Point .

Definition at line 27 of file shapefunctionMLS3D.cpp.

8.63.2.3 ~ShapeFunctionMLS3D() `mknix::ShapeFunctionMLS3D::~~ShapeFunctionMLS3D ( )`Destructor for [ShapeFunctionMLS3D](#).

Definition at line 78 of file shapefunctionMLS3D.cpp.

8.63.3 Member Function Documentation**8.63.3.1 calc()** `void mknix::ShapeFunctionMLS3D::calc ( ) [virtual]`

Computes 3D MLS shape functions and derivatives.

Implements [mknix::ShapeFunction](#).

Definition at line 85 of file shapefunctionMLS3D.cpp.

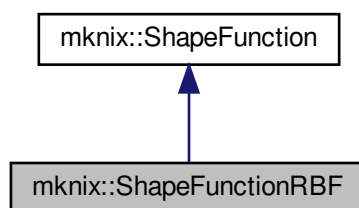
The documentation for this class was generated from the following files:

- [shapefunctionMLS3D.h](#)
- [shapefunctionMLS3D.cpp](#)

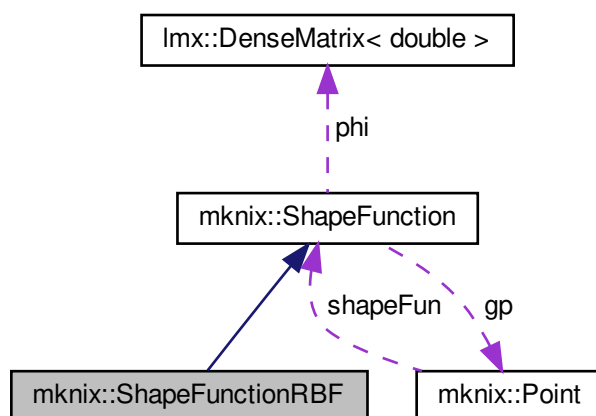
8.64 mknix::ShapeFunctionRBF Class Reference

#include <shapefunctionRBF.h>

Inheritance diagram for mknix::ShapeFunctionRBF:



Collaboration diagram for mknix::ShapeFunctionRBF:



Public Member Functions

- [ShapeFunctionRBF](#) ()
Default constructor for [ShapeFunctionRBF](#).
- [ShapeFunctionRBF](#) (size_t, size_t, int, double &, double &, double &, [Point](#) *)
Constructs a Radial Basis Function (RBF) shape function object.
- [~ShapeFunctionRBF](#) ()
Destructor for [ShapeFunctionRBF](#).
- void [calc](#) ()
Computes RBF shape functions by building the moment matrix and solving for phi.

Additional Inherited Members

8.64.1 Detailed Description

Author

Daniel Iglesias

Definition at line 43 of file shapefunctionRBF.h.

8.64.2 Constructor & Destructor Documentation**8.64.2.1 ShapeFunctionRBF()** [1/2] `mknix::ShapeFunctionRBF::ShapeFunctionRBF ( )`Default constructor for [ShapeFunctionRBF](#).

Definition at line 13 of file shapefunctionRBF.cpp.

8.64.2.2 ShapeFunctionRBF() [2/2] `mknix::ShapeFunctionRBF::ShapeFunctionRBF (`

```

    size_t nn_in,
    size_t mm_in,
    int rbfType_in,
    double & alpha_c_in,
    double & d_c_in,
    double & q_in,
    Point * gp_in )

```

Constructs a Radial Basis Function (RBF) shape function object.

Parameters

<i>nn_in</i>	Number of support nodes (radial basis functions).
<i>mm_in</i>	Number of polynomial basis terms.
<i>rbfType_in</i>	Type of RBF kernel (0 = multiquadric, etc.).
<i>alpha_c_in</i>	Influence radius scaling factor.
<i>d_c_in</i>	Characteristic nodal spacing.
<i>q_in</i>	Shape parameter for the RBF kernel.
<i>gp_in</i>	Pointer to the evaluation Point .

Definition at line 28 of file shapefunctionRBF.cpp.

8.64.2.3 ~ShapeFunctionRBF() `mknix::ShapeFunctionRBF::~~ShapeFunctionRBF ( )`Destructor for [ShapeFunctionRBF](#).

Definition at line 72 of file shapefunctionRBF.cpp.

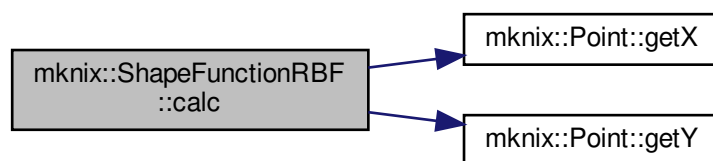
8.64.3 Member Function Documentation**8.64.3.1 calc()** `void mknix::ShapeFunctionRBF::calc ( ) [virtual]`

Computes RBF shape functions by building the moment matrix and solving for phi.

Implements [mknix::ShapeFunction](#).

Definition at line 79 of file shapefunctionRBF.cpp.

Here is the call graph for this function:



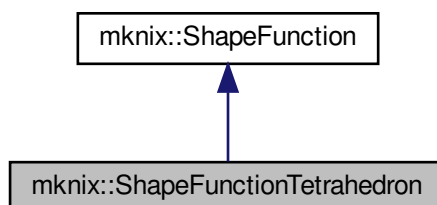
The documentation for this class was generated from the following files:

- [shapefunctionRBF.h](#)
- [shapefunctionRBF.cpp](#)

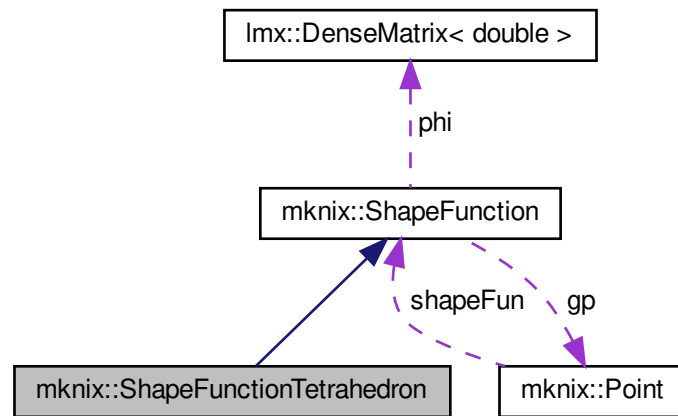
8.65 mknix::ShapeFunctionTetrahedron Class Reference

```
#include <shapefunctiontetrahedron.h>
```

Inheritance diagram for `mknix::ShapeFunctionTetrahedron`:



Collaboration diagram for `mknix::ShapeFunctionTetrahedron`:



Public Member Functions

- [ShapeFunctionTetrahedron](#) ()
Default constructor for [ShapeFunctionTetrahedron](#).
- [ShapeFunctionTetrahedron](#) ([Point](#) *)
Constructs a tetrahedral FEM shape function for the given Gauss point.
- [~ShapeFunctionTetrahedron](#) ()
Destructor for [ShapeFunctionTetrahedron](#).
- void [calc](#) ()
Computes the tetrahedral FEM shape functions and their spatial derivatives using the nodal coordinate sub-determinants (Zienkiewicz & Taylor formulation).
- void [compute_abcd](#) (int, int, int)
Computes the a, b, c, d coefficients for one tetrahedral shape function using the 3x3 sub-determinant method.

Additional Inherited Members

8.65.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file `shapefunctiontetrahedron.h`.

8.65.2 Constructor & Destructor Documentation

8.65.2.1 [ShapeFunctionTetrahedron](#)() [1/2] `mknix::ShapeFunctionTetrahedron::ShapeFunctionTetrahedron ( )`

Default constructor for [ShapeFunctionTetrahedron](#).

Definition at line 30 of file `shapefunctiontetrahedron.cpp`.

8.65.2.2 ShapeFunctionTetrahedron() [2/2] `mknix::ShapeFunctionTetrahedron::ShapeFunctionTetrahedron (
 Point * gp_in)`

Constructs a tetrahedral FEM shape function for the given Gauss point.

Parameters

<i>gp_in</i>	Pointer to the evaluation Point (expects exactly 4 support nodes).
--------------	--

Definition at line 40 of file `shapefunctiontetrahedron.cpp`.

8.65.2.3 ~ShapeFunctionTetrahedron() `mknix::ShapeFunctionTetrahedron::~~ShapeFunctionTetrahedron ( )`

Destructor for [ShapeFunctionTetrahedron](#).

Definition at line 52 of file `shapefunctiontetrahedron.cpp`.

8.65.3 Member Function Documentation

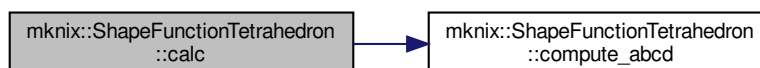
8.65.3.1 calc() `void mknix::ShapeFunctionTetrahedron::calc ( ) [virtual]`

Computes the tetrahedral FEM shape functions and their spatial derivatives using the nodal coordinate sub-determinants (Zienkiewicz & Taylor formulation).

Implements [mknix::ShapeFunction](#).

Definition at line 61 of file `shapefunctiontetrahedron.cpp`.

Here is the call graph for this function:



8.65.3.2 compute_abcd() `void mknix::ShapeFunctionTetrahedron::compute_abcd (
 int i1,
 int i2,
 int i3)`

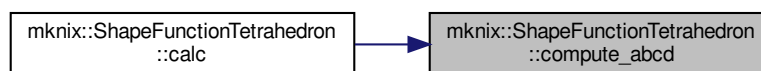
Computes the a, b, c, d coefficients for one tetrahedral shape function using the 3x3 sub-determinant method.

Parameters

<i>i1</i>	Index of the first vertex in the opposite face.
<i>i2</i>	Index of the second vertex in the opposite face.
<i>i3</i>	Index of the third vertex in the opposite face.

Definition at line 108 of file `shapefunctiontetrahedron.cpp`.

Here is the caller graph for this function:



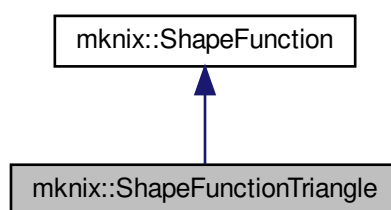
The documentation for this class was generated from the following files:

- [shapefunctiontetrahedron.h](#)
- [shapefunctiontetrahedron.cpp](#)

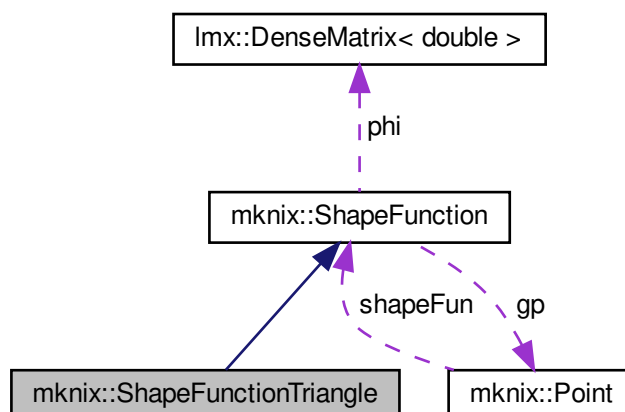
8.66 mknix::ShapeFunctionTriangle Class Reference

```
#include <shapefunctiontriangle.h>
```

Inheritance diagram for `mknix::ShapeFunctionTriangle`:



Collaboration diagram for `mknix::ShapeFunctionTriangle`:



Public Member Functions

- [ShapeFunctionTriangle](#) ()
Default constructor for [ShapeFunctionTriangle](#).
- [ShapeFunctionTriangle](#) ([Point](#) *)
Constructs a triangular FEM shape function for the given Gauss point.
- [~ShapeFunctionTriangle](#) ()
Destructor for [ShapeFunctionTriangle](#).
- void [calc](#) ()
Computes linear triangular (area-coordinate) FEM shape functions and first spatial derivatives using the standard Zienkiewicz & Taylor formula.

Additional Inherited Members

8.66.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file shapefunctiontriangle.h.

8.66.2 Constructor & Destructor Documentation

8.66.2.1 ShapeFunctionTriangle() [1/2] mknix::ShapeFunctionTriangle::ShapeFunctionTriangle ()

Default constructor for [ShapeFunctionTriangle](#).

Definition at line 32 of file shapefunctiontriangle.cpp.

8.66.2.2 ShapeFunctionTriangle() [2/2] mknix::ShapeFunctionTriangle::ShapeFunctionTriangle ([Point](#) * gp_in)

Constructs a triangular FEM shape function for the given Gauss point.

Parameters

gp_in	Pointer to the evaluation Point (expects exactly 3 support nodes).
-----------------------	--

Definition at line 43 of file shapefunctiontriangle.cpp.

8.66.2.3 ~ShapeFunctionTriangle() mknix::ShapeFunctionTriangle::~~ShapeFunctionTriangle ()

Destructor for [ShapeFunctionTriangle](#).

Definition at line 53 of file shapefunctiontriangle.cpp.

8.66.3 Member Function Documentation

8.66.3.1 calc() void mknix::ShapeFunctionTriangle::calc () [virtual]

Computes linear triangular (area-coordinate) FEM shape functions and first spatial derivatives using the standard Zienkiewicz & Taylor formula.

Implements [mknix::ShapeFunction](#).

Definition at line 62 of file shapefunctiontriangle.cpp.

The documentation for this class was generated from the following files:

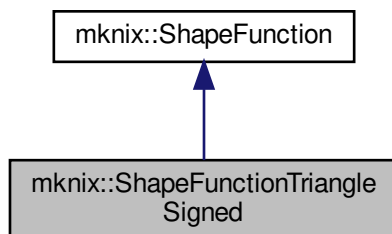
- [shapefunctiontriangle.h](#)

- [shapefunctiontriangle.cpp](#)

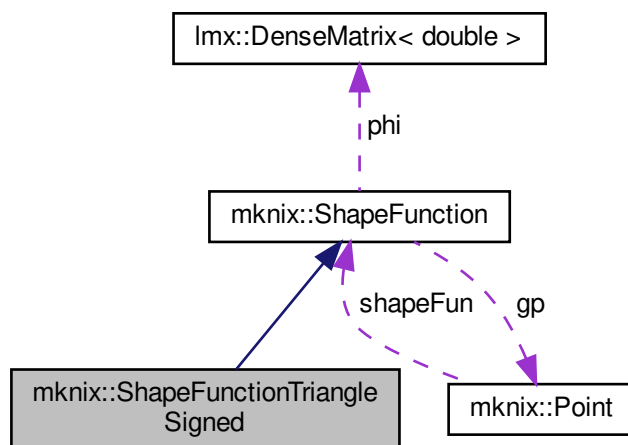
8.67 mknix::ShapeFunctionTriangleSigned Class Reference

```
#include <shapefunctiontriangle3D.h>
```

Inheritance diagram for mknix::ShapeFunctionTriangleSigned:



Collaboration diagram for mknix::ShapeFunctionTriangleSigned:



Public Member Functions

- [ShapeFunctionTriangleSigned \(\)](#)
Default constructor for *ShapeFunctionTriangleSigned*.
- [ShapeFunctionTriangleSigned \(Point *\)](#)
Constructs a signed triangular shape function for a triangle embedded in 3D space.
- [~ShapeFunctionTriangleSigned \(\)](#)
Destructor for *ShapeFunctionTriangleSigned*.
- void [calc \(\)](#)
Computes barycentric (signed-area) shape functions for a triangle embedded in 3D space, using cross products to obtain area ratios relative to the total triangle area.

Additional Inherited Members

8.67.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 31 of file shapefunctiontriangle3D.h.

8.67.2 Constructor & Destructor Documentation

8.67.2.1 ShapeFunctionTriangleSigned() [1/2] mknix::ShapeFunctionTriangleSigned::ShapeFunctionTriangleSigned ()

Default constructor for [ShapeFunctionTriangleSigned](#).

Definition at line 30 of file shapefunctiontriangle3D.cpp.

8.67.2.2 ShapeFunctionTriangleSigned() [2/2] mknix::ShapeFunctionTriangleSigned::ShapeFunctionTriangleSigned (

[Point](#) * gp_in)

Constructs a signed triangular shape function for a triangle embedded in 3D space.

Parameters

gp_in	Pointer to the evaluation Point (expects exactly 3 support nodes).
-----------------------	--

Definition at line 41 of file shapefunctiontriangle3D.cpp.

8.67.2.3 ~ShapeFunctionTriangleSigned() mknix::ShapeFunctionTriangleSigned::~ShapeFunctionTriangleSigned ()

Destructor for [ShapeFunctionTriangleSigned](#).

Definition at line 51 of file shapefunctiontriangle3D.cpp.

8.67.3 Member Function Documentation

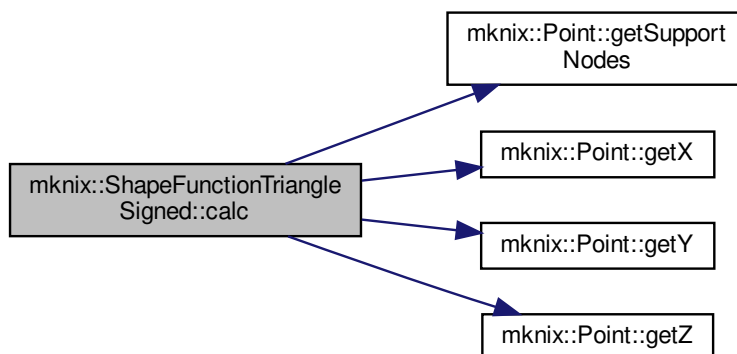
8.67.3.1 calc() void mknix::ShapeFunctionTriangleSigned::calc () [virtual]

Computes barycentric (signed-area) shape functions for a triangle embedded in 3D space, using cross products to obtain area ratios relative to the total triangle area.

Implements [mknix::ShapeFunction](#).

Definition at line 60 of file shapefunctiontriangle3D.cpp.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [shapefunctiontriangle3D.h](#)
- [shapefunctiontriangle3D.cpp](#)

8.68 mknix::Simulation Class Reference

```
#include <simulation.h>
```

Public Types

- enum class [OutputType](#) { [POSITION](#) , [FORCE](#) , [STRESS](#) }

Public Member Functions

- [Simulation](#) ()
Constructs a [Simulation](#) object and initializes output files and the global timer.
- [~Simulation](#) ()
Destroys the [Simulation](#) object and frees allocated timers and output file streams.
- [Simulation](#) (const [Simulation](#) &)=delete
- [Simulation](#) & operator= (const [Simulation](#) &)=delete
- void [inputFromFile](#) (const std::string &fileIn)
Reads and parses the simulation configuration from an input file.
- size_t [getInterfaceNumberOfNodes](#) (const std::string &name) const
Returns the number of nodes in a named subsystem interface.
- [Node](#) * [getInterfaceNode](#) (const std::string &system_name, size_t num) const
Returns a node from a named subsystem by index.
- std::vector< [Node](#) * > [getSignalNodes](#) (const std::string &system_name, const std::string &name) const
Returns the signal nodes associated with a named signal in a subsystem.
- [Node](#) * [getOutputNode](#) (const std::string &system_name, const std::string &name) const
Returns an output node identified by name from a subsystem.
- std::vector< string > [getConstraintNames](#) (const std::string &systemName="") const
Returns the names of all constraints defined in a subsystem.
- [Constraint](#) * [getConstraint](#) (const std::string &constraintName, const std::string &systemName="") const
Returns a constraint by name from a subsystem, searching both mechanical and thermal constraints.

- double [getConstraintOutput](#) (const std::string &constraintName, const std::string &systemName="", size_t component=0)
Returns the internal reaction force of a constraint for a given component.
- std::vector< double > [getInterfaceNodesCoords](#) ()
Returns the coordinates of all thermal interface nodes.
- void [setOutputFilesDetail](#) (int level_in)
- void [init](#) (int verbosity=2)
Initializes all analyses without executing the full simulation loop.
- void [setInitialTemperatures](#) (double)
Sets the uniform initial temperature applied to all thermal nodes.
- void [setThermalNodeInitialTemperature](#) (Node *node, double temperature)
Sets the initial temperature for a specific thermal node, overriding the uniform value.
- double [getInitialTemperature](#) () const
- void [solveStep](#) ()
Advances the current analysis by one time step.
- void [solveStep](#) (double *, double *o_output=0)
Advances the current analysis by one time step, applying an input signal and capturing an output signal.
- void [endSimulation](#) ()
Finalizes the simulation, closes output files, and writes remaining results.
- std::vector< std::string > [bodyNames](#) ()
Returns the names of all flexible bodies in the base system.
- std::vector< double > [bodyPoints](#) (const std::string &system_name, const std::string &body_name) const
Returns the current nodal coordinates of all nodes belonging to a named body in a subsystem.
- void [run](#) ()
Executes the complete simulation by running all defined analyses in sequence.
- void [runThermalAnalysis](#) (Analysis *)
Initializes and solves a thermal or thermalstatic analysis.
- void [runMechanicalAnalysis](#) (Analysis *)
Initializes and solves a mechanical (static, dynamic, or thermo-mechanical dynamic) analysis.
- void [writeSystem](#) ()
Writes the initial system topology (nodes, rigid/flex bodies, joints) to the main output file and a nodes.dat file.
- void [staticThermalResidue](#) (Imx::Vector< data_type > &residue, Imx::Vector< data_type > &q)
Computes the residue vector for the static thermal equilibrium problem.
- void [staticThermalTangent](#) (Imx::Matrix< data_type > &tangent_in, Imx::Vector< data_type > &q)
Computes the tangent (Jacobian) matrix for the static thermal problem.
- bool [staticThermalConvergence](#) (Imx::Vector< data_type > &res, Imx::Vector< data_type > &q)
Checks whether the static thermal solver has converged and handles post-convergence output.
- void [explicitThermalEvaluation](#) (const Imx::Vector< data_type > &q, Imx::Vector< data_type > &qdot, double time)
Evaluates the thermal time derivative for explicit time integration.
- void [dynamicThermalEvaluation](#) (const Imx::Vector< data_type > &q, Imx::Vector< data_type > &qdot, double time)
Evaluates the thermal time derivative for implicit dynamic time integration, recomputing the conductivity and capacity matrices at each call.
- void [dynamicThermalResidue](#) (Imx::Vector< data_type > &residue, const Imx::Vector< data_type > &q, const Imx::Vector< data_type > &qdot, double time)
Computes the residue vector for the dynamic thermal problem ($C \cdot qdot + K \cdot q + f_{int} - f_{ext}$).
- void [dynamicThermalTangent](#) (Imx::Matrix< data_type > &tangent_in, const Imx::Vector< data_type > &q, double partial_qdot, double time)
*Computes the tangent matrix for the dynamic thermal problem as $partial_qdot * C + K$.*
- bool [dynamicThermalConvergence](#) (const Imx::Vector< data_type > &q, const Imx::Vector< data_type > &qdot, double time)

Checks convergence for the dynamic thermal solver and handles post-convergence output.

- bool `dynamicThermalConvergenceInThermomechanical` (const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, double time)

Checks thermal convergence within a coupled thermo-mechanical dynamic analysis.

- void `explicitAcceleration` (const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, `Imx::Vector< data_type >` &qddot, double time)

Computes the nodal acceleration for explicit mechanical time integration by solving $M \cdot qddot = f_{ext} - f_{int}$.

- void `dynamicAcceleration` (const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, `Imx::Vector< data_type >` &qddot, double time)

Computes the nodal acceleration for implicit dynamic mechanical analysis, recomputing the mass matrix at each call.

- void `dynamicResidue` (`Imx::Vector< data_type >` &residue, const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, const `Imx::Vector< data_type >` &qddot, double time)

Computes the residue vector for the dynamic mechanical problem ($M \cdot qddot + f_{int} - f_{ext}$).

- void `dynamicTangent` (`Imx::Matrix< data_type >` &tangent_in, const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, double partial_qdot, double partial_qddot, double time)

*Computes the tangent matrix for the dynamic mechanical problem as $K + partial_qddot * M$.*

- bool `dynamicConvergence` (const `Imx::Vector< data_type >` &q, const `Imx::Vector< data_type >` &qddot, const `Imx::Vector< data_type >` &qddot, double time)

Checks convergence for the dynamic mechanical solver and handles post-convergence output.

- void `staticResidue` (`Imx::Vector< data_type >` &residue, `Imx::Vector< data_type >` &q)

Computes the residue vector for the static mechanical equilibrium problem ($f_{int} - f_{ext}$).

- void `staticTangent` (`Imx::Matrix< data_type >` &tangent_in, `Imx::Vector< data_type >` &q)

Computes the tangent (stiffness) matrix for the static mechanical problem.

- bool `staticConvergence` (`Imx::Vector< data_type >` &res, `Imx::Vector< data_type >` &q)

Checks convergence for the static mechanical solver and handles post-convergence output.

- void `stepTriggered` ()

Callback invoked after a successful step convergence; writes configuration and updates timing output.

- void `writeConfStep` ()

Writes the current nodal configuration at the current step time to the displacement output file.

- `Imx::DenseMatrix< data_type >` & `getSparsePattern` ()

Static Public Member Functions

- static double `getGravity` (int component)

Returns a component of the global gravity vector.

- static double `getAlpha` ()

Returns the penalty parameter alpha used for constraint enforcement.

- static double `getAugmentedTolerance` ()

Returns the convergence tolerance for the augmented Lagrangian method.

- static void `setAugmentedTolerance` (double tolerance)

Sets the convergence tolerance for the augmented Lagrangian method.

- static double `getTime` ()

Returns the current simulation step time.

- static int `getDim` ()

Returns the spatial dimension of the simulation (2 or 3).

- static std::string `getConstraintMethod` ()

Returns the name of the constraint enforcement method in use.

- static std::string `getSmoothingType` ()

Returns the smoothing type used in the simulation.

Friends

- class [Reader](#)
- class [ReaderConstraints](#)
- class [ReaderFlex](#)
- class [ReaderRigid](#)
- class [Contact](#)
- class [SystemChain](#)

8.68.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 57 of file simulation.h.

8.68.2 Member Enumeration Documentation

8.68.2.1 OutputType `enum mknix::Simulation::OutputType [strong]`

Enumerator

POSITION	
FORCE	
STRESS	

Definition at line 108 of file simulation.h.

8.68.3 Constructor & Destructor Documentation

8.68.3.1 Simulation() [1/2] `mknix::Simulation::Simulation ( )`

Constructs a [Simulation](#) object and initializes output files and the global timer.

Definition at line 123 of file simulation.cpp.

8.68.3.2 ~Simulation() `mknix::Simulation::~Simulation ( )`

Destroys the [Simulation](#) object and frees allocated timers and output file streams.

Definition at line 148 of file simulation.cpp.

8.68.3.3 Simulation() [2/2] `mknix::Simulation::Simulation ( const Simulation & ) [delete]`

8.68.4 Member Function Documentation

8.68.4.1 bodyNames() `std::vector< std::string > mknix::Simulation::bodyNames ( )`

Returns the names of all flexible bodies in the base system.

Returns

Vector of flexible body name strings.

Definition at line 1498 of file simulation.cpp.

8.68.4.2 bodyPoints() `std::vector< double > mknix::Simulation::bodyPoints (`
`const std::string & system_name,`
`const std::string & name ) const`

Returns the current nodal coordinates of all nodes belonging to a named body in a subsystem.

Parameters

<i>system_name</i>	Name of the subsystem containing the body.
<i>name</i>	Name of the body.

Returns

Flat vector of (x, y, z) coordinate triples for each node.

Definition at line 1509 of file simulation.cpp.

8.68.4.3 dynamicAcceleration() `void mknix::Simulation::dynamicAcceleration (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`lmx::Vector< data_type > & qddot,`
`double time )`

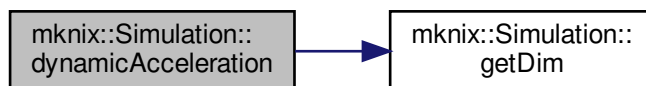
Computes the nodal acceleration for implicit dynamic mechanical analysis, recomputing the mass matrix at each call.

Parameters

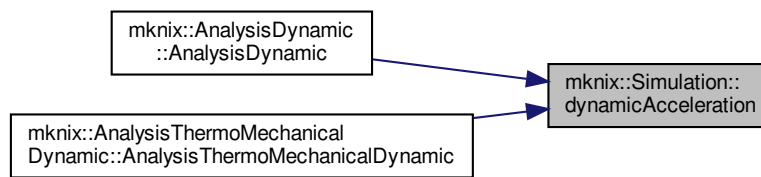
<i>q</i>	Current displacement state vector.
<i>qdot</i>	Current velocity state vector.
<i>qddot</i>	Output acceleration vector.

Definition at line 1177 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.4 dynamicConvergence() `bool mknix::Simulation::dynamicConvergence (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`const lmx::Vector< data_type > & qddot,`
`double time )`

Checks convergence for the dynamic mechanical solver and handles post-convergence output.

Parameters

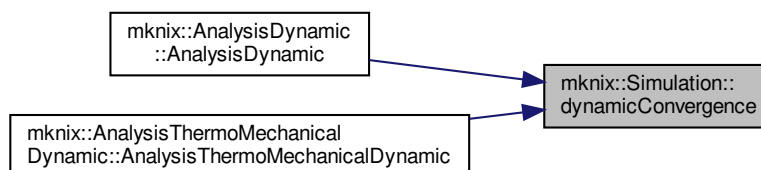
<i>q</i>	Current displacement state vector.
<i>qdot</i>	Current velocity state vector.
<i>qddot</i>	Current acceleration state vector.
<i>time</i>	Current simulation time.

Returns

True if the step has converged, false otherwise.

Definition at line 1282 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.5 dynamicResidue() `void mknix::Simulation::dynamicResidue (`
`lmx::Vector< data_type > & residue,`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`const lmx::Vector< data_type > & qddot,`
`double time )`

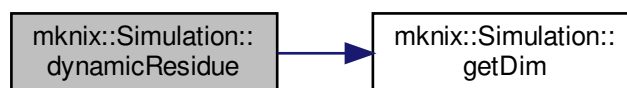
Computes the residue vector for the dynamic mechanical problem ($M \cdot \ddot{q} + f_{int} - f_{ext}$).

Parameters

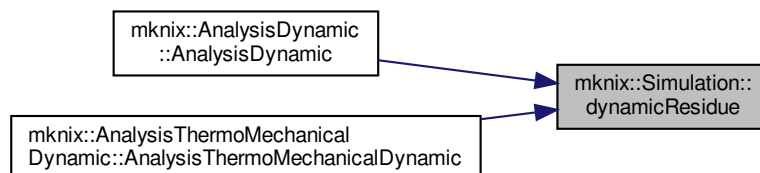
<i>residue</i>	Output residue vector.
<i>q</i>	Current displacement state vector.
<i>qdot</i>	Current velocity state vector.
<i>qddot</i>	Current acceleration state vector.
<i>time</i>	Current simulation time.

Definition at line 1215 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



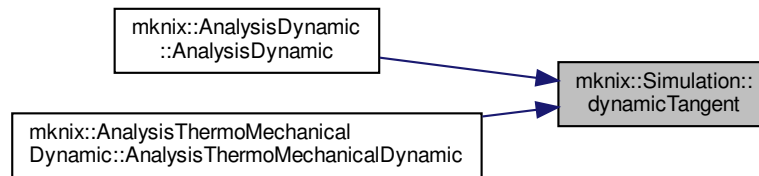
8.68.4.6 dynamicTangent() `void mknix::Simulation::dynamicTangent (`
`lmx::Matrix< data_type > & tangent_in,`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`double partial_qdot,`
`double partial_qddot,`
`double time )`

Computes the tangent matrix for the dynamic mechanical problem as $K + \text{partial_qddot} \cdot M$.

Parameters

<i>tangent_in</i>	Output tangent matrix.
<i>q</i>	Current displacement state vector.
<i>qdot</i>	Current velocity state vector.
<i>partial_qddot</i>	Partial derivative of qddot with respect to the unknown (from the time integrator).

Definition at line 1260 of file simulation.cpp.
Here is the caller graph for this function:



8.68.4.7 dynamicThermalConvergence() `bool mknix::Simulation::dynamicThermalConvergence (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`double time )`

Checks convergence for the dynamic thermal solver and handles post-convergence output.

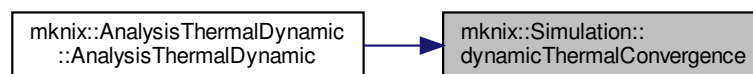
Parameters

<i>q</i>	Current temperature state vector.
<i>qdot</i>	Current temperature rate vector.
<i>time</i>	Current simulation time.

Returns

True if the step has converged, false otherwise.

Definition at line 1061 of file simulation.cpp.
Here is the caller graph for this function:



8.68.4.8 dynamicThermalConvergenceInThermomechanical() `bool mknix::Simulation::dynamicThermalConvergenceInThermomechanical (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot,`
`double time )`

Checks thermal convergence within a coupled thermo-mechanical dynamic analysis.

Parameters

<i>q</i>	Current temperature state vector.
----------	-----------------------------------

Parameters

<i>qdot</i>	Current temperature rate vector.
<i>time</i>	Current simulation time.

Returns

True if the thermal step has converged, false otherwise.

Definition at line 1107 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.9 dynamicThermalEvaluation() `void mknix::Simulation::dynamicThermalEvaluation (`
`const lmx::Vector< data_type > & qt,`
`lmx::Vector< data_type > & qtdot,`
`double time )`

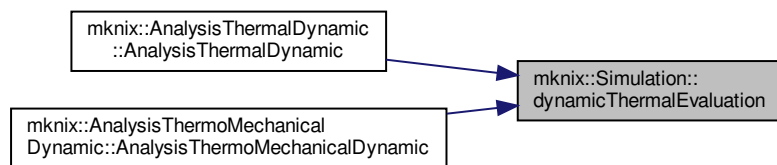
Evaluates the thermal time derivative for implicit dynamic time integration, recomputing the conductivity and capacity matrices at each call.

Parameters

<i>qt</i>	Current temperature state vector.
<i>qtdot</i>	Output temperature rate vector.
<i>time</i>	Current simulation time.

Definition at line 952 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.10 dynamicThermalResidue() `void mknix::Simulation::dynamicThermalResidue (`
`lmx::Vector< data_type > & residue,`
`const lmx::Vector< data_type > & q,`

```
const lmx::Vector< data_type > & qdot,
double time )
```

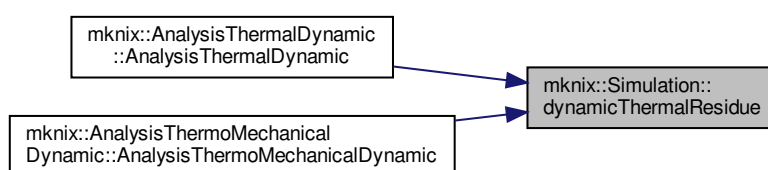
Computes the residue vector for the dynamic thermal problem ($C \cdot \dot{q} + K \cdot q + f_{\text{int}} - f_{\text{ext}}$).

Parameters

<i>residue</i>	Output residue vector.
<i>q</i>	Current temperature state vector.
<i>qdot</i>	Current temperature rate vector.

Definition at line 990 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.11 dynamicThermalTangent() void mknix::Simulation::dynamicThermalTangent (
 lmx::Matrix< data_type > & tangent_in,
 const lmx::Vector< data_type > & q,
 double partial_qdot,
 double time)

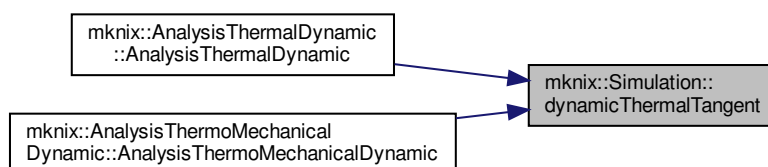
Computes the tangent matrix for the dynamic thermal problem as $\text{partial_qdot} * C + K$.

Parameters

<i>tangent_in</i>	Output tangent matrix.
<i>q</i>	Current temperature state vector.
<i>partial_qdot</i>	Partial derivative of qdot with respect to the unknown (from the time integrator).

Definition at line 1041 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.12 endSimulation() `void mknix::Simulation::endSimulation ( )`

Finalizes the simulation, closes output files, and writes remaining results.

Definition at line 513 of file simulation.cpp.

8.68.4.13 explicitAcceleration() `void mknix::Simulation::explicitAcceleration (`

```
const lmx::Vector< data_type > & q,
const lmx::Vector< data_type > & qdot,
lmx::Vector< data_type > & qddot,
double time )
```

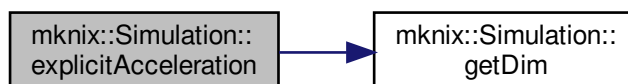
Computes the nodal acceleration for explicit mechanical time integration by solving $M \cdot qddot = f_{ext} - f_{int}$.

Parameters

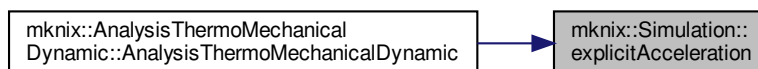
<i>q</i>	Current displacement state vector.
<i>qdot</i>	Current velocity state vector.
<i>qddot</i>	Output acceleration vector.
<i>time</i>	Current simulation time.

Definition at line 1137 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.14 explicitThermalEvaluation() `void mknix::Simulation::explicitThermalEvaluation (`

```
const lmx::Vector< data_type > & qt,
lmx::Vector< data_type > & qtdot,
double time )
```

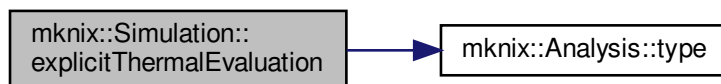
Evaluates the thermal time derivative for explicit time integration.

Parameters

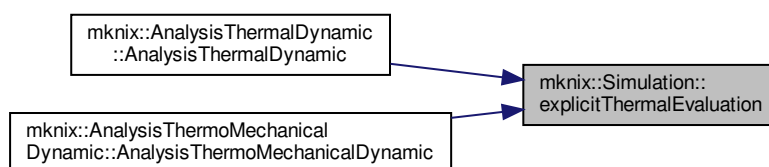
<i>qt</i>	Current temperature state vector.
<i>qtdot</i>	Output temperature rate vector (solved from $C \cdot qtdot = -(K \cdot qt + f_{int} - f_{ext})$).
<i>time</i>	Current simulation time.

Definition at line 906 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.15 getAlpha() `double mknix::Simulation::getAlpha ( ) [static]`

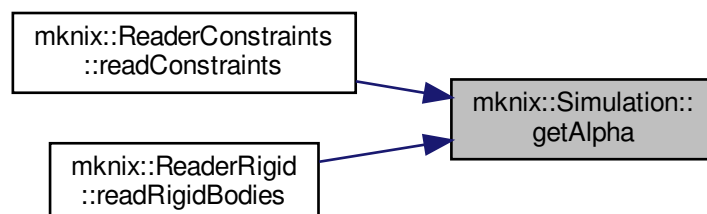
Returns the penalty parameter alpha used for constraint enforcement.

Returns

The alpha penalty parameter.

Definition at line 61 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.16 getAugmentedTolerance() `double mknix::Simulation::getAugmentedTolerance ( ) [static]`

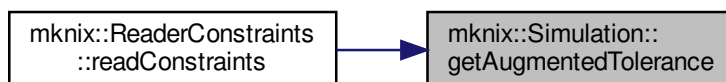
Returns the convergence tolerance for the augmented Lagrangian method.

Returns

The augmented Lagrangian tolerance value.

Definition at line 70 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.17 getConstraint() `Constraint * mknix::Simulation::getConstraint ( const std::string & constraintName, const std::string & systemName = "" ) const`

Returns a constraint by name from a subsystem, searching both mechanical and thermal constraints.

Parameters

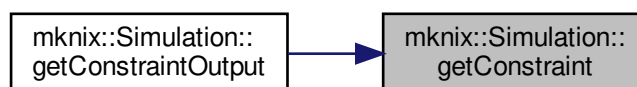
<i>constraintName</i>	Name of the constraint.
<i>systemName</i>	Name of the subsystem.

Returns

Pointer to the matching [Constraint](#), or nullptr if not found.

Definition at line 244 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.18 getConstraintMethod() `std::string mknix::Simulation::getConstraintMethod ( ) [static]`
Returns the name of the constraint enforcement method in use.

Returns

A string identifying the constraint method (e.g. "PENALTY").

Definition at line 106 of file simulation.cpp.

8.68.4.19 getConstraintNames() `std::vector< std::string > mknix::Simulation::getConstraintNames (`

`const std::string & systemName = "" ) const`

Returns the names of all constraints defined in a subsystem.

Parameters

<i>systemName</i>	Name of the subsystem.
-------------------	------------------------

Returns

Vector of constraint name strings.

Definition at line 232 of file simulation.cpp.

8.68.4.20 getConstraintOutput() `double mknix::Simulation::getConstraintOutput (`
`const std::string & constraintName,`
`const std::string & systemName = "",`
`size_t component = 0 )`

Returns the internal reaction force of a constraint for a given component.

Parameters

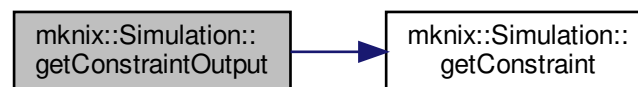
<i>constraintName</i>	Name of the constraint.
<i>systemName</i>	Name of the subsystem containing the constraint.
<i>component</i>	Index of the force component to retrieve.

Returns

The negated internal force value for the specified component.

Definition at line 262 of file simulation.cpp.

Here is the call graph for this function:



8.68.4.21 getDim() `int mknix::Simulation::getDim ( ) [static]`

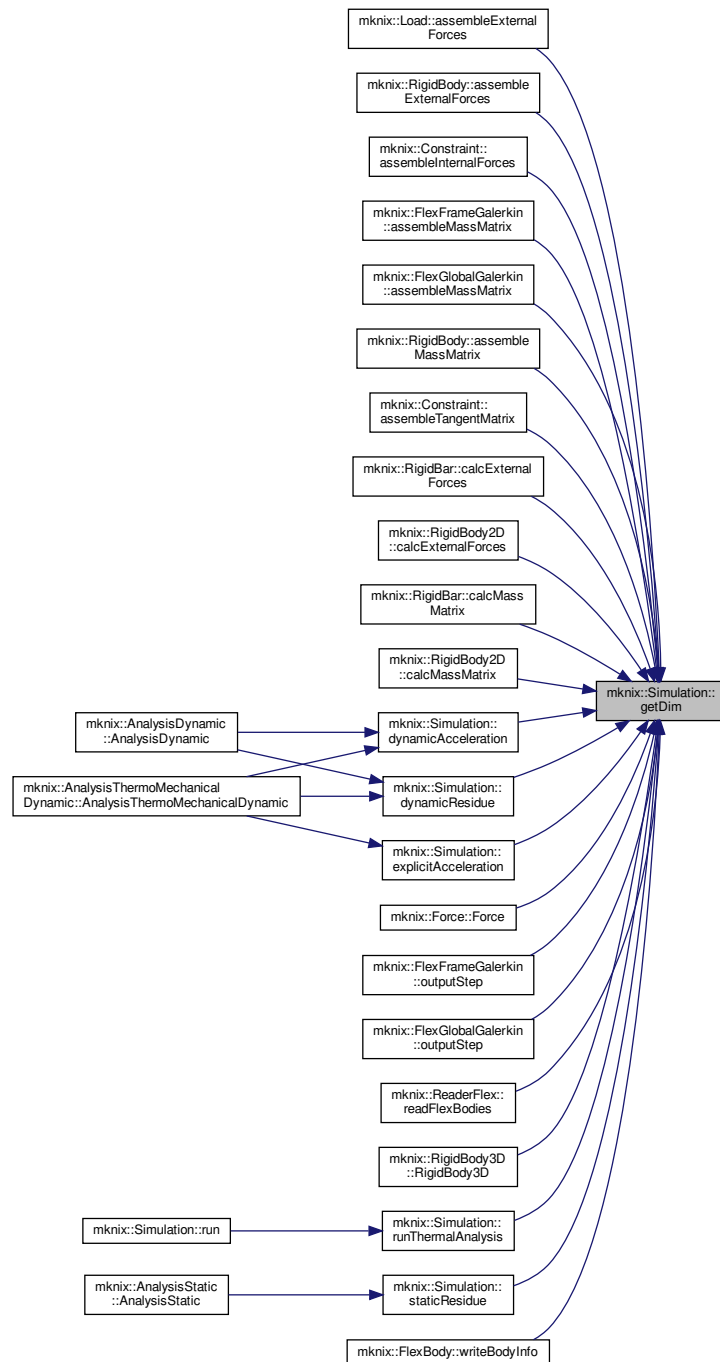
Returns the spatial dimension of the simulation (2 or 3).

Returns

The number of spatial dimensions.

Definition at line 97 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.22 `getGravity()` `double mknix::Simulation::getGravity (`
`int component ) [static]`

Returns a component of the global gravity vector.

Parameters

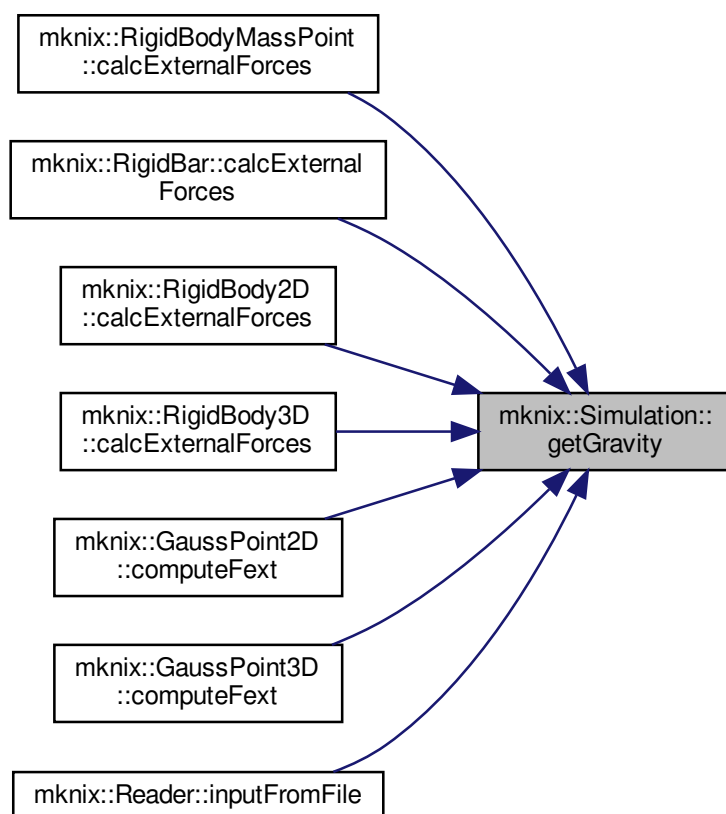
<i>component</i>	Index of the gravity vector component (0, 1, or 2).
------------------	---

Returns

The gravity value for the specified component.

Definition at line 52 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.23 `getInitialTemperature()` `double mknix::Simulation::getInitialTemperature ( ) const [inline]`

Definition at line 121 of file simulation.h.

8.68.4.24 `getInterfaceNode()` `Node * mknix::Simulation::getInterfaceNode ( const std::string & system_name, size_t num ) const`

Returns a node from a named subsystem by index.

Parameters

<i>system_name</i>	Name of the subsystem.
<i>num</i>	Zero-based index of the node.

Returns

Pointer to the requested [Node](#).

Definition at line 186 of file simulation.cpp.

8.68.4.25 getInterfaceNodesCoords() `std::vector< double > mknix::Simulation::getInterfaceNodesCoords ( )`

Returns the coordinates of all thermal interface nodes.

Returns

A vector of coordinate values for all thermal nodes.

Definition at line 217 of file simulation.cpp.

8.68.4.26 getInterfaceNumberOfNodes() `size_t mknix::Simulation::getInterfaceNumberOfNodes ( const std::string & name ) const`

Returns the number of nodes in a named subsystem interface.

Parameters

<i>name</i>	Name of the subsystem.
-------------	------------------------

Returns

Number of nodes in the specified subsystem.

Definition at line 175 of file simulation.cpp.

8.68.4.27 getOutputNode() `Node * mknix::Simulation::getOutputNode ( const std::string & system_name, const std::string & name ) const`

Returns an output node identified by name from a subsystem.

Parameters

<i>system_name</i>	Name of the subsystem.
<i>name</i>	Name of the output node.

Returns

Pointer to the requested output [Node](#).

Definition at line 208 of file simulation.cpp.

8.68.4.28 getSignalNodes() `std::vector< Node * > mknix::Simulation::getSignalNodes ( const std::string & system_name, const std::string & name ) const`

Returns the signal nodes associated with a named signal in a subsystem.

Parameters

<i>system_name</i>	Name of the subsystem.
<i>name</i>	Name of the signal.

Returns

Vector of pointers to the signal nodes.

Definition at line 197 of file simulation.cpp.

8.68.4.29 getSmoothingType() `std::string mknix::Simulation::getSmoothingType ( ) [static]`

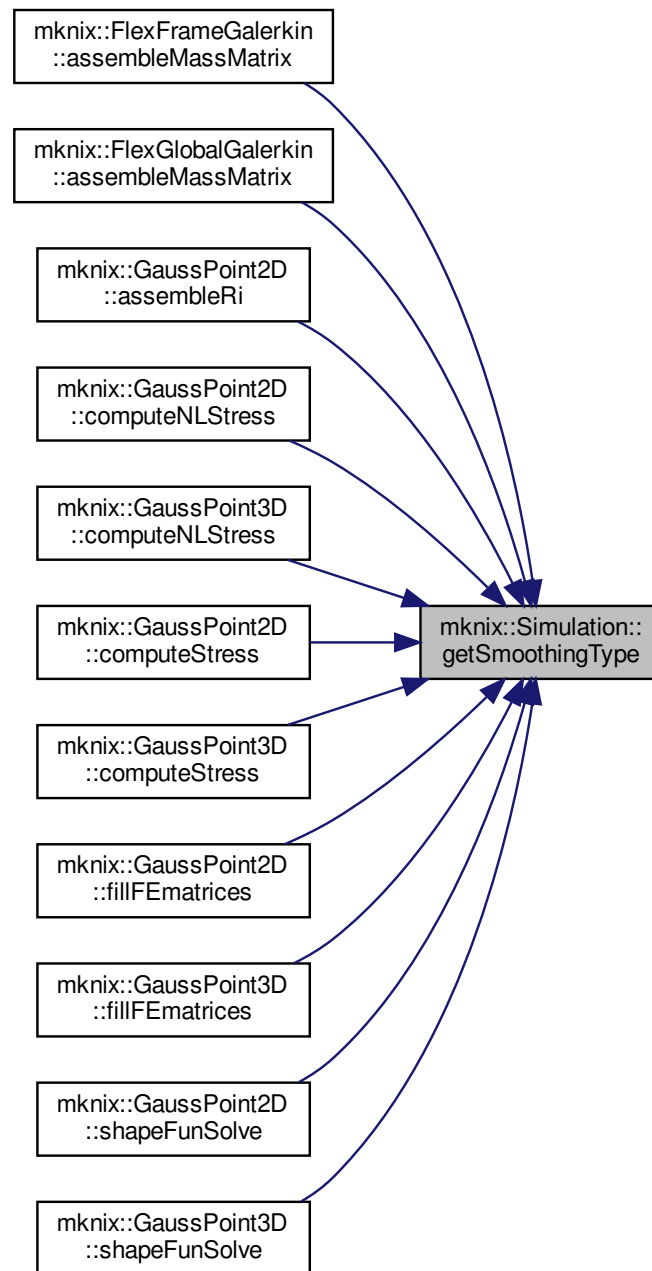
Returns the smoothing type used in the simulation.

Returns

A string identifying the smoothing type (e.g. "GLOBAL").

Definition at line 115 of file simulation.cpp.

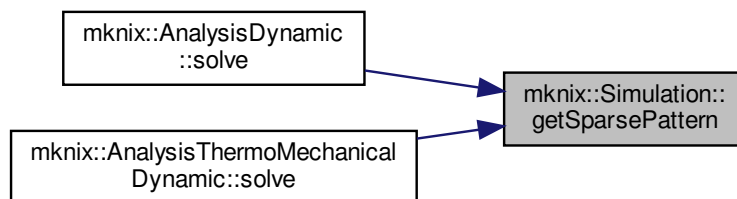
Here is the caller graph for this function:



8.68.4.30 `getSparsePattern()` `lmx::DenseMatrix<data_type>& mknix::Simulation::getSparsePattern (`
`) [inline]`

Definition at line 235 of file `simulation.h`.

Here is the caller graph for this function:



8.68.4.31 `getTime()` `double mknix::Simulation::getTime ( ) [static]`

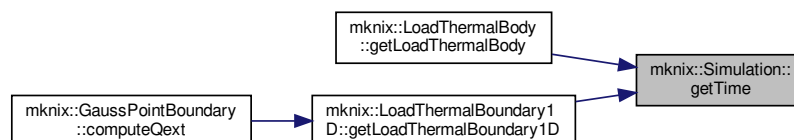
Returns the current simulation step time.

Returns

The current value of `stepTime`.

Definition at line 88 of file `simulation.cpp`.

Here is the caller graph for this function:



8.68.4.32 `init()` `void mknix::Simulation::init ( int verbosity = 2 )`

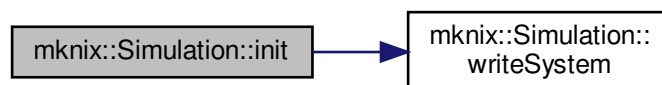
Initializes all analyses without executing the full simulation loop.

Parameters

<i>verbosity</i>	Verbosity level controlling diagnostic output.
------------------	--

Definition at line 308 of file `simulation.cpp`.

Here is the call graph for this function:



8.68.4.33 inputFromFile() `void mknix::Simulation::inputFromFile ( const std::string & FileIn )`

Reads and parses the simulation configuration from an input file.

Parameters

<i>FileIn</i>	Path to the input file to read.
---------------	---------------------------------

Definition at line 160 of file simulation.cpp.

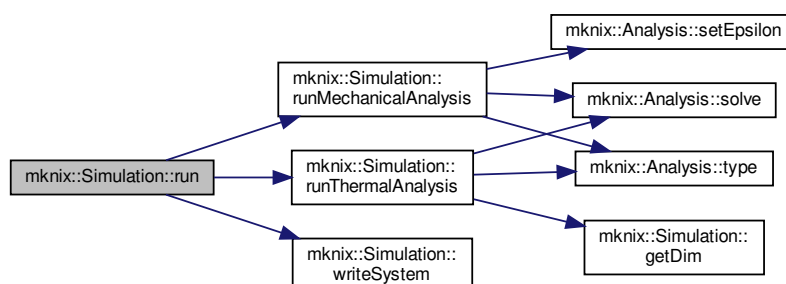
8.68.4.34 operator=() `Simulation& mknix::Simulation::operator= ( const Simulation & ) [delete]`

8.68.4.35 run() `void mknix::Simulation::run ( )`

Executes the complete simulation by running all defined analyses in sequence.

Definition at line 563 of file simulation.cpp.

Here is the call graph for this function:



8.68.4.36 runMechanicalAnalysis() `void mknix::Simulation::runMechanicalAnalysis ( Analysis * theAnalysis_in )`

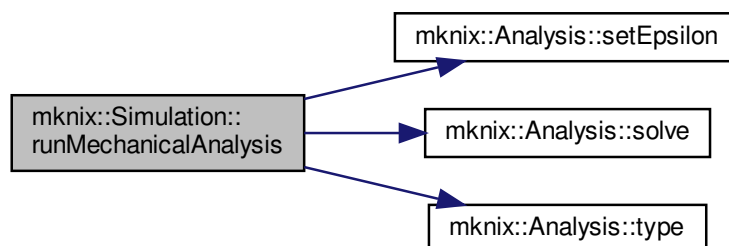
Initializes and solves a mechanical (static, dynamic, or thermo-mechanical dynamic) analysis.

Parameters

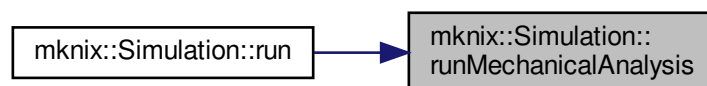
<i>theAnalysis</i> ↔ _in	Pointer to the mechanical Analysis object to run.
-----------------------------	---

Definition at line 675 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.37 runThermalAnalysis() `void mknix::Simulation::runThermalAnalysis (
 Analysis * theAnalysis_in)`

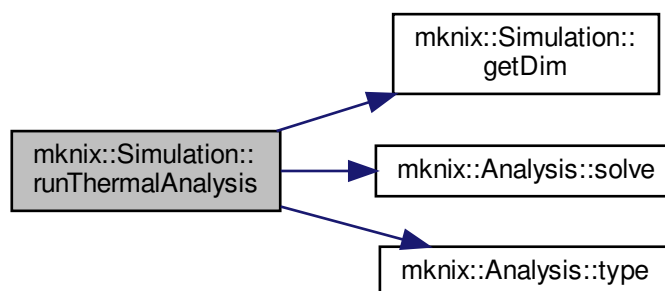
Initializes and solves a thermal or thermalstatic analysis.

Parameters

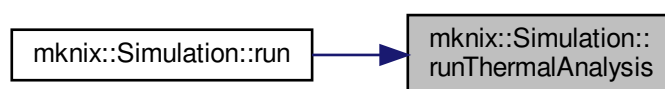
<i>theAnalysis</i> ↔ _in	Pointer to the thermal Analysis object to run.
-----------------------------	--

Definition at line 607 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.38 setAugmentedTolerance() `void mknix::Simulation::setAugmentedTolerance ( double tolerance ) [static]`

Sets the convergence tolerance for the augmented Lagrangian method.

Parameters

<i>tolerance</i>	The new tolerance value.
------------------	--------------------------

Definition at line 79 of file `simulation.cpp`.

Here is the caller graph for this function:



8.68.4.39 setInitialTemperatures() `void mknix::Simulation::setInitialTemperatures ( double temp_in )`

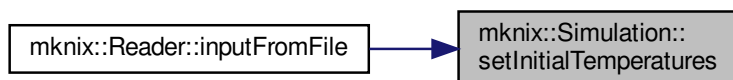
Sets the uniform initial temperature applied to all thermal nodes.

Parameters

<i>temp_in</i>	Initial temperature value.
----------------	----------------------------

Definition at line 274 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.40 setOutputFilesDetail() `void mknix::Simulation::setOutputFilesDetail ( int level_in ) [inline]`

Definition at line 100 of file simulation.h.

8.68.4.41 setThermalNodeInitialTemperature() `void mknix::Simulation::setThermalNodeInitialTemperature ( Node * node, double temperature )`

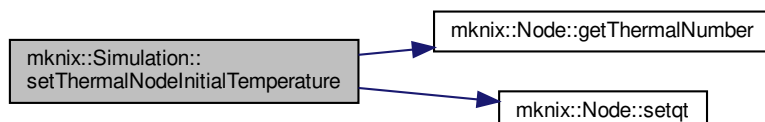
Sets the initial temperature for a specific thermal node, overriding the uniform value.

Parameters

<i>node</i>	Pointer to the node whose temperature is to be set.
<i>temperature</i>	Initial temperature value for this node.

Definition at line 284 of file simulation.cpp.

Here is the call graph for this function:

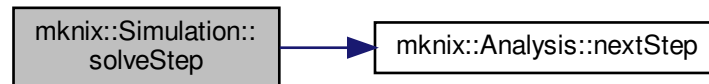


8.68.4.42 solveStep() [1/2] `void mknix::Simulation::solveStep ( )`

Advances the current analysis by one time step.

Definition at line 490 of file simulation.cpp.

Here is the call graph for this function:

**8.68.4.43 solveStep()** [2/2] `void mknix::Simulation::solveStep (`

`double * signal,`

`double * outputSignal = 0 )`

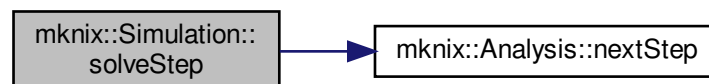
Advances the current analysis by one time step, applying an input signal and capturing an output signal.

Parameters

<i>signal</i>	Pointer to the input signal array used to update thermal loads.
<i>outputSignal</i>	Pointer to the output signal array; filled with thermal output if non-null.

Definition at line 500 of file simulation.cpp.

Here is the call graph for this function:

**8.68.4.44 staticConvergence()** `bool mknix::Simulation::staticConvergence (`

`lmx::Vector< data_type > & res,`

`lmx::Vector< data_type > & q )`

Checks convergence for the static mechanical solver and handles post-convergence output.

Parameters

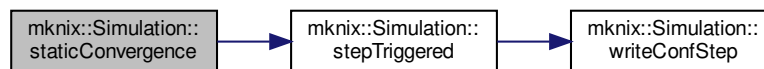
<i>res</i>	Current residue vector.
<i>q</i>	Current displacement state vector.

Returns

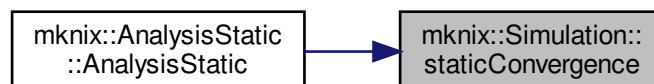
True if the step has converged, false otherwise.

Definition at line 1381 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.45 staticResidue() `void mknix::Simulation::staticResidue (`
`lmx::Vector< data_type > & residue,`
`lmx::Vector< data_type > & q )`

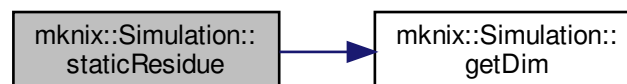
Computes the residue vector for the static mechanical equilibrium problem ($f_{int} - f_{ext}$).

Parameters

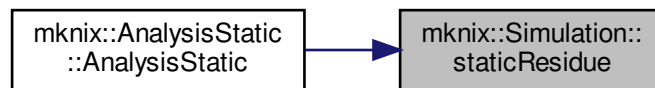
<i>residue</i>	Output residue vector.
<i>q</i>	Current displacement state vector.

Definition at line 1333 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.46 staticTangent() `void mknix::Simulation::staticTangent (`
`lmx::Matrix< data_type > & tangent_in,`
`lmx::Vector< data_type > & q )`

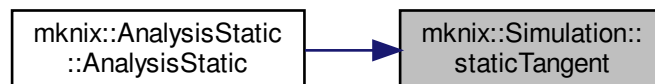
Computes the tangent (stiffness) matrix for the static mechanical problem.

Parameters

<i>tangent_in</i>	Output tangent matrix.
<i>q</i>	Current displacement state vector.

Definition at line 1365 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.47 staticThermalConvergence() `bool mknix::Simulation::staticThermalConvergence (`
`lmx::Vector< data_type > & res,`
`lmx::Vector< data_type > & q )`

Checks whether the static thermal solver has converged and handles post-convergence output.

Parameters

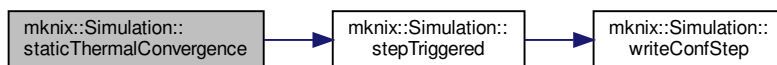
<i>res</i>	Current residue vector.
<i>q</i>	Current temperature state vector.

Returns

True if the step has converged, false otherwise.

Definition at line 877 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:



8.68.4.48 staticThermalResidue() `void mknix::Simulation::staticThermalResidue (`
`lmx::Vector< data_type > & residue,`
`lmx::Vector< data_type > & q )`

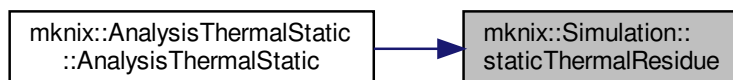
Computes the residue vector for the static thermal equilibrium problem.

Parameters

<i>residue</i>	Output residue vector ($K*q + f_{int} - f_{ext}$).
<i>q</i>	Current temperature state vector.

Definition at line 825 of file simulation.cpp.

Here is the caller graph for this function:



8.68.4.49 staticThermalTangent() `void mknix::Simulation::staticThermalTangent (`
`lmx::Matrix< data_type > & tangent_in,`
`lmx::Vector< data_type > & q )`

Computes the tangent (Jacobian) matrix for the static thermal problem.

Parameters

<i>tangent</i> _↔ <i>_in</i>	Output tangent matrix.
<i>q</i>	Current temperature state vector.

Definition at line 859 of file simulation.cpp.

Here is the caller graph for this function:

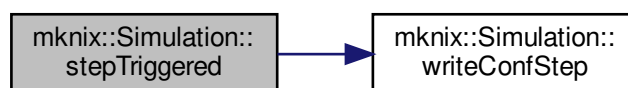


8.68.4.50 stepTriggered() `void mknix::Simulation::stepTriggered ( )`

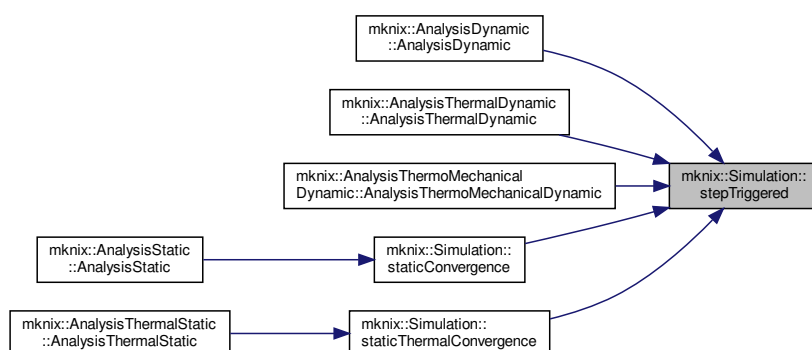
Callback invoked after a successful step convergence; writes configuration and updates timing output.

Definition at line 1409 of file simulation.cpp.

Here is the call graph for this function:



Here is the caller graph for this function:

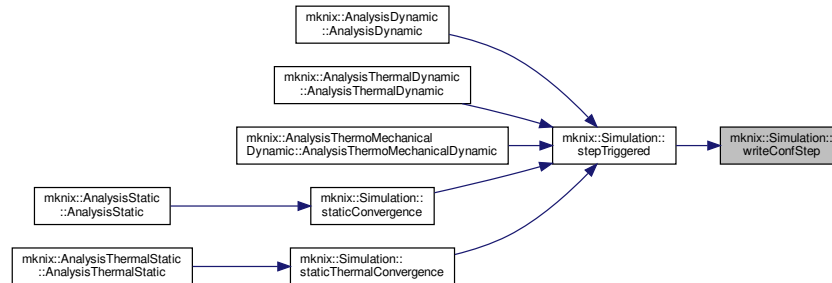


8.68.4.51 writeConfStep() `void mknix::Simulation::writeConfStep ( )`

Writes the current nodal configuration at the current step time to the displacement output file.

Definition at line 1440 of file simulation.cpp.

Here is the caller graph for this function:

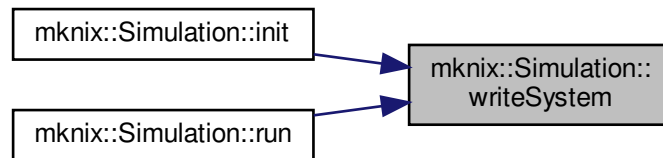


8.68.4.52 writeSystem() `void mknix::Simulation::writeSystem ( )`

Writes the initial system topology (nodes, rigid/flex bodies, joints) to the main output file and a nodes.dat file.

Definition at line 763 of file simulation.cpp.

Here is the caller graph for this function:



8.68.5 Friends And Related Function Documentation

8.68.5.1 Contact `friend class Contact [friend]`

Definition at line 68 of file simulation.h.

8.68.5.2 Reader `friend class Reader [friend]`

Definition at line 60 of file simulation.h.

8.68.5.3 ReaderConstraints `friend class ReaderConstraints [friend]`

Definition at line 62 of file simulation.h.

8.68.5.4 ReaderFlex `friend class ReaderFlex [friend]`
 Definition at line 64 of file simulation.h.

8.68.5.5 ReaderRigid `friend class ReaderRigid [friend]`
 Definition at line 66 of file simulation.h.

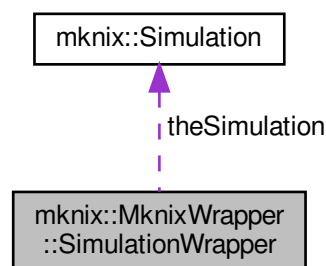
8.68.5.6 SystemChain `friend class SystemChain [friend]`
 Definition at line 70 of file simulation.h.

The documentation for this class was generated from the following files:

- [simulation.h](#)
- [simulation.cpp](#)

8.69 mknix::MknixWrapper::SimulationWrapper Struct Reference

Collaboration diagram for mknix::MknixWrapper::SimulationWrapper:



Public Attributes

- [mknix::Simulation theSimulation](#)

8.69.1 Detailed Description

Definition at line 5 of file mknixWrapper.cpp.

8.69.2 Member Data Documentation

8.69.2.1 theSimulation `mknix::Simulation mknix::MknixWrapper::SimulationWrapper::theSimulation`
 Definition at line 6 of file mknixWrapper.cpp.

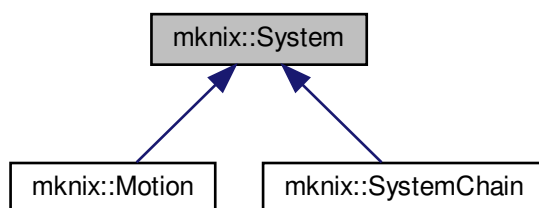
The documentation for this struct was generated from the following file:

- [mknixWrapper.cpp](#)

8.70 mknix::System Class Reference

```
#include <system.h>
```

Inheritance diagram for `mknix::System`:



Public Member Functions

- `System ()`
Default constructor.
- `System (const std::string &title)`
Constructor with title.
- `virtual ~System ()`
Destructor. Releases memory for sub-systems, bodies, loads and thermal loads.
- `std::string getTitle ()`
- `size_t getNumberOfNodes ()`
- `virtual Node * getNode (size_t index)`
- `virtual Node * getOutputNode (const std::string &name)`
- `Node * getNode (const std::string &name)`
- `System * getSystem (const std::string &sysName)`
- `std::vector< std::string > getConstraintNames ()`
- `ConstraintThermal * getConstraintThermal (const std::string &constraintName)`
- `Constraint * getConstraint (const std::string &constraintName)`
- `void getThermalNodes (std::vector< double > &)`
Collects the X coordinates of all thermal-load nodes in the "tiles" sub-system.
- `void getOutputSignalThermal (double *)`
Writes the current temperature of each output-signal node in the "tiles" sub-system (and optionally the maximum interface temperature) into the provided array.
- `void updateThermalLoads (double *)`
Updates the thermal loads in the "tiles" sub-system from an external array.
- `virtual void update (double)`
Propagates the current time to all motions and recursively to all sub-systems.
- `void initFlexBodies ()`
Initializes all flexible bodies in this system.
- `void writeRigidBodies (std::ofstream *)`
Writes rigid-body descriptions to the output file for this system and all sub-systems.
- `void writeFlexBodies (std::ofstream *)`
Writes flexible-body descriptions to the output file for this system and all sub-systems.
- `void writeJoints (std::ofstream *)`
Writes joint/constraint descriptions to the output file for this system and all sub-systems.
- `void calcCapacityMatrix ()`
Computes the thermal capacity matrices for all thermal bodies and sub-systems.
- `void calcConductivityMatrix ()`

- Computes the thermal conductivity matrices for all thermal bodies and sub-systems.*

 - void [calcInternalHeat](#) ()
- Computes the internal heat vectors for all thermal constraints and sub-systems.*

 - void [calcExternalHeat](#) ()
- Computes the external heat vectors for all thermal bodies and sub-systems.*

 - void [calcThermalTangentMatrix](#) ()
- Computes the thermal tangent matrices for all thermal constraints and sub-systems.*

 - void [assembleCapacityMatrix](#) (Imx::Matrix< [data_type](#) > &)
- Assembles the capacity matrices of all thermal bodies and sub-systems into the global matrix.*

 - void [assembleConductivityMatrix](#) (Imx::Matrix< [data_type](#) > &)
- Assembles the conductivity matrices of all thermal bodies and sub-systems into the global matrix.*

 - void [assembleExternalHeat](#) (Imx::Vector< [data_type](#) > &)
- Assembles external heat contributions from all thermal bodies, thermal loads, and sub-systems.*

 - void [assembleInternalHeat](#) (Imx::Vector< [data_type](#) > &)
- Assembles internal heat contributions from all thermal constraints and sub-systems.*

 - void [assembleThermalTangentMatrix](#) (Imx::Matrix< [data_type](#) > &)
- Assembles the thermal tangent matrix from all thermal constraints and sub-systems.*

 - void [calcMassMatrix](#) ()
- Computes the mass matrices for all flexible and rigid bodies and sub-systems.*

 - void [calcInternalForces](#) ()
- Computes internal forces for all flexible bodies, constraints, and sub-systems.*

 - void [calcExternalForces](#) ()
- Computes external forces for all flexible bodies, rigid bodies, and sub-systems.*

 - void [calcTangentMatrix](#) ()
- Computes tangent stiffness matrices for all flexible bodies, constraints, and sub-systems.*

 - void [assembleMassMatrix](#) (Imx::Matrix< [data_type](#) > &)
- Assembles mass matrices from all bodies and sub-systems into the global matrix.*

 - void [assembleInternalForces](#) (Imx::Vector< [data_type](#) > &)
- Assembles internal forces from all bodies, constraints, and sub-systems into the global vector.*

 - void [assembleExternalForces](#) (Imx::Vector< [data_type](#) > &)
- Assembles external forces from all bodies, loads, and sub-systems into the global vector.*

 - void [assembleTangentMatrix](#) (Imx::Matrix< [data_type](#) > &)
- Assembles the tangent stiffness matrix from all bodies, constraints, and sub-systems.*

 - void [assembleConstraintForces](#) (Imx::Vector< [data_type](#) > &)
- Recomputes and assembles constraint internal forces into the global force vector.*

 - void [setMechanical](#) ()
- Marks all rigid and flexible bodies as mechanical (non-thermal), propagating to sub-systems.*

 - void [outputStep](#) (const Imx::Vector< [data_type](#) > &, const Imx::Vector< [data_type](#) > &)
- Collects step output data for a dynamic step from all bodies, constraints, and sub-systems.*

 - void [outputStep](#) (const Imx::Vector< [data_type](#) > &)
- Collects step output data for a static step from all bodies, constraints, and sub-systems.*

 - void [outputToFile](#) (std::ofstream *)
- Streams all stored results for bodies, loads, and constraints to the output file.*

 - bool [checkAugmented](#) ()
- Checks whether all constraints (mechanical and thermal) in this system and sub-systems have satisfied the augmented-Lagrangian convergence criterion.*

 - void [clearAugmented](#) ()
- Resets the Lagrange multipliers of all constraints and sub-systems to zero.*

 - void [writeBoundaryNodes](#) (std::vector< [Point](#) * > &)
- Collects boundary-node pointers from all bodies and sub-systems.*

 - void [writeBoundaryConnectivity](#) (std::vector< std::vector< [Point](#) * > > &)

Builds the ordered boundary connectivity from all bodies and sub-systems.

- `std::vector< std::string > flexBodyNames ()`
- `Body * getBody (const std::string &system_name, const std::string &body_name)`

Finds and returns a body by its system and body names.

- `std::vector< Node * > getSignalNodes (const std::string &name)`

Public Attributes

- `bool outputMaxInterfaceTemp`

Protected Attributes

- `std::string title`
- `std::vector< Node * > groundNodes`
- `std::map< std::string, Node * > groundNodesMap`
- `std::map< std::string, System * > subSystems`
- `std::map< std::string, RigidBody * > rigidBodies`
- `std::map< std::string, FlexBody * > flexBodies`
- `std::map< std::string, Body * > thermalBodies`
- `std::map< std::string, Constraint * > constraints`
- `std::map< std::string, ConstraintThermal * > constraintsThermal`
- `std::vector< Load * > loads`
- `std::vector< LoadThermal * > loadsThermal`
- `std::vector< Node * > outputSignalThermal`
- `std::vector< Motion * > motions`
- `std::map< std::string, Node * > outputSignals`
- `std::map< std::string, std::vector< Node * > > inputSignals`

Friends

- `class Reader`
- `class ReaderFlex`
- `class ReaderRigid`
- `class ReaderConstraints`
- `class Contact`

8.70.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 56 of file system.h.

8.70.2 Constructor & Destructor Documentation

8.70.2.1 System() [1/2] `mknix::System::System ( )`

Default constructor.

Definition at line 37 of file system.cpp.

8.70.2.2 System() [2/2] `mknix::System::System ( const std::string & title_in )`

Constructor with title.

Parameters

<i>title</i> ↔ _in	Name identifier for this system.
-----------------------	----------------------------------

Definition at line 47 of file system.cpp.

8.70.2.3 ~System() mknix::System::~~System () [virtual]

Destructor. Releases memory for sub-systems, bodies, loads and thermal loads.

Definition at line 57 of file system.cpp.

8.70.3 Member Function Documentation

8.70.3.1 assembleCapacityMatrix() void mknix::System::assembleCapacityMatrix (lmx::Matrix< data_type > & globalCapacity_in)

Assembles the capacity matrices of all thermal bodies and sub-systems into the global matrix.

Parameters

<i>globalCapacity</i> ↔ _in	Reference to the global thermal capacity matrix.
--------------------------------	--

Definition at line 323 of file system.cpp.

8.70.3.2 assembleConductivityMatrix() void mknix::System::assembleConductivityMatrix (lmx::Matrix< data_type > & globalConductivity_in)

Assembles the conductivity matrices of all thermal bodies and sub-systems into the global matrix.

Parameters

<i>globalConductivity</i> ↔ _in	Reference to the global thermal conductivity matrix.
------------------------------------	--

Definition at line 341 of file system.cpp.

8.70.3.3 assembleConstraintForces() void mknix::System::assembleConstraintForces (lmx::Vector< data_type > & internalForces_in)

Recomputes and assembles constraint internal forces into the global force vector.

Parameters

<i>internalForces</i> ↔ _in	Reference to the global internal force vector.
--------------------------------	--

Definition at line 615 of file system.cpp.

8.70.3.4 assembleExternalForces() void mknix::System::assembleExternalForces (lmx::Vector< data_type > & externalForces_in)

Assembles external forces from all bodies, loads, and sub-systems into the global vector.

Parameters

<i>externalForces</i> ↔ _in	Reference to the global external force vector.
--------------------------------	--

Definition at line 561 of file system.cpp.

8.70.3.5 assembleExternalHeat() `void mknix::System::assembleExternalHeat (
 lmx::Vector< data_type > & externalHeat_in)`

Assembles external heat contributions from all thermal bodies, thermal loads, and sub-systems.

Parameters

<i>externalHeat</i> ↔ _in	Reference to the global external heat vector.
------------------------------	---

Definition at line 367 of file system.cpp.

8.70.3.6 assembleInternalForces() `void mknix::System::assembleInternalForces (
 lmx::Vector< data_type > & internalForces_in)`

Assembles internal forces from all bodies, constraints, and sub-systems into the global vector.

Parameters

<i>internalForces</i> ↔ _in	Reference to the global internal force vector.
--------------------------------	--

Definition at line 538 of file system.cpp.

8.70.3.7 assembleInternalHeat() `void mknix::System::assembleInternalHeat (
 lmx::Vector< data_type > & internalHeat_in)`

Assembles internal heat contributions from all thermal constraints and sub-systems.

Parameters

<i>internalHeat</i> ↔ _in	Reference to the global internal heat vector.
------------------------------	---

Definition at line 391 of file system.cpp.

8.70.3.8 assembleMassMatrix() `void mknix::System::assembleMassMatrix (
 lmx::Matrix< data_type > & globalMass_in)`

Assembles mass matrices from all bodies and sub-systems into the global matrix.

Parameters

<i>globalMass</i> ↔ _in	Reference to the global mass matrix.
----------------------------	--------------------------------------

Definition at line 515 of file system.cpp.

8.70.3.9 assembleTangentMatrix() `void mknix::System::assembleTangentMatrix (
 lmx::Matrix< data_type > & globalTangent_in)`

Assembles the tangent stiffness matrix from all bodies, constraints, and sub-systems.

Parameters

<i>globalTangent_in</i>	Reference to the global tangent matrix.
-------------------------	---

Definition at line 592 of file system.cpp.

8.70.3.10 assembleThermalTangentMatrix() `void mknix::System::assembleThermalTangentMatrix (
 lmx::Matrix< data_type > & globalTangent_in)`

Assembles the thermal tangent matrix from all thermal constraints and sub-systems.

Parameters

<i>globalTangent_in</i>	Reference to the global thermal tangent matrix.
-------------------------	---

Definition at line 409 of file system.cpp.

8.70.3.11 calcCapacityMatrix() `void mknix::System::calcCapacityMatrix ( )`

Computes the thermal capacity matrices for all thermal bodies and sub-systems.

Definition at line 232 of file system.cpp.

8.70.3.12 calcConductivityMatrix() `void mknix::System::calcConductivityMatrix ( )`

Computes the thermal conductivity matrices for all thermal bodies and sub-systems.

Definition at line 248 of file system.cpp.

8.70.3.13 calcExternalForces() `void mknix::System::calcExternalForces ( )`

Computes external forces for all flexible bodies, rigid bodies, and sub-systems.

Definition at line 469 of file system.cpp.

8.70.3.14 calcExternalHeat() `void mknix::System::calcExternalHeat ( )`

Computes the external heat vectors for all thermal bodies and sub-systems.

Definition at line 272 of file system.cpp.

8.70.3.15 calcInternalForces() `void mknix::System::calcInternalForces ( )`

Computes internal forces for all flexible bodies, constraints, and sub-systems.

Definition at line 447 of file system.cpp.

8.70.3.16 calcInternalHeat() `void mknix::System::calcInternalHeat ( )`

Computes the internal heat vectors for all thermal constraints and sub-systems.

Definition at line 288 of file system.cpp.

8.70.3.17 calcMassMatrix() `void mknix::System::calcMassMatrix ( )`

Computes the mass matrices for all flexible and rigid bodies and sub-systems.

Definition at line 426 of file system.cpp.

8.70.3.18 calcTangentMatrix() `void mknix::System::calcTangentMatrix ( )`

Computes tangent stiffness matrices for all flexible bodies, constraints, and sub-systems.

Definition at line 491 of file system.cpp.

8.70.3.19 calcThermalTangentMatrix() `void mknix::System::calcThermalTangentMatrix ( )`

Computes the thermal tangent matrices for all thermal constraints and sub-systems.

Definition at line 305 of file system.cpp.

8.70.3.20 checkAugmented() `bool mknix::System::checkAugmented ( )`

Checks whether all constraints (mechanical and thermal) in this system and sub-systems have satisfied the augmented-Lagrangian convergence criterion.

Returns

True if all constraints converged; false otherwise.

Definition at line 755 of file system.cpp.

8.70.3.21 clearAugmented() `void mknix::System::clearAugmented ( )`

Resets the Lagrange multipliers of all constraints and sub-systems to zero.

Definition at line 791 of file system.cpp.

8.70.3.22 flexBodyNames() `std::vector<std::string> mknix::System::flexBodyNames ( ) [inline]`

Definition at line 203 of file system.h.

8.70.3.23 getBody() `mknix::Body * mknix::System::getBody (`
`const std::string & system_name,`
`const std::string & body_name )`

Finds and returns a body by its system and body names.

Parameters

<i>system_name</i>	Name of the sub-system containing the body.
<i>body_name</i>	Name of the body to retrieve.

Returns

Pointer to the found [Body](#).

Exceptions

<i>std::out_of_range</i>	if the system or body is not found.
--------------------------	-------------------------------------

Definition at line 861 of file system.cpp.

8.70.3.24 getConstraint() `Constraint*` mknix::System::getConstraint (
 const std::string & *constraintName*) [inline]

Definition at line 124 of file system.h.

8.70.3.25 getConstraintNames() `std::vector<std::string>` mknix::System::getConstraintNames ()
 [inline]

Definition at line 108 of file system.h.

8.70.3.26 getConstraintThermal() `ConstraintThermal*` mknix::System::getConstraintThermal (
 const std::string & *constraintName*) [inline]

Definition at line 119 of file system.h.

8.70.3.27 getNode() [1/2] `Node*` mknix::System::getNode (
 const std::string & *name*) [inline]

Definition at line 98 of file system.h.

8.70.3.28 getNode() [2/2] `virtual Node*` mknix::System::getNode (
 `size_t` *index*) [inline], [virtual]

Reimplemented in [mknix::SystemChain](#).

Definition at line 88 of file system.h.

8.70.3.29 getNumberOfNodes() `size_t` mknix::System::getNumberOfNodes () [inline]

Definition at line 83 of file system.h.

8.70.3.30 getOutputNode() `virtual Node*` mknix::System::getOutputNode (
 const std::string & *name*) [inline], [virtual]

Definition at line 93 of file system.h.

8.70.3.31 getOutputSignalThermal() `void` mknix::System::getOutputSignalThermal (
 `double *` *vector_in*)

Writes the current temperature of each output-signal node in the "tiles" sub-system (and optionally the maximum interface temperature) into the provided array.

Parameters

<code>vector↵ _in</code>	Output array filled with temperature values.
------------------------------	--

Definition at line 113 of file system.cpp.

8.70.3.32 getSignalNodes() `std::vector<Node*>` mknix::System::getSignalNodes (
 const std::string & *name*) [inline]

Definition at line 216 of file system.h.

8.70.3.33 getSystem() `System*` mknix::System::getSystem (
 const std::string & *sysName*) [inline]

Definition at line 103 of file system.h.

8.70.3.34 getThermalNodes() `void mknix::System::getThermalNodes (`
`std::vector< double > & x_coordinates )`

Collects the X coordinates of all thermal-load nodes in the "tiles" sub-system.

Parameters

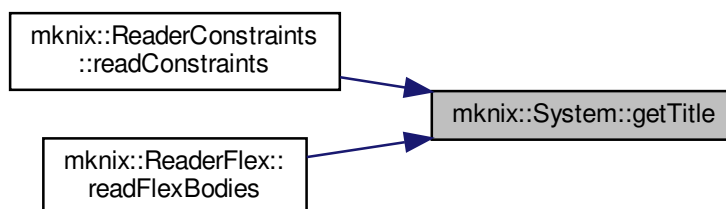
<code>x_coordinates</code>	Vector to which the node X coordinates are appended.
----------------------------	--

Definition at line 100 of file system.cpp.

8.70.3.35 getTitle() `std::string mknix::System::getTitle ( ) [inline]`

Definition at line 78 of file system.h.

Here is the caller graph for this function:

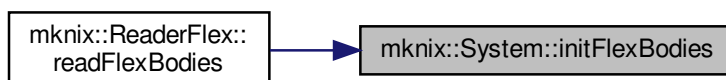


8.70.3.36 initFlexBodies() `void mknix::System::initFlexBodies ( )`

Initializes all flexible bodies in this system.

Definition at line 166 of file system.cpp.

Here is the caller graph for this function:



8.70.3.37 outputStep() `[1/2] void mknix::System::outputStep (`
`const lmx::Vector< data_type > & q )`

Collects step output data for a static step from all bodies, constraints, and sub-systems.

Parameters

<i>q</i>	Global configuration vector.
----------	------------------------------

Definition at line 693 of file system.cpp.

8.70.3.38 outputStep() [2/2] `void mknix::System::outputStep (`
`const lmx::Vector< data_type > & q,`
`const lmx::Vector< data_type > & qdot )`

Collects step output data for a dynamic step from all bodies, constraints, and sub-systems.

Parameters

<i>q</i>	Global configuration vector.
<i>qdot</i>	Global velocity vector.

Definition at line 665 of file system.cpp.

8.70.3.39 outputToFile() `void mknix::System::outputToFile (`
`std::ofstream * outFile )`

Streams all stored results for bodies, loads, and constraints to the output file.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 721 of file system.cpp.

8.70.3.40 setMechanical() `void mknix::System::setMechanical ( )`

Marks all rigid and flexible bodies as mechanical (non-thermal), propagating to sub-systems.

Definition at line 642 of file system.cpp.

8.70.3.41 update() `void mknix::System::update (`
`double time ) [virtual]`

Propagates the current time to all motions and recursively to all sub-systems.

Parameters

<i>time</i>	Current simulation time.
-------------	--------------------------

Reimplemented in [mknix::SystemChain](#), and [mknix::Motion](#).

Definition at line 150 of file system.cpp.

8.70.3.42 updateThermalLoads() `void mknix::System::updateThermalLoads (`
`double * vector_in )`

Updates the thermal loads in the "tiles" sub-system from an external array.

Parameters

<i>vector_{in}</i>	Array of new load values, one per thermal load.
----------------------------	---

Definition at line 136 of file system.cpp.

8.70.3.43 writeBoundaryConnectivity() `void mknix::System::writeBoundaryConnectivity (`
`std::vector< std::vector< Point * >> & )`

Builds the ordered boundary connectivity from all bodies and sub-systems.

Parameters

<i>connectivity_nodes</i>	Vector of node chains to which each body's boundary is appended.
---------------------------	--

Definition at line 836 of file system.cpp.

8.70.3.44 writeBoundaryNodes() `void mknix::System::writeBoundaryNodes (`
`std::vector< Point * > & boundary_nodes )`

Collects boundary-node pointers from all bodies and sub-systems.

Parameters

<i>boundary_nodes</i>	Vector to which the boundary node pointers are appended.
-----------------------	--

Definition at line 813 of file system.cpp.

8.70.3.45 writeFlexBodies() `void mknix::System::writeFlexBodies (`
`std::ofstream * outFile )`

Writes flexible-body descriptions to the output file for this system and all sub-systems.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 196 of file system.cpp.

8.70.3.46 writeJoints() `void mknix::System::writeJoints (`
`std::ofstream * outFile )`

Writes joint/constraint descriptions to the output file for this system and all sub-systems.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 213 of file system.cpp.

8.70.3.47 writeRigidBodies() `void mknix::System::writeRigidBodies (`
`std::ofstream * outFile )`

Writes rigid-body descriptions to the output file for this system and all sub-systems.

Parameters

<i>outFile</i>	Pointer to the output file stream.
----------------	------------------------------------

Definition at line 179 of file system.cpp.

8.70.4 Friends And Related Function Documentation

8.70.4.1 Contact `friend class Contact [friend]`

Definition at line 67 of file system.h.

8.70.4.2 Reader `friend class Reader [friend]`

Definition at line 59 of file system.h.

8.70.4.3 ReaderConstraints `friend class ReaderConstraints [friend]`

Definition at line 65 of file system.h.

8.70.4.4 ReaderFlex `friend class ReaderFlex [friend]`

Definition at line 61 of file system.h.

8.70.4.5 ReaderRigid `friend class ReaderRigid [friend]`

Definition at line 63 of file system.h.

8.70.5 Member Data Documentation

8.70.5.1 constraints `std::map<std::string, Constraint*> mknix::System::constraints [protected]`

Definition at line 231 of file system.h.

8.70.5.2 constraintsThermal `std::map<std::string, ConstraintThermal*> mknix::System::constraintsThermal [protected]`

Definition at line 232 of file system.h.

8.70.5.3 flexBodies `std::map<std::string, FlexBody*> mknix::System::flexBodies [protected]`

Definition at line 229 of file system.h.

8.70.5.4 groundNodes `std::vector<Node*> mknix::System::groundNodes [protected]`

Definition at line 225 of file system.h.

8.70.5.5 groundNodesMap `std::map<std::string, Node*> mknix::System::groundNodesMap [protected]`

Definition at line 226 of file system.h.

8.70.5.6 inputSignals `std::map<std::string, std::vector<Node*> > mknix::System::inputSignals [protected]`

Definition at line 238 of file system.h.

8.70.5.7 loads `std::vector<Load*> mknix::System::loads` [protected]

Definition at line 233 of file system.h.

8.70.5.8 loadsThermal `std::vector<LoadThermal*> mknix::System::loadsThermal` [protected]

Definition at line 234 of file system.h.

8.70.5.9 motions `std::vector<Motion*> mknix::System::motions` [protected]

Definition at line 236 of file system.h.

8.70.5.10 outputMaxInterfaceTemp `bool mknix::System::outputMaxInterfaceTemp`

Definition at line 76 of file system.h.

8.70.5.11 outputSignals `std::map<std::string, Node*> mknix::System::outputSignals` [protected]

Definition at line 237 of file system.h.

8.70.5.12 outputSignalThermal `std::vector<Node*> mknix::System::outputSignalThermal` [protected]

Definition at line 235 of file system.h.

8.70.5.13 rigidBodies `std::map<std::string, RigidBody*> mknix::System::rigidBodies` [protected]

Definition at line 228 of file system.h.

8.70.5.14 subSystems `std::map<std::string, System*> mknix::System::subSystems` [protected]

Definition at line 227 of file system.h.

8.70.5.15 thermalBodies `std::map<std::string, Body*> mknix::System::thermalBodies` [protected]

Definition at line 230 of file system.h.

8.70.5.16 title `std::string mknix::System::title` [protected]

Definition at line 223 of file system.h.

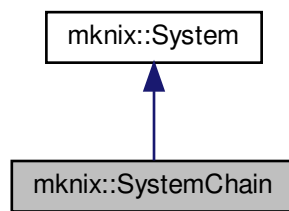
The documentation for this class was generated from the following files:

- [system.h](#)
- [system.cpp](#)

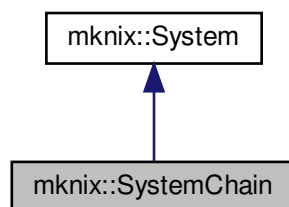
8.71 mknix::SystemChain Class Reference

```
#include <systemchain.h>
```

Inheritance diagram for mknix::SystemChain:



Collaboration diagram for mknix::SystemChain:



Public Member Functions

- [SystemChain](#) ()
Default constructor.
- [SystemChain](#) (const char *)
Constructor with title.
- [~SystemChain](#) ()
Destructor.
- void [setInterfaceNodeA](#) (double x, double y, double z)
- void [setInterfaceNodeB](#) (double x, double y, double z)
- void [setProperties](#) (int segments_in, double length_in)
- void [setMass](#) (double mass_in)
- void [setTimeLengths](#) (std::map< double, double > &timeLengths_in)
- [Node](#) * [getNode](#) (size_t) override
Returns a specific node of the chain by index. Index 0 returns the first frame node of the first bar; index 1 returns the last frame node of the last bar.
- void [addTimeLength](#) (double time_in, double length_in)
- void [populate](#) ([Simulation](#) *, std::string &)
Creates and connects all bar segments, distance constraints, and spherical joints that form this chain system, registering them with the simulation.
- void [update](#) (double) override
Updates each bar's constraint distance based on the interpolated chain length at the given time.

Additional Inherited Members

8.71.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 36 of file systemchain.h.

8.71.2 Constructor & Destructor Documentation

8.71.2.1 SystemChain() [1/2] `mknix::SystemChain::SystemChain ( )`

Default constructor.

Definition at line 34 of file systemchain.cpp.

8.71.2.2 SystemChain() [2/2] `mknix::SystemChain::SystemChain ( const char * title_in )`

Constructor with title.

Parameters

<i>title</i> ↔ _in	C-string name identifier for this chain system.
-----------------------	---

Definition at line 43 of file systemchain.cpp.

8.71.2.3 ~SystemChain() `mknix::SystemChain::~~SystemChain ( )`

Destructor.

Definition at line 51 of file systemchain.cpp.

8.71.3 Member Function Documentation

8.71.3.1 addTimeLenght() `void mknix::SystemChain::addTimeLenght ( double time_in, double length_in ) [inline]`

Definition at line 85 of file systemchain.h.

8.71.3.2 getNode() `Node * mknix::SystemChain::getNode ( size_t node_i ) [override], [virtual]`

Returns a specific node of the chain by index. Index 0 returns the first frame node of the first bar; index 1 returns the last frame node of the last bar.

Parameters

<i>node</i> ↔ _i	<code>Node</code> index (0 or 1).
---------------------	-----------------------------------

Returns

Pointer to the requested [Node](#), or nullptr for other indices.

Reimplemented from [mknix::System](#).

Definition at line 61 of file systemchain.cpp.

8.71.3.3 populate() `void mknix::SystemChain::populate (
 Simulation * theSimulation,
 std::string & energyKeyword)`

Creates and connects all bar segments, distance constraints, and spherical joints that form this chain system, registering them with the simulation.

Parameters

<i>theSimulation</i>	Pointer to the main Simulation object (used to create nodes).
<i>energyKeyword</i>	Keyword to activate energy output for each bar segment.

Definition at line 79 of file systemchain.cpp.

8.71.3.4 setInterfaceNodeA() `void mknix::SystemChain::setInterfaceNodeA (
 double x,
 double y,
 double z) [inline]`

Definition at line 52 of file systemchain.h.

8.71.3.5 setInterfaceNodeB() `void mknix::SystemChain::setInterfaceNodeB (
 double x,
 double y,
 double z) [inline]`

Definition at line 59 of file systemchain.h.

8.71.3.6 setMass() `void mknix::SystemChain::setMass (
 double mass_in) [inline]`

Definition at line 72 of file systemchain.h.

8.71.3.7 setProperties() `void mknix::SystemChain::setProperties (
 int segments_in,
 double length_in) [inline]`

Definition at line 66 of file systemchain.h.

8.71.3.8 setTimeLengths() `void mknix::SystemChain::setTimeLengths (
 std::map< double, double > & timeLengths_in) [inline]`

Definition at line 78 of file systemchain.h.

8.71.3.9 update() `void mknix::SystemChain::update (
 double theTime) [override], [virtual]`

Updates each bar's constraint distance based on the interpolated chain length at the given time.

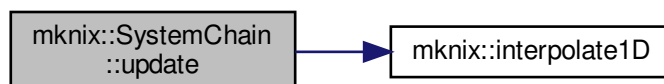
Parameters

<i>theTime</i>	Current simulation time.
----------------	--------------------------

Reimplemented from [mknix::System](#).

Definition at line 145 of file systemchain.cpp.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

- [systemchain.h](#)
- [systemchain.cpp](#)

8.72 mknix::ThermalBody Class Reference

```
#include <bodythermal.h>
```

Public Member Functions

- [ThermalBody](#) ()
Default constructor. Initializes the thermal body with energy computation disabled.
- [ThermalBody](#) (std::string)
Constructor with 1 parameter.
- virtual [~ThermalBody](#) ()
Destructor.
- virtual void [initialize](#) ()
Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.
- virtual void [calcCapacityMatrix](#) ()
Computes the local Capacity of the material body by calling each cell's cascade function.
- virtual void [calcConductivityMatrix](#) ()
Computes the local Conductivity of the material body by calling each cell's cascade function.
- virtual void [calcExternalHeat](#) ()
Computes the local volumetric heat vector of the material body by calling each cell's cascade function.
- virtual void [assembleCapacityMatrix](#) (Imx::Matrix< [data_type](#) > &)
Assembles the local conductivity into the global matrix by calling each cell's cascade function.
- virtual void [assembleConductivityMatrix](#) (Imx::Matrix< [data_type](#) > &)
Assembles the local conductivity into the global matrix by calling each cell's cascade function.
- virtual void [assembleExternalHeat](#) (Imx::Vector< [data_type](#) > &)
Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.
- virtual void [setOutput](#) (std::string)
Activates a flag for output data at the end of the analysis.
- virtual void [outputStep](#) (const Imx::Vector< [data_type](#) > &, const Imx::Vector< [data_type](#) > &)
Postprocess and store step results for dynamic analysis.

- virtual void `outputStep` (const `Imx::Vector< data_type >` &)
Postprocess and store step results for static analysis.
- virtual void `outputToFile` (std::ofstream *)
Streams the data stored during the analysis to a file. The idea is that all thermalBodies will be linked to a solid body with the same name (title). The postprocessor will treat this values as if it was the same body, so it must take into account that it can need to read several ENERGY() sections for the same body.*
- virtual void `addNode` (`Node *node_in`)
- virtual `Node *` `getNode` (int node_number)
- virtual `Node *` `getLastNode` ()
- virtual void `addCell` (int num, `Cell *cell_in`)
- virtual void `addLoadThermal` (`LoadThermalBody *theLoad`)

Protected Attributes

- std::string `title`
- bool `computeEnergy`
- std::vector< `Node *` > `nodes`
- std::map< int, `Cell *` > `cells`
- std::vector< `Imx::Vector< data_type > *` > `temperature`
- std::vector< `LoadThermalBody *` > `volumetricHeatSources`

8.72.1 Detailed Description

Author

AUTHORS <MAILS>

Definition at line 36 of file bodythermal.h.

8.72.2 Constructor & Destructor Documentation

8.72.2.1 ThermalBody() [1/2] `mknix::ThermalBody::ThermalBody ( )`

Default constructor. Initializes the thermal body with energy computation disabled.

Definition at line 32 of file bodythermal.cpp.

8.72.2.2 ThermalBody() [2/2] `mknix::ThermalBody::ThermalBody ( std::string title_in )`

Constructor with 1 parameter.

Parameters

<code>title_in</code>	Name of body in the system. Will be the same as the associated material body
-----------------------	--

Definition at line 44 of file bodythermal.cpp.

8.72.2.3 ~ThermalBody() `mknix::ThermalBody::~ThermalBody ( )` [virtual]

Destructor.

Definition at line 54 of file bodythermal.cpp.

8.72.3 Member Function Documentation

8.72.3.1 addCell() `virtual void mknix::ThermalBody::addCell (`
`int num,`
`Cell * cell_in ) [inline], [virtual]`

Definition at line 95 of file bodythermal.h.

8.72.3.2 addLoadThermal() `virtual void mknix::ThermalBody::addLoadThermal (`
`LoadThermalBody * theLoad ) [inline], [virtual]`

Definition at line 101 of file bodythermal.h.

8.72.3.3 addNode() `virtual void mknix::ThermalBody::addNode (`
`Node * node_in ) [inline], [virtual]`

Definition at line 80 of file bodythermal.h.

8.72.3.4 assembleCapacityMatrix() `void mknix::ThermalBody::assembleCapacityMatrix (`
`lmx::Matrix< data_type > & globalCapacity ) [virtual]`

Assembles the local conductivity into the global matrix by calling each cell's cascade function.

Parameters

<i>globalCapacity</i>	Reference to the global matrix of the thermal simulation.
-----------------------	---

Returns

void

Definition at line 152 of file bodythermal.cpp.

8.72.3.5 assembleConductivityMatrix() `void mknix::ThermalBody::assembleConductivityMatrix (`
`lmx::Matrix< data_type > & globalConductivity ) [virtual]`

Assembles the local conductivity into the global matrix by calling each cell's cascade function.

Parameters

<i>globalConductivity</i>	Reference to the global matrix of the thermal simulation.
---------------------------	---

Returns

void

Definition at line 170 of file bodythermal.cpp.

8.72.3.6 assembleExternalHeat() `void mknix::ThermalBody::assembleExternalHeat (`
`lmx::Vector< data_type > & globalExternalHeat ) [virtual]`

Assembles the local volumetric heat into the global heat load vector by calling each cell's cascade function.

Returns

void

Definition at line 187 of file bodythermal.cpp.

8.72.3.7 calcCapacityMatrix() `void mknix::ThermalBody::calcCapacityMatrix ( ) [virtual]`
 Computes the local Capacity of the material body by calling each cell's cascade function.

Returns

void

Definition at line 99 of file bodythermal.cpp.

8.72.3.8 calcConductivityMatrix() `void mknix::ThermalBody::calcConductivityMatrix ( ) [virtual]`
 Computes the local Conductivity of the material body by calling each cell's cascade function.

Returns

void

Definition at line 116 of file bodythermal.cpp.

8.72.3.9 calcExternalHeat() `void mknix::ThermalBody::calcExternalHeat ( ) [virtual]`
 Computes the local volumetric heat vector of the material body by calling each cell's cascade function.

Returns

void

Definition at line 133 of file bodythermal.cpp.

8.72.3.10 getLastNode() `virtual Node* mknix::ThermalBody::getLastNode ( ) [inline], [virtual]`
 Definition at line 90 of file bodythermal.h.

8.72.3.11 getNode() `virtual Node* mknix::ThermalBody::getNode ( int node_number ) [inline], [virtual]`

Definition at line 85 of file bodythermal.h.

8.72.3.12 initialize() `void mknix::ThermalBody::initialize ( ) [virtual]`
 Cascade initialization funtion. Calls the initialize methods for each of the Cells and tells them to compute their shapefunction values. Both loops are parallelized.

Returns

void

Definition at line 64 of file bodythermal.cpp.

8.72.3.13 outputStep() [1/2] `void mknix::ThermalBody::outputStep ( const lmx::Vector< data_type > & q ) [virtual]`

Postprocess and store step results for static analysis.

Parameters

q	Global configuration vector
-----	-----------------------------

Returns

void

Definition at line 297 of file bodythermal.cpp.

8.72.3.14 outputStep() [2/2] void mknix::ThermalBody::outputStep (
 const lmx::Vector< data_type > & q,
 const lmx::Vector< data_type > & qdot) [virtual]

Postprocess and store step results for dynamic analysis.

Parameters

<i>q</i>	Global configuration vector
<i>qdot</i>	Global configuration first derivative vector

Returns

void

Definition at line 268 of file bodythermal.cpp.

8.72.3.15 outputToFile() void mknix::ThermalBody::outputToFile (
 std::ofstream * outFile) [virtual]

Streams the data stored during the analysis to a file. The idea is that all thermalBodies will be linked to a solid body with the same name (title). The postprocessor will treat this values as if it was the same body, so it must take into account that it can need to read several ENERGY(*) sections for the same body.

Parameters

<i>outFile</i>	Output files
----------------	--------------

Returns

void

Definition at line 226 of file bodythermal.cpp.

8.72.3.16 setOutput() void mknix::ThermalBody::setOutput (
 std::string outputType_in) [virtual]

Activates a flag for output data at the end of the analysis.

See also

[outputToFile\(\)](#)
[outputStep\(\)](#)

Parameters

<i>outputType↔ _in</i>	Keyword of the flag. Options are: [ENERGY]
----------------------------	--

Returns

void

Definition at line 208 of file `bodythermal.cpp`.**8.72.4 Member Data Documentation****8.72.4.1 cells** `std::map<int, Cell*> mknix::ThermalBody::cells` [protected]

Map of integration cells.

Definition at line 111 of file `bodythermal.h`.**8.72.4.2 computeEnergy** `bool mknix::ThermalBody::computeEnergy` [protected]Definition at line 109 of file `bodythermal.h`.**8.72.4.3 nodes** `std::vector<Node*> mknix::ThermalBody::nodes` [protected]Definition at line 110 of file `bodythermal.h`.**8.72.4.4 temperature** `std::vector< Imx::Vector<data_type>*> mknix::ThermalBody::temperature` [protected]Definition at line 112 of file `bodythermal.h`.**8.72.4.5 title** `std::string mknix::ThermalBody::title` [protected]Definition at line 107 of file `bodythermal.h`.**8.72.4.6 volumetricHeatSources** `std::vector<LoadThermalBody*> mknix::ThermalBody::volumetricHeatSources` [protected]Definition at line 113 of file `bodythermal.h`.

The documentation for this class was generated from the following files:

- [bodythermal.h](#)
- [bodythermal.cpp](#)

8.73 `Imx::Vector< T >` Class Template Reference**8.73.1 Detailed Description**

```
template<typename T>
class Imx::Vector< T >
```

Definition at line 40 of file `cell.h`.

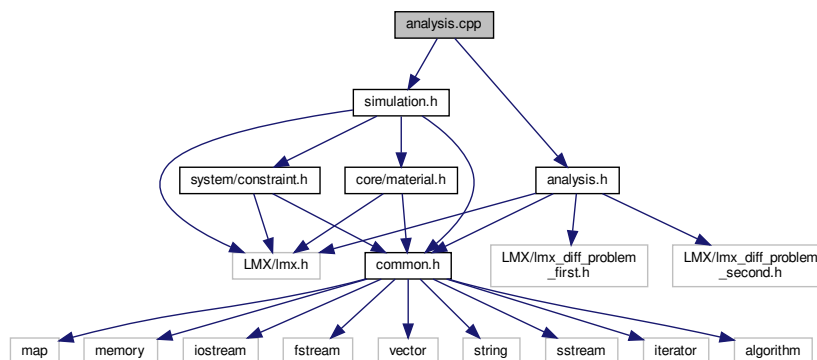
The documentation for this class was generated from the following file:

- [cell.h](#)

9 File Documentation**9.1 analysis.cpp File Reference**

```
#include "analysis.h"
#include "simulation.h"
```


Include dependency graph for analysis.cpp:



Namespaces

- [mnix](#)

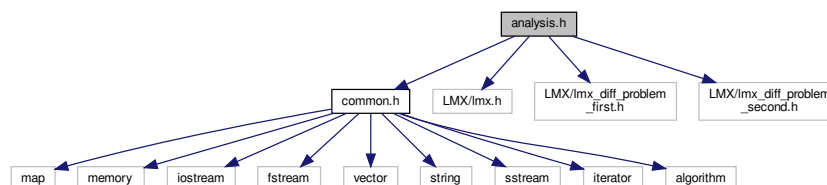
9.2 analysis.h File Reference

```

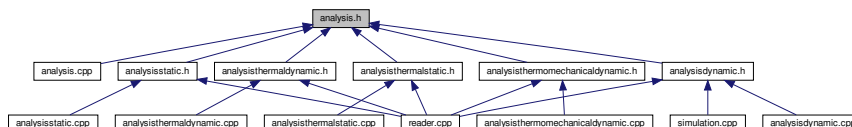
#include "common.h"
#include "LMX/lmx.h"
#include "LMX/lmx_diff_problem_first.h"
#include "LMX/lmx_diff_problem_second.h"

```

Include dependency graph for analysis.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::Analysis](#)

Namespaces

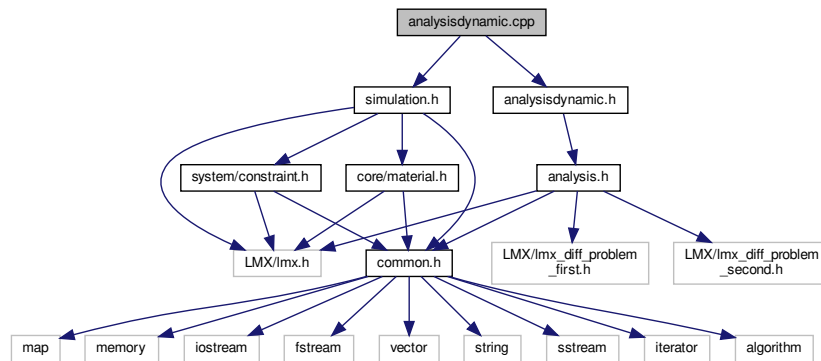
- [mnix](#)

9.3 analysisdynamic.cpp File Reference

```
#include "analysisdynamic.h"
```

```
#include "simulation.h"
```

Include dependency graph for analysisdynamic.cpp:



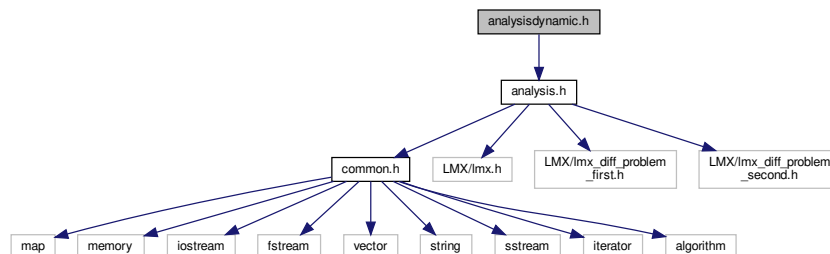
Namespaces

- [mknix](#)

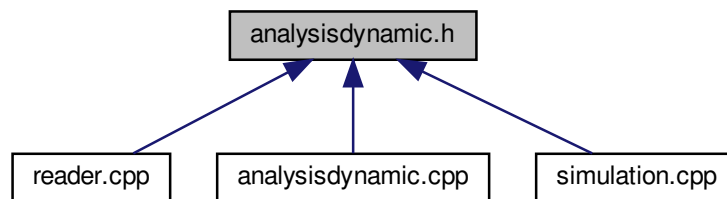
9.4 analysisdynamic.h File Reference

```
#include "analysis.h"
```

Include dependency graph for analysisdynamic.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mnix::AnalysisDynamic`

Namespaces

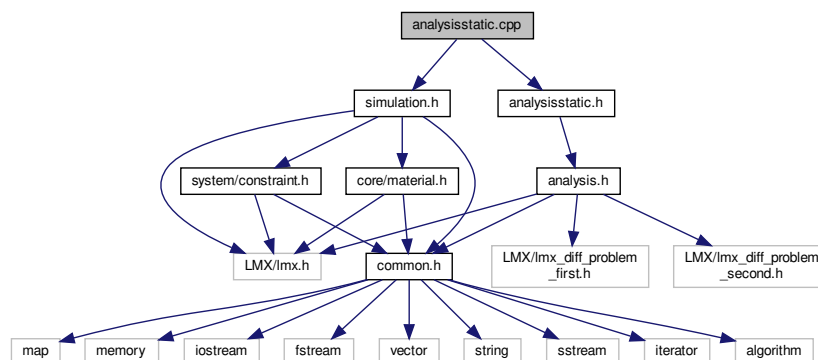
- `mnix`

9.5 analysisstatic.cpp File Reference

```
#include "analysisstatic.h"
```

```
#include "simulation.h"
```

Include dependency graph for `analysisstatic.cpp`:



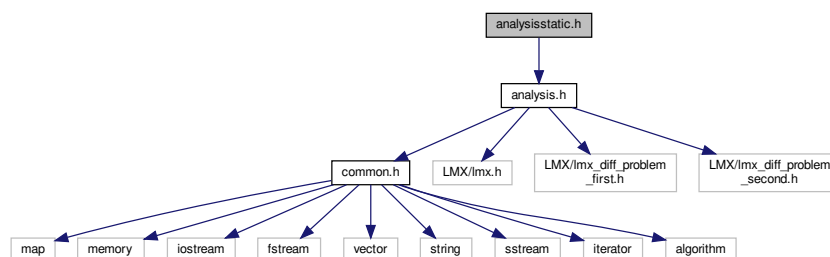
Namespaces

- `mnix`

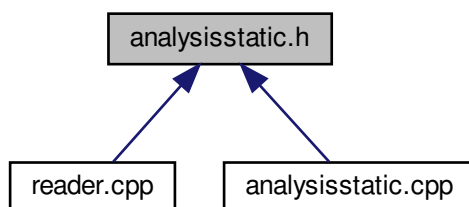
9.6 analysisstatic.h File Reference

```
#include "analysis.h"
```

Include dependency graph for analysisstatic.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::AnalysisStatic](#)

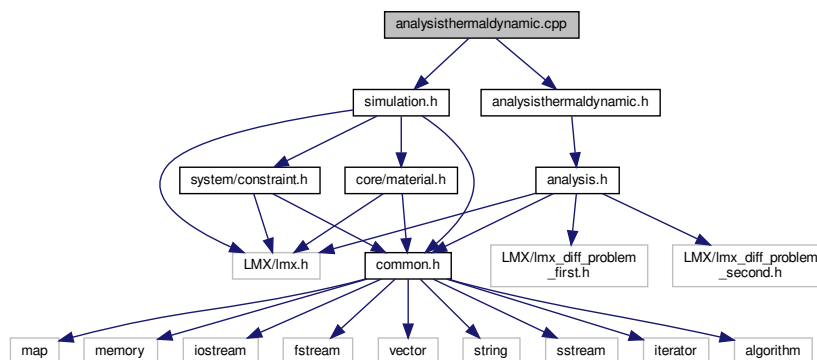
Namespaces

- [mknix](#)

9.7 analysisthermaldynamic.cpp File Reference

```
#include "analysisthermaldynamic.h"
#include "simulation.h"
```

Include dependency graph for `analysisthermaldynamic.cpp`:



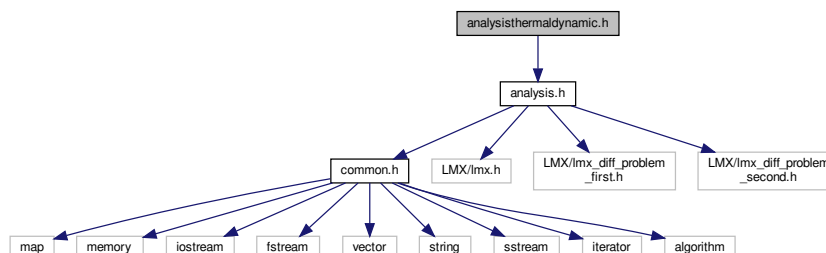
Namespaces

- [mknix](#)

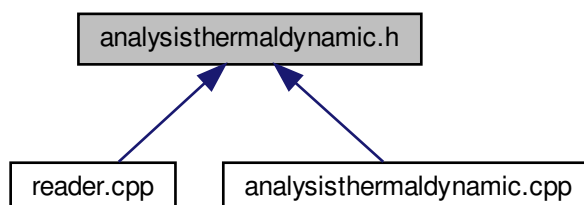
9.8 analysisthermaldynamic.h File Reference

```
#include "analysis.h"
```

Include dependency graph for `analysisthermaldynamic.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::AnalysisThermalDynamic](#)

Namespaces

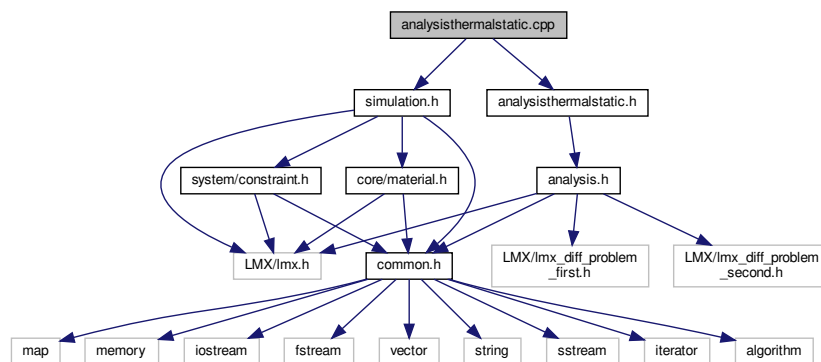
- [mknix](#)

9.9 analysisthermalstatic.cpp File Reference

```
#include "analysisthermalstatic.h"
```

```
#include "simulation.h"
```

Include dependency graph for analysisthermalstatic.cpp:

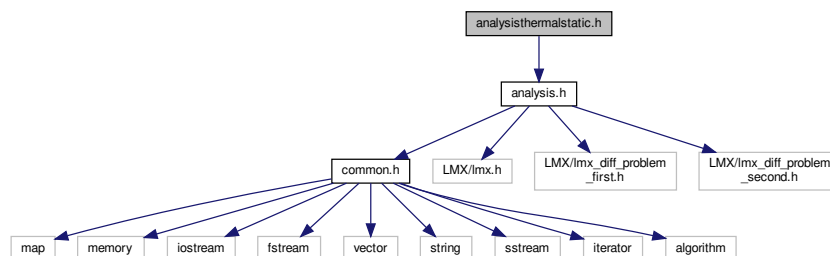
**Namespaces**

- [mknix](#)

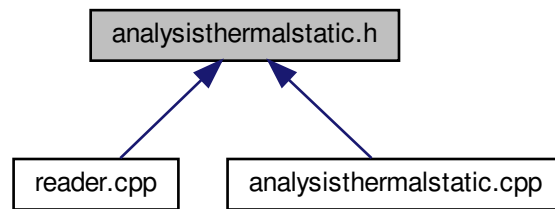
9.10 analysisthermalstatic.h File Reference

```
#include "analysis.h"
```

Include dependency graph for analysisthermalstatic.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::AnalysisThermalStatic](#)

Namespaces

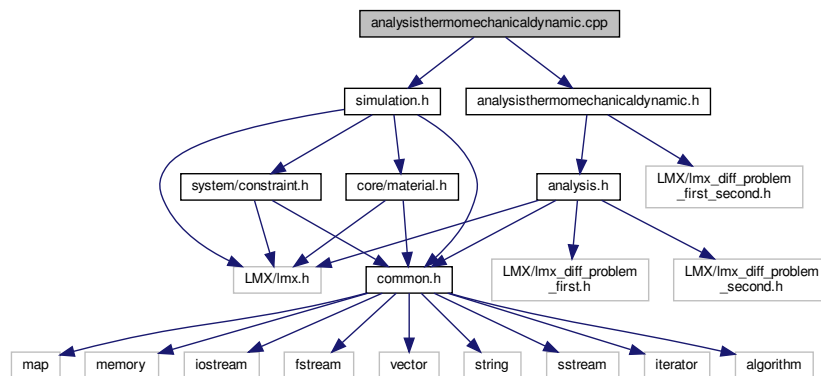
- [mknix](#)

9.11 analysisisthermomechanicaldynamic.cpp File Reference

```
#include "analysisisthermomechanicaldynamic.h"
```

```
#include "simulation.h"
```

Include dependency graph for `analysisisthermomechanicaldynamic.cpp`:



Namespaces

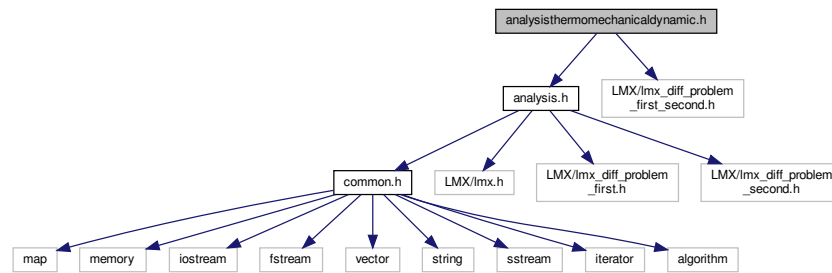
- [mknix](#)

9.12 analysisisthermomechanicaldynamic.h File Reference

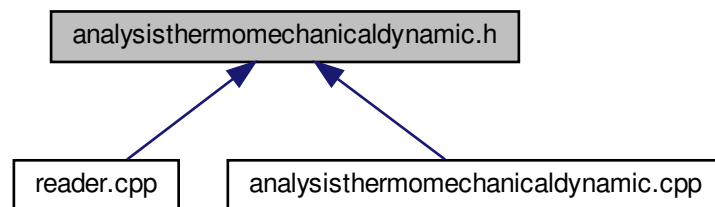
```
#include "analysis.h"
```

```
#include <LMX/lmx_diff_problem_first_second.h>
```

Include dependency graph for analysisthermomechanicaldynamic.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mnix::AnalysisThermoMechanicalDynamic`

Namespaces

- `mnix`

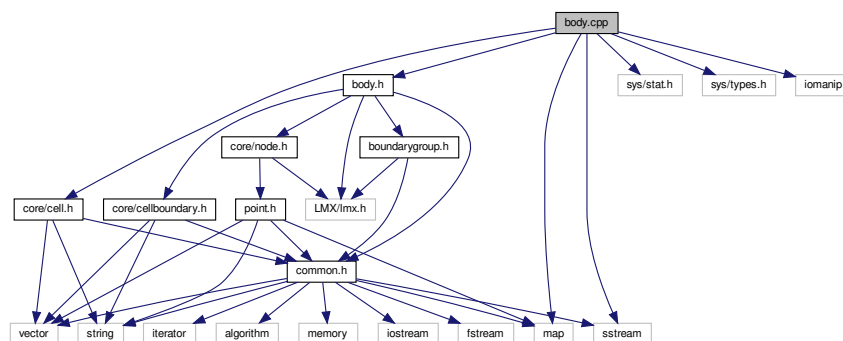
9.13 body.cpp File Reference

```

#include "body.h"
#include <core/cell.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <map>
#include <sstream>
#include <iomanip>

```


Include dependency graph for body.cpp:



Namespaces

- [mknix](#)

Functions

- constexpr double [mknix::deg2rad](#) (double deg)

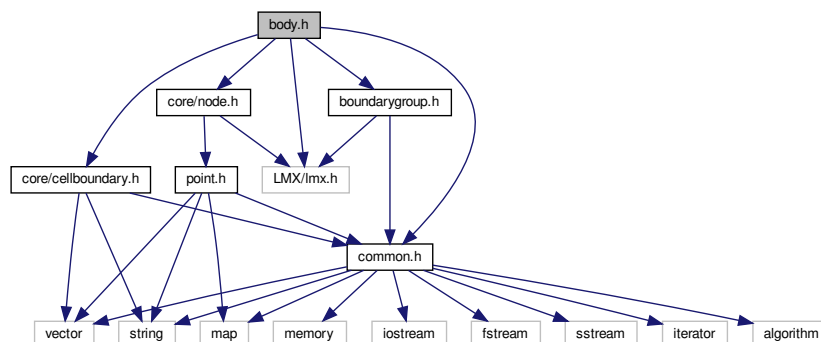
9.14 body.h File Reference

```

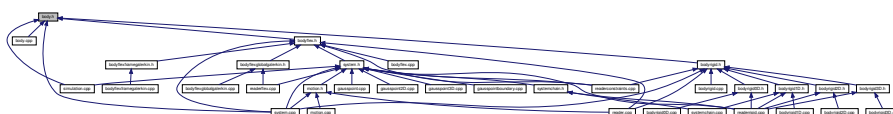
#include "common.h"
#include "LMX/lmx.h"
#include "boundarygroup.h"
#include <core/cellboundary.h>
#include <core/node.h>

```

Include dependency graph for body.h:



This graph shows which files directly or indirectly include this file:



Classes

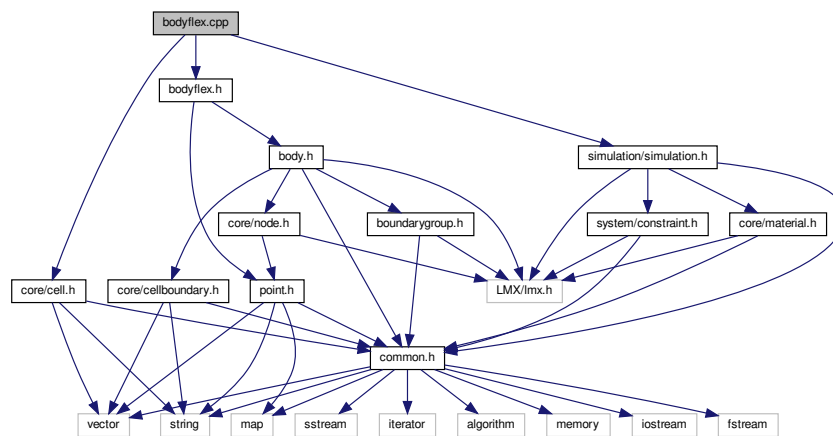
- class [mknix::Body](#)

Namespaces

- [mknix](#)

9.15 bodyflex.cpp File Reference

```
#include "bodyflex.h"
#include <core/cell.h>
#include <simulation/simulation.h>
Include dependency graph for bodyflex.cpp:
```

**Namespaces**

- [mknix](#)

9.16 bodyflex.h File Reference

```
#include "body.h"
#include <core/point.h>
```



- class `mknix::FlexBody`

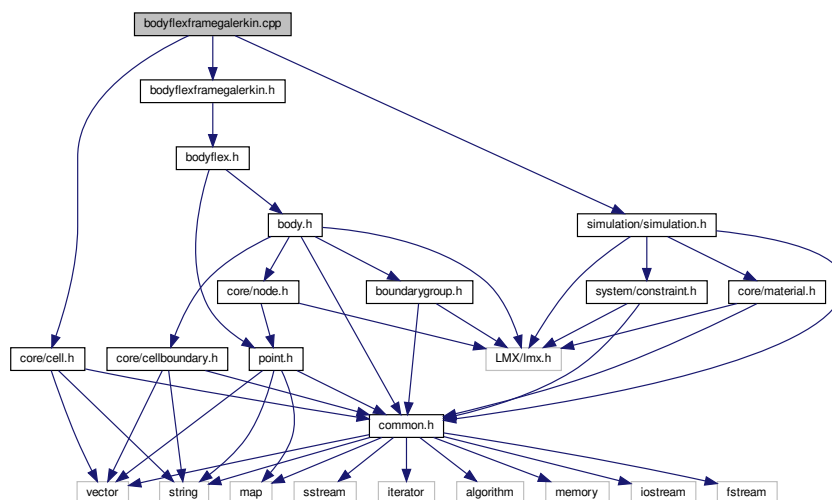
Namespaces

- mknix

9.17 bodyflexframegalerkin.cpp File Reference

```
#include "bodyflexframegalerkin.h"
#include <core/cell.h>
#include <simulation/simulation.h>
```

Include dependency graph for bodyflexframegalerkin.cpp:



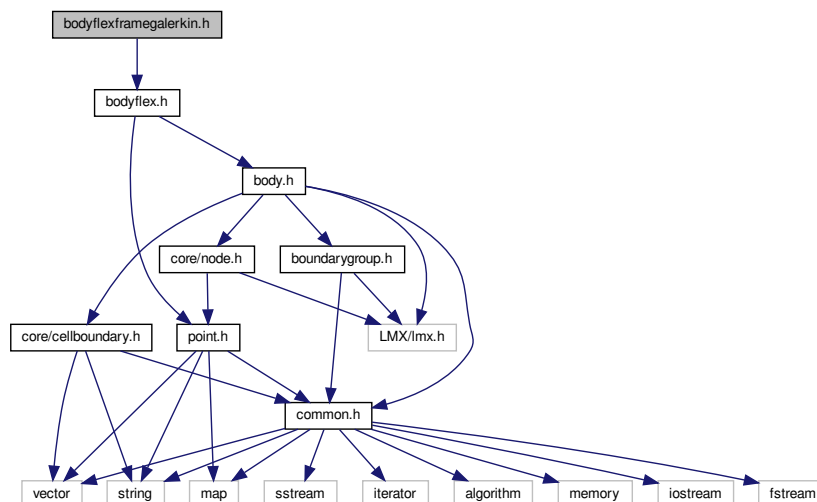
Namespaces

- [mknix](#)

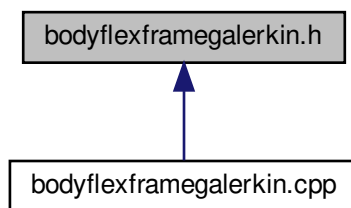
9.18 bodyflexframegalerkin.h File Reference

```
#include "bodyflex.h"
```

Include dependency graph for bodyflexframegalerkin.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::FlexFrameGalerkin](#)

Namespaces

- [mknix](#)

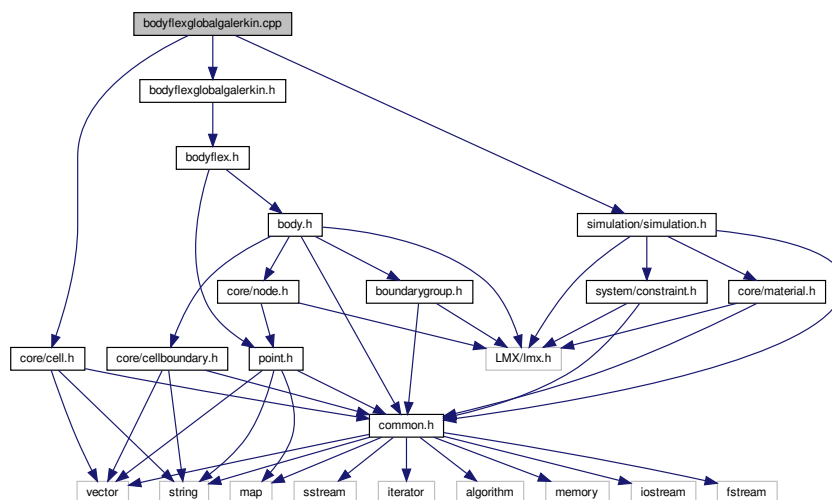
9.19 bodyflexglobalgalerkin.cpp File Reference

```
#include "bodyflexglobalgalerkin.h"
```

```
#include <core/cell.h>
```

```
#include <simulation/simulation.h>
```

Include dependency graph for `bodyflexglobalgalerkin.cpp`:



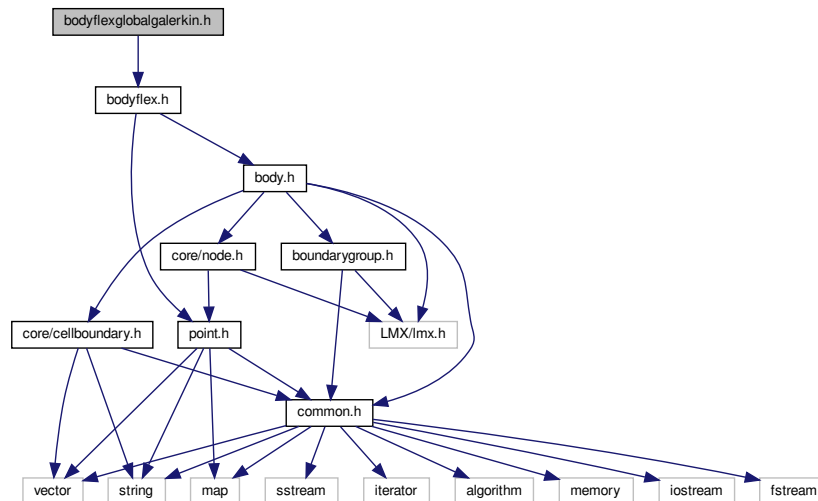
Namespaces

- [mknix](#)

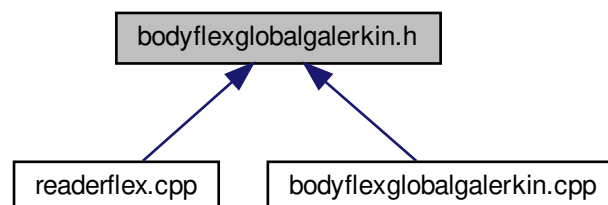
9.20 bodyflexglobalgalerkin.h File Reference

```
#include "bodyflex.h"
```

Include dependency graph for bodyflexglobalgalerkin.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::FlexGlobalGalerkin](#)

Namespaces

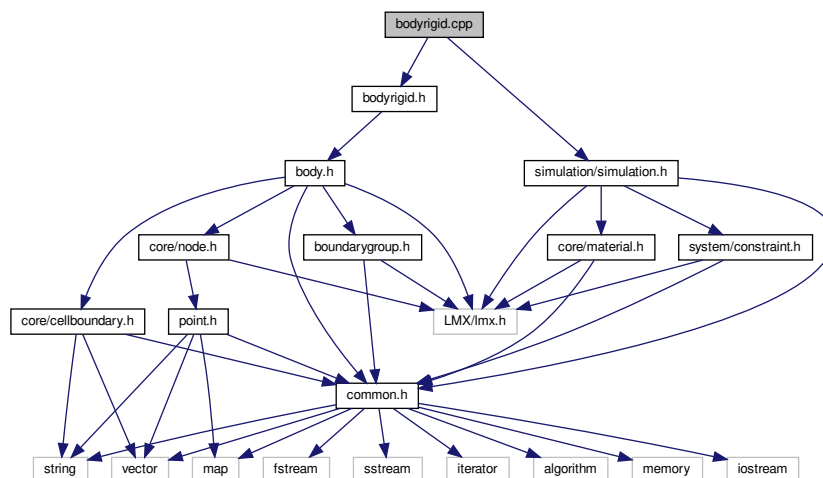
- [mknix](#)

9.21 bodyrigid.cpp File Reference

```
#include "bodyrigid.h"
```

```
#include <simulation/simulation.h>
```

Include dependency graph for bodyrigid.cpp:



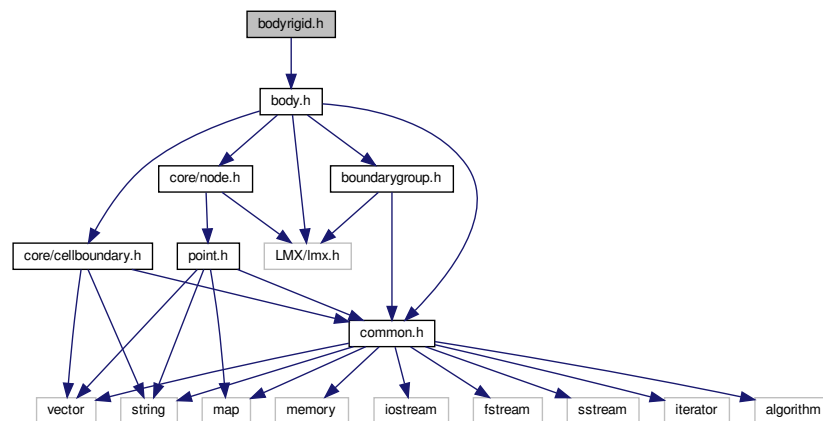
Namespaces

- [mknix](#)

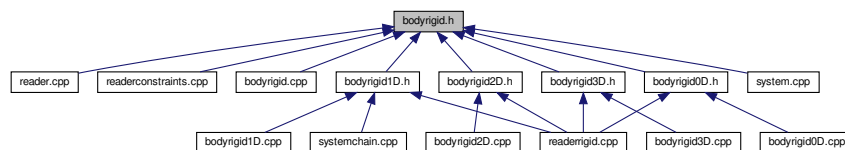
9.22 bodyrigid.h File Reference

```
#include "body.h"
```

Include dependency graph for bodyrigid.h:



This graph shows which files directly or indirectly include this file:



Classes

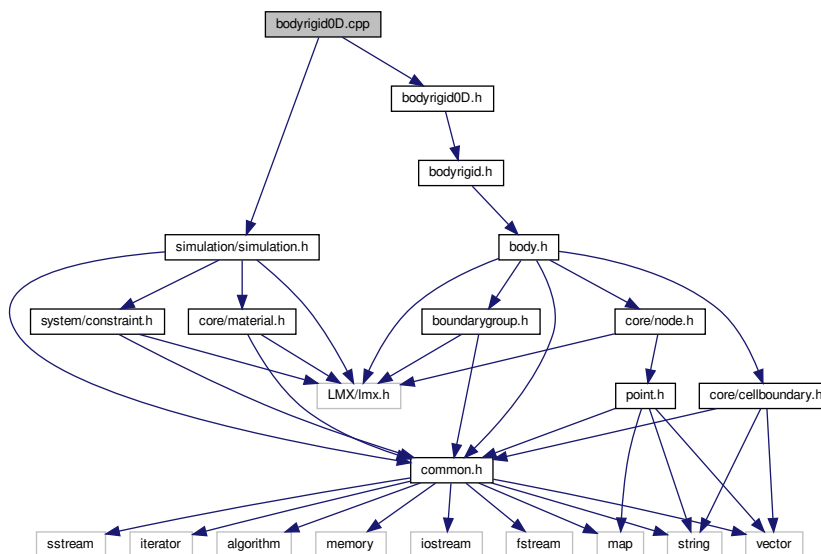
- class [mknix::RigidBody](#)

Namespaces

- [mknix](#)

9.23 bodyrigid0D.cpp File Reference

```
#include "bodyrigid0D.h"
#include <simulation/simulation.h>
Include dependency graph for bodyrigid0D.cpp:
```



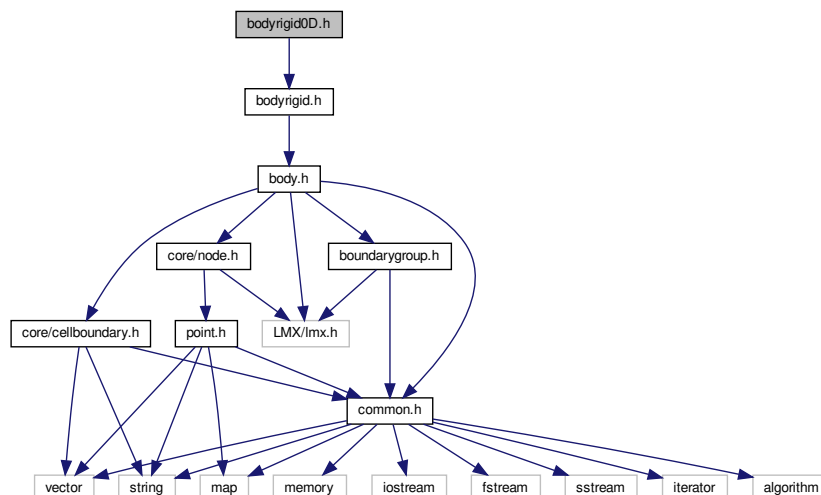
Namespaces

- [mknix](#)

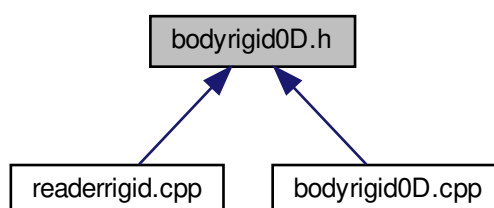
9.24 bodyrigid0D.h File Reference

```
#include "bodyrigid.h"
```


Include dependency graph for bodyrigid0D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::RigidBodyMassPoint](#)

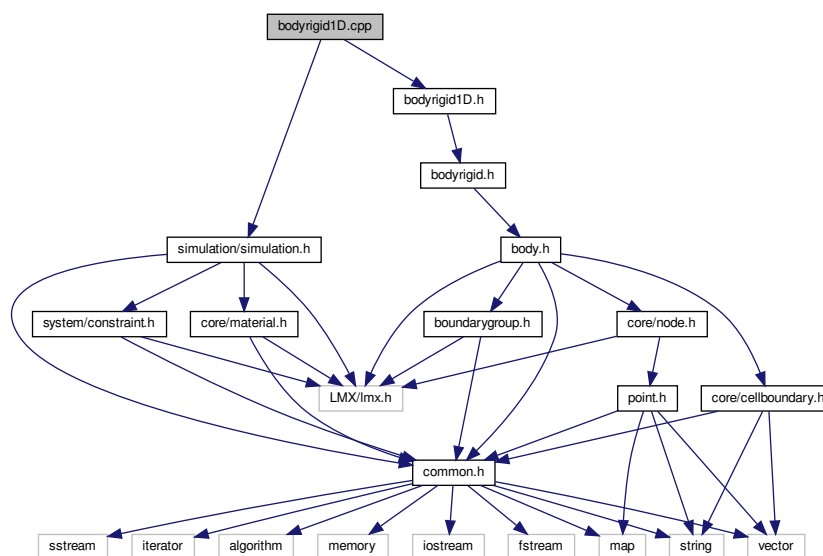
Namespaces

- [mknix](#)

9.25 bodyrigid1D.cpp File Reference

```
#include "bodyrigid1D.h"
#include <simulation/simulation.h>
```

Include dependency graph for bodyrigid1D.cpp:



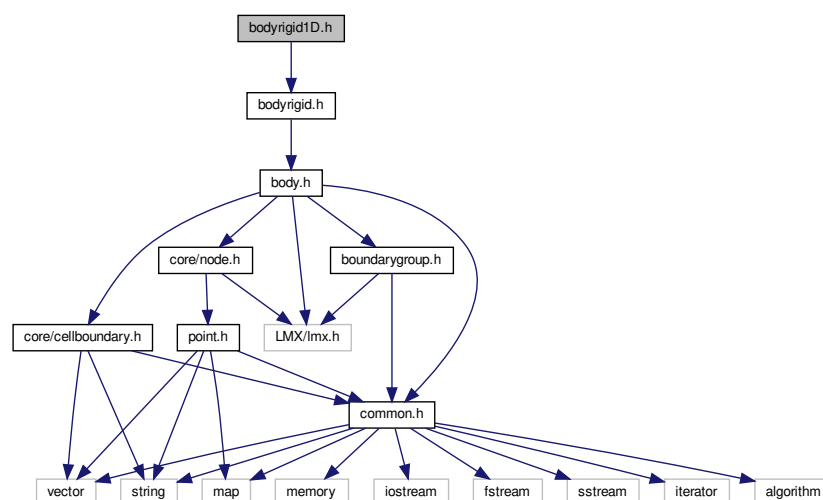
Namespaces

- [mknix](#)

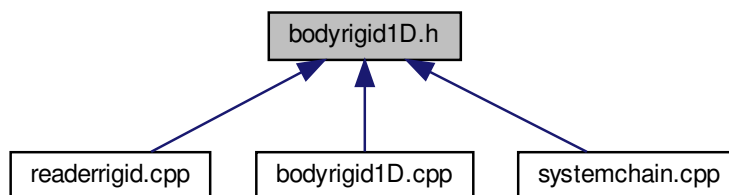
9.26 bodyrigid1D.h File Reference

```
#include "bodyrigid.h"
```

Include dependency graph for bodyrigid1D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mknix::RigidBar`

Namespaces

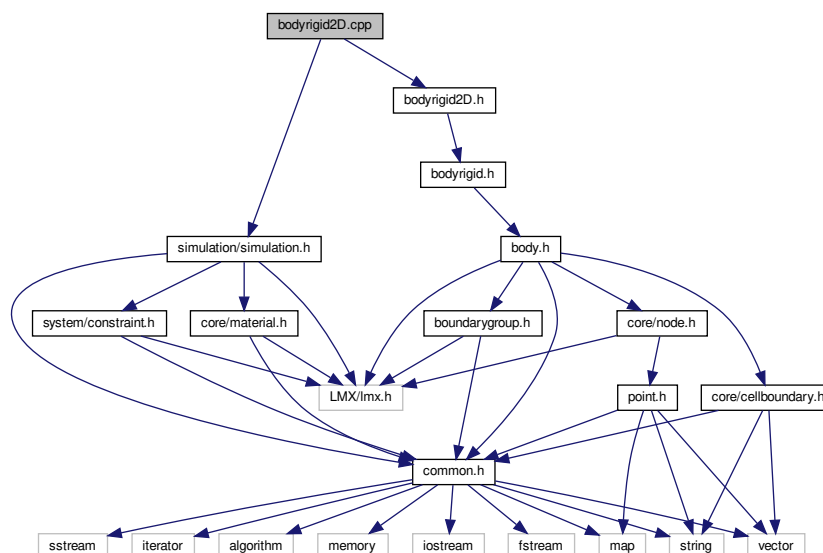
- `mknix`

9.27 bodyrigid2D.cpp File Reference

```
#include "bodyrigid2D.h"
```

```
#include <simulation/simulation.h>
```

Include dependency graph for `bodyrigid2D.cpp`:



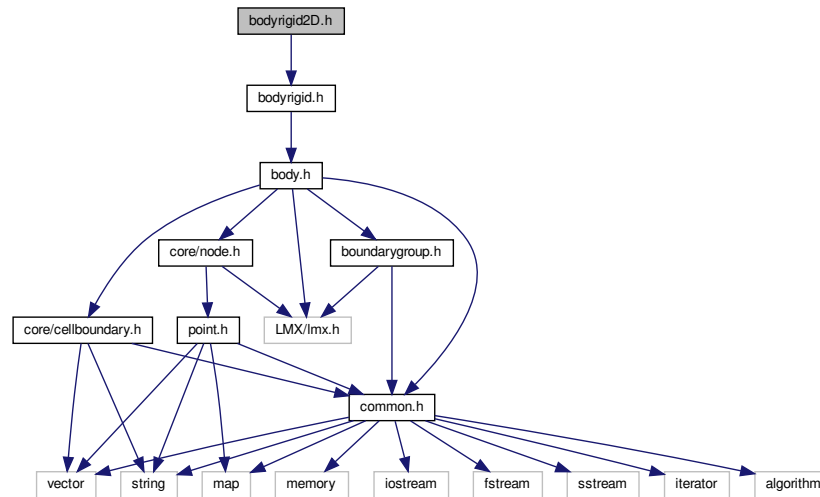
Namespaces

- `mknix`

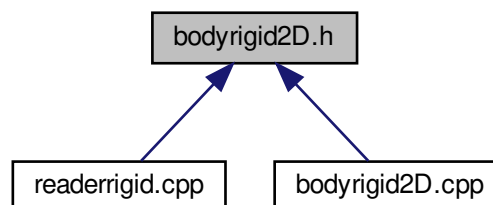
9.28 bodyrigid2D.h File Reference

```
#include "bodyrigid.h"
```

Include dependency graph for bodyrigid2D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::RigidBody2D](#)

Namespaces

- [mnix](#)

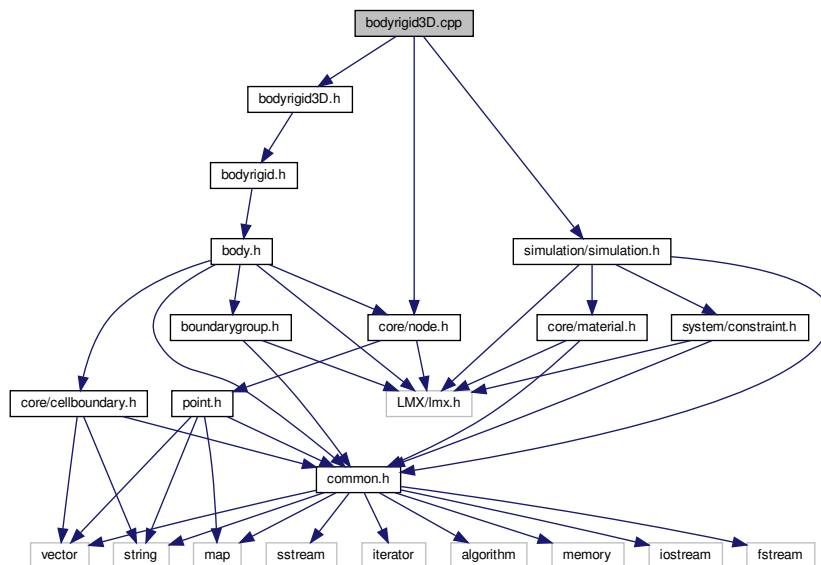
9.29 bodyrigid3D.cpp File Reference

```
#include "bodyrigid3D.h"
```

```
#include <core/node.h>
```

```
#include <simulation/simulation.h>
```

Include dependency graph for bodyrigid3D.cpp:



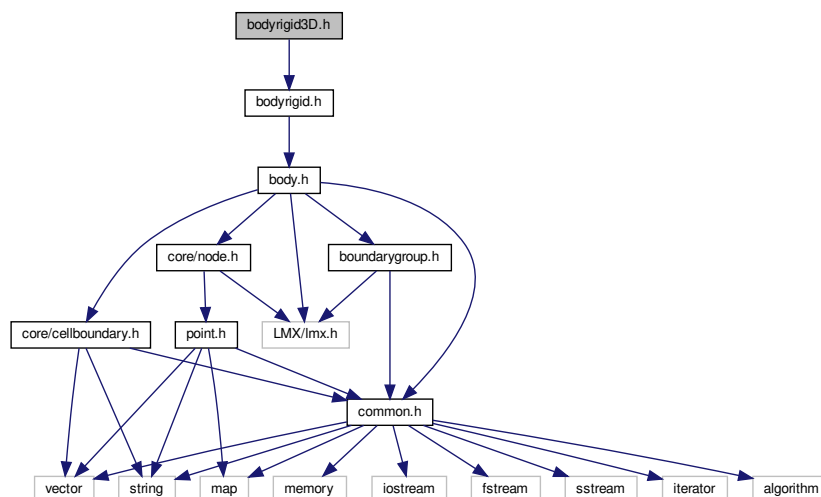
Namespaces

- [mknix](#)

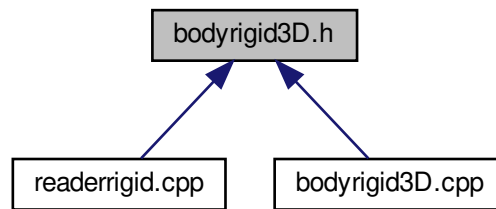
9.30 bodyrigid3D.h File Reference

```
#include "bodyrigid.h"
```

Include dependency graph for bodyrigid3D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::RigidBody3D](#)

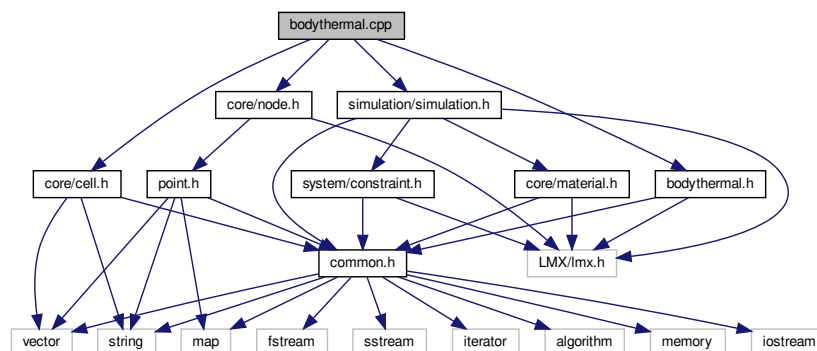
Namespaces

- [mknix](#)

9.31 bodythermal.cpp File Reference

```
#include "bodythermal.h"
#include <core/node.h>
#include <core/cell.h>
#include <simulation/simulation.h>
```

Include dependency graph for `bodythermal.cpp`:



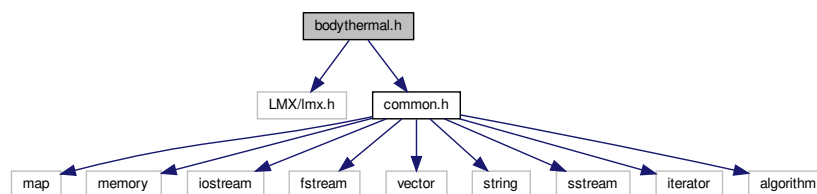
Namespaces

- [mknix](#)

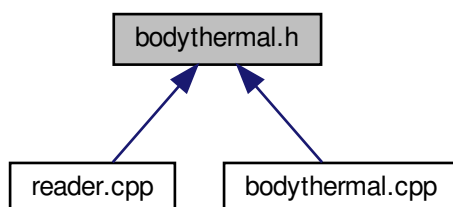
9.32 bodythermal.h File Reference

```
#include "LMX/lmx.h"
#include "common.h"
```

Include dependency graph for bodythermal.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::ThermalBody](#)

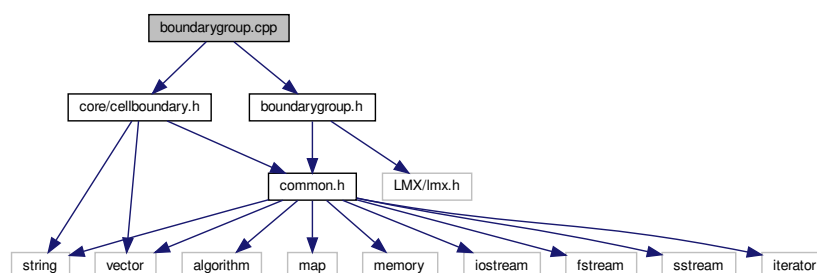
Namespaces

- [mnix](#)

9.33 boundarygroup.cpp File Reference

```
#include "boundarygroup.h"
#include <core/cellboundary.h>
```

Include dependency graph for boundarygroup.cpp:



Namespaces

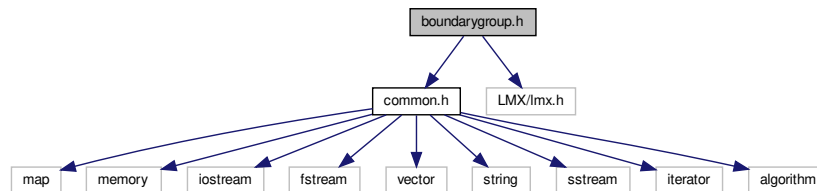
- [mknix](#)

9.34 boundarygroup.h File Reference

```
#include "common.h"
```

```
#include "LMX/lmx.h"
```

Include dependency graph for boundarygroup.h:



This graph shows which files directly or indirectly include this file:

**Classes**

- class [mknix::BoundaryGroup](#)

Namespaces

- [mknix](#)

9.35 cell.cpp File Reference

```
#include "LMX/lmx.h"
```

```
#include "cell.h"
```

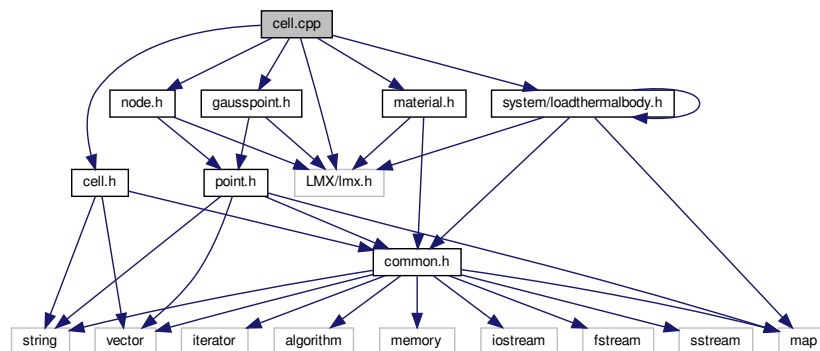
```
#include "material.h"
```

```
#include "gausspoint.h"
```

```
#include "node.h"
```

```
#include <system/loadthermalbody.h>
```


Include dependency graph for cell.cpp:



Namespaces

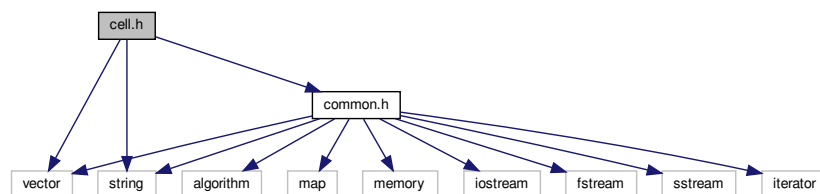
- [mnix](#)

9.36 cell.h File Reference

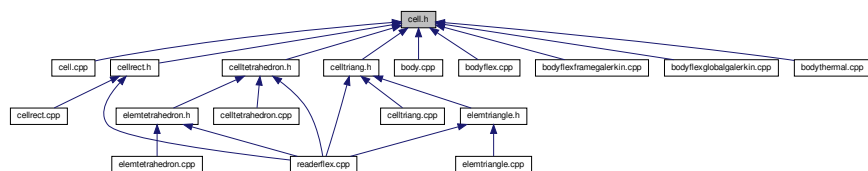
Background cells for integration.

```
#include <vector>
#include <string>
#include <common.h>
```

Include dependency graph for cell.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::Cell](#)

Namespaces

- [lmx](#)
- [mknix](#)

9.36.1 Detailed Description

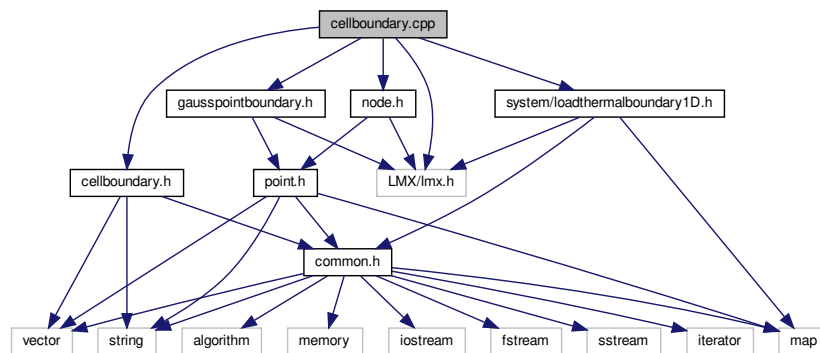
Background cells for integration.

Author

Daniel Iglesias

9.37 cellboundary.cpp File Reference

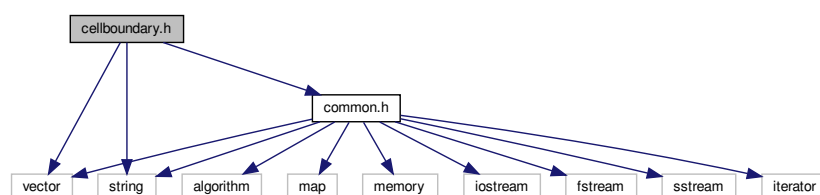
```
#include "LMX/lmx.h"
#include "node.h"
#include "cellboundary.h"
#include "gausspointboundary.h"
#include <system/loadthermalboundary1D.h>
Include dependency graph for cellboundary.cpp:
```

**Namespaces**

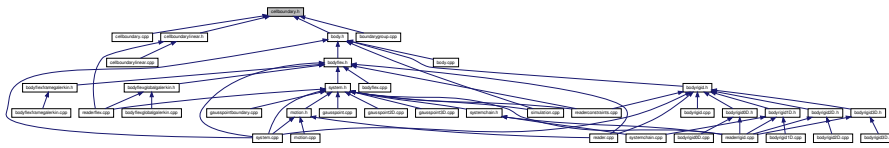
- [mknix](#)

9.38 cellboundary.h File Reference

```
#include <vector>
#include <string>
#include "common.h"
Include dependency graph for cellboundary.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::CellBoundary](#)

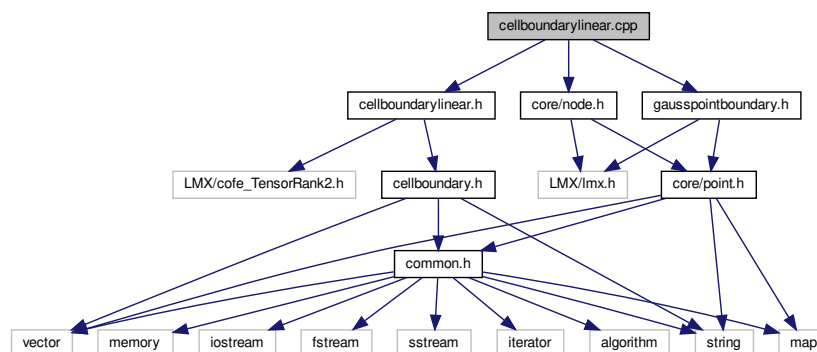
Namespaces

- [lmx](#)
- [mnix](#)

9.39 cellboundarylinear.cpp File Reference

```
#include "cellboundarylinear.h"
#include "gausspointboundary.h"
#include <core/node.h>
```

Include dependency graph for cellboundarylinear.cpp:



Namespaces

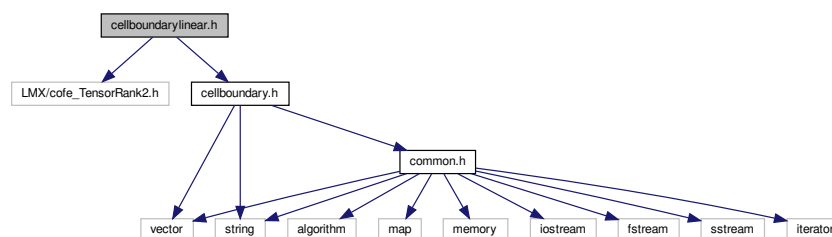
- [mnix](#)

9.40 cellboundarylinear.h File Reference

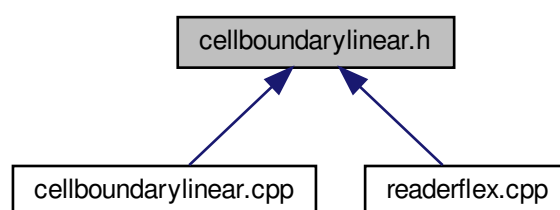
Background cells for boundary integration.

```
#include "LMX/cofe_TensorRank2.h"
#include "cellboundary.h"
```

Include dependency graph for cellboundarylinear.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::CellBoundaryLinear](#)

Namespaces

- [mknix](#)

9.40.1 Detailed Description

Background cells for boundary integration.

Author

Daniel Iglesias

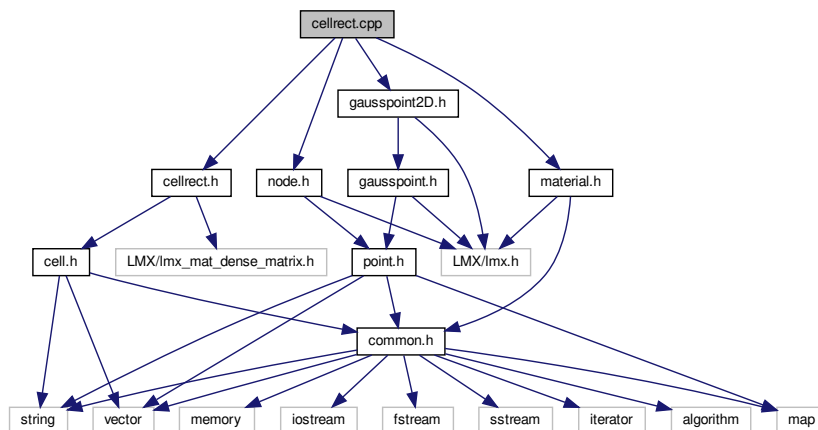
9.41 cellrect.cpp File Reference

```

#include "cellrect.h"
#include "material.h"
#include "node.h"
#include "gausspoint2D.h"

```

Include dependency graph for cellrect.cpp:



Namespaces

- [mknix](#)

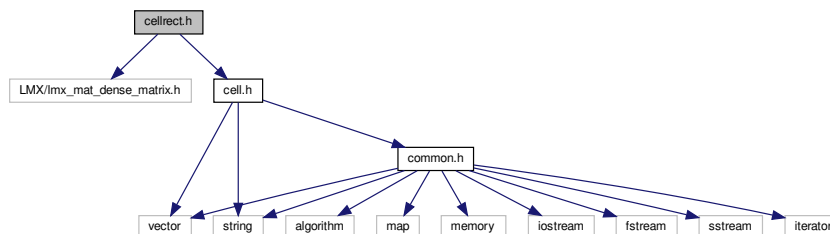
9.42 cellrect.h File Reference

Background cells for integration.

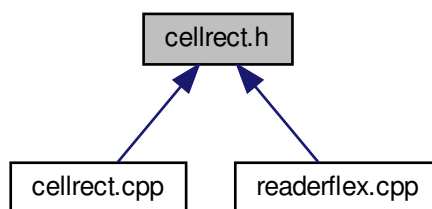
```
#include "LMX/lmx_mat_dense_matrix.h"
```

```
#include "cell.h"
```

Include dependency graph for cellrect.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::CellRect](#)

Namespaces

- [lmx](#)
- [mnix](#)

9.42.1 Detailed Description

Background cells for integration.

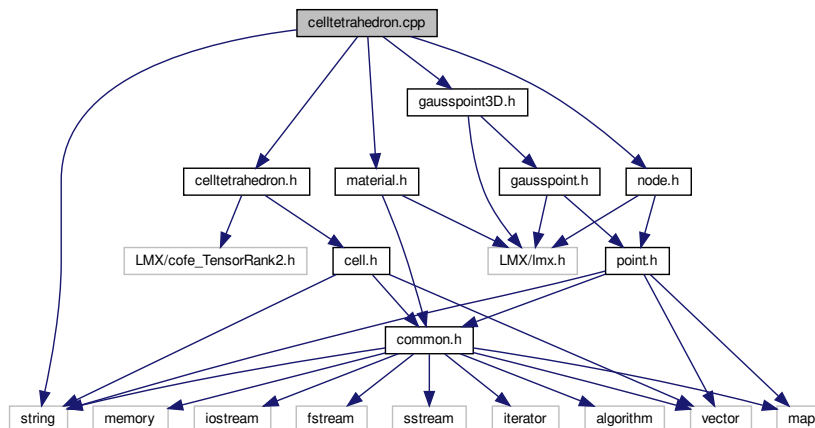
Author

Daniel Iglesias

9.43 celltetrahedron.cpp File Reference

```
#include "celltetrahedron.h"
#include "material.h"
#include "node.h"
#include "gausspoint3D.h"
#include <string>
```

Include dependency graph for `celltetrahedron.cpp`:



Namespaces

- [mknix](#)

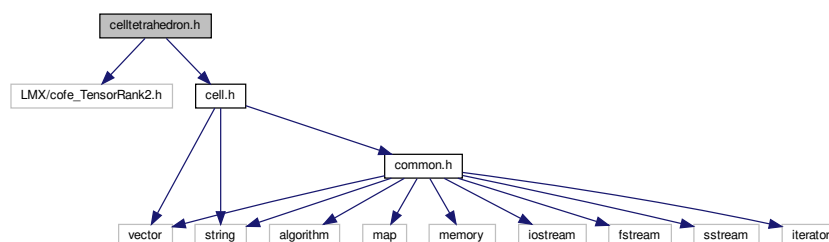
9.44 celltetrahedron.h File Reference

Background cells for integration.

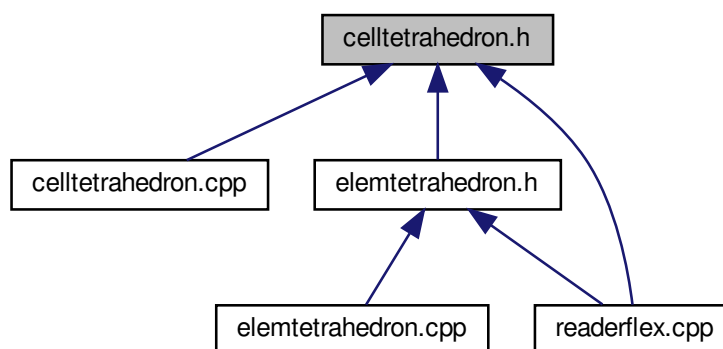
```
#include "LMX/cofe_TensorRank2.h"
```

```
#include "cell.h"
```

Include dependency graph for celltetrahedron.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::CellTetrahedron](#)

Namespaces

- [mknix](#)

9.44.1 Detailed Description

Background cells for integration.

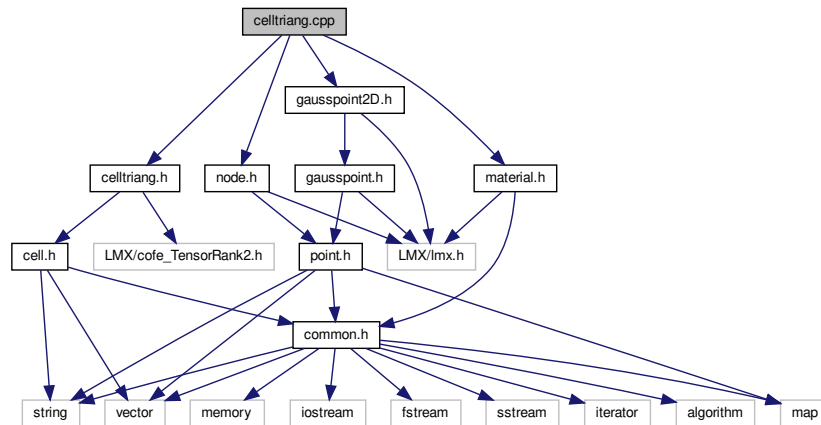
Author

Daniel Iglesias

9.45 celltriang.cpp File Reference

```
#include "celltriang.h"
#include "material.h"
#include "node.h"
#include "gausspoint2D.h"
```

Include dependency graph for celltriang.cpp:



Namespaces

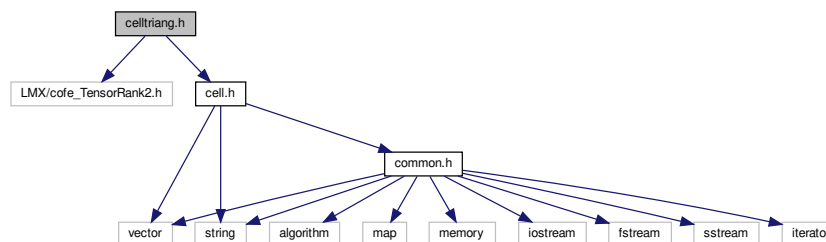
- [mknix](#)

9.46 celltriang.h File Reference

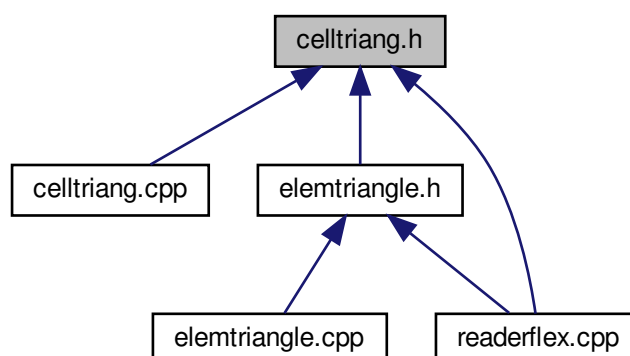
Background cells for integration.

```
#include "LMX/cofe_TensorRank2.h"
#include "cell.h"
```

Include dependency graph for celltriang.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::CellTriang](#)

Namespaces

- [mknix](#)

9.46.1 Detailed Description

Background cells for integration.

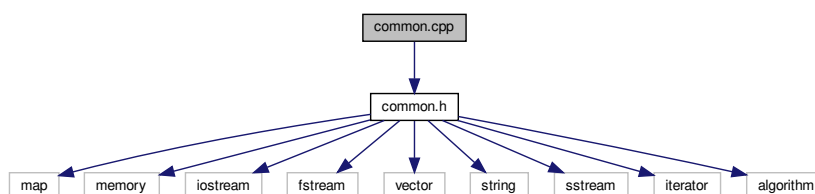
Author

Daniel Iglesias

9.47 common.cpp File Reference

```
#include "common.h"
```

Include dependency graph for `common.cpp`:



Namespaces

- [mknix](#)

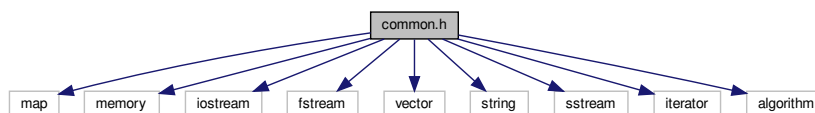
Functions

- double [mknix::interpolate1D](#) (double key, const std::map< double, double > &theMap)
Performs linear interpolation on a 1D map of (key, value) pairs.
- double [mknix::interpolate2D](#) (double key1, double key2, const std::map< double, std::map< double, double >> &the2DMap)
Performs bilinear interpolation on a 2D nested map of data points.
- double [mknix::interpolate3D](#) (double key1, double key2, double key3, const std::map< double, std::map< double, std::map< double, double >>> &the3DMap)
Performs trilinear interpolation on a 3D nested map of data points.
- std::vector< double > [mknix::doubles_in_vector](#) (const std::string &str)
Parses all whitespace-separated double values from a string.
- std::vector< std::vector< double > > [mknix::read_lines](#) (std::istream &stm)
Reads all lines from an input stream and parses each as a row of doubles.
- bool [mknix::isNumber](#) (const std::string &str)
Checks if a string represents a number.
- void [mknix::readFile3D](#) (const std::string &fileName, std::map< double, std::map< double, std::map< double, double >>> &dest)
Reads a 3D source distribution file into a nested map.

9.48 common.h File Reference

```
#include <map>
#include <memory>
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <iterator>
#include <algorithm>
```

Include dependency graph for common.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::boxFIR](#)

Namespaces

- [mknix](#)

Typedefs

- typedef double [mknix::data_type](#)

Functions

- double [mknix::interpolate1D](#) (double key, const std::map< double, double > &theMap)
Performs linear interpolation on a 1D map of (key, value) pairs.
- double [mknix::interpolate2D](#) (double key1, double key2, const std::map< double, std::map< double, double >> &the2DMap)
Performs bilinear interpolation on a 2D nested map of data points.
- double [mknix::interpolate3D](#) (double key1, double key2, double key3, const std::map< double, std::map< double, std::map< double, double >>> &the3DMap)
Performs trilinear interpolation on a 3D nested map of data points.
- template<class T, class... Args>
std::unique_ptr< T > [mknix::make_unique](#) (Args &&... args)
- std::vector< double > [mknix::doubles_in_vector](#) (const std::string &str)
Parses all whitespace-separated double values from a string.
- std::vector< std::vector< double > > [mknix::read_lines](#) (std::istream &stm)
Reads all lines from an input stream and parses each as a row of doubles.
- bool [mknix::isNumber](#) (const std::string &str)
Checks if a string represents a number.
- void [mknix::readFile3D](#) (const std::string &fileName, std::map< double, std::map< double, std::map< double, double >>> &dest)
Reads a 3D source distribution file into a nested map.

9.49 compbar.cpp File Reference

9.50 compbar.h File Reference

Classes

- class [mknix::CompBar](#)

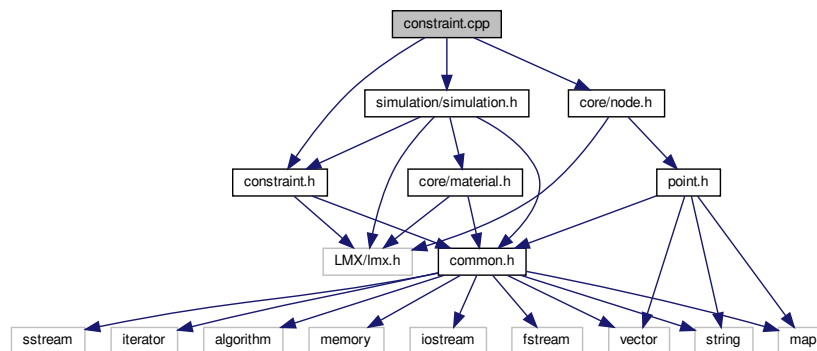
Namespaces

- [mknix](#)

9.51 constraint.cpp File Reference

```
#include "constraint.h"  
#include <core/node.h>  
#include <simulation/simulation.h>
```

Include dependency graph for constraint.cpp:



Namespaces

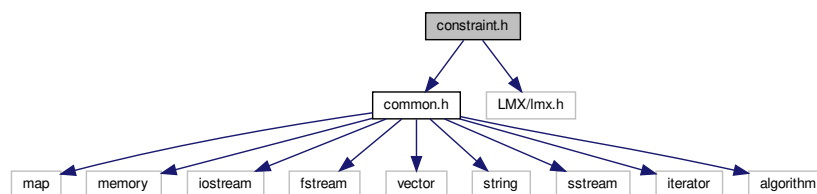
- [mnix](#)

9.52 constraint.h File Reference

```
#include "common.h"
```

```
#include "LMX/lmx.h"
```

Include dependency graph for constraint.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::Constraint](#)

Namespaces

- [mnix](#)

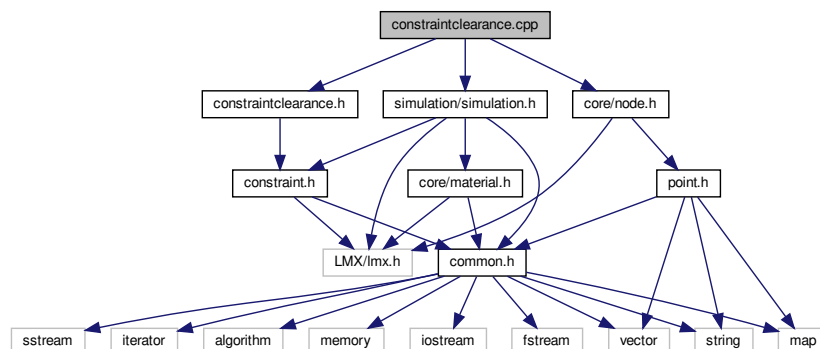
9.53 constraintclearance.cpp File Reference

```
#include "constraintclearance.h"
```

```
#include <core/node.h>
```

```
#include <simulation/simulation.h>
```

Include dependency graph for constraintclearance.cpp:



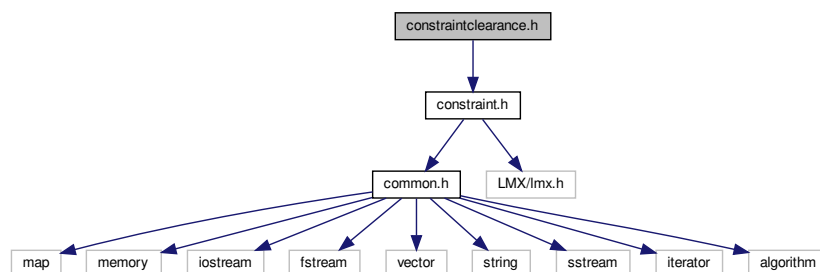
Namespaces

- [mknix](#)

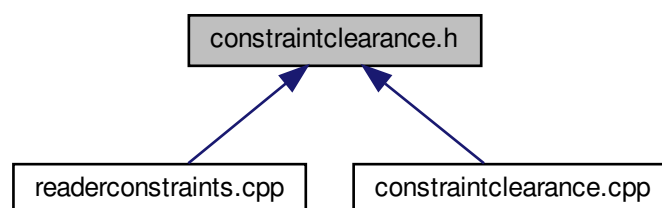
9.54 constraintclearance.h File Reference

```
#include "constraint.h"
```

Include dependency graph for constraintclearance.h:



This graph shows which files directly or indirectly include this file:



Classes

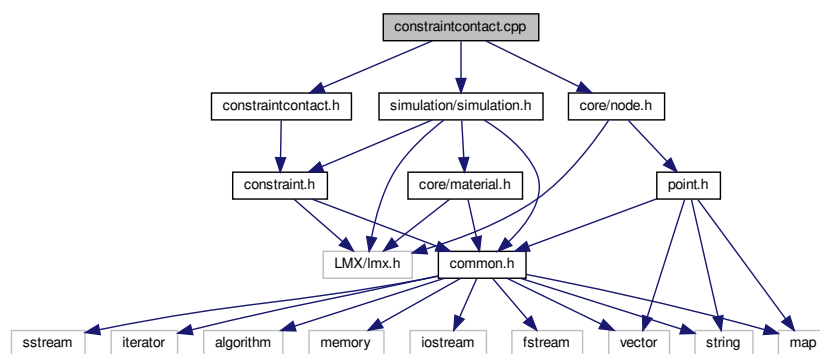
- class [mknix::ConstraintClearance](#)

Namespaces

- [mknix](#)

9.55 constraintcontact.cpp File Reference

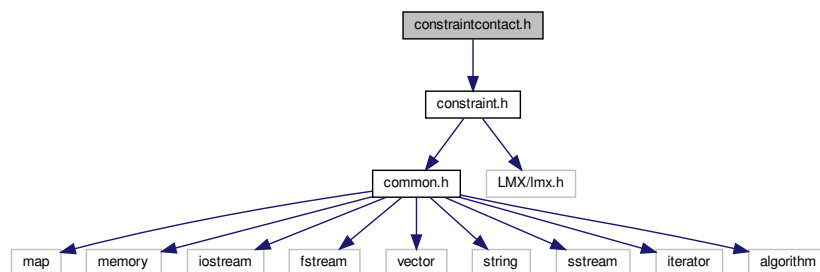
```
#include "constraintcontact.h"
#include <core/node.h>
#include <simulation/simulation.h>
Include dependency graph for constraintcontact.cpp:
```

**Namespaces**

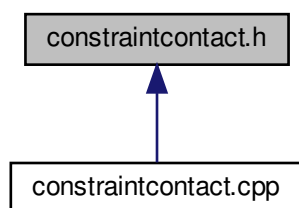
- [mknix](#)

9.56 constraintcontact.h File Reference

```
#include "constraint.h"
Include dependency graph for constraintcontact.h:
```



This graph shows which files directly or indirectly include this file:



Classes

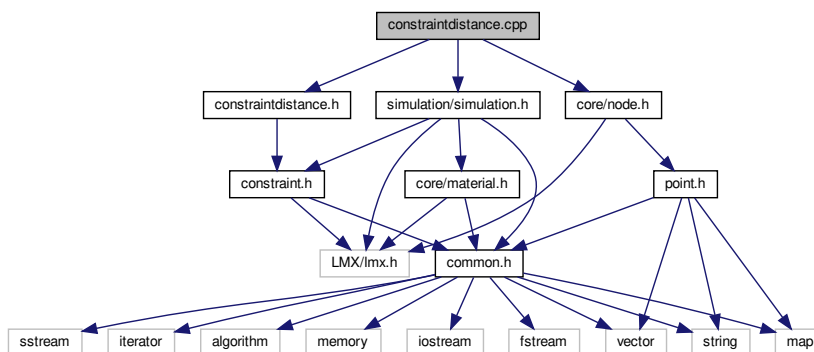
- class [mknix::ConstraintContact](#)

Namespaces

- [mknix](#)

9.57 constraintdistance.cpp File Reference

```
#include "constraintdistance.h"
#include <core/node.h>
#include <simulation/simulation.h>
Include dependency graph for constraintdistance.cpp:
```



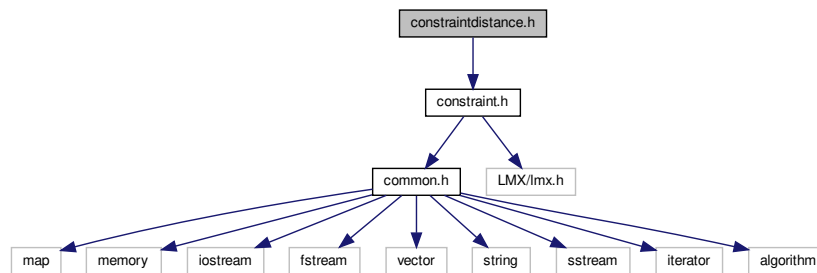
Namespaces

- [mknix](#)

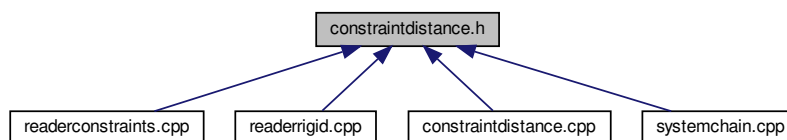
9.58 constraintdistance.h File Reference

```
#include "constraint.h"
```

Include dependency graph for constraintdistance.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ConstraintDistance](#)

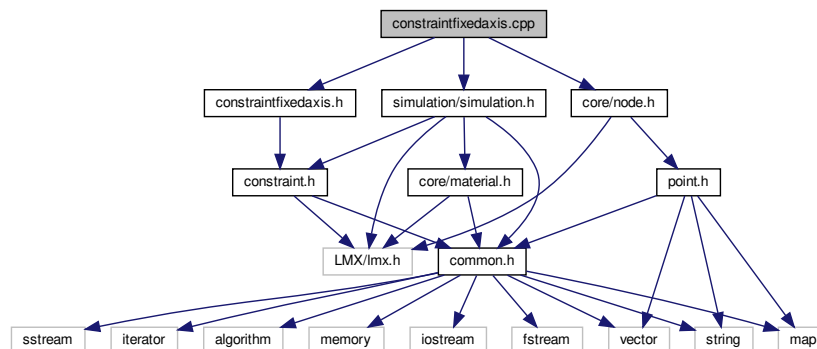
Namespaces

- [mknix](#)

9.59 constraintfixedaxis.cpp File Reference

```
#include "constraintfixedaxis.h"
#include <core/node.h>
#include <simulation/simulation.h>
```

Include dependency graph for constraintfixedaxis.cpp:



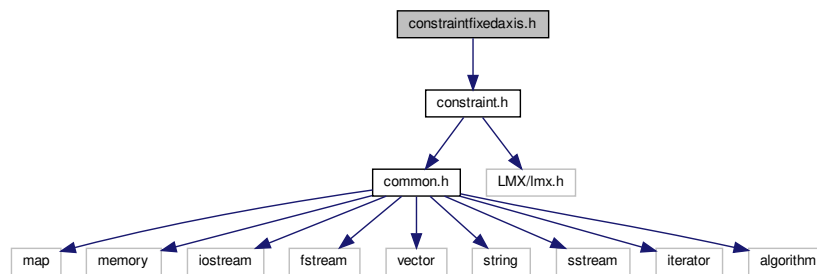
Namespaces

- [mknix](#)

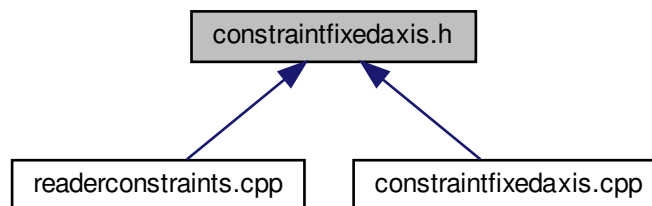
9.60 constraintfixedaxis.h File Reference

```
#include "constraint.h"
```

Include dependency graph for constraintfixedaxis.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ConstraintFixedAxis](#)

Namespaces

- [mknix](#)

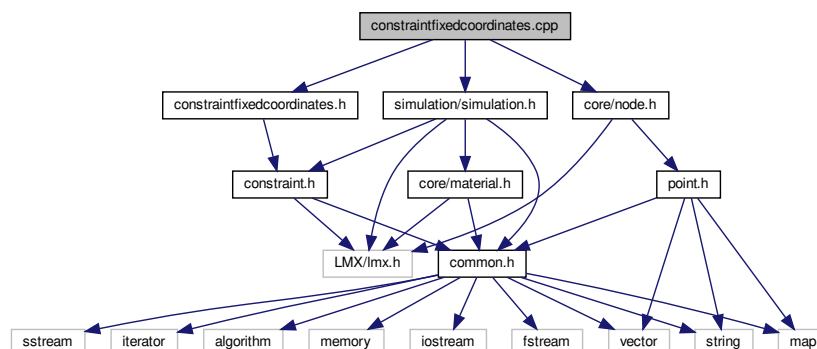
9.61 constraintfixedcoordinates.cpp File Reference

```
#include "constraintfixedcoordinates.h"
```

```
#include <simulation/simulation.h>
```

```
#include <core/node.h>
```

Include dependency graph for constraintfixedcoordinates.cpp:



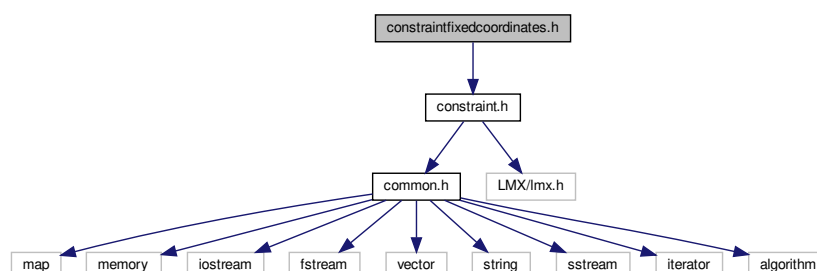
Namespaces

- [mnix](#)

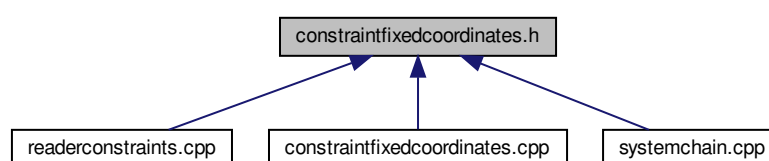
9.62 constraintfixedcoordinates.h File Reference

```
#include "constraint.h"
```

Include dependency graph for constraintfixedcoordinates.h:



This graph shows which files directly or indirectly include this file:



Classes

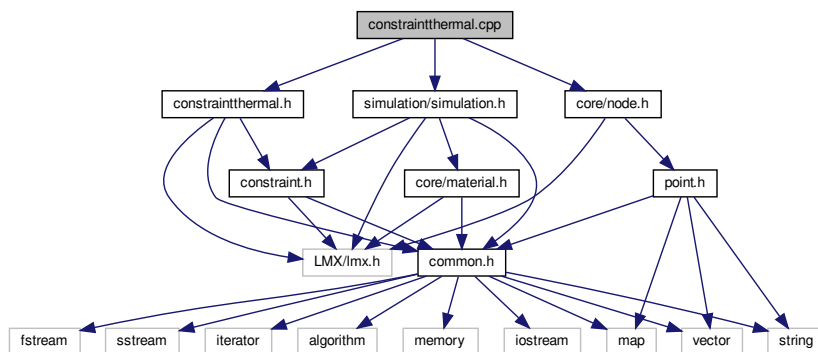
- class [mnix::ConstraintFixedCoordinates](#)

Namespaces

- [mknix](#)

9.63 constraintthermal.cpp File Reference

```
#include "constraintthermal.h"
#include <core/node.h>
#include <simulation/simulation.h>
Include dependency graph for constraintthermal.cpp:
```

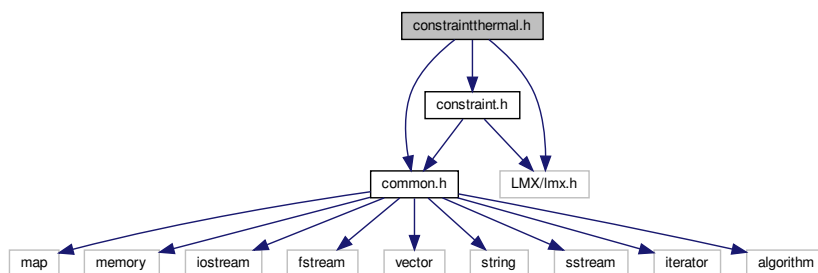


Namespaces

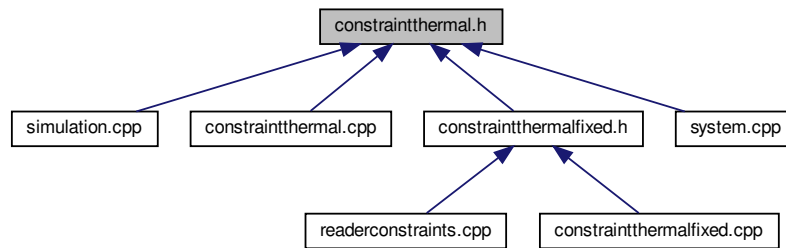
- [mknix](#)

9.64 constraintthermal.h File Reference

```
#include "common.h"
#include "LMX/lmx.h"
#include "constraint.h"
Include dependency graph for constraintthermal.h:
```



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ConstraintThermal](#)

Namespaces

- [mknix](#)

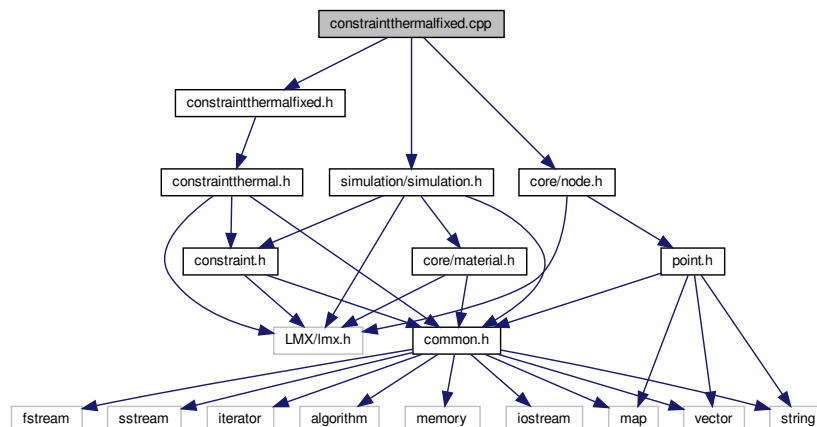
9.65 constraintthermalfixed.cpp File Reference

```
#include "constraintthermalfixed.h"
```

```
#include <core/node.h>
```

```
#include <simulation/simulation.h>
```

Include dependency graph for `constraintthermalfixed.cpp`:



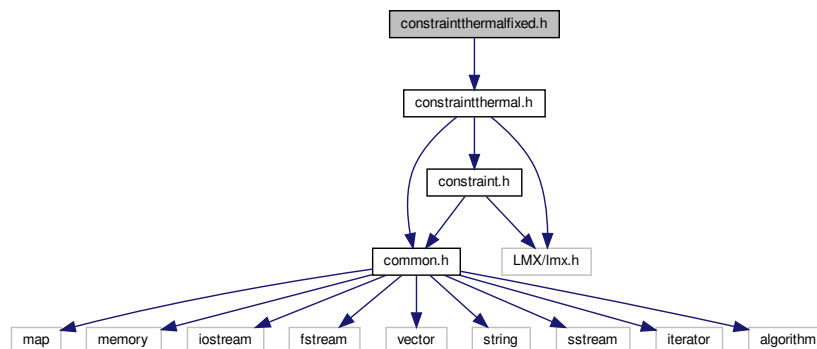
Namespaces

- [mknix](#)

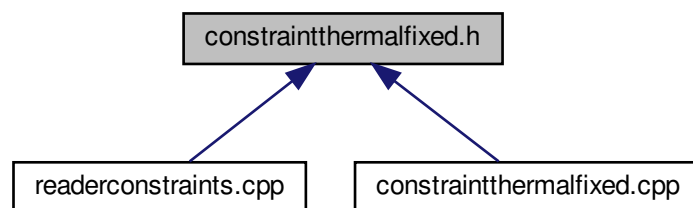
9.66 constraintthermalfixed.h File Reference

```
#include "constraintthermal.h"
```

Include dependency graph for `constraintthermalfixed.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ConstraintThermalFixed](#)

Namespaces

- [mknix](#)

9.67 dictionary.h File Reference

Namespaces

- [mknix](#)

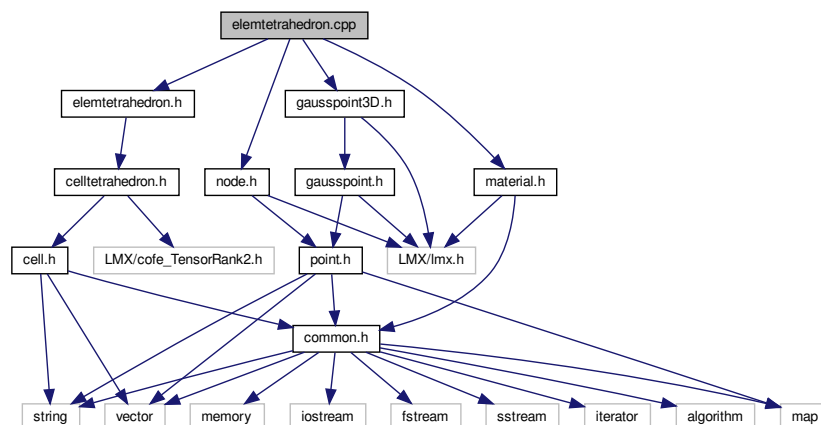
9.68 elemtetrahedron.cpp File Reference

```

#include "elemtetrahedron.h"
#include "material.h"
#include "node.h"
#include "gausspoint3D.h"

```

Include dependency graph for elemtetrahedron.cpp:



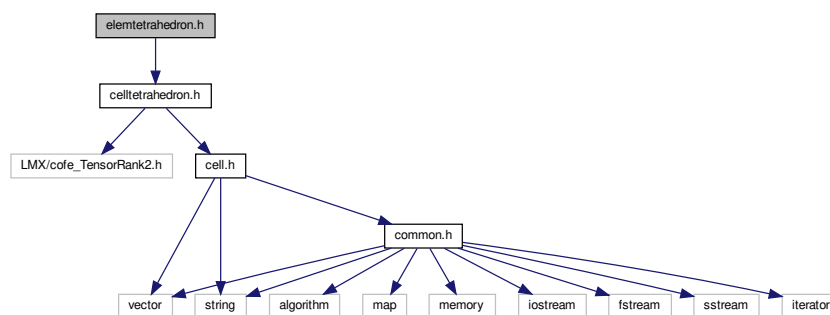
Namespaces

- [mknix](#)

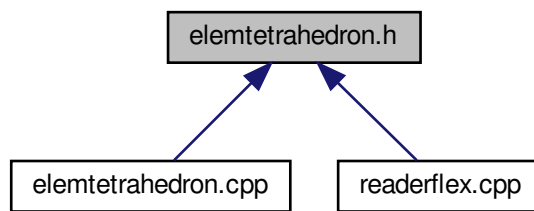
9.69 elemtetrahedron.h File Reference

```
#include "celltetrahedron.h"
```

Include dependency graph for elemtetrahedron.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ElemTetrahedron](#)

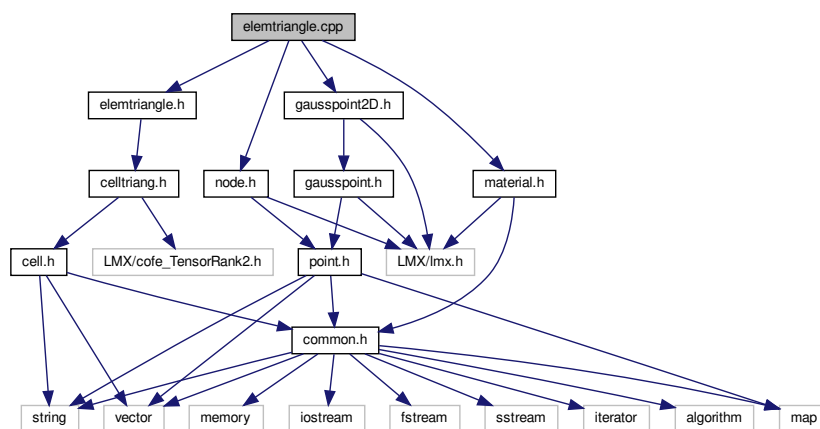
Namespaces

- [mknix](#)

9.70 elemtriangle.cpp File Reference

```
#include "elemtriangle.h"
#include "material.h"
#include "node.h"
#include "gausspoint2D.h"
```

Include dependency graph for `elemtriangle.cpp`:



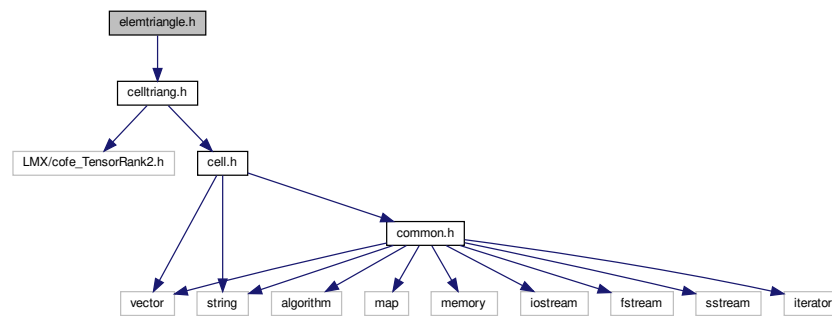
Namespaces

- [mknix](#)

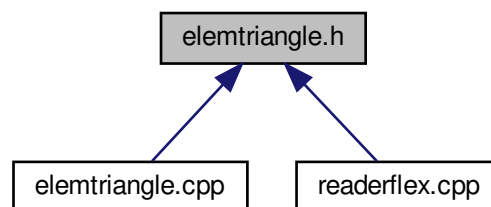
9.71 elemtriangle.h File Reference

```
#include "celltriang.h"
```

Include dependency graph for elemtriangle.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ElemTriangle](#)

Namespaces

- [mknix](#)

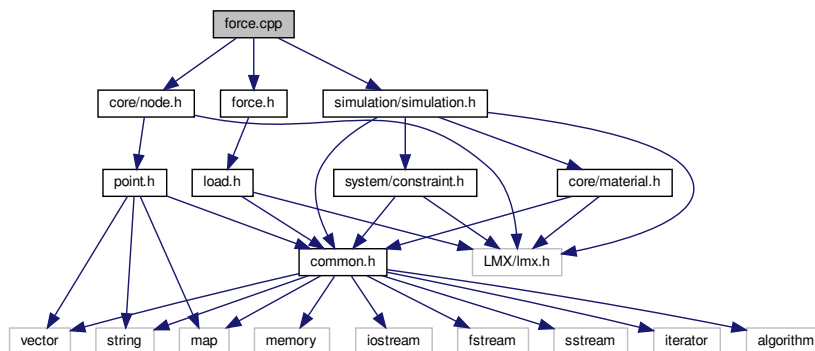
9.72 force.cpp File Reference

```

#include "force.h"
#include <core/node.h>
#include <simulation/simulation.h>

```


Include dependency graph for force.cpp:



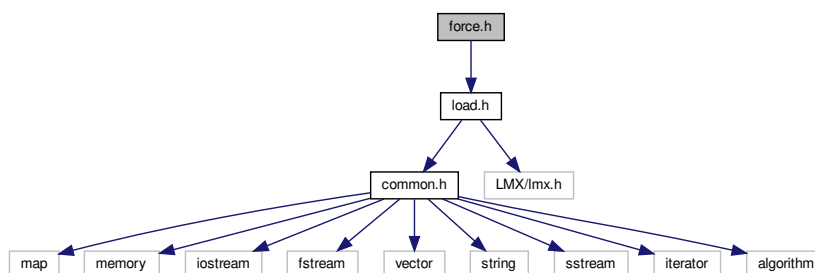
Namespaces

- [mknix](#)

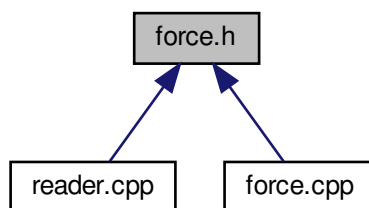
9.73 force.h File Reference

```
#include "load.h"
```

Include dependency graph for force.h:



This graph shows which files directly or indirectly include this file:



Classes

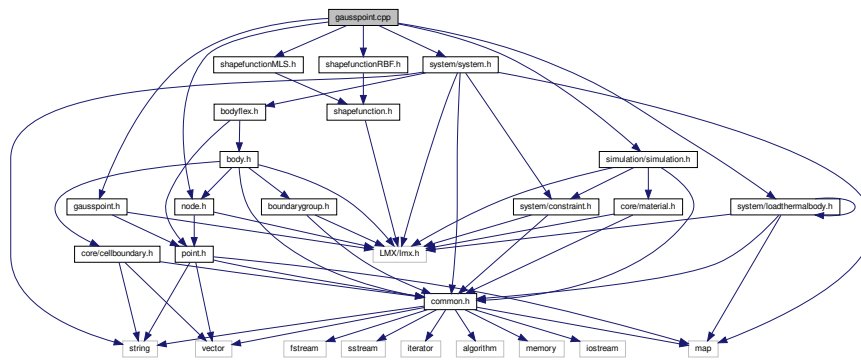
- class [mknix::Force](#)

Namespaces

- [mknix](#)

9.74 gausspoint.cpp File Reference

```
#include "gausspoint.h"
#include "node.h"
#include "shapefunctionRBF.h"
#include "shapefunctionMLS.h"
#include <simulation/simulation.h>
#include <system/system.h>
#include <system/loadthermalbody.h>
Include dependency graph for gausspoint.cpp:
```

**Namespaces**

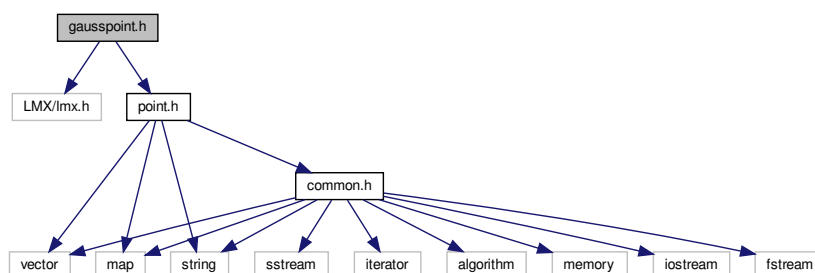
- [mknix](#)

9.75 gausspoint.h File Reference

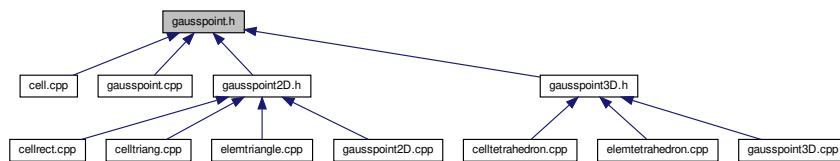
Point for numerical integration.

```
#include "LMX/lmx.h"
#include "point.h"
```

Include dependency graph for `gausspoint.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::GaussPoint](#)

Namespaces

- [mknix](#)

9.75.1 Detailed Description

Point for numerical integration.

Author

Daniel Iglesias

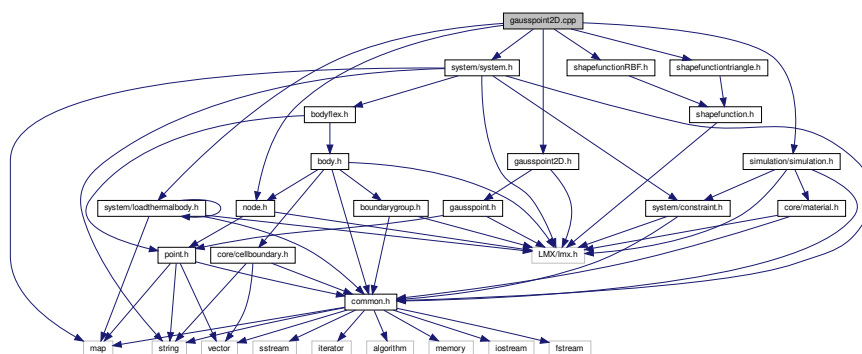
9.76 gausspoint2D.cpp File Reference

```

#include "gausspoint2D.h"
#include "node.h"
#include "shapefunctionRBF.h"
#include "shapefunctiontriangle.h"
#include <simulation/simulation.h>
#include <system/system.h>
#include <system/loadthermalbody.h>

```

Include dependency graph for gausspoint2D.cpp:



Namespaces

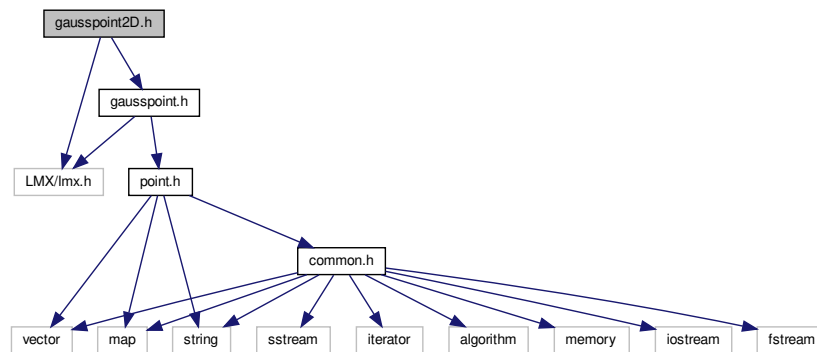
- [mknix](#)

9.77 gausspoint2D.h File Reference

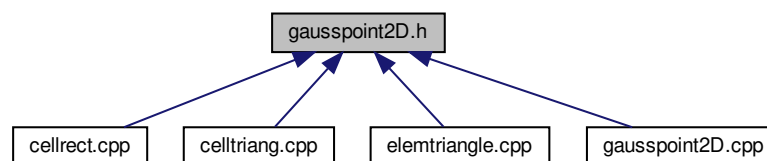
```
#include "LMX/lmx.h"
```

```
#include "gausspoint.h"
```

Include dependency graph for gausspoint2D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::GaussPoint2D](#)

Namespaces

- [mknix](#)

9.78 gausspoint3D.cpp File Reference

```
#include "gausspoint3D.h"
```

```
#include "node.h"
```

```
#include "shapefunctionRBF.h"
```

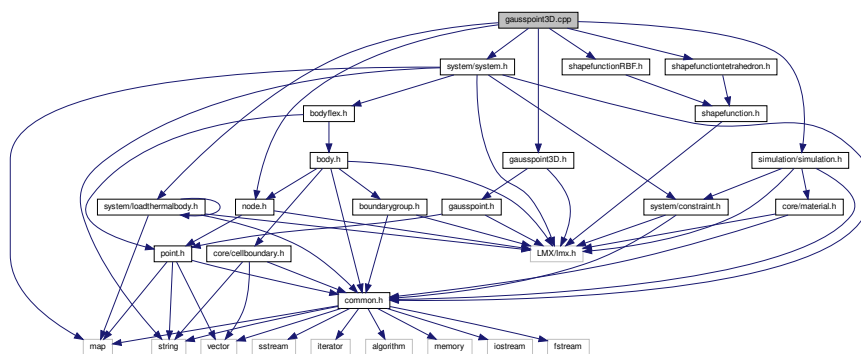
```
#include "shapefunctiontetrahedron.h"
```

```
#include <simulation/simulation.h>
```

```
#include <system/system.h>
```

```
#include <system/loadthermalbody.h>
```

Include dependency graph for gausspoint3D.cpp:



Namespaces

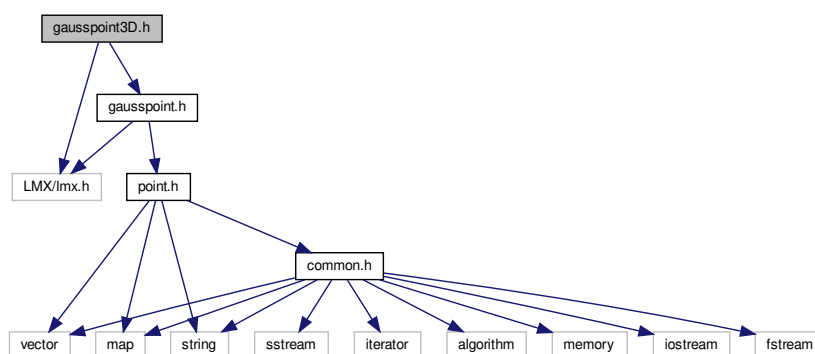
- [mknix](#)

9.79 gausspoint3D.h File Reference

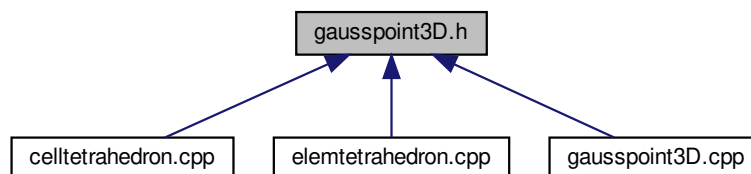
```
#include "LMX/lmx.h"
```

```
#include "gausspoint.h"
```

Include dependency graph for gausspoint3D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::GaussPoint3D](#)

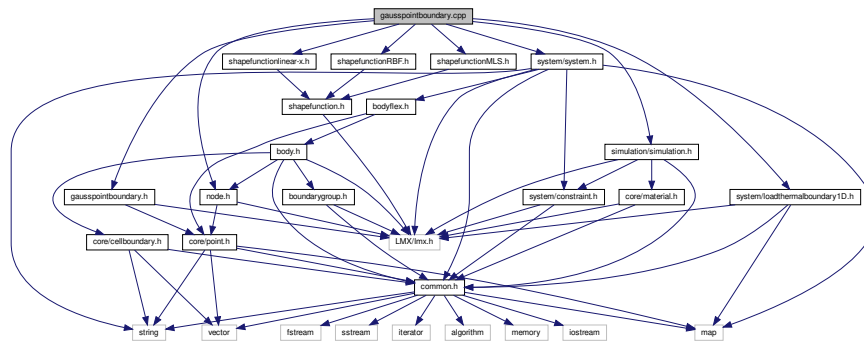
Namespaces

- [mknix](#)

9.80 gausspointboundary.cpp File Reference

```
#include "gausspointboundary.h"
#include "node.h"
#include "shapefunctionRBF.h"
#include "shapefunctionMLS.h"
#include "shapefunctionlinear-x.h"
#include <simulation/simulation.h>
#include <system/system.h>
#include <system/loadthermalboundary1D.h>
```

Include dependency graph for gausspointboundary.cpp:

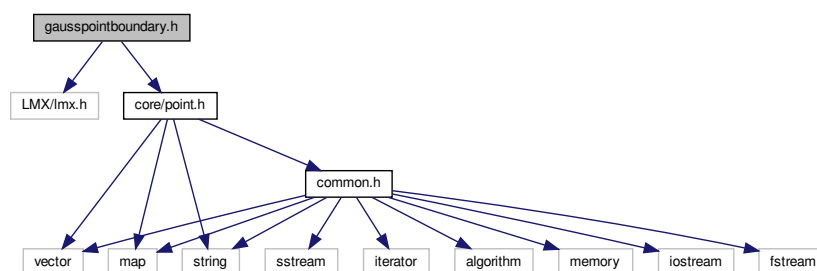
**Namespaces**

- [mknix](#)

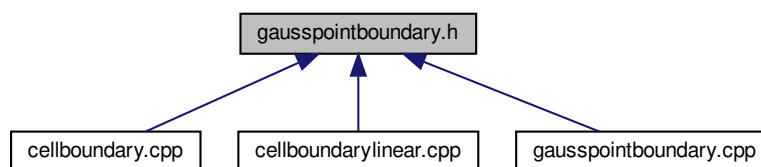
9.81 gausspointboundary.h File Reference

```
#include <LMX/lmx.h>
#include <core/point.h>
```

Include dependency graph for gausspointboundary.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mnix::GaussPointBoundary`

Namespaces

- `mnix`

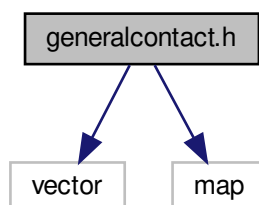
9.82 generalcontact.cpp File Reference

9.83 generalcontact.h File Reference

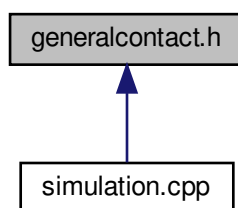
```
#include <vector>
```

```
#include <map>
```

Include dependency graph for `generalcontact.h`:



This graph shows which files directly or indirectly include this file:



Classes

- class `mnix::Contact`

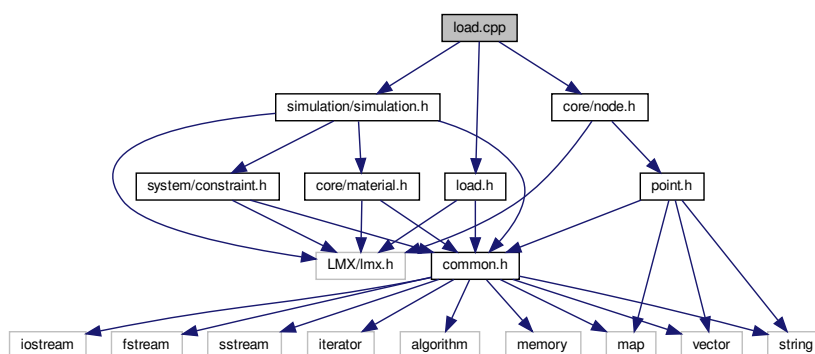
Namespaces

- `mnix`

9.84 input_manual.md File Reference

9.85 load.cpp File Reference

```
#include "load.h"
#include <simulation/simulation.h>
#include <core/node.h>
Include dependency graph for load.cpp:
```



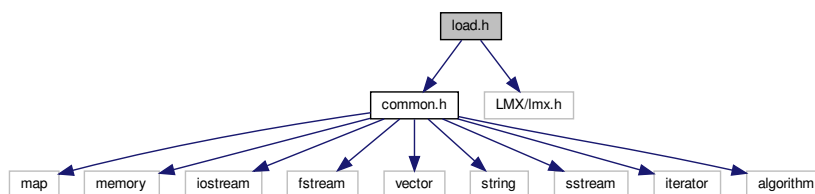
Namespaces

- `mnix`

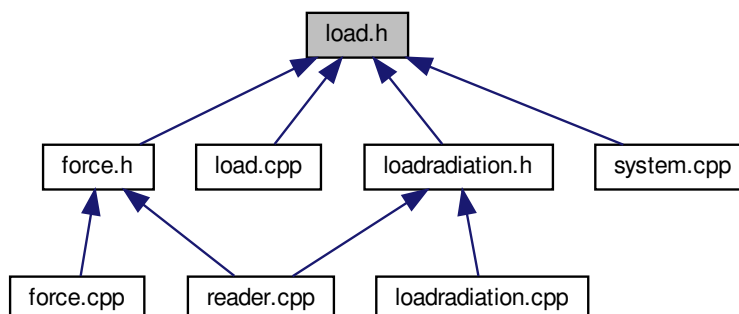
9.86 load.h File Reference

```
#include "common.h"
#include "LMX/lmx.h"
```


Include dependency graph for load.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::Load](#)

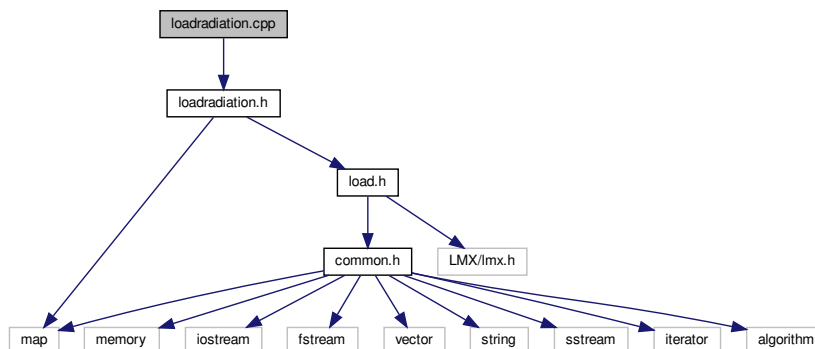
Namespaces

- [mnix](#)

9.87 loadradiation.cpp File Reference

```
#include "loadradiation.h"
```

Include dependency graph for loadradiation.cpp:



Namespaces

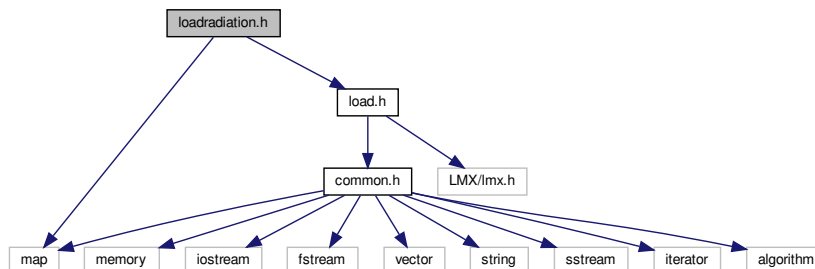
- [mknix](#)

9.88 loadradiation.h File Reference

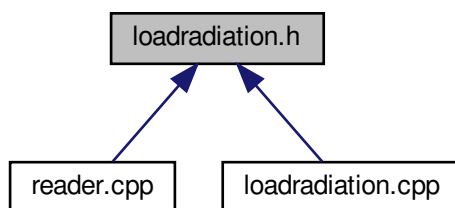
```
#include <map>
```

```
#include "load.h"
```

Include dependency graph for loadradiation.h:



This graph shows which files directly or indirectly include this file:



Classes

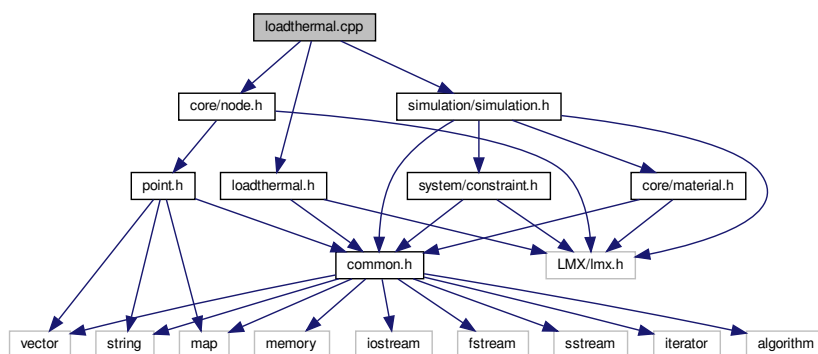
- class [mknix::Radiation](#)

Namespaces

- [mknix](#)

9.89 loadthermal.cpp File Reference

```
#include "loadthermal.h"
#include <core/node.h>
#include <simulation/simulation.h>
Include dependency graph for loadthermal.cpp:
```

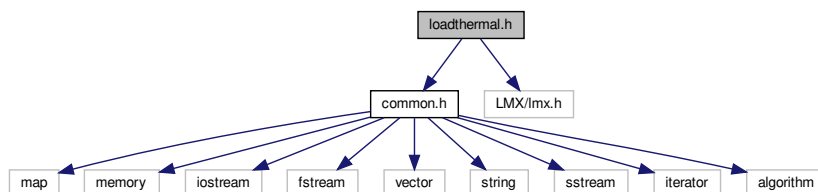


Namespaces

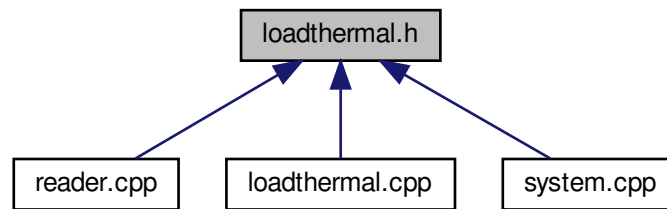
- [mknix](#)

9.90 loadthermal.h File Reference

```
#include "common.h"
#include "LMX/lmx.h"
Include dependency graph for loadthermal.h:
```



This graph shows which files directly or indirectly include this file:



Classes

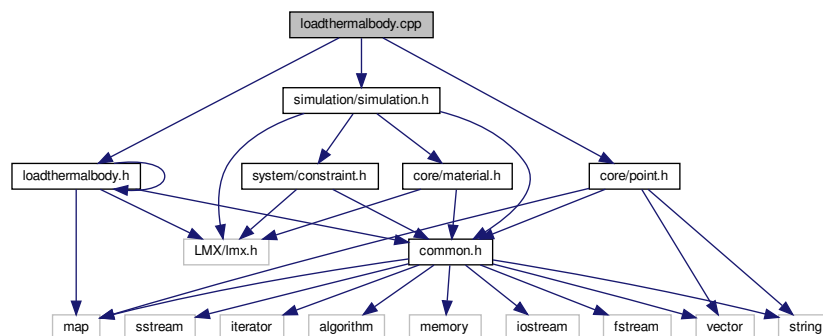
- class [mknix::LoadThermal](#)

Namespaces

- [mknix](#)

9.91 loadthermalbody.cpp File Reference

```
#include "loadthermalbody.h"
#include <core/point.h>
#include <simulation/simulation.h>
Include dependency graph for loadthermalbody.cpp:
```



Namespaces

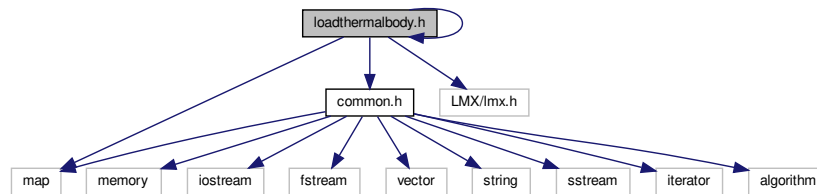
- [mknix](#)

9.92 loadthermalbody.h File Reference

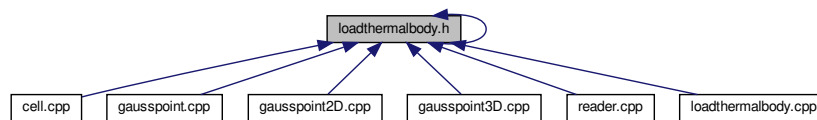
```
#include "common.h"
#include "LMX/lmx.h"
#include <map>
```

```
#include "loadthermalbody.h"
```

Include dependency graph for loadthermalbody.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::LoadThermalBody](#)

Namespaces

- [mknix](#)

Enumerations

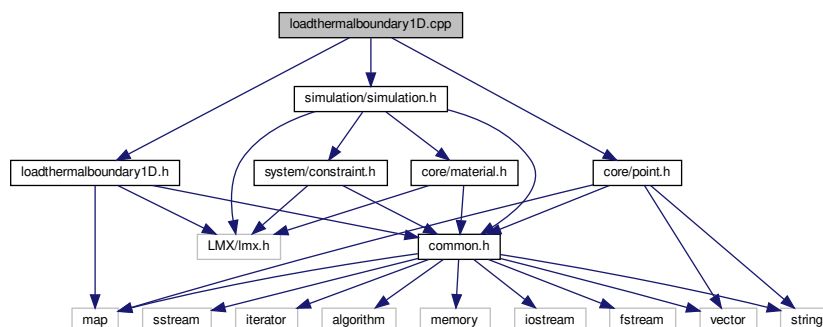
- enum class [mknix::SourceType](#) {
[mknix::CONSTANT](#) = 0 , [mknix::SOURCE_1D](#) = 1 , [mknix::SOURCE_2D](#) = 2 , [mknix::SOURCE_3D](#) = 3 ,
[mknix::SOURCE_VTK](#) = 4 }

Identifies which spatial source is active, in ascending priority order.

9.93 loadthermalboundary1D.cpp File Reference

```
#include "loadthermalboundary1D.h"
#include <core/point.h>
#include <simulation/simulation.h>
```

Include dependency graph for loadthermalboundary1D.cpp:



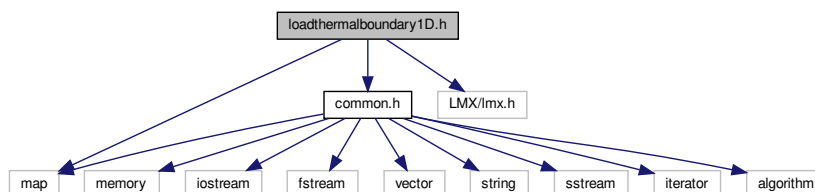
Namespaces

- [mnix](#)

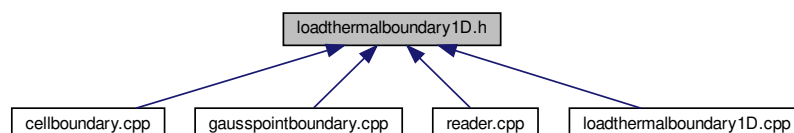
9.94 loadthermalboundary1D.h File Reference

```
#include "common.h"
#include "LMX/lmx.h"
#include <map>
```

Include dependency graph for loadthermalboundary1D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::LoadThermalBoundary1D](#)

Namespaces

- [mnix](#)

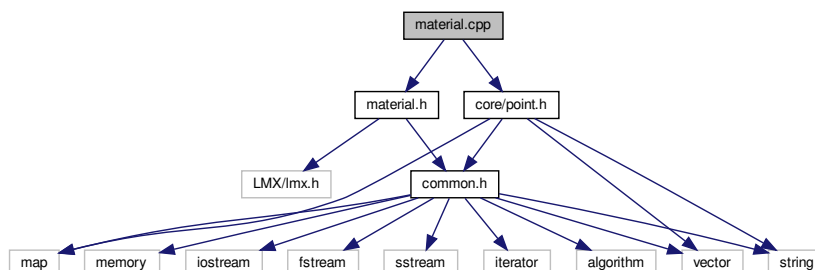
9.95 mainpage.md File Reference

9.96 material.cpp File Reference

```
#include "material.h"
```

```
#include <core/point.h>
```

Include dependency graph for material.cpp:



Namespaces

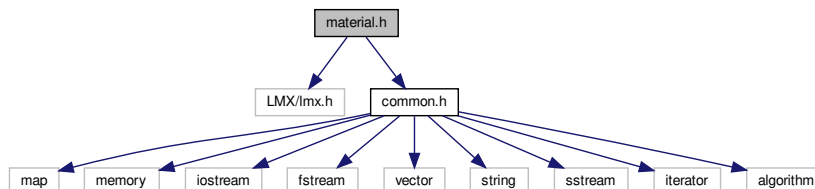
- [mknix](#)

9.97 material.h File Reference

```
#include "LMX/lmx.h"
```

```
#include "common.h"
```

Include dependency graph for material.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::Material](#)

Namespaces

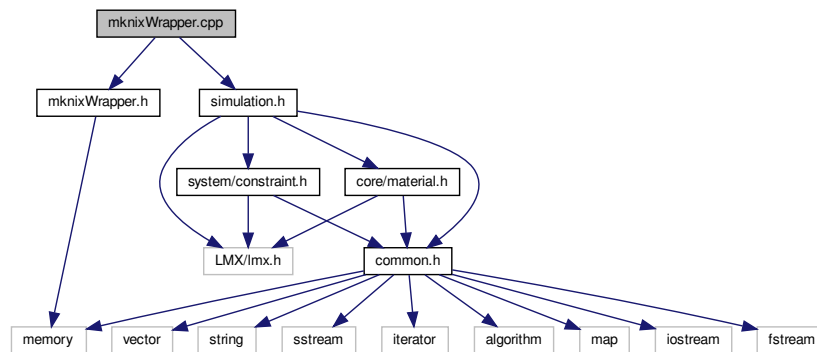
- [mknix](#)

9.98 mknixWrapper.cpp File Reference

```
#include "mknixWrapper.h"
```

```
#include "simulation.h"
```

Include dependency graph for mknixWrapper.cpp:



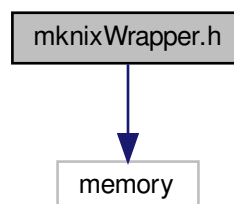
Classes

- struct [mknix::MknixWrapper::SimulationWrapper](#)

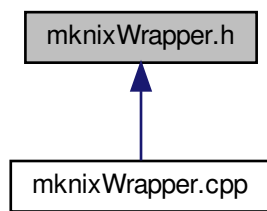
9.99 mknixWrapper.h File Reference

```
#include <memory>
```

Include dependency graph for mknixWrapper.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mknix::MknixWrapper`

Namespaces

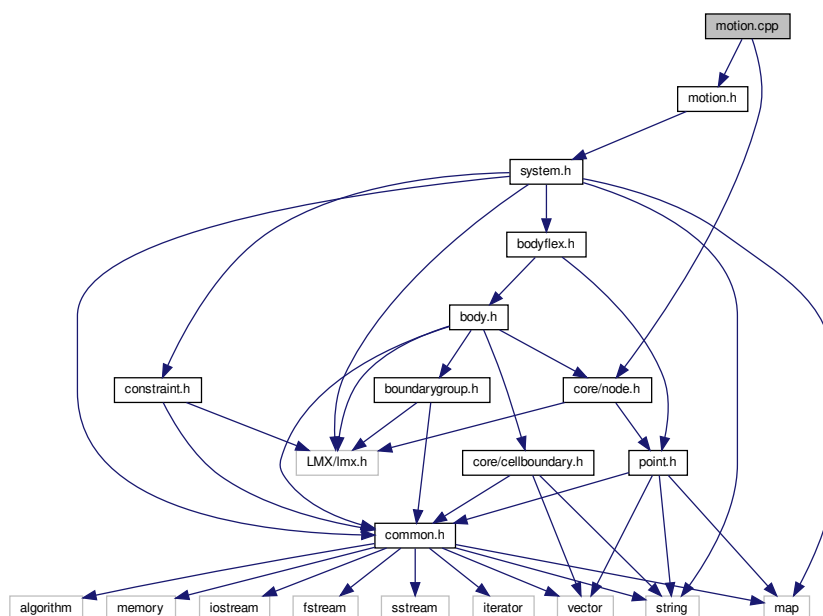
- `mknix`

9.100 motion.cpp File Reference

```
#include "motion.h"
```

```
#include <core/node.h>
```

Include dependency graph for `motion.cpp`:



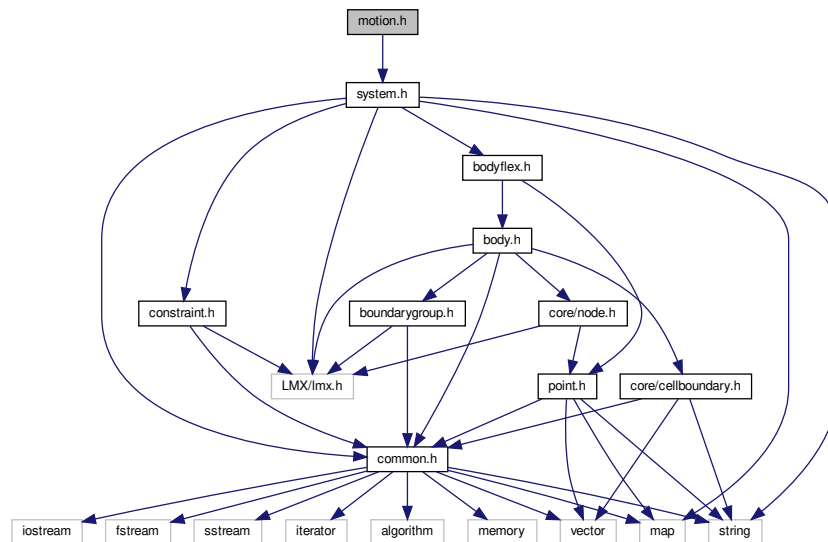
Namespaces

- `mknix`

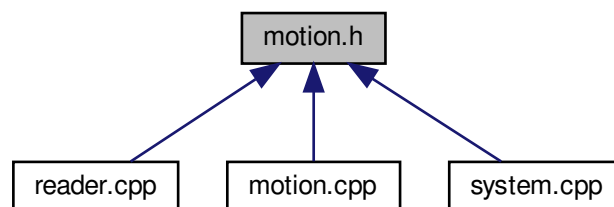
9.101 motion.h File Reference

```
#include "system.h"
```

Include dependency graph for motion.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::Motion](#)

Namespaces

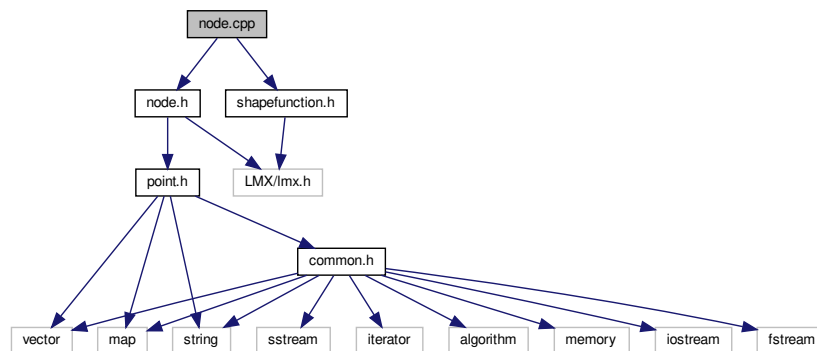
- [mknix](#)

9.102 node.cpp File Reference

```
#include "node.h"
```

```
#include "shapefunction.h"
```

Include dependency graph for node.cpp:



Namespaces

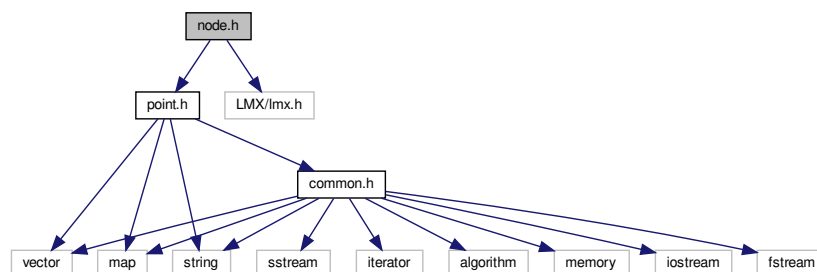
- [mknix](#)

9.103 node.h File Reference

```
#include "point.h"
```

```
#include "LMX/lmx.h"
```

Include dependency graph for node.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::Node](#)

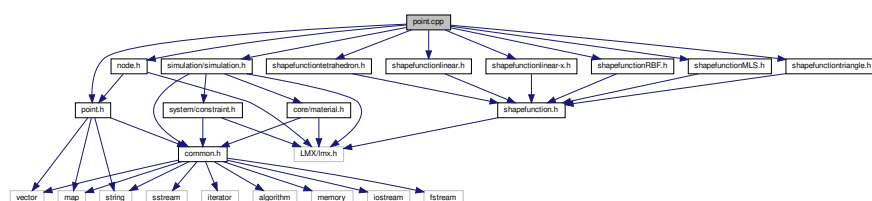
Namespaces

- [mknix](#)

9.104 point.cpp File Reference

```
#include "point.h"
#include "node.h"
#include "shapefunctionRBF.h"
#include "shapefunctionMLS.h"
#include "shapefunctiontriangle.h"
#include "shapefunctiontetrahedron.h"
#include "shapefunctionlinear.h"
#include "shapefunctionlinear-x.h"
#include <simulation/simulation.h>
```

Include dependency graph for point.cpp:



Namespaces

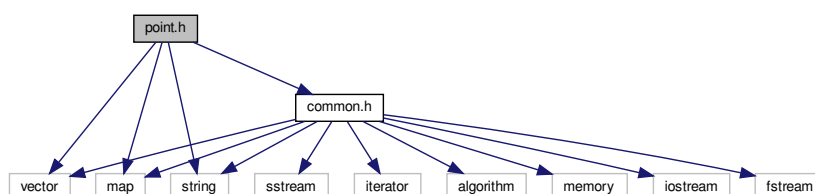
- [mknix](#)

9.105 point.h File Reference

Point of interest.

```
#include <vector>
#include <map>
#include <string>
#include "common.h"
```

Include dependency graph for point.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::Point](#)

Namespaces

- [mknix](#)

9.105.1 Detailed Description

Point of interest.

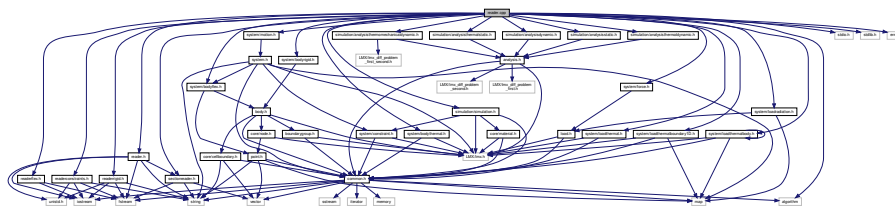
Author

Daniel Iglesias

9.106 reader.cpp File Reference

```
#include "reader.h"
#include "readerrigid.h"
#include "readerflex.h"
#include "readerconstraints.h"
#include "sectionreader.h"
#include <simulation/simulation.h>
#include <simulation/analysisdynamic.h>
#include <simulation/analysisstatic.h>
#include <simulation/analysisthermaldynamic.h>
#include <simulation/analysisthermalstatic.h>
#include <simulation/analysisthermomechanicaldynamic.h>
#include <system/bodyflex.h>
#include <system/bodyrigid.h>
#include <system/bodythermal.h>
#include <system/force.h>
#include <system/loadradiation.h>
#include <system/loadthermal.h>
#include <system/motion.h>
#include <system/loadthermalbody.h>
#include <system/loadthermalboundary1D.h>
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <algorithm>
```

Include dependency graph for reader.cpp:



Namespaces

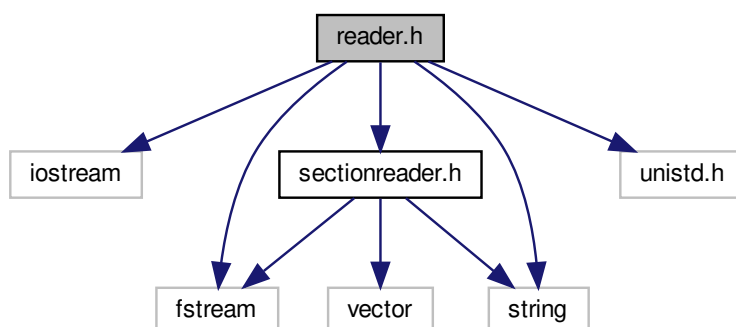
- [mknix](#)

9.107 reader.h File Reference

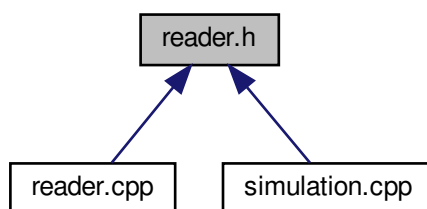
```
#include <iostream>
#include <fstream>
#include <string>
#include "sectionreader.h"
```

```
#include <unistd.h>
```

Include dependency graph for reader.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::Reader](#)

Namespaces

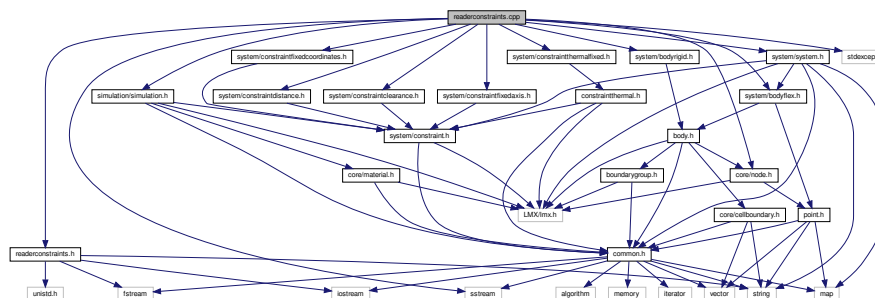
- [mnix](#)

9.108 readerconstraints.cpp File Reference

```
#include "readerconstraints.h"
#include <core/node.h>
#include <simulation/simulation.h>
#include <system/bodyflex.h>
#include <system/bodyrigid.h>
#include <system/constraintclearance.h>
#include <system/constraintdistance.h>
#include <system/constraintfixedaxis.h>
#include <system/constraintfixedcoordinates.h>
#include <system/constraintthermalfixed.h>
```

```
#include <system/system.h>
#include <sstream>
#include <stdexcept>
```

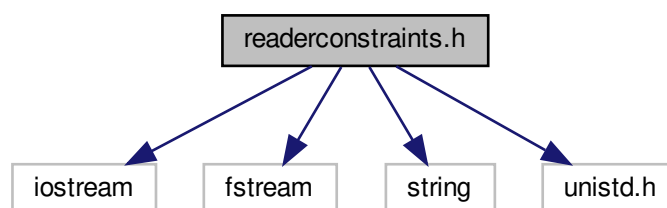
Include dependency graph for readerconstraints.cpp:



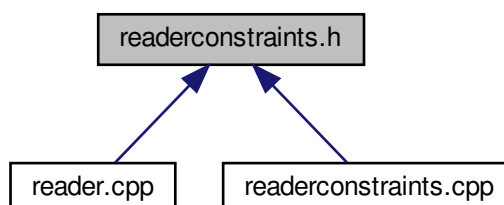
9.109 readerconstraints.h File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include <unistd.h>
```

Include dependency graph for readerconstraints.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mknix::ReaderConstraints`

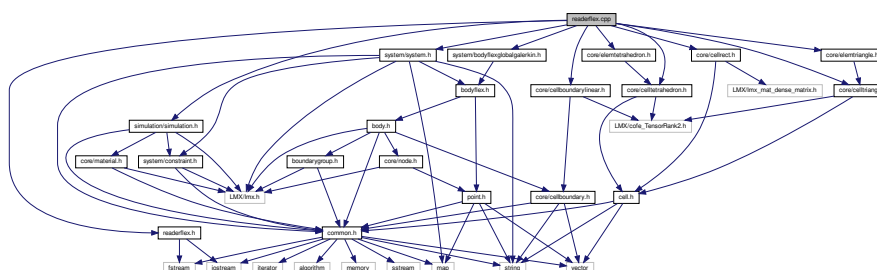
Namespaces

- `mknix`

9.110 readerflex.cpp File Reference

```
#include "readerflex.h"
#include <simulation/simulation.h>
#include <system/bodyflexglobalgalerkin.h>
#include <core/cellrect.h>
#include <core/celltriang.h>
#include <core/celltetrahedron.h>
#include <core/cellboundarylinear.h>
#include <core/elemtriangle.h>
#include <core/elemtetrahedron.h>
#include <system/system.h>
```

Include dependency graph for readerflex.cpp:



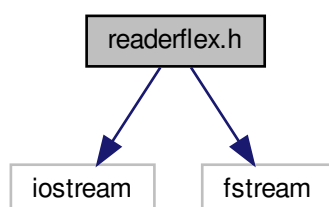
Namespaces

- `mknix`

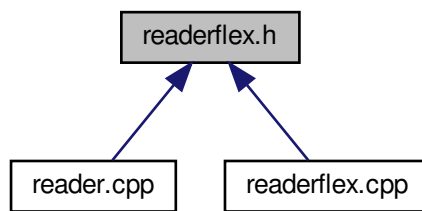
9.111 readerflex.h File Reference

```
#include <iostream>
#include <fstream>
```

Include dependency graph for readerflex.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ReaderFlex](#)

Namespaces

- [mknix](#)

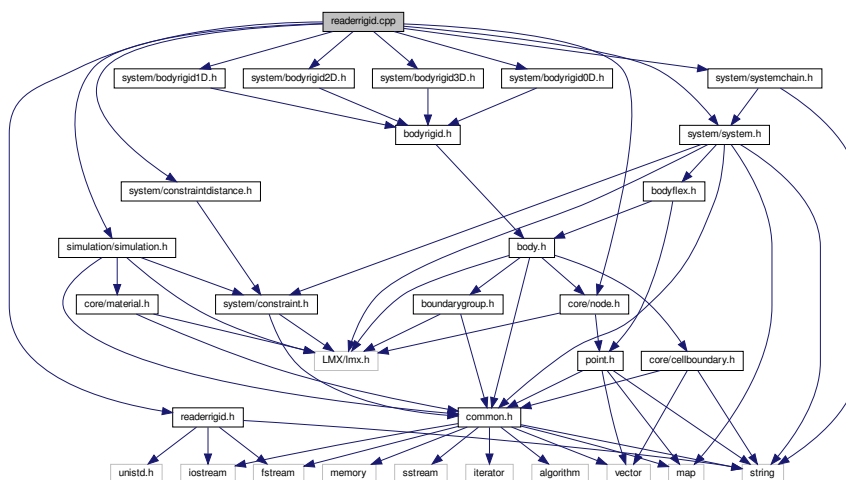
9.112 readerrigid.cpp File Reference

```

#include "readerrigid.h"
#include <core/node.h>
#include <simulation/simulation.h>
#include <system/bodyrigid0D.h>
#include <system/bodyrigid1D.h>
#include <system/bodyrigid2D.h>
#include <system/bodyrigid3D.h>
#include <system/constraintdistance.h>
#include <system/system.h>
#include <system/systemchain.h>

```

Include dependency graph for readerrigid.cpp:



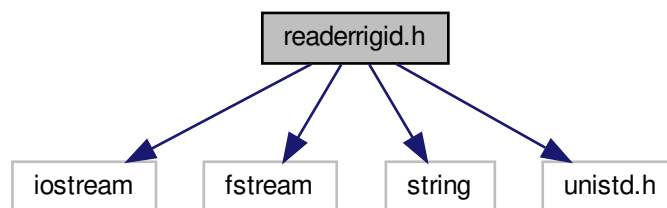
Namespaces

- [mknix](#)

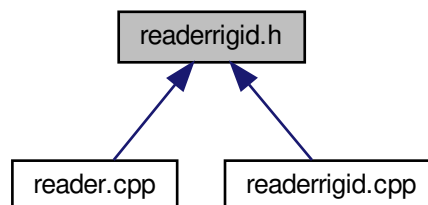
9.113 readerrigid.h File Reference

```
#include <iostream>
#include <fstream>
#include <string>
#include <unistd.h>
```

Include dependency graph for readerrigid.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ReaderRigid](#)

Namespaces

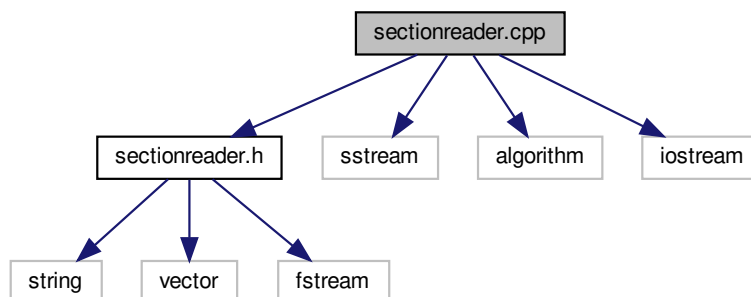
- [mknix](#)

9.114 sectionreader.cpp File Reference

```
#include "sectionreader.h"
#include <sstream>
#include <algorithm>
```

```
#include <iostream>
```

Include dependency graph for sectionreader.cpp:



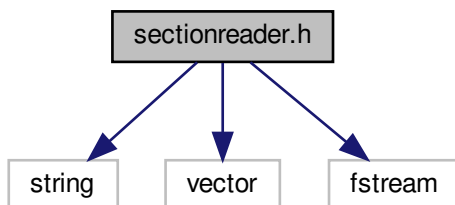
9.115 sectionreader.h File Reference

```
#include <string>
```

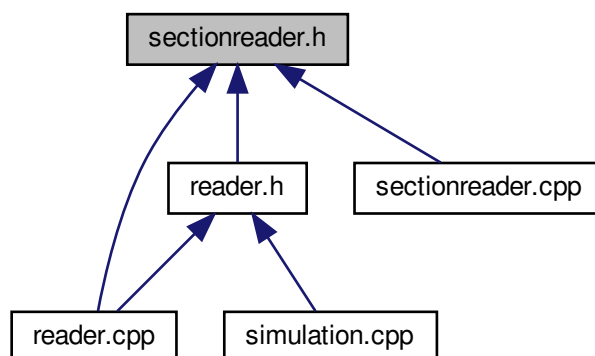
```
#include <vector>
```

```
#include <fstream>
```

Include dependency graph for sectionreader.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::SectionReader](#)

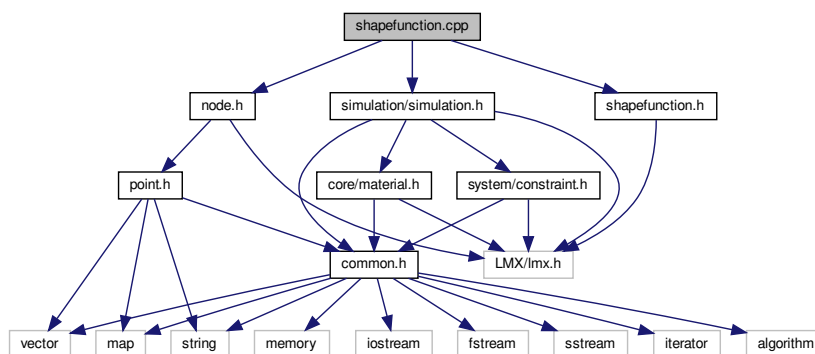
Namespaces

- [mknix](#)

9.116 shapefunction.cpp File Reference

```
#include "shapefunction.h"
#include "node.h"
#include <simulation/simulation.h>
```

Include dependency graph for `shapefunction.cpp`:



Namespaces

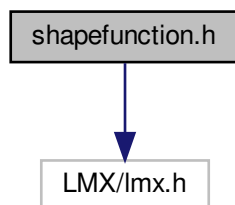
- [mknix](#)

9.117 shapefunction.h File Reference

Function for interpolation of basic variables.

```
#include "LMX/lmx.h"
```

Include dependency graph for shapefunction.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::ShapeFunction](#)

Namespaces

- [mnix](#)

9.117.1 Detailed Description

Function for interpolation of basic variables.

Author

Daniel Iglesias

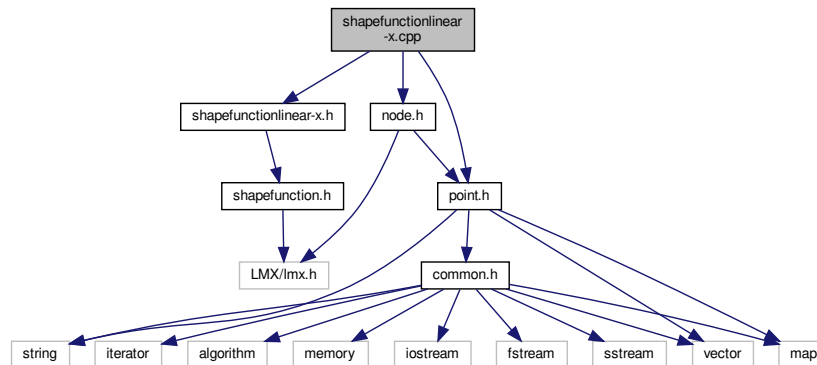
9.118 shapefunctionlinear-x.cpp File Reference

```
#include "shapefunctionlinear-x.h"
```

```
#include "point.h"
```

```
#include "node.h"
```

Include dependency graph for shapefunctionlinear-x.cpp:



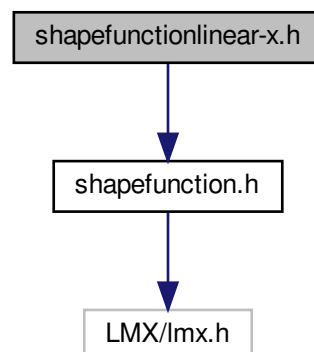
Namespaces

- [mknix](#)

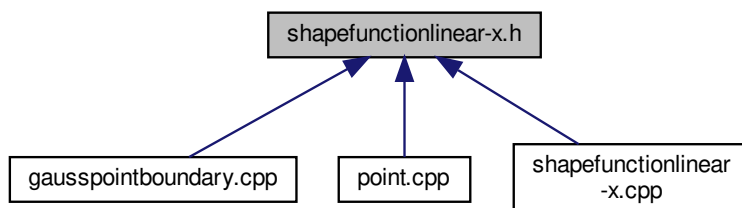
9.119 shapefunctionlinear-x.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionlinear-x.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::ShapeFunctionLinearX](#)

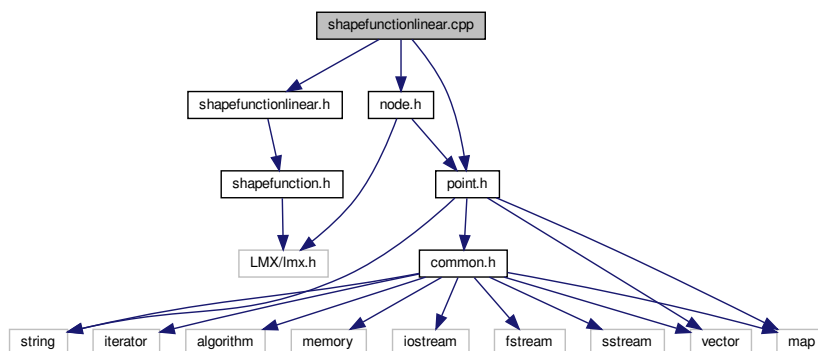
Namespaces

- [mnix](#)

9.120 shapefunctionlinear.cpp File Reference

```
#include "shapefunctionlinear.h"
#include "point.h"
#include "node.h"
```

Include dependency graph for `shapefunctionlinear.cpp`:



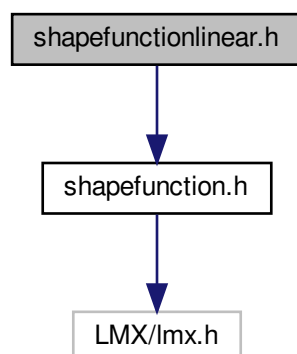
Namespaces

- [mnix](#)

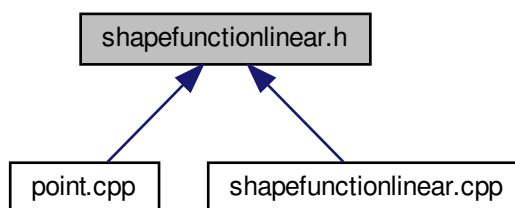
9.121 shapefunctionlinear.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionlinear.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ShapeFunctionLinear](#)

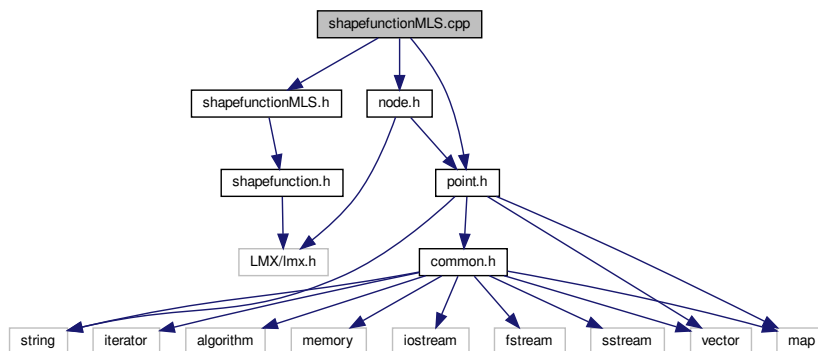
Namespaces

- [mknix](#)

9.122 shapefunctionMLS.cpp File Reference

```
#include "shapefunctionMLS.h"  
#include "point.h"  
#include "node.h"
```


Include dependency graph for shapefunctionMLS.cpp:



Namespaces

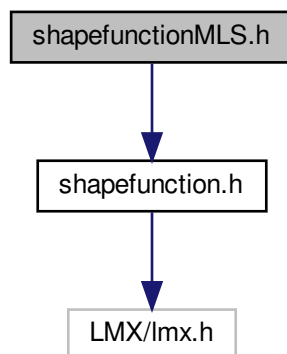
- [mknix](#)

9.123 shapefunctionMLS.h File Reference

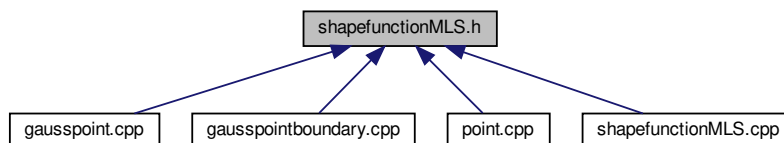
Function for aproximation of basic variables by Moving Least Squares fit.

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionMLS.h:



This graph shows which files directly or indirectly include this file:



Classes

- class `mnix::ShapeFunctionMLS`

Namespaces

- `mnix`

9.123.1 Detailed Description

Function for aproximation of basic variables by Moving Least Squares fit.

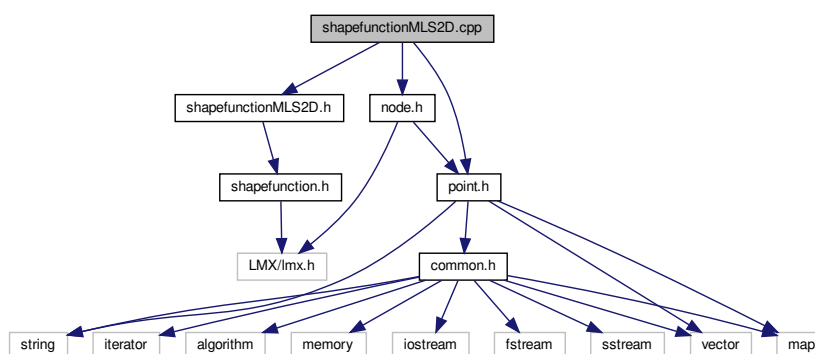
Author

Daniel Iglesias

9.124 shapefunctionMLS2D.cpp File Reference

```
#include "shapefunctionMLS2D.h"
#include "point.h"
#include "node.h"
```

Include dependency graph for `shapefunctionMLS2D.cpp`:



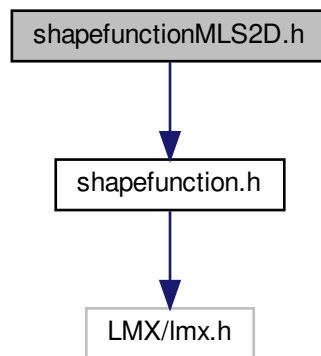
Namespaces

- `mnix`

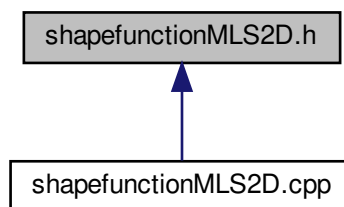
9.125 shapefunctionMLS2D.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionMLS2D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::ShapeFunctionMLS2D](#)

Namespaces

- [mnix](#)

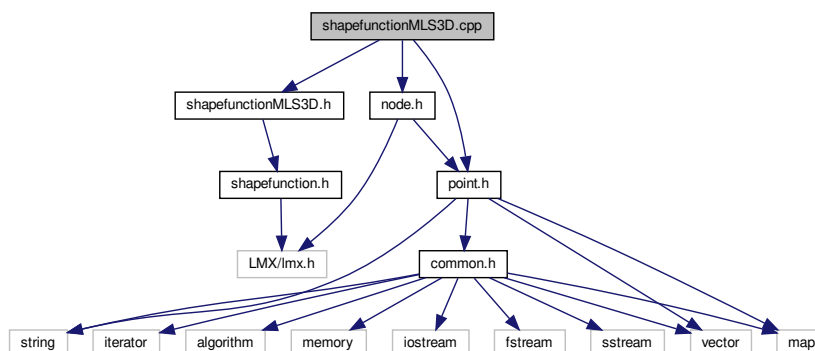
9.126 shapefunctionMLS3D.cpp File Reference

```
#include "shapefunctionMLS3D.h"
```

```
#include "point.h"
```

```
#include "node.h"
```

Include dependency graph for shapefunctionMLS3D.cpp:



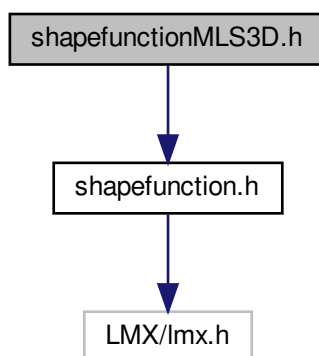
Namespaces

- [mknix](#)

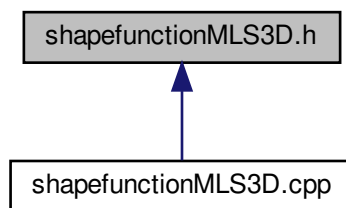
9.127 shapefunctionMLS3D.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionMLS3D.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ShapeFunctionMLS3D](#)

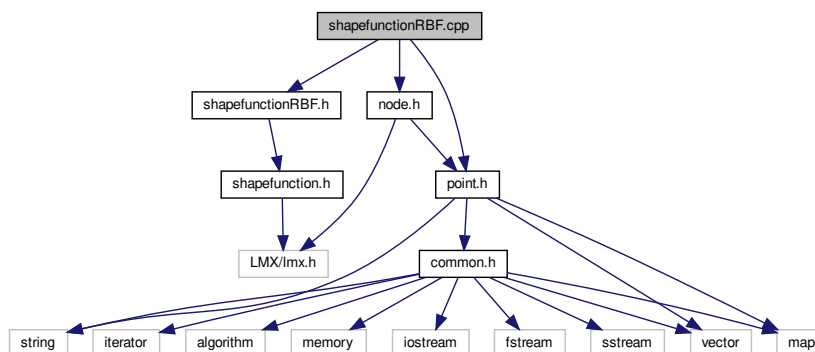
Namespaces

- [mknix](#)

9.128 shapefunctionRBF.cpp File Reference

```
#include "shapefunctionRBF.h"
#include "point.h"
#include "node.h"
```

Include dependency graph for `shapefunctionRBF.cpp`:



Namespaces

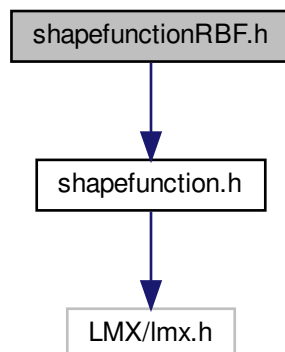
- [mknix](#)

9.129 shapefunctionRBF.h File Reference

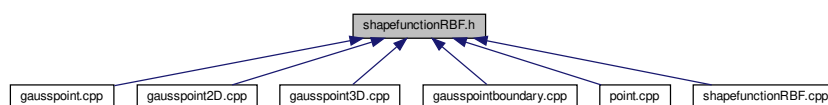
Function for interpolation of basic variables by means of Radial basis Functions.

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctionRBF.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ShapeFunctionRBF](#)

Namespaces

- [mknix](#)

9.129.1 Detailed Description

Function for interpolation of basic variables by means of Radial basis Functions.

Author

Daniel Iglesias

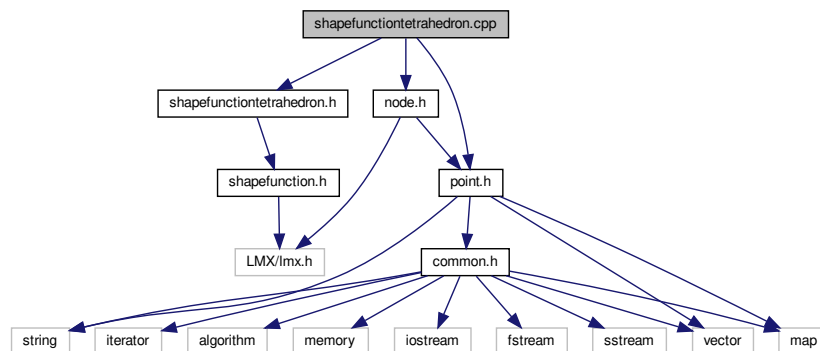
9.130 shapefunctiontetrahedron.cpp File Reference

```
#include "shapefunctiontetrahedron.h"
```

```
#include "point.h"
```

```
#include "node.h"
```

Include dependency graph for shapefunctiontetrahedron.cpp:



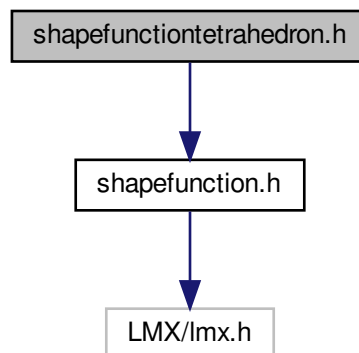
Namespaces

- [mknix](#)

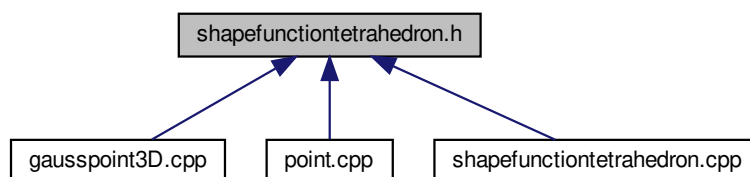
9.131 shapefunctiontetrahedron.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctiontetrahedron.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ShapeFunctionTetrahedron](#)

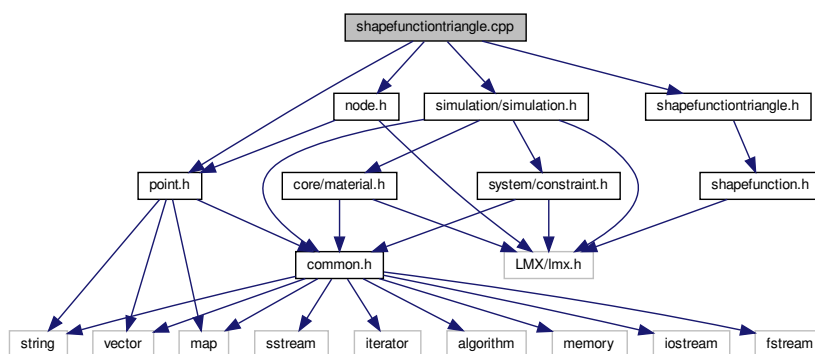
Namespaces

- [mknix](#)

9.132 shapefunctiontriangle.cpp File Reference

```
#include "shapefunctiontriangle.h"
#include "point.h"
#include "node.h"
#include <simulation/simulation.h>
```

Include dependency graph for `shapefunctiontriangle.cpp`:



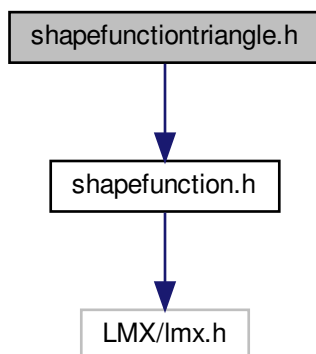
Namespaces

- [mknix](#)

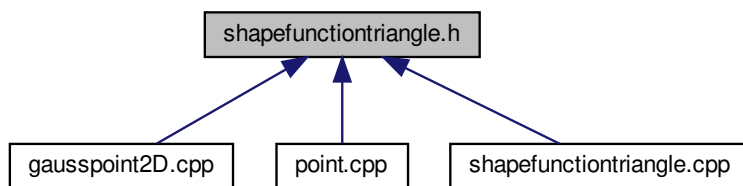
9.133 shapefunctiontriangle.h File Reference

```
#include "shapefunction.h"
```


Include dependency graph for shapefunctiontriangle.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::ShapeFunctionTriangle](#)

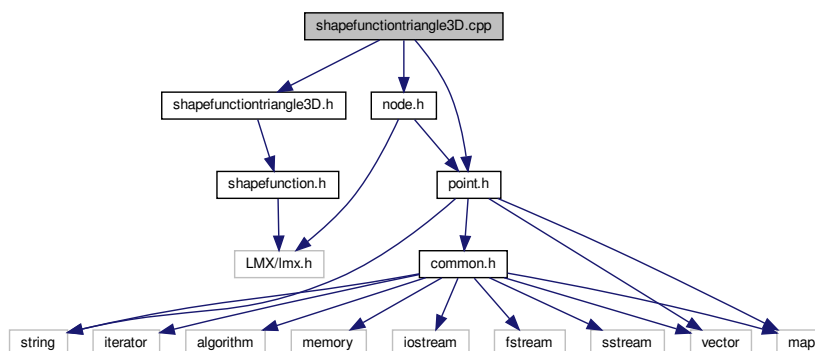
Namespaces

- [mknix](#)

9.134 shapefunctiontriangle3D.cpp File Reference

```
#include "shapefunctiontriangle3D.h"  
#include "point.h"  
#include "node.h"
```

Include dependency graph for shapefunctiontriangle3D.cpp:



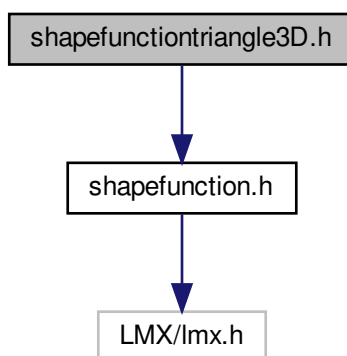
Namespaces

- [mknix](#)

9.135 shapefunctiontriangle3D.h File Reference

```
#include "shapefunction.h"
```

Include dependency graph for shapefunctiontriangle3D.h:





- ## Namespaces

- ### 9.136 simulation.cpp File Reference

Include dependency graph for simulation.cpp:



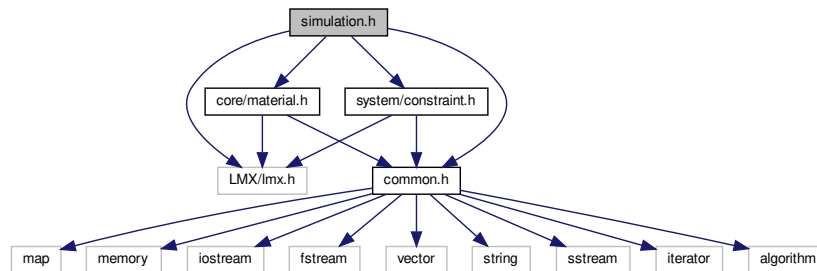
- ## Functions

- Generated by Doxygen

9.137 simulation.h File Reference

```
#include "LMX/lmx.h"
#include "common.h"
#include <core/material.h>
#include <system/constraint.h>
```

Include dependency graph for simulation.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mknix::Simulation](#)

Namespaces

- [mknix](#)

Macros

- #define [MKNIX_API](#)

9.137.1 Macro Definition Documentation

9.137.1.1 MKNIX_API #define MKNIX_API

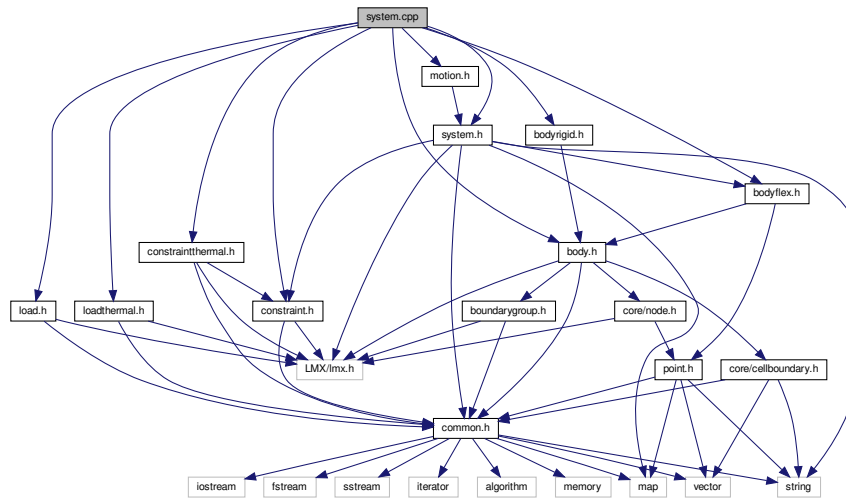
Definition at line 36 of file simulation.h.

9.138 system.cpp File Reference

```
#include "system.h"
#include "body.h"
#include "bodyflex.h"
#include "bodyrigid.h"
#include "constraint.h"
#include "constraintthermal.h"
#include "load.h"
#include "loadthermal.h"
```

```
#include "motion.h"
```

Include dependency graph for system.cpp:



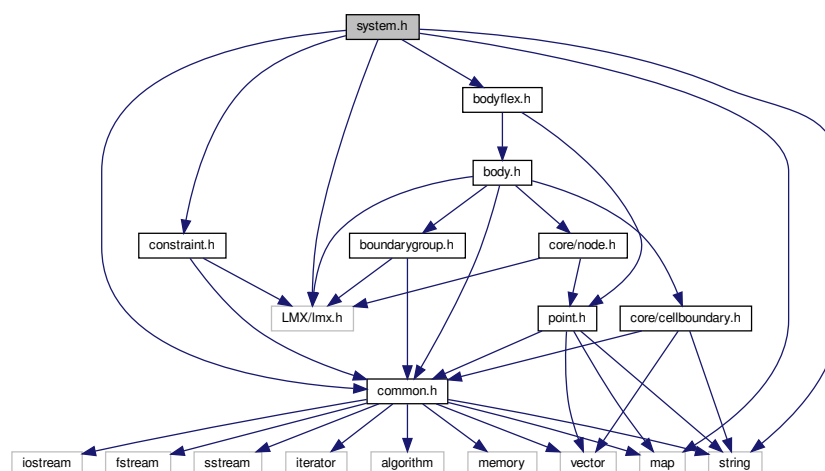
Namespaces

- [mknix](#)

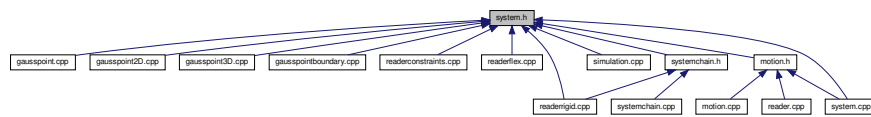
9.139 system.h File Reference

```
#include <map>
#include <string>
#include "LMX/lmx.h"
#include "common.h"
#include "constraint.h"
#include "bodyflex.h"
```

Include dependency graph for system.h:



This graph shows which files directly or indirectly include this file:



Classes

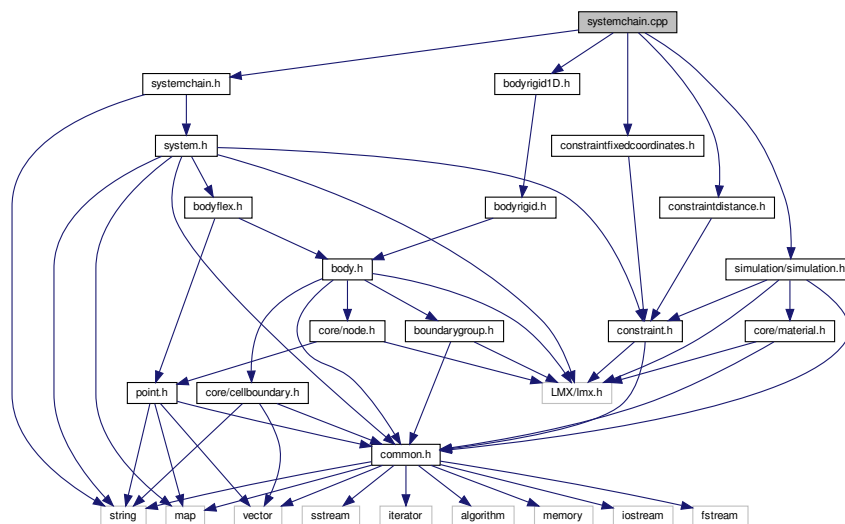
- class [mknix::System](#)

Namespaces

- [mknix](#)

9.140 systemchain.cpp File Reference

```
#include "systemchain.h"
#include "bodyrigid1D.h"
#include "constraintdistance.h"
#include "constraintfixedcoordinates.h"
#include <simulation/simulation.h>
Include dependency graph for systemchain.cpp:
```



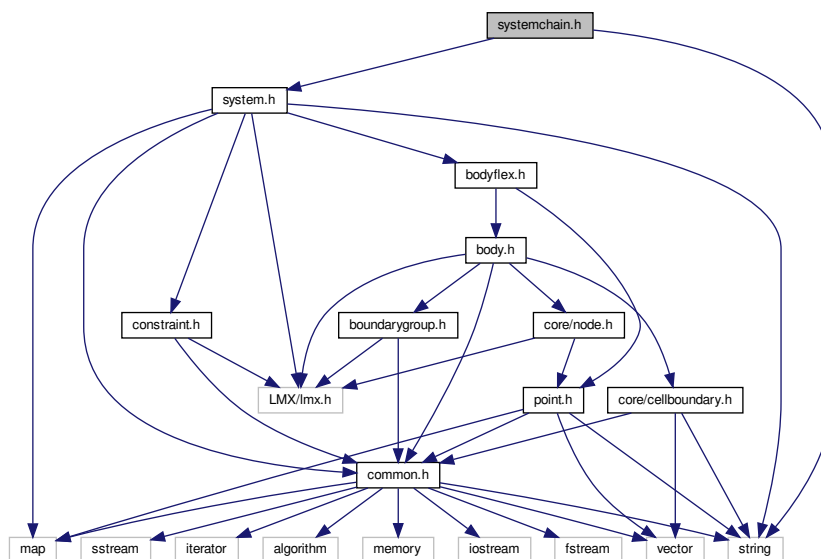
Namespaces

- [mknix](#)

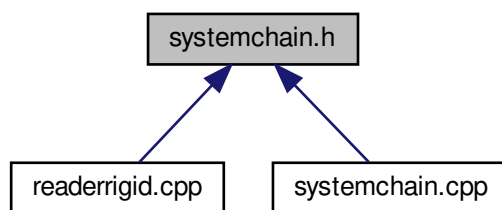
9.141 systemchain.h File Reference

```
#include "system.h"
#include <string>
```

Include dependency graph for systemchain.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [mnix::SystemChain](#)

Namespaces

- [mnix](#)

Index

- ~Analysis
 - mknix::Analysis, [29](#)
- ~AnalysisDynamic
 - mknix::AnalysisDynamic, [33](#)
- ~AnalysisStatic
 - mknix::AnalysisStatic, [36](#)
- ~AnalysisThermalDynamic
 - mknix::AnalysisThermalDynamic, [38](#)
- ~AnalysisThermalStatic
 - mknix::AnalysisThermalStatic, [41](#)
- ~AnalysisThermoMechanicalDynamic
 - mknix::AnalysisThermoMechanicalDynamic, [43](#)
- ~Body
 - mknix::Body, [47](#)
- ~BoundaryGroup
 - mknix::BoundaryGroup, [58](#)
- ~Cell
 - mknix::Cell, [63](#)
- ~CellBoundary
 - mknix::CellBoundary, [73](#)
- ~CellBoundaryLinear
 - mknix::CellBoundaryLinear, [78](#)
- ~CellRect
 - mknix::CellRect, [82](#)
- ~CellTetrahedron
 - mknix::CellTetrahedron, [85](#)
- ~CellTriang
 - mknix::CellTriang, [90](#)
- ~CompBar
 - mknix::CompBar, [91](#)
- ~Constraint
 - mknix::Constraint, [94](#)
- ~ConstraintClearance
 - mknix::ConstraintClearance, [102](#)
- ~ConstraintContact
 - mknix::ConstraintContact, [105](#)
- ~ConstraintDistance
 - mknix::ConstraintDistance, [108](#)
- ~ConstraintFixedAxis
 - mknix::ConstraintFixedAxis, [111](#)
- ~ConstraintFixedCoordinates
 - mknix::ConstraintFixedCoordinates, [115](#)
- ~ConstraintThermal
 - mknix::ConstraintThermal, [117](#)
- ~ConstraintThermalFixed
 - mknix::ConstraintThermalFixed, [120](#)
- ~Contact
 - mknix::Contact, [122](#)
- ~ElemTetrahedron
 - mknix::ElemTetrahedron, [124](#)
- ~ElemTriangle
 - mknix::ElemTriangle, [127](#)
- ~FlexBody
 - mknix::FlexBody, [131](#)
- ~FlexFrameGalerkin
 - mknix::FlexFrameGalerkin, [138](#)
- ~FlexGlobalGalerkin
 - mknix::FlexGlobalGalerkin, [144](#)
- ~Force
 - mknix::Force, [149](#)
- ~GaussPoint
 - mknix::GaussPoint, [153](#)
- ~GaussPoint2D
 - mknix::GaussPoint2D, [162](#)
- ~GaussPoint3D
 - mknix::GaussPoint3D, [172](#)
- ~GaussPointBoundary
 - mknix::GaussPointBoundary, [181](#)
- ~Load
 - mknix::Load, [184](#)
- ~LoadThermal
 - mknix::LoadThermal, [186](#)
- ~LoadThermalBody
 - mknix::LoadThermalBody, [188](#)
- ~LoadThermalBoundary1D
 - mknix::LoadThermalBoundary1D, [191](#)
- ~Material
 - mknix::Material, [195](#)
- ~MknixWrapper
 - mknix::MknixWrapper, [207](#)
- ~Motion
 - mknix::Motion, [209](#)
- ~Node
 - mknix::Node, [213](#)
- ~Point
 - mknix::Point, [225](#)
- ~Radiation
 - mknix::Radiation, [237](#)
- ~Reader
 - mknix::Reader, [239](#)
- ~ReaderConstraints
 - mknix::ReaderConstraints, [240](#)
- ~ReaderFlex
 - mknix::ReaderFlex, [242](#)
- ~ReaderRigid
 - mknix::ReaderRigid, [244](#)
- ~RigidBar
 - mknix::RigidBar, [247](#)
- ~RigidBody
 - mknix::RigidBody, [251](#)
- ~RigidBody2D
 - mknix::RigidBody2D, [258](#)
- ~RigidBody3D
 - mknix::RigidBody3D, [263](#)
- ~RigidBodyMassPoint
 - mknix::RigidBodyMassPoint, [266](#)
- ~ShapeFunction
 - mknix::ShapeFunction, [272](#)
- ~ShapeFunctionLinear
 - mknix::ShapeFunctionLinear, [276](#)

- ~ShapeFunctionLinearX
 - mknix::ShapeFunctionLinearX, 278
- ~ShapeFunctionMLS
 - mknix::ShapeFunctionMLS, 280
- ~ShapeFunctionMLS2D
 - mknix::ShapeFunctionMLS2D, 282
- ~ShapeFunctionMLS3D
 - mknix::ShapeFunctionMLS3D, 284
- ~ShapeFunctionRBF
 - mknix::ShapeFunctionRBF, 286
- ~ShapeFunctionTetrahedron
 - mknix::ShapeFunctionTetrahedron, 289
- ~ShapeFunctionTriangle
 - mknix::ShapeFunctionTriangle, 291
- ~ShapeFunctionTriangleSigned
 - mknix::ShapeFunctionTriangleSigned, 293
- ~Simulation
 - mknix::Simulation, 297
- ~System
 - mknix::System, 329
- ~SystemChain
 - mknix::SystemChain, 340
- ~ThermalBody
 - mknix::ThermalBody, 343
- addBodyPoint
 - mknix::FlexBody, 131
- addBoundaryConnectivity
 - mknix::Body, 47
- addBoundaryGroup
 - mknix::Body, 48
- addBoundaryLine
 - mknix::Body, 48
- addCell
 - mknix::Body, 48
 - mknix::BoundaryGroup, 58
 - mknix::ThermalBody, 343
- addCellToBoundaryGroup
 - mknix::Body, 48
- addField
 - mknix::SectionReader, 268
- addLoad
 - mknix::LoadThermalBody, 188, 189
- addLoadThermal
 - mknix::Body, 48
 - mknix::ThermalBody, 344
- addNode
 - mknix::Body, 48
 - mknix::BoundaryGroup, 58
 - mknix::RigidBar, 247
 - mknix::RigidBody2D, 258
 - mknix::RigidBody3D, 263
 - mknix::ThermalBody, 344
- addNodes
 - mknix::Body, 49
- addNodeToBoundaryGroup
 - mknix::Body, 49
- addPoint
 - mknix::FlexBody, 131, 132
- addSubSection
 - mknix::SectionReader, 268
- addSupportNode
 - mknix::Point, 225
- addThermalCapacity
 - mknix::Material, 195
- addThermalConductivity
 - mknix::Material, 195
- addThermalDensity
 - mknix::Material, 195
- addThermalExpansion
 - mknix::Material, 195
- addTimeLenght
 - mknix::SystemChain, 340
- addToRender
 - mknix::CompBar, 91
- addVariableResistance
 - mknix::Material, 195
- addVoxel
 - mknix::Radiation, 238
- addWeight
 - mknix::Node, 213
- alpha
 - mknix::Cell, 70
 - mknix::CellBoundary, 74
 - mknix::Constraint, 99
- alphaI
 - mknix::Point, 235
- Analysis
 - mknix::Analysis, 29
- analysis.cpp, 347
- analysis.h, 348
- AnalysisDynamic
 - mknix::AnalysisDynamic, 32
- analysisdynamic.cpp, 349
- analysisdynamic.h, 349
- AnalysisStatic
 - mknix::AnalysisStatic, 35
- analysisstatic.cpp, 350
- analysisstatic.h, 350
- AnalysisThermalDynamic
 - mknix::AnalysisThermalDynamic, 38
- analysisthermaldynamic.cpp, 351
- analysisthermaldynamic.h, 352
- AnalysisThermalStatic
 - mknix::AnalysisThermalStatic, 40
- analysisthermalstatic.cpp, 353
- analysisthermalstatic.h, 353
- AnalysisThermoMechanicalDynamic
 - mknix::AnalysisThermoMechanicalDynamic, 43
- analysisthermomechanicaldynamic.cpp, 354
- analysisthermomechanicaldynamic.h, 354
- assembleCapacityGaussPoints
 - mknix::Cell, 63
- assembleCapacityMatrix
 - mknix::Body, 49
 - mknix::System, 329
 - mknix::ThermalBody, 344

- assembleCij
 - mknix::GaussPoint, [153](#)
- assembleConductivityGaussPoints
 - mknix::Cell, [65](#)
- assembleConductivityMatrix
 - mknix::Body, [49](#)
 - mknix::System, [329](#)
 - mknix::ThermalBody, [344](#)
- assembleConstraintForces
 - mknix::System, [329](#)
- assembleExternalForces
 - mknix::Body, [50](#)
 - mknix::FlexFrameGalerkin, [139](#)
 - mknix::FlexGlobalGalerkin, [144](#)
 - mknix::Load, [184](#)
 - mknix::RigidBody, [251](#)
 - mknix::System, [329](#)
- assembleExternalHeat
 - mknix::Body, [50](#)
 - mknix::BoundaryGroup, [58](#)
 - mknix::LoadThermal, [186](#)
 - mknix::System, [330](#)
 - mknix::ThermalBody, [344](#)
- assembleFext
 - mknix::GaussPoint, [153](#)
 - mknix::GaussPoint2D, [162](#)
 - mknix::GaussPoint3D, [172](#)
- assembleFextGaussPoints
 - mknix::Cell, [65](#)
- assembleFint
 - mknix::GaussPoint, [153](#)
 - mknix::GaussPoint2D, [162](#)
 - mknix::GaussPoint3D, [172](#)
- assembleFintGaussPoints
 - mknix::Cell, [65](#)
- assembleHij
 - mknix::GaussPoint, [153](#)
- assembleInternalForces
 - mknix::Constraint, [94](#)
 - mknix::ConstraintThermal, [117](#)
 - mknix::FlexBody, [132](#)
 - mknix::FlexFrameGalerkin, [139](#)
 - mknix::FlexGlobalGalerkin, [145](#)
 - mknix::System, [330](#)
- assembleInternalHeat
 - mknix::System, [330](#)
- assembleKGaussPoints
 - mknix::Cell, [65](#)
- assembleKij
 - mknix::GaussPoint, [154](#)
 - mknix::GaussPoint2D, [162](#)
 - mknix::GaussPoint3D, [172](#)
- assembleMassMatrix
 - mknix::Body, [50](#)
 - mknix::FlexFrameGalerkin, [139](#)
 - mknix::FlexGlobalGalerkin, [145](#)
 - mknix::RigidBody, [252](#)
 - mknix::System, [330](#)
- assembleMGaussPoints
 - mknix::Cell, [65](#)
- assembleMij
 - mknix::GaussPoint, [154](#)
 - mknix::GaussPoint2D, [162](#)
 - mknix::GaussPoint3D, [172](#)
- assembleNLRGaussPoints
 - mknix::Cell, [66](#)
- assembleQext
 - mknix::GaussPoint, [154](#)
 - mknix::GaussPointBoundary, [181](#)
- assembleQextGaussPoints
 - mknix::Cell, [66](#)
 - mknix::CellBoundary, [73](#)
- assembleRGaussPoints
 - mknix::Cell, [66](#)
- assembleRi
 - mknix::GaussPoint, [154](#)
 - mknix::GaussPoint2D, [163](#)
 - mknix::GaussPoint3D, [173](#)
- assembleTangentMatrix
 - mknix::Constraint, [95](#)
 - mknix::ConstraintThermal, [118](#)
 - mknix::FlexBody, [132](#)
 - mknix::FlexFrameGalerkin, [140](#)
 - mknix::FlexGlobalGalerkin, [145](#)
 - mknix::System, [330](#)
- assembleThermalTangentMatrix
 - mknix::System, [331](#)
- augmentedTolerance
 - mknix::Constraint, [99](#)
- avgTemp
 - mknix::GaussPoint, [157](#)
- axisName
 - mknix::ConstraintFixedAxis, [112](#)
- B
 - mknix::GaussPoint, [157](#)
- Body
 - mknix::Body, [47](#)
- body.cpp, [355](#)
- body.h, [356](#)
- bodyflex.cpp, [357](#)
- bodyflex.h, [357](#)
- bodyflexframegalerkin.cpp, [358](#)
- bodyflexframegalerkin.h, [359](#)
- bodyflexglobalgalerkin.cpp, [360](#)
- bodyflexglobalgalerkin.h, [361](#)
- bodyNames
 - mknix::Simulation, [297](#)
- bodyPoints
 - mknix::Cell, [71](#)
 - mknix::CellBoundary, [74](#)
 - mknix::FlexBody, [136](#)
 - mknix::Simulation, [298](#)
- bodyrigid.cpp, [361](#)
- bodyrigid.h, [362](#)
- bodyrigid0D.cpp, [363](#)
- bodyrigid0D.h, [363](#)

- bodyrigid1D.cpp, 364
- bodyrigid1D.h, 365
- bodyrigid2D.cpp, 366
- bodyrigid2D.h, 367
- bodyrigid3D.cpp, 367
- bodyrigid3D.h, 368
- bodythermal.cpp, 369
- bodythermal.h, 369
- bondedBodyNodes
 - mknix::Body, 56
- boundaryConnectivity
 - mknix::Body, 56
- BoundaryGroup
 - mknix::BoundaryGroup, 58
- boundarygroup.cpp, 370
- boundarygroup.h, 371
- boundaryGroups
 - mknix::Body, 56
- boxFIR
 - mknix::boxFIR, 60
- C
 - mknix::GaussPoint, 157
- calc
 - mknix::ShapeFunction, 272
 - mknix::ShapeFunctionLinear, 276
 - mknix::ShapeFunctionLinearX, 278
 - mknix::ShapeFunctionMLS, 280
 - mknix::ShapeFunctionMLS2D, 282
 - mknix::ShapeFunctionMLS3D, 284
 - mknix::ShapeFunctionRBF, 286
 - mknix::ShapeFunctionTetrahedron, 289
 - mknix::ShapeFunctionTriangle, 291
 - mknix::ShapeFunctionTriangleSigned, 293
- calcCapacityMatrix
 - mknix::Body, 50
 - mknix::System, 331
 - mknix::ThermalBody, 344
- calcConductivityMatrix
 - mknix::Body, 50
 - mknix::System, 331
 - mknix::ThermalBody, 345
- calcElasticE
 - mknix::GaussPoint, 154
 - mknix::GaussPoint2D, 163
 - mknix::GaussPoint3D, 173
- calcElasticEGaussPoints
 - mknix::Cell, 66
- calcExternalForces
 - mknix::Body, 50
 - mknix::FlexFrameGalerkin, 140
 - mknix::FlexGlobalGalerkin, 145
 - mknix::RigidBar, 247
 - mknix::RigidBody2D, 259
 - mknix::RigidBody3D, 263
 - mknix::RigidBodyMassPoint, 266
 - mknix::System, 331
- calcExternalHeat
 - mknix::Body, 50
- mknix::BoundaryGroup, 59
- mknix::System, 331
- mknix::ThermalBody, 345
- calcInternalForces
 - mknix::Constraint, 95
 - mknix::FlexBody, 132
 - mknix::FlexFrameGalerkin, 140
 - mknix::FlexGlobalGalerkin, 146
 - mknix::System, 331
- calcInternalHeat
 - mknix::System, 331
- calcKineticE
 - mknix::GaussPoint, 154
 - mknix::GaussPoint2D, 163
 - mknix::GaussPoint3D, 173
- calcKineticEGaussPoints
 - mknix::Cell, 66
- calcMassMatrix
 - mknix::Body, 50
 - mknix::FlexFrameGalerkin, 140
 - mknix::FlexGlobalGalerkin, 146
 - mknix::RigidBar, 248
 - mknix::RigidBody2D, 259
 - mknix::RigidBody3D, 264
 - mknix::RigidBodyMassPoint, 267
 - mknix::System, 331
- calcPhi
 - mknix::Constraint, 96
 - mknix::ConstraintClearance, 102
 - mknix::ConstraintContact, 105
 - mknix::ConstraintDistance, 108
 - mknix::ConstraintFixedAxis, 112
 - mknix::ConstraintFixedCoordinates, 115
 - mknix::ConstraintThermalFixed, 120
- calcPhiq
 - mknix::Constraint, 96
 - mknix::ConstraintClearance, 102
 - mknix::ConstraintContact, 105
 - mknix::ConstraintDistance, 108
 - mknix::ConstraintFixedAxis, 112
 - mknix::ConstraintFixedCoordinates, 115
 - mknix::ConstraintThermalFixed, 120
- calcPhiqq
 - mknix::Constraint, 96
 - mknix::ConstraintClearance, 102
 - mknix::ConstraintContact, 105
 - mknix::ConstraintDistance, 109
 - mknix::ConstraintFixedAxis, 112
 - mknix::ConstraintFixedCoordinates, 115
 - mknix::ConstraintThermalFixed, 121
- calcPotentialE
 - mknix::GaussPoint, 154
 - mknix::GaussPoint2D, 164
 - mknix::GaussPoint3D, 174
- calcPotentialEGaussPoints
 - mknix::Cell, 68
- calcRo
 - mknix::ConstraintDistance, 109

- calcTangentMatrix
 - mknix::Constraint, 97
 - mknix::FlexBody, 132
 - mknix::FlexFrameGalerkin, 140
 - mknix::FlexGlobalGalerkin, 146
 - mknix::System, 332
- calcThermalTangentMatrix
 - mknix::System, 332
- Cell
 - mknix::Cell, 63
- cell.cpp, 371
- cell.h, 372
- CellBoundary
 - mknix::CellBoundary, 72, 73
- cellboundary.cpp, 373
- cellboundary.h, 373
- CellBoundaryLinear
 - mknix::CellBoundaryLinear, 76, 77
- cellboundarylinear.cpp, 374
- cellboundarylinear.h, 374
- CellRect
 - mknix::CellRect, 81
- cellrect.cpp, 375
- cellrect.h, 376
- cells
 - mknix::Body, 56
 - mknix::BoundaryGroup, 59
 - mknix::ThermalBody, 347
- CellTetrahedron
 - mknix::CellTetrahedron, 84
- celltetrahedron.cpp, 377
- celltetrahedron.h, 378
- CellTriang
 - mknix::CellTriang, 88, 89
- celltriang.cpp, 379
- celltriang.h, 379
- checkAugmented
 - mknix::Constraint, 97
 - mknix::ConstraintThermalFixed, 121
 - mknix::System, 332
- clearAugmented
 - mknix::Constraint, 97
 - mknix::System, 332
- common.cpp, 380
- common.h, 381
- CompBar
 - mknix::CompBar, 91
- compbar.cpp, 382
- compbar.h, 382
- compute_abcd
 - mknix::ShapeFunctionTetrahedron, 289
- computeC
 - mknix::Material, 195
- computeCapacityGaussPoints
 - mknix::Cell, 68
- computeCij
 - mknix::GaussPoint, 154
- computeConductivityGaussPoints
 - mknix::Cell, 68
- computeD
 - mknix::Material, 196
- computeEnergy
 - mknix::Body, 56
 - mknix::FlexBody, 136
 - mknix::Material, 196, 197
 - mknix::RigidBody, 255
 - mknix::ThermalBody, 347
- computeFext
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 164
 - mknix::GaussPoint3D, 174
- computeFextGaussPoints
 - mknix::Cell, 68
- computeFint
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 165
 - mknix::GaussPoint3D, 175
- computeFintGaussPoints
 - mknix::Cell, 68
- computeHij
 - mknix::GaussPoint, 155
- computeKGaussPoints
 - mknix::Cell, 68
- computeKij
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 165
 - mknix::GaussPoint3D, 175
- computeMGaussPoints
 - mknix::Cell, 68
- computeMij
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 166
 - mknix::GaussPoint3D, 176
- computeNLFint
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 166
 - mknix::GaussPoint3D, 176
- computeNLFintGaussPoints
 - mknix::Cell, 68
- computeNLKGaussPoints
 - mknix::Cell, 69
- computeNLKij
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 166
 - mknix::GaussPoint3D, 176
- computeNLStress
 - mknix::GaussPoint, 155
 - mknix::GaussPoint2D, 167
 - mknix::GaussPoint3D, 177
- computePorosityLoad
 - mknix::Material, 197
- computeQext
 - mknix::GaussPoint, 156
 - mknix::GaussPointBoundary, 182
- computeQextGaussPoints
 - mknix::Cell, 69

- mknix::CellBoundary, 73
- computeS
 - mknix::Material, 198, 199
- computeShapeFunctions
 - mknix::Cell, 69
 - mknix::CellBoundary, 73
 - mknix::CellBoundaryLinear, 78
 - mknix::ElemTetrahedron, 125
 - mknix::ElemTriangle, 128
- computeStress
 - mknix::FlexBody, 136
 - mknix::GaussPoint, 156
 - mknix::GaussPoint2D, 167
 - mknix::GaussPoint3D, 177
- CONSTANT
 - mknix, 22
- constantSouce
 - mknix::LoadThermalBody, 190
- Constraint
 - mknix::Constraint, 94
- constraint.cpp, 382
- constraint.h, 383
- ConstraintClearance
 - mknix::ConstraintClearance, 101, 102
- constraintclearance.cpp, 383
- constraintclearance.h, 384
- ConstraintContact
 - mknix::ConstraintContact, 105
- constraintcontact.cpp, 385
- constraintcontact.h, 385
- ConstraintDistance
 - mknix::ConstraintDistance, 108
- constraintdistance.cpp, 386
- constraintdistance.h, 386
- ConstraintFixedAxis
 - mknix::ConstraintFixedAxis, 111
- constraintfixedaxis.cpp, 387
- constraintfixedaxis.h, 388
- ConstraintFixedCoordinates
 - mknix::ConstraintFixedCoordinates, 114
- constraintfixedcoordinates.cpp, 388
- constraintfixedcoordinates.h, 389
- constraints
 - mknix::System, 337
- constraintsThermal
 - mknix::System, 337
- ConstraintThermal
 - mknix::ConstraintThermal, 117
- constraintthermal.cpp, 390
- constraintthermal.h, 390
- ConstraintThermalFixed
 - mknix::ConstraintThermalFixed, 120
- constraintthermalfixed.cpp, 391
- constraintthermalfixed.h, 391
- Contact
 - mknix::Contact, 121, 122
 - mknix::Simulation, 324
 - mknix::System, 337
- createDelaunay
 - mknix::Contact, 122
- createDrawingContactObjects
 - mknix::Contact, 122
- createDrawingObjects
 - mknix::Contact, 122
- createGaussPoints
 - mknix::CellBoundaryLinear, 78
 - mknix::CellTetrahedron, 85
 - mknix::CellTriang, 90
- createGaussPoints_MC
 - mknix::ElemTetrahedron, 125
 - mknix::ElemTriangle, 128
- createPoints
 - mknix::Contact, 122
- createPolys
 - mknix::Contact, 122
- data_type
 - mknix, 22
- dc
 - mknix::Cell, 71
 - mknix::CellBoundary, 74
 - mknix::Point, 235
- deg2rad
 - mknix, 23
- densityFactor
 - mknix::RigidBody, 255
- dictionary.h, 392
- dim
 - mknix::Constraint, 99
 - mknix::Point, 235
 - mknix::RigidBody, 255
 - mknix::ShapeFunction, 274
- distance
 - mknix::Point, 225
- domainConf
 - mknix::RigidBody, 255
- doubles_in_vector
 - mknix, 23
- drawObjects
 - mknix::Contact, 122
- dynamicAcceleration
 - mknix::Simulation, 298
- dynamicConvergence
 - mknix::Simulation, 299
- dynamicResidue
 - mknix::Simulation, 299
- dynamicTangent
 - mknix::Simulation, 300
- dynamicThermalConvergence
 - mknix::Simulation, 301
- dynamicThermalConvergenceInThermomechanical
 - mknix::Simulation, 301
- dynamicThermalEvaluation
 - mknix::Simulation, 302
- dynamicThermalResidue
 - mknix::Simulation, 302
- dynamicThermalTangent

- mknix::Simulation, 303
- ElemTetrahedron
 - mknix::ElemTetrahedron, 124
- elemtetrahedron.cpp, 392
- elemtetrahedron.h, 393
- ElemTriangle
 - mknix::ElemTriangle, 127
- elemtriangle.cpp, 394
- elemtriangle.h, 394
- endSimulation
 - mknix::Simulation, 303
- energies
 - mknix::FlexBody, 136
- energy
 - mknix::RigidBody, 255
- epsilon
 - mknix::Analysis, 31
- explicitAcceleration
 - mknix::Simulation, 304
- explicitThermalEvaluation
 - mknix::Simulation, 304
- externalForces
 - mknix::Load, 185
 - mknix::RigidBody, 255
- externalHeat
 - mknix::Load, 185
 - mknix::LoadThermal, 187
- fext
 - mknix::GaussPoint, 158
- fillFEmatrices
 - mknix::GaussPoint, 156
 - mknix::GaussPoint2D, 168
 - mknix::GaussPoint3D, 178
- filter
 - mknix::boxFIR, 60
- findSupportNodes
 - mknix::Point, 226, 227
- fint
 - mknix::GaussPoint, 158
- flexBodies
 - mknix::System, 337
- FlexBody
 - mknix::FlexBody, 131
- flexBodyNames
 - mknix::System, 332
- FlexFrameGalerkin
 - mknix::FlexFrameGalerkin, 138
- FlexGlobalGalerkin
 - mknix::FlexGlobalGalerkin, 144
- FORCE
 - mknix::Simulation, 297
- Force
 - mknix::Force, 149
- force.cpp, 395
- force.h, 396
- formulation
 - mknix::Cell, 71
 - mknix::CellBoundary, 74
 - mknix::FlexBody, 136
- frameNodes
 - mknix::RigidBody, 255
- GaussPoint
 - mknix::GaussPoint, 152
 - mknix::Point, 234
- gausspoint.cpp, 397
- gausspoint.h, 397
- GaussPoint2D
 - mknix::GaussPoint2D, 160, 161
- gausspoint2D.cpp, 398
- gausspoint2D.h, 399
- GaussPoint3D
 - mknix::GaussPoint3D, 171
- gausspoint3D.cpp, 399
- gausspoint3D.h, 400
- GaussPointBoundary
 - mknix::GaussPointBoundary, 180, 181
- gausspointboundary.cpp, 401
- gausspointboundary.h, 401
- generalcontact.cpp, 402
- generalcontact.h, 402
- getAlpha
 - mknix::Simulation, 305
- getAugmentedTolerance
 - mknix::Simulation, 305
- getBeta
 - mknix::Material, 199
- getBody
 - mknix::System, 332
- getBodyPoint
 - mknix::FlexBody, 132
- getBoundaryFirstNode
 - mknix::Body, 51
- getBoundaryNextNode
 - mknix::Body, 51
- getBoundarySize
 - mknix::Body, 51
- getC
 - mknix::Material, 200
- getCapacity
 - mknix::Material, 200
- getCellLastNumber
 - mknix::Body, 51
- getConf
 - mknix::Node, 213
 - mknix::Point, 227
- getConstraint
 - mknix::Simulation, 306
 - mknix::System, 332
- getConstraintMethod
 - mknix::Simulation, 306
- getConstraintNames
 - mknix::Simulation, 306
 - mknix::System, 333
- getConstraintOutput
 - mknix::Simulation, 307

- getConstraintThermal
 - mknix::System, 333
- getCsym
 - mknix::Material, 201
- getD
 - mknix::Material, 201
- getDensity
 - mknix::Material, 201
- getDim
 - mknix::Point, 228
 - mknix::Simulation, 307
- getE
 - mknix::Material, 202
- getFields
 - mknix::SectionReader, 269
- getFluidTempHistory
 - mknix::Material, 202
- getGap
 - mknix::ConstraintContact, 105
- getGravity
 - mknix::Simulation, 308
- getInitialTemperature
 - mknix::Simulation, 310
- getInterfaceNode
 - mknix::Simulation, 310
- getInterfaceNodesCoords
 - mknix::Simulation, 311
- getInterfaceNumberOfNodes
 - mknix::Simulation, 311
- getInternalForces
 - mknix::Constraint, 97
 - mknix::ConstraintClearance, 102
 - mknix::ConstraintContact, 106
 - mknix::ConstraintDistance, 109
 - mknix::ConstraintFixedAxis, 112
 - mknix::ConstraintFixedCoordinates, 115
- getKappa
 - mknix::Material, 202
- getLastBodyPoint
 - mknix::FlexBody, 132
- getLastNode
 - mknix::Body, 51
 - mknix::ThermalBody, 345
- getLoadThermalBody
 - mknix::LoadThermalBody, 189
- getLoadThermalBoundary1D
 - mknix::LoadThermalBoundary1D, 192
- getMaxTemp
 - mknix::LoadThermal, 187
- getMu
 - mknix::Material, 203
- getNode
 - mknix::Body, 52
 - mknix::Constraint, 97
 - mknix::FlexBody, 133
 - mknix::RigidBody, 252
 - mknix::System, 333
 - mknix::SystemChain, 340
 - mknix::ThermalBody, 345
- getNodeCount
 - mknix::Constraint, 98
- getNodeNumbers
 - mknix::Cell, 69
- getNodes
 - mknix::Body, 52
- getNodesSize
 - mknix::Body, 52
- getNumber
 - mknix::Point, 228
- getNumberOfNodes
 - mknix::System, 333
- getNumberOfNodes
 - mknix::Cell, 69
- getNumberOfPoints
 - mknix::FlexBody, 133
- getOutputNode
 - mknix::Simulation, 311
- getOutputNode
 - mknix::System, 333
- getOutputSignalThermal
 - mknix::System, 333
- getPhi
 - mknix::ShapeFunction, 272
- getPorosityFluidTemp
 - mknix::Material, 203
- getPorosityResistance
 - mknix::Material, 203
- getqt
 - mknix::Node, 214
- getqx
 - mknix::Node, 214
- getShapeFunType
 - mknix::Point, 228
- getShapeFunValue
 - mknix::Node, 215
- getSignal
 - mknix, 23
- getSignalNodes
 - mknix::Simulation, 311
 - mknix::System, 333
- getSmoothingType
 - mknix::Simulation, 312
- getSparsePattern
 - mknix::Simulation, 313
- getStiffnessMatrix
 - mknix::Constraint, 98
 - mknix::ConstraintClearance, 103
 - mknix::ConstraintContact, 106
 - mknix::ConstraintDistance, 109
 - mknix::ConstraintFixedAxis, 112
 - mknix::ConstraintFixedCoordinates, 115
- getSubSections
 - mknix::SectionReader, 269
- getSupportNodeNumber
 - mknix::Node, 215
 - mknix::Point, 229

- getSupportNodes
 - mknix::Point, [229](#)
- getSupportSize
 - mknix::Node, [216](#)
 - mknix::Point, [229](#)
- getSystem
 - mknix::System, [333](#)
- getTemp
 - mknix::Node, [216](#)
 - mknix::Point, [229](#)
- getThermalNodes
 - mknix::System, [334](#)
- getThermalNumber
 - mknix::Node, [217](#)
- getTime
 - mknix::Simulation, [314](#)
- getTitle
 - mknix::Constraint, [98](#)
 - mknix::System, [334](#)
- getType
 - mknix::Body, [52](#)
 - mknix::FlexFrameGalerkin, [140](#)
 - mknix::FlexGlobalGalerkin, [146](#)
 - mknix::RigidBar, [248](#)
 - mknix::RigidBody2D, [259](#)
 - mknix::RigidBody3D, [264](#)
 - mknix::RigidBodyMassPoint, [267](#)
- getU
 - mknix::Node, [217](#)
- getUx
 - mknix::Node, [217](#)
- getUy
 - mknix::Node, [218](#)
- getUz
 - mknix::Node, [218](#)
- getWeight
 - mknix::Node, [218](#)
- getX
 - mknix::Point, [230](#)
- getY
 - mknix::Point, [231](#)
- getZ
 - mknix::Point, [232](#)
- gnuplotOut
 - mknix::Cell, [69](#)
 - mknix::CellRect, [83](#)
 - mknix::CellTetrahedron, [86](#)
 - mknix::CellTriang, [90](#)
 - mknix::Point, [232](#)
 - mknix::ShapeFunction, [273](#)
- gnuplotOutStress
 - mknix::Cell, [69](#)
 - mknix::GaussPoint, [156](#)
- gp
 - mknix::ShapeFunction, [274](#)
- gPoints
 - mknix::Cell, [71](#)
 - mknix::CellBoundary, [74](#)
- gPoints_MC
 - mknix::Cell, [71](#)
- groundNodes
 - mknix::System, [337](#)
- groundNodesMap
 - mknix::System, [337](#)
- H
 - mknix::GaussPoint, [158](#)
- init
 - mknix::Analysis, [29](#)
 - mknix::AnalysisDynamic, [33](#)
 - mknix::AnalysisStatic, [36](#)
 - mknix::AnalysisThermalDynamic, [38](#)
 - mknix::AnalysisThermalStatic, [41](#)
 - mknix::AnalysisThermoMechanicalDynamic, [43](#)
 - mknix::MknixWrapper, [207](#)
 - mknix::Simulation, [314](#)
- initFlexBodies
 - mknix::System, [334](#)
- initialize
 - mknix::Body, [52](#)
 - mknix::BoundaryGroup, [59](#)
 - mknix::Cell, [70](#)
 - mknix::CellBoundary, [74](#)
 - mknix::CellBoundaryLinear, [79](#)
 - mknix::CellRect, [83](#)
 - mknix::ElemTetrahedron, [125](#)
 - mknix::ElemTriangle, [128](#)
 - mknix::FlexBody, [133](#)
 - mknix::ThermalBody, [345](#)
- initializeMatVecs
 - mknix::GaussPointBoundary, [182](#)
- input_manual.md, [403](#)
- inputFromFile
 - mknix::Reader, [239](#)
 - mknix::Simulation, [315](#)
- inputSignals
 - mknix::System, [337](#)
- insertNodesXCoordinates
 - mknix::LoadThermal, [187](#)
- internalForces
 - mknix::Constraint, [99](#)
- internalForcesOutput
 - mknix::Constraint, [99](#)
- interpolate1D
 - mknix, [24](#)
- interpolate2D
 - mknix, [24](#)
- interpolate3D
 - mknix, [25](#)
- isNumber
 - mknix, [26](#)
- isPorous
 - mknix::Material, [204](#)
- isThermal
 - mknix::Body, [56](#)
- iter_augmented

- mknix::Constraint, 99
- jacobian
 - mknix::Cell, 71
 - mknix::CellBoundary, 75
 - mknix::Point, 235
- K
 - mknix::GaussPoint, 158
- lambda
 - mknix::Constraint, 99
- lastNode
 - mknix::Body, 57
- linearBoundary
 - mknix::Body, 57
- Imx, 20
- Imx::DenseMatrix< T >, 122
- Imx::Matrix< T >, 206
- Imx::Vector< T >, 347
- Load
 - mknix::Load, 184
- load.cpp, 403
- load.h, 403
- loadFile
 - mknix::LoadThermalBoundary1D, 192
- loadFile3D
 - mknix::LoadThermalBody, 189
- loadmap
 - mknix::LoadThermalBoundary1D, 193
- loadradiation.cpp, 404
- loadradiation.h, 405
- loads
 - mknix::System, 337
- loadsThermal
 - mknix::System, 338
- LoadThermal
 - mknix::LoadThermal, 186
- loadthermal.cpp, 406
- loadthermal.h, 406
- LoadThermalBody
 - mknix::LoadThermalBody, 188
- loadthermalbody.cpp, 407
- loadthermalbody.h, 407
- LoadThermalBoundary1D
 - mknix::LoadThermalBoundary1D, 191
- loadthermalboundary1D.cpp, 408
- loadthermalboundary1D.h, 409
- loadThermalBoundaryGroup
 - mknix::BoundaryGroup, 59
- loadTimeFile
 - mknix::LoadThermalBody, 190
 - mknix::LoadThermalBoundary1D, 193
- localMassMatrix
 - mknix::RigidBody, 255
- M
 - mknix::GaussPoint, 158
- m_hasTimeScale
 - mknix::LoadThermalBody, 190
- m_source
 - mknix::LoadThermalBody, 190
- m_source2D
 - mknix::LoadThermalBody, 190
- m_source3D
 - mknix::LoadThermalBody, 190
- m_sourceType
 - mknix::LoadThermalBody, 191
- m_time
 - mknix::LoadThermalBody, 191
- mainpage.md, 410
- make_unique
 - mknix, 26
- mass
 - mknix::RigidBody, 256
- mat
 - mknix::Cell, 71
 - mknix::GaussPoint, 158
- Material
 - mknix::Material, 195
- material.cpp, 410
- material.h, 410
- method
 - mknix::Constraint, 99
- mknix, 20
 - CONSTANT, 22
 - data_type, 22
 - deg2rad, 23
 - doubles_in_vector, 23
 - getSignal, 23
 - interpolate1D, 24
 - interpolate2D, 24
 - interpolate3D, 25
 - isNumber, 26
 - make_unique, 26
 - read_lines, 26
 - readFile3D, 27
 - setSignal, 27
 - SOURCE_1D, 22
 - SOURCE_2D, 22
 - SOURCE_3D, 22
 - SOURCE_VTK, 22
 - SourceType, 22
- mknix::Analysis, 28
 - ~Analysis, 29
 - Analysis, 29
 - epsilon, 31
 - init, 29
 - nextStep, 29
 - setEpsilon, 30
 - solve, 30
 - theSimulation, 31
 - type, 30
- mknix::AnalysisDynamic, 31
 - ~AnalysisDynamic, 33
 - AnalysisDynamic, 32
 - init, 33

- nextStep, 33
- solve, 33
- type, 34
- mknix::AnalysisStatic, 34
 - ~AnalysisStatic, 36
 - AnalysisStatic, 35
 - init, 36
 - nextStep, 36
 - solve, 36
 - type, 36
- mknix::AnalysisThermalDynamic, 36
 - ~AnalysisThermalDynamic, 38
 - AnalysisThermalDynamic, 38
 - init, 38
 - nextStep, 38
 - solve, 39
 - type, 39
- mknix::AnalysisThermalStatic, 39
 - ~AnalysisThermalStatic, 41
 - AnalysisThermalStatic, 40
 - init, 41
 - nextStep, 41
 - solve, 41
 - type, 41
- mknix::AnalysisThermoMechanicalDynamic, 41
 - ~AnalysisThermoMechanicalDynamic, 43
 - AnalysisThermoMechanicalDynamic, 43
 - init, 43
 - nextStep, 44
 - solve, 44
 - type, 44
- mknix::Body, 44
 - ~Body, 47
 - addBoundaryConnectivity, 47
 - addBoundaryGroup, 48
 - addBoundaryLine, 48
 - addCell, 48
 - addCellToBoundaryGroup, 48
 - addLoadThermal, 48
 - addNode, 48
 - addNodes, 49
 - addNodeToBoundaryGroup, 49
 - assembleCapacityMatrix, 49
 - assembleConductivityMatrix, 49
 - assembleExternalForces, 50
 - assembleExternalHeat, 50
 - assembleMassMatrix, 50
 - Body, 47
 - bondedBodyNodes, 56
 - boundaryConnectivity, 56
 - boundaryGroups, 56
 - calcCapacityMatrix, 50
 - calcConductivityMatrix, 50
 - calcExternalForces, 50
 - calcExternalHeat, 50
 - calcMassMatrix, 50
 - cells, 56
 - computeEnergy, 56
 - getBoundaryFirstNode, 51
 - getBoundaryNextNode, 51
 - getBoundarySize, 51
 - getCellLastNumber, 51
 - getLastNode, 51
 - getNode, 52
 - getNodes, 52
 - getNodesSize, 52
 - getType, 52
 - initialize, 52
 - isThermal, 56
 - lastNode, 57
 - linearBoundary, 57
 - nodes, 57
 - outputStep, 52, 53
 - outputToFile, 53
 - outputVTK, 54
 - rotate, 54
 - setLoadThermalInBoundaryGroup, 55
 - setMechanical, 55
 - setOutput, 55
 - setTemperature, 55
 - temperature, 57
 - title, 57
 - translate, 55
 - volumetricHeatSources, 57
 - writeBodyInfo, 56
 - writeBoundaryConnectivity, 56
 - writeBoundaryNodes, 56
- mknix::BoundaryGroup, 57
 - ~BoundaryGroup, 58
 - addCell, 58
 - addNode, 58
 - assembleExternalHeat, 58
 - BoundaryGroup, 58
 - calcExternalHeat, 59
 - cells, 59
 - initialize, 59
 - loadThermalBoundaryGroup, 59
 - nodes, 59
 - setLoadThermal, 59
- mknix::boxFIR, 59
 - boxFIR, 60
 - filter, 60
- mknix::Cell, 60
 - ~Cell, 63
 - alpha, 70
 - assembleCapacityGaussPoints, 63
 - assembleConductivityGaussPoints, 65
 - assembleFextGaussPoints, 65
 - assembleFintGaussPoints, 65
 - assembleKGaussPoints, 65
 - assembleMGaussPoints, 65
 - assembleNLRGaussPoints, 66
 - assembleQextGaussPoints, 66
 - assembleRGaussPoints, 66
 - bodyPoints, 71
 - calcElasticEGaussPoints, 66

- calcKineticEGaussPoints, 66
- calcPotentialEGaussPoints, 68
- Cell, 63
- computeCapacityGaussPoints, 68
- computeConductivityGaussPoints, 68
- computeFextGaussPoints, 68
- computeFintGaussPoints, 68
- computeKGaussPoints, 68
- computeMGaussPoints, 68
- computeNLFintGaussPoints, 68
- computeNLKGaussPoints, 69
- computeQextGaussPoints, 69
- computeShapeFunctions, 69
- dc, 71
- formulation, 71
- getNodeNumbers, 69
- getNumberOfNodes, 69
- gnuplotOut, 69
- gnuplotOutStress, 69
- gPoints, 71
- gPoints_MC, 71
- initialize, 70
- jacobian, 71
- mat, 71
- nGPoints, 71
- outputConnectivityToFile, 70
- setMaterialIfLayer, 70
- mknix::CellBoundary, 71
 - ~CellBoundary, 73
 - alpha, 74
 - assembleQextGaussPoints, 73
 - bodyPoints, 74
 - CellBoundary, 72, 73
 - computeQextGaussPoints, 73
 - computeShapeFunctions, 73
 - dc, 74
 - formulation, 74
 - gPoints, 74
 - initialize, 74
 - jacobian, 75
 - nGPoints, 75
- mknix::CellBoundaryLinear, 75
 - ~CellBoundaryLinear, 78
 - CellBoundaryLinear, 76, 77
 - computeShapeFunctions, 78
 - createGaussPoints, 78
 - initialize, 79
 - points, 79
- mknix::CellRect, 79
 - ~CellRect, 82
 - CellRect, 81
 - gnuplotOut, 83
 - initialize, 83
- mknix::CellTetrahedron, 83
 - ~CellTetrahedron, 85
 - CellTetrahedron, 84
 - createGaussPoints, 85
 - gnuplotOut, 86
 - points, 86
- mknix::CellTriang, 86
 - ~CellTriang, 90
 - CellTriang, 88, 89
 - createGaussPoints, 90
 - gnuplotOut, 90
 - points, 91
- mknix::CompBar, 91
 - ~CompBar, 91
 - addToRender, 91
 - CompBar, 91
 - removeFromRender, 91
 - updatePoints, 91
- mknix::Constraint, 92
 - ~Constraint, 94
 - alpha, 99
 - assembleInternalForces, 94
 - assembleTangentMatrix, 95
 - augmentedTolerance, 99
 - calcInternalForces, 95
 - calcPhi, 96
 - calcPhiq, 96
 - calcPhiqq, 96
 - calcTangentMatrix, 97
 - checkAugmented, 97
 - clearAugmented, 97
 - Constraint, 94
 - dim, 99
 - getInternalForces, 97
 - getNode, 97
 - getNodeCount, 98
 - getStiffnessMatrix, 98
 - getTitle, 98
 - internalForces, 99
 - internalForcesOutput, 99
 - iter_augmented, 99
 - lambda, 99
 - method, 99
 - nodes, 99
 - outputStep, 98
 - outputToFile, 98
 - phi, 99
 - phi_q, 100
 - phi_qq, 100
 - setTitle, 98
 - stiffnessMatrix, 100
 - title, 100
 - writeJointInfo, 99
- mknix::ConstraintClearance, 100
 - ~ConstraintClearance, 102
 - calcPhi, 102
 - calcPhiq, 102
 - calcPhiqq, 102
 - ConstraintClearance, 101, 102
 - getInternalForces, 102
 - getStiffnessMatrix, 103
 - rh, 103
 - rt, 103

- mknix::ConstraintContact, 103
 - ~ConstraintContact, 105
 - calcPhi, 105
 - calcPhiq, 105
 - calcPhiqq, 105
 - ConstraintContact, 105
 - getGap, 105
 - getInternalForces, 106
 - getStiffnessMatrix, 106
 - normal, 106
 - rh, 106
 - rt, 106
- mknix::ConstraintDistance, 106
 - ~ConstraintDistance, 108
 - calcPhi, 108
 - calcPhiq, 108
 - calcPhiqq, 109
 - calcRo, 109
 - ConstraintDistance, 108
 - getInternalForces, 109
 - getStiffnessMatrix, 109
 - ro, 109
 - rt, 109
 - setLenght, 109
- mknix::ConstraintFixedAxis, 110
 - ~ConstraintFixedAxis, 111
 - axisName, 112
 - calcPhi, 112
 - calcPhiq, 112
 - calcPhiqq, 112
 - ConstraintFixedAxis, 111
 - getInternalForces, 112
 - getStiffnessMatrix, 112
 - ro, 112
 - rt, 112
- mknix::ConstraintFixedCoordinates, 113
 - ~ConstraintFixedCoordinates, 115
 - calcPhi, 115
 - calcPhiq, 115
 - calcPhiqq, 115
 - ConstraintFixedCoordinates, 114
 - getInternalForces, 115
 - getStiffnessMatrix, 115
 - rxo, 115
 - rxt, 115
 - ryo, 115
 - ryt, 115
 - rzo, 115
 - rzt, 116
- mknix::ConstraintThermal, 116
 - ~ConstraintThermal, 117
 - assembleInternalForces, 117
 - assembleTangentMatrix, 118
 - ConstraintThermal, 117
- mknix::ConstraintThermalFixed, 118
 - ~ConstraintThermalFixed, 120
 - calcPhi, 120
 - calcPhiq, 120
 - calcPhiqq, 121
 - checkAugmented, 121
 - ConstraintThermalFixed, 120
 - Tt, 121
- mknix::Contact, 121
 - ~Contact, 122
 - Contact, 121, 122
 - createDelaunay, 122
 - createDrawingContactObjects, 122
 - createDrawingObjects, 122
 - createPoints, 122
 - createPolys, 122
 - drawObjects, 122
 - updateDelaunay, 122
 - updateLines, 122
 - updatePoints, 122
- mknix::ElemTetrahedron, 123
 - ~ElemTetrahedron, 124
 - computeShapeFunctions, 125
 - createGaussPoints_MC, 125
 - ElemTetrahedron, 124
 - initialize, 125
- mknix::ElemTriangle, 126
 - ~ElemTriangle, 127
 - computeShapeFunctions, 128
 - createGaussPoints_MC, 128
 - ElemTriangle, 127
 - initialize, 128
- mknix::FlexBody, 129
 - ~FlexBody, 131
 - addBodyPoint, 131
 - addPoint, 131, 132
 - assembleInternalForces, 132
 - assembleTangentMatrix, 132
 - bodyPoints, 136
 - calcInternalForces, 132
 - calcTangentMatrix, 132
 - computeEnergy, 136
 - computeStress, 136
 - energies, 136
 - FlexBody, 131
 - formulation, 136
 - getBodyPoint, 132
 - getLastBodyPoint, 132
 - getNode, 133
 - getNumberOfPoints, 133
 - initialize, 133
 - outputToFile, 133
 - points, 136
 - setFormulation, 134
 - setOutput, 134
 - setType, 134
 - smoothingMassMatrix, 136
 - stresses, 136
 - writeBodyInfo, 134
 - writeBoundaryConnectivity, 135
 - writeBoundaryNodes, 135
- mknix::FlexFrameGalerkin, 136

- ~FlexFrameGalerkin, 138
- assembleExternalForces, 139
- assembleInternalForces, 139
- assembleMassMatrix, 139
- assembleTangentMatrix, 140
- calcExternalForces, 140
- calcInternalForces, 140
- calcMassMatrix, 140
- calcTangentMatrix, 140
- FlexFrameGalerkin, 138
- getType, 140
- outputStep, 140, 141
- setType, 142
- mknix::FlexGlobalGalerkin, 142
 - ~FlexGlobalGalerkin, 144
 - assembleExternalForces, 144
 - assembleInternalForces, 145
 - assembleMassMatrix, 145
 - assembleTangentMatrix, 145
 - calcExternalForces, 145
 - calcInternalForces, 146
 - calcMassMatrix, 146
 - calcTangentMatrix, 146
 - FlexGlobalGalerkin, 144
 - getType, 146
 - outputStep, 146, 147
 - setType, 147
- mknix::Force, 148
 - ~Force, 149
 - Force, 149
 - outputToFile, 149
- mknix::GaussPoint, 150
 - ~GaussPoint, 153
 - assembleCij, 153
 - assembleFext, 153
 - assembleFint, 153
 - assembleHij, 153
 - assembleKij, 154
 - assembleMij, 154
 - assembleQext, 154
 - assembleRi, 154
 - avgTemp, 157
 - B, 157
 - C, 157
 - calcElasticE, 154
 - calcKineticE, 154
 - calcPotentialE, 154
 - computeCij, 154
 - computeFext, 155
 - computeFint, 155
 - computeHij, 155
 - computeKij, 155
 - computeMij, 155
 - computeNLFint, 155
 - computeNLKij, 155
 - computeNLStress, 155
 - computeQext, 156
 - computeStress, 156
 - fext, 158
 - fillFEmatrices, 156
 - fint, 158
 - GaussPoint, 152
 - gnuplotOutStress, 156
 - H, 158
 - K, 158
 - M, 158
 - mat, 158
 - num, 158
 - Qext, 158
 - r, 158
 - setMaterial, 156
 - shapeFunSolve, 156
 - stressPoint, 158
 - tension, 158
 - weight, 158
- mknix::GaussPoint2D, 159
 - ~GaussPoint2D, 162
 - assembleFext, 162
 - assembleFint, 162
 - assembleKij, 162
 - assembleMij, 162
 - assembleRi, 163
 - calcElasticE, 163
 - calcKineticE, 163
 - calcPotentialE, 164
 - computeFext, 164
 - computeFint, 165
 - computeKij, 165
 - computeMij, 166
 - computeNLFint, 166
 - computeNLKij, 166
 - computeNLStress, 167
 - computeStress, 167
 - fillFEmatrices, 168
 - GaussPoint2D, 160, 161
 - shapeFunSolve, 168
- mknix::GaussPoint3D, 169
 - ~GaussPoint3D, 172
 - assembleFext, 172
 - assembleFint, 172
 - assembleKij, 172
 - assembleMij, 172
 - assembleRi, 173
 - calcElasticE, 173
 - calcKineticE, 173
 - calcPotentialE, 174
 - computeFext, 174
 - computeFint, 175
 - computeKij, 175
 - computeMij, 176
 - computeNLFint, 176
 - computeNLKij, 176
 - computeNLStress, 177
 - computeStress, 177
 - fillFEmatrices, 178
 - GaussPoint3D, 171

- shapeFunSolve, 178
- mknix::GaussPointBoundary, 179
 - ~GaussPointBoundary, 181
 - assembleQext, 181
 - computeQext, 182
 - GaussPointBoundary, 180, 181
 - initializeMatVecs, 182
 - num, 183
 - Qext, 183
 - shapeFunSolve, 182
 - weight, 183
- mknix::Load, 183
 - ~Load, 184
 - assembleExternalForces, 184
 - externalForces, 185
 - externalHeat, 185
 - Load, 184
 - nodes, 185
 - outputToFile, 185
- mknix::LoadThermal, 185
 - ~LoadThermal, 186
 - assembleExternalHeat, 186
 - externalHeat, 187
 - getMaxTemp, 187
 - insertNodesXCoordinates, 187
 - LoadThermal, 186
 - nodes, 187
 - outputToFile, 187
 - updateLoad, 187
- mknix::LoadThermalBody, 188
 - ~LoadThermalBody, 188
 - addLoad, 188, 189
 - constantSouce, 190
 - getLoadThermalBody, 189
 - loadFile3D, 189
 - LoadThermalBody, 188
 - loadTimeFile, 190
 - m_hasTimeScale, 190
 - m_source, 190
 - m_source2D, 190
 - m_source3D, 190
 - m_sourceType, 191
 - m_time, 191
 - setConstantValue, 190
- mknix::LoadThermalBoundary1D, 191
 - ~LoadThermalBoundary1D, 191
 - getLoadThermalBoundary1D, 192
 - loadFile, 192
 - loadmap, 193
 - LoadThermalBoundary1D, 191
 - loadTimeFile, 193
 - scaleLoad, 193
 - timemap, 193
- mknix::Material, 193
 - ~Material, 195
 - addThermalCapacity, 195
 - addThermalConductivity, 195
 - addThermalDensity, 195
 - addThermalExpansion, 195
 - addVariableResistance, 195
 - computeC, 195
 - computeD, 196
 - computeEnergy, 196, 197
 - computePorosityLoad, 197
 - computeS, 198, 199
 - getBeta, 199
 - getC, 200
 - getCapacity, 200
 - getCsym, 201
 - getD, 201
 - getDensity, 201
 - getE, 202
 - getFluidTempHistory, 202
 - getKappa, 202
 - getMu, 203
 - getPorosityFluidTemp, 203
 - getPorosityResistance, 203
 - isPorous, 204
 - Material, 195
 - outputToFile, 204
 - setMechanicalProps, 205
 - setPorosityFluidTemp, 205
 - setPorosityProps, 205
 - setThermalProps, 206
 - setVariableResistanceCoords, 206
 - update, 206
- mknix::MknixWrapper, 206
 - ~MknixWrapper, 207
 - init, 207
 - MknixWrapper, 207
 - run, 207
- mknix::MknixWrapper::SimulationWrapper, 325
 - theSimulation, 325
- mknix::Motion, 207
 - ~Motion, 209
 - Motion, 209
 - setNode, 209
 - setTimeUx, 209
 - setTimeUy, 209
 - setTimeUz, 209
 - update, 209
- mknix::Node, 210
 - ~Node, 213
 - addWeight, 213
 - getConf, 213
 - getqt, 214
 - getqx, 214
 - getShapeFunValue, 215
 - getSupportNodeNumber, 215
 - getSupportSize, 216
 - getTemp, 216
 - getThermalNumber, 217
 - getU, 217
 - getUx, 217
 - getUy, 218
 - getUz, 218

- getWeight, 218
- Node, 212
- setNumber, 218
- setqt, 219
- setqx, 219
- setThermalNumber, 219
- setUx, 219
- setUy, 220
- setUz, 220
- setX, 220
- setY, 220
- setZ, 220
- mknix::Point, 221
 - ~Point, 225
 - addSupportNode, 225
 - alpha_i, 235
 - dc, 235
 - dim, 235
 - distance, 225
 - findSupportNodes, 226, 227
 - GaussPoint, 234
 - getConf, 227
 - getDim, 228
 - getNumber, 228
 - getShapeFunType, 228
 - getSupportNodeNumber, 229
 - getSupportNodes, 229
 - getSupportSize, 229
 - getTemp, 229
 - getX, 230
 - getY, 231
 - getZ, 232
 - gnuplotOut, 232
 - jacobian, 235
 - num, 235
 - Point, 223, 224
 - setAlpha_i, 233
 - setDc, 233
 - setJacobian, 233
 - setShapeFunType, 233
 - shapeFun, 235
 - ShapeFunction, 234
 - ShapeFunctionLinear, 234
 - ShapeFunctionLinearX, 234
 - ShapeFunctionMLS, 234
 - ShapeFunctionMLS2D, 235
 - ShapeFunctionMLS3D, 235
 - ShapeFunctionRBF, 235
 - ShapeFunctionTetrahedron, 235
 - ShapeFunctionTriangle, 235
 - shapeFunSolve, 233
 - shapeFunType, 235
 - supportNodes, 235
 - supportNodesSize, 236
 - X, 236
 - Y, 236
 - Z, 236
- mknix::Radiation, 236
 - ~Radiation, 237
 - addVoxel, 238
 - outputToFile, 238
 - Radiation, 237
- mknix::Reader, 238
 - ~Reader, 239
 - inputFromFile, 239
 - Reader, 239
- mknix::ReaderConstraints, 240
 - ~ReaderConstraints, 240
 - readConstraints, 241
 - ReaderConstraints, 240
- mknix::ReaderFlex, 241
 - ~ReaderFlex, 242
 - ReaderFlex, 242
 - readFlexBodies, 242
- mknix::ReaderRigid, 243
 - ~ReaderRigid, 244
 - ReaderRigid, 243
 - readRigidBodies, 244
- mknix::RigidBar, 244
 - ~RigidBar, 247
 - addNode, 247
 - calcExternalForces, 247
 - calcMassMatrix, 248
 - getType, 248
 - RigidBar, 246
 - setInertia, 248
 - setPosition, 248
 - writeBoundaryConnectivity, 249
 - writeBoundaryNodes, 249
- mknix::RigidBody, 249
 - ~RigidBody, 251
 - assembleExternalForces, 251
 - assembleMassMatrix, 252
 - computeEnergy, 255
 - densityFactor, 255
 - dim, 255
 - domainConf, 255
 - energy, 255
 - externalForces, 255
 - frameNodes, 255
 - getNode, 252
 - localMassMatrix, 255
 - mass, 256
 - outputStep, 252, 253
 - outputToFile, 253
 - RigidBody, 251
 - setDensityFactor, 254
 - setInertia, 254
 - setMass, 254
 - setOutput, 254
 - setPosition, 254
 - writeBodyInfo, 254
 - writeBoundaryConnectivity, 255
 - writeBoundaryNodes, 255
- mknix::RigidBody2D, 256
 - ~RigidBody2D, 258

- addNode, [258](#)
- calcExternalForces, [259](#)
- calcMassMatrix, [259](#)
- getType, [259](#)
- RigidBody2D, [258](#)
- setInertia, [260](#)
- setPosition, [260](#)
- mknix::RigidBody3D, [260](#)
 - ~RigidBody3D, [263](#)
 - addNode, [263](#)
 - calcExternalForces, [263](#)
 - calcMassMatrix, [264](#)
 - getType, [264](#)
 - RigidBody3D, [262](#)
 - setInertia, [264](#)
 - setPosition, [264](#)
- mknix::RigidBodyMassPoint, [265](#)
 - ~RigidBodyMassPoint, [266](#)
 - calcExternalForces, [266](#)
 - calcMassMatrix, [267](#)
 - getType, [267](#)
 - RigidBodyMassPoint, [266](#)
 - setInertia, [267](#)
 - setPosition, [267](#)
- mknix::SectionReader, [268](#)
 - addField, [268](#)
 - addSubSection, [268](#)
 - getFields, [269](#)
 - getSubSections, [269](#)
 - read, [269](#)
 - SectionReader, [268](#)
- mknix::ShapeFunction, [269](#)
 - ~ShapeFunction, [272](#)
 - calc, [272](#)
 - dim, [274](#)
 - getPhi, [272](#)
 - gnuplotOut, [273](#)
 - gp, [274](#)
 - outputValues, [274](#)
 - phi, [274](#)
 - setPhi, [274](#)
 - ShapeFunction, [271](#)
- mknix::ShapeFunctionLinear, [274](#)
 - ~ShapeFunctionLinear, [276](#)
 - calc, [276](#)
 - ShapeFunctionLinear, [276](#)
- mknix::ShapeFunctionLinearX, [277](#)
 - ~ShapeFunctionLinearX, [278](#)
 - calc, [278](#)
 - ShapeFunctionLinearX, [278](#)
- mknix::ShapeFunctionMLS, [279](#)
 - ~ShapeFunctionMLS, [280](#)
 - calc, [280](#)
 - ShapeFunctionMLS, [280](#)
- mknix::ShapeFunctionMLS2D, [281](#)
 - ~ShapeFunctionMLS2D, [282](#)
 - calc, [282](#)
 - ShapeFunctionMLS2D, [282](#)
- mknix::ShapeFunctionMLS3D, [282](#)
 - ~ShapeFunctionMLS3D, [284](#)
 - calc, [284](#)
 - ShapeFunctionMLS3D, [284](#)
- mknix::ShapeFunctionRBF, [284](#)
 - ~ShapeFunctionRBF, [286](#)
 - calc, [286](#)
 - ShapeFunctionRBF, [286](#)
- mknix::ShapeFunctionTetrahedron, [287](#)
 - ~ShapeFunctionTetrahedron, [289](#)
 - calc, [289](#)
 - compute_abcd, [289](#)
 - ShapeFunctionTetrahedron, [288](#)
- mknix::ShapeFunctionTriangle, [290](#)
 - ~ShapeFunctionTriangle, [291](#)
 - calc, [291](#)
 - ShapeFunctionTriangle, [291](#)
- mknix::ShapeFunctionTriangleSigned, [292](#)
 - ~ShapeFunctionTriangleSigned, [293](#)
 - calc, [293](#)
 - ShapeFunctionTriangleSigned, [293](#)
- mknix::Simulation, [294](#)
 - ~Simulation, [297](#)
 - bodyNames, [297](#)
 - bodyPoints, [298](#)
 - Contact, [324](#)
 - dynamicAcceleration, [298](#)
 - dynamicConvergence, [299](#)
 - dynamicResidue, [299](#)
 - dynamicTangent, [300](#)
 - dynamicThermalConvergence, [301](#)
 - dynamicThermalConvergenceInThermomechanical, [301](#)
 - dynamicThermalEvaluation, [302](#)
 - dynamicThermalResidue, [302](#)
 - dynamicThermalTangent, [303](#)
 - endSimulation, [303](#)
 - explicitAcceleration, [304](#)
 - explicitThermalEvaluation, [304](#)
 - FORCE, [297](#)
 - getAlpha, [305](#)
 - getAugmentedTolerance, [305](#)
 - getConstraint, [306](#)
 - getConstraintMethod, [306](#)
 - getConstraintNames, [306](#)
 - getConstraintOutput, [307](#)
 - getDim, [307](#)
 - getGravity, [308](#)
 - getInitialTemperature, [310](#)
 - getInterfaceNode, [310](#)
 - getInterfaceNodesCoords, [311](#)
 - getInterfaceNumberOfNodes, [311](#)
 - getOutputNode, [311](#)
 - getSignalNodes, [311](#)
 - getSmoothingType, [312](#)
 - getSparsePattern, [313](#)
 - getTime, [314](#)
 - init, [314](#)

- inputFromFile, 315
- operator=, 315
- OutputType, 297
- POSITION, 297
- Reader, 324
- ReaderConstraints, 324
- ReaderFlex, 324
- ReaderRigid, 325
- run, 315
- runMechanicalAnalysis, 315
- runThermalAnalysis, 316
- setAugmentedTolerance, 317
- setInitialTemperatures, 317
- setOutputFilesDetail, 318
- setThermalNodeInitialTemperature, 318
- Simulation, 297
- solveStep, 318, 319
- staticConvergence, 319
- staticResidue, 320
- staticTangent, 321
- staticThermalConvergence, 321
- staticThermalResidue, 322
- staticThermalTangent, 322
- stepTriggered, 323
- STRESS, 297
- SystemChain, 325
- writeConfStep, 323
- writeSystem, 324
- mnknix::System, 325
 - ~System, 329
 - assembleCapacityMatrix, 329
 - assembleConductivityMatrix, 329
 - assembleConstraintForces, 329
 - assembleExternalForces, 329
 - assembleExternalHeat, 330
 - assembleInternalForces, 330
 - assembleInternalHeat, 330
 - assembleMassMatrix, 330
 - assembleTangentMatrix, 330
 - assembleThermalTangentMatrix, 331
 - calcCapacityMatrix, 331
 - calcConductivityMatrix, 331
 - calcExternalForces, 331
 - calcExternalHeat, 331
 - calcInternalForces, 331
 - calcInternalHeat, 331
 - calcMassMatrix, 331
 - calcTangentMatrix, 332
 - calcThermalTangentMatrix, 332
 - checkAugmented, 332
 - clearAugmented, 332
 - constraints, 337
 - constraintsThermal, 337
 - Contact, 337
 - flexBodies, 337
 - flexBodyNames, 332
 - getBody, 332
 - getConstraint, 332
 - getConstraintNames, 333
 - getConstraintThermal, 333
 - getNode, 333
 - getNumberOfNodes, 333
 - getOutputNode, 333
 - getOutputSignalThermal, 333
 - getSignalNodes, 333
 - getSystem, 333
 - getThermalNodes, 334
 - getTitle, 334
 - groundNodes, 337
 - groundNodesMap, 337
 - initFlexBodies, 334
 - inputSignals, 337
 - loads, 337
 - loadsThermal, 338
 - motions, 338
 - outputMaxInterfaceTemp, 338
 - outputSignals, 338
 - outputSignalThermal, 338
 - outputStep, 334, 335
 - outputToFile, 335
 - Reader, 337
 - ReaderConstraints, 337
 - ReaderFlex, 337
 - ReaderRigid, 337
 - rigidBodies, 338
 - setMechanical, 335
 - subSystems, 338
 - System, 328
 - thermalBodies, 338
 - title, 338
 - update, 335
 - updateThermalLoads, 335
 - writeBoundaryConnectivity, 336
 - writeBoundaryNodes, 336
 - writeFlexBodies, 336
 - writeJoints, 336
 - writeRigidBodies, 336
- mnknix::SystemChain, 338
 - ~SystemChain, 340
 - addTimeLenght, 340
 - getNode, 340
 - populate, 341
 - setInterfaceNodeA, 341
 - setInterfaceNodeB, 341
 - setMass, 341
 - setProperties, 341
 - setTimeLengths, 341
 - SystemChain, 340
 - update, 341
- mnknix::ThermalBody, 342
 - ~ThermalBody, 343
 - addCell, 343
 - addLoadThermal, 344
 - addNode, 344
 - assembleCapacityMatrix, 344
 - assembleConductivityMatrix, 344

- assembleExternalHeat, 344
- calcCapacityMatrix, 344
- calcConductivityMatrix, 345
- calcExternalHeat, 345
- cells, 347
- computeEnergy, 347
- getLastNode, 345
- getNode, 345
- initialize, 345
- nodes, 347
- outputStep, 345, 346
- outputToFile, 346
- setOutput, 346
- temperature, 347
- ThermalBody, 343
- title, 347
- volumetricHeatSources, 347
- MKNIX_API
 - simulation.h, 439
- MknixWrapper
 - mknix::MknixWrapper, 207
- mknixWrapper.cpp, 411
- mknixWrapper.h, 411
- Motion
 - mknix::Motion, 209
- motion.cpp, 412
- motion.h, 413
- motions
 - mknix::System, 338
- nextStep
 - mknix::Analysis, 29
 - mknix::AnalysisDynamic, 33
 - mknix::AnalysisStatic, 36
 - mknix::AnalysisThermalDynamic, 38
 - mknix::AnalysisThermalStatic, 41
 - mknix::AnalysisThermoMechanicalDynamic, 44
- nGPoints
 - mknix::Cell, 71
 - mknix::CellBoundary, 75
- Node
 - mknix::Node, 212
- node.cpp, 413
- node.h, 414
- nodes
 - mknix::Body, 57
 - mknix::BoundaryGroup, 59
 - mknix::Constraint, 99
 - mknix::Load, 185
 - mknix::LoadThermal, 187
 - mknix::ThermalBody, 347
- normal
 - mknix::ConstraintContact, 106
- num
 - mknix::GaussPoint, 158
 - mknix::GaussPointBoundary, 183
 - mknix::Point, 235
- operator=
 - mknix::Simulation, 315
- outputConnectivityToFile
 - mknix::Cell, 70
- outputMaxInterfaceTemp
 - mknix::System, 338
- outputSignals
 - mknix::System, 338
- outputSignalThermal
 - mknix::System, 338
- outputStep
 - mknix::Body, 52, 53
 - mknix::Constraint, 98
 - mknix::FlexFrameGalerkin, 140, 141
 - mknix::FlexGlobalGalerkin, 146, 147
 - mknix::RigidBody, 252, 253
 - mknix::System, 334, 335
 - mknix::ThermalBody, 345, 346
- outputToFile
 - mknix::Body, 53
 - mknix::Constraint, 98
 - mknix::FlexBody, 133
 - mknix::Force, 149
 - mknix::Load, 185
 - mknix::LoadThermal, 187
 - mknix::Material, 204
 - mknix::Radiation, 238
 - mknix::RigidBody, 253
 - mknix::System, 335
 - mknix::ThermalBody, 346
- OutputType
 - mknix::Simulation, 297
- outputValues
 - mknix::ShapeFunction, 274
- outputVTK
 - mknix::Body, 54
- phi
 - mknix::Constraint, 99
 - mknix::ShapeFunction, 274
- phi_q
 - mknix::Constraint, 100
- phi_qq
 - mknix::Constraint, 100
- Point
 - mknix::Point, 223, 224
- point.cpp, 415
- point.h, 415
- points
 - mknix::CellBoundaryLinear, 79
 - mknix::CellTetrahedron, 86
 - mknix::CellTriang, 91
 - mknix::FlexBody, 136
- populate
 - mknix::SystemChain, 341
- POSITION
 - mknix::Simulation, 297
- Qext
 - mknix::GaussPoint, 158

- mknix::GaussPointBoundary, 183
- r
 - mknix::GaussPoint, 158
- Radiation
 - mknix::Radiation, 237
- read
 - mknix::SectionReader, 269
- read_lines
 - mknix, 26
- readConstraints
 - mknix::ReaderConstraints, 241
- Reader
 - mknix::Reader, 239
 - mknix::Simulation, 324
 - mknix::System, 337
- reader.cpp, 416
- reader.h, 416
- ReaderConstraints
 - mknix::ReaderConstraints, 240
 - mknix::Simulation, 324
 - mknix::System, 337
- readerconstraints.cpp, 417
- readerconstraints.h, 418
- ReaderFlex
 - mknix::ReaderFlex, 242
 - mknix::Simulation, 324
 - mknix::System, 337
- readerflex.cpp, 419
- readerflex.h, 419
- ReaderRigid
 - mknix::ReaderRigid, 243
 - mknix::Simulation, 325
 - mknix::System, 337
- readerrigid.cpp, 420
- readerrigid.h, 421
- readFile3D
 - mknix, 27
- readFlexBodies
 - mknix::ReaderFlex, 242
- readRigidBodies
 - mknix::ReaderRigid, 244
- removeFromRender
 - mknix::CompBar, 91
- rh
 - mknix::ConstraintClearance, 103
 - mknix::ConstraintContact, 106
- RigidBar
 - mknix::RigidBar, 246
- rigidBodies
 - mknix::System, 338
- RigidBody
 - mknix::RigidBody, 251
- RigidBody2D
 - mknix::RigidBody2D, 258
- RigidBody3D
 - mknix::RigidBody3D, 262
- RigidBodyMassPoint
 - mknix::RigidBodyMassPoint, 266
- ro
 - mknix::ConstraintDistance, 109
 - mknix::ConstraintFixedAxis, 112
- rotate
 - mknix::Body, 54
- rt
 - mknix::ConstraintClearance, 103
 - mknix::ConstraintContact, 106
 - mknix::ConstraintDistance, 109
 - mknix::ConstraintFixedAxis, 112
- run
 - mknix::MknixWrapper, 207
 - mknix::Simulation, 315
- runMechanicalAnalysis
 - mknix::Simulation, 315
- runThermalAnalysis
 - mknix::Simulation, 316
- rxo
 - mknix::ConstraintFixedCoordinates, 115
- rxz
 - mknix::ConstraintFixedCoordinates, 115
- ryo
 - mknix::ConstraintFixedCoordinates, 115
- ryt
 - mknix::ConstraintFixedCoordinates, 115
- rzo
 - mknix::ConstraintFixedCoordinates, 115
- rzt
 - mknix::ConstraintFixedCoordinates, 116
- scaleLoad
 - mknix::LoadThermalBoundary1D, 193
- SectionReader
 - mknix::SectionReader, 268
- sectionreader.cpp, 421
- sectionreader.h, 422
- setAlphaI
 - mknix::Point, 233
- setAugmentedTolerance
 - mknix::Simulation, 317
- setConstantValue
 - mknix::LoadThermalBody, 190
- setDc
 - mknix::Point, 233
- setDensityFactor
 - mknix::RigidBody, 254
- setEpsilon
 - mknix::Analysis, 30
- setFormulation
 - mknix::FlexBody, 134
- setInertia
 - mknix::RigidBar, 248
 - mknix::RigidBody, 254
 - mknix::RigidBody2D, 260
 - mknix::RigidBody3D, 264
 - mknix::RigidBodyMassPoint, 267
- setInitialTemperatures
 - mknix::Simulation, 317
- setInterfaceNodeA

- mknix::SystemChain, 341
- setInterfaceNodeB
 - mknix::SystemChain, 341
- setJacobian
 - mknix::Point, 233
- setLenght
 - mknix::ConstraintDistance, 109
- setLoadThermal
 - mknix::BoundaryGroup, 59
- setLoadThermalInBoundaryGroup
 - mknix::Body, 55
- setMass
 - mknix::RigidBody, 254
 - mknix::SystemChain, 341
- setMaterial
 - mknix::GaussPoint, 156
- setMaterialIfLayer
 - mknix::Cell, 70
- setMechanical
 - mknix::Body, 55
 - mknix::System, 335
- setMechanicalProps
 - mknix::Material, 205
- setNode
 - mknix::Motion, 209
- setNumber
 - mknix::Node, 218
- setOutput
 - mknix::Body, 55
 - mknix::FlexBody, 134
 - mknix::RigidBody, 254
 - mknix::ThermalBody, 346
- setOutputFilesDetail
 - mknix::Simulation, 318
- setPhi
 - mknix::ShapeFunction, 274
- setPorosityFluidTemp
 - mknix::Material, 205
- setPorosityProps
 - mknix::Material, 205
- setPosition
 - mknix::RigidBar, 248
 - mknix::RigidBody, 254
 - mknix::RigidBody2D, 260
 - mknix::RigidBody3D, 264
 - mknix::RigidBodyMassPoint, 267
- setProperties
 - mknix::SystemChain, 341
- setqt
 - mknix::Node, 219
- setqx
 - mknix::Node, 219
- setShapeFunType
 - mknix::Point, 233
- setSignal
 - mknix, 27
- setTemperature
 - mknix::Body, 55
- setThermalNodeInitialTemperature
 - mknix::Simulation, 318
- setThermalNumber
 - mknix::Node, 219
- setThermalProps
 - mknix::Material, 206
- setTimeLengths
 - mknix::SystemChain, 341
- setTimeUx
 - mknix::Motion, 209
- setTimeUy
 - mknix::Motion, 209
- setTimeUz
 - mknix::Motion, 209
- setTitle
 - mknix::Constraint, 98
- setType
 - mknix::FlexBody, 134
 - mknix::FlexFrameGalerkin, 142
 - mknix::FlexGlobalGalerkin, 147
- setUx
 - mknix::Node, 219
- setUy
 - mknix::Node, 220
- setUz
 - mknix::Node, 220
- setVariableResistanceCoords
 - mknix::Material, 206
- setX
 - mknix::Node, 220
- setY
 - mknix::Node, 220
- setZ
 - mknix::Node, 220
- shapeFun
 - mknix::Point, 235
- ShapeFunction
 - mknix::Point, 234
 - mknix::ShapeFunction, 271
- shapefunction.cpp, 423
- shapefunction.h, 424
- ShapeFunctionLinear
 - mknix::Point, 234
 - mknix::ShapeFunctionLinear, 276
- shapefunctionlinear-x.cpp, 425
- shapefunctionlinear-x.h, 425
- shapefunctionlinear.cpp, 426
- shapefunctionlinear.h, 426
- ShapeFunctionLinearX
 - mknix::Point, 234
 - mknix::ShapeFunctionLinearX, 278
- ShapeFunctionMLS
 - mknix::Point, 234
 - mknix::ShapeFunctionMLS, 280
- shapefunctionMLS.cpp, 427
- shapefunctionMLS.h, 428
- ShapeFunctionMLS2D
 - mknix::Point, 235

- mknix::ShapeFunctionMLS2D, 282
- shapefunctionMLS2D.cpp, 429
- shapefunctionMLS2D.h, 430
- ShapeFunctionMLS3D
 - mknix::Point, 235
 - mknix::ShapeFunctionMLS3D, 284
- shapefunctionMLS3D.cpp, 430
- shapefunctionMLS3D.h, 431
- ShapeFunctionRBF
 - mknix::Point, 235
 - mknix::ShapeFunctionRBF, 286
- shapefunctionRBF.cpp, 432
- shapefunctionRBF.h, 432
- ShapeFunctionTetrahedron
 - mknix::Point, 235
 - mknix::ShapeFunctionTetrahedron, 288
- shapefunctiontetrahedron.cpp, 433
- shapefunctiontetrahedron.h, 434
- ShapeFunctionTriangle
 - mknix::Point, 235
 - mknix::ShapeFunctionTriangle, 291
- shapefunctiontriangle.cpp, 435
- shapefunctiontriangle.h, 435
- shapefunctiontriangle3D.cpp, 436
- shapefunctiontriangle3D.h, 437
- ShapeFunctionTriangleSigned
 - mknix::ShapeFunctionTriangleSigned, 293
- shapeFunSolve
 - mknix::GaussPoint, 156
 - mknix::GaussPoint2D, 168
 - mknix::GaussPoint3D, 178
 - mknix::GaussPointBoundary, 182
 - mknix::Point, 233
- shapeFunType
 - mknix::Point, 235
- Simulation
 - mknix::Simulation, 297
- simulation.cpp, 438
- simulation.h, 439
 - MKNIX_API, 439
- smoothingMassMatrix
 - mknix::FlexBody, 136
- solve
 - mknix::Analysis, 30
 - mknix::AnalysisDynamic, 33
 - mknix::AnalysisStatic, 36
 - mknix::AnalysisThermalDynamic, 39
 - mknix::AnalysisThermalStatic, 41
 - mknix::AnalysisThermoMechanicalDynamic, 44
- solveStep
 - mknix::Simulation, 318, 319
- SOURCE_1D
 - mknix, 22
- SOURCE_2D
 - mknix, 22
- SOURCE_3D
 - mknix, 22
- SOURCE_VTK
 - mknix, 22
- Source Type
 - mknix, 22
- staticConvergence
 - mknix::Simulation, 319
- staticResidue
 - mknix::Simulation, 320
- staticTangent
 - mknix::Simulation, 321
- staticThermalConvergence
 - mknix::Simulation, 321
- staticThermalResidue
 - mknix::Simulation, 322
- staticThermalTangent
 - mknix::Simulation, 322
- stepTriggered
 - mknix::Simulation, 323
- stiffnessMatrix
 - mknix::Constraint, 100
- STRESS
 - mknix::Simulation, 297
- stresses
 - mknix::FlexBody, 136
- stressPoint
 - mknix::GaussPoint, 158
- subSystems
 - mknix::System, 338
- supportNodes
 - mknix::Point, 235
- supportNodesSize
 - mknix::Point, 236
- System
 - mknix::System, 328
- system.cpp, 439
- system.h, 440
- SystemChain
 - mknix::Simulation, 325
 - mknix::SystemChain, 340
- systemchain.cpp, 441
- systemchain.h, 441
- temperature
 - mknix::Body, 57
 - mknix::ThermalBody, 347
- tension
 - mknix::GaussPoint, 158
- thermalBodies
 - mknix::System, 338
- ThermalBody
 - mknix::ThermalBody, 343
- theSimulation
 - mknix::Analysis, 31
 - mknix::MknixWrapper::SimulationWrapper, 325
- timemap
 - mknix::LoadThermalBoundary1D, 193
- title
 - mknix::Body, 57
 - mknix::Constraint, 100
 - mknix::System, 338

- mknix::ThermalBody, 347
- translate
 - mknix::Body, 55
- Tt
 - mknix::ConstraintThermalFixed, 121
- type
 - mknix::Analysis, 30
 - mknix::AnalysisDynamic, 34
 - mknix::AnalysisStatic, 36
 - mknix::AnalysisThermalDynamic, 39
 - mknix::AnalysisThermalStatic, 41
 - mknix::AnalysisThermoMechanicalDynamic, 44
- update
 - mknix::Material, 206
 - mknix::Motion, 209
 - mknix::System, 335
 - mknix::SystemChain, 341
- updateDelaunay
 - mknix::Contact, 122
- updateLines
 - mknix::Contact, 122
- updateLoad
 - mknix::LoadThermal, 187
- updatePoints
 - mknix::CompBar, 91
 - mknix::Contact, 122
- updateThermalLoads
 - mknix::System, 335
- volumetricHeatSources
 - mknix::Body, 57
 - mknix::ThermalBody, 347
- weight
 - mknix::GaussPoint, 158
 - mknix::GaussPointBoundary, 183
- writeBodyInfo
 - mknix::Body, 56
 - mknix::FlexBody, 134
 - mknix::RigidBody, 254
- writeBoundaryConnectivity
 - mknix::Body, 56
 - mknix::FlexBody, 135
 - mknix::RigidBar, 249
 - mknix::RigidBody, 255
 - mknix::System, 336
- writeBoundaryNodes
 - mknix::Body, 56
 - mknix::FlexBody, 135
 - mknix::RigidBar, 249
 - mknix::RigidBody, 255
 - mknix::System, 336
- writeConfStep
 - mknix::Simulation, 323
- writeFlexBodies
 - mknix::System, 336
- writeJointInfo
 - mknix::Constraint, 99
- writeJoints
 - mknix::System, 336
- writeRigidBodies
 - mknix::System, 336
- writeSystem
 - mknix::Simulation, 324
- X
 - mknix::Point, 236
- Y
 - mknix::Point, 236
- Z
 - mknix::Point, 236